

刑務所 (Jail)

解説 nxteru (戸高空)

問題概要

- ▶ N 頂点の木に M 人の人がいる
- ▶ 人 j は最初頂点 s_j におり頂点 t_j に移動させたい
- ▶ ある人を他の人がいない隣接した頂点に移動させるという操作を好きなだけできる（同時に2人以上が同じ頂点にいてはならない）
- ▶ 全員が最短経路で遠回りをせずに移動をすることはできるか
- ▶ $N, M \leq 120\,000$

小課題 1

- ▶ $A_i = i, B_i = i + 1$
- ▶ 木はパスグラフである

小課題 1

- ▶ 直線上を人が左右に移動するとみなせる
 - ▶ 移動前と移動後で人の並びの順番が一致している $\Leftrightarrow$ Yes
 - ▶ なぜか？
-
- ▶ 一致していない $\Rightarrow$ どこかで人のswapが必要なのでNo
 - ▶ 一致している $\Rightarrow$ 誰かを隣り合う頂点に移動させることで終わりの状態に近づくような移動が常に存在する

小課題 2

- ▶ $Q \leq 20, N \leq 250, M = 2$
- ▶ Noになるような位置関係で適当な場合分けをすればよい
- ▶ 2人のいる場所の状態は N^2 通りしかないのでDPでも解ける

ここで大胆予想

ここで大胆予想

- ▶ ある人の移動を複数回に分けて行う必要って本当にあるの？

ここで大胆予想

- ▶ ある人の移動を複数回に分けて行う必要って本当にあるの？
- ▶ 実は「Yes⇒同じ人の移動はすべて連続して行うような移動のさせ方が存在する」ことが証明できる

証明

• • •

1	2	2	3	2	3	4	1
---	---	---	---	---	---	---	---

 • • •

- ▶ Yesである仮定したときそれを達成するような移動のさせ方で、操作で指定する人を順番に並べた列を考える

証明

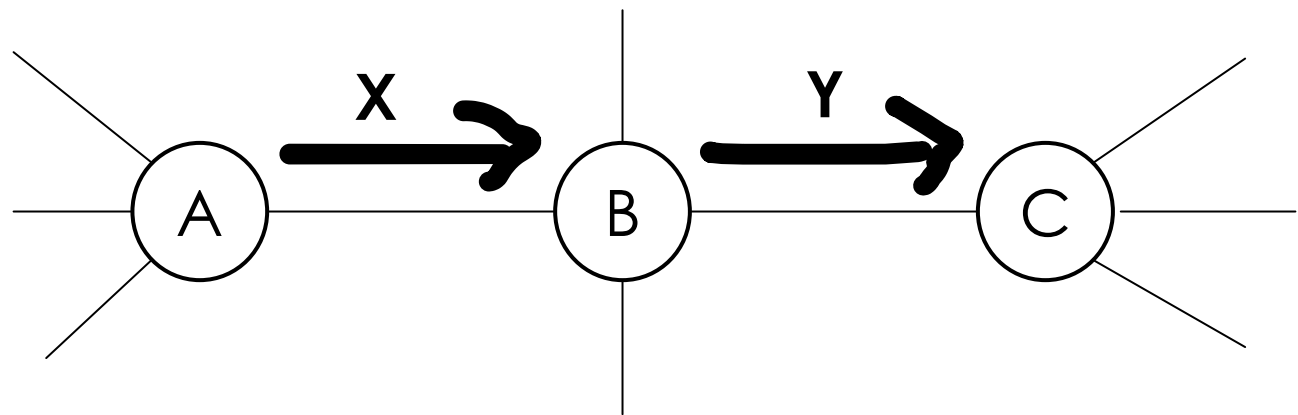


- ▶ ある人にとって連続するが操作の列では離れている2つの操作に注目 (X, Y とする)
- ▶ ここで X の前に操作の列で連続している同じ人の移動、 Y の後に連続している同じ人の移動もひとかたまりとして扱う

証明



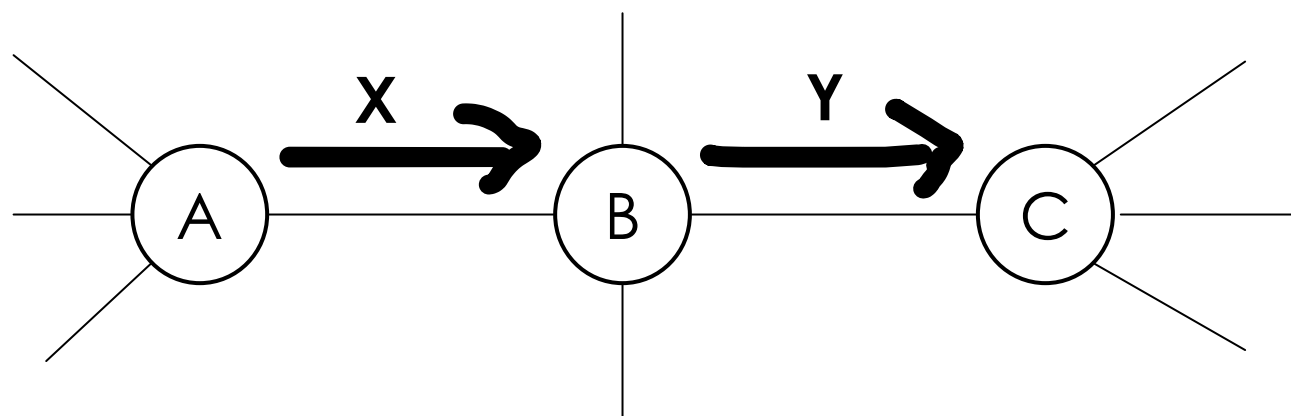
▶ その2つが右図の移動



証明



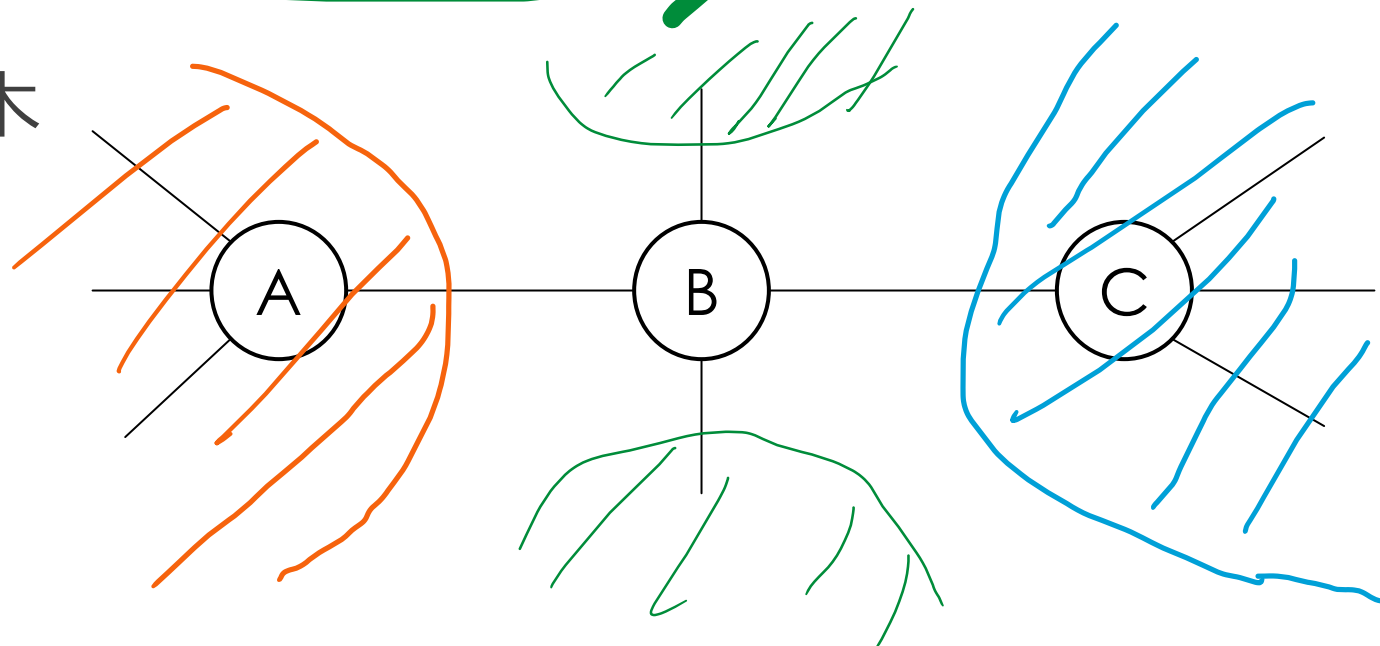
- ▶ その間人1は頂点 B にいるので緑の区間の移動は B を通らない



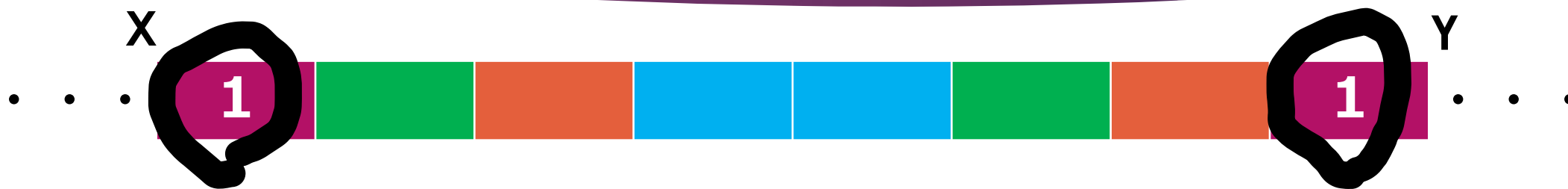
証明



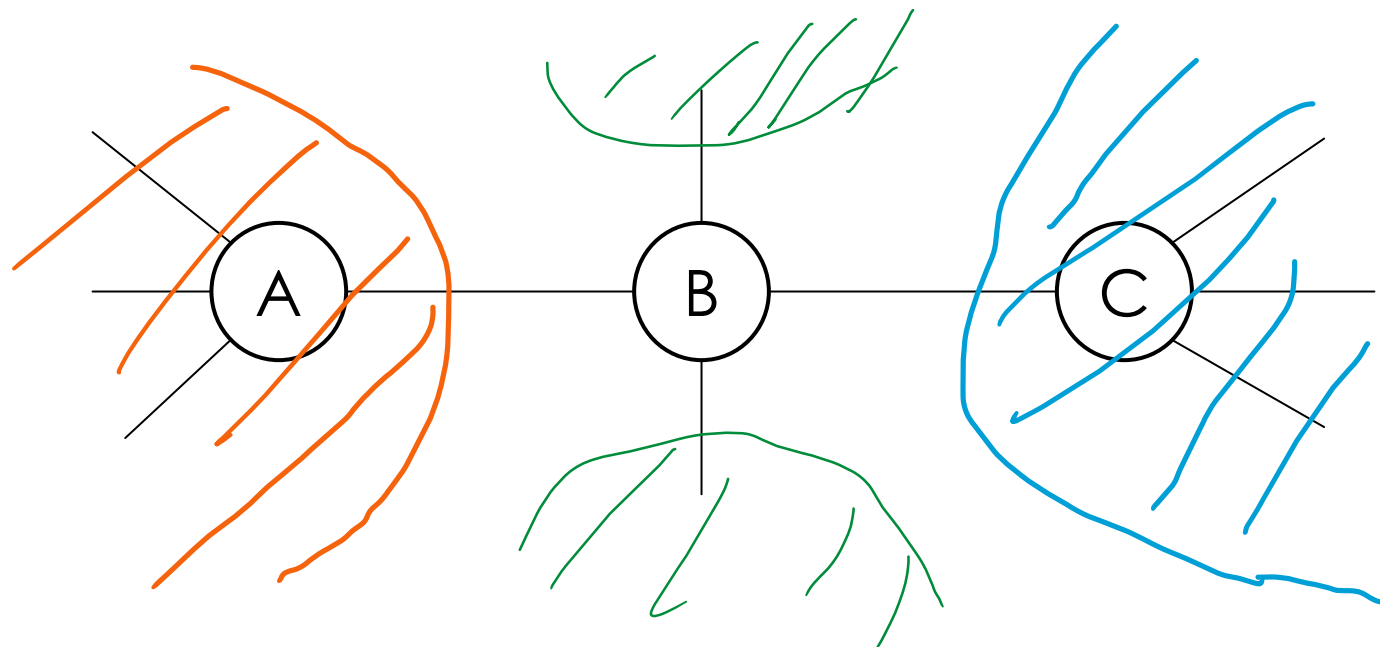
- ▶ B で分けたときの部分木ごとに間の操作を分類する



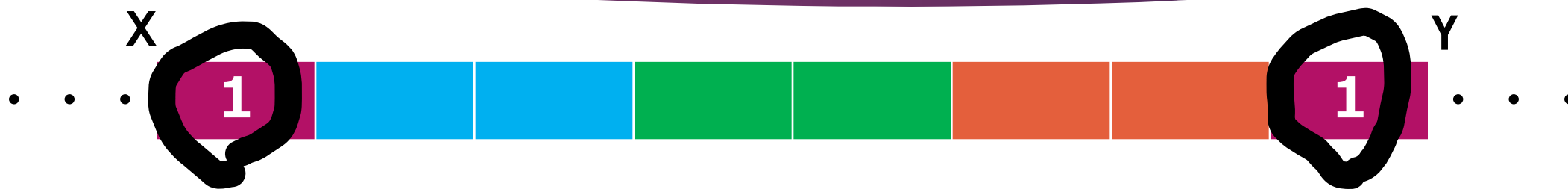
証明



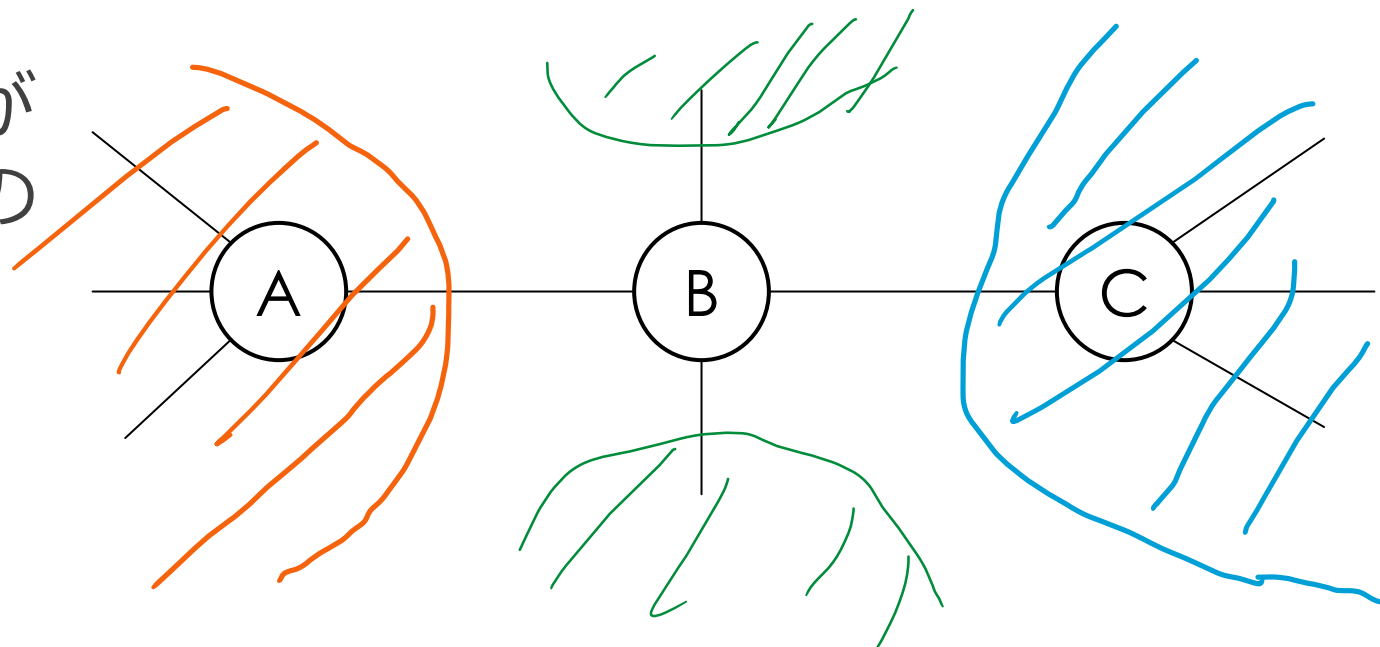
- ▶ 違う部分木なら操作は干渉しないので、部分木内での操作の順序を守れば順序を入れ替えることができる



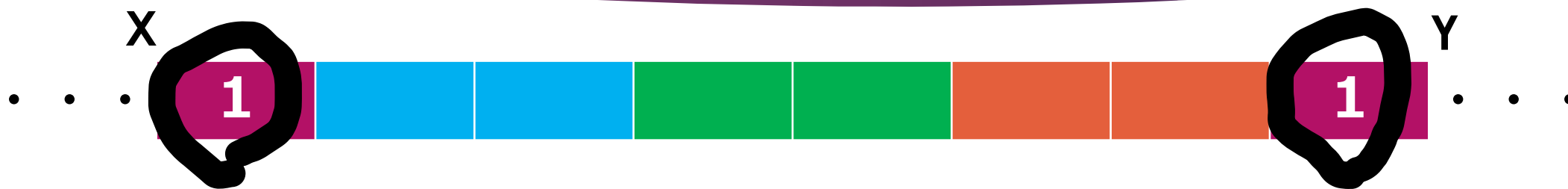
証明



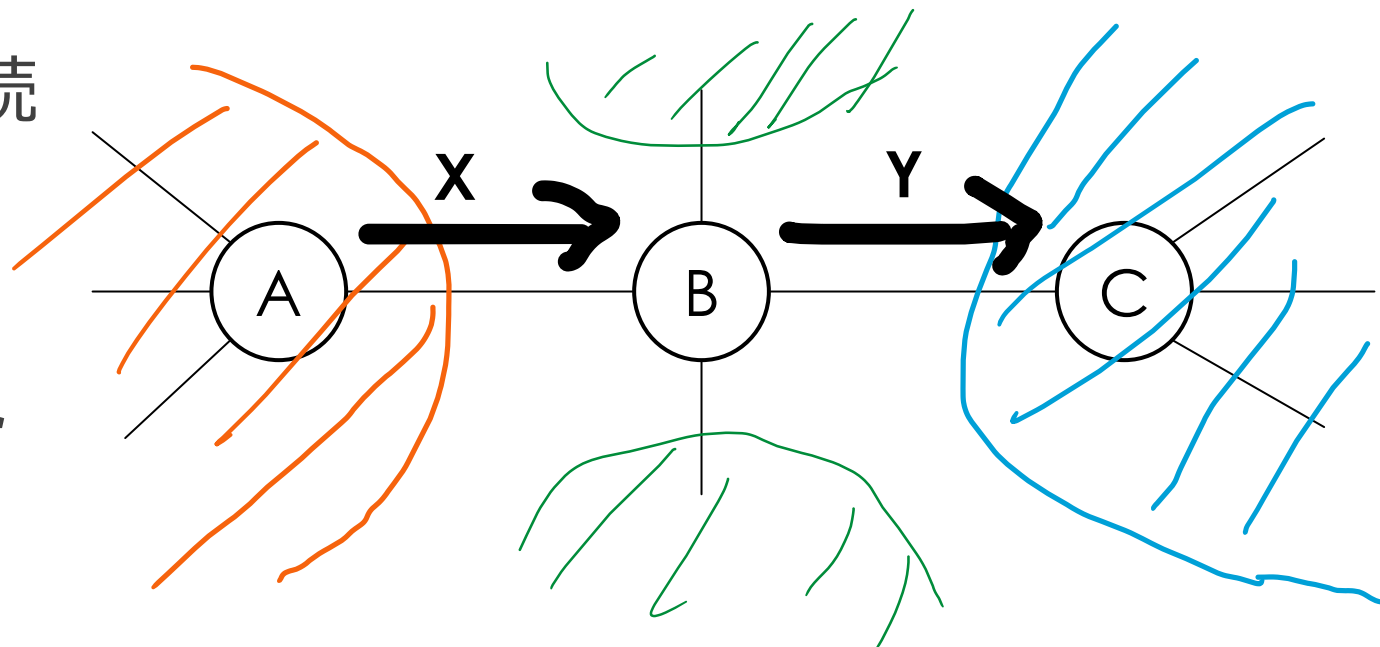
- ▶ C を含む部分木の操作が
すべて A を含む部分木の
操作の前になるように
順序を入れ替える



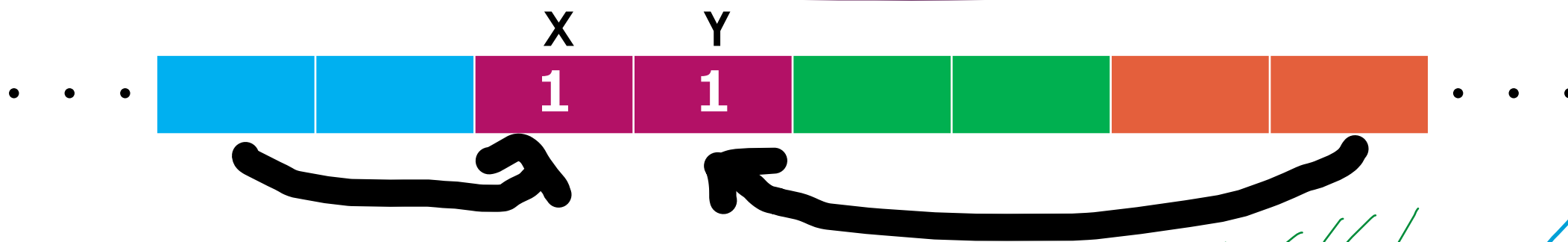
証明



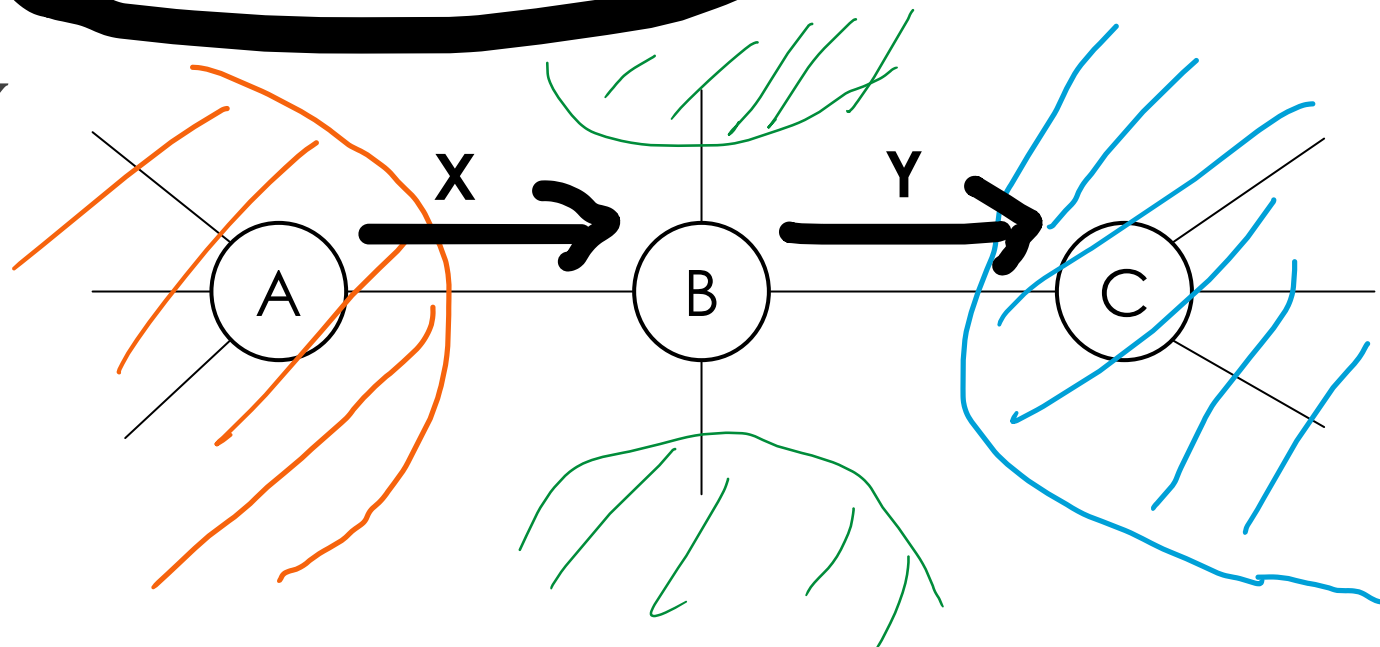
- ▶ 操作X（とその前に連続する同じ人の移動のかたまり）と干渉するのはAを含む部分木だけ
- ▶ 操作Yと干渉するのはCを含む部分木だけ



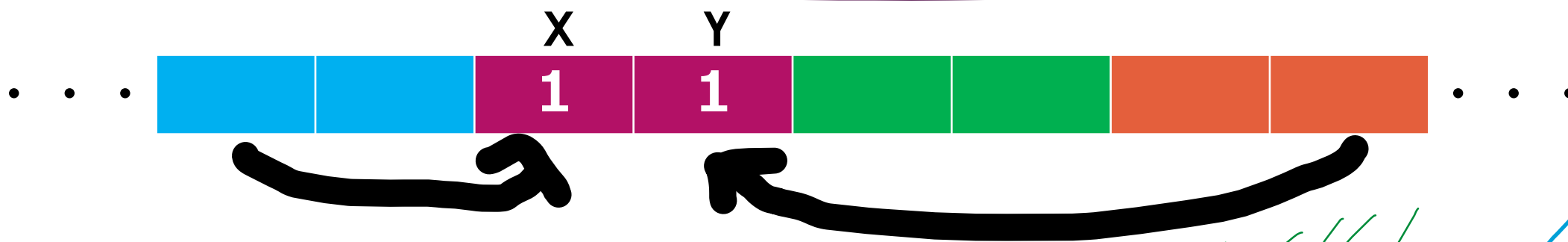
証明



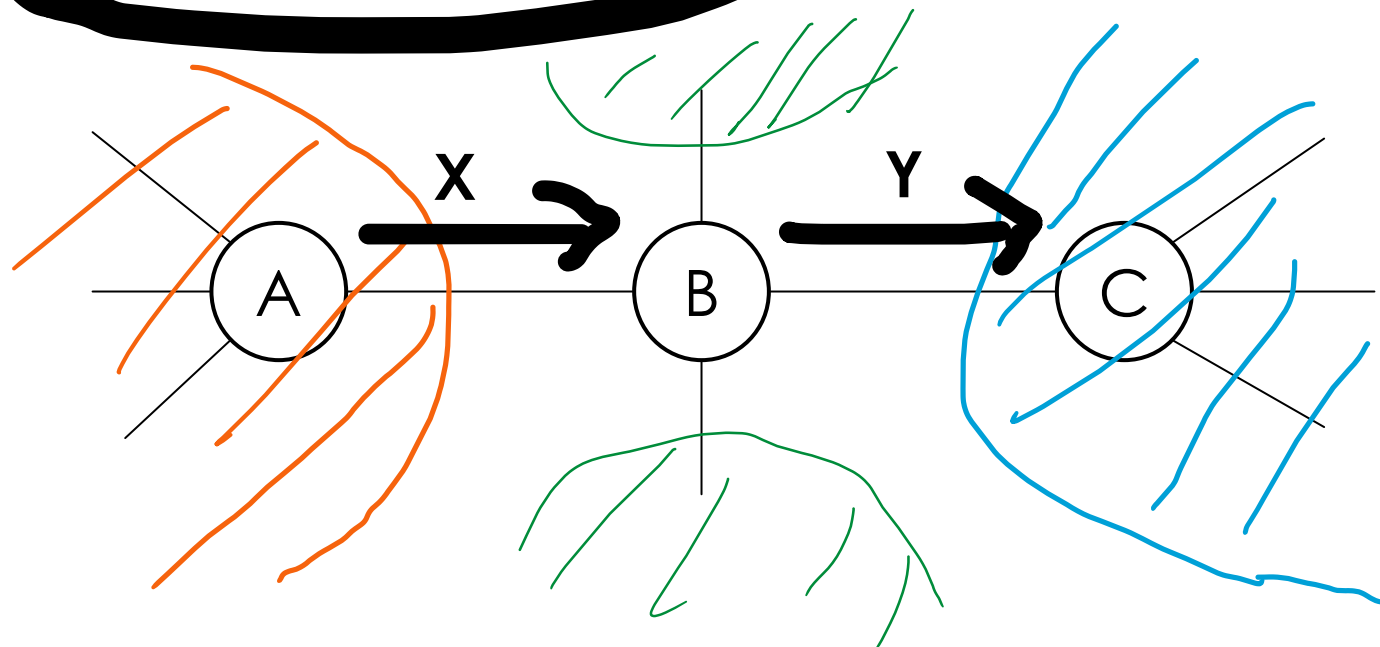
- ▶ 順序を入れ替えてXとYのかたまりを連続させることができる



証明



- ▶ 同じ人の操作は同じ部分木に属するからこれらの入れ替えで連続した人が離れてしまうことはない



証明

...

2	2	1	1	1	4	4	4
---	---	---	---	---	---	---	---

 ...

- ▶ これを繰り返すことで全員が自分の操作は連続するような操作列をつくることできる

より簡単な問題への帰着

- ▶ 操作は次のようにしてもよい
- ▶ 人を1人選んでその人を初期地点から目的地点まで移動させる
- ▶ ただしその経路上に他の人がいてはならない

小課題 3

- ▶ $Q \leq 20, N \leq 250, M \leq 6$
- ▶ M が小さいので移動する順番を全探索できる
- ▶ $O(QM!N)$

考察

▶ 満たすべき条件「操作のとき移動する人の経路に他の人がいない」

=条件「人 X の移動のときその経路上に人 Y がいない」を
($1 \leq X < Y \leq M$) ですべて満たす

考察

- ▶ 「人 X の移動のときその経路上に人 Y がいない」
- ▶ 人 Y は初期地点 S_Y にいる（まだ移動していない）か目的地点 T_Y にいる（移動を終えた）のどちらかである

考察

- ▶ 「人 X の移動のときその経路上に人 Y がない」
- ▶ 人 Y は初期地点 S_Y にいる（まだ移動していない）か目的地点 T_Y にいる（移動を終えた）のどちらかである
- ▶ 人 X の経路上に S_Y があるとき
「人 Y の移動は人 X の移動より先でなければならない」
- ▶ 人 X の経路上に T_Y があるとき
「人 X の移動は人 Y の移動より先でなければならない」

考察

- ▶ つまり順番について2つの順序がたくさん与えられるのでそれを満たす順番が存在するかを判定する問題となる

考察

- ▶ M 頂点のグラフを用意して、「 x が y より先」という条件を x から y への有向辺であらわす

考察

- ▶ M 頂点のグラフを用意して、「 x が y より先」という条件を x から y への有向辺であらわす
- ▶ すると条件を満たす順番が存在する $\Leftrightarrow$ グラフに閉路がない
- ▶ 閉路がないならトポロジカルソートをすることで実際に順番を構築できる

小課題 4

- ▶ $Q \leq 20, N \leq 250, M \leq 100$
- ▶ 人 X, Y の組を全探索して X と Y の間の有向辺を調べる

小課題 4

- ▶ X の移動を考えた時 Y との間に辺がはられる条件
- ▶ $S_X - T_X$ パス上に S_Y がある($X \leftarrow Y$ の辺)
- ▶ $S_X - T_X$ パス上に T_Y がある($Y \leftarrow X$ の辺)
- ▶ N が小さいのでパスを1つずつたどって判定しても間に合う
- ▶ $O(QM^2N)$

小課題 5

- ▶ $Q \leq 20, M \leq 500$
- ▶ 人 X, Y の組を全探索して X と Y の間の有向辺を調べる
- ▶ 「 A - B パス上に C があるか」の判定を高速化する

小課題 5

- ▶ 根付き木にしてオイラーツアーをする
- ▶ すると「頂点 X が頂点 Y の先祖か」を $O(1)$ で判定できる
- ▶ 「 A - B パス上に C がある」 = 「 $LCA(A, B)$ は C の先祖かつ C は A の先祖」 or 「 $LCA(A, B)$ は C の先祖かつ C は B の先祖」

小課題 5

- ▶ M 人それぞれのパスの LCA をあらかじめ求めておくと判定が $O(1)$ でできる
- ▶ $O(Q(M^2 + M \log N))$

小課題 6

- ▶ どの部屋からどの部屋へも、高々 20 本の通路を使って移動することができる。
- ▶ それぞれの人のパスも21頂点以下である

小課題 6

- ▶ 辺をはるのはパス上に s または t がある人のみ
- ▶ 各頂点にそこに s がある人の番号、 t がある人の番号を記録
- ▶ 各人についてパスをたどってそこに書いてある番号の人と辺をはる

小課題 6

- ▶ 根付き木にしてパスの LCA を求めておくと端点から親へたどっていくことでパスだけみることができる
- ▶ $O(M \log N + 20M)$

満点

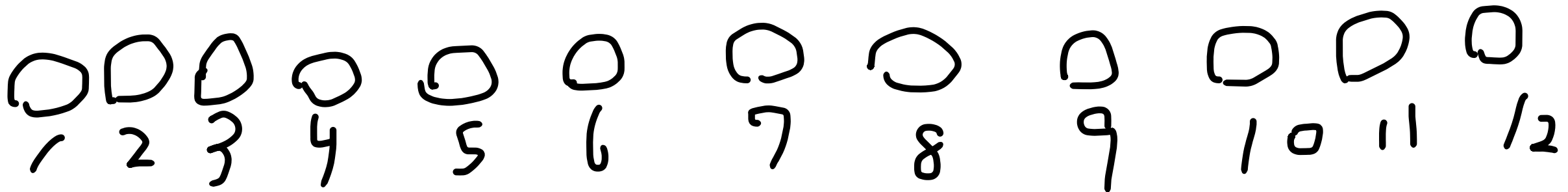
- ▶ M 人それぞれについてその経路上に有向辺をはる問題
- ▶ パスに対して辺をはる操作を高速に行いたい

満点

- ▶ 有向辺をはるという操作は（閉路判定においては）最大値を求める操作と同じように前計算でいくつかのものをまとめておくとその計算結果が利用できる

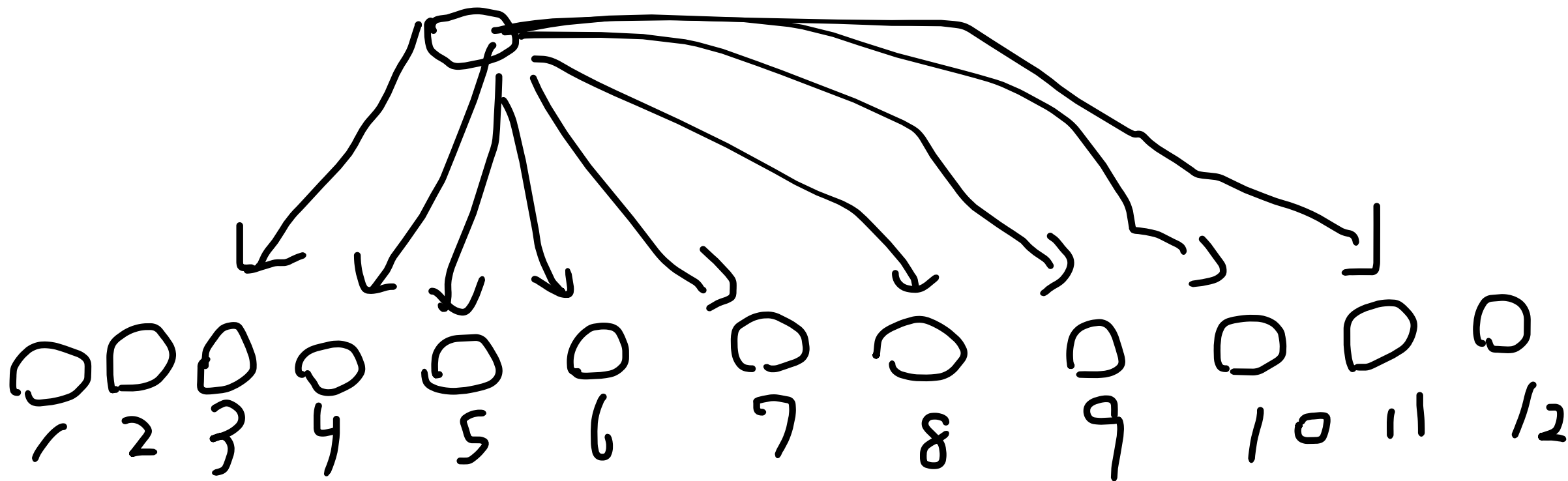
満点

▶ 例えば下の頂点の列について3 ... 11に有向辺をはるとき



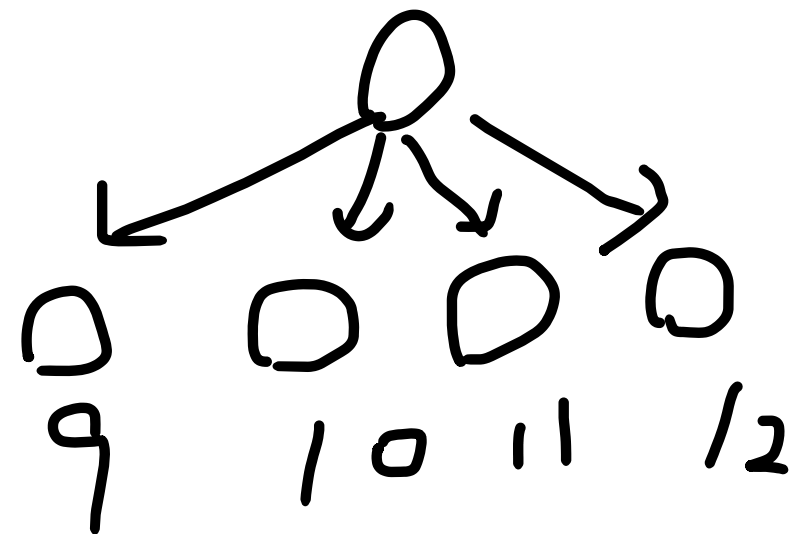
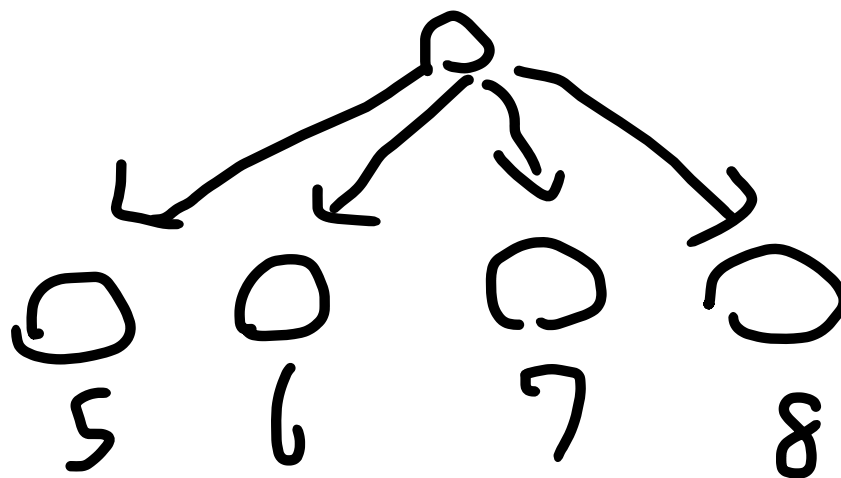
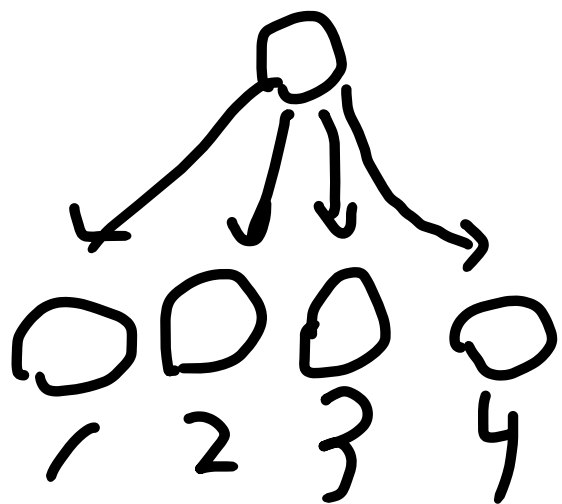
満点

▶ 愚直にはると時間がかかるが

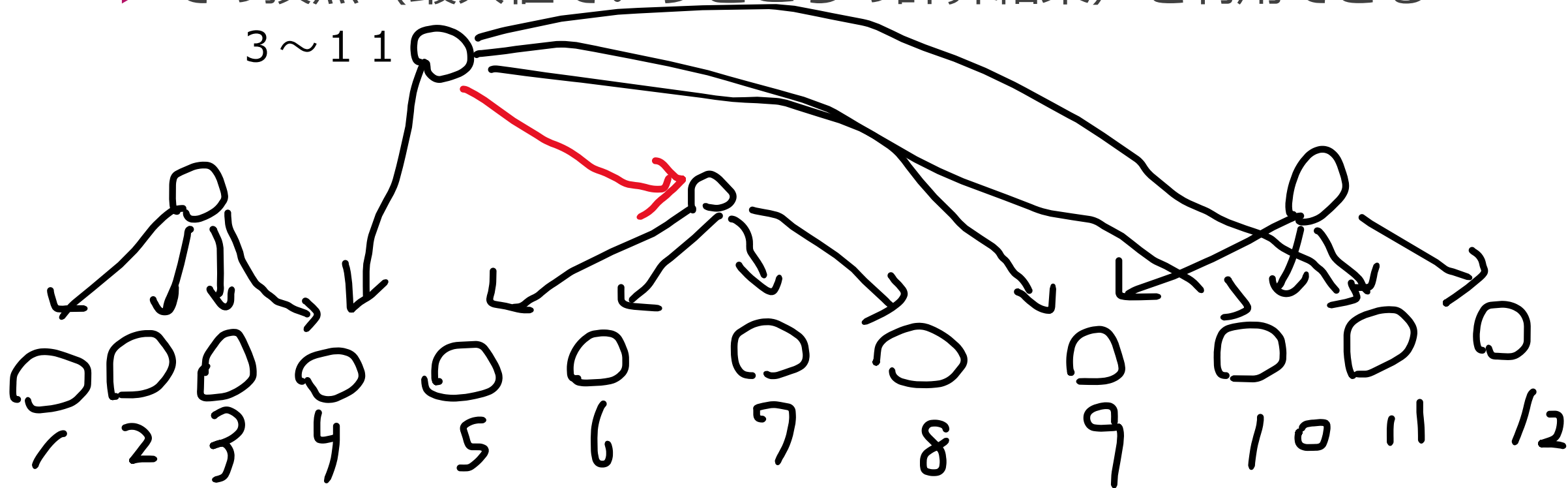


満点

- ▶ 前計算で4つずつまとめて有向辺をはった頂点を作っておくと

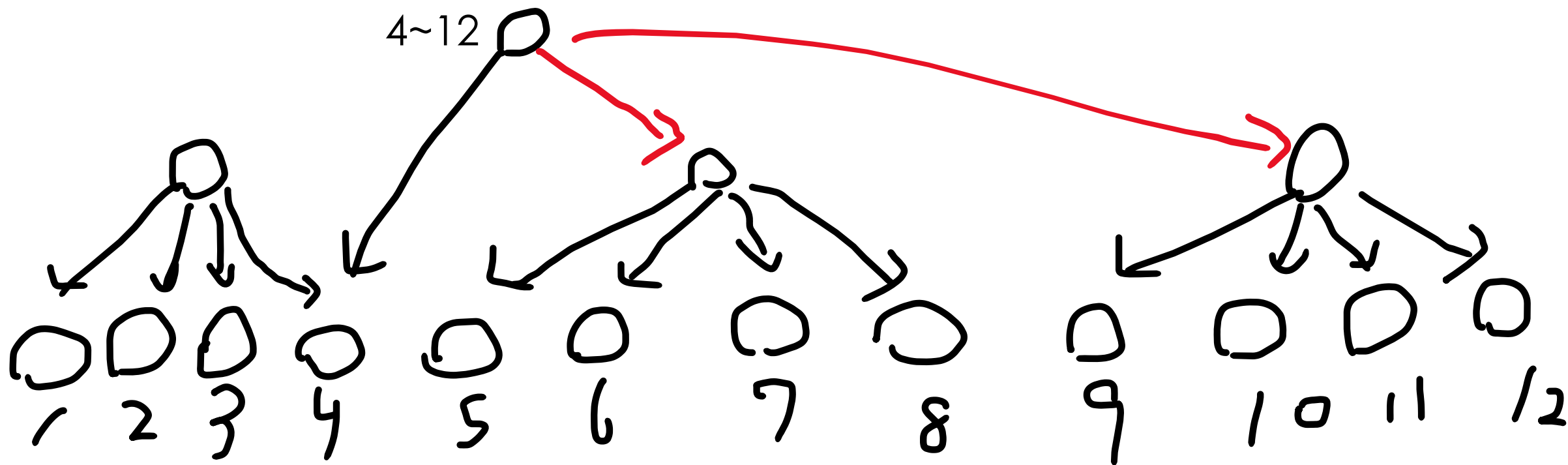


100



満点

- ▶ その頂点（最大値でいうところの計算結果）を利用できる



満点

- ▶ パス上に辺を高速にはりたい
- ▶ パス上の最大値を求める操作と同じようにできる
- ▶ ダブリングでできる！

満点

- ▶ LCA をダブリングで求める手法を思い出してみよう
- ▶ 各頂点から 2^0 回, 2^1 回, 2^2 回, ..., 2^k 回先の祖先を計算している
- ▶ 同じように各頂点から根まで 2^0 回, 2^1 回, 2^2 回, ..., 2^k 回たどった経路上にある頂点すべてに有向辺がはられた頂点を用意する

満点

- ▶ k の小さい順に構築していく
- ▶ v から 2^k 個に辺をはる頂点 $e[v][k]$ からは
 v から 2^{k-1} 個に辺をはった頂点

$$e[v][k - 1]$$

と v の 2^{k-1} 回先の頂点から 2^{k-1} 個に辺をはった頂点

$$e[\text{par}[v][k - 1]][k - 1]$$

の2つに辺をはれば構築できる

満点

- ▶ $v - u$ パスに有向辺をはるとき
- ▶ $v - LCA(v, u)$ と $u - LCA(v, u)$ にはるとして

満点

▶ $r = 2^k, 2^{k-1}, 2^{k-2}, \dots, 2^0$ として

r 個先を確認して LCA 以下ならそこから r 個に辺をはって

($e[v][r]$ に辺をはって)

r 個先にいく ($v = par[v][r]$)

満点

- ▶ できあがった $(N + M)\log N$ 頂点のグラフで閉路判定をする
- ▶ $O((N + M)\log N)$ で解ける

満点（別解）

- ▶ 根で分けたとき、 s と T が違う部分木にあるものはもっと簡単に辺をはれる
- ▶ s から根まで、 T から根までに辺をはることになるが
- ▶ どちらも根まではるので、各頂点について根からの累積和のような頂点を用意しておけばよい（ v の累積頂点は親の累積頂点と v に有向辺をはって構築）

満点 (別解)

- ▶ では $S - T$ が同じ部分木にあるものは
- ▶ その部分木について再帰的に問題を解けばよい
- ▶ 根として重心を選んでいくと再帰の深さが $\log N$ でおさまる (重心分解)
- ▶ $O((N + M)\log N)$

得点分布

- ▶ 100点 5人
- ▶ 77点 6人
- ▶ 66点 3人
- ▶ 54点 2人
- ▶ 26点 4人
- ▶ 10点 6人
- ▶ 5点 1人
- ▶ 0点 2人

Sightseeing in Kyoto

Editorial: Hirotaka Yoneda (square1001)

[Front cover taken on Nov 23rd, 2021]



京都市は有名な観光地ですが...



[Google Maps より]

京都市は「碁盤目状の道路」でも有名です

問題概要

横方向の道路が H 本、縦方向の道路が W 本あり

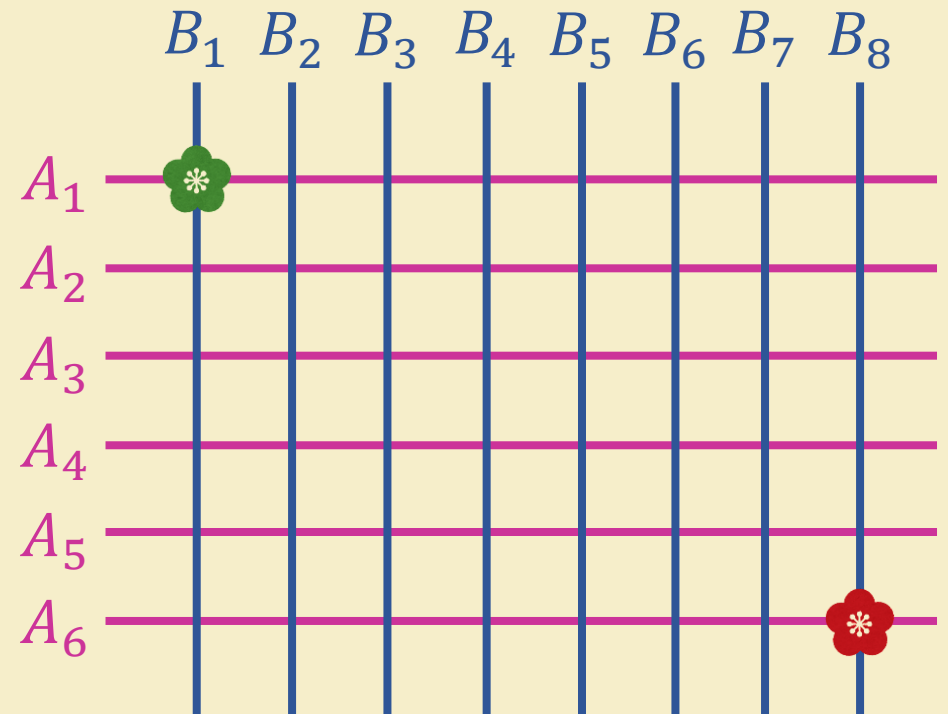
右図のように「碁盤目状」になっています

道路によって歩く速度が異なってきます

- 北から i 本目の横方向の道路は
1 マス分歩くのに A_i 秒かかる
- 西から j 本目の横方向の道路は
1 マス分歩くのに B_j 秒かかる

左上端 → 右下端 には最短で何秒かかる？

ただし、遠回りをしてはならないものとします（つまり、右方向 or 下方向にしか動けない）



問題概要

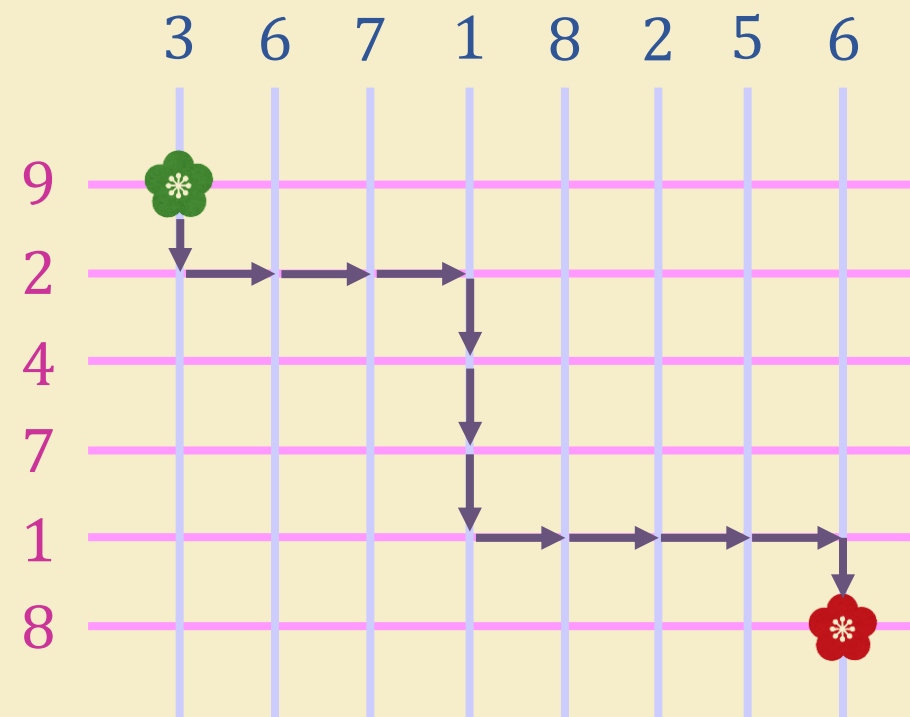
右図は $H = 6, W = 8$ の例

このように進むのが「最短」になります

移動時間: $3+2+2+2+1+1+1+1+1+1+6 = 22$

制約

- 小課題 1 (10 点): $H, W \leq 10^3$
- 小課題 2 (30 点): $H, W \leq 10^5; A_i, B_j \leq 10^3$
- 小課題 3 (60 点): $H, W \leq 10^5$



小課題 I (10 点)

制約: $H, W \leq 1000$



小課題 1 — $H, W \leq 1000$

動的計画法 (DP) で解くことを考えます

$d[i][j]$ = 「交差点 (i, j) に到達するための最短の時間」を順々に求めていきます

小課題 1 — $H, W \leq 1000$

動的計画法 (DP) で解くことを考えます

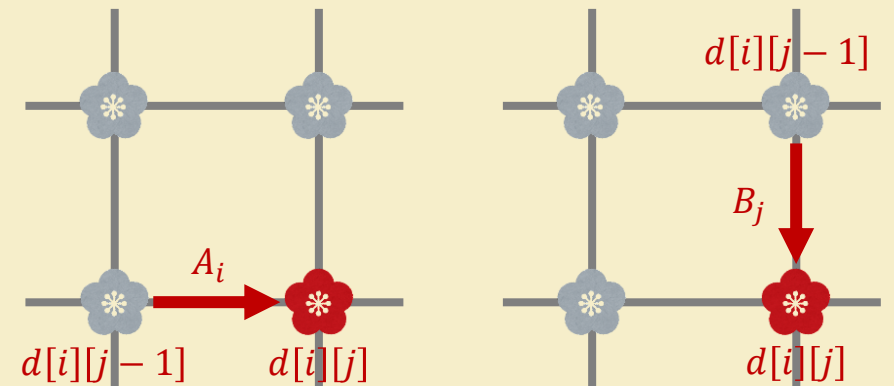
$d[i][j]$ = 「交差点 (i, j) に到達するための最短の時間」を順々に求めていきます

$d[i][j]$ を求めたい

交差点 (i, j) に到達する方法は右図の 2 通りに分けられます

- 左から進む場合: 最短距離 $d[i][j-1] + A_i$
- 上から進む場合: 最短距離 $d[i-1][j] + B_j$

よって $d[i][j] = \min(d[i][j-1] + A_i, d[i-1][j] + B_j)$



小課題 1 — $H, W \leq 1000$

この式にしたがって、左上の交差点から順に求めます

答えは $d[H][W]$ になります

計算量は $O(HW) \rightarrow$ 小課題 1 には通ります

```
#include <iostream>
#include <algorithm>
using namespace std;

const long long INF = (1LL << 62);
int H, W, A[1009], B[1009]; long long d[1009][1009];

int main() {
    cin >> H >> W;
    for (int i = 1; i <= H; i++) cin >> A[i];
    for (int i = 1; i <= W; i++) cin >> B[i];
    for (int i = 1; i <= H; i++) {
        for (int j = 1; j <= W; j++) {
            if (i == 1 && j == 1) continue;
            long long v1 = (j >= 2 ? d[i][j-1] + A[i] : INF);
            long long v2 = (i >= 2 ? d[i-1][j] + B[j] : INF);
            d[i][j] = min(v1, v2);
        }
    }
    cout << d[H][W] << endl;
}
```


小課題 2 (30 点)

制約: $H, W \leq 10^5$; $A_i, B_j \leq 1000$



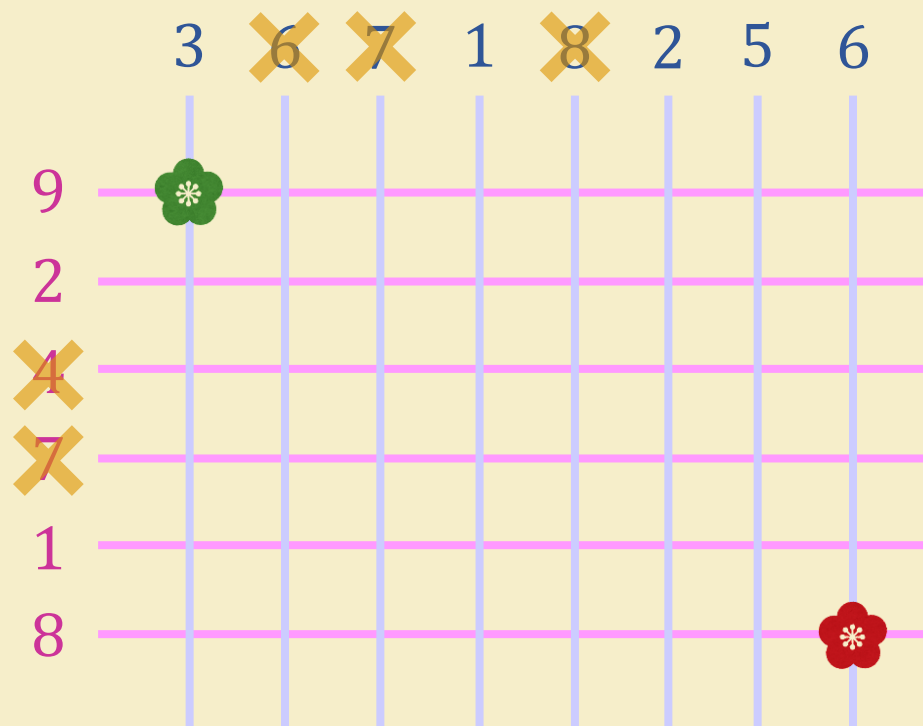
小課題 2 — $A_i, B_j \leq 1000$

右図の例を考えてみよう

どのような道路を通ることになるか？

逆に、どのような道路は確実に通らないか？

実は、× を書いた道路は「確実に通らない」！



小課題 2 — $A_i, B_j \leq 1000$

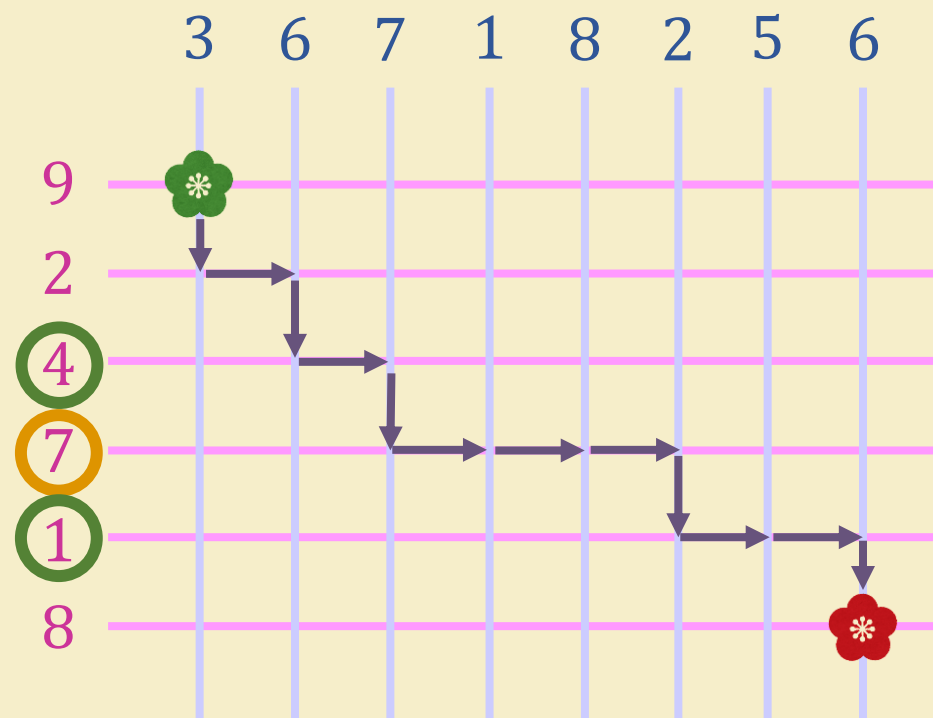
重要な性質

以下の道路は絶対に通らない

- $A_{i-1} \leq A_i \geq A_{i+1}$ となるような、横方向の道路 i
- $B_{i-1} \leq B_i \geq B_{i+1}$ となるような、縦方向の道路 j

具体例

仮に右図の「7」の道路を通る経路が最適だとする



小課題 2 — $A_i, B_j \leq 1000$

重要な性質

以下の道路は絶対に通らない

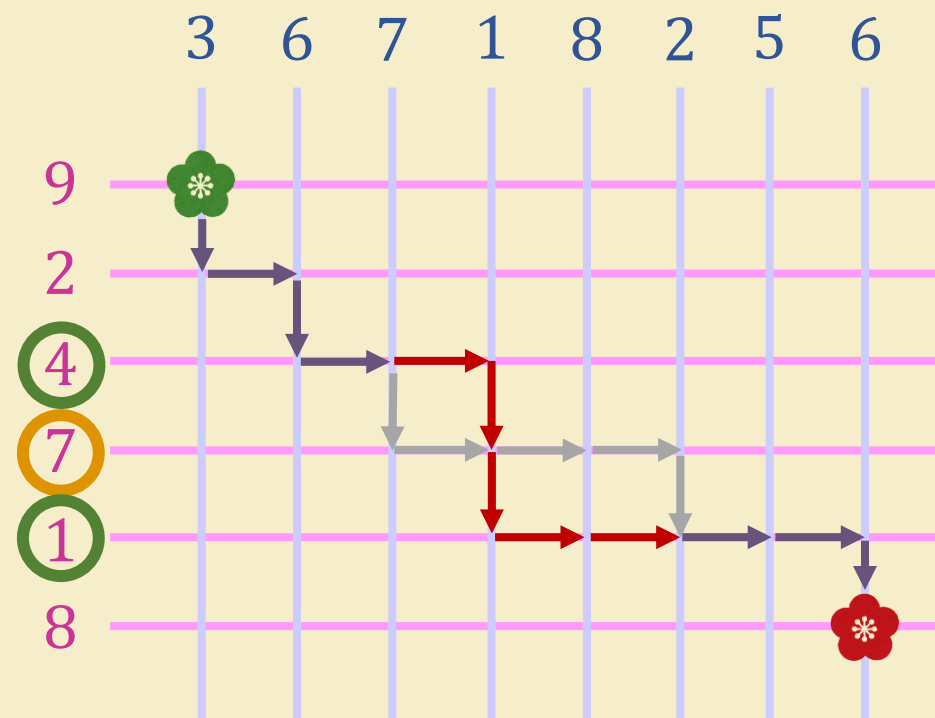
- $A_{i-1} \leq A_i \geq A_{i+1}$ となるような、横方向の道路 i
- $B_{i-1} \leq B_i \geq B_{i+1}$ となるような、縦方向の道路 j

具体例

仮に右図の「7」の道路を通る経路が最適だとする

これは、赤い（縦の「1」を通る）経路に変えた方が明らかに得

（理由は、縦の部分 $[7+2 > 1+1]$ と横の部分 $[7+7+7 > 4+1+1]$ の両方で、全区間で短縮されるから）



小課題 2 — $A_i, B_j \leq 1000$

重要な性質

以下の道路は絶対に通らない

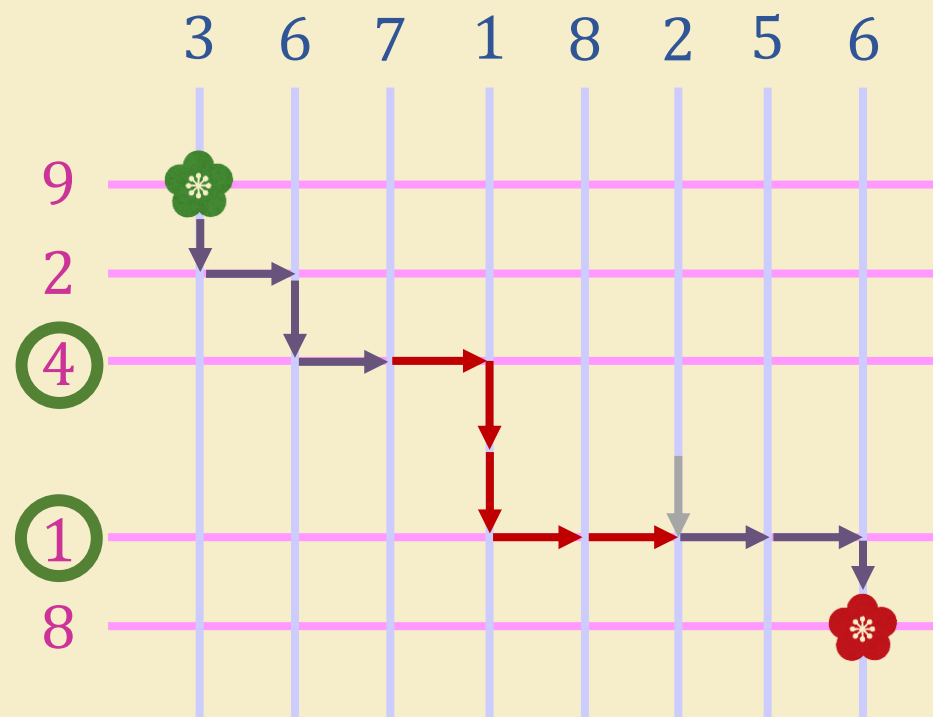
- $A_{i-1} \leq A_i \geq A_{i+1}$ となるような、横方向の道路 i
- $B_{i-1} \leq B_i \geq B_{i+1}$ となるような、縦方向の道路 j

証明

A_i の道路を交差点 $(i, l) \rightarrow \dots \rightarrow (i, r)$ で通るとする

ここで、 $B_l, B_{l+1}, \dots, B_r$ の最小値を B_k とする

すると、 $(i-1, l) \rightarrow \dots \rightarrow (i-1, k) \rightarrow (i, k) \rightarrow (i+1, k) \rightarrow \dots \rightarrow (i+1, r)$ の経路にした方が絶対に得だから、 A_i の道路は“存在意義がない” (B_j の道路についても同様)



小課題 2 — $A_i, B_j \leq 1000$

重要な性質

以下の道路は絶対に通らない

- $A_{i-1} \leq A_i \geq A_{i+1}$ となるような、横方向の道路 i
- $B_{i-1} \leq B_i \geq B_{i+1}$ となるような、縦方向の道路 j

この要領で道路を消していくとどうなる…?



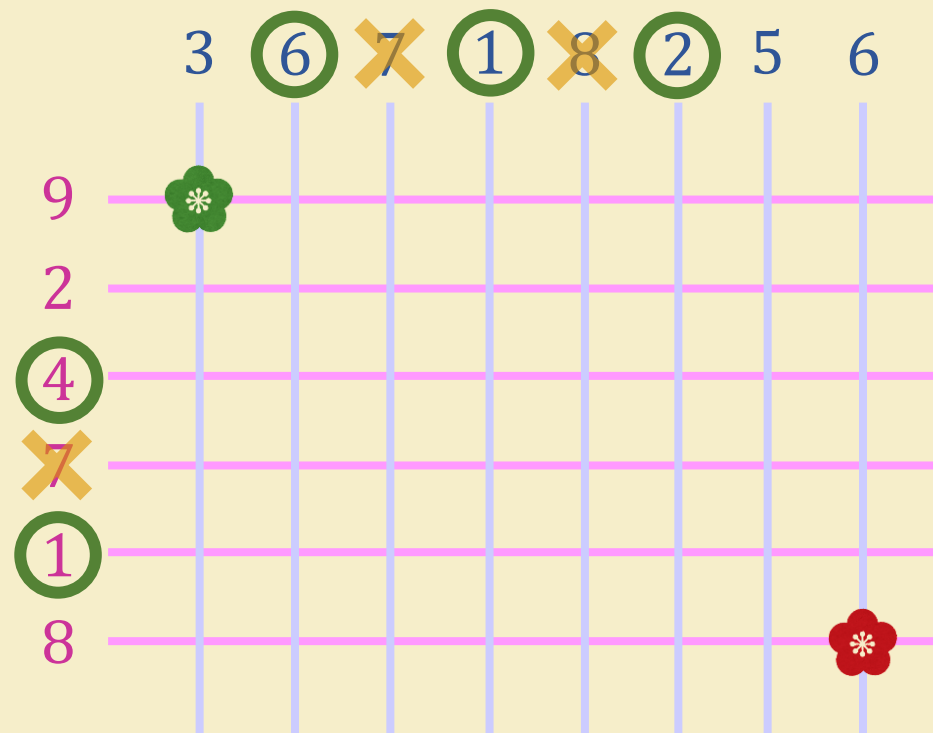
小課題 2 — $A_i, B_j \leq 1000$

重要な性質

以下の道路は絶対に通らない

- $A_{i-1} \leq A_i \geq A_{i+1}$ となるような、横方向の道路 i
- $B_{i-1} \leq B_i \geq B_{i+1}$ となるような、縦方向の道路 j

この要領で道路を消していくとどうなる…？



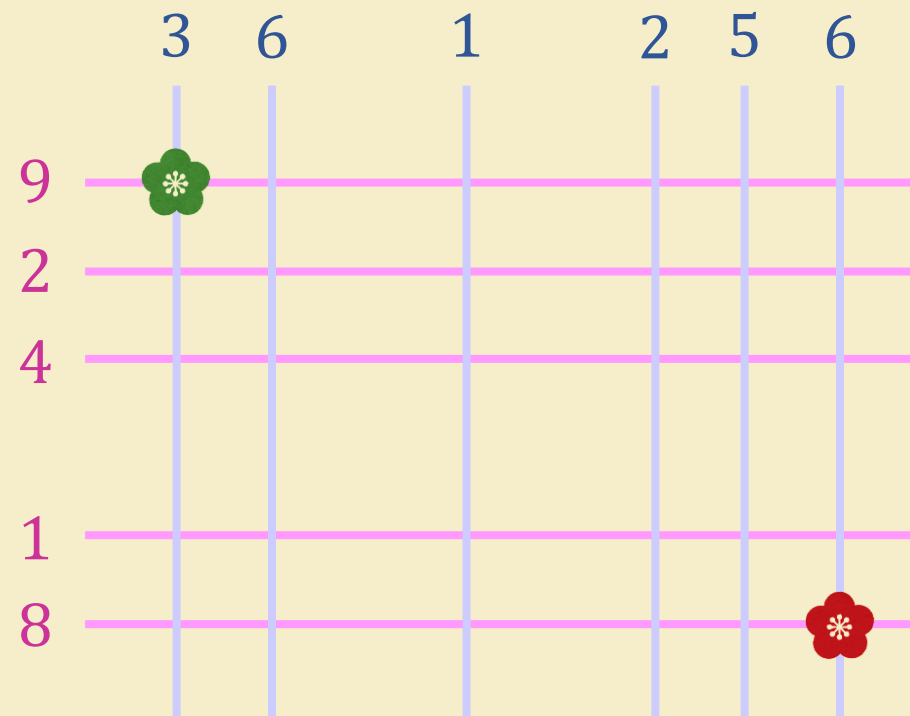
小課題 2 — $A_i, B_j \leq 1000$

重要な性質

以下の道路は絶対に通らない

- $A_{i-1} \leq A_i \geq A_{i+1}$ となるような、横方向の道路 i
- $B_{i-1} \leq B_i \geq B_{i+1}$ となるような、縦方向の道路 j

この要領で道路を消していくとどうなる…?



小課題 2 — $A_i, B_j \leq 1000$

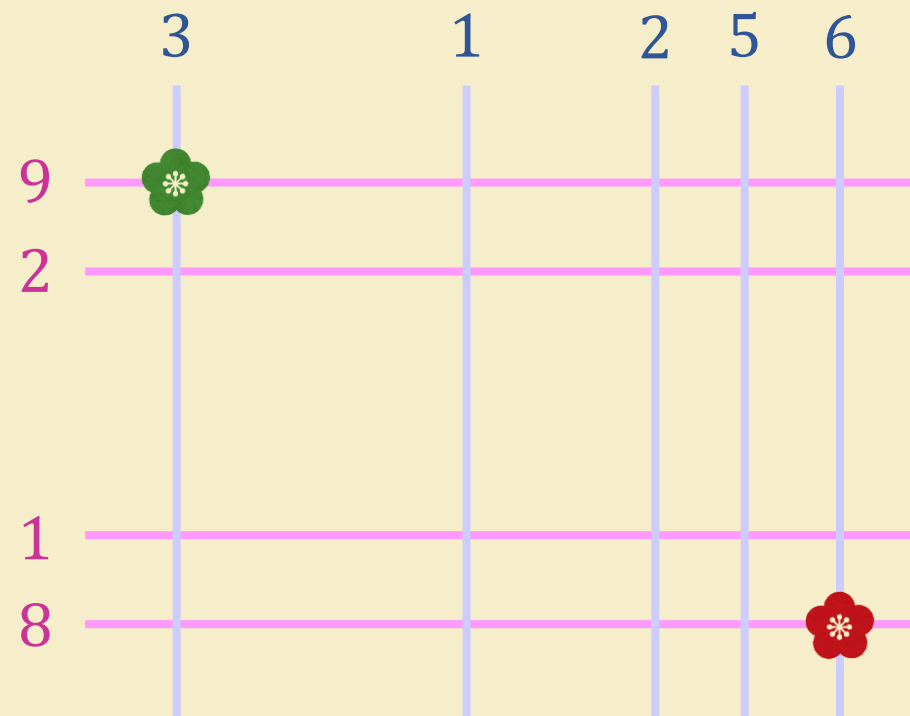
重要な性質

以下の道路は絶対に通らない

- $A_{i-1} \leq A_i \geq A_{i+1}$ となるような、横方向の道路 i
- $B_{i-1} \leq B_i \geq B_{i+1}$ となるような、縦方向の道路 j

この要領で道路を消していくとどうなる…？

→ たくさんの道路が消えた！



小課題 2 — $A_i, B_j \leq 1000$

結局、残る道路は…？

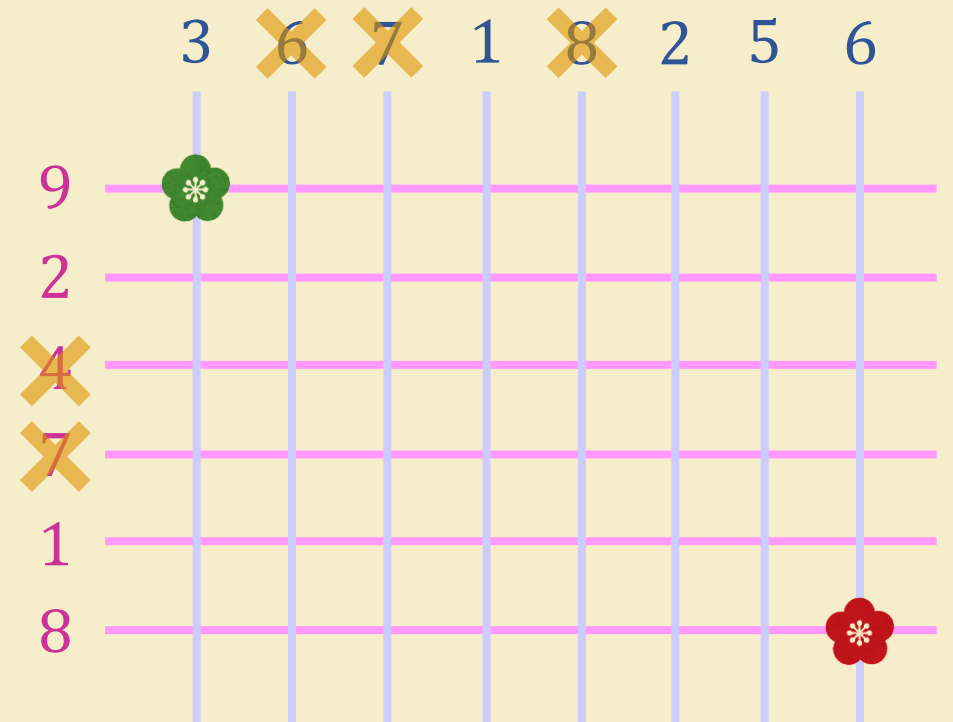
- A_i の左から or 右からの最小値を更新する道路
- B_j の左から or 右からの最小値を更新する道路

$A_i, B_j \leq 1000$ の場合…

残る道路は縦横それぞれ 2000 本以下

→ これなら DP で最短経路が求まる！

全体計算量は $O(MAX^2)$ [ただし MAX は A_i, B_j の最大値]



満点解法

制約: $H, W \leq 10^5$



小課題 3 — $H, W \leq 100000$

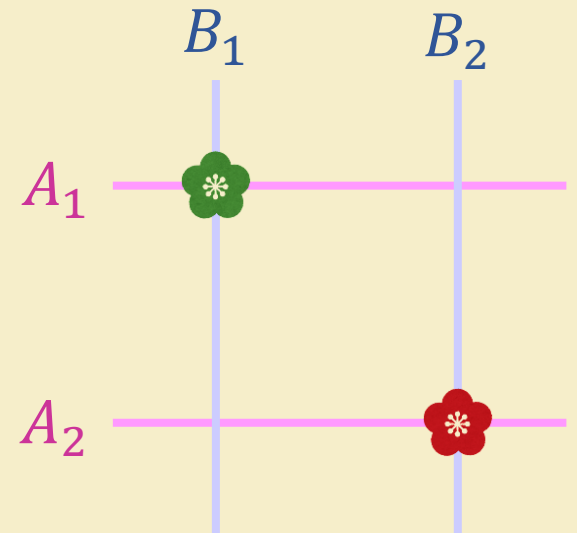
ところで、 $H = 2, W = 2$ の場合を考えよう

$H = 2, W = 2$ の場合

経路は 2 つあるが、どちらを通った方がよいか？

- $(1, 1) \rightarrow (1, 2) \rightarrow (2, 2)$: $A_1 + B_2$ 秒
- $(1, 1) \rightarrow (2, 1) \rightarrow (2, 2)$: $B_1 + A_2$ 秒

$A_1 + B_2 < B_1 + A_2 \Leftrightarrow A_2 - A_1 > B_2 - B_1$ のとき前者の方が得



小課題 3 — $H, W \leq 100000$

ところで、 $H = 2, W = 2$ の場合を考えよう

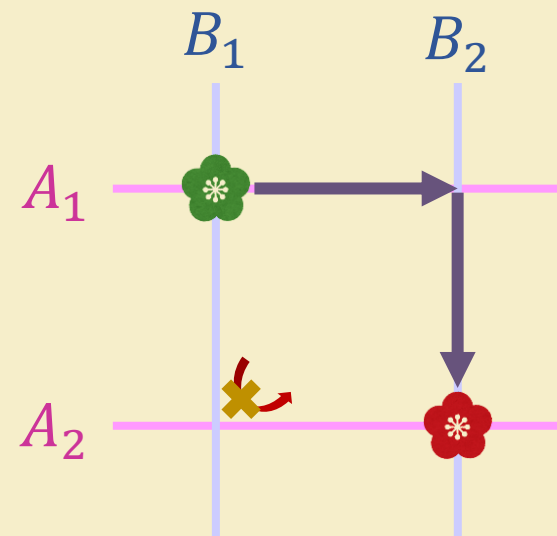
$H = 2, W = 2$ の場合

経路は 2 つあるが、どちらを通った方がよいか？

- $(1, 1) \rightarrow (1, 2) \rightarrow (2, 2)$: $A_1 + B_2$ 秒
- $(1, 1) \rightarrow (2, 1) \rightarrow (2, 2)$: $B_1 + A_2$ 秒

$A_1 + B_2 < B_1 + A_2 \Leftrightarrow A_2 - A_1 > B_2 - B_1$ のとき前者の方が得

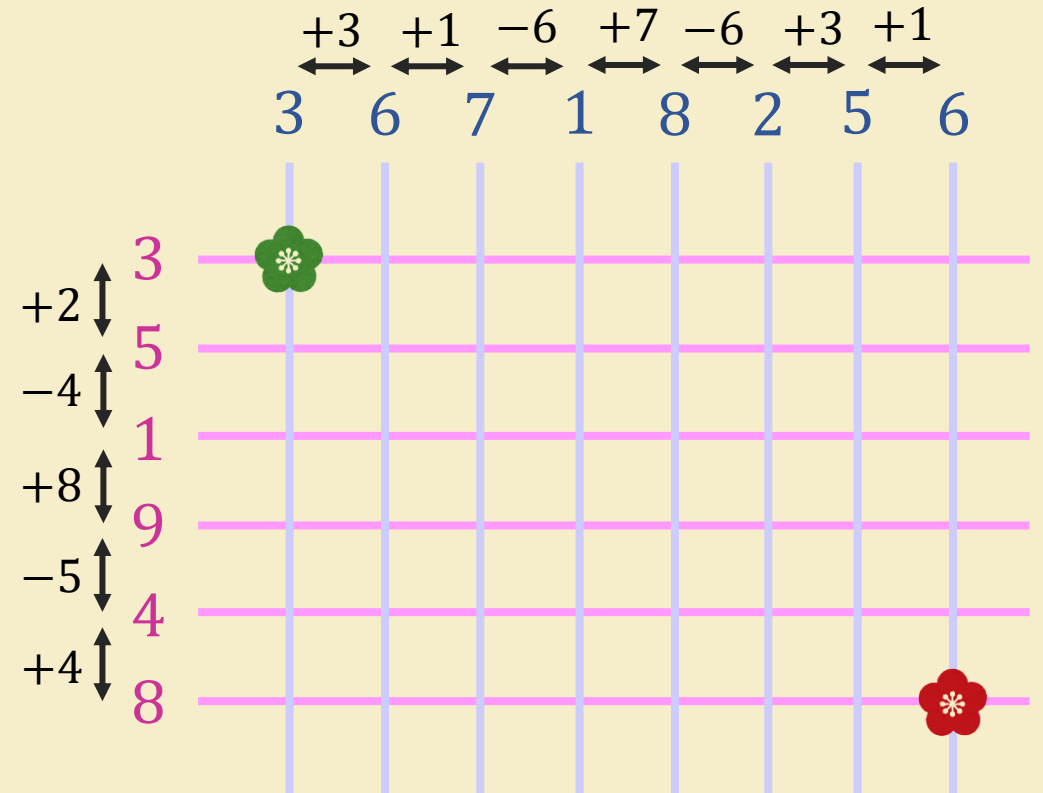
つまり、交差点 $(2, 1)$ で左向きに曲がることは許されない！



小課題 3 — $H, W \leq 100000$

これを、一般の場合に適用してみましょう

(右図の黒い数は「差分」を表しています)



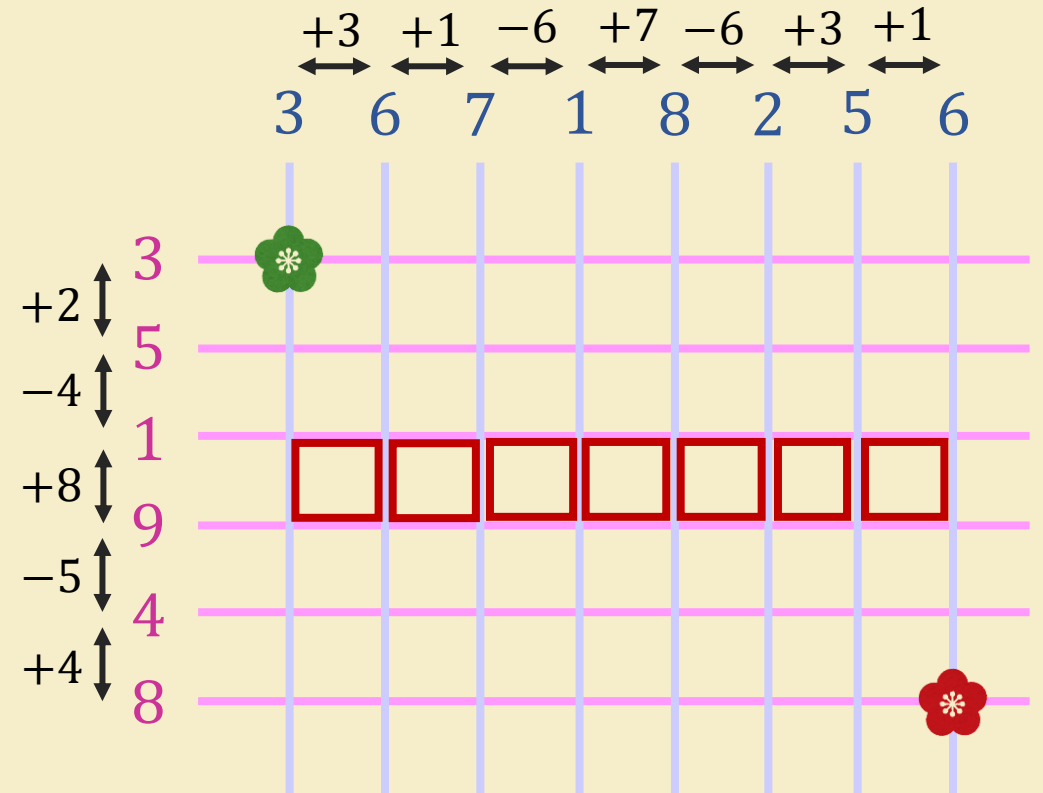
小課題 3 — $H, W \leq 100000$

これを、一般の場合に適用してみましょう

(右図の黒い数は「差分」を表しています)

「+8」は全体で最も大きい差分なので

この行のすべての四角に条件を適用させると…



小課題 3 — $H, W \leq 100000$

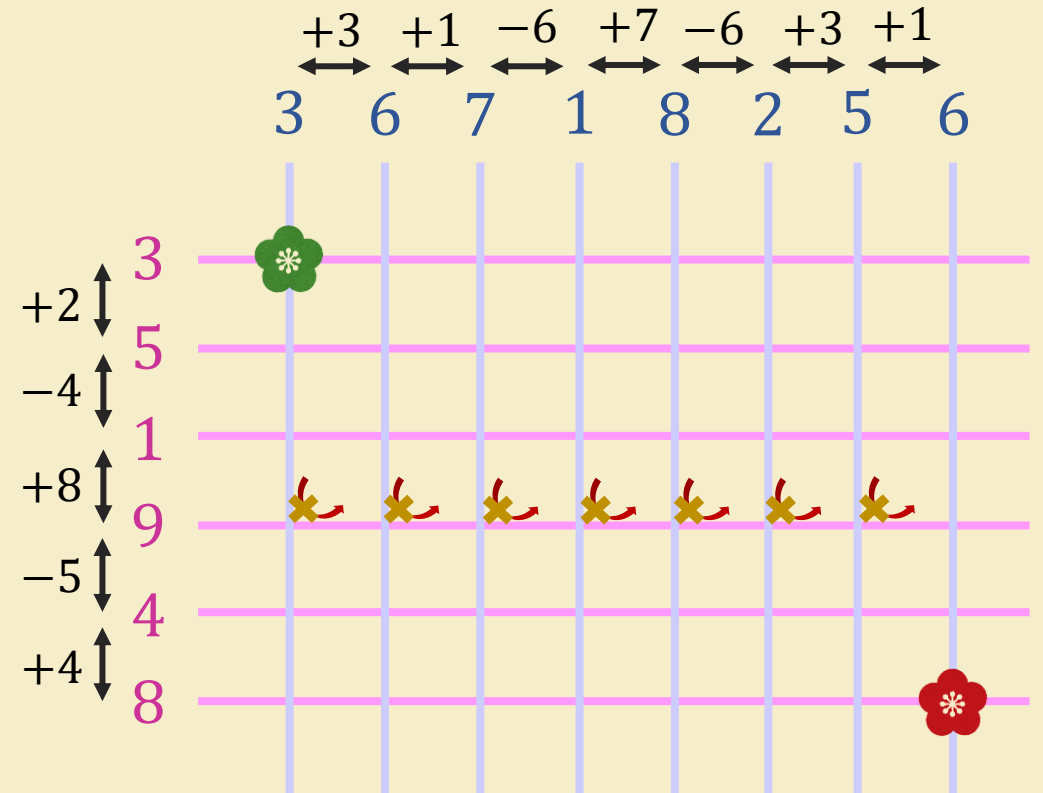
これを、一般の場合に適用してみましょう

(右図の黒い数は「差分」を表しています)

「+8」は全体で最も大きい差分なので

この行のすべての四角に条件を適用させると…

“左に曲がることはできない”



小課題 3 — $H, W \leq 100000$

これを、一般の場合に適用してみましょう

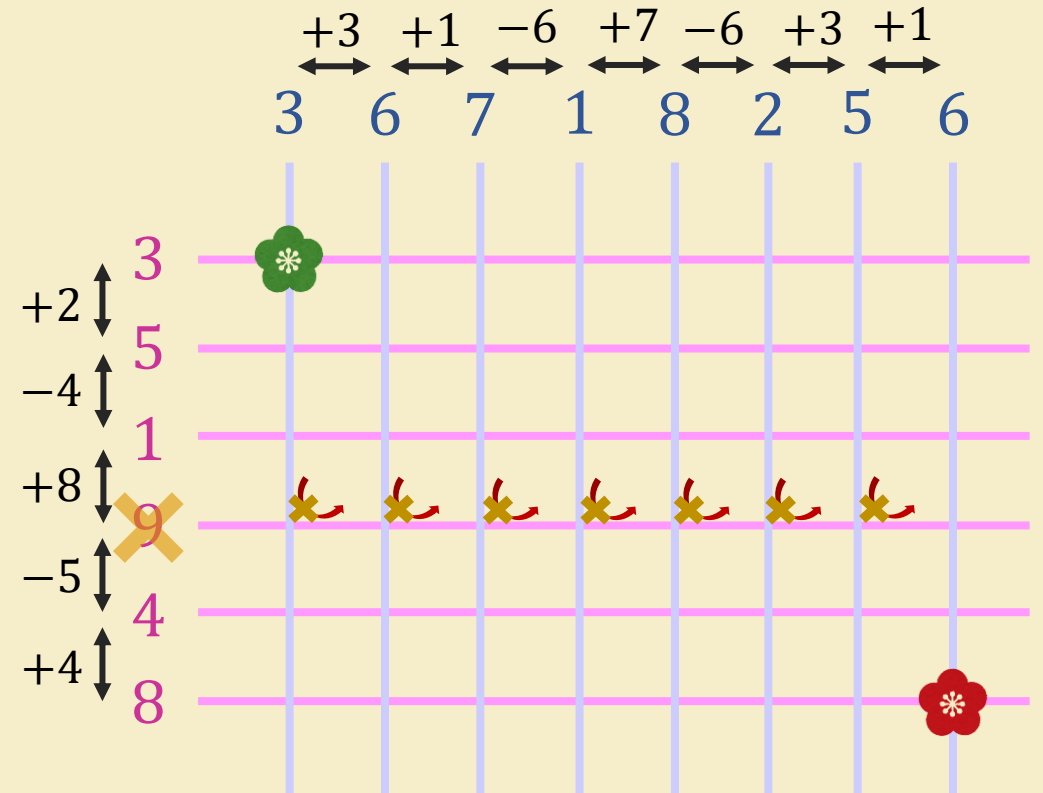
(右図の黒い数は「差分」を表しています)

「+8」は全体で最も大きい差分なので

この行のすべての四角に条件を適用させると…

“左に曲がることはできない”

つまり、「9」の道路は絶対に通ることができない!



小課題 3 — $H, W \leq 100000$

これを、一般の場合に適用してみましょう

(右図の黒い数は「差分」を表しています)

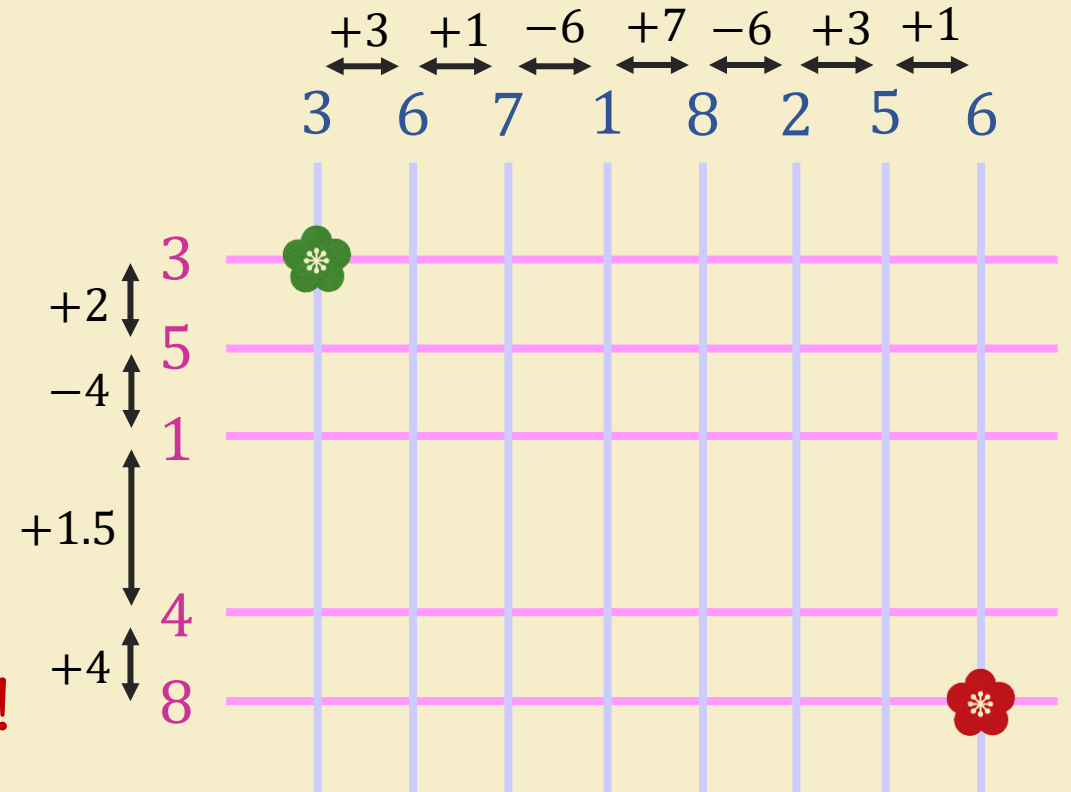
「+8」は全体で最も大きい差分なので

この行のすべての四角に条件を適用させると…

“左に曲がることはできない”

つまり、「9」の道路は絶対に通ることができない!

これで 1 つ道路が消えた



小課題 3 — $H, W \leq 100000$

これを、一般の場合に適用してみましょう

(右図の黒い数は「差分」を表しています)

「+8」は全体で最も大きい差分なので

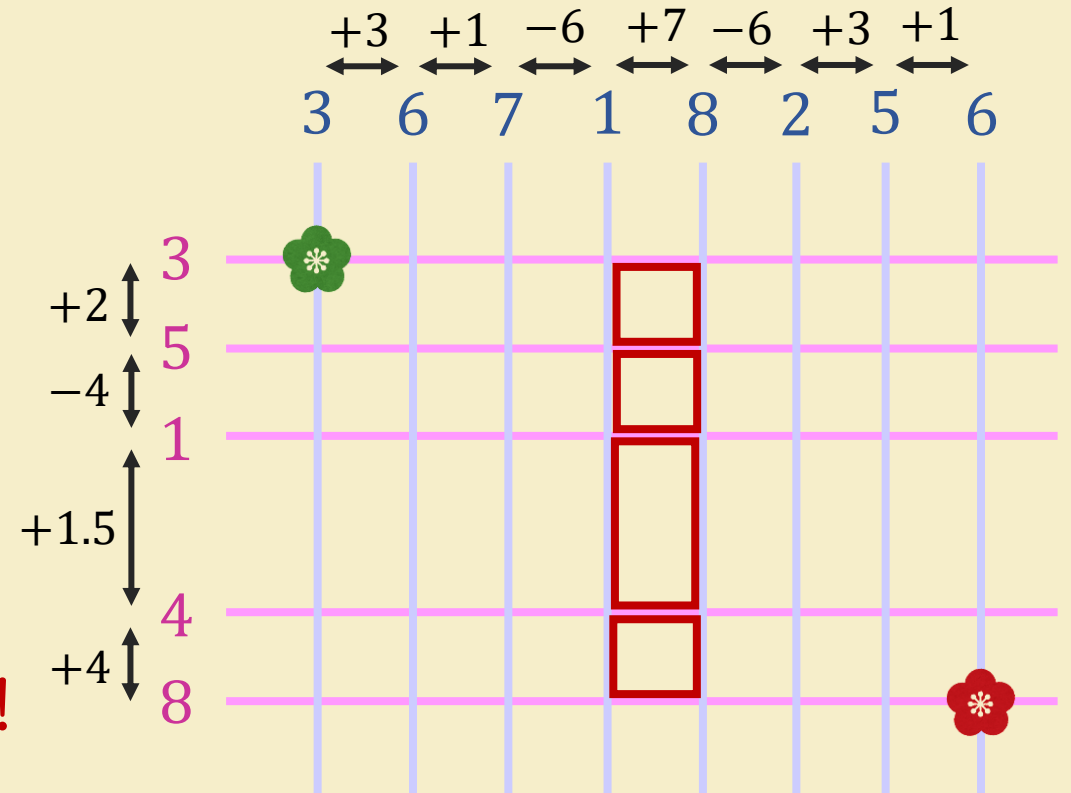
この行のすべての四角に条件を適用させると…

“左に曲がることはできない”

つまり、「9」の道路は絶対に通ることができない!

これで 1 つ道路が消えた

次に大きい「+7」に条件を適用させたい



小課題 3 — $H, W \leq 100000$

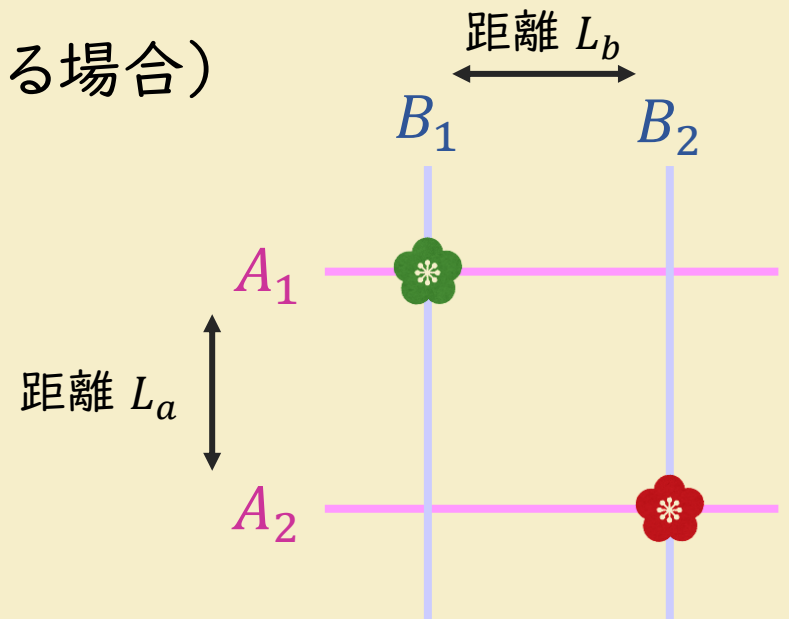
ここで、 $H = 2, W = 2$ に戻って考えよう（道路が離れている場合）

$H = 2, W = 2$ の場合

経路は 2 つあるが、どちらを通った方がよいか？

- $(1, 1) \rightarrow (1, 2) \rightarrow (2, 2)$: $A_1 \cdot L_b + B_2 \cdot L_a$ 秒
- $(1, 1) \rightarrow (2, 1) \rightarrow (2, 2)$: $B_1 \cdot L_a + A_2 \cdot L_b$ 秒

$A_1 \cdot L_b + B_2 \cdot L_a < B_1 \cdot L_a + A_2 \cdot L_b \Leftrightarrow \frac{A_2 - A_1}{L_a} > \frac{B_2 - B_1}{L_b}$ のとき前者の方が得



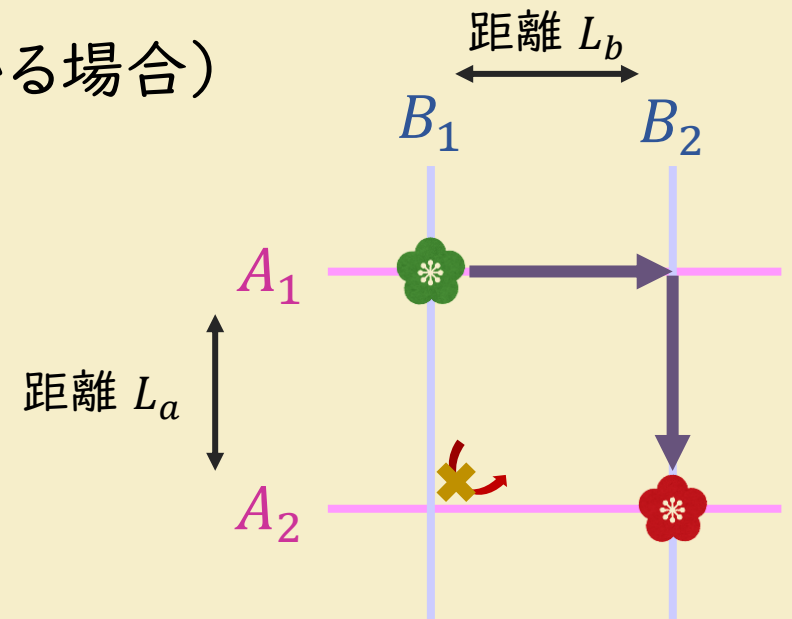
小課題 3 — $H, W \leq 100000$

ここで、 $H = 2, W = 2$ に戻って考えよう（道路が離れている場合）

$H = 2, W = 2$ の場合

経路は 2 つあるが、どちらを通った方がよいか？

- $(1, 1) \rightarrow (1, 2) \rightarrow (2, 2)$: $A_1 \cdot L_b + B_2 \cdot L_a$ 秒
- $(1, 1) \rightarrow (2, 1) \rightarrow (2, 2)$: $B_1 \cdot L_a + A_2 \cdot L_b$ 秒



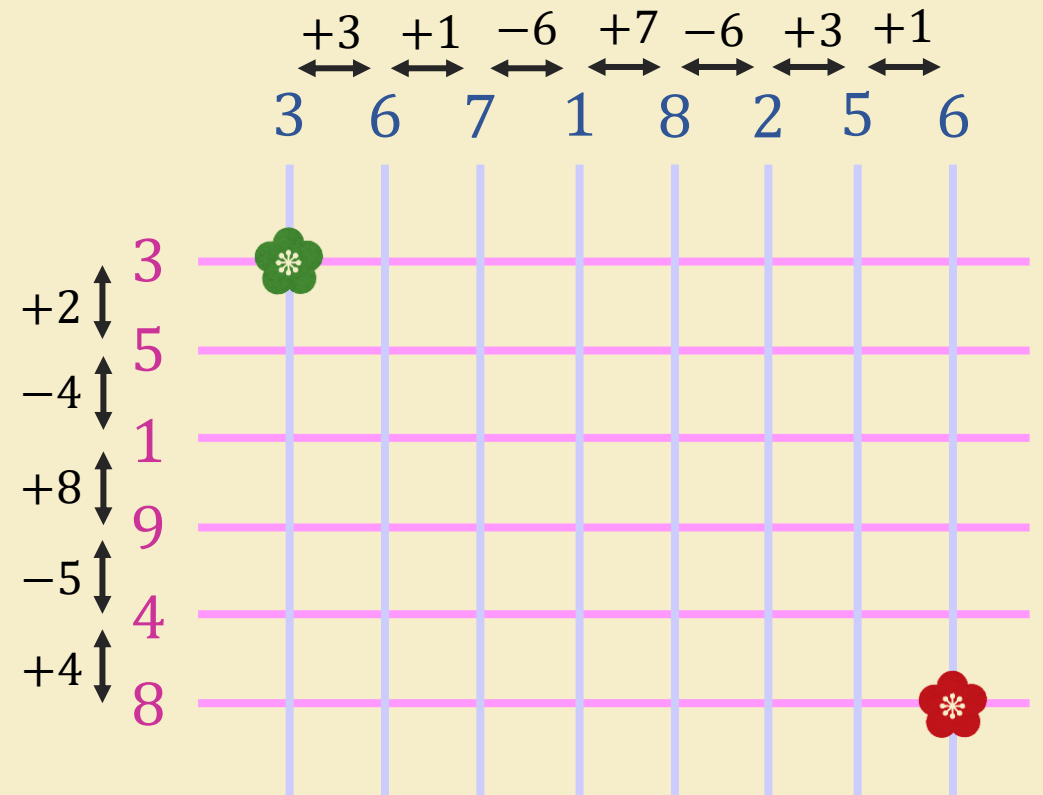
$A_1 \cdot L_b + B_2 \cdot L_a < B_1 \cdot L_a + A_2 \cdot L_b \Leftrightarrow \frac{A_2 - A_1}{L_a} > \frac{B_2 - B_1}{L_b}$ のとき前者の方が得

→ つまり、(差分) ÷ (距離) で運命が分かれる！（これを“傾き”ということにする）

小課題 3 — $H, W \leq 100000$

最初からやってみよう!

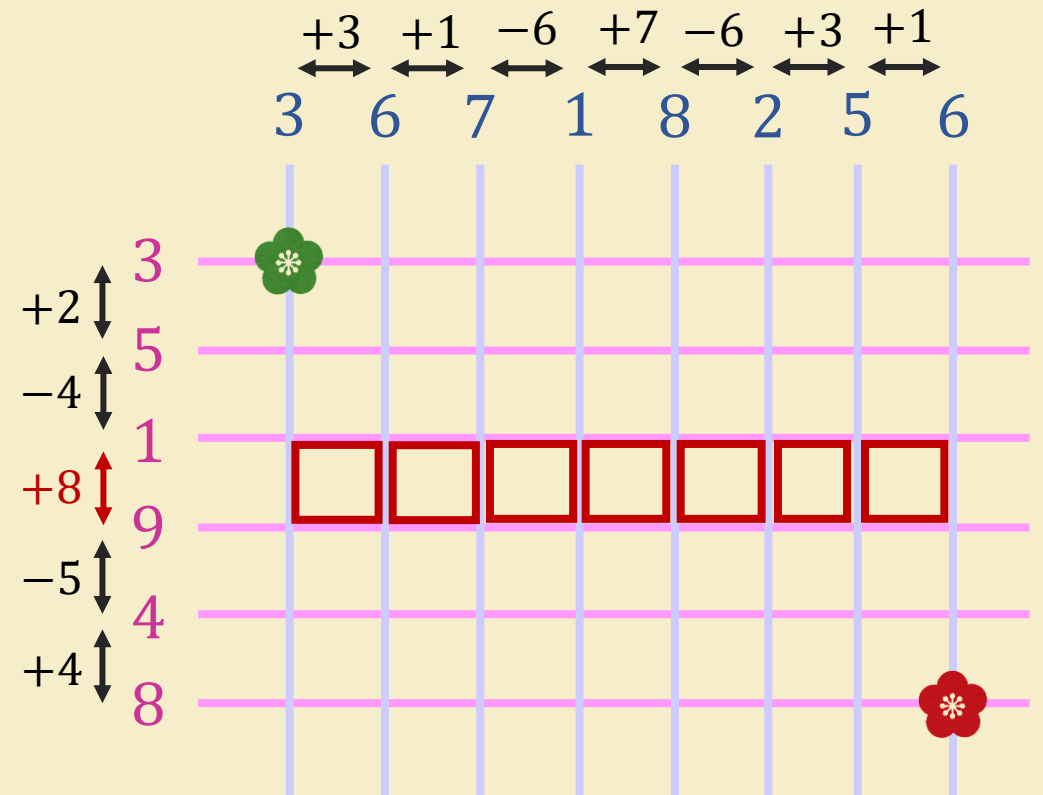
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

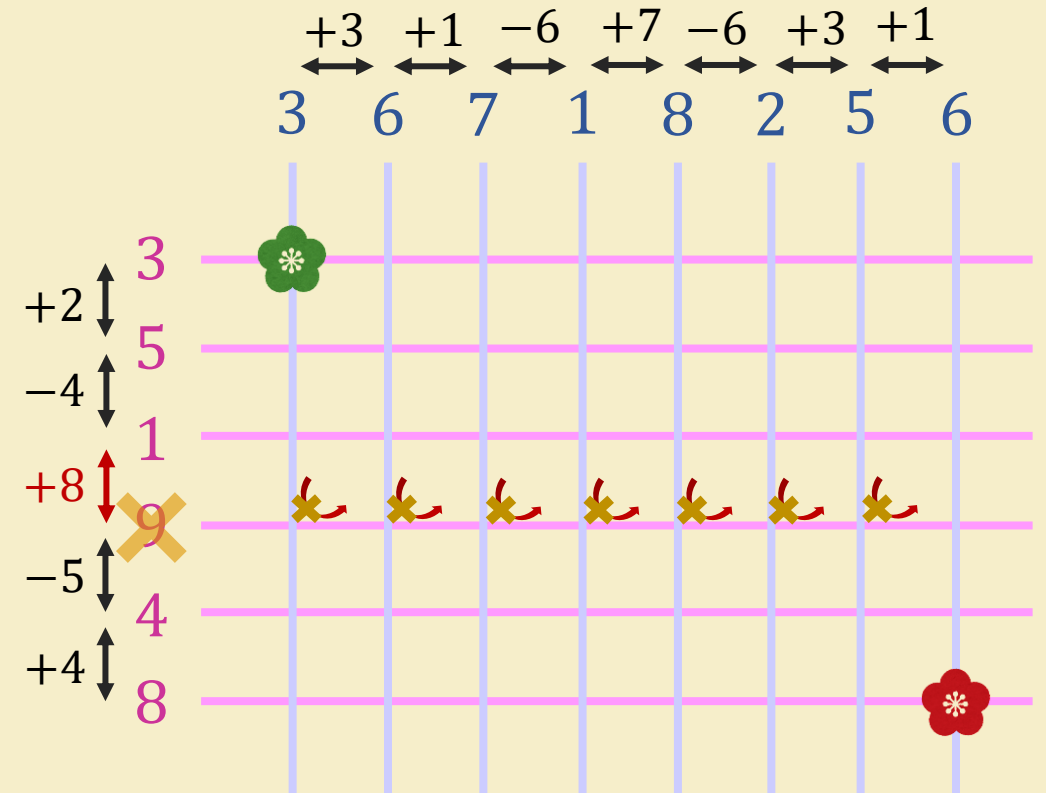
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

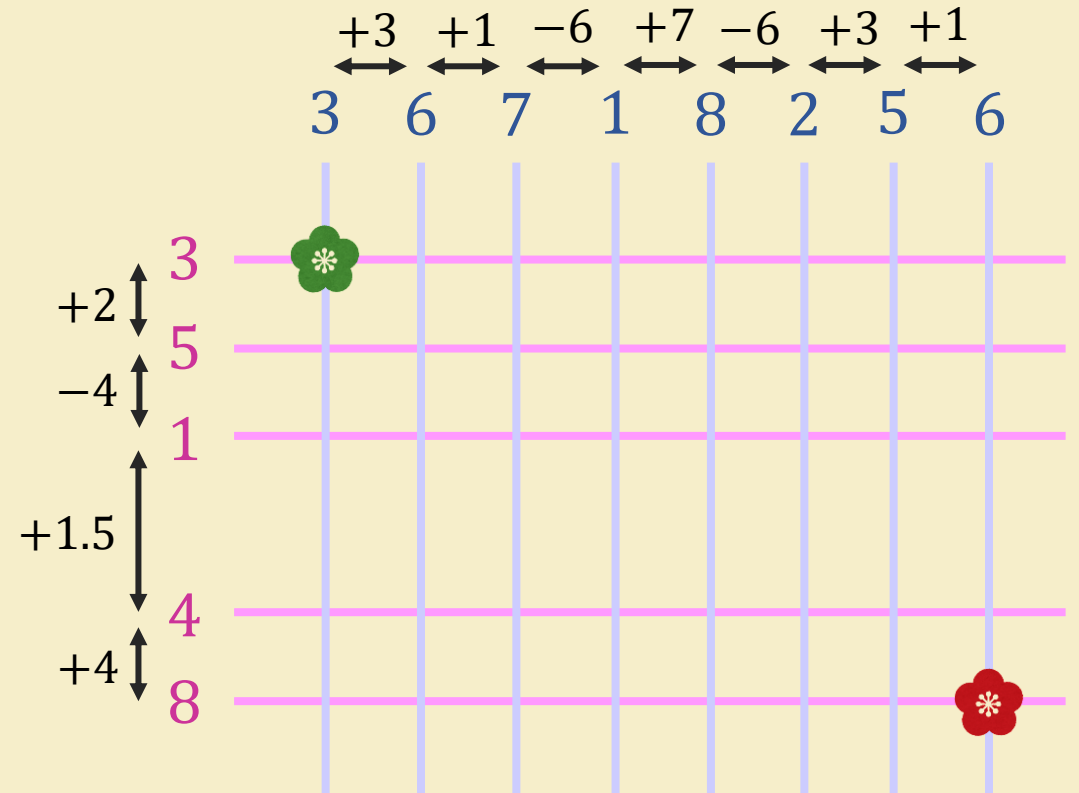
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

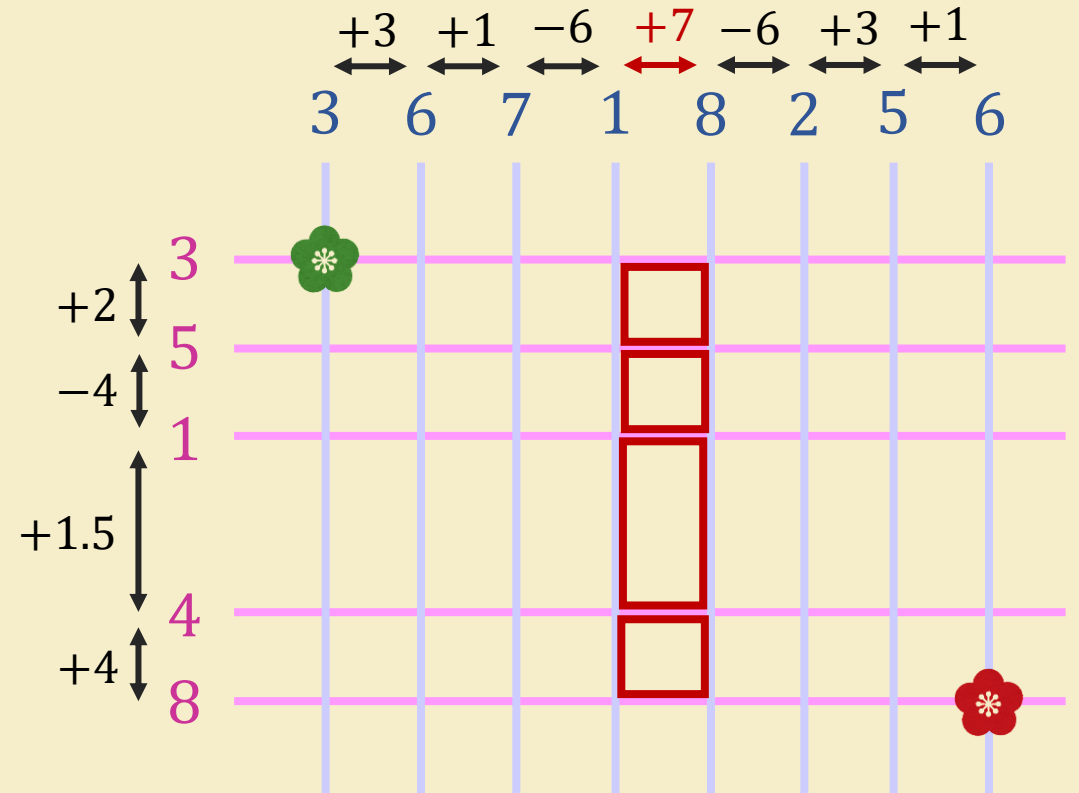
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

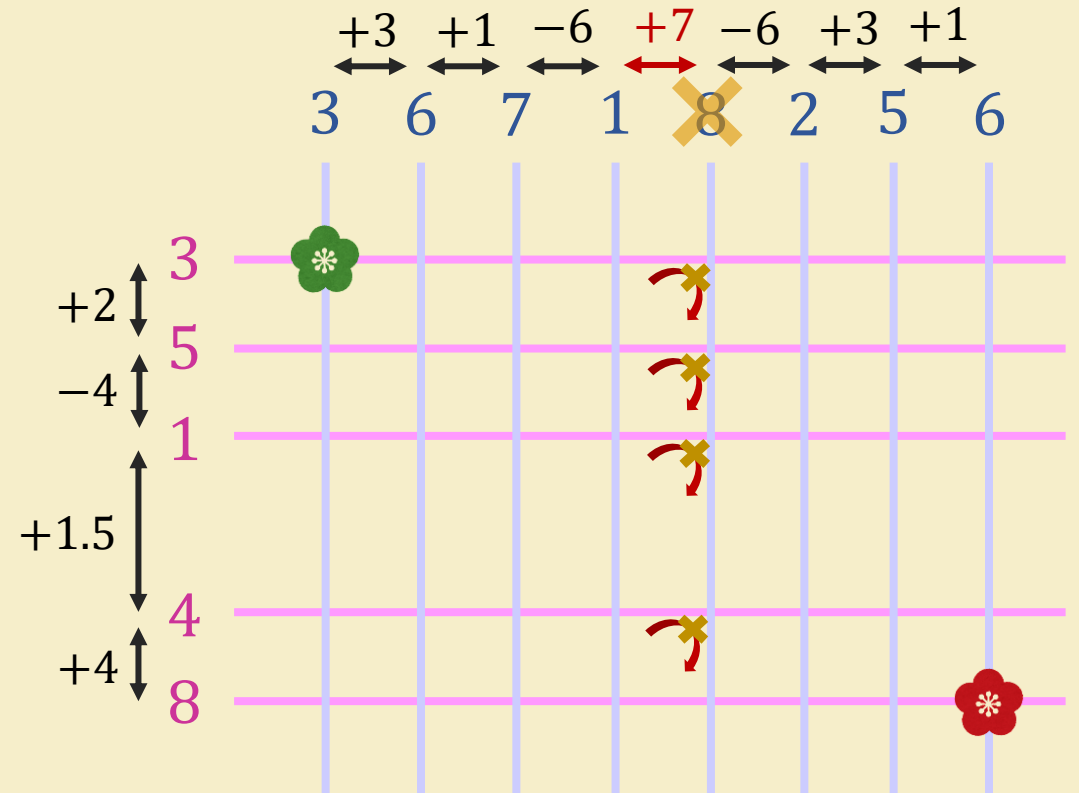
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

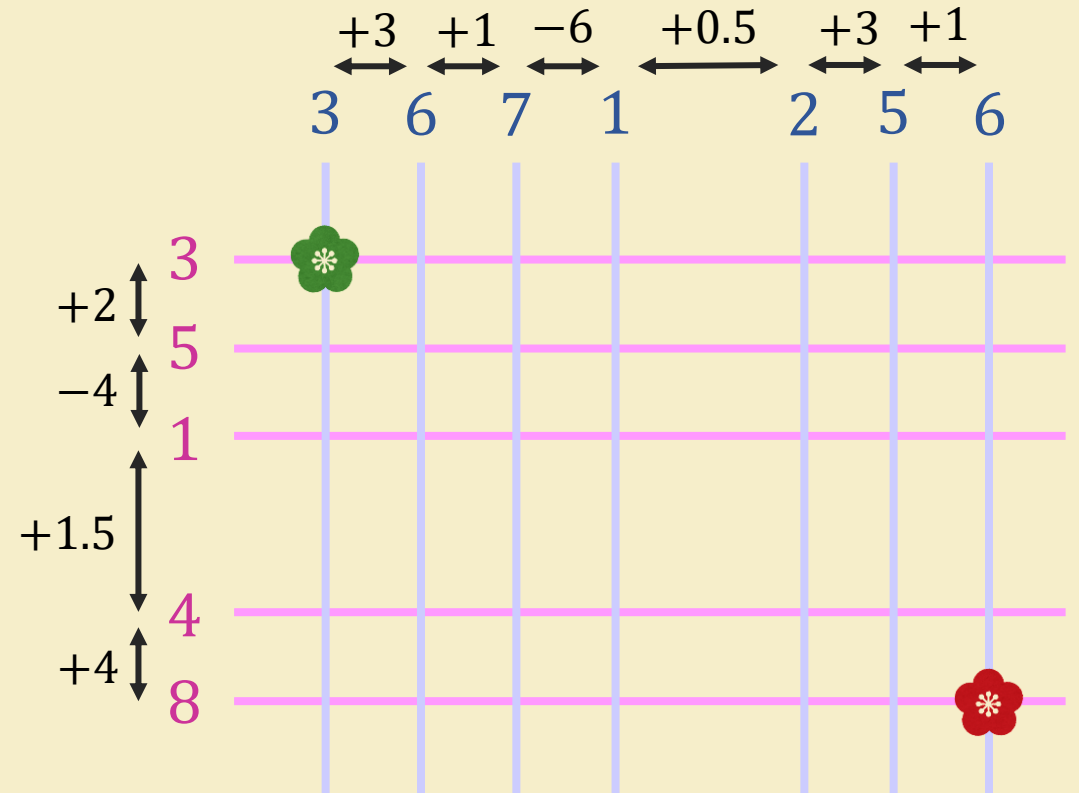
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

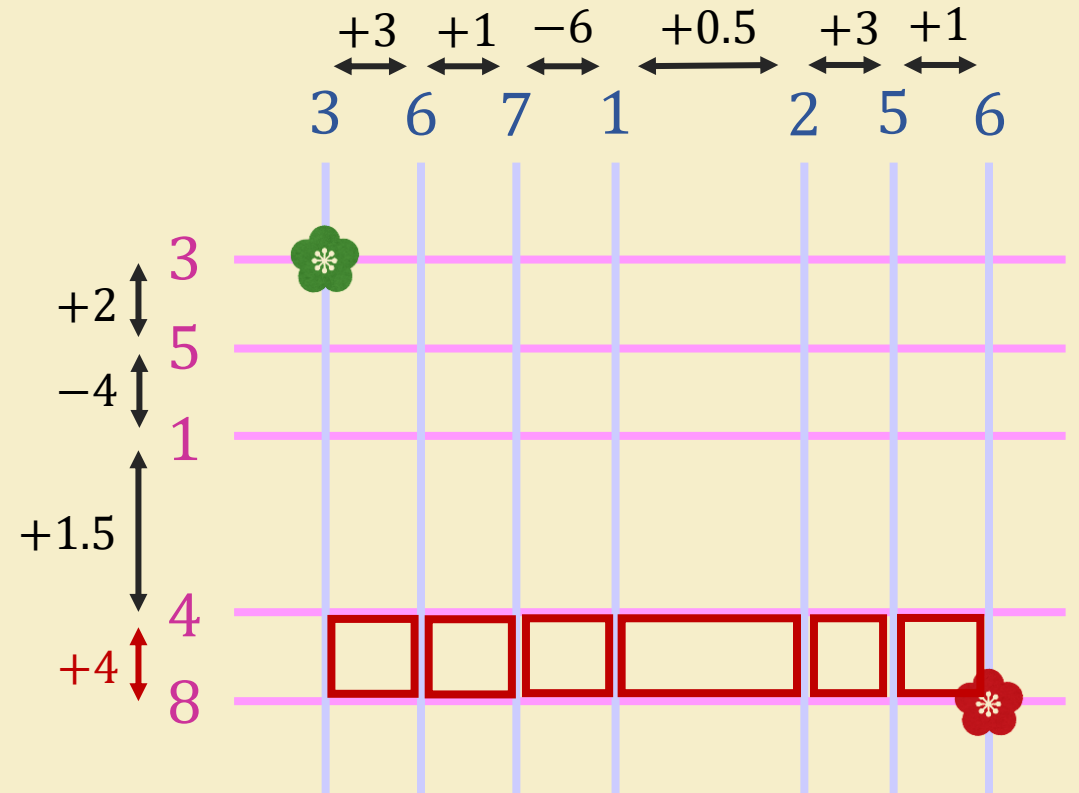
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

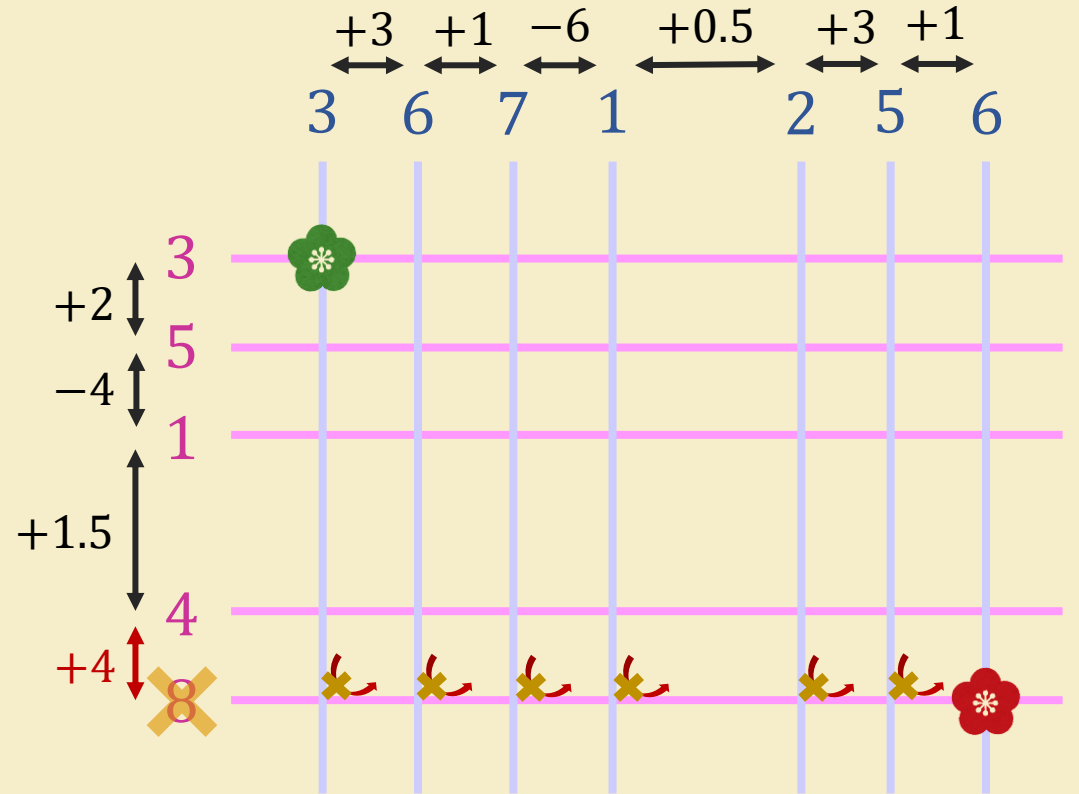
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

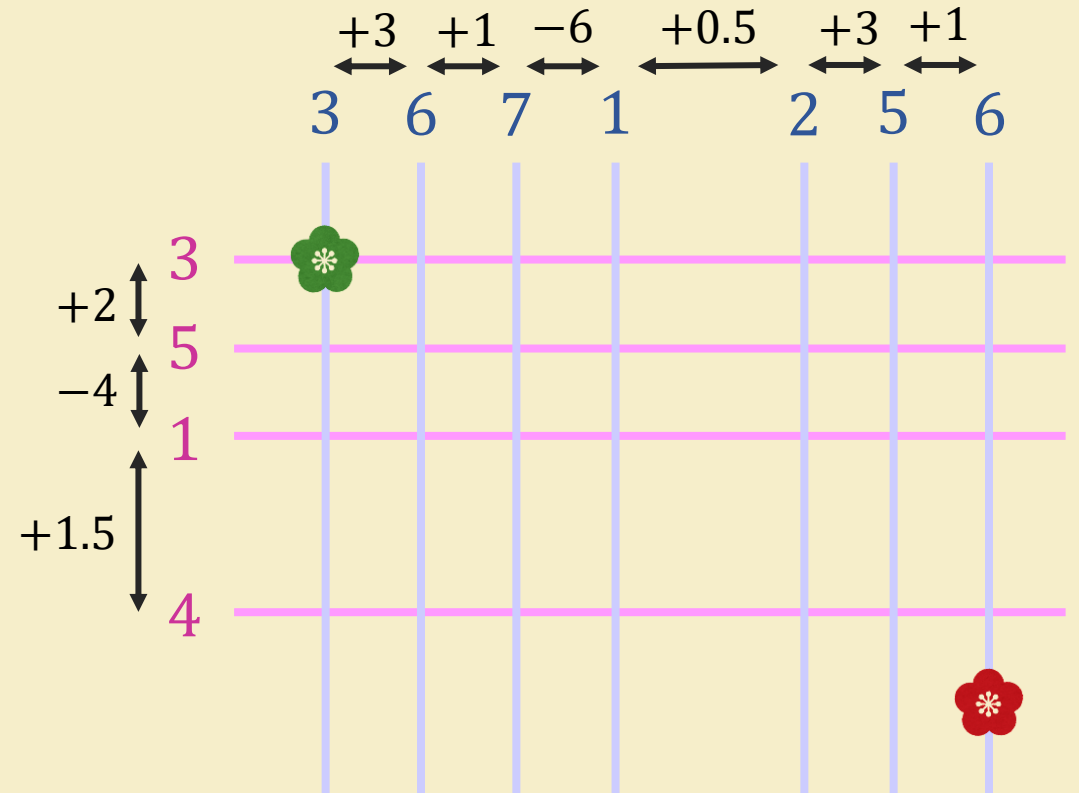
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

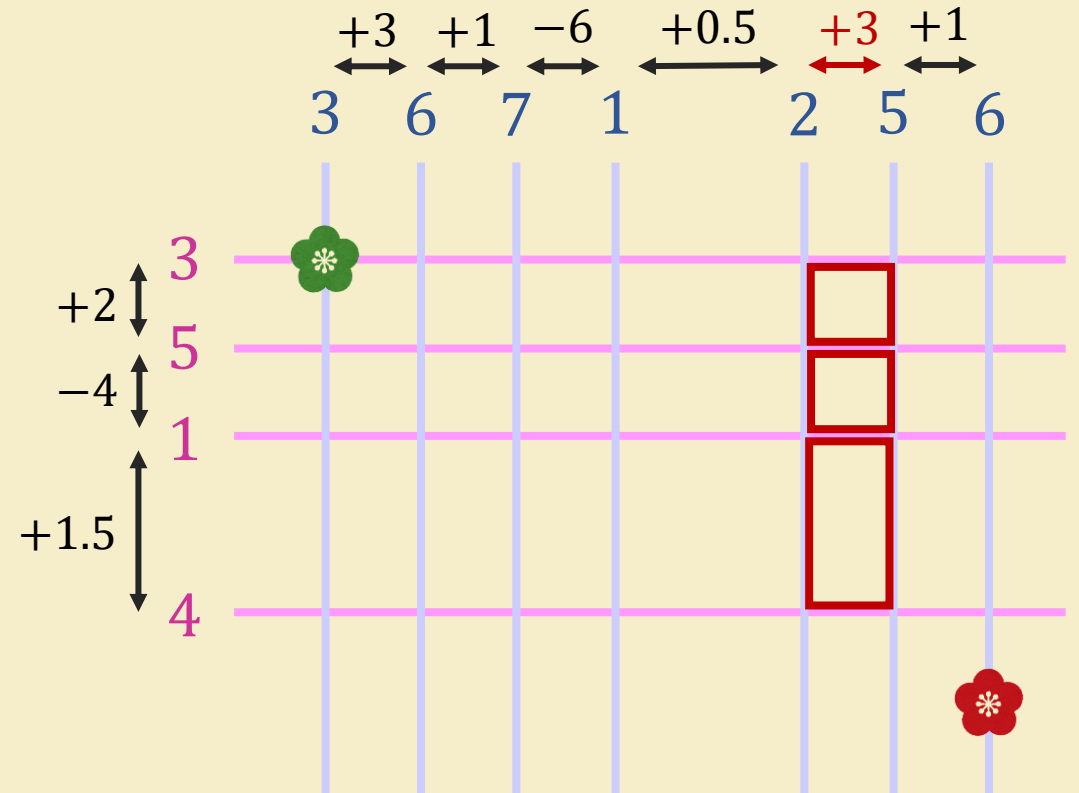
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

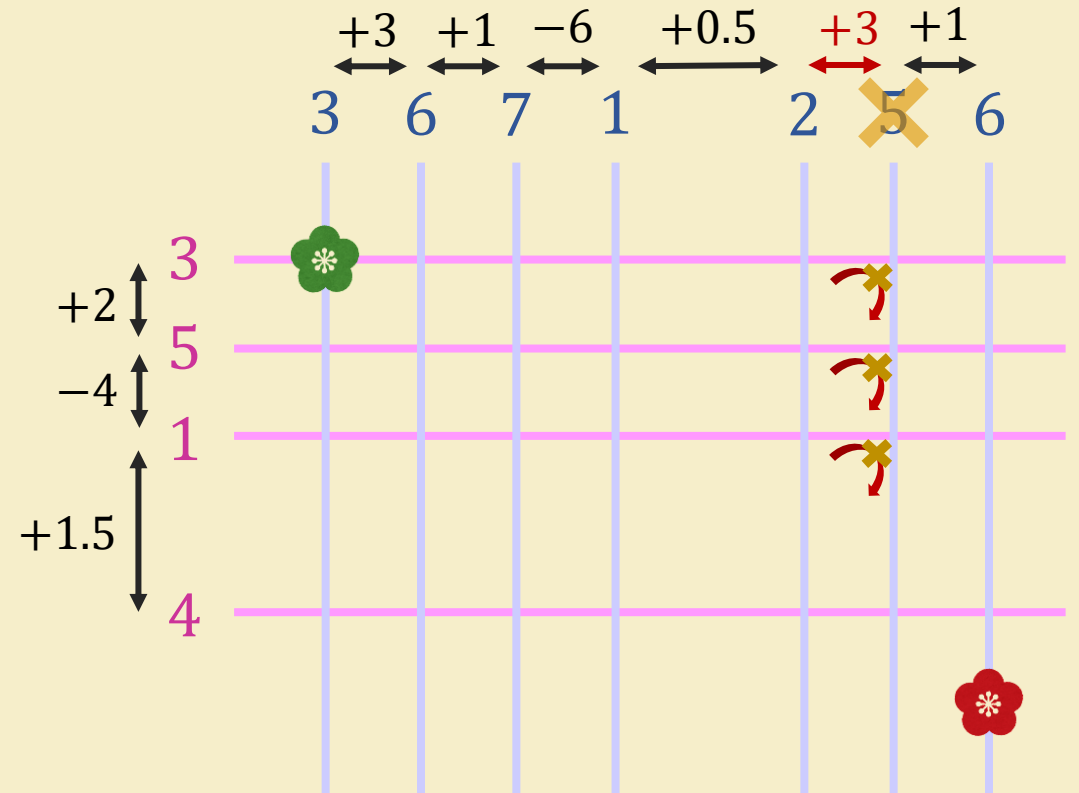
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

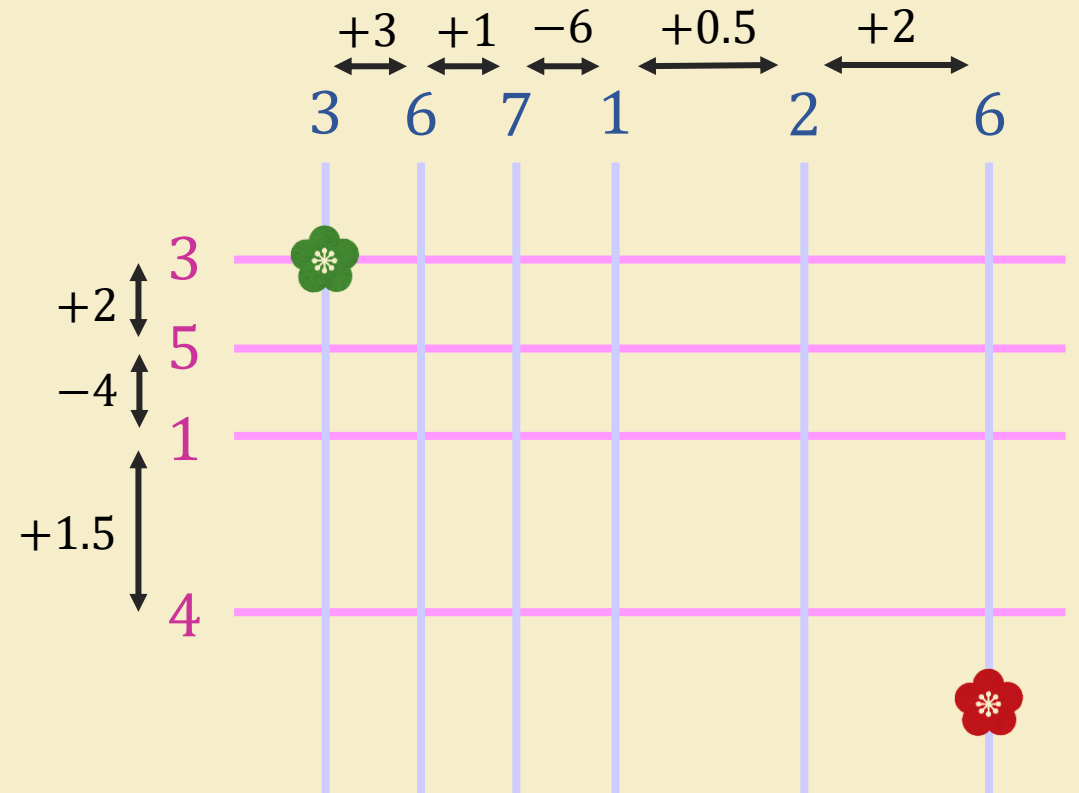
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

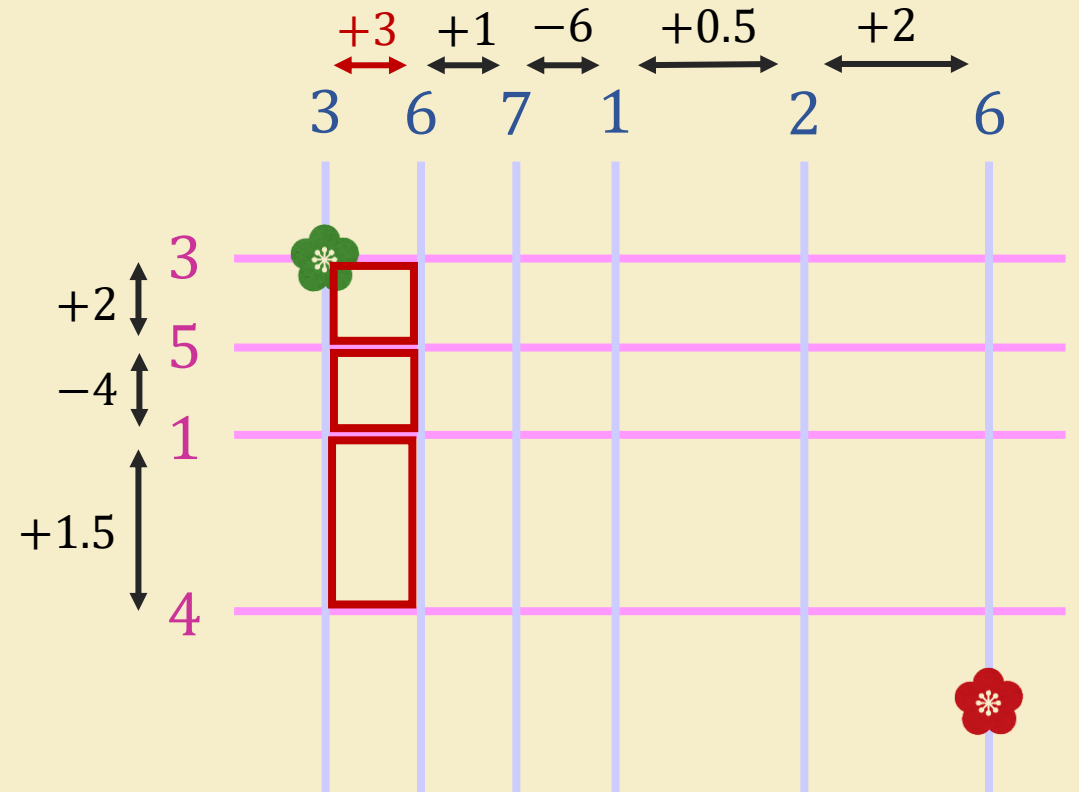
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

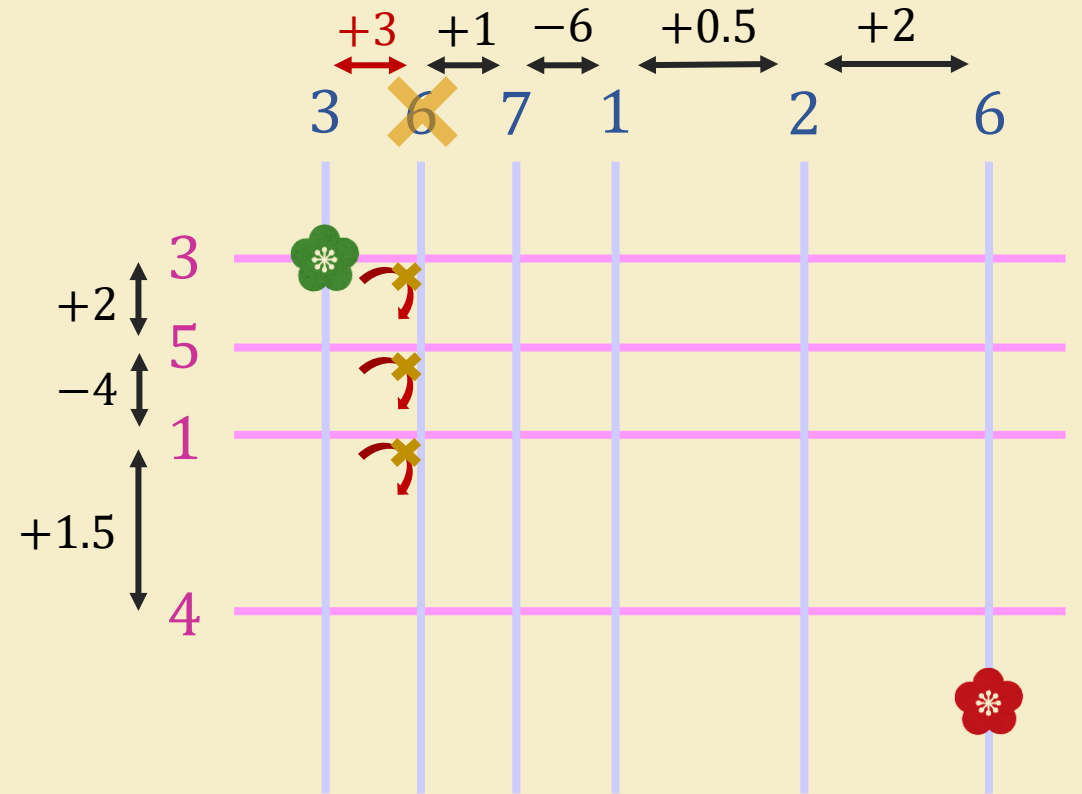
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

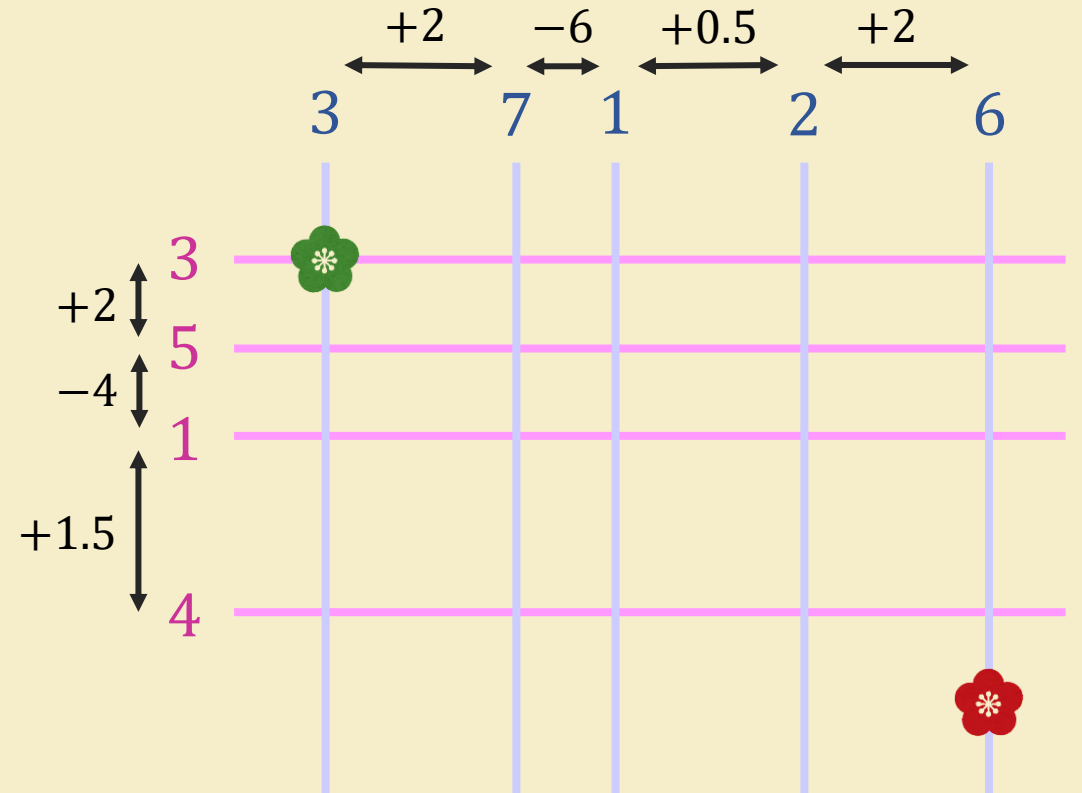
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

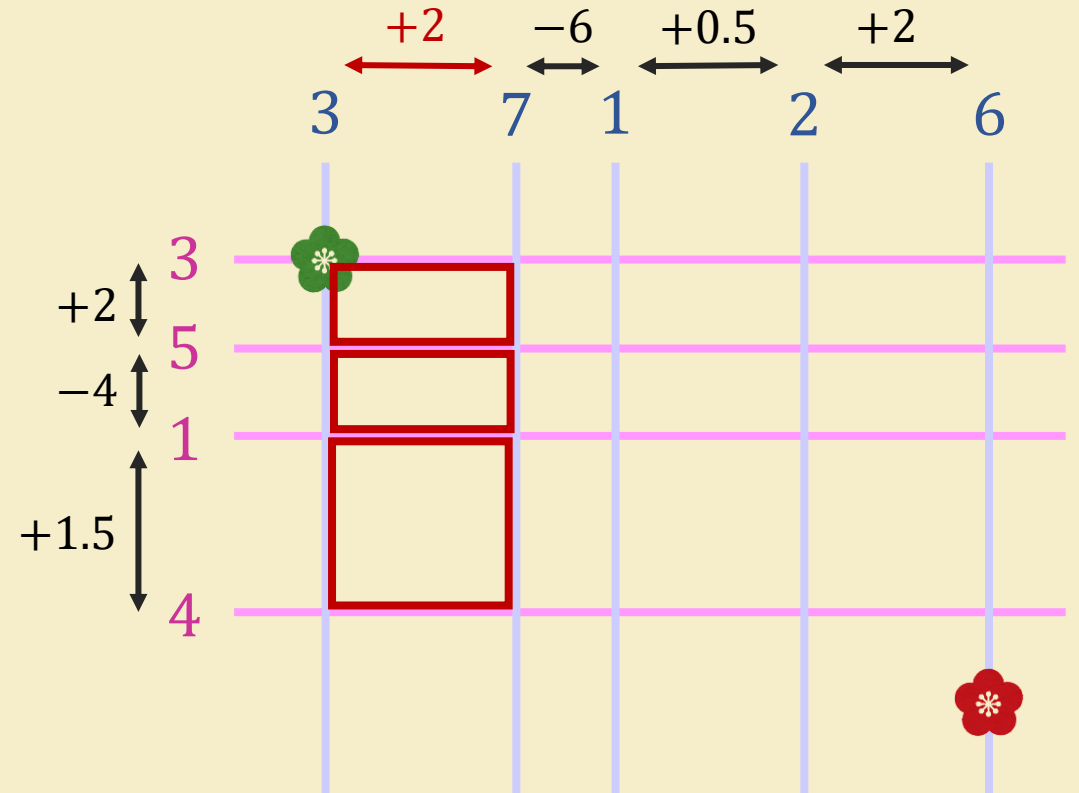
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

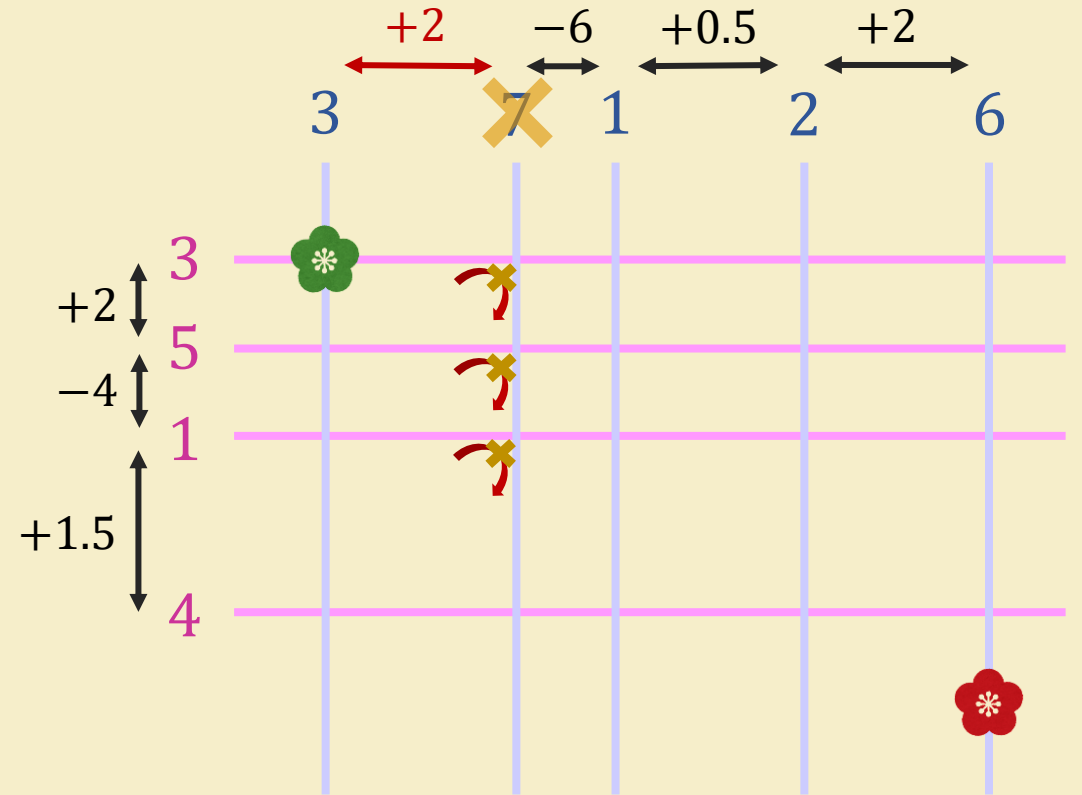
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

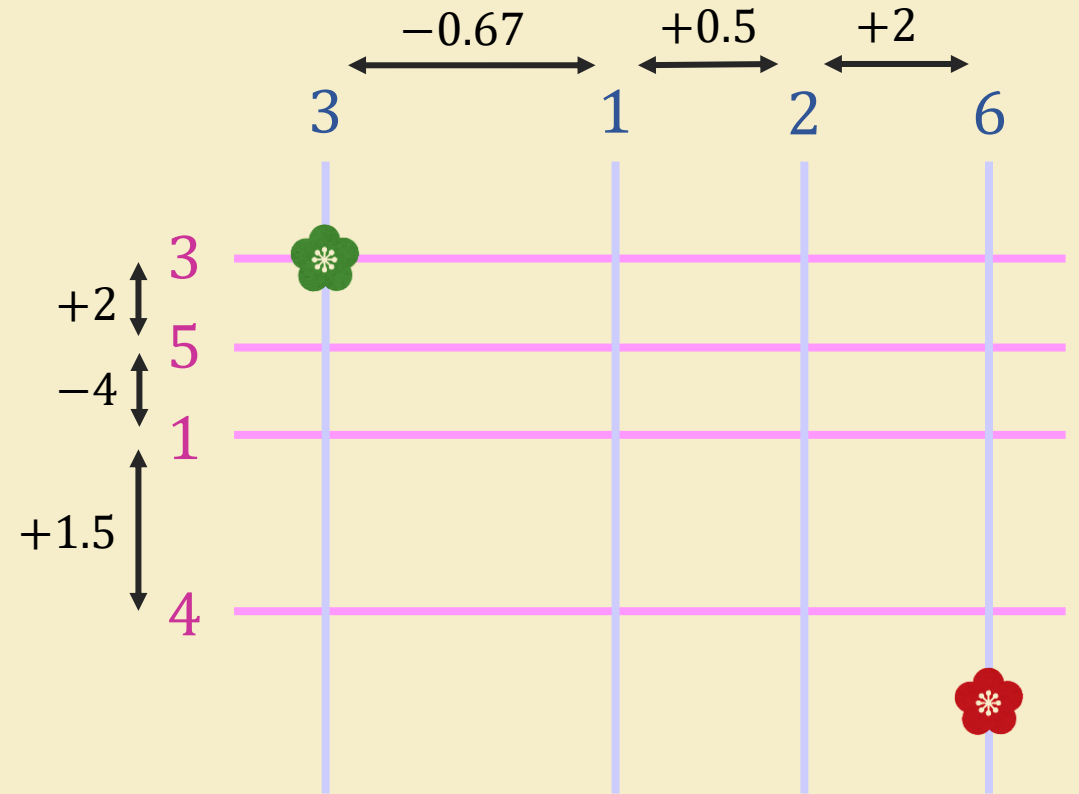
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

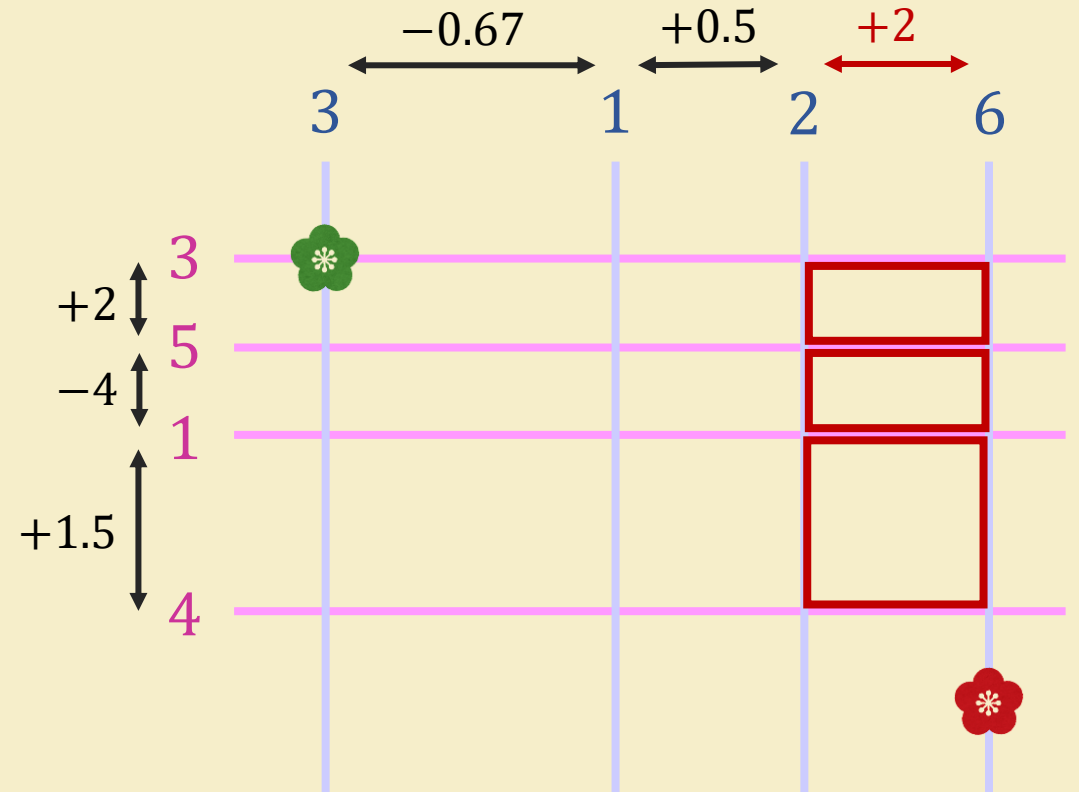
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

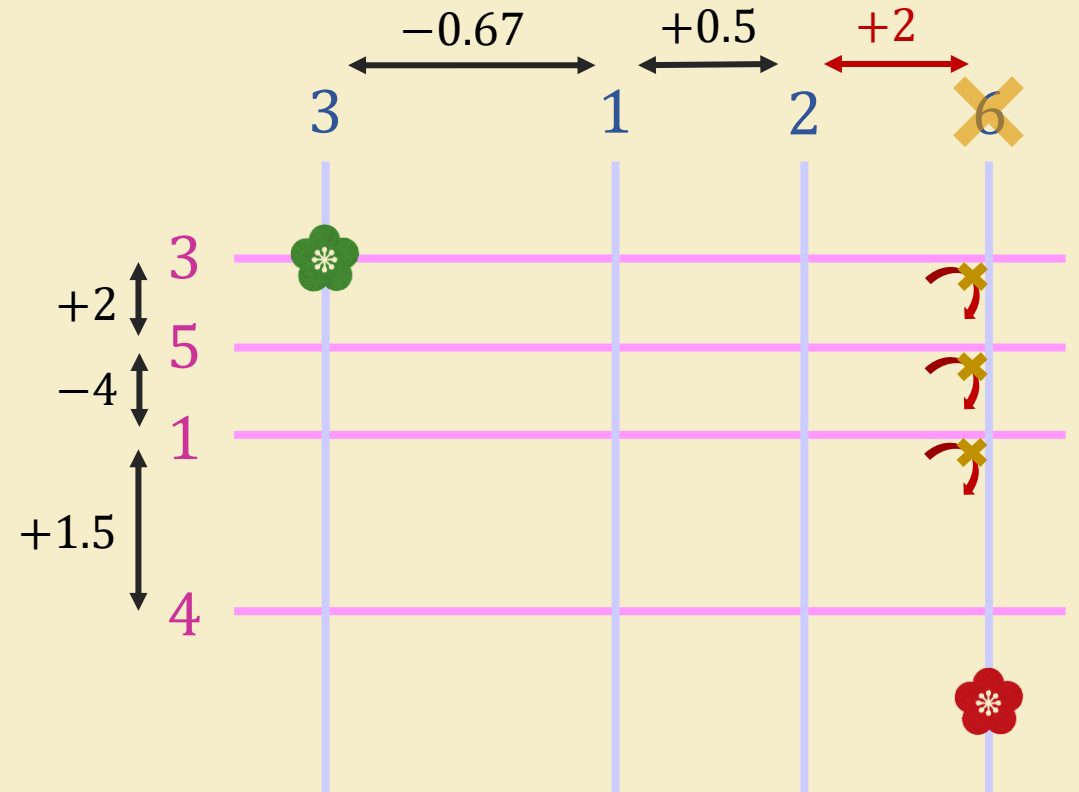


小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

四角に対して条件が適用されるので
道路の最後の部分だけ残ることに注意

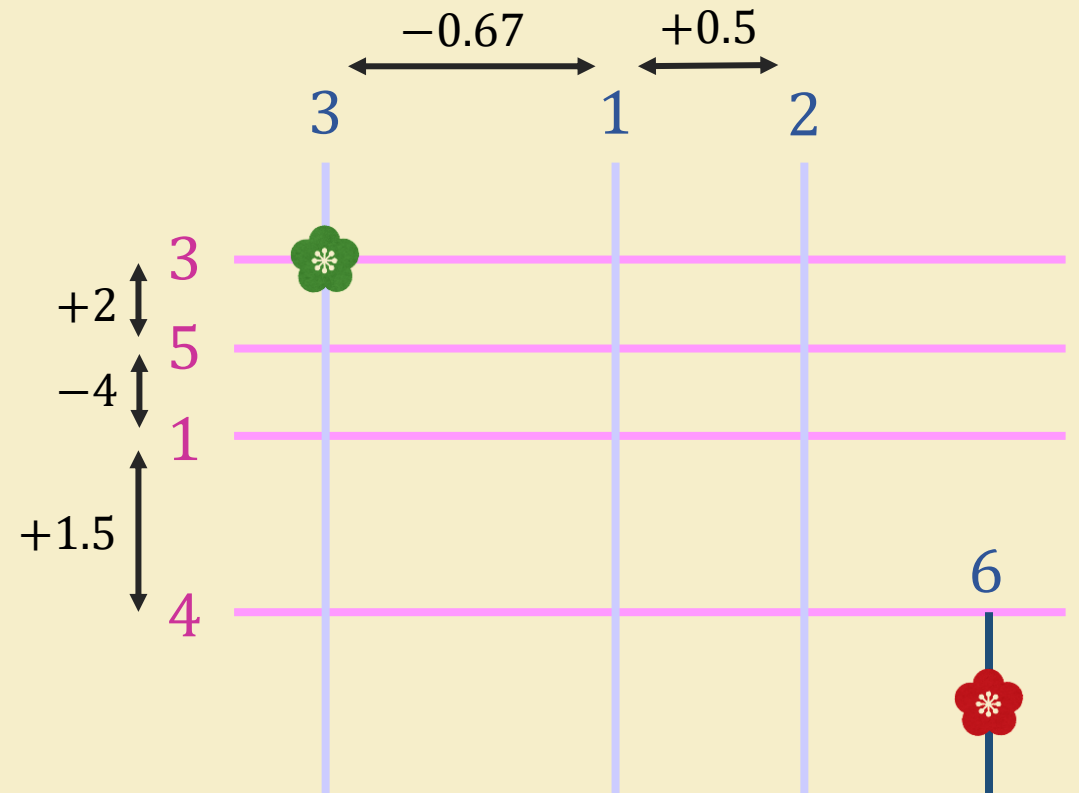


小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

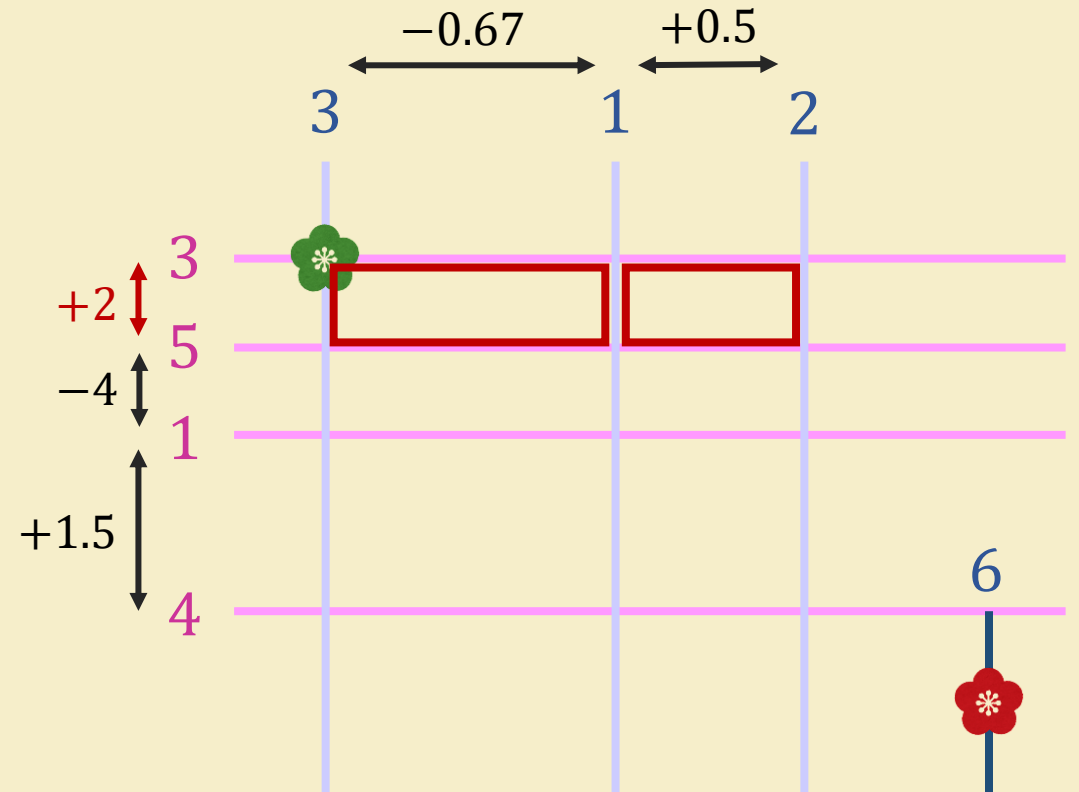
四角に対して条件が適用されるので
道路の最後の部分だけ残ることに注意



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

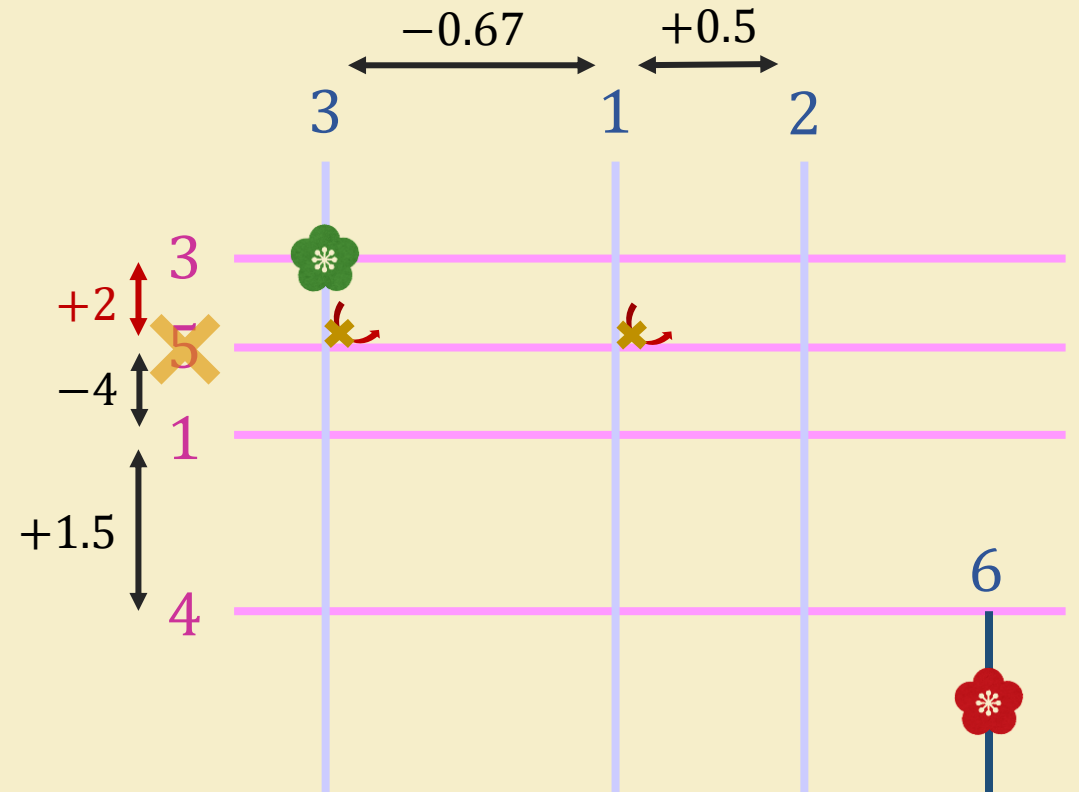
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

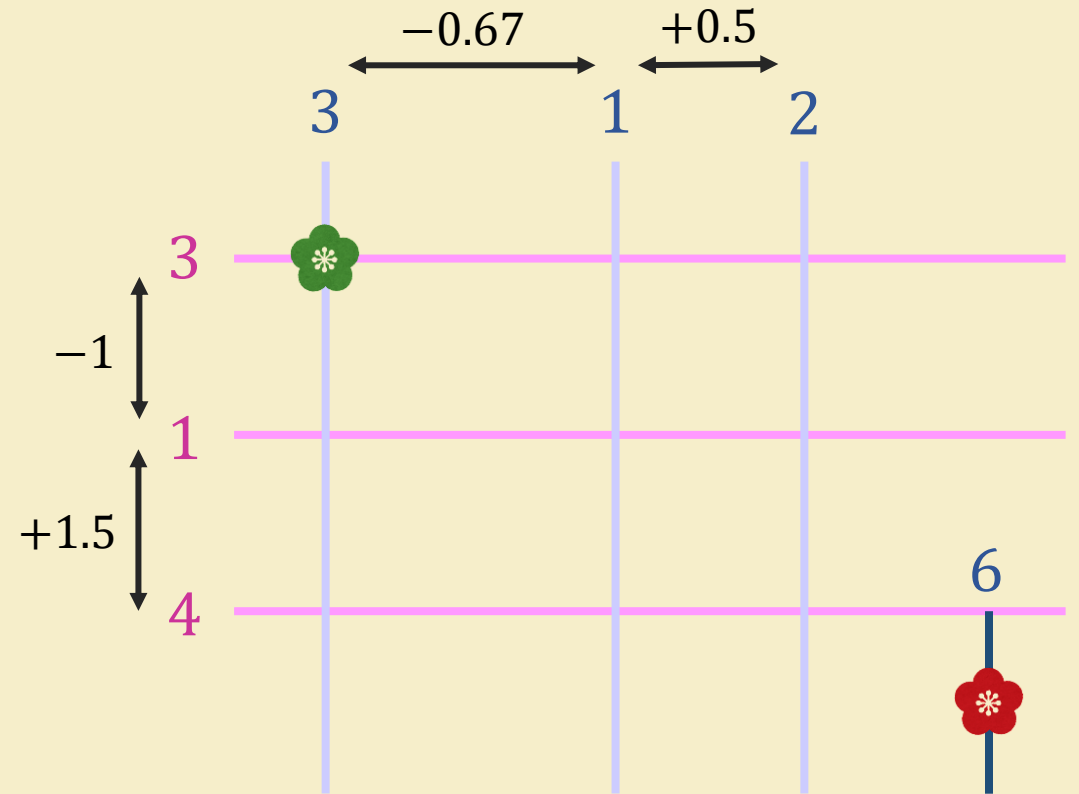
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

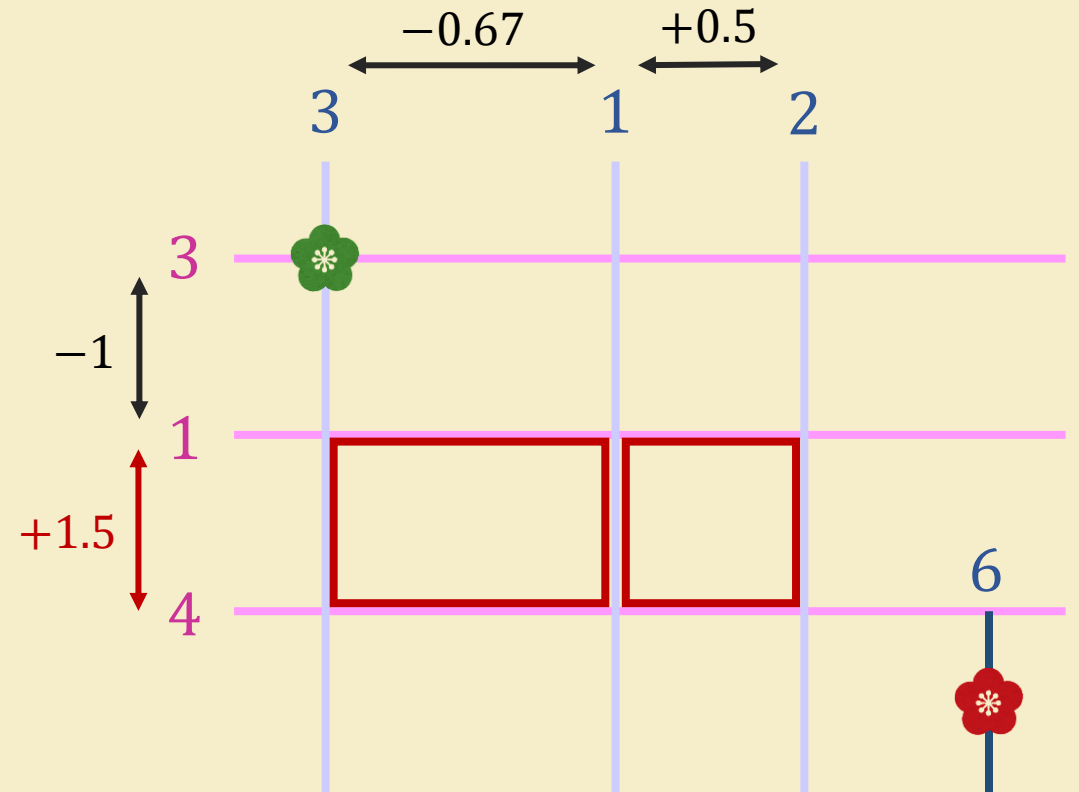
(右図の黒い数は「傾き」を表しています)



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

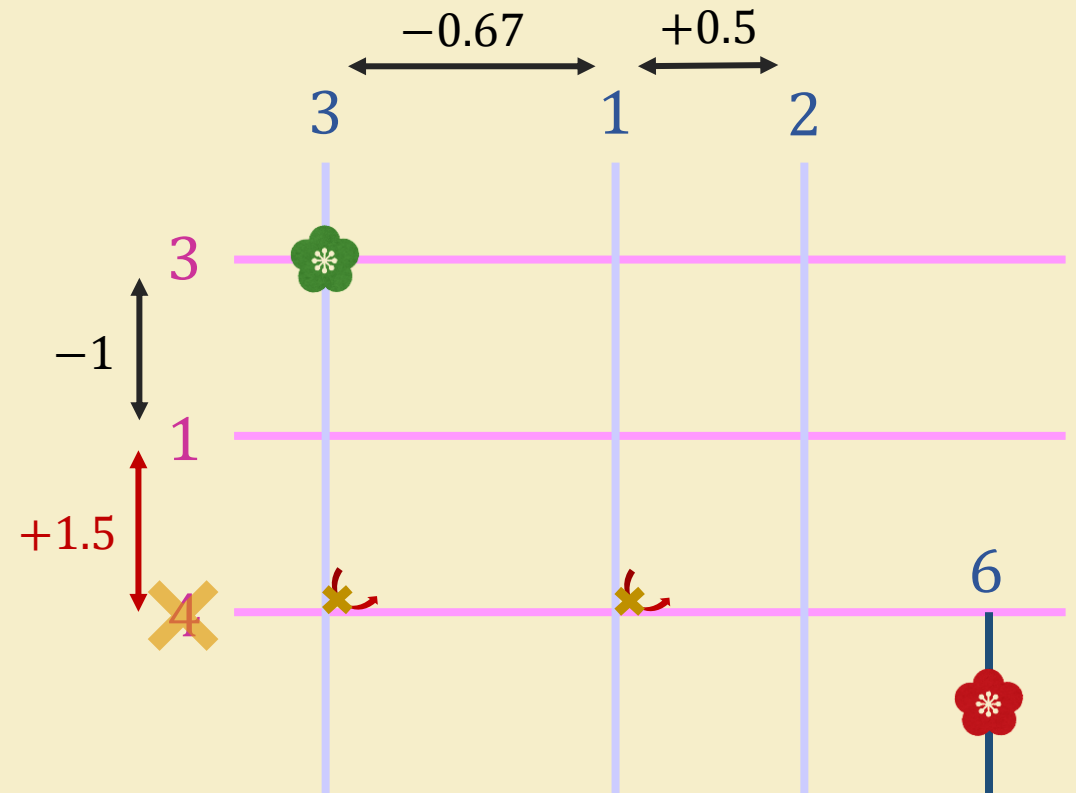


小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

四角に対して条件が適用されるので
道路の最後の部分だけ残ることに注意

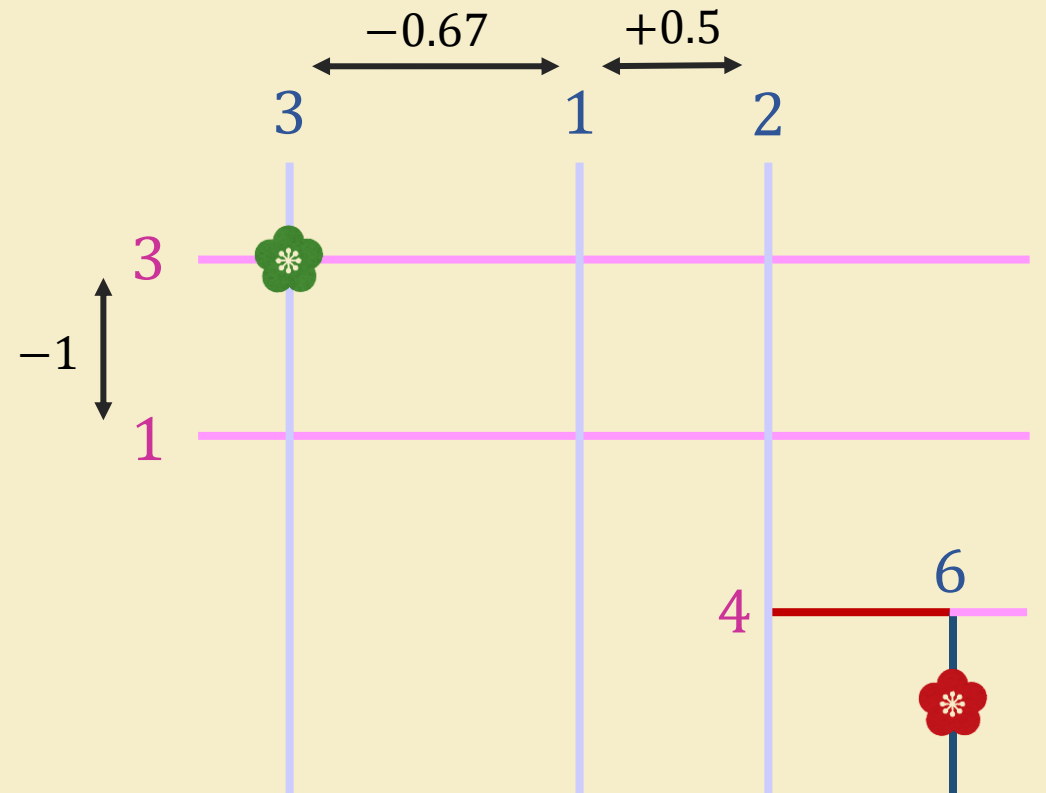


小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

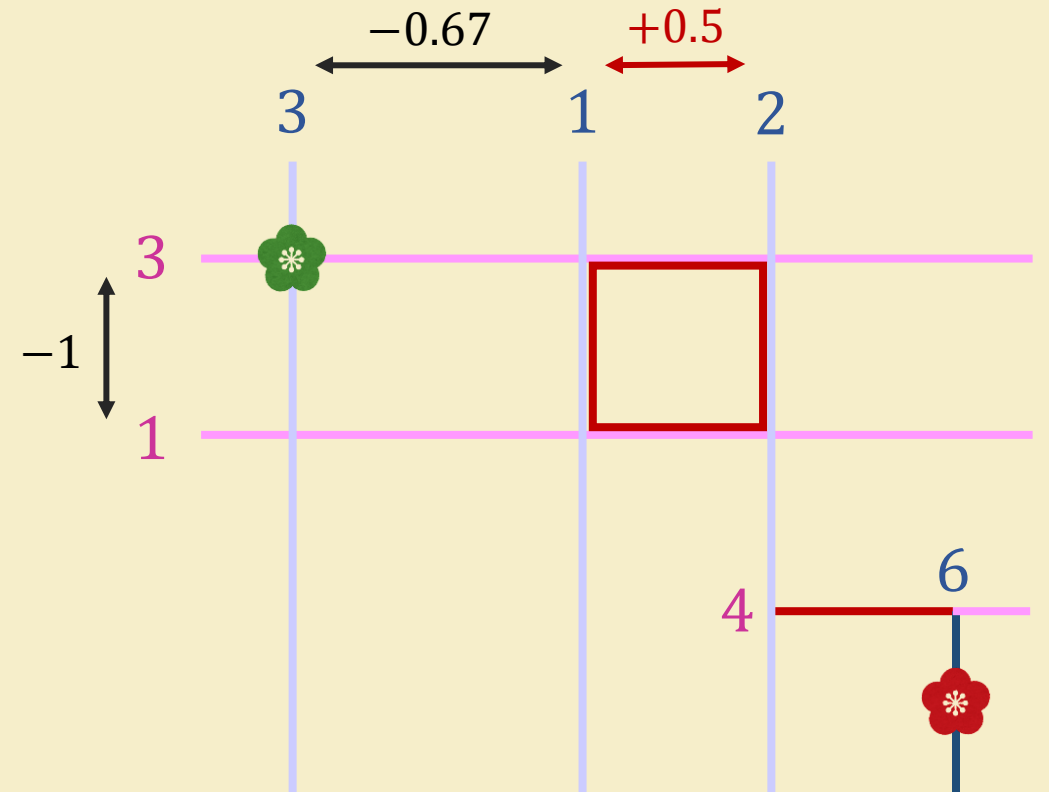
四角に対して条件が適用されるので
道路の最後の部分だけ残ることに注意



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

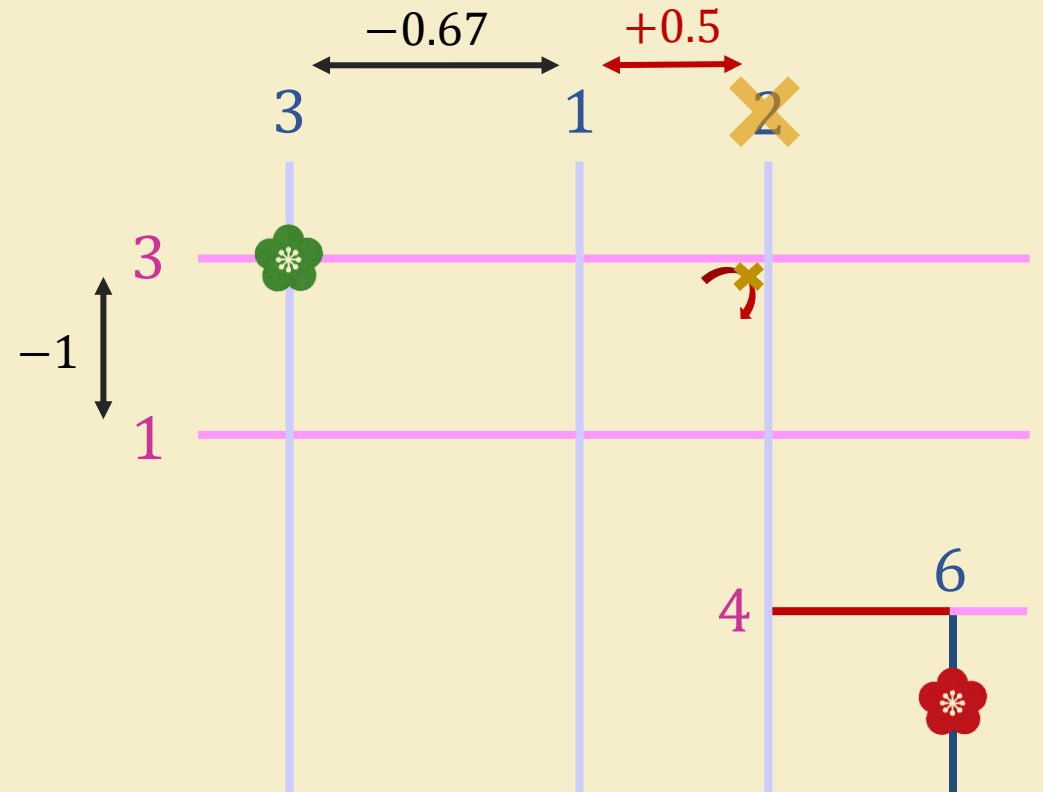


小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

四角に対して条件が適用されるので
道路の最後の部分だけ残ることに注意

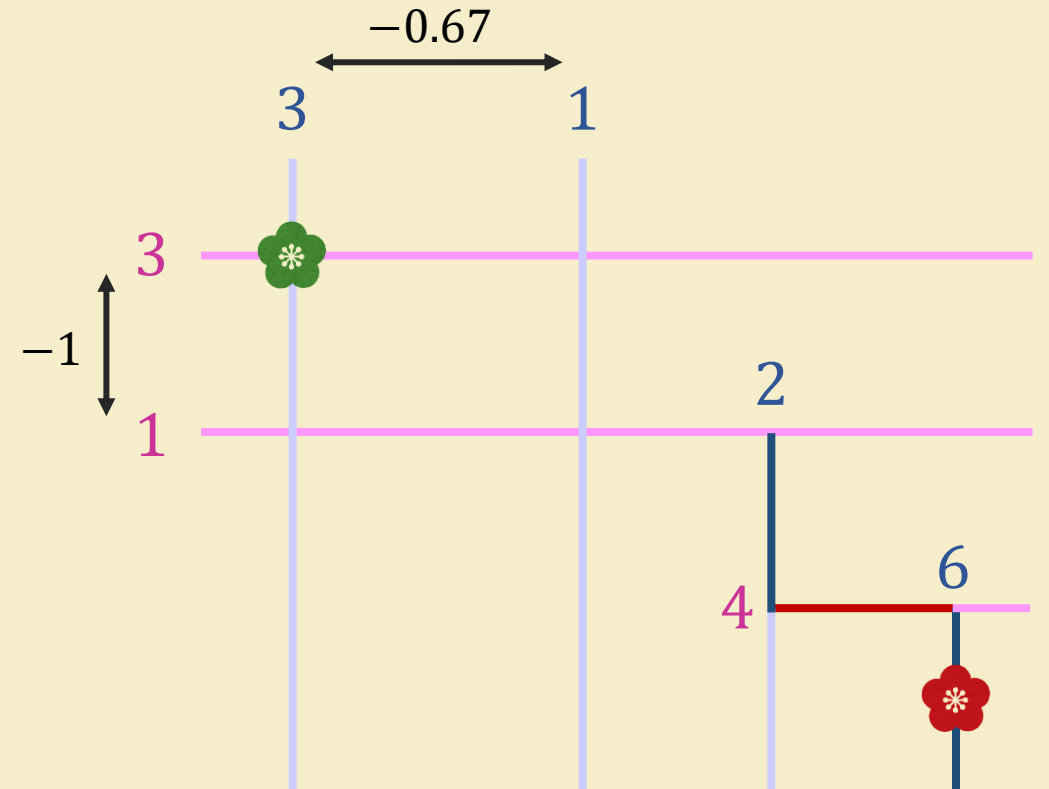


小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

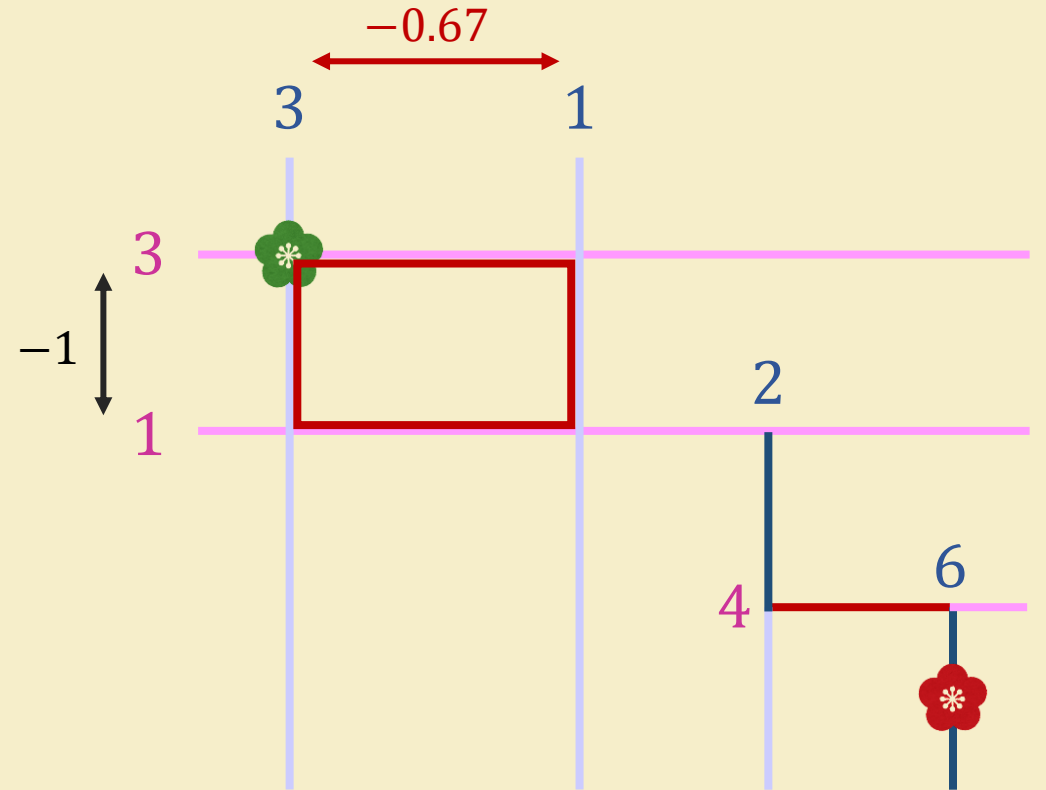
四角に対して条件が適用されるので
道路の最後の部分だけ残ることに注意



小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

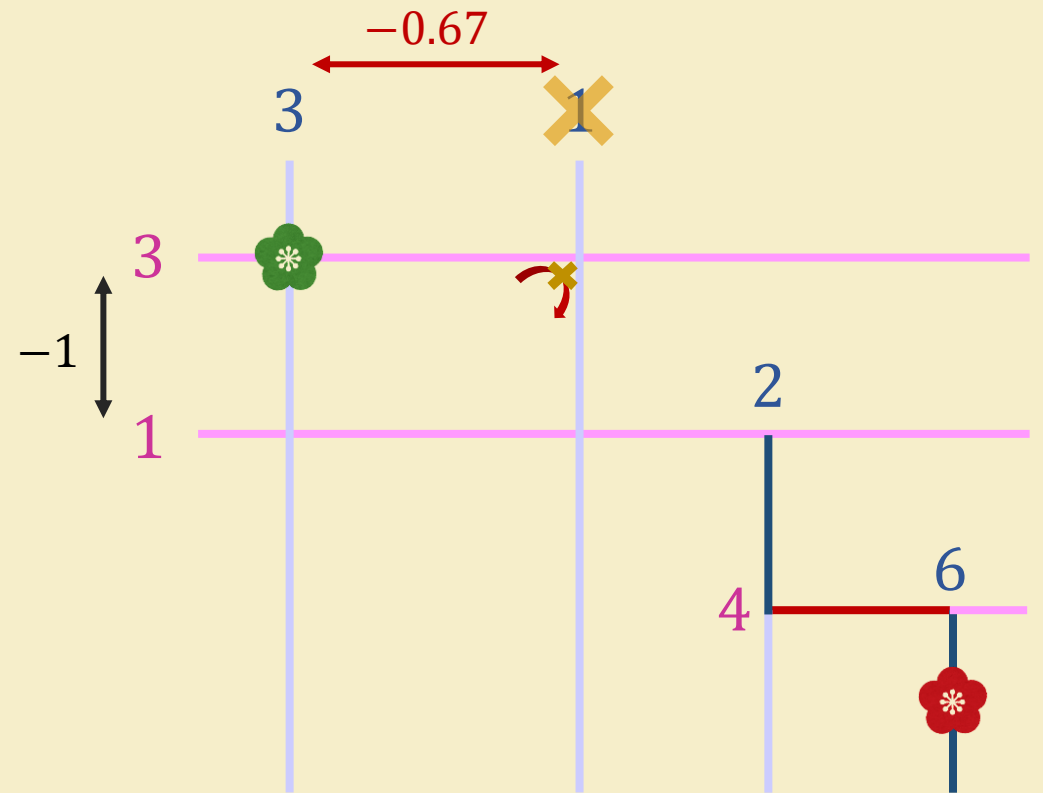


小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

四角に対して条件が適用されるので
道路の最後の部分だけ残ることに注意

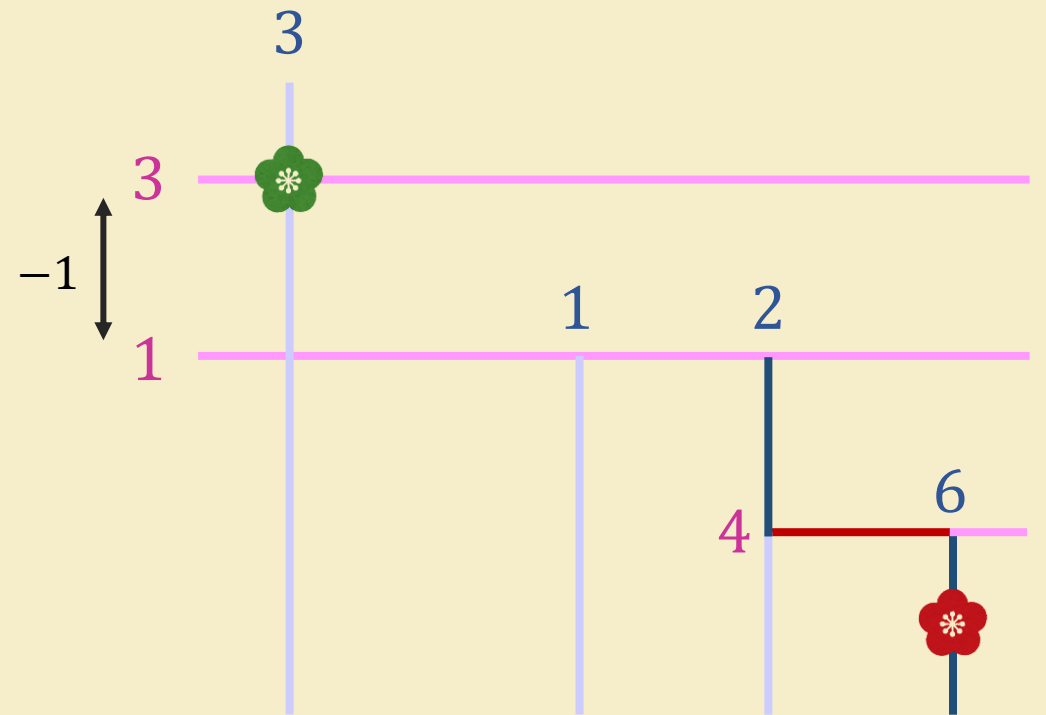


小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

四角に対して条件が適用されるので
道路の最後の部分だけ残ることに注意

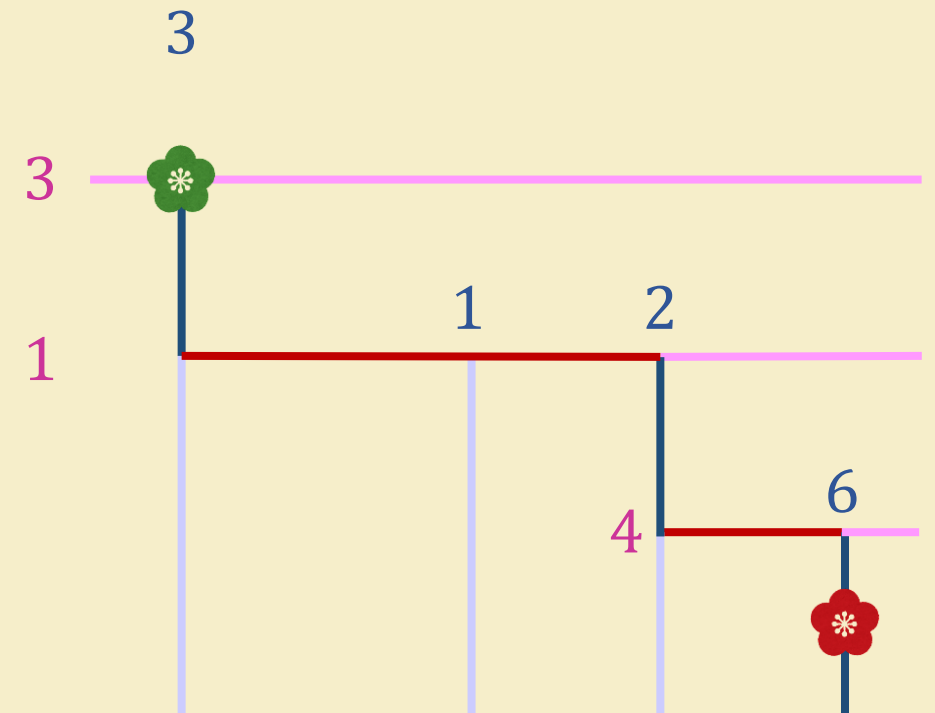


小課題 3 — $H, W \leq 100000$

最初からやってみよう!

(右図の黒い数は「傾き」を表しています)

→ これで経路が決まった!!!



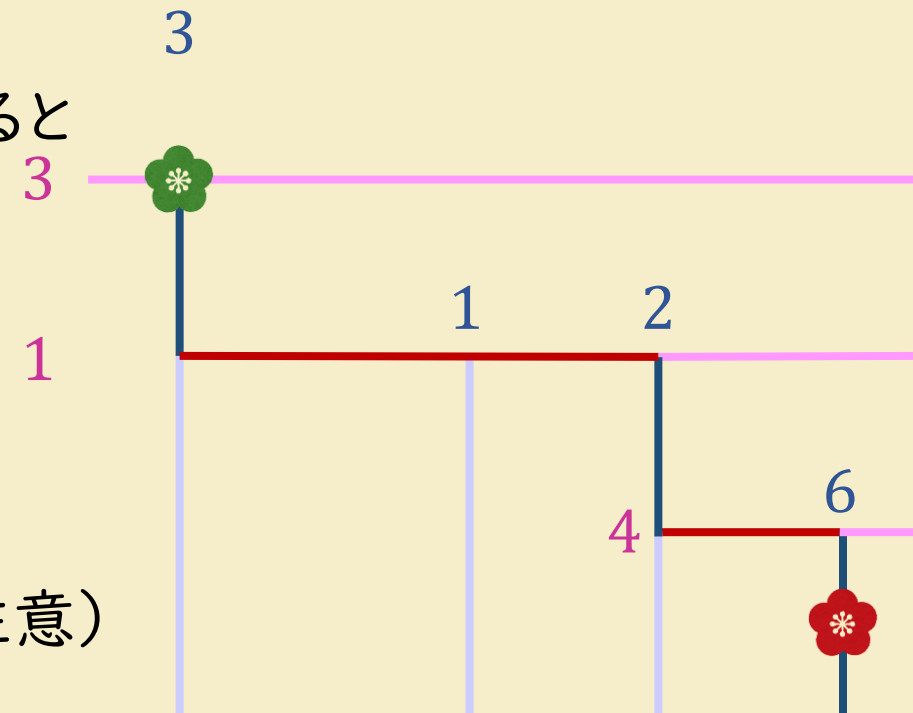
小課題 3 — $H, W \leq 100000$

解法のまとめ

このように、傾きが最小の行／列に条件を適用し続けると最終的には必ず 1 つの経路ができる

このシミュレーションは、`std::set` などを用いて全体計算量 $O((H + W) \log(H + W))$ でできる

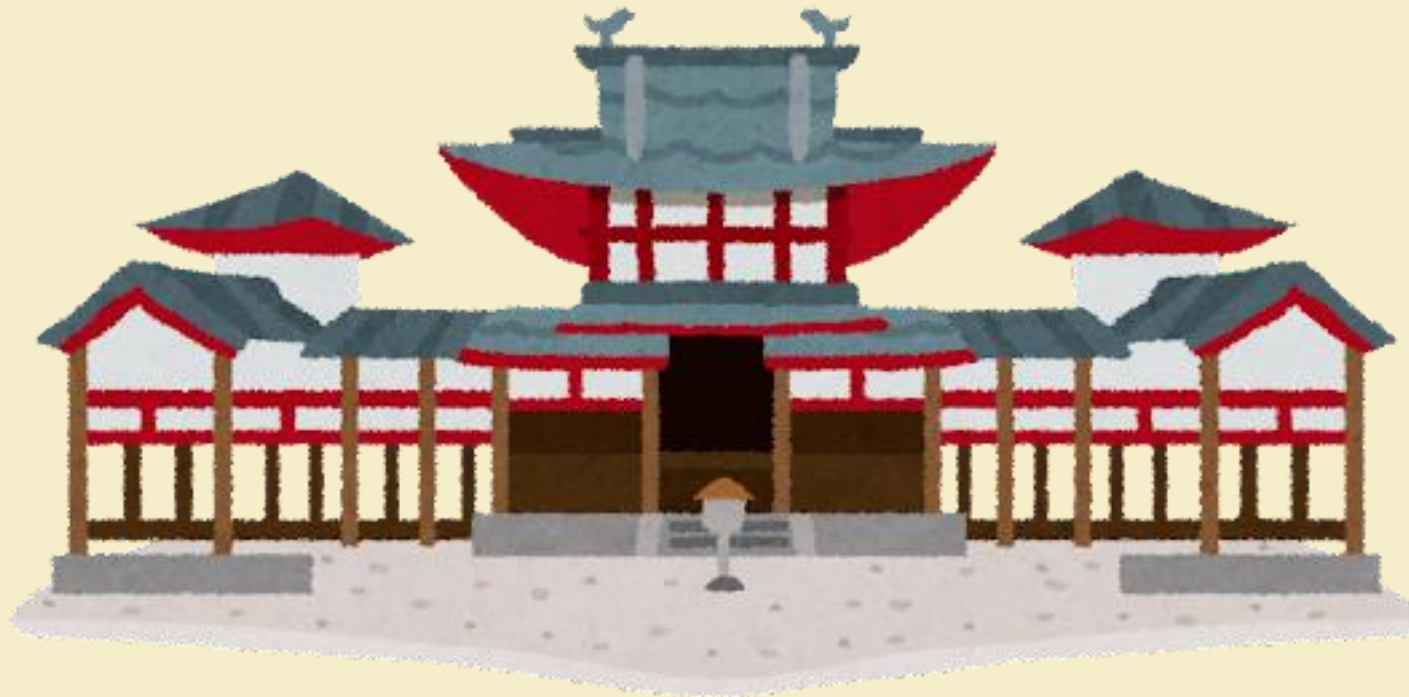
(一番右・下の値が消されたときに経路が残ることに注意)



小課題 3 — $H, W \leq 100000$

解法のまとめ

よって、この問題は $O((H + W) \log(H + W))$ で解けた！



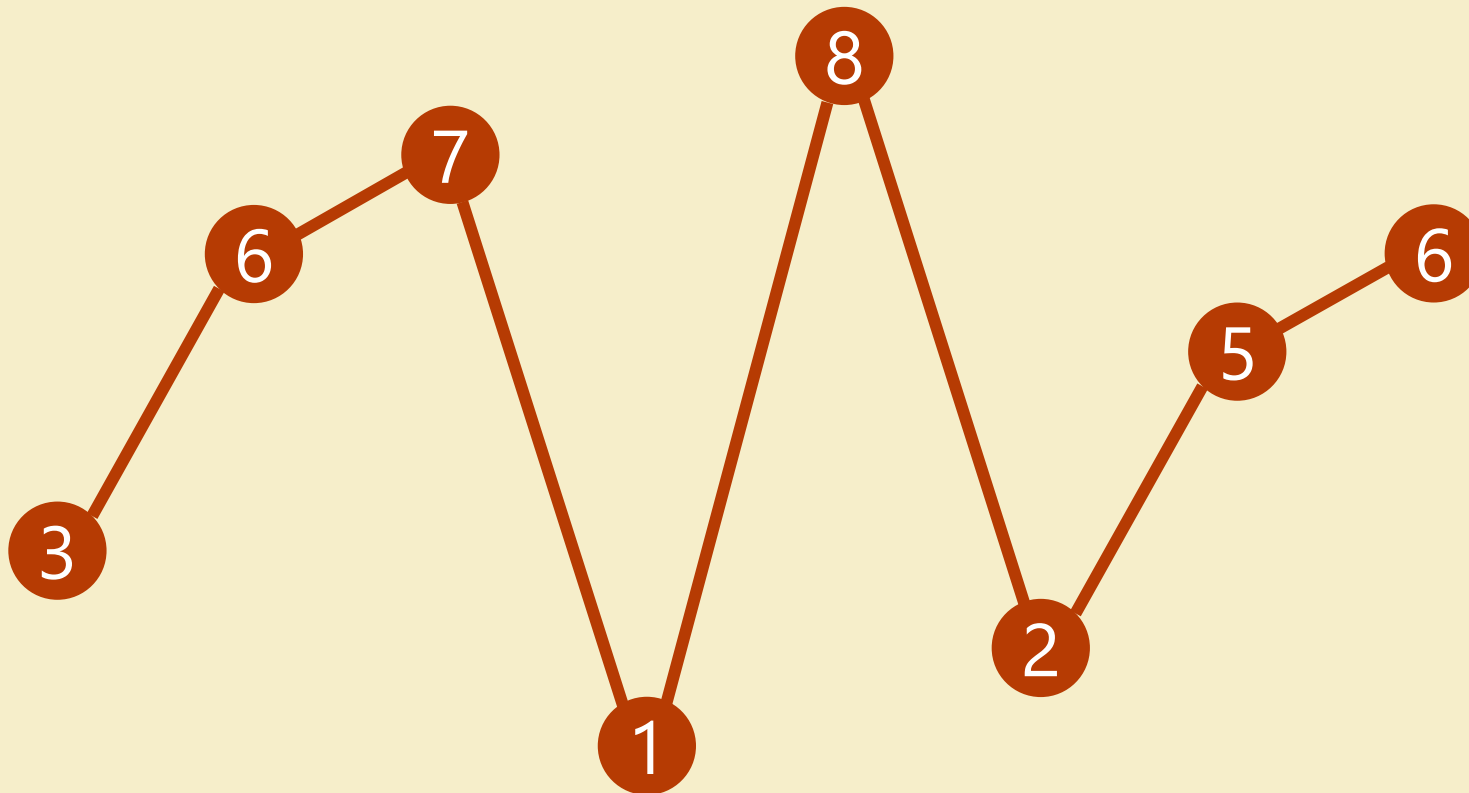
別解

計算量 $O(H + W)$ 解法



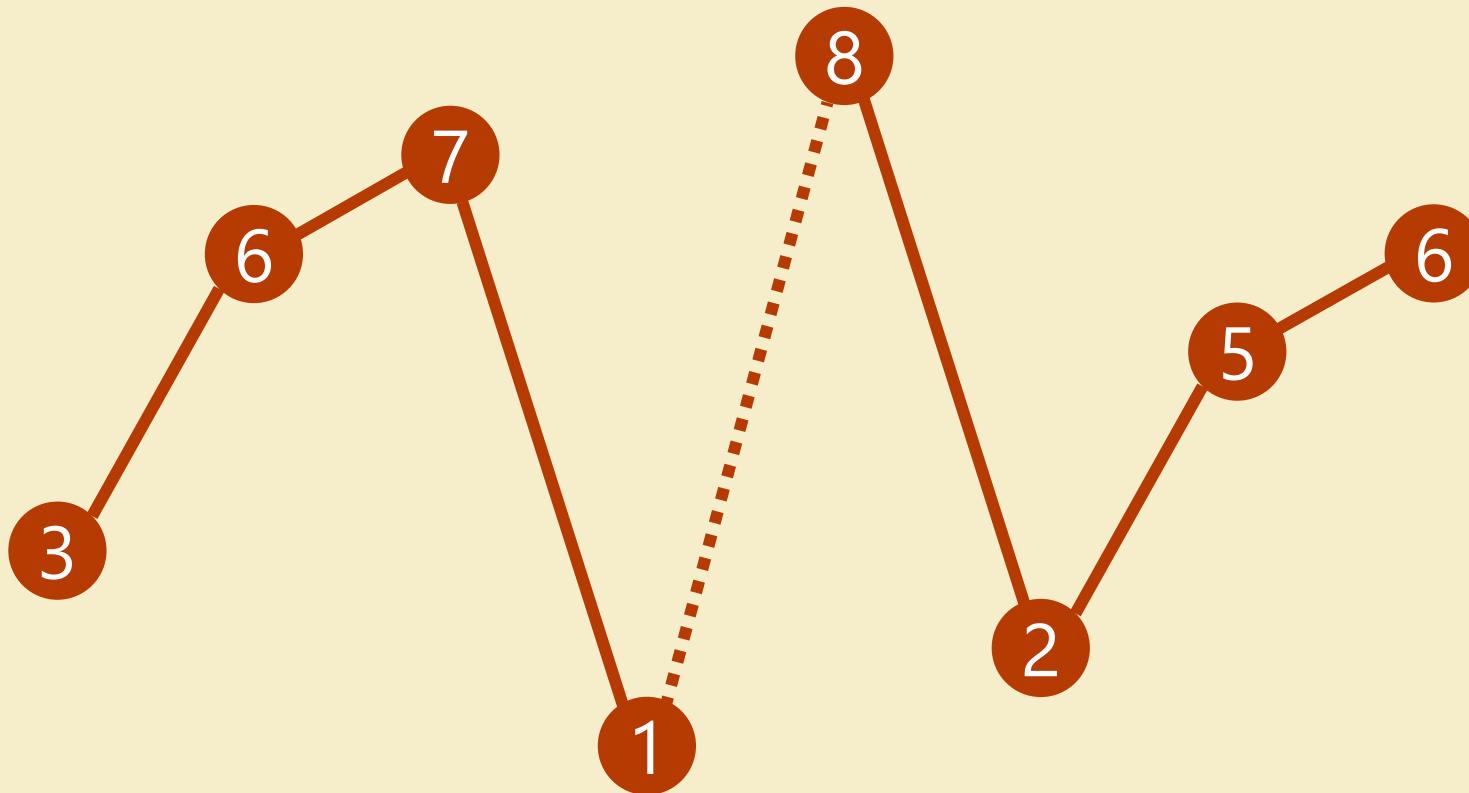
別解

「傾きが最大を取り除く」というのは、ということ？



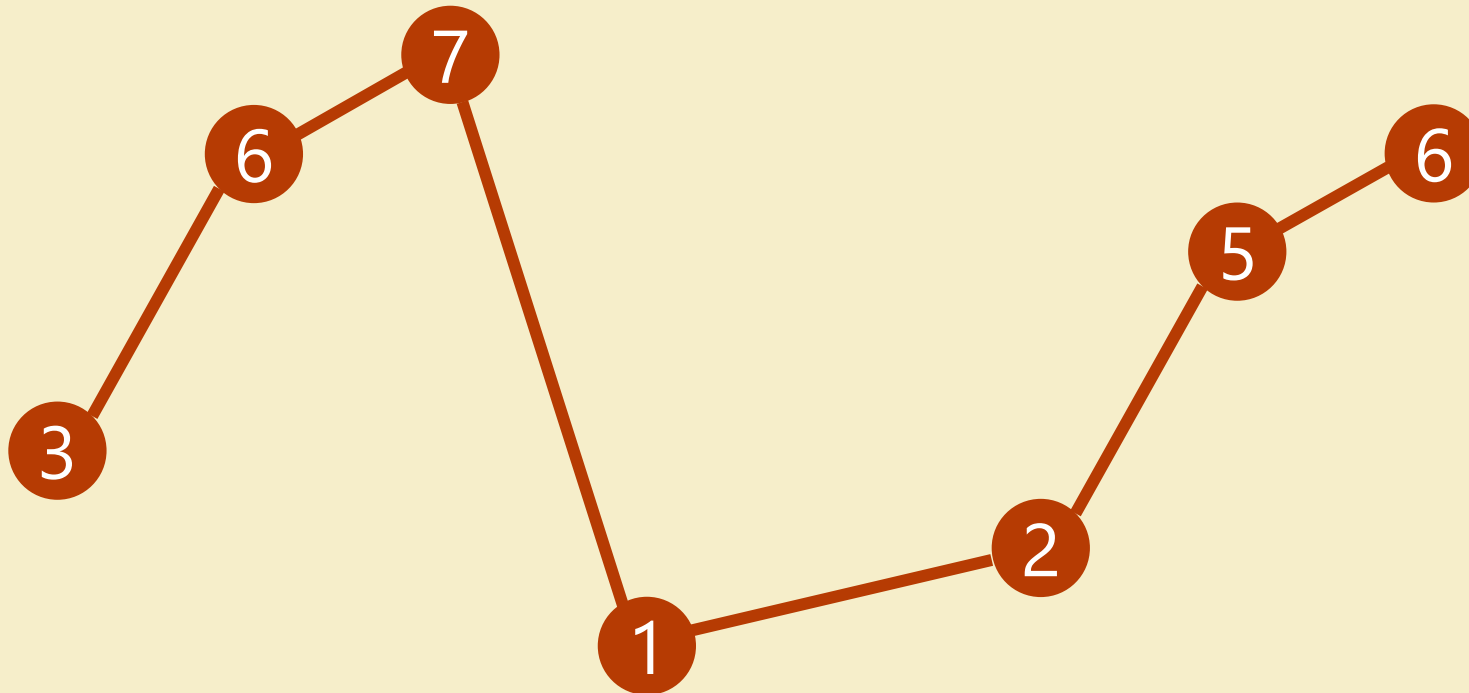
別解

「傾きが最大を取り除く」というのは、どういうこと？



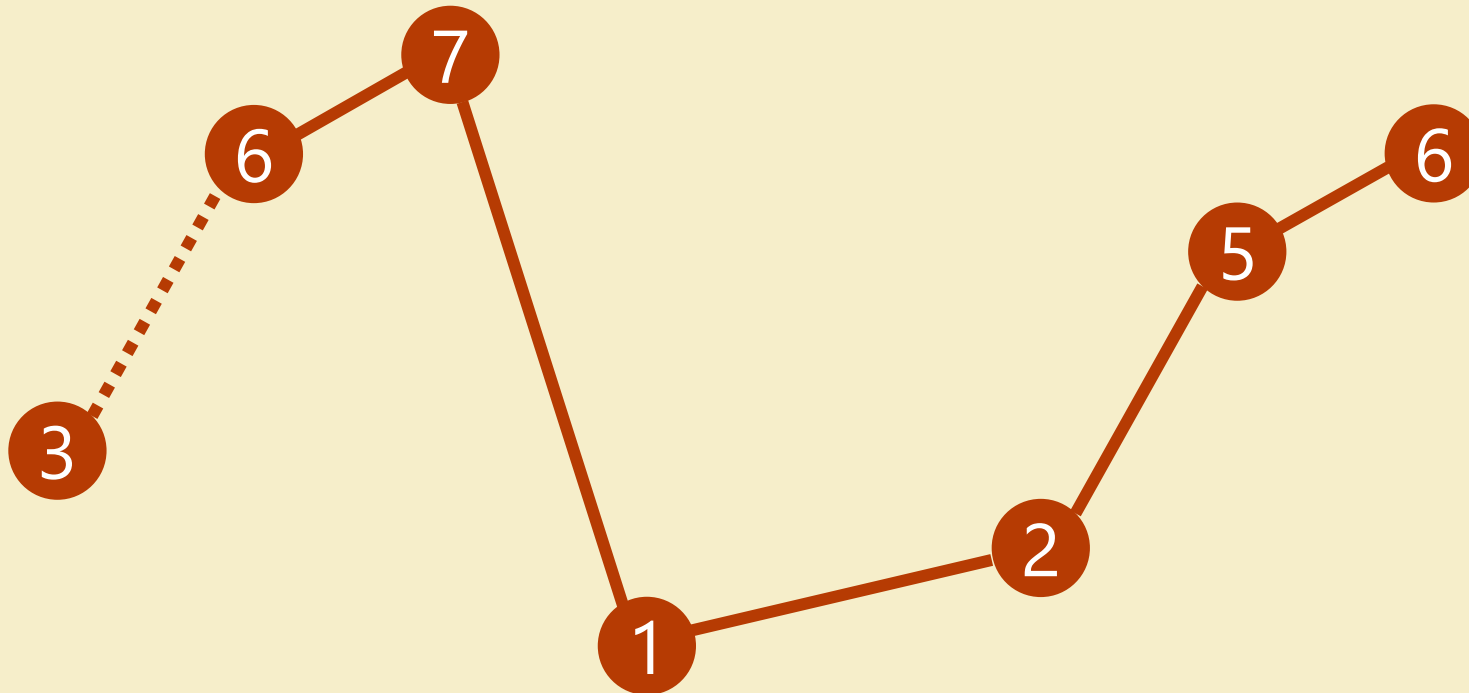
別解

「傾きが最大を取り除く」というのは、どういうこと？



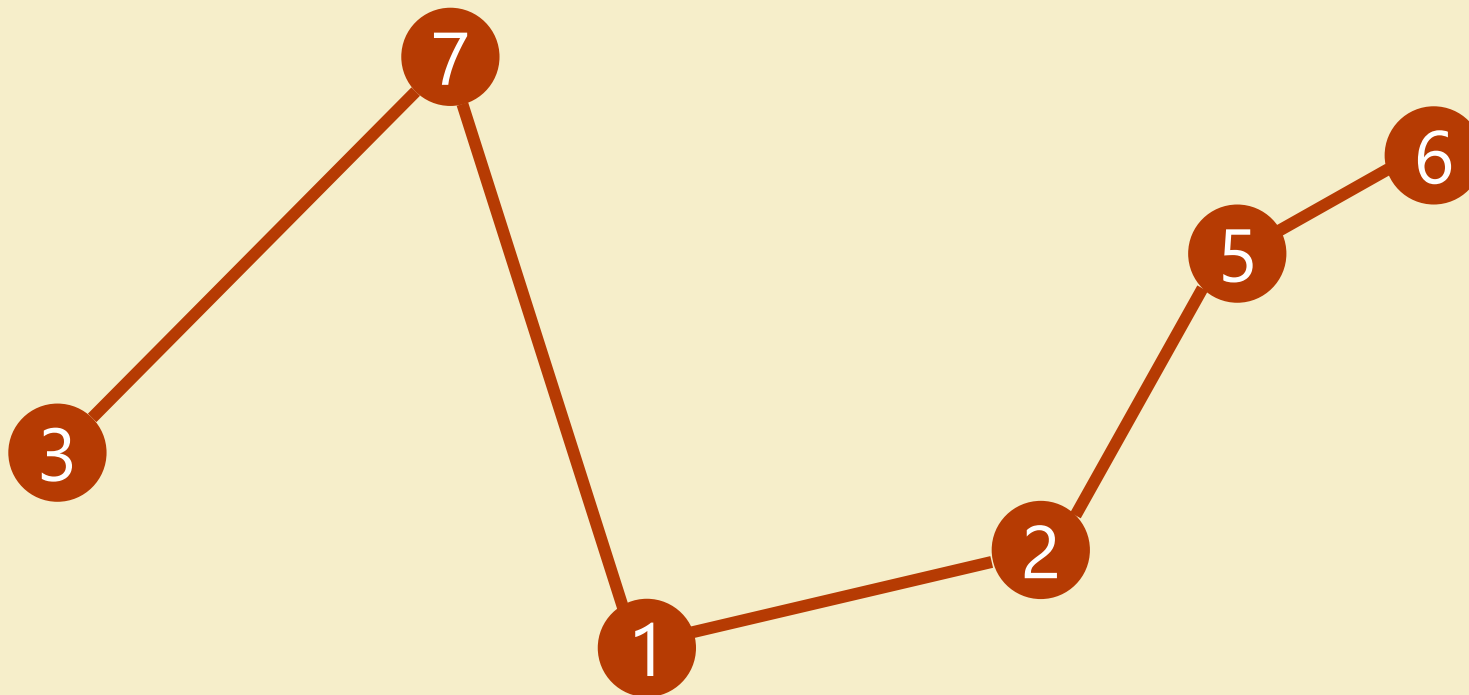
別解

「傾きが最大を取り除く」というのは、ということ？



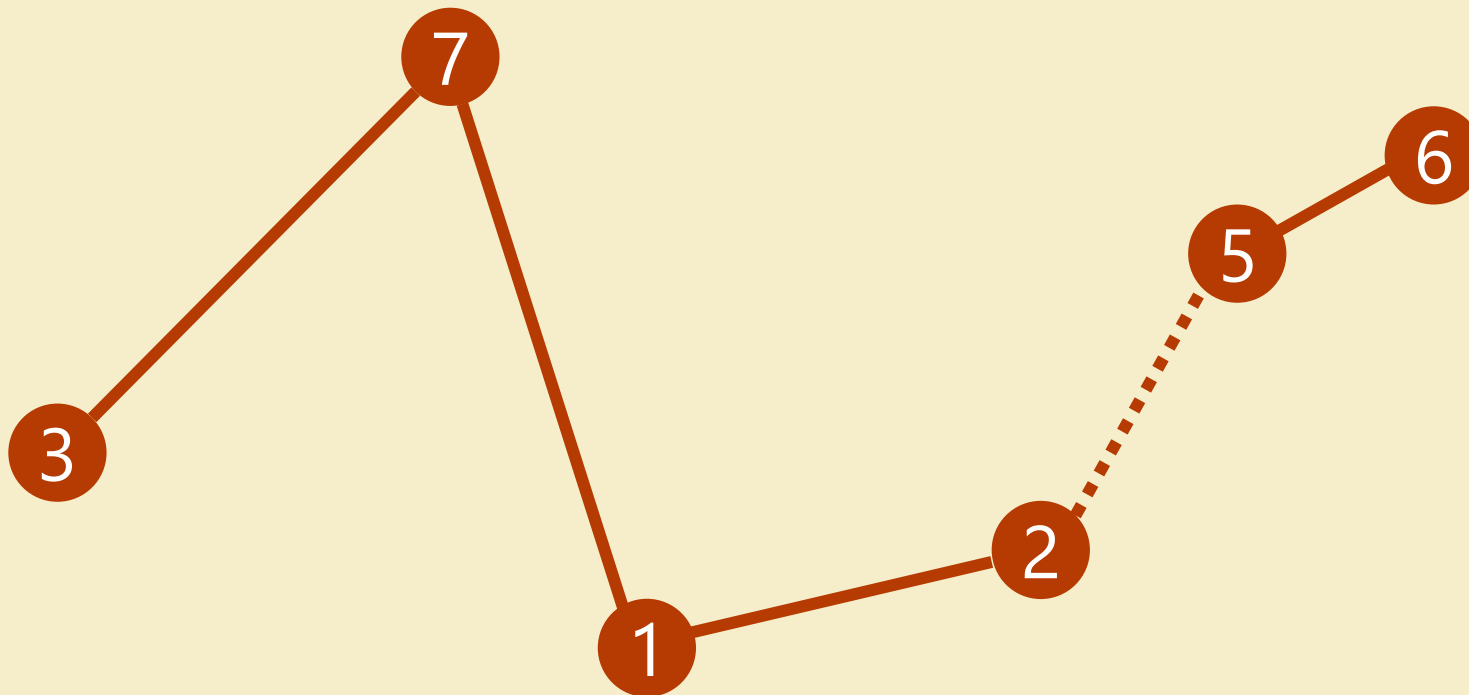
別解

「傾きが最大を取り除く」というのは、ということ？



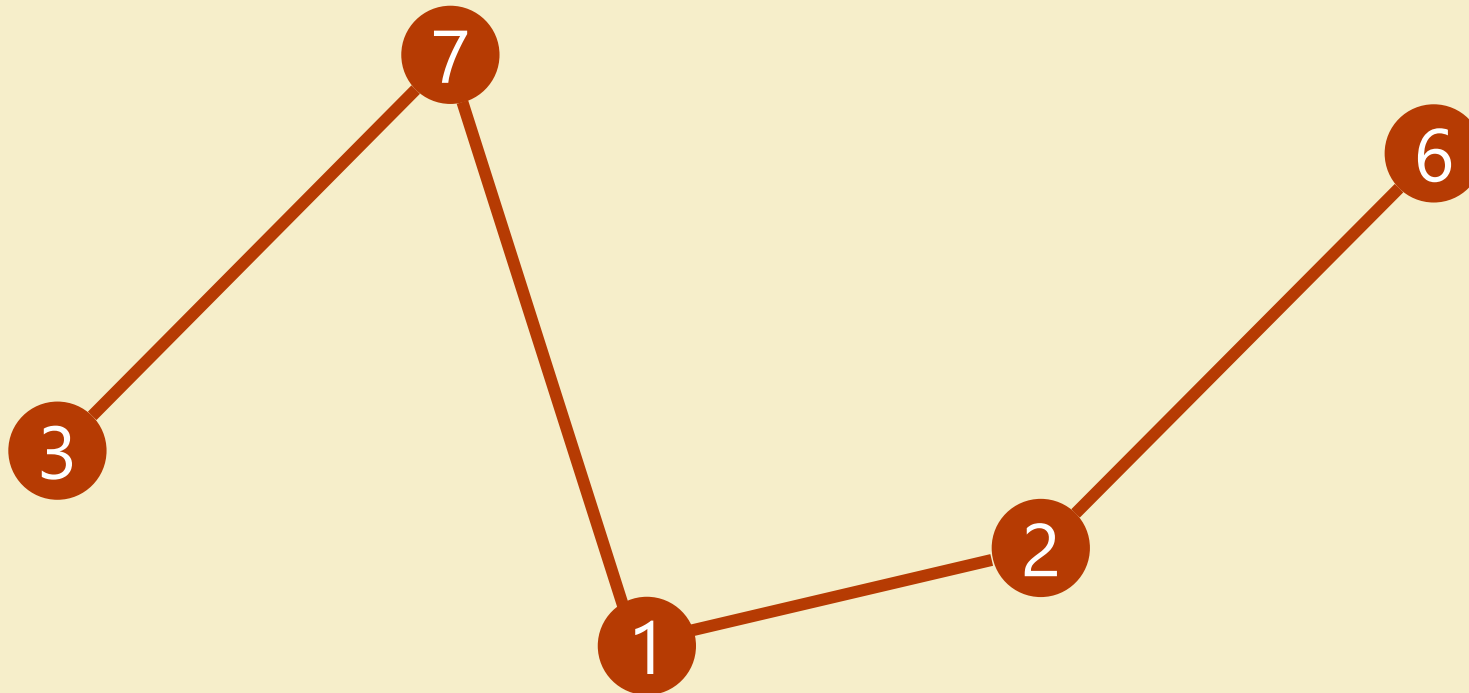
別解

「傾きが最大を取り除く」というのは、ということ？



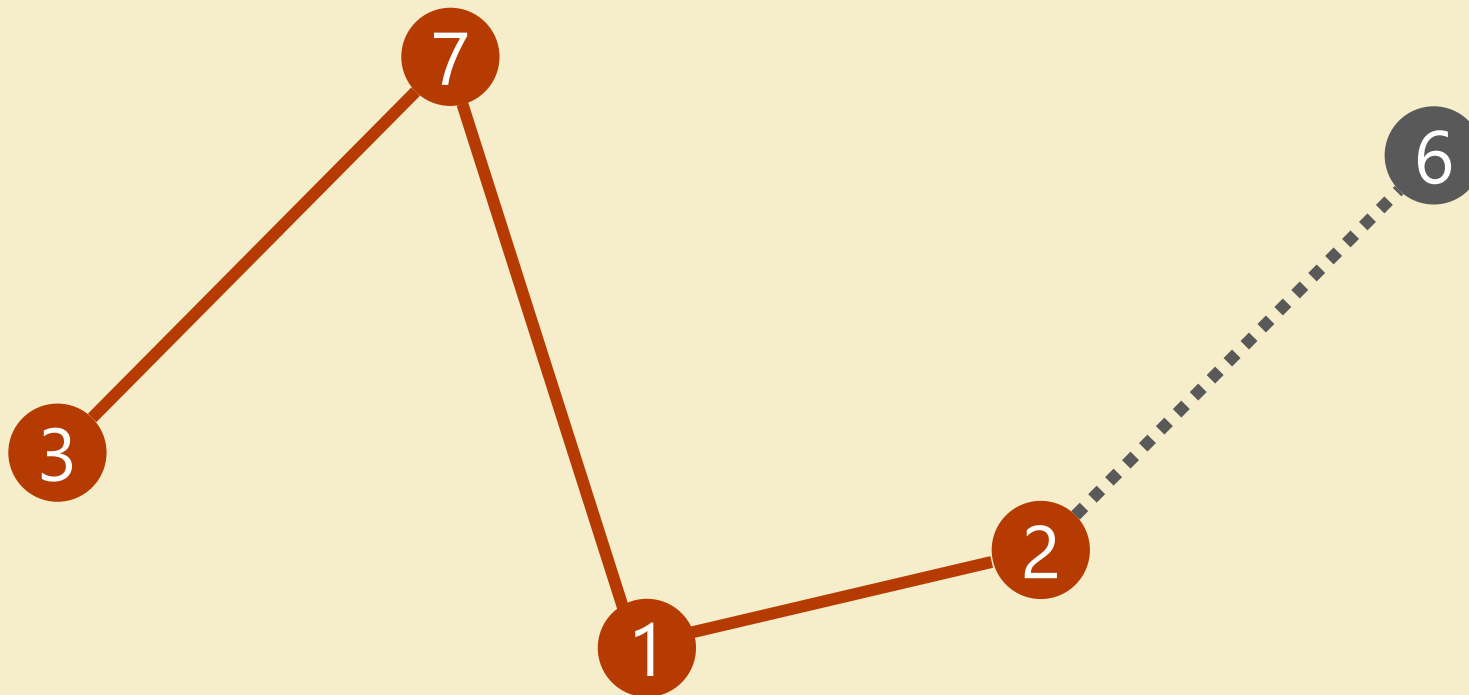
別解

「傾きが最大を取り除く」というのは、どういうこと？



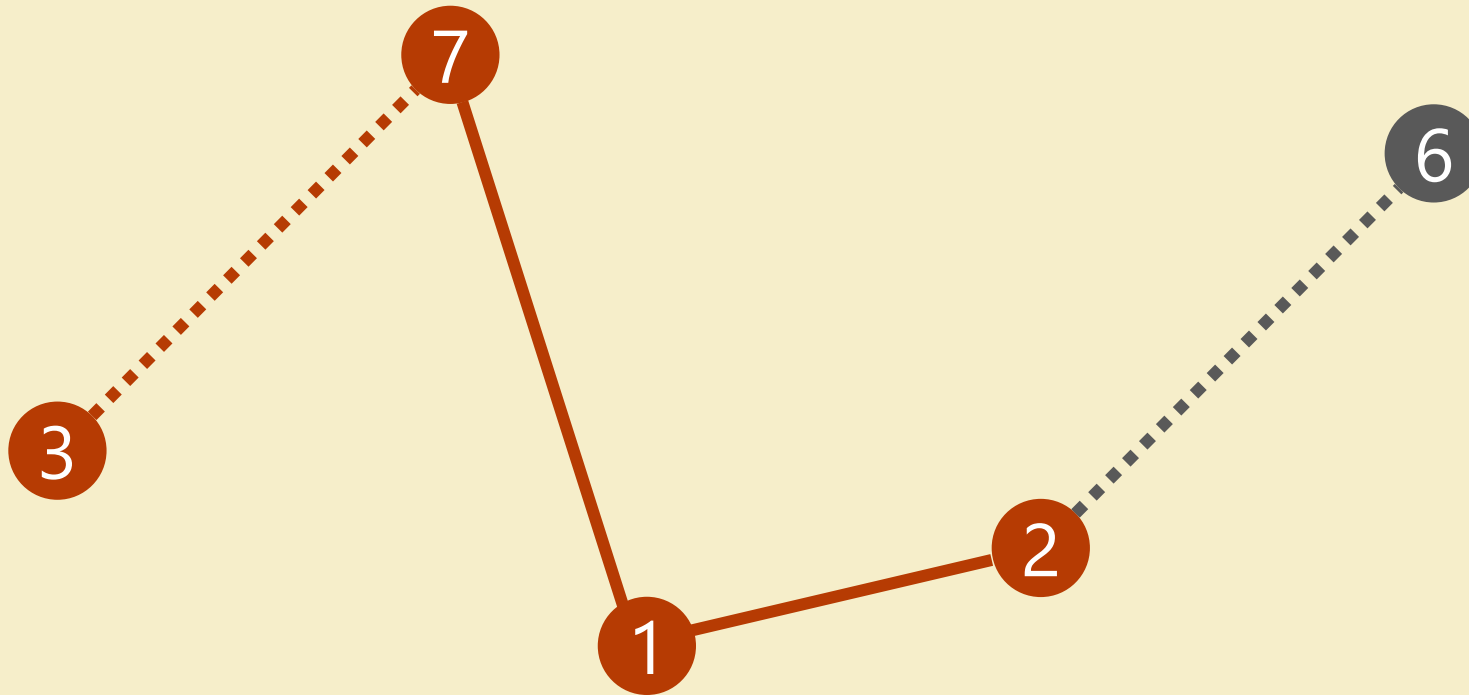
別解

「傾きが最大を取り除く」というのは、ということ？



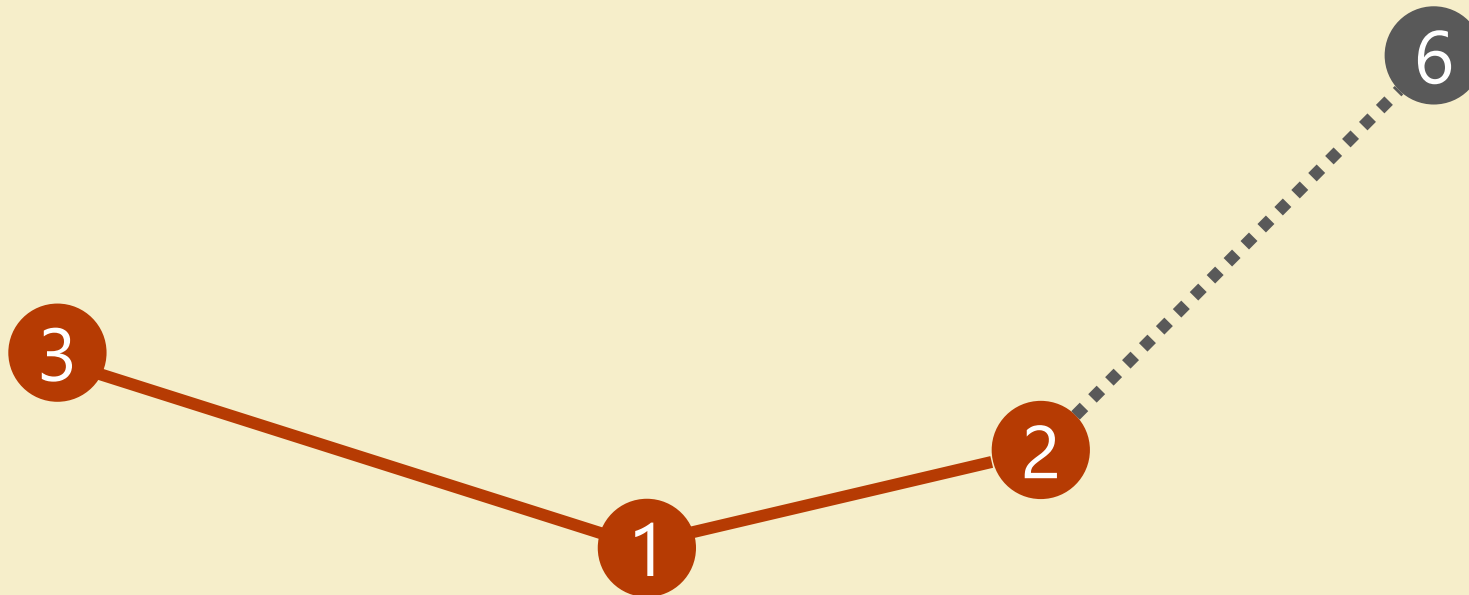
別解

「傾きが最大を取り除く」というのは、ということ？



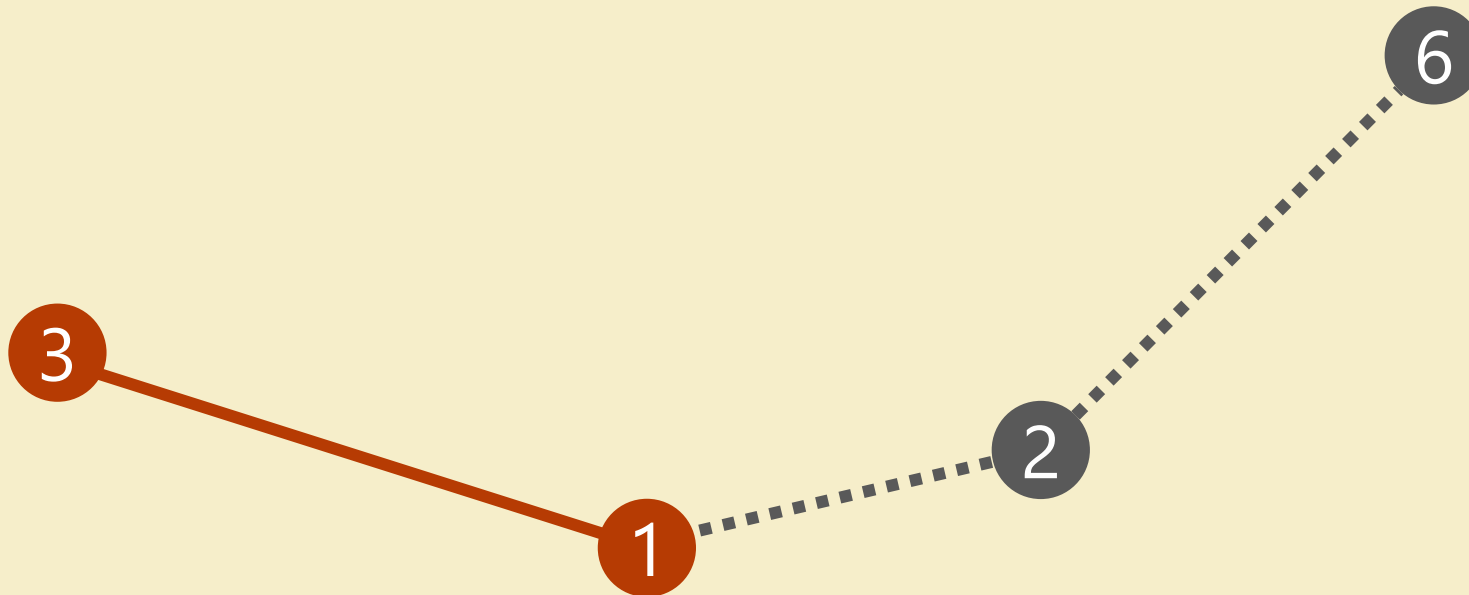
別解

「傾きが最大を取り除く」というのは、ということ？



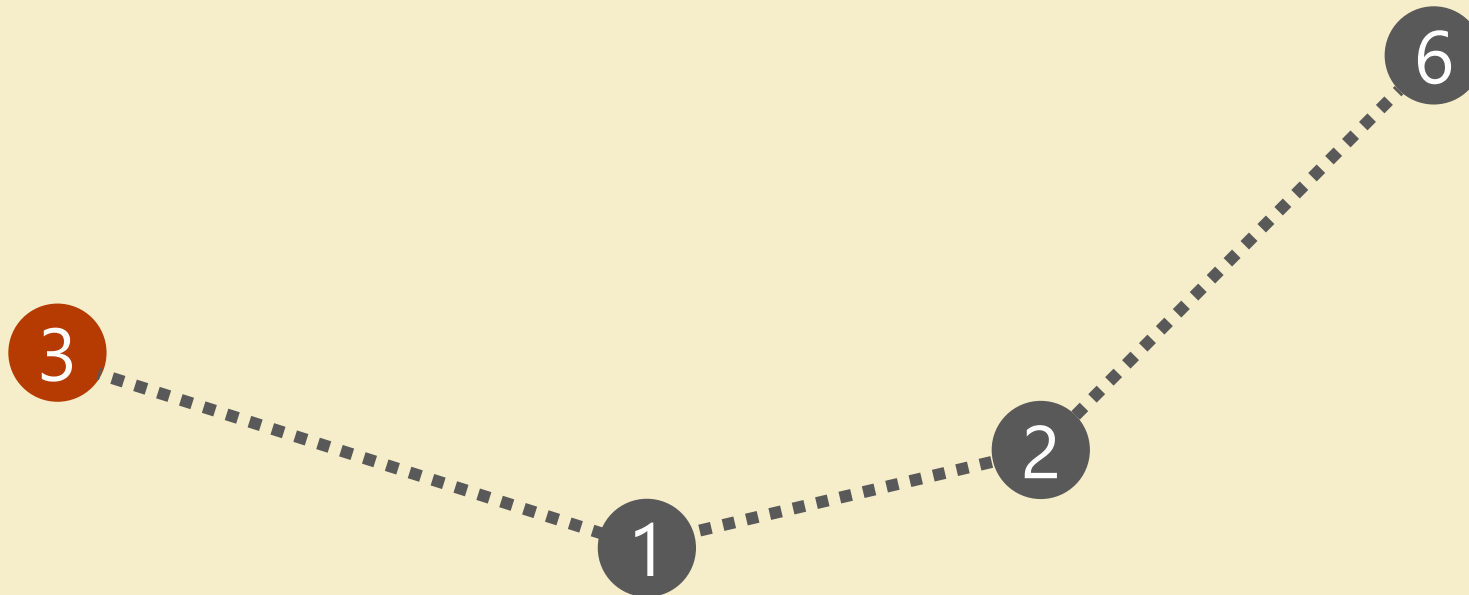
別解

「傾きが最大を取り除く」というのは、ということ？



別解

「傾きが最大を取り除く」というのは、ということ？

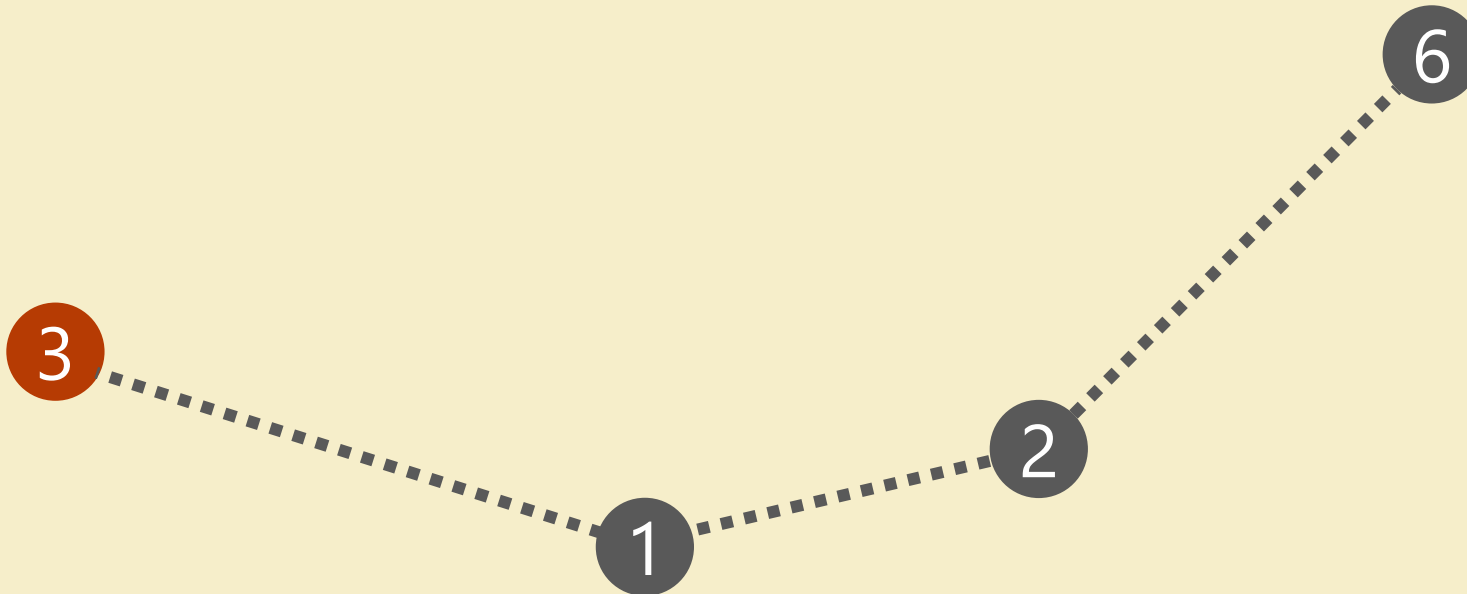


別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

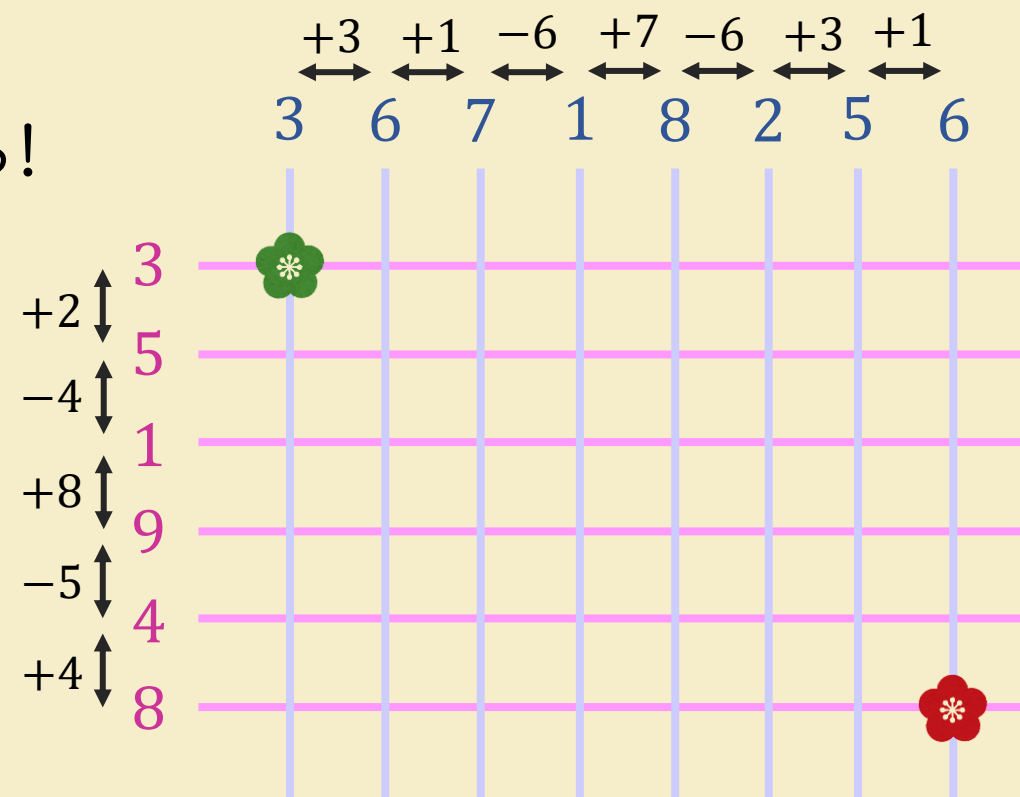


別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！



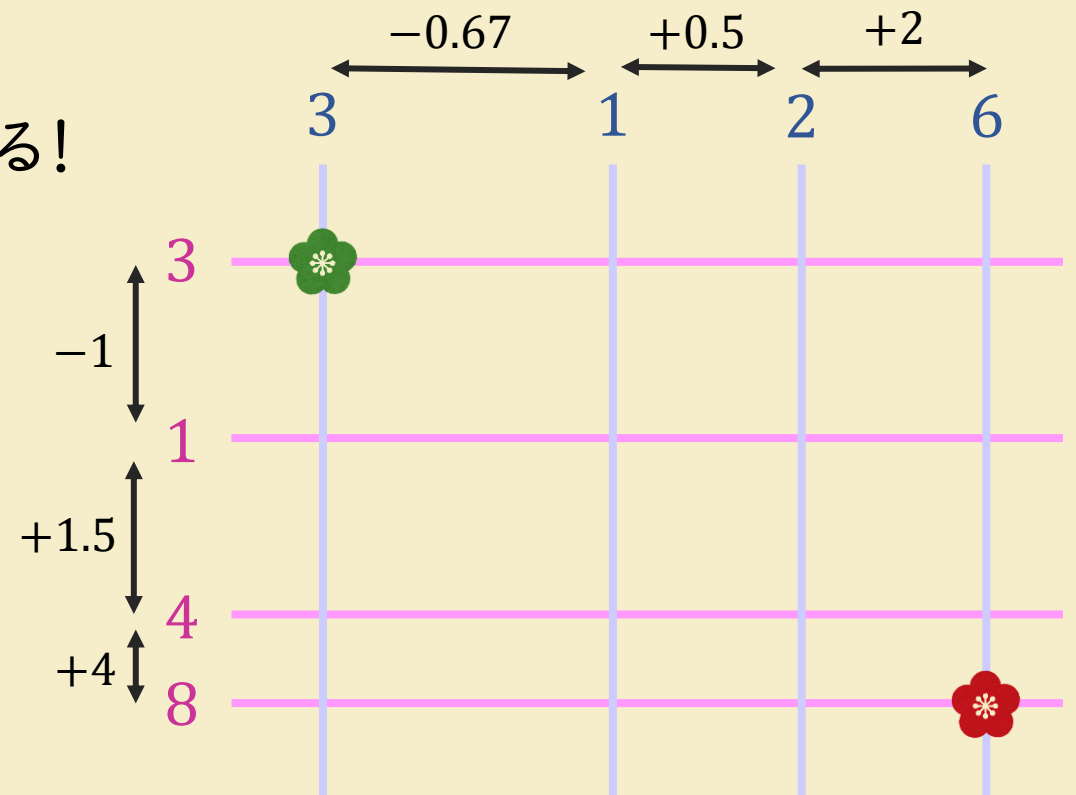
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



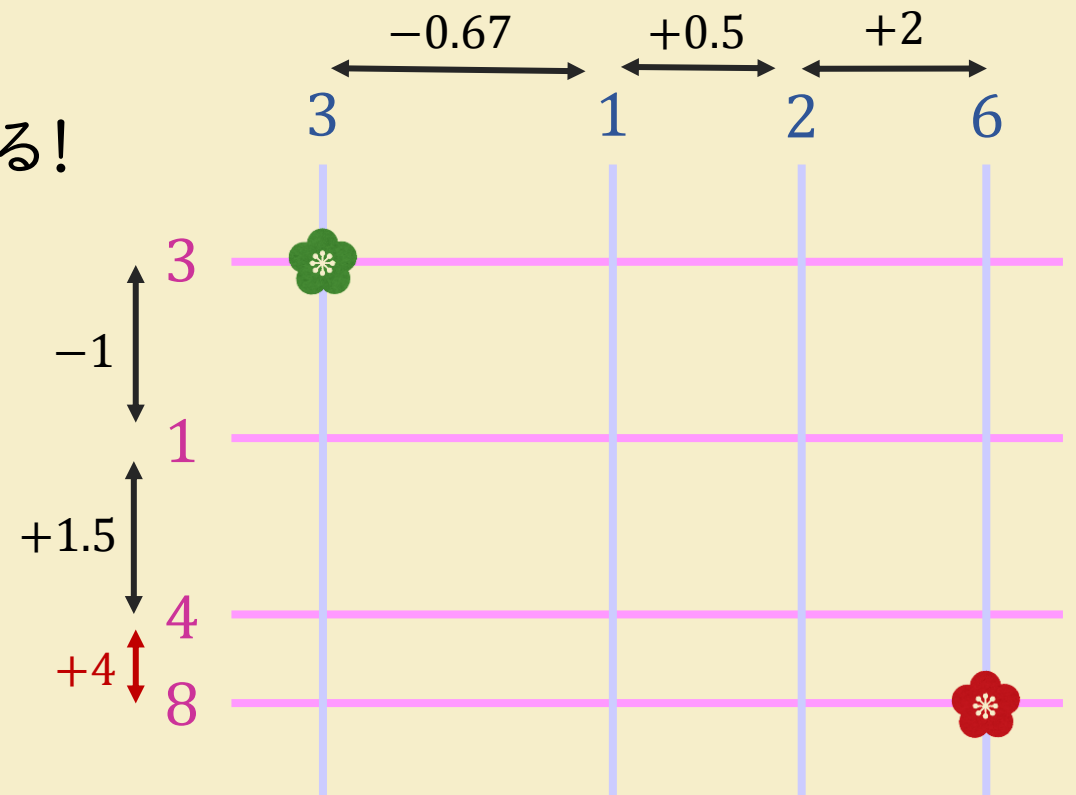
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



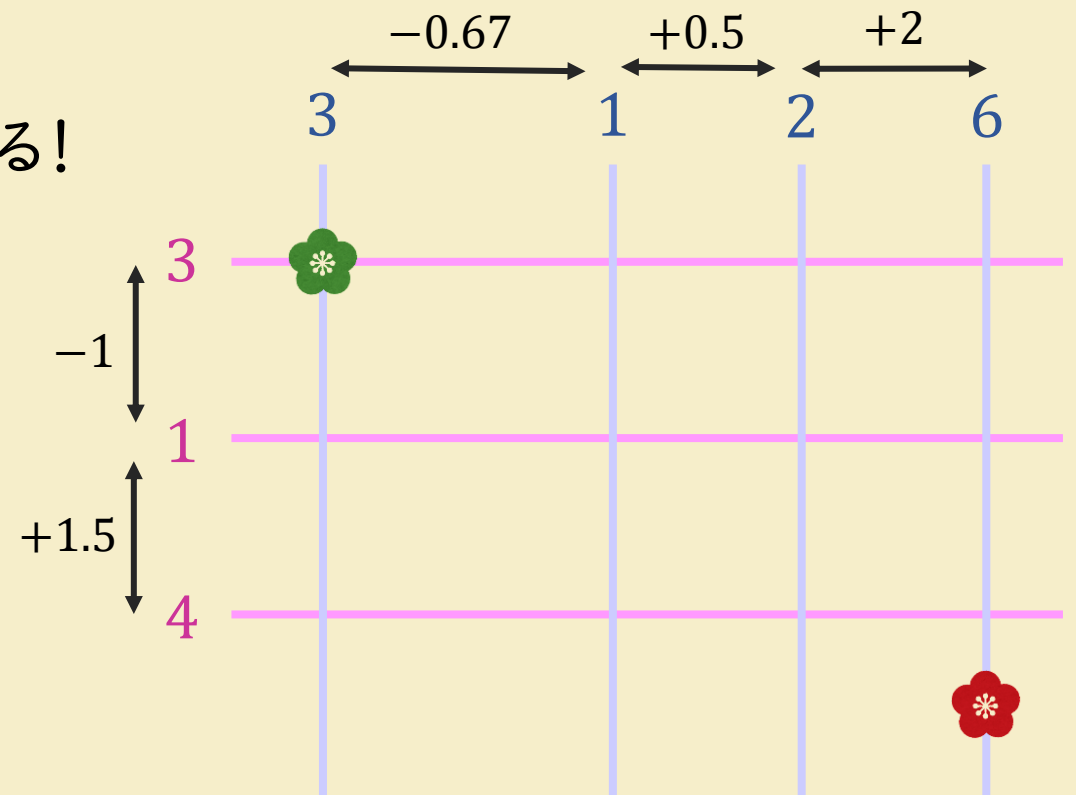
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



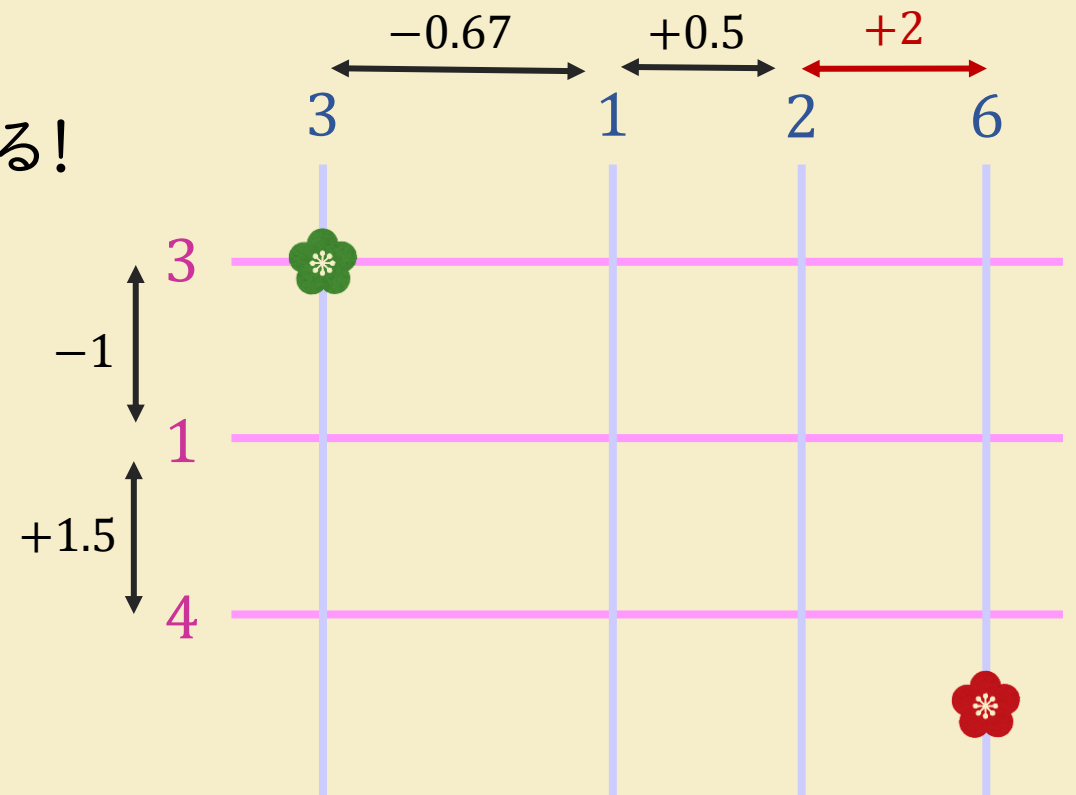
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



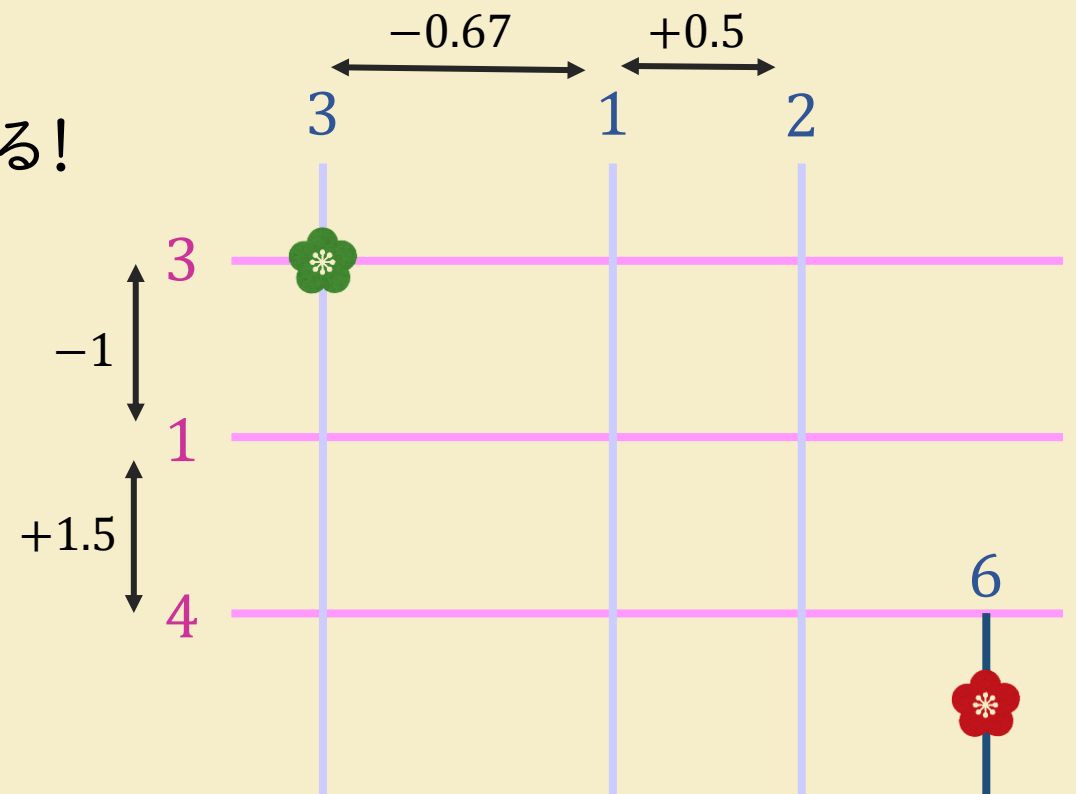
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



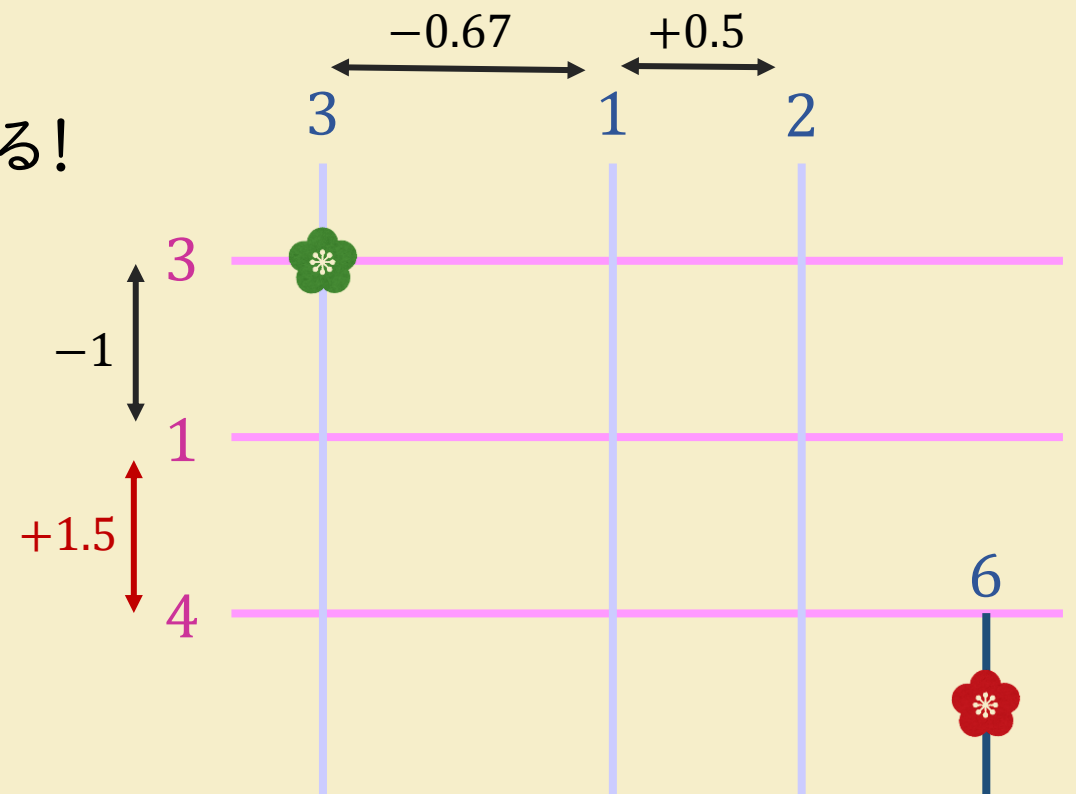
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



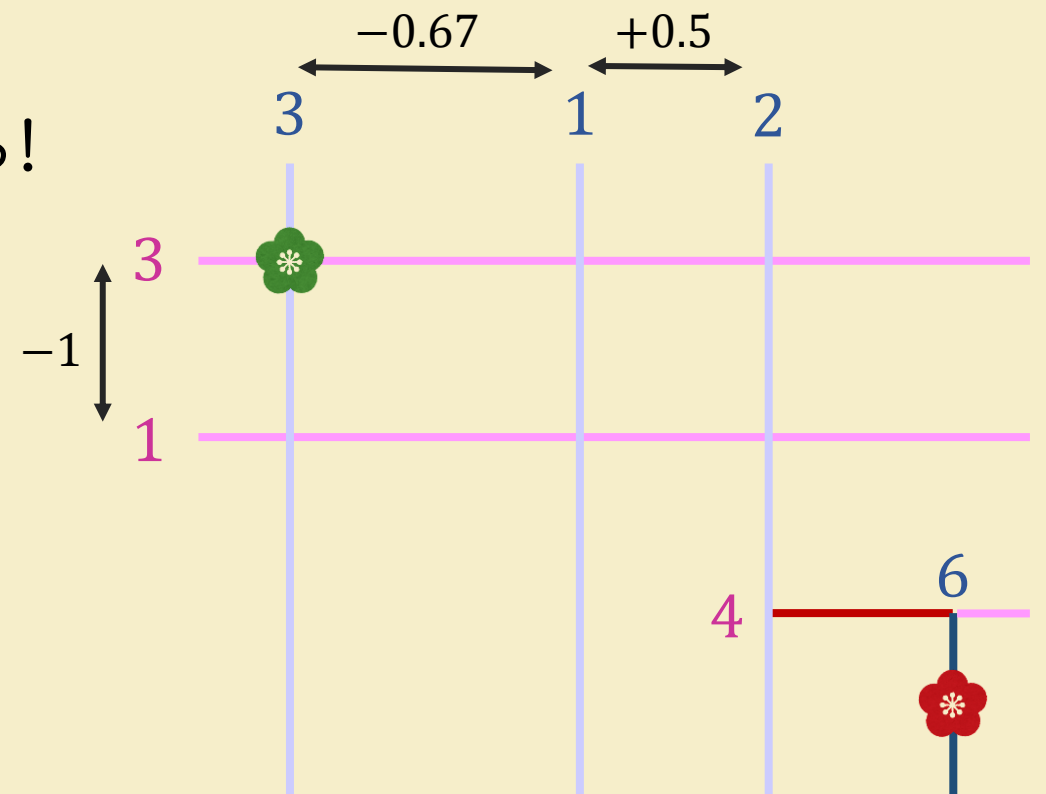
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



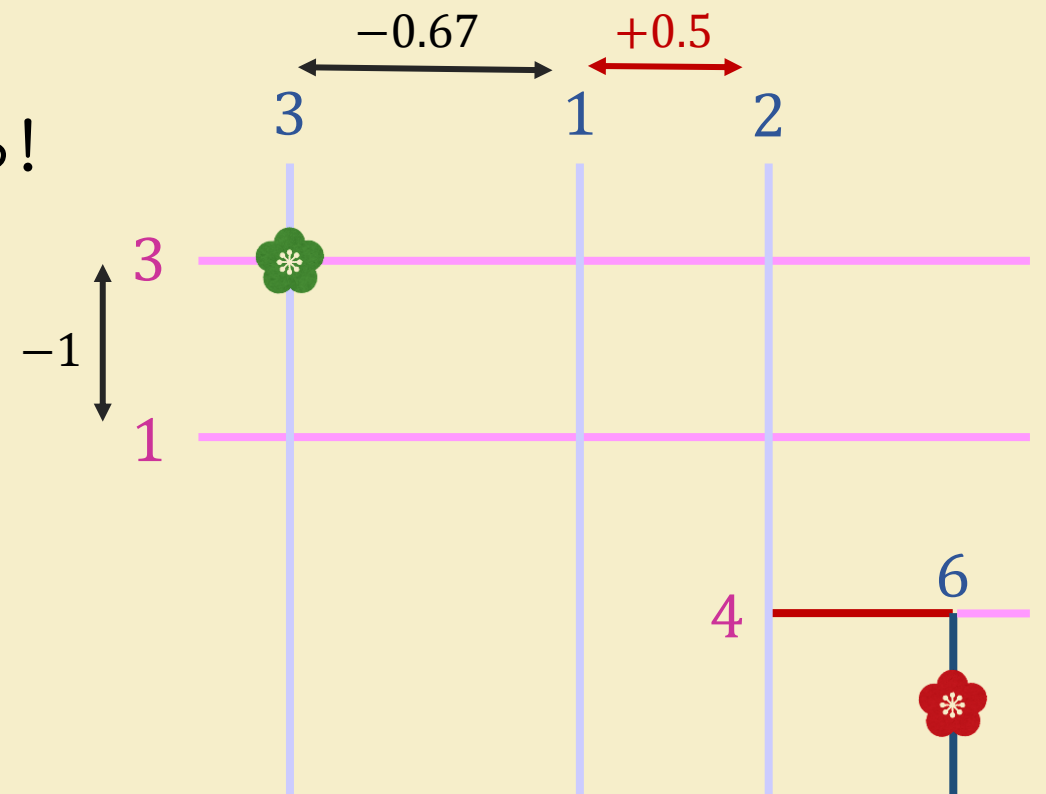
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



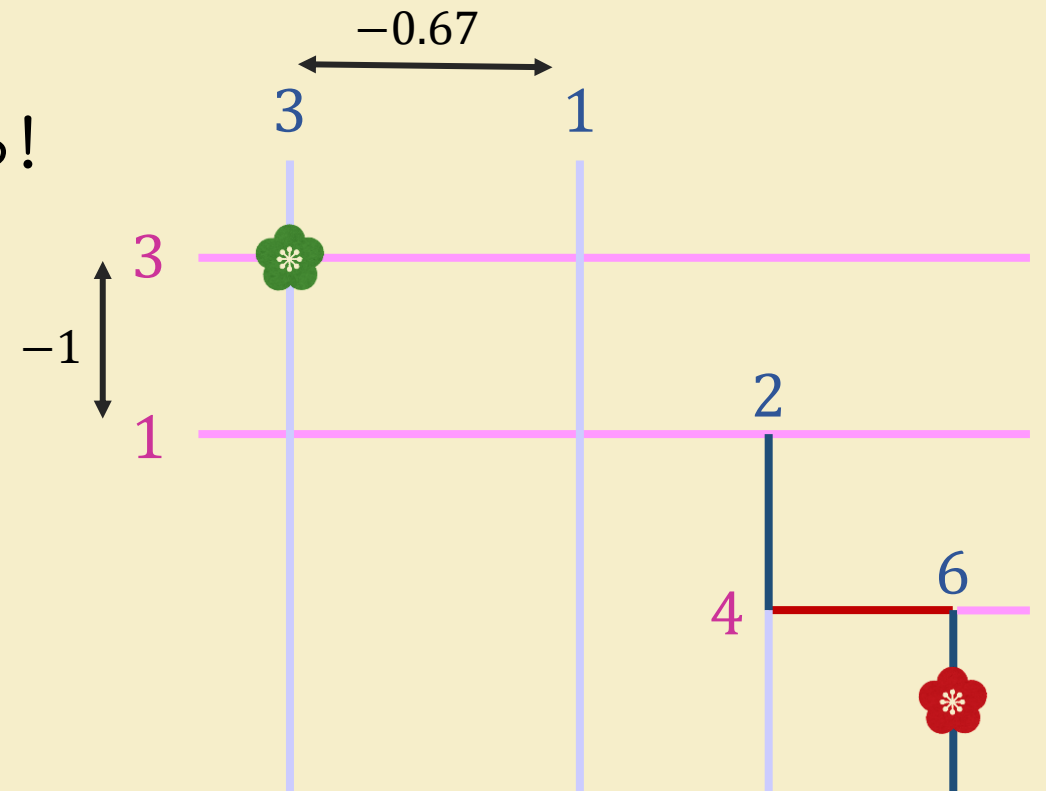
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



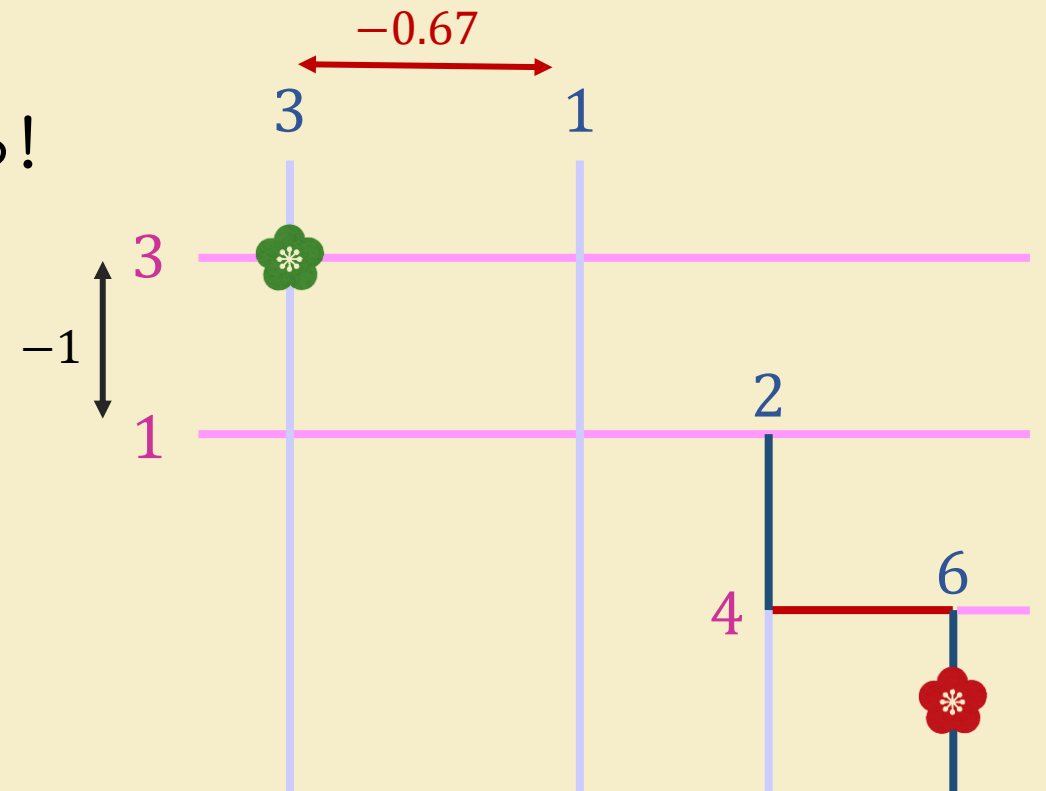
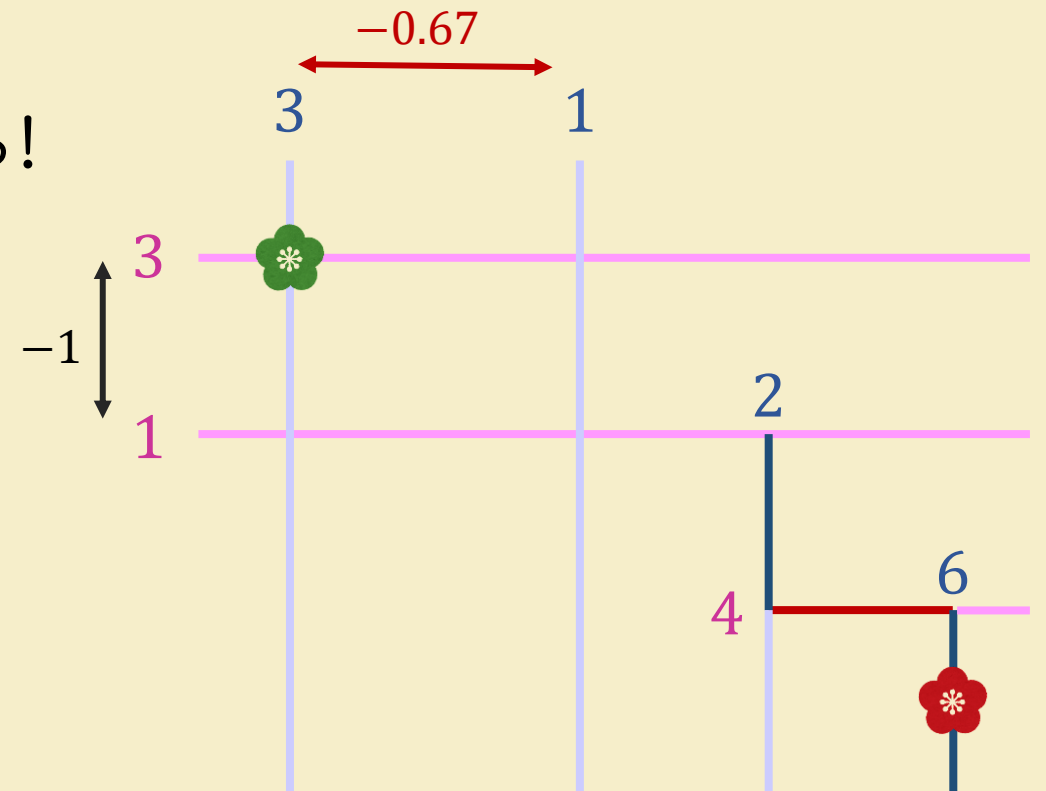
別解

「傾きが最大のを取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい!

「点 (i, A_i) の凸包」に含まれる道だけに削れる!

これでシミュレーションするとどうなる？



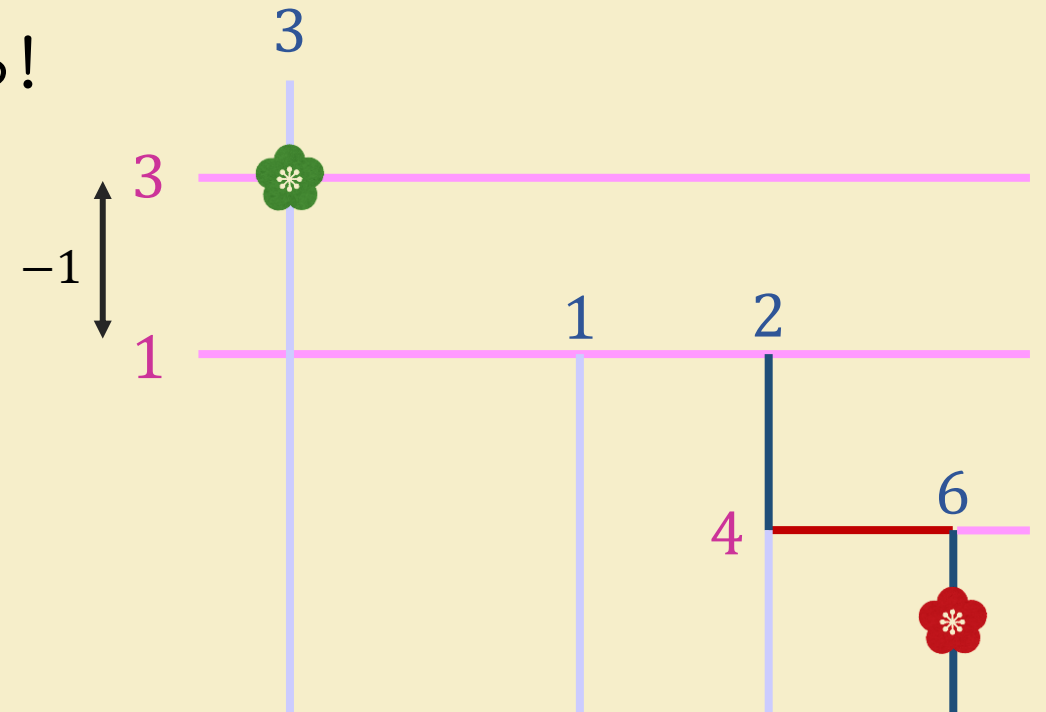
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



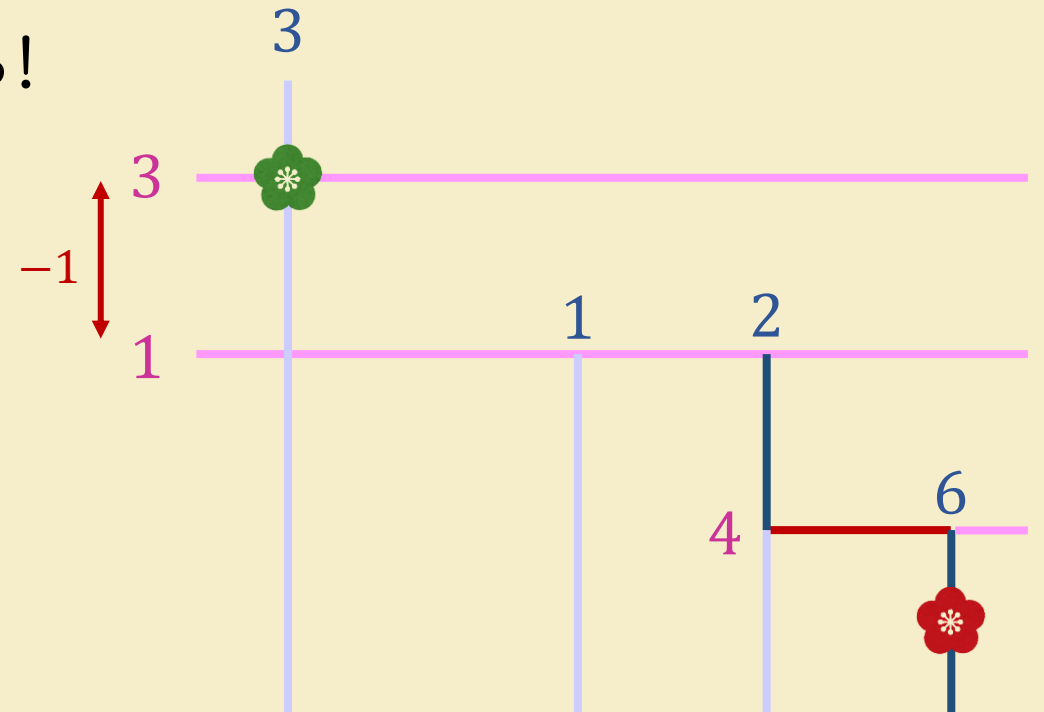
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



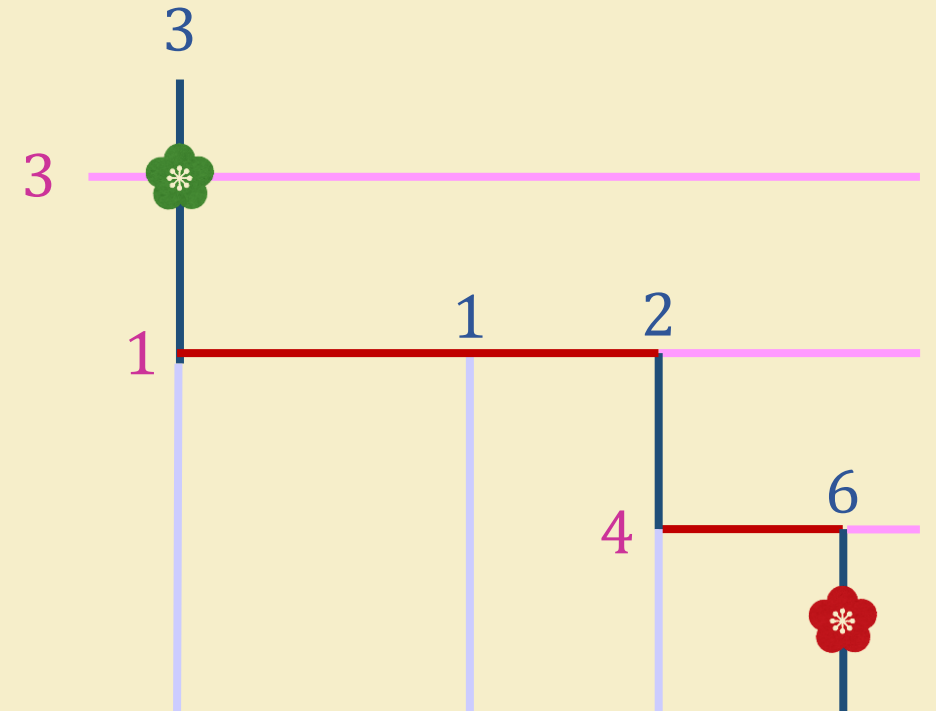
別解

「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？



別解

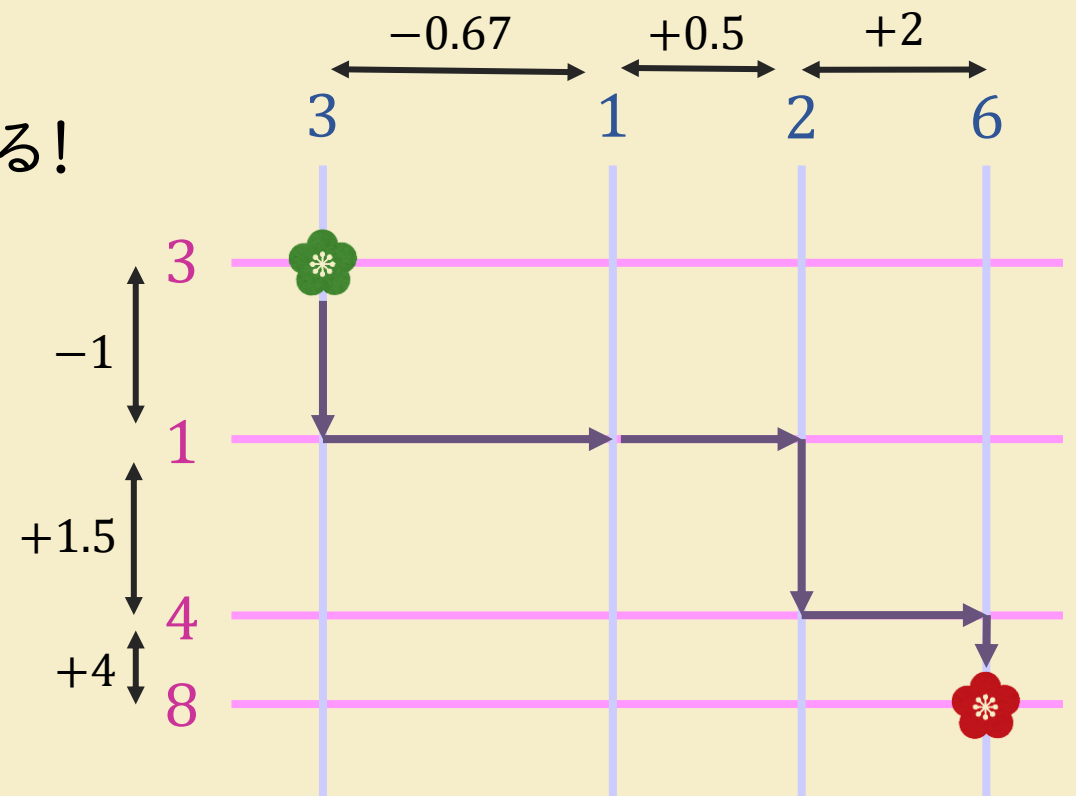
「傾きが最大を取り除く」というのは、どういうこと？

→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？

よって、答えはこうなります！



別解

「傾きが最大を取り除く」というのは、どういうこと？

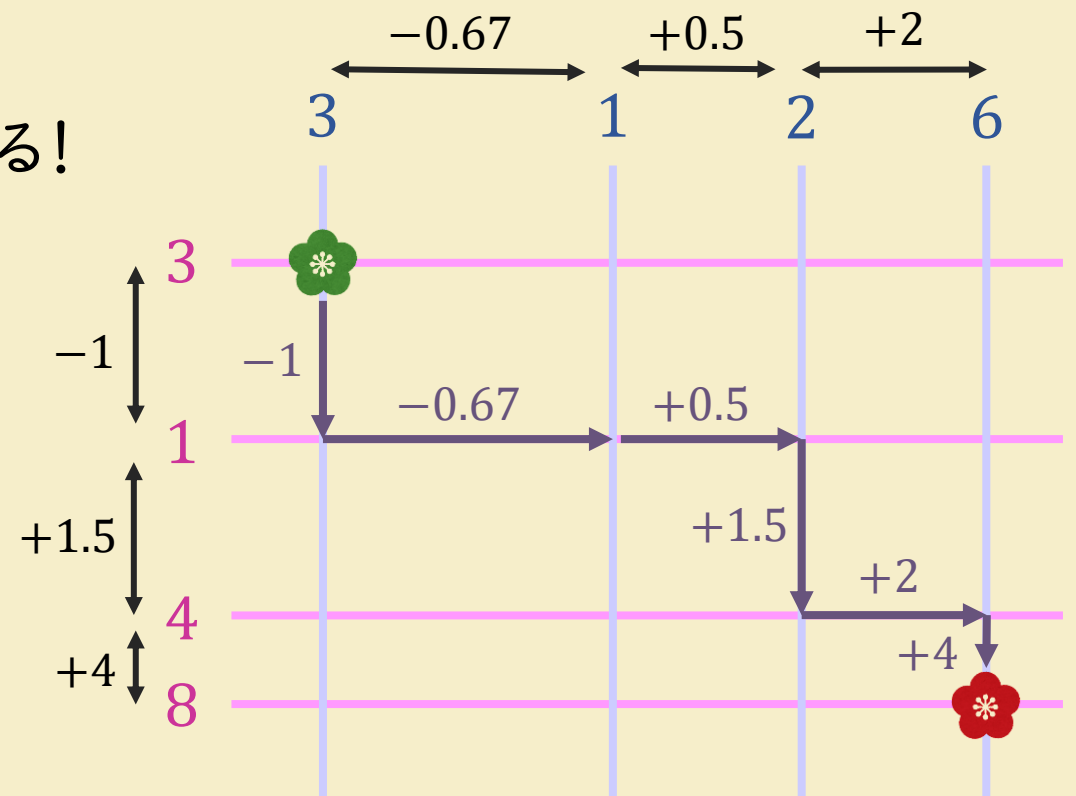
→ 結局は「点 (i, A_i) の凸包」を考えればよい！

「点 (i, A_i) の凸包」に含まれる道だけに削れる！

これでシミュレーションするとどうなる？

よって、答えはこうなります！

「傾きの小さい道路」から通るのが最適！



別解

アルゴリズム

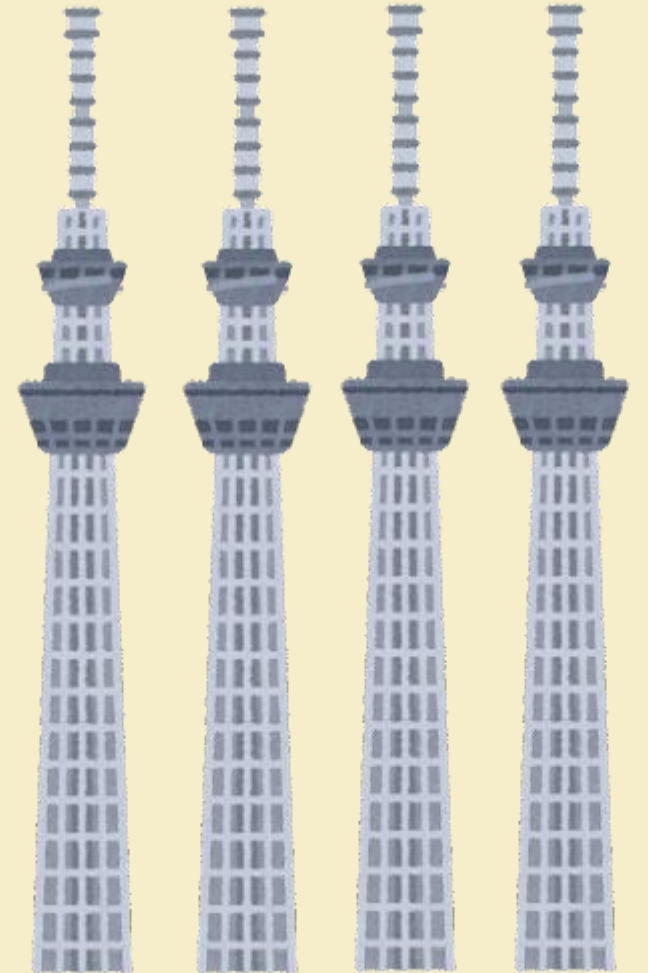
- 1 点 (i, A_i) の凸包を求める $\rightarrow$ 計算量 $O(H)$
- 2 点 (j, B_j) の凸包を求める $\rightarrow$ 計算量 $O(W)$
- 3 凸包の傾きをマージする $\rightarrow$ 計算量 $O(H + W)$

別解

アルゴリズム

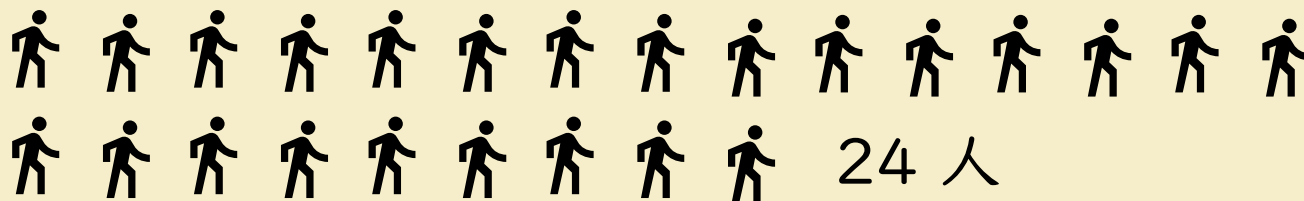
- 1 点 (i, A_i) の凸包を求める $\rightarrow$ 計算量 $O(H)$
- 2 点 (j, B_j) の凸包を求める $\rightarrow$ 計算量 $O(W)$
- 3 凸包の傾きをマージする $\rightarrow$ 計算量 $O(H + W)$

全体計算量 $O(H + W)$



得点分布

10 点



40 点



100 点

0 人

得点分布

全員 10 点以上

Congratulations!!!



Misspelling 解説

ynymxiaolongbao (米山 瑛士)

問題概要

$S =$ acccbba

英小文字からなる文
字列 S について考える

問題概要

$S =$ acccbba

$T_1 =$ cccbba

$T_2 =$ a ccbba

$T_3 =$ ac cbba

$T_4 =$ acc bba

$T_5 =$ accc ba

$T_6 =$ acccb a

$T_7 =$ acccbb

T_i を S の i 文字目を削除して
詰めた文字列とする

問題概要

$S =$ acccbba

$T_1 =$ cccbba

$T_2 =$ accbba

$T_3 =$ accbba

$T_4 =$ accbba

$T_5 =$ acccba

$T_6 =$ acccba

$T_7 =$ acccbb

T_i を S の i 文字目を削除して
詰めた文字列とする

問題概要

$S =$ acccbba
 $T_1 =$ cccbba
 $T_2 =$ accbba
 $T_3 =$ accbba
 $T_4 =$ accbba
 $T_5 =$ acccba
 $T_6 =$ acccba
 $T_7 =$ acccbb

$T_i \leq T_j$ (辞書順) の形の
条件が M 個与えられる

それらを満たす S を数え
上げる

小課題 1 ($N \leq 10$)

$S =$ acccbba

$T_1 =$ cccbba

$T_2 =$ accbba

$T_3 =$ accbba

$T_4 =$ accbba

$T_5 =$ acccba

$T_6 =$ acccba

$T_7 =$ acccbb

$T_i \leq T_j$ とは？ ($i < j$)

小課題 1 ($N \leq 10$)

$S =$ acccbba

$T_1 =$ cccbba

$T_2 =$ accbba

$T_3 =$ accbba

$T_4 =$ accbba

$T_5 =$ acccba

$T_6 =$ acccba

$T_7 =$ acccbb

$T_i \leq T_j$ とは？

例として、 T_2 と T_6
の大小を考える

小課題 1 ($N \leq 10$)

$S =$ acccbba

$T_1 =$ cccbba

$T_2 =$ a ccbba

$T_3 =$ ac cbba

$T_4 =$ acc bba

$T_5 =$ accc ba

$T_6 =$ acccb a

$T_7 =$ acccbb

$T_i \leq T_j$ とは？

例として、 T_2 と T_6
の大小を考える

小課題 1 ($N \leq 10$)

$S =$ acccbba

$T_2 =$ a ccbba

$T_6 =$ acccb a

$T_i \leq T_j$ とは？

例として、 T_2 と T_6
の大小を考える

小課題 1 ($N \leq 10$)

$S =$ a c c c b b a

$T_2 =$ a c c b b a

$T_6 =$ a c c c b a

$T_i \leq T_j$ とは？

例として、 T_2 と T_6
の大小を考える

- ・ はじめと終わりは同じなので S の $[i, j]$ だけ見る

小課題 1 ($N \leq 10$)

$S =$ a c c c b b a

$T_2 =$ a c c b b a

$T_6 =$ a c c c b a

$T_i \leq T_j$ とは？

例として、 T_2 と T_6
の大小を考える

- ・ はじめと終わりは同じなので S の $[i, j]$ だけ見る

小課題 1 ($N \leq 10$)

$S =$ a c c c b b a

$T_2 =$ a c c b b a

$T_6 =$ a c c c b a

$T_i \leq T_j$ とは？

例として、 T_2 と T_6
の大小を考える

- ・ はじめと終わりは同じなので S の $[i, j]$ だけ見る
- ・ S の **i 以降で初めて文字の種類が変わる所**で初めて T_i と T_j に違いが出る (T_i で S_{k+1} 、 T_j で S_k)

小課題 1 ($N \leq 10$)

$S =$ a**ccccca**

$T_2 =$ a **cccca**

$T_6 =$ a**cccc** a

$T_i \leq T_j$ とは？

例として、 T_2 と T_6
の大小を考える

- ・ はじめと終わりは同じなので S の $[i, j]$ だけ見る
- ・ S の i 以降で初めて文字の種類が変わる所で初めて T_i と T_j に違いが出る (T_i で S_{k+1} 、 T_j で S_k)
- ・ もし **j までに文字の種類が変わらない**なら $T_i = T_j$

小課題 1 ($N \leq 10$)

- T_i と T_j の大小関係は、各 k についての S_k と S_{k+1} の大小関係のみから判断できる
- 各 k について $S_k < S_{k+1}$ 、 $S_k = S_{k+1}$ 、 $S_k > S_{k+1}$ のどれが成り立つか決め打つ ($3^{\{N-1\}}$ 通り)
- これを満たす S の個数を dp で求め、 T の大小関係の条件を満たしているか確認し、足し合わせる
- $K = 26$ として計算量は $O(3^N \text{poly}(N) \text{poly}(K))$

小課題 2 ($N \leq 200$)

- S を後ろから決めていくことを考える

隣合う文字の大小関係も書いていく

$$S_2 = S_3 < S_4 = S_5 > S_6 < S_7$$

T_2 と T_6 の大小関係が知りたいとする

小課題 2 ($N \leq 200$)

- T_2 と T_6 の大小関係が知りたい

$$S_2 = S_3 < S_4 = S_5 > S_6 < S_7$$

小課題 2 ($N \leq 200$)

- T_2 と T_6 の大小関係が知りたい

$$S_2 = S_3 < S_4 = S_5 > S_6 < S_7$$

$i(= 2)$ 以降で初めて文字の種類が変わる所が $j(= 6)$ までにあるか、あるなら $S_k < S_{k+1}$ か $S_k > S_{k+1}$ かが分かればよい

小課題 2 ($N \leq 200$)

- T_2 と T_6 の大小関係が知りたい

$$S_2 = S_3 < S_4 = S_5 > S_6 < S_7$$

一番左（＝一番最後に書いた） $<$ や $>$ の
場所と種類をメモしておけばよい
→ dp のキーにする

小課題 2 ($N \leq 200$)

- S を後ろから決めていく
- $dp(pos, x, c, op) :=$ 以下を満たす決め方の個数
 - S_{pos} まで決めて、 $S_{pos} = c$
 - x は $pos \leq x$ かつ $S_x \neq S_{x+1}$ を満たす最小の値
 - op は $< \text{か} >$ であり、 $S_x op S_{x+1}$
- 計算量は $O(N^2 poly(K))$

小課題 3 (T をソートした順が定まっている)

- T をソートしたときの順番について考える
- $S_k \neq S_{k+1}$ のとき、 $T_{k+1}, T_{k+2}, \dots, T_N$ は一つの連続した区間になる

$$S_1 < S_2 = S_3 > S_4 = S_5 < S_6 = S_7 = S_8$$

小課題 3 (T をソートした順が定まっている)

- T をソートしたときの順番について考える
- $S_k \neq S_{k+1}$ のとき、 $T_{k+1}, T_{k+2}, \dots, T_N$ は一つの連続した区間になる

$$S_1 < S_2 = S_3 > S_4 = S_5 < S_6 = S_7 = S_8$$

小課題3 (T をソートした順が定まってる)

- T をソートしたときの順番について考える
- $S_k \neq S_{k+1}$ のとき、 $T_{k+1}, T_{k+2}, \dots, T_N$ は一つの連続した区間になる

$$S_1 < S_2 = S_3 > S_4 = S_5 < S_6 = S_7 = S_8$$

↑ T_3 は $T_4, T_5, \dots, T_8$ よりも小さい

↑ T_2 は $T_4, T_5, \dots, T_8$ よりも小さい

↑ T_1 は $T_4, T_5, \dots, T_8$ よりも大きい

小課題3 (T をソートした順が定まってる)

- T をソートしたときの順番について考える
- $S_k \neq S_{k+1}$ のとき、 $T_{k+1}, T_{k+2}, \dots, T_N$ は一つの連続した区間になる

$$S_1 < S_2 = S_3 > S_4 = S_5 < S_6 = S_7 = S_8$$

↑ T_3 は $T_4, T_5, \dots, T_8$ よりも小さい

↑ T_2 は $T_4, T_5, \dots, T_8$ よりも小さい

- $S_k = S_{k+1}$ のとき T_k と T_{k+1} は同じ側にある

小課題 3 (T をソートした順が定まってる)

- T をソートしたときの順番について考える
- $S_k \neq S_{k+1}$ のとき、 $T_{k+1}, T_{k+2}, \dots, T_N$ は一つの連続した区間になる
- $S_k = S_{k+1}$ のとき T_k と T_{k+1} は同じ側にある

小課題 3 (T をソートした順が定まってる)

- T をソートしたときの順番について考える
- $S_k \neq S_{k+1}$ のとき、 $T_{k+1}, T_{k+2}, \dots, T_N$ は一つの連続した区間になる
- $S_k = S_{k+1}$ のとき T_k と T_{k+1} は同じ側にある
- これが満たされていれば十分 $\rightarrow$ 必要十分

小課題 3 (T をソートした順が定まってる)

- S を後ろから決めていく
- 小課題 2 の dp から x (最後に見た不等号の位置) を抜いて $dp(pos, c, op)$ とする
- $T_{pos}, T_{pos+1}, \dots, T_N$ が T をソートしたとき一つの連続した区間になるときだけ $S_{pos-1} \neq S_{pos}$ の遷移をする
- 計算量は $O(Npoly(K))$

小課題 4, 5 ($N \leq 2 \times 10^4, 5 \times 10^5$)

- 小課題 2 の dp をデータ構造で高速化する
- $dp(pos, x, c, op) \rightarrow dp(pos - 1, x, c, op)$
という遷移がある (前と同じ文字にする操作)
- (x, c, op) ごとに値を使いまわすことにすると、
 pos が変わったとき、何もしなければこの遷移ができる

小課題 4, 5 ($N \leq 2 \times 10^4, 5 \times 10^5$)

- 他の遷移は以下ようになる
 - ・ 特定の (c, op) の組に対して、 $dp(x, c, op)$ の $x < r$ の部分を 0 にする (T の大小関係の条件)
 - ・ 特定の (c, c', op, op') の組に対して、 $dp(pos, c', op')$ に $dp(x, c, op)$ の和を足す (前と別の文字にする操作)

小課題 4, 5 ($N \leq 2 \times 10^4, 5 \times 10^5$)

- (c, op) ごとに遅延セグ木を持つ $\rightarrow O(N \log N \text{poly}(K))$

- 特定の (c, op) の組に対して、 $dp(x, c, op)$ の $x < r$ の部分を0にする (T の大小関係の条件)

- $\rightarrow$ 区間に0を代入

- 特定の (c, c', op, op') の組に対して、 $dp(pos, c', op')$ に $dp(x, c, op)$ の和を足す (前と別の文字にする操作)

- $\rightarrow$ 全体の和を求める、一点更新

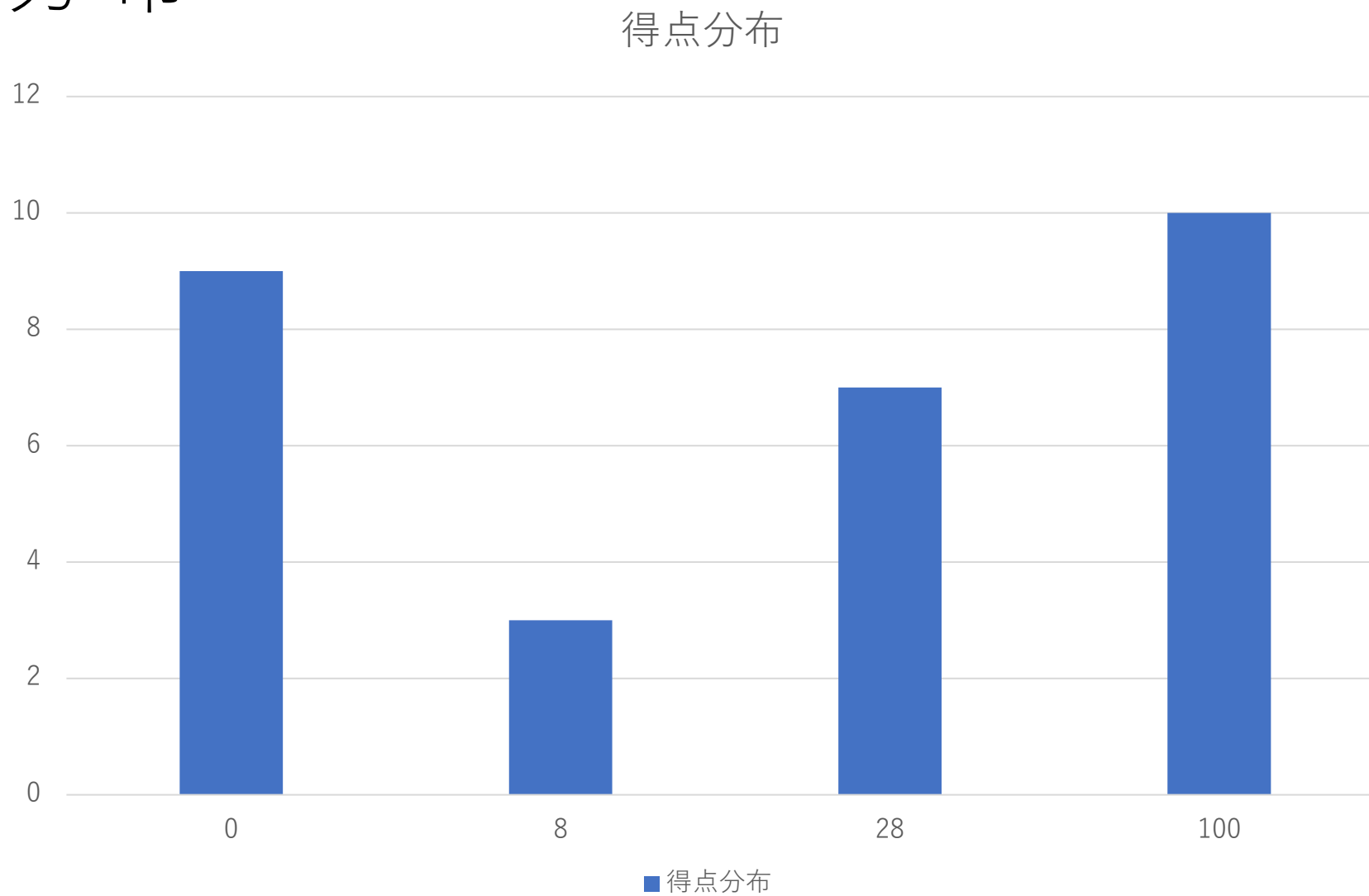
小課題 4, 5 ($N \leq 2 \times 10^4, 5 \times 10^5$)

- (c, op) ごとにスタックを持つ $\rightarrow O(N \text{poly}(K))$
 - 特定の (c, op) の組に対して、 $dp(x, c, op)$ の $x < r$ の部分を0にする (T の大小関係の条件)
 $\rightarrow$ 左からいくつかの要素を削除
 - 特定の (c, c', op, op') の組に対して、 $dp(pos, c', op')$ に $dp(x, c, op)$ の和を足す (前と別の文字にする操作)
 $\rightarrow$ 全体の和を求める、左に要素を追加

別解？

- setやpriority_queueを使う（やってることは同じ）
- 二次元累積和を使って頑張る

得点分布



コピー & ペースト 3 解説

松尾 凜太郎 / QCFium

問題概要

x, y は最初空文字列。 x を S に等しくしたい。

できる操作

- A) x の末尾に任意の 1 文字追加
- B) y を x に等しくし、 x を空文字列にする
- C) x の末尾に y を結合

それぞれ A, B, C のコストがかかるとき、 S を入力するためにかかるコストの最小値は？

考察 1

できる操作

- A) x の末尾に任意の 1 文字追加
- B) y を x に等しくし、 x を空文字列にする
- C) x の末尾に y を結合

どう考えても

- 操作 A で入力する文字は S に含まれる文字のみ
- x, y を S の長さ以上にするのは無駄

小課題 1

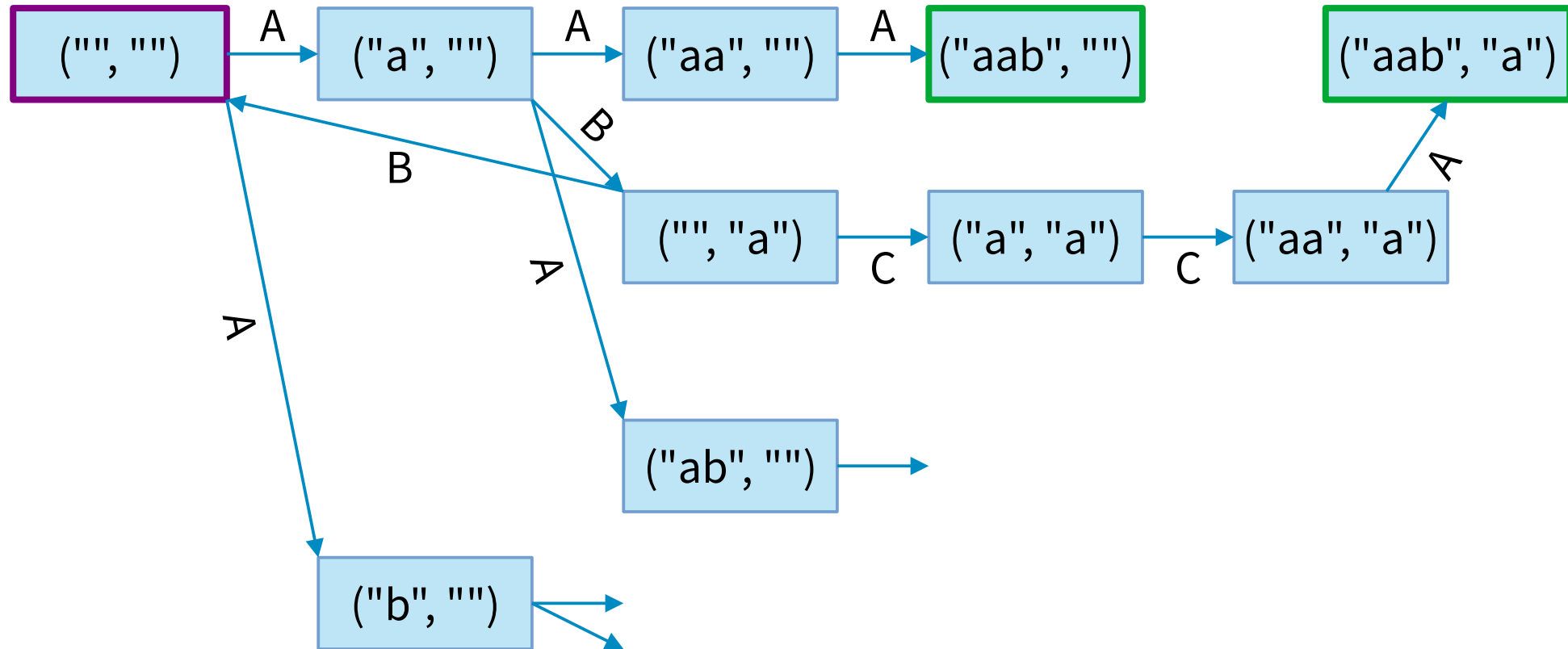
制約 : $N = 3$

N が小さい $\rightarrow$ x, y の状態を全て管理してダイクストラ法
あり得る x, y の種類はそれぞれ $1+3+9+27=40$ (S が全て異なる
文字の場合)

40×40 頂点のグラフでダイクストラ法

小課題 1

例 : $S = \text{"aab"}$



小課題 1

頂点数は 40×40 以下

各頂点からは

- 操作 A の辺が高々 3 本
- 操作 B の辺が 1 本
- 操作 C の辺が高々 1 本

→ 間に合う

小課題 2

制約：S は 'a' のみからなる

x, y として考えられるのはそれぞれ $O(N)$ 個

→ 最短経路を求めるグラフの頂点数は $O(N^2)$

各頂点からでる辺の本数は多くとも

- 操作 A の辺が 1 本
- 操作 B, C も 1 本ずつ

→ $O(N^2 \log(N))$

考察 2

できる操作

- A) x の末尾に任意の 1 文字追加
- B) y を x に等しくし、 x を空文字列にする
- C) x の末尾に y を結合

x は空文字列 $\rightarrow y$ を捨てるのは無駄
 $\rightarrow$ 切り取り直後の y は x の部分文字列になる
 $\rightarrow y$ は S の (連続する) 部分文字列になるとしてよい

任意の時点での x も S の (連続する) 部分文字列になるとしてよい

小課題 3

制約 : $N \leq 30$

x, y は S の連続する部分列なので $O(N^2)$ 個

少し工夫して辺を張れば最短経路を求めるグラフ (頂点数 $O(N^4)$ 辺数 $O(N^5)$) を $O(N^6)$ で構築できる

- 操作 A

各 x について $x+c$ で表すことのできる S の部分文字列の位置を前計算 ($O(N^4)$) しておけばよい

- 操作 B

頂点ごとに S の部分文字列で $x+y$ に等しいものを検索 (愚直にやると $O(N^6)$)

考察 3

できる操作

- A) x の末尾に任意の 1 文字追加
- B) y を x に等しくし、 x を空文字列にする
- C) x の末尾に y を結合

グラフを作って最短経路を求めるのは限界そう

((A,C を何回か) $\rightarrow$ B) の繰り返しと考える

$S[l:r]$: S の l 番目から $r-1$ 番目までの部分文字列 とする

$dp[l][r]$: 操作 B の直後で $x=""$, $y=S[l:r]$ である状態にするまでの最小コスト

$r-1$ の昇順に遷移していけばよい

$dp[s][t]$ から $dp[u][v]$ への遷移は？

考察 3

できる操作

- A) x の末尾に任意の 1 文字追加
- B) y を x に等しくし、 x を空文字列にする
- C) x の末尾に y を結合

$S[s:t] : \text{bba}$

$S[u:v] : \text{aa}\text{bba}\text{bbab}$

$A(t-s) < C$ だと $S[s:t]$ をクリップボードにコピーした意味がない

$A(t-s) \geq C$ ならできるだけ限りクリップボードを使う回数を多くしたい

→ 前から貪欲に貼り付けを使っていく

考察 3

クリップボード : bab

abababaababbbab



太青線部は貼り付け、それ以外は
1 文字入力するのが最適

考察 3

クリップボード : bab

ababaababbab



赤線（遷移先）が多すぎるので左のような赤線のみへの遷移を考える
自分と被らない次の青線に移ることを繰り返して更新可能

考察 3

クリップボード : bab

ababaababbab



はみ出した分は 1 文字入力するしかない
→ 赤線に遷移した後左右に 1 伸ばすの
をそれぞれコスト A で自由にすることに
すればよい

考察 3

最初に全ての s, t について $S[s:t] = S[i:i+(t-s)]$ となる i のリスト $a[s, t]$ を作る

$a[s, t]$ の各要素 i について、 $a[s, t]$ に含まれ $j \geq i+t-s$ を満たす最小の要素 j を前計算する

$dp[s][t]$ からの遷移は

- $dp[s][t+1]$ にコスト A で遷移 : $S[s:t]$ に遷移した過程の一番最後に操作 A で $S[t]$ を入力したことにする
- $dp[s-1][t]$ にコスト A で遷移 : $S[s:t]$ に遷移した過程の一番最初に操作 A で $S[s-1]$ を入力したことにする
- $a[s, t]$ に含まれる全ての i について :
 $a[s, t]$ のうち $i+(t-s)$ 以上の最小のものを辿っていき、更新

考察 3

最初に全ての s, t について $S[s:t] = S[i, i+(t-s)]$ となる i のリスト $a[s, t]$ を作る

$a[s, t]$ の各要素 i について、 $a[s, t]$ に含まれ $j \geq i+t-s$ を満たす最小の要素 j を前計算する

全部分文字列の Rolling Hash の結果を map に入れば $O(N^2 \log(N))$

考察 3

最初に全ての s, t について $S[s:t] = S[i, i+(t-s)]$ となる i のリスト $a[s, t]$ を作る

$a[s, t]$ の要素 j で $j \geq t$ を満たす最小のものも前計算しておく

$dp[s][t]$ からの遷移は

- $dp[s][t+1]$ にコスト A で遷移 : $S[s:t]$ に遷移した過程の一番最後に操作 A で $S[t]$ を入力したことにする
- $dp[s-1][t]$ にコスト A で遷移 : $S[s:t]$ に遷移した過程の一番最初に操作 A で $S[s-1]$ を入力したことにする
- $a[s, t]$ に含まれる全ての i について :
 $a[s, t]$ のうち $i+(t-s)$ 以上の最小のものを辿っていき、更新

小課題 4

クリップボード : bab

ababaababbab

=====

=====

=====

=====

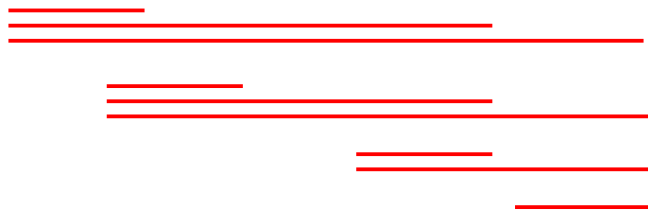
$dp[1][4], dp[3][6], dp[8][11], dp[11][14]$ をまとめて遷移
→ 1 遷移あたり $O(N)$

全体で $O(N^3)$

小課題 5

$N \leq 1000$

ababaababbab



よく見ると 1 遷移あたり
 $O(N / (\text{長さ}))$ になっているので
合計 $O(N^2 \log(N))$

小課題 6 (満点)

$$N \leq 2500$$

$O(N^2 \log(N))$ で通る場合と通らない場合がある

小課題 6 (満点)

$N \leq 2500$

最初に全ての s, t について $S[s:t] = S[i, i+(t-s)]$ となる i のリスト $a[s, t]$ を作る
 $a[s, t]$ の各要素 i について、 $a[s, t]$ に含まれ $j \geq i+t-s$ を満たす最小の要素 j を前計算する

全部分文字列の Rolling Hash の結果を map に入れば $O(N^2 \log(N))$

小課題 6 (満点)

$N \leq 2500$

最初に全ての s, t について $S[s:t] = S[i, i+(t-s)]$ となる i のリスト $a[s, t]$ を作る
 $a[s, t]$ の各要素 i について、 $a[s, t]$ に含まれ $j \geq i+t-s$ を満たす最小の要素 j を前計算する

全部分文字列の Rolling Hash の結果を map に入れば $O(N^2 \log(N))$

小課題 6 (満点)

$$N \leq 2500$$

最初に全ての s, t について $S[s:t] = S[i, i+(t-s)]$ となる i のリスト $a[s, t]$ を作る

$a[s, t]$ の各要素 i について、 $a[s, t]$ に含まれ $j \geq i+t-s$ を満たす最小の要素 j を前計算する

全部分文字列の Rolling Hash の結果を map に入れば $O(N^2 \log(N))$

全部分文字列のハッシュを求めたあと長さごとに分け map で処理すると TLE が消える

Z algorithm 等で上の処理を $O(N^2)$ にすることもできる

おわり

ありがとうございました

飛行機旅行 解説

JOI 2021/2022 春季トレーニング合宿

米田 優峻 / E869120

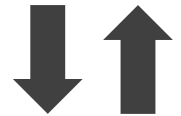
- 競技プログラミングでは、通常 **Batch 型課題**（出力の正誤によって採点される）が出題されます。
- しかしながら、情報オリンピックでは**通信型課題**などの特殊な形式が出題されることがあります。

通信型課題って
一体なに？

- 2つのプログラムを書く
- それらを問題で指定された形式でやりとりさせることによって、正解を導き出す。



Program 1



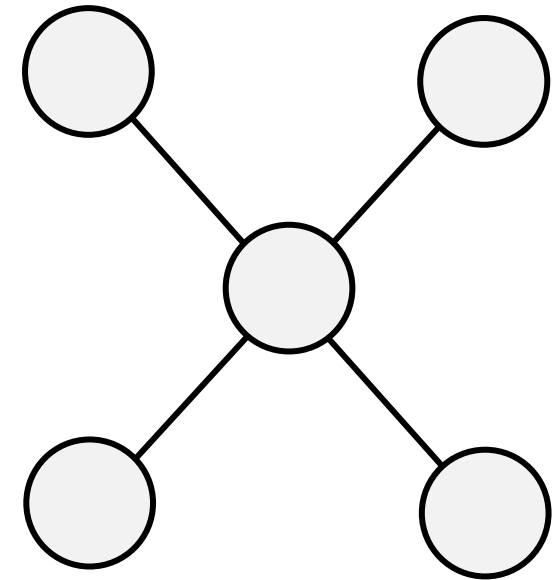
Program 2

Communication Task

とも呼ばれる

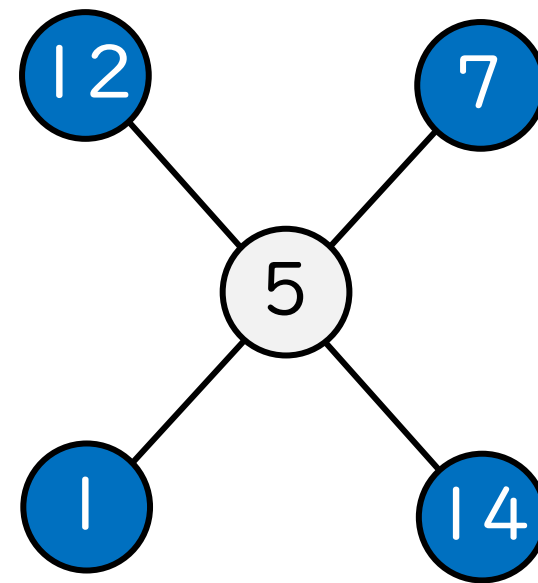
0

N 頂点の木で表される航空路線図がある (Ali だけが知る)



0 N 頂点の木で表される航空路線図がある (Ali だけが知る)

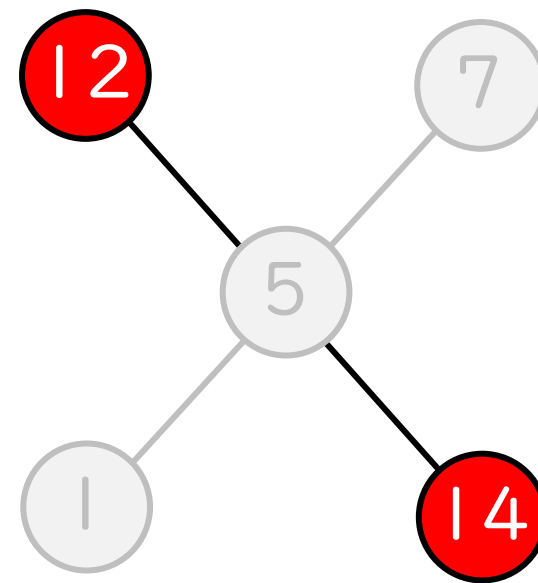
1 まず Ali は各頂点に ID を付ける



0 N 頂点の木で表される航空路線図がある (Ali だけが知る)

1 まず Ali は各頂点に ID を付ける

2 Benjamin は頂点 u, v の ID を得る



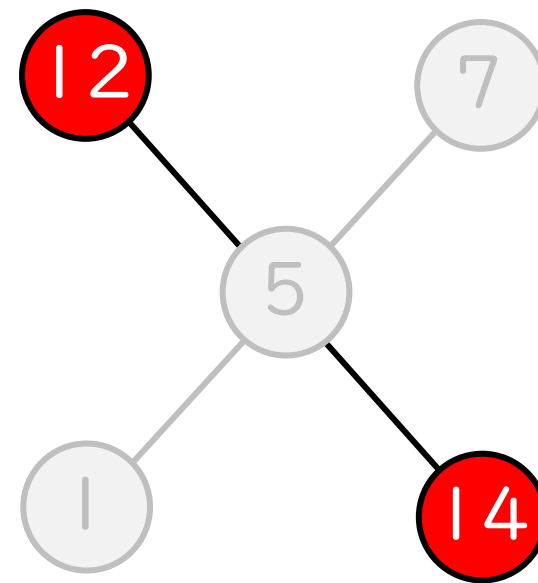
0 N 頂点の木で表される航空路線図がある (Ali だけが知る)

1 まず Ali は各頂点に ID を付ける

2 Benjamin は頂点 u, v の ID を得る

3 2 人が通信をして

Benjamin が 2 頂点間の距離※を答える



※距離 = 最短経路の長さ。

満点を取るためには…？

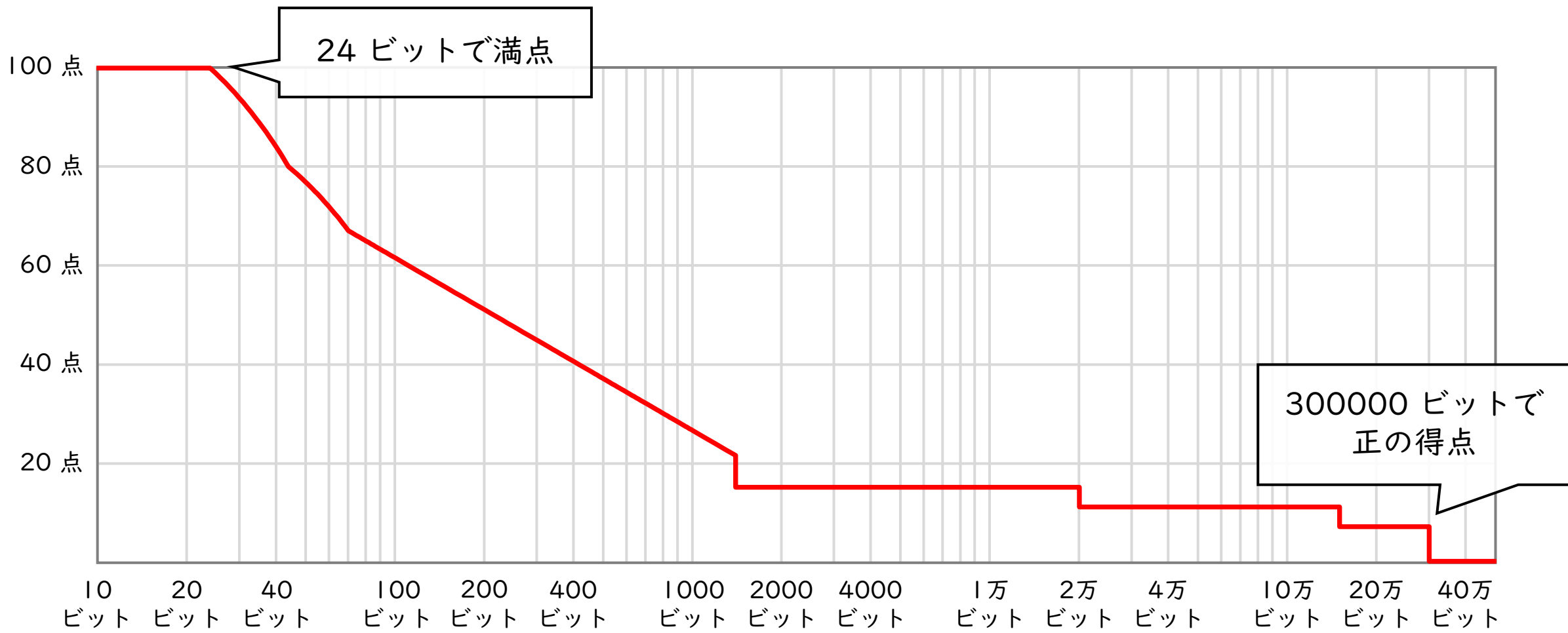
Benjamin→Ali

20 ビット

Ali→Benjamin

24 ビット以内

※以降は解説の都合上、「文字数」を「ビット数」として表します



Ali → Benjamin の最大ビット数 (文字数)

Stage 1

20000 ビットまでの解法

Ali が Benjamin に
木の情報を全部送ることを考える

必要なビット数は？

送るべき個数

20000個

辺の数は約 10000 本
それぞれの辺について
端点の番号を送るので
 $10000 \times 2 = 20000$ 個

必要なビット数は？

送るべき個数

1 個当たり

20000 個 × 14 ビット

辺の数は約 10000 本
それぞれの辺について
端点の番号を送るので
 $10000 \times 2 = 20000$ 個

頂点番号は 10000 まで
14 ビットあれば
 $2^{14} - 1 = 16383$ までを
表せるので十分

必要なビット数は？

$$\begin{array}{l} \text{送るべき個数} \\ 20000 \text{ 個} \end{array} \times \begin{array}{l} \text{1 個当たり} \\ 14 \text{ ビット} \end{array} = 280000 \text{ ビット}$$

辺の数は約 10000 本
それぞれの辺について
端点の番号を送るので
 $10000 \times 2 = 20000$ 個

頂点番号は 10000 まで
14 ビットあれば
 $2^{14} - 1 = 16383$ までを
表せるので十分



必要なビット数は？

送るべき個数

1 個当たり

ここまでの点数：**7**点

辺の数は約 10000 本
それぞれの辺について
端点の番号を送るので
 $10000 \times 2 = 20000$ 個

頂点番号は 10000 まで
14 ビットあれば
 $2^{14} - 1 = 16383$ までを
表せるので十分

Q. Benjamin は頂点番号 X, Y を
両方送れるのか？

頂点番号 X, Y を両方送るには？

頂点番号 X は 10000 まで ▶ 14 ビット

頂点番号 Y は 10000 まで ▶ 14 ビット

頂点番号 X, Y を両方送るには？

頂点番号 X は 10000 まで ▶ 14 ビット

頂点番号 Y は 10000 まで ▶ 14 ビット



合計 28 ビット

20 ビット制限に収まらない



点解法

20
222

頂点番号 X, Y を両方送るには？

頂点番号 X は 10000 だが 14 ビット

頂点番号 Y は 10000 だが 14 ビット
どちらか片方なら送れる



合計 28 ビット

20 ビット制限に収まらない

Benjamin が頂点番号 X を送り
Ali が番号 X から各頂点への距離を送る

Ali は何ビット必要か？

送るべき個数

10000個

各頂点までの距離を
全部送るので
頂点数 $N=10000$ 個

Ali は何ビット必要か？

送るべき個数

1 個当たり

10000 個 $\times$ 14 ビット

各頂点までの距離を
全部送るので
頂点数 $N=10000$ 個

頂点番号は 10000 まで
14 ビットあれば
 $2^{14}-1=16383$ までを
表せるので十分

Ali は何ビット必要か？

送るべき個数

1 個当たり

$$10000_{\text{個}} \times 14_{\text{ビット}} = 140000_{\text{ビット}}$$

各頂点までの距離を
全部送るので
頂点数 $N=10000$ 個

頂点番号は 10000 まで
14 ビットあれば
 $2^{14}-1=16383$ までを
表せるので十分



点解法

25
222

Ali は何ビット必要か？

送るべき個数

1 個当たり

ここまでの点数 := 140000 点

各頂点までの距離を
全部送るので
頂点数 $N=10000$ 個

頂点番号は 10000 まで
14 ビットあれば
 $2^{14}-1=16383$ までを
表せるので十分

7 点解法では木の情報を 280000 ビットかけて送ったが...

実は 2 万ビットで送れる

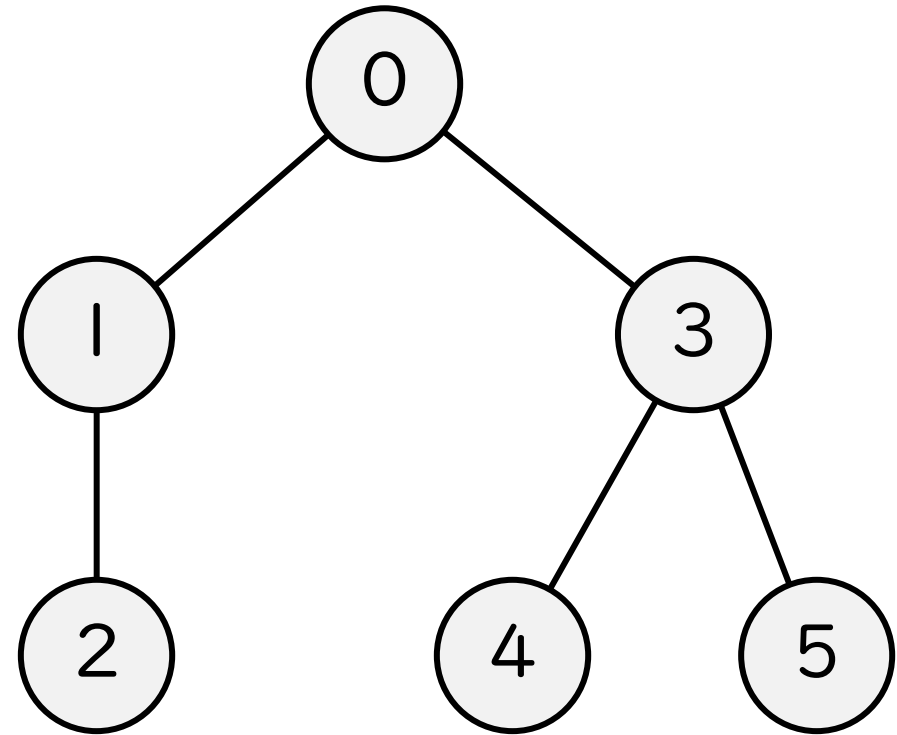


Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

Ali が送る方法

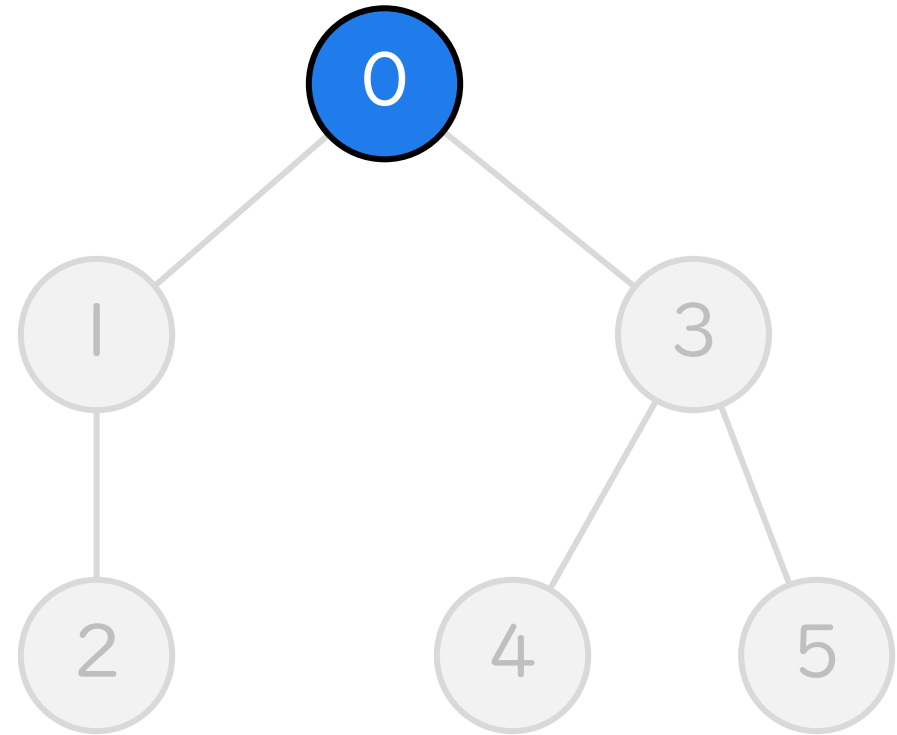
- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列：

Ali が送る方法

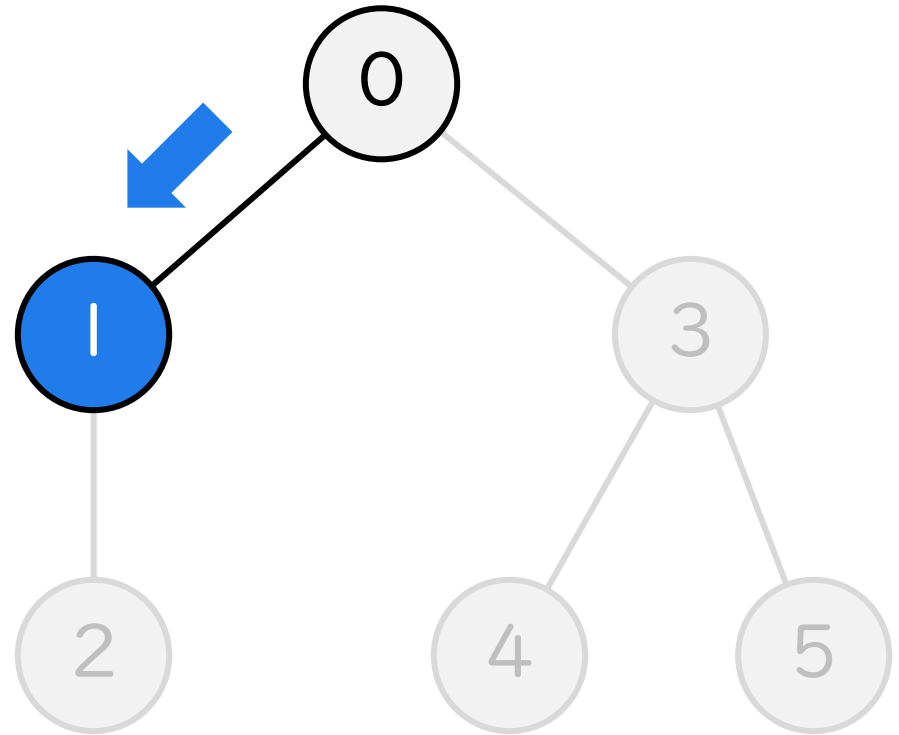
- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列：

Ali が送る方法

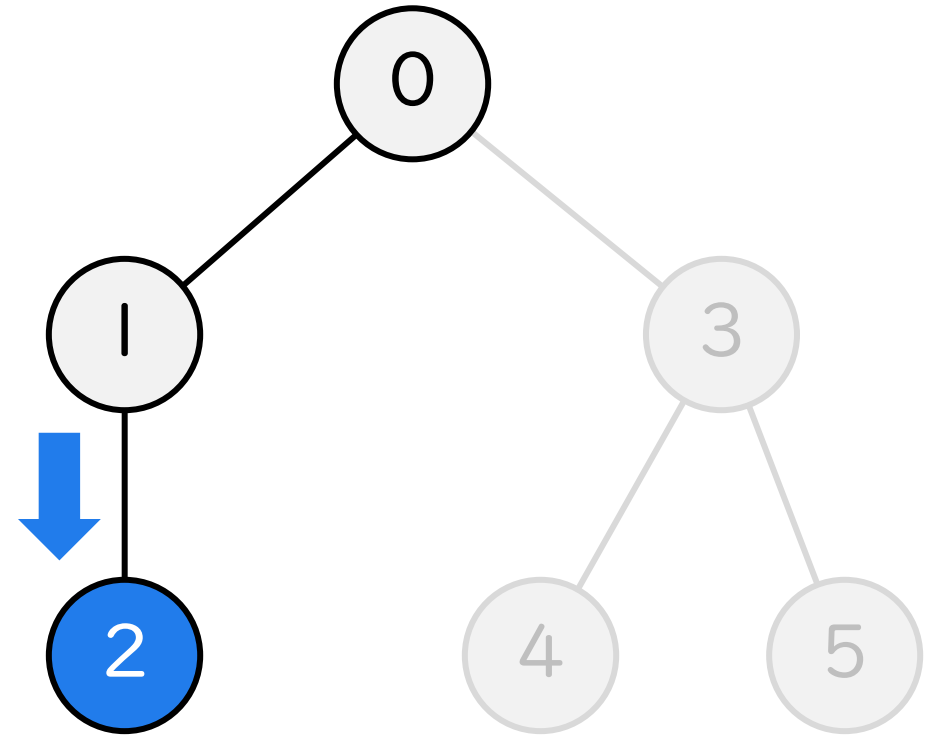
- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列： 0

Ali が送る方法

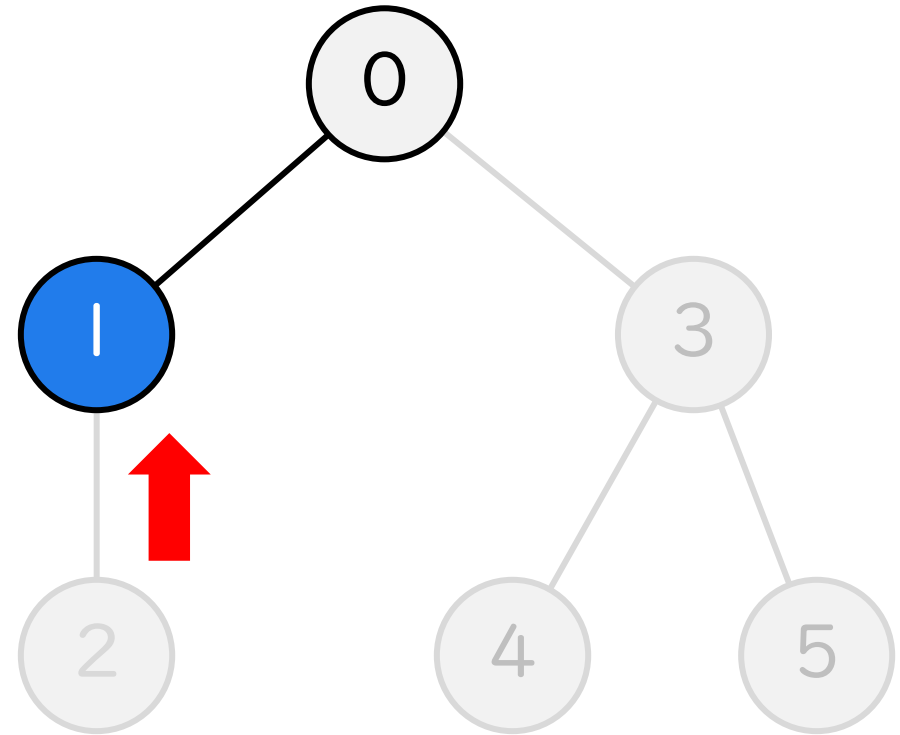
- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列： 00

Ali が送る方法

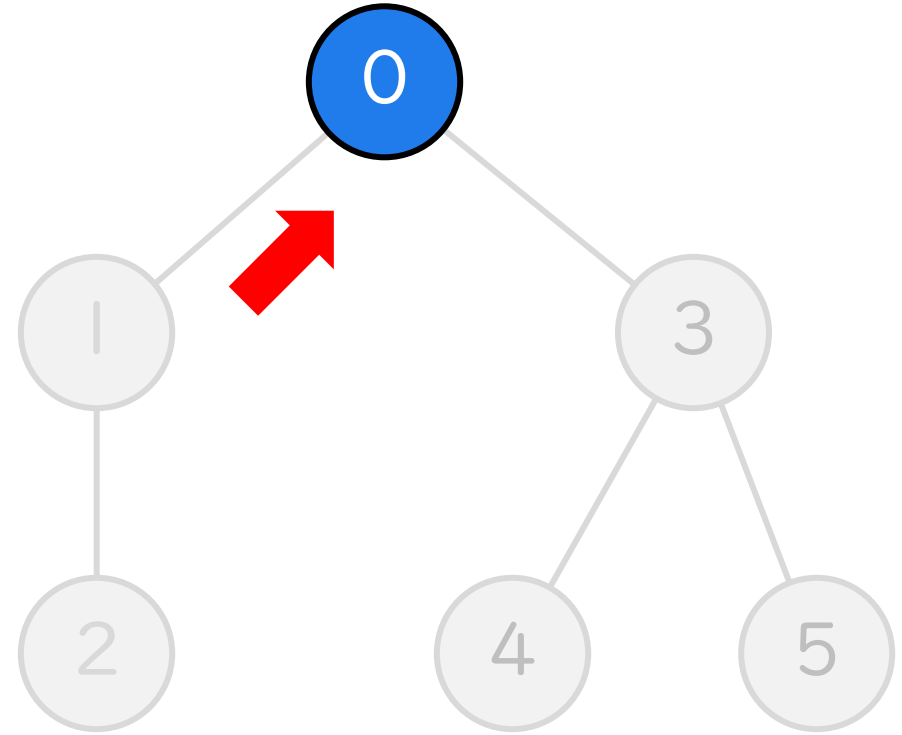
- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列：001

Ali が送る方法

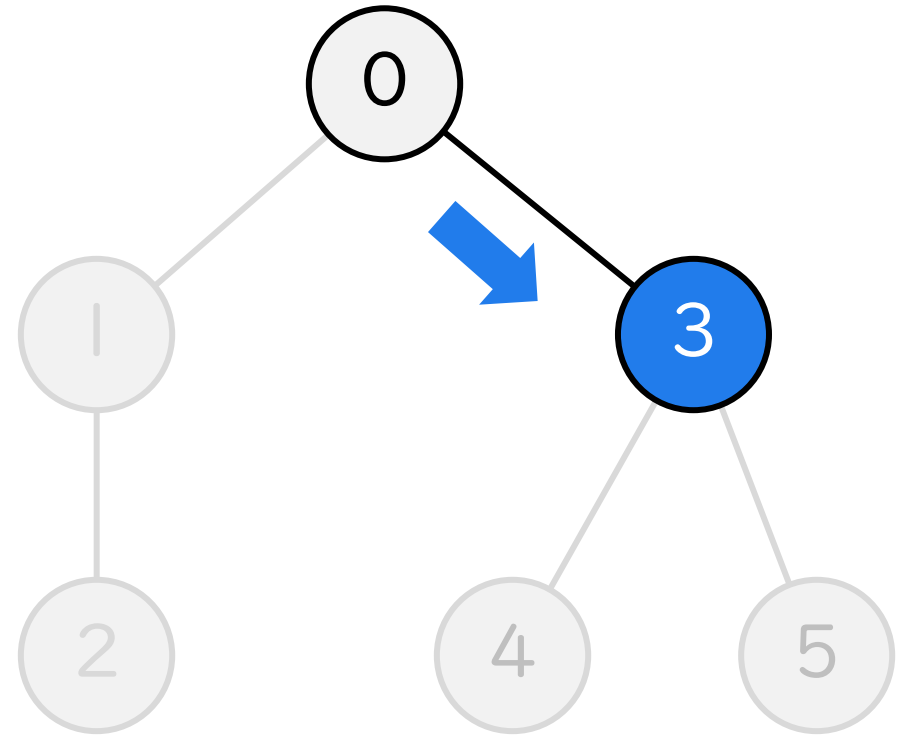
- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列：0011

Ali が送る方法

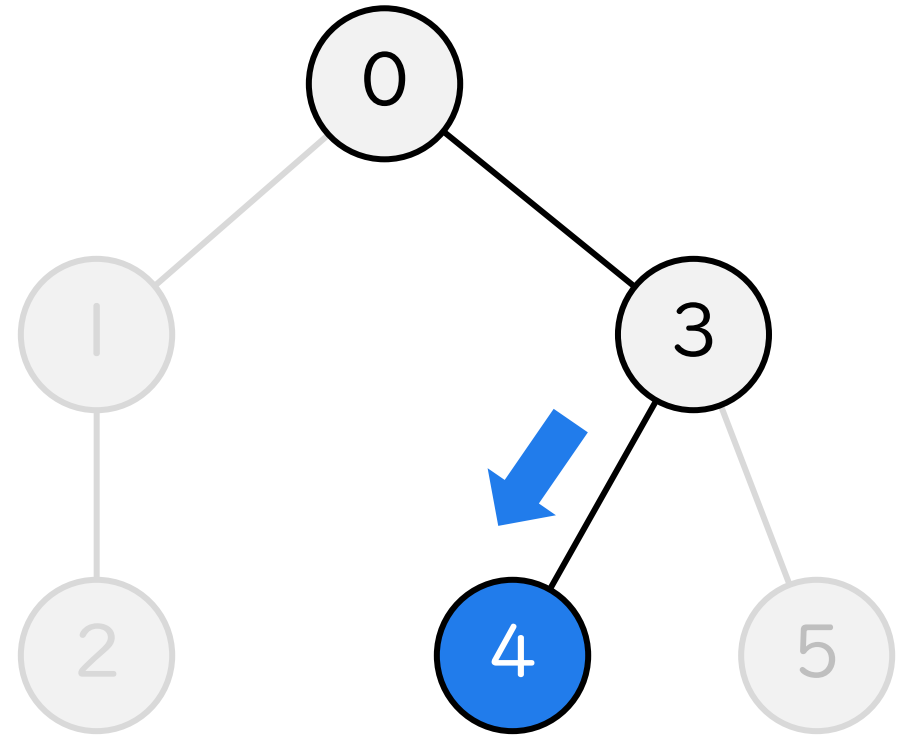
- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列：00110

Ali が送る方法

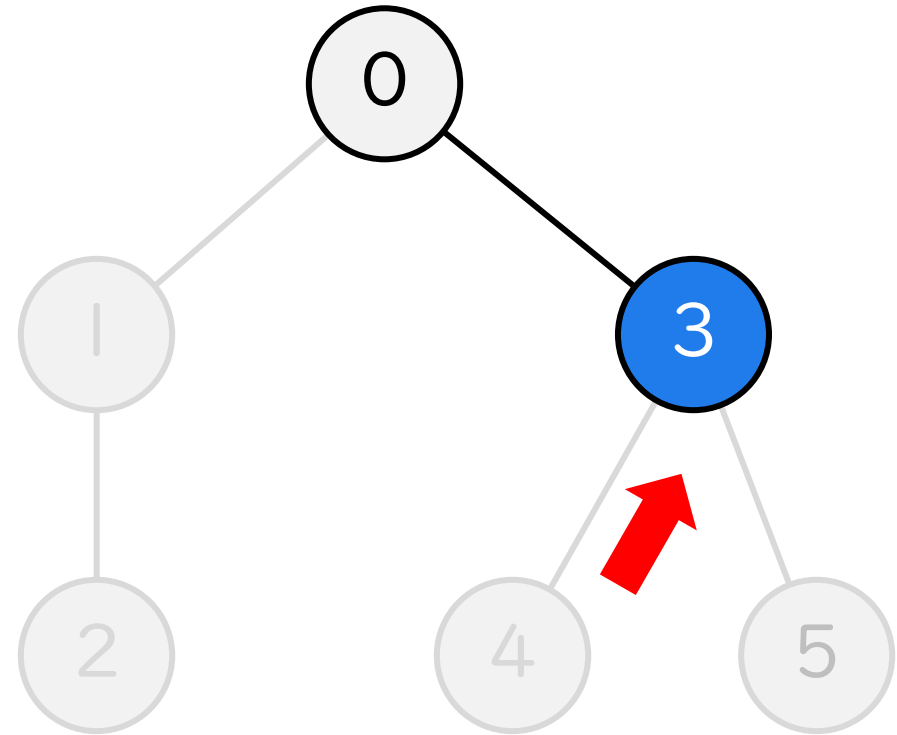
- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列： 001100

Ali が送る方法

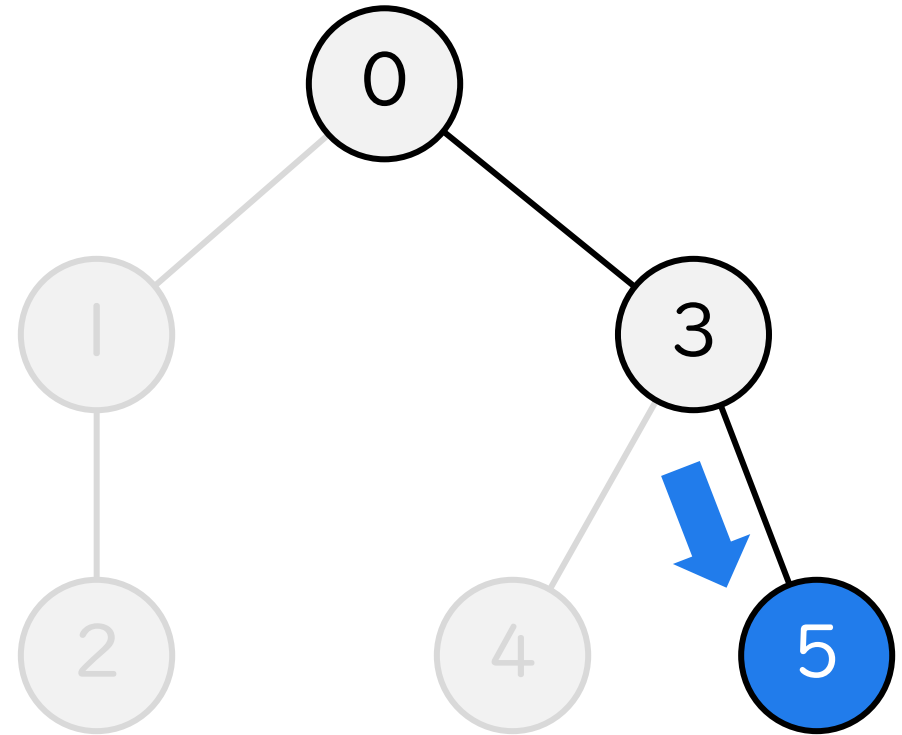
- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列： 0011001

Ali が送る方法

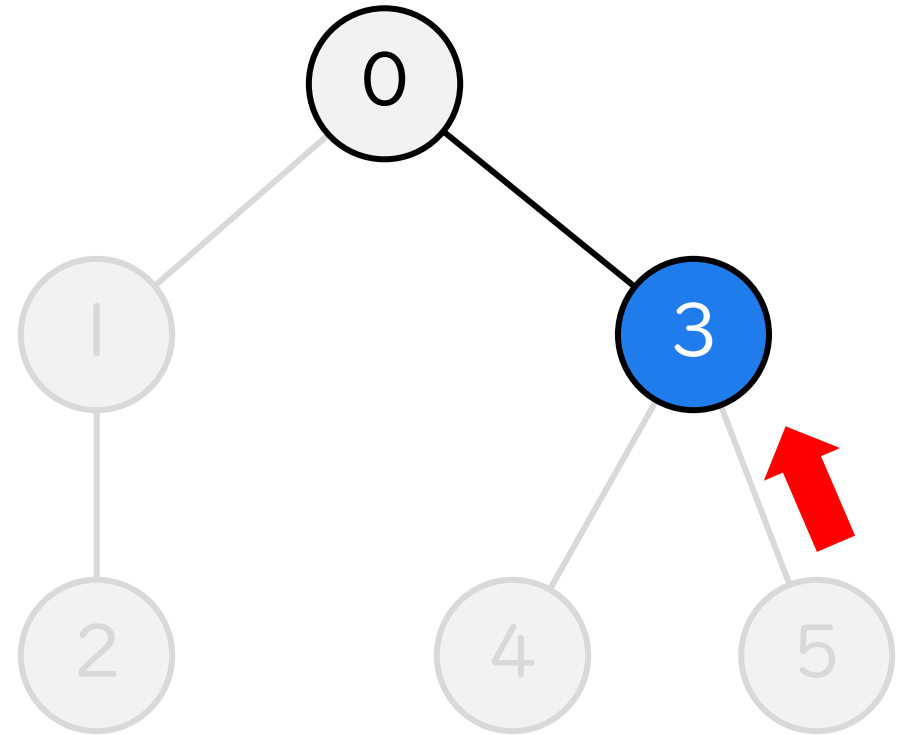
- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列： 00110010

Ali が送る方法

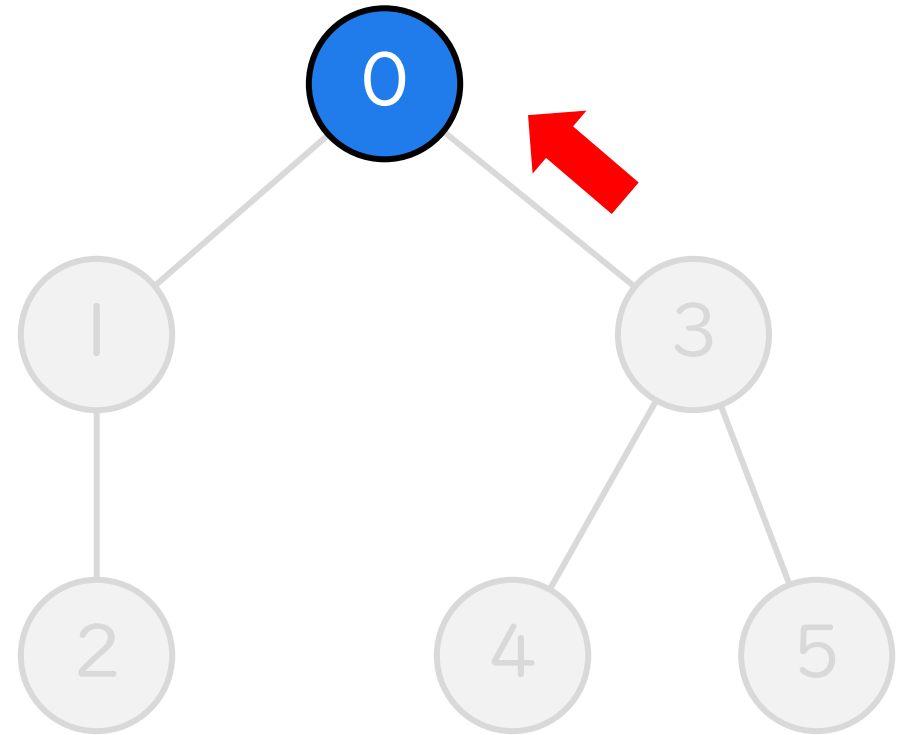
- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列： 001100101

Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



現在の文字列： 0011001011



Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻る時 1 を送る

Benjamin はどうやって
木を復元するか？

現在の文字列：0011001011



Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る



Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一歩戻る

受信した文字列：0011001011

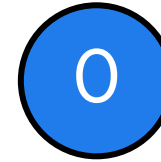


Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

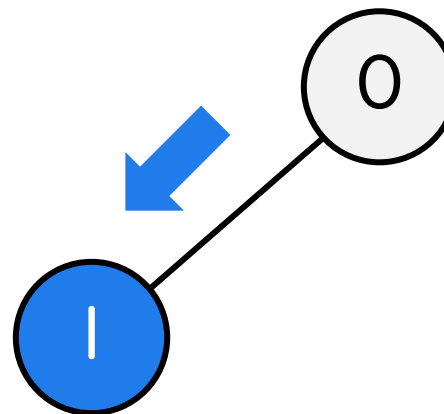
- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一歩戻る



受信した文字列：0011001011

Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加



復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る

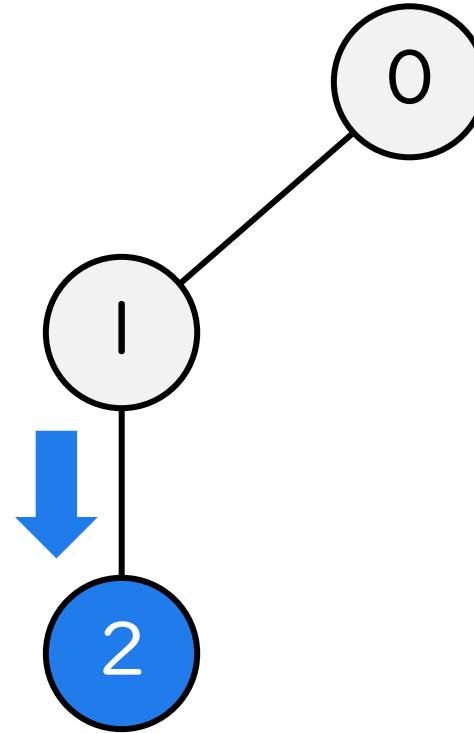
受信した文字列: 0011001011

Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る



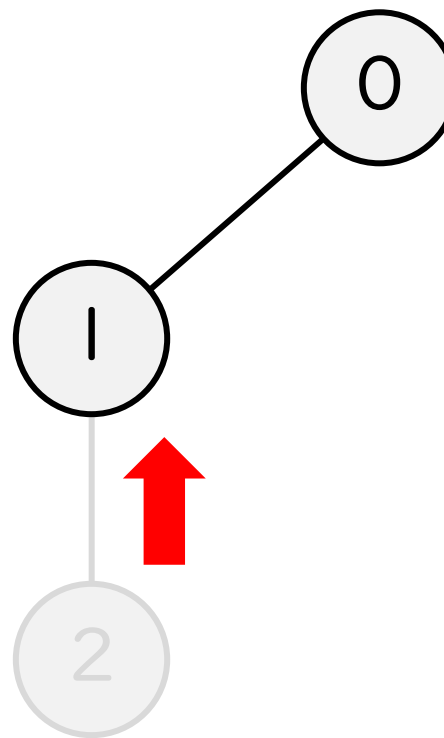
受信した文字列： 00 1 1 00 1 0 1 1

Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る



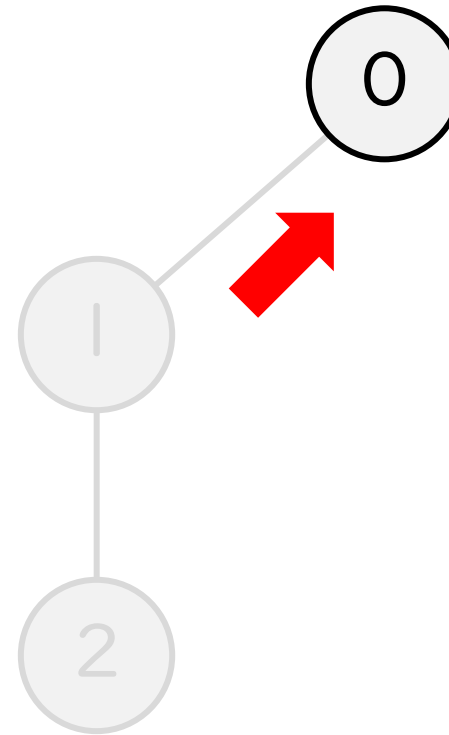
受信した文字列: 0011001011

Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る



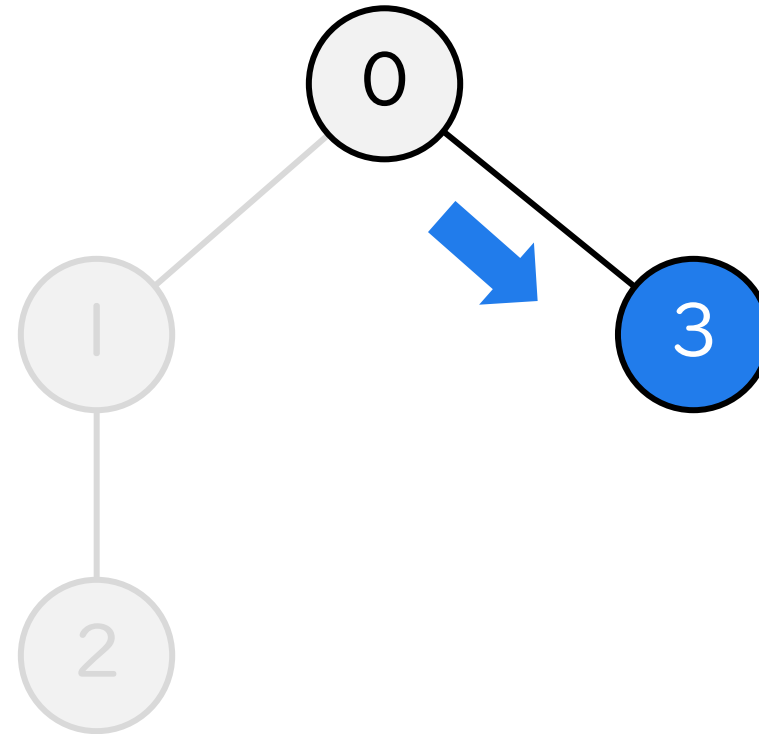
受信した文字列： 0011001011

Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る



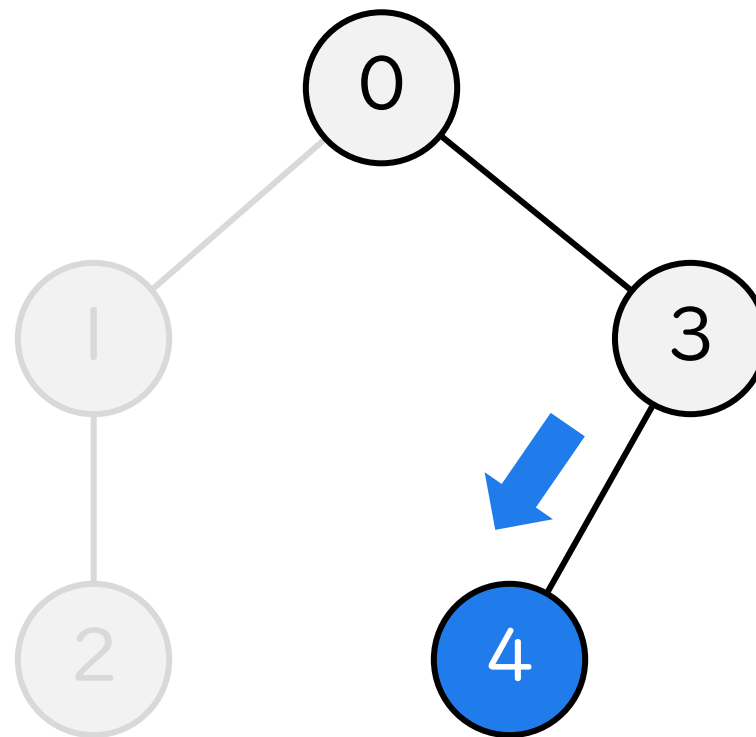
受信した文字列： 0011001011

Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る



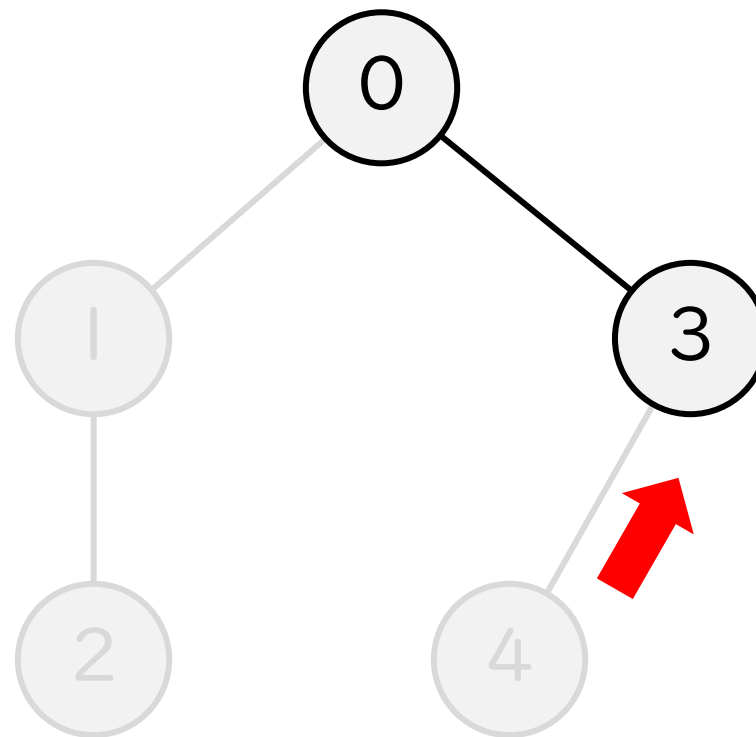
受信した文字列： 0011001011

Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る



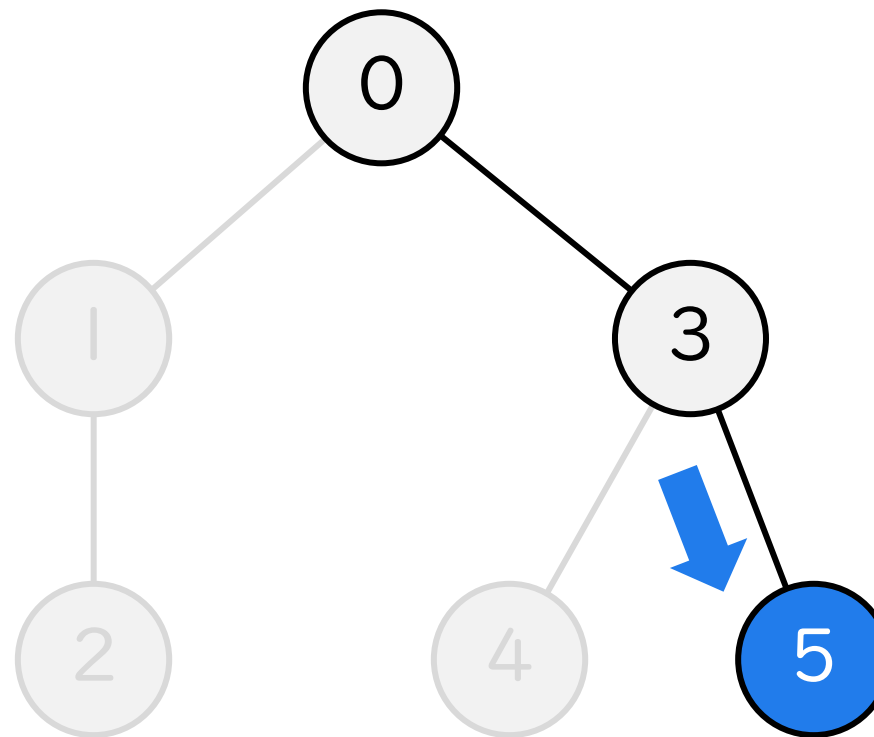
受信した文字列: 0011001011

Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る



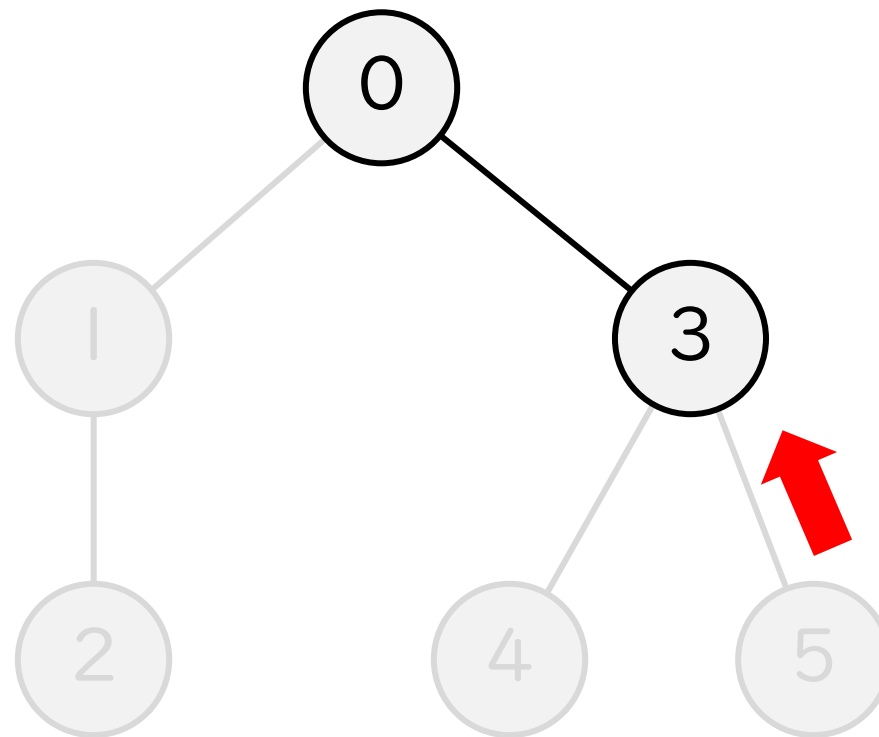
受信した文字列: 0011001011

Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る



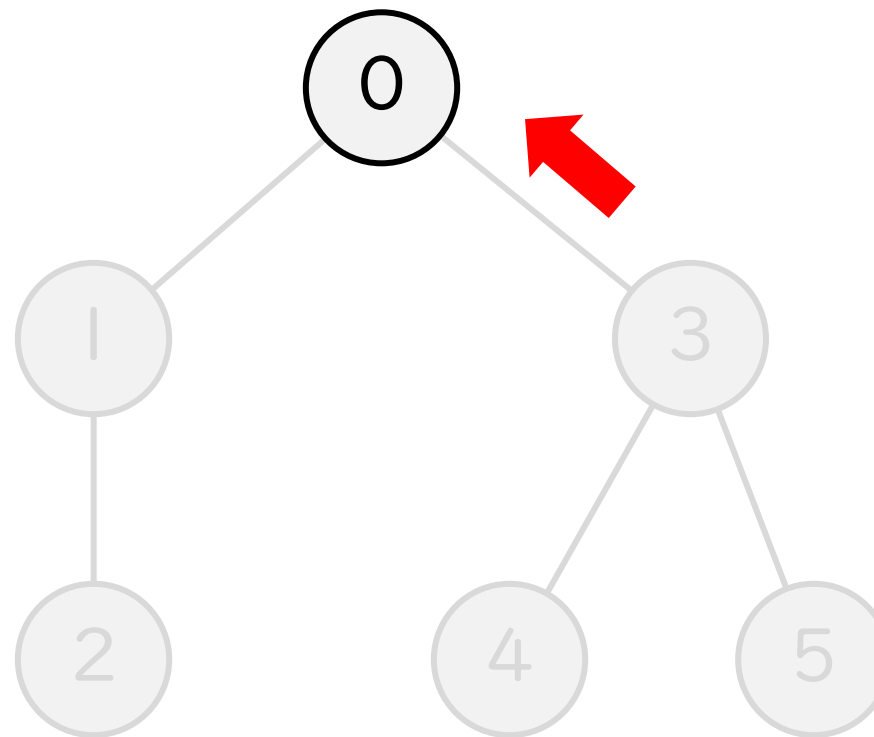
受信した文字列：0011001011

Ali が送る方法

- 適当な根から DFS を行う
- 進むとき 0、戻るとき 1 を追加

復元する方法

- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る



受信した文字列：0011001011



木が正しく復元できた！

復元する方法

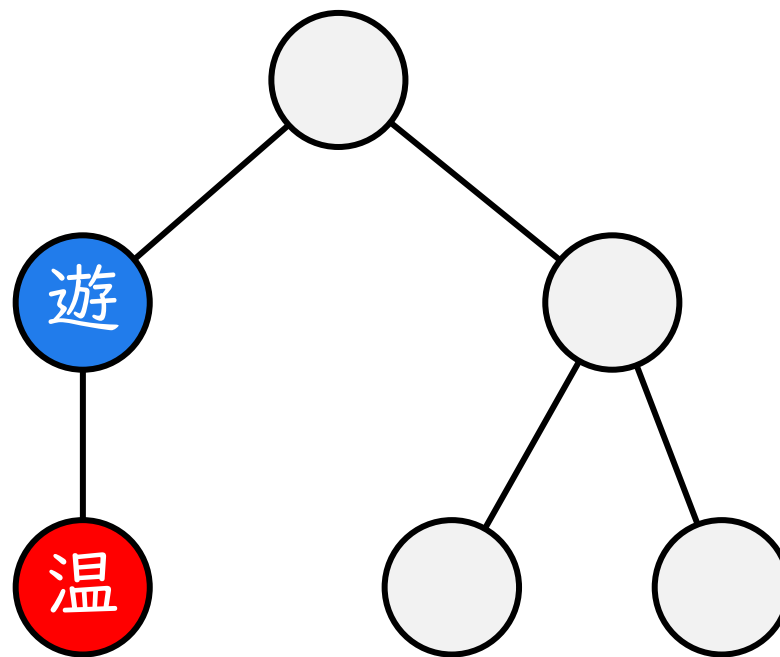
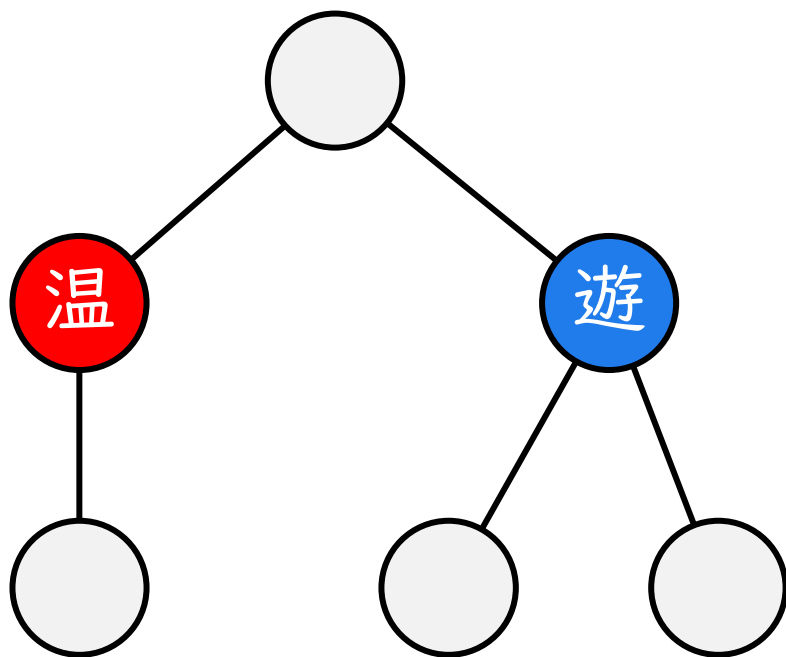
- 文字列を 1 文字ずつ読んで行く
- 0 で頂点追加、1 で一步戻る

受信した文字列： 0011001011



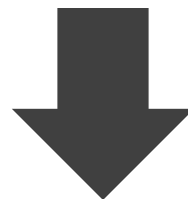
重要な問題

どの頂点が遊園地／温泉かが分からない！



重要な問題

どの頂点が遊園地／温泉かが分からない！



空港の ID を DFS の行きがけ順に
設定しよう！

必要なビット数は？

$$10000_{\text{頂点}} \times 2_{\text{ビット}} = 20000_{\text{ビット}}$$

※この解法を選んだ人は少ないかもしれませんが、解説の後半で役立ちます



必要なビット数は？

ここまでの点数： $10000_{\text{頂点}} \times 2_{\text{ビット}} = 15_{\text{点}}$

※この解法を選んだ人は少ないかもしれませんが、解説の後半で役立ちます

Stage 2

672 ビットまでの解法

Q. Benjamin の 20 ビットを
有効に使えないか？



Benjamin が送る情報の例

頂点番号 X の下 10 ビット

頂点番号 Y の下 10 ビット

Benjamin が送る情報の例

頂点番号 X の下 10 ビット $\blacktriangleright$ 10 通りに絞れる

頂点番号 Y の下 10 ビット $\blacktriangleright$ 10 通りに絞れる

$$10000 \div 2^{10} \doteq 10$$



Benjamin が送る情報の例

$$10000 \div 2^{10} \div 10$$

Benjamin の情報によって
Ali は (X, Y) を 100 通りに絞れる



Ali が送る情報

100 通りの距離を全部、順番に送れば良い

Ali が送る情報

100 通りの距離を全部、順番に送れば良い

送るべき個数

1 個当たり

100 個

×

14 ビット

=

1400 ビット

距離は 10000 まで

$2^{14} = 16384$ より小さい

2



Ali はどうするか？

66

222

Ali が送る情報

100 通りの距離を全部、順番に送れば良い

送るべき個数

1 個当たり

ここまでの点数：**21** 点

距離は 10000 まで

$2^{14} = 16384$ より小さい



改善方法

頂点番号 X, Y をまとめて送ろう

改善方法

頂点番号 X, Y をまとめて送ろう

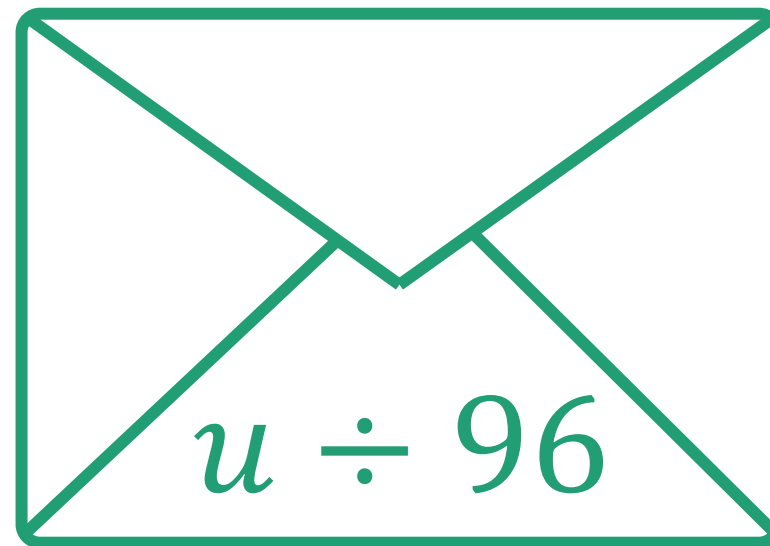
		Y の値			
		0	1	2	3
X の値	0	0	1	2	3
	1	4	5	6	7
	2	8	9	10	11
	3	12	13	14	15

各 (X, Y) に対して番号 u を定める

改善方法

頂点番号 X, Y をまとめて送ろう

		Y の値			
		0	1	2	3
X の値	0	0	1	2	3
	1	4	5	6	7
	2	8	9	10	11
	3	12	13	14	15

各 (X, Y) に対して番号 u を定める

番号を 96 で割った値を送る

そうすると、Ali は **96** 通りしか送る必要が無くなる



×14

最大 **1344** ビット



そうすると、Ali は 96 通りしか送る必要がなくなる

ここまでの点数： 22×4 点

最大 1344 ビット

遊園地 X と温泉 Y の順序は

関係ない

$X < Y$ を仮定すると？

番号の最大値が 1 億 $\rightarrow$ 5000 万に！

		Y の値			
		0	1	2	3
X の値	0	0	1	2	3
	1	4	5	6	7
	2	8	9	10	11
	3	12	13	14	15

$X < Y$ を仮定しない



		Y の値			
		0	1	2	3
X の値	0		0	1	2
	1			3	4
	2				5
	3				

$X < Y$ を仮定する

そうすると、Ali は **48** 通りしか送る必要が無くなる



最大 **672** ビット



そうすると、Ali は 48 通りしか送る必要がなくなる

ここまでの点数： $\times 4$ 32 点

最大 672 ビット

※ここまでの方針とオイラーツアーを組み合わせると 50 点台まで狙えますが

満点解法とは関係ないので解説では詳しく扱いません

※なぜ 48 か？ → 番号の最大値は 5000 万であり、48 で割ればギリギリ 2^{20} に収まるため

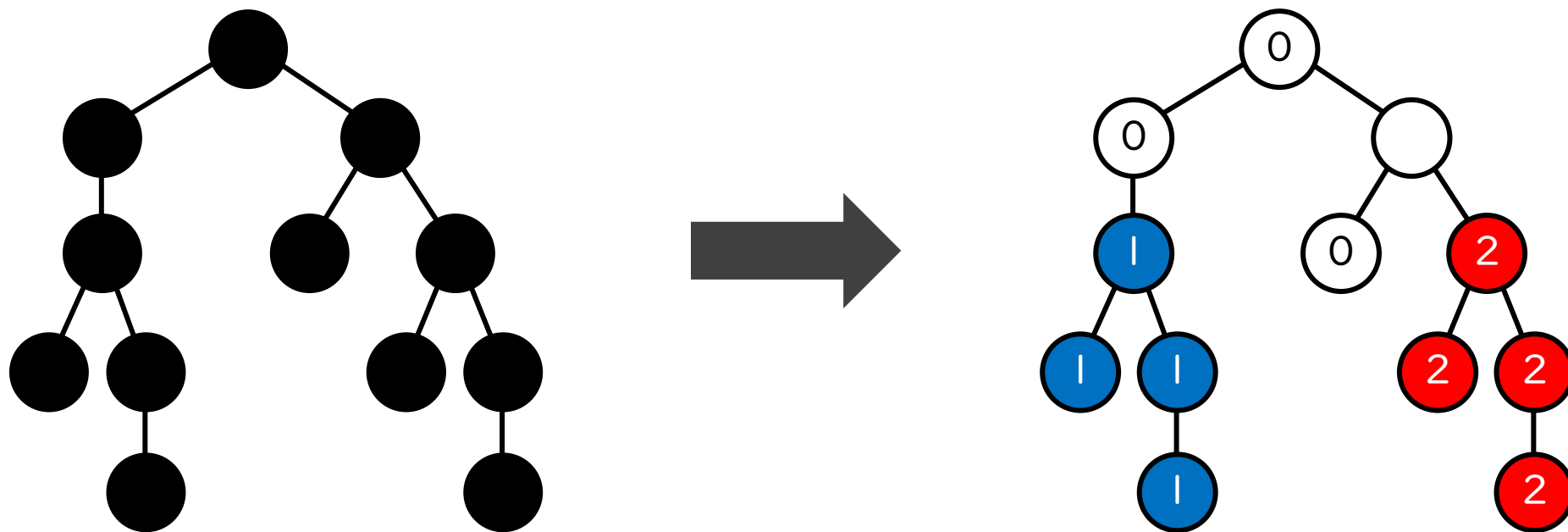
Stage 3

118 ビットまでの解法

本質的な考察

頂点数の差が 2 倍以内になるように
木を連結なグループに分解することができる

※この問題の制約の場合。また、1つのグループを除く。



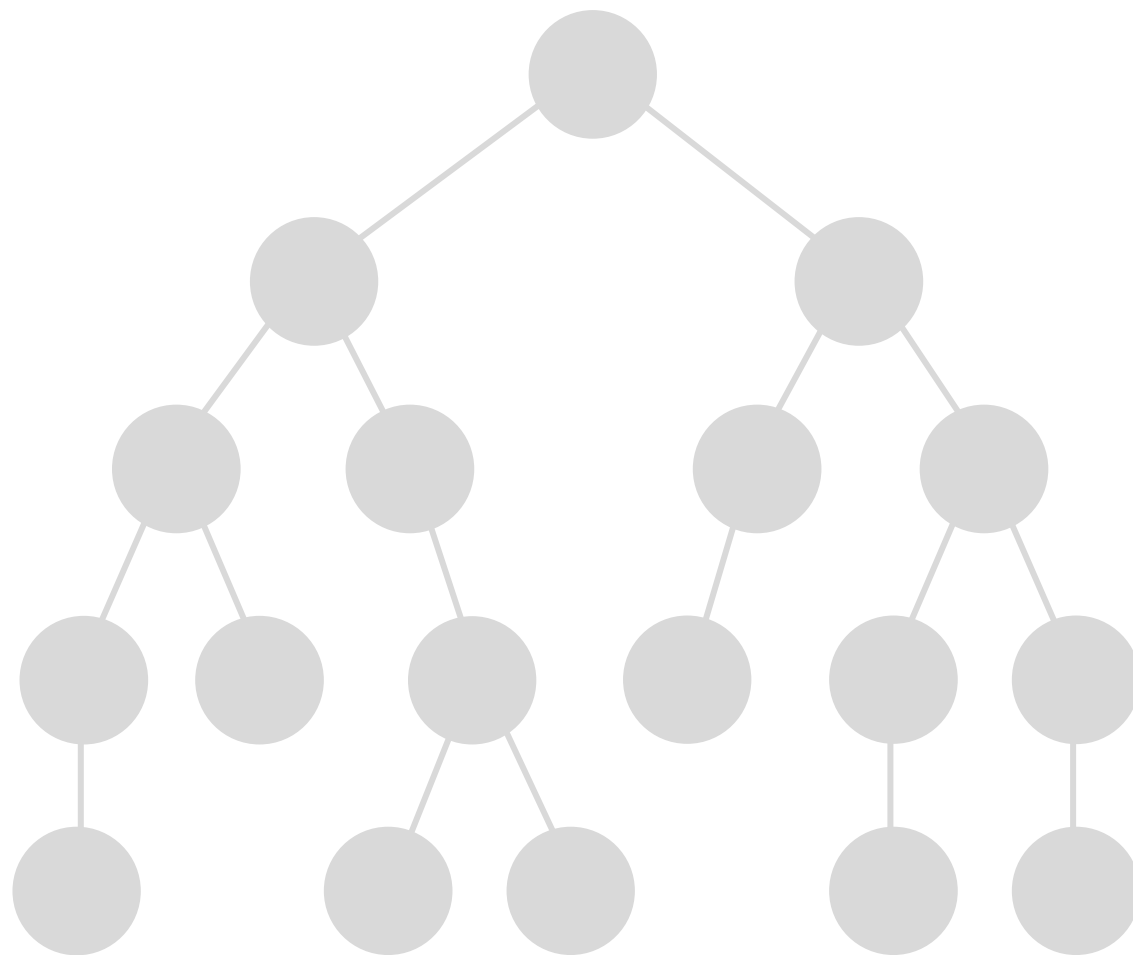


木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る

木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

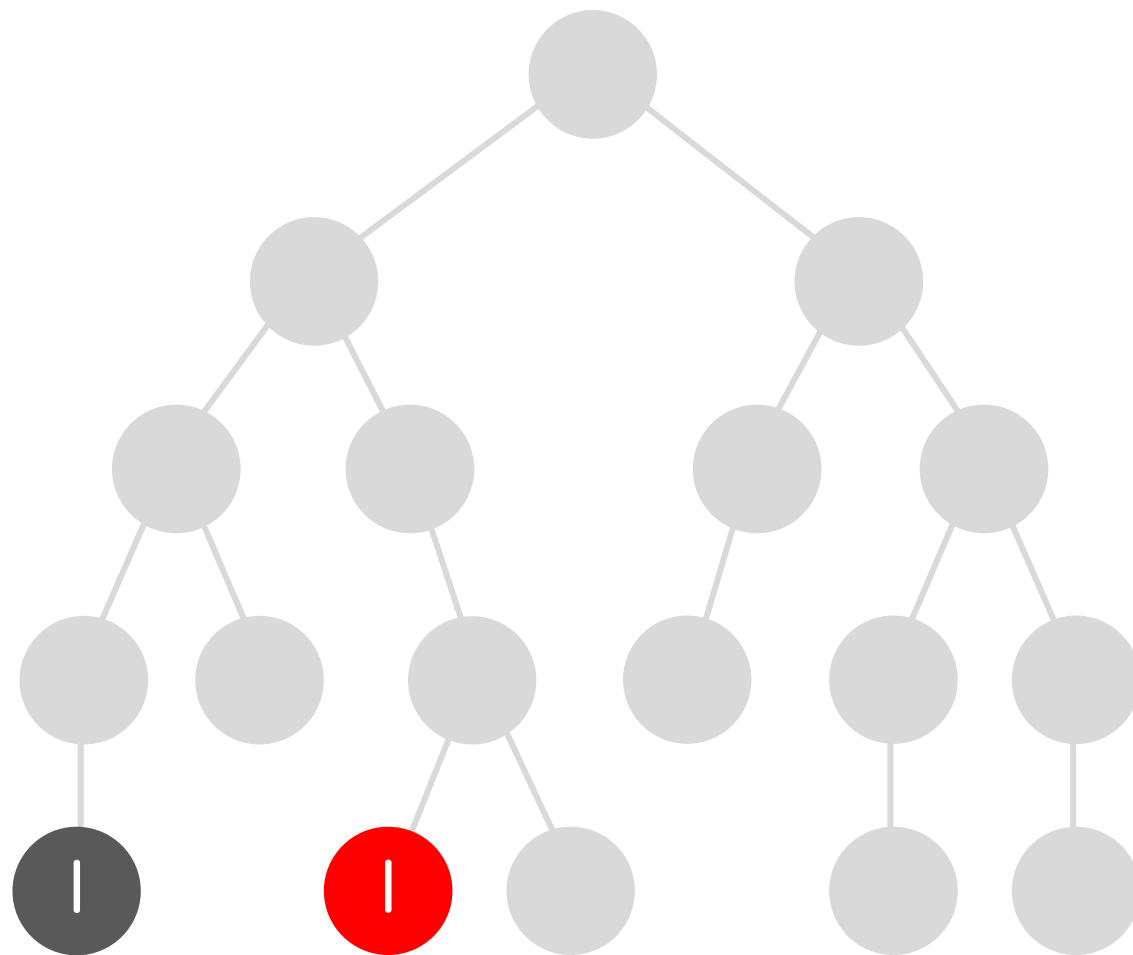


- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

木の分解方法

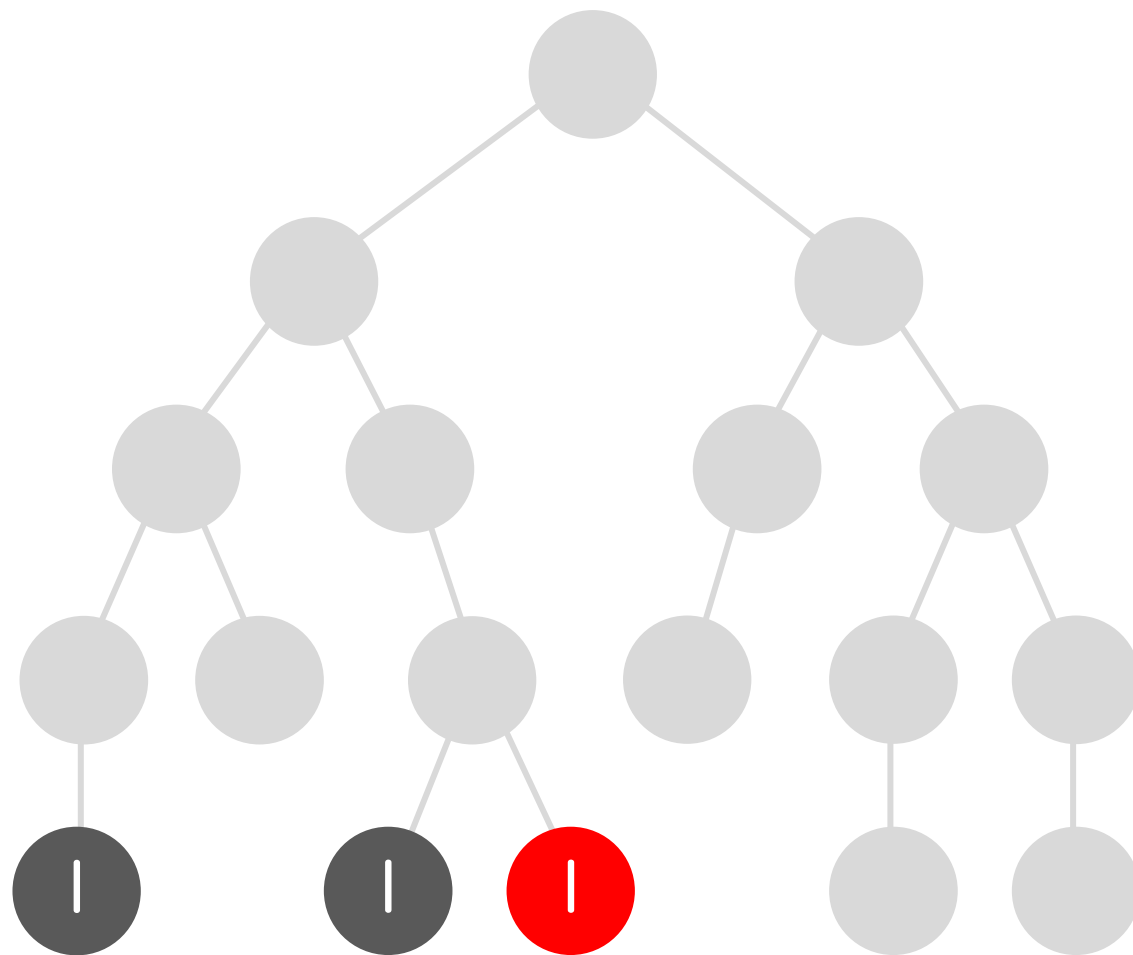
- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

※頂点の数字は
現在のグループのサイズ



- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

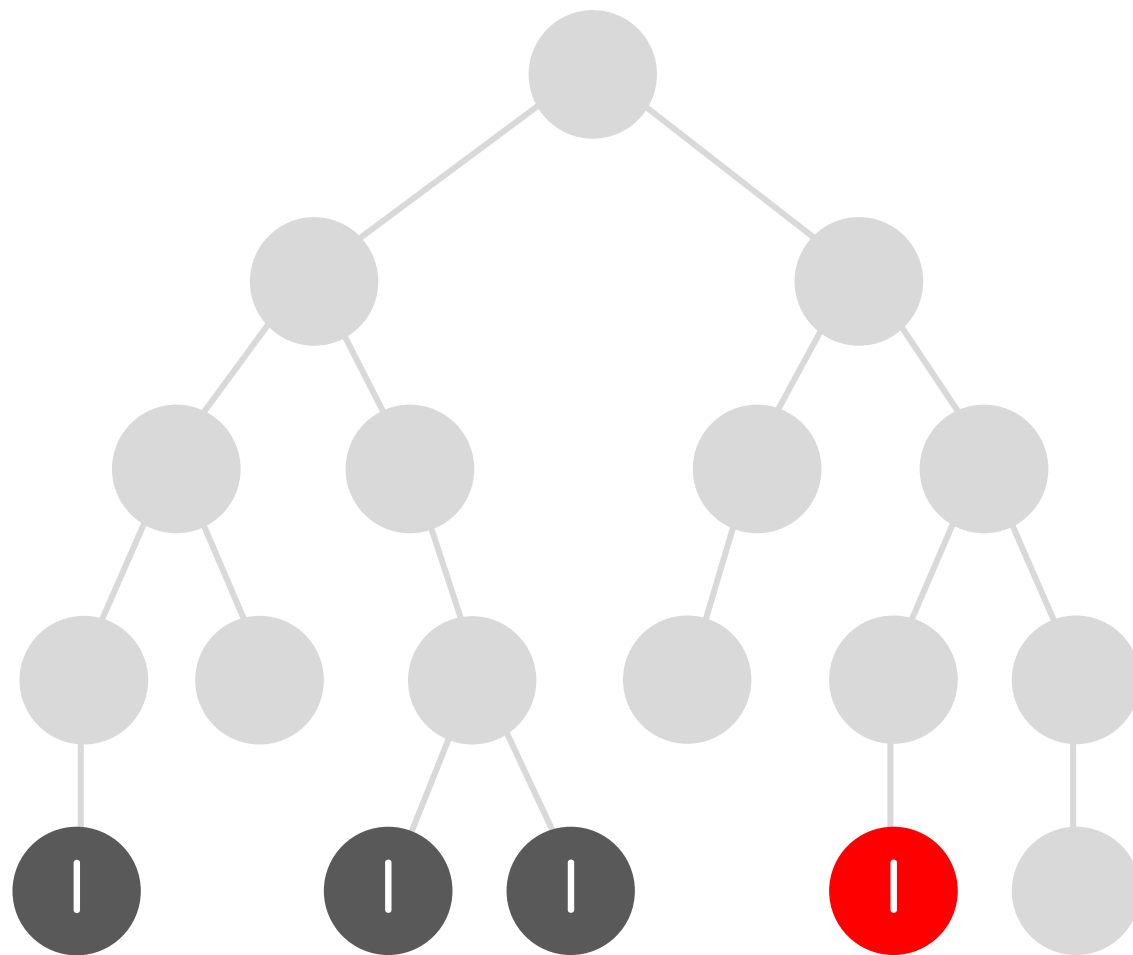
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

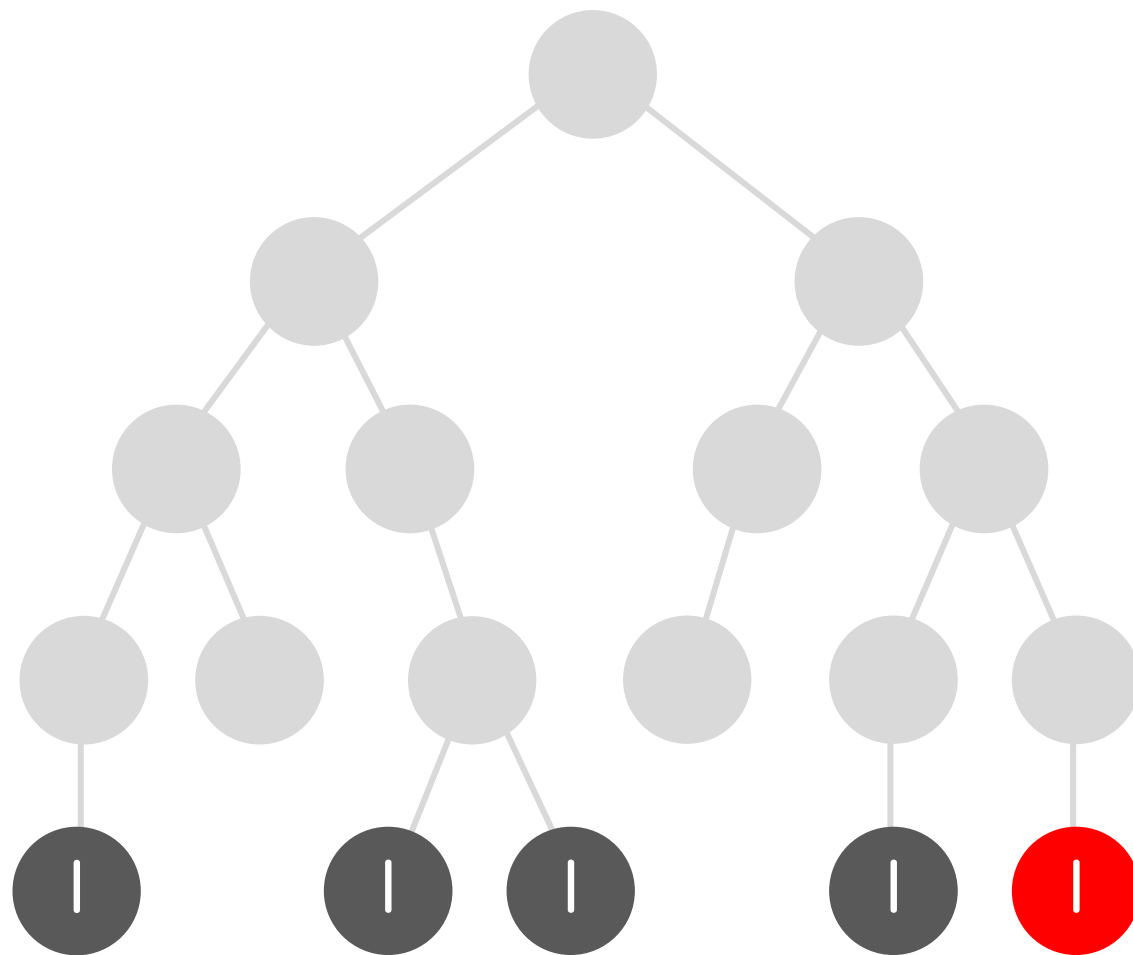
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

※頂点の数字は
現在のグループのサイズ

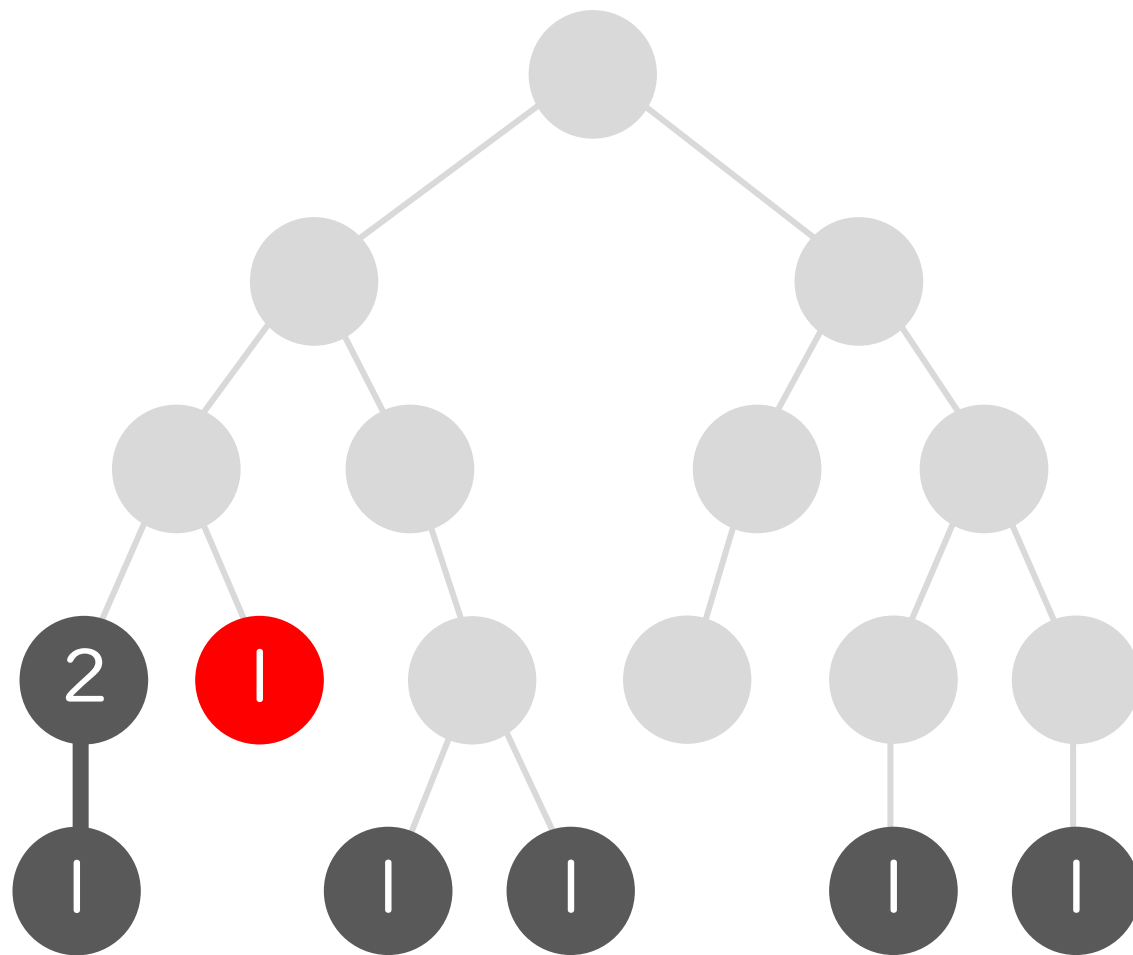


- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

※頂点の数字は
現在のグループのサイズ

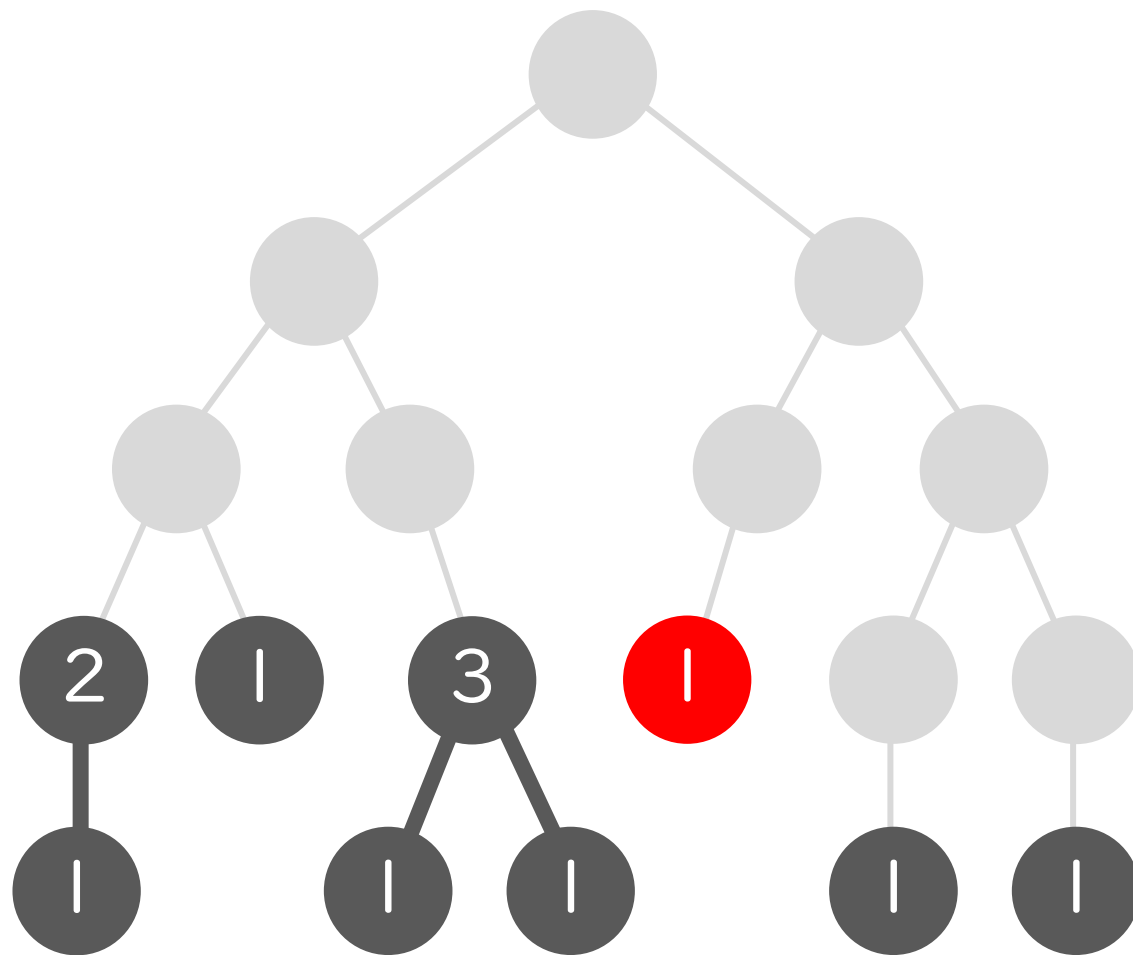


- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

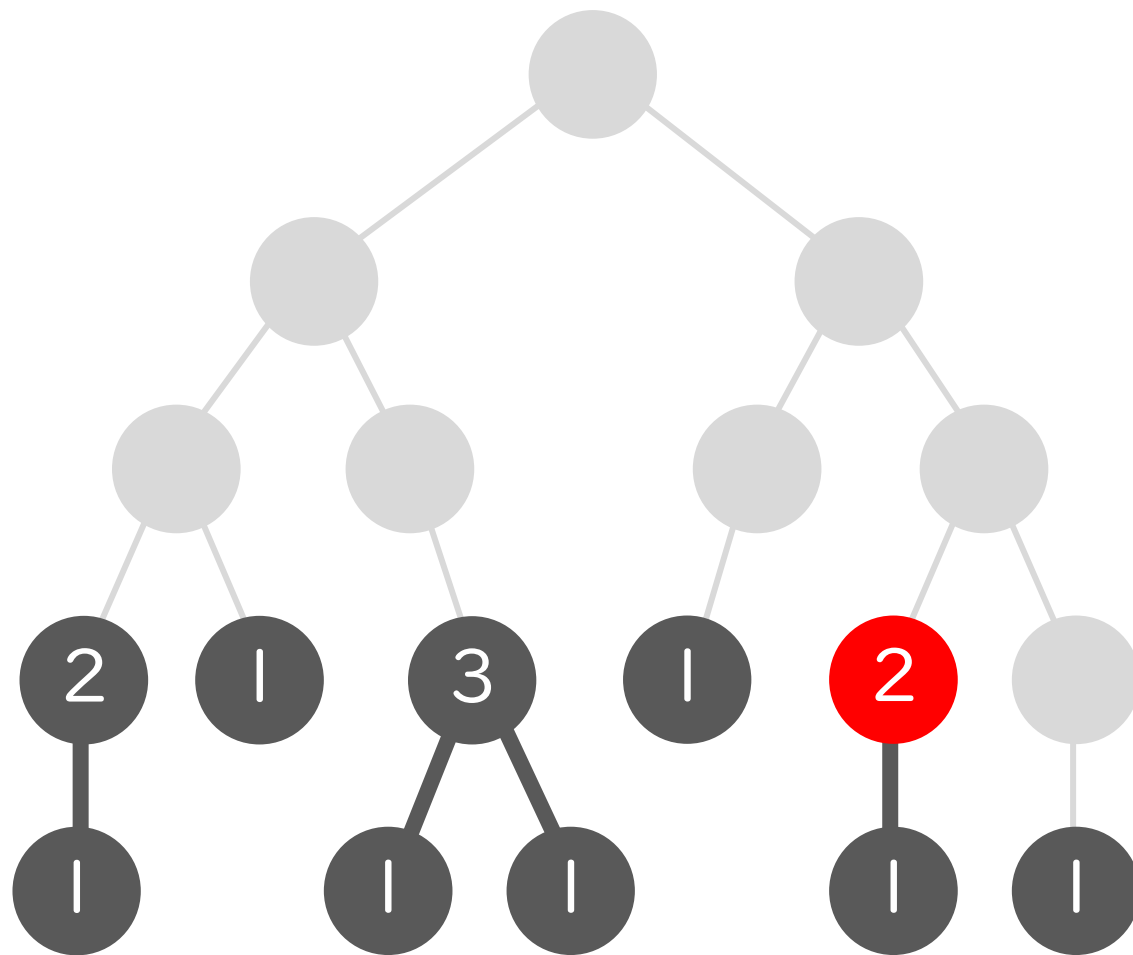
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

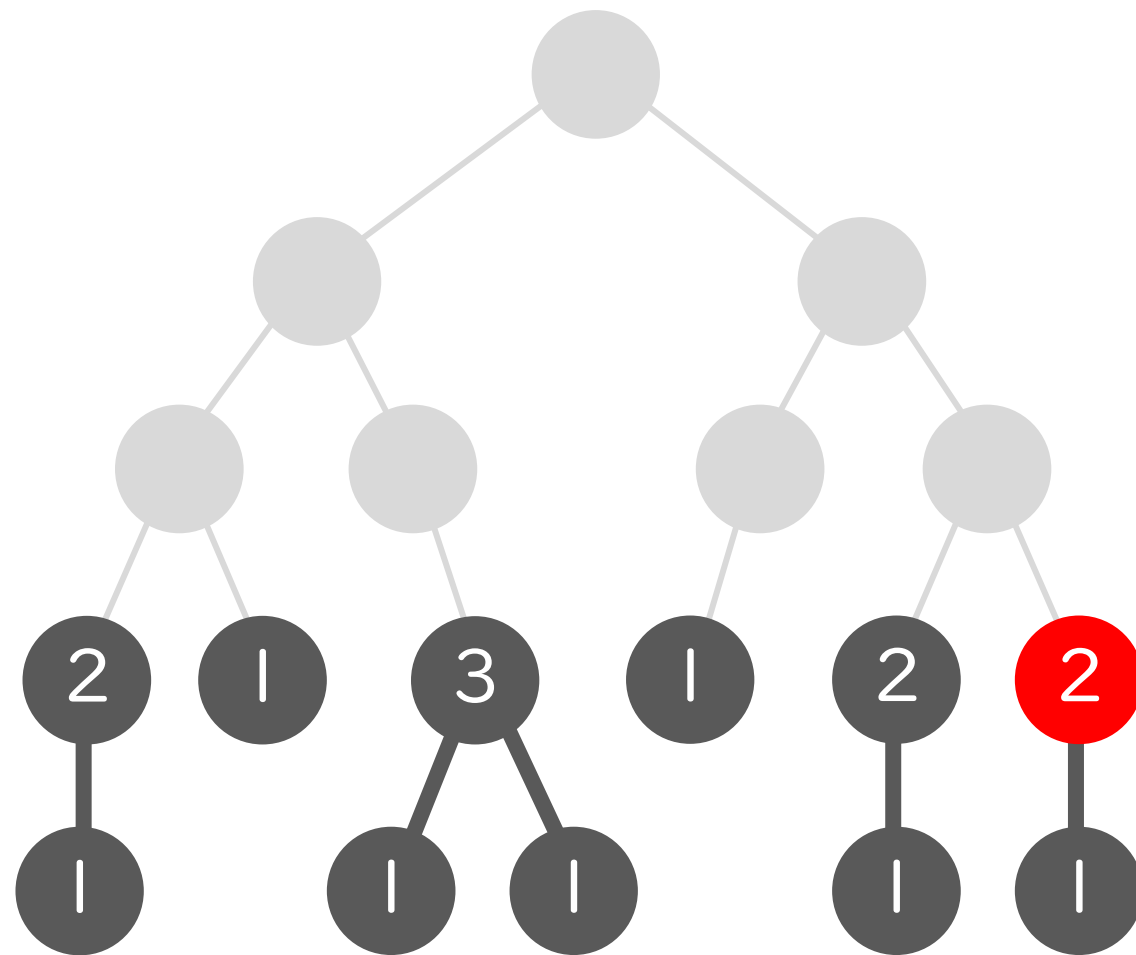
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

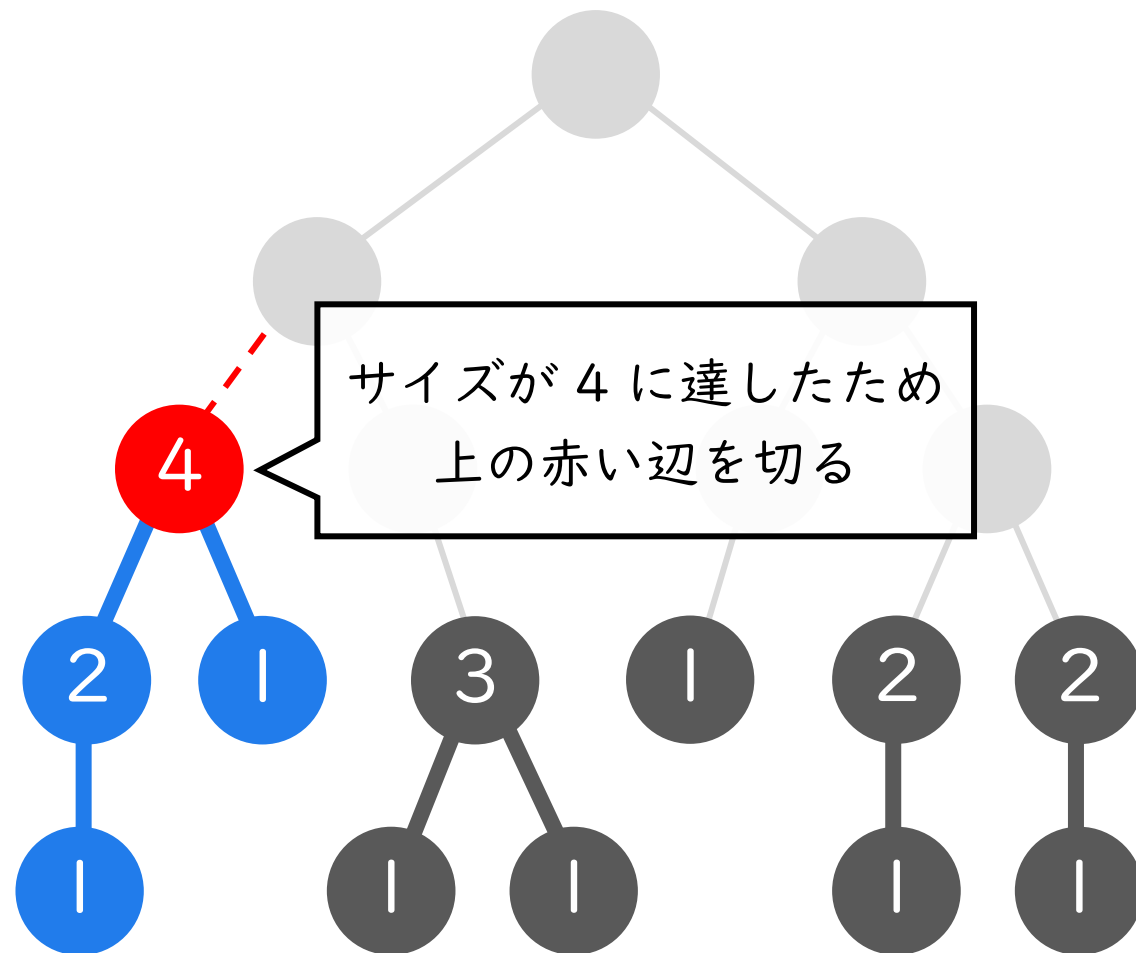
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

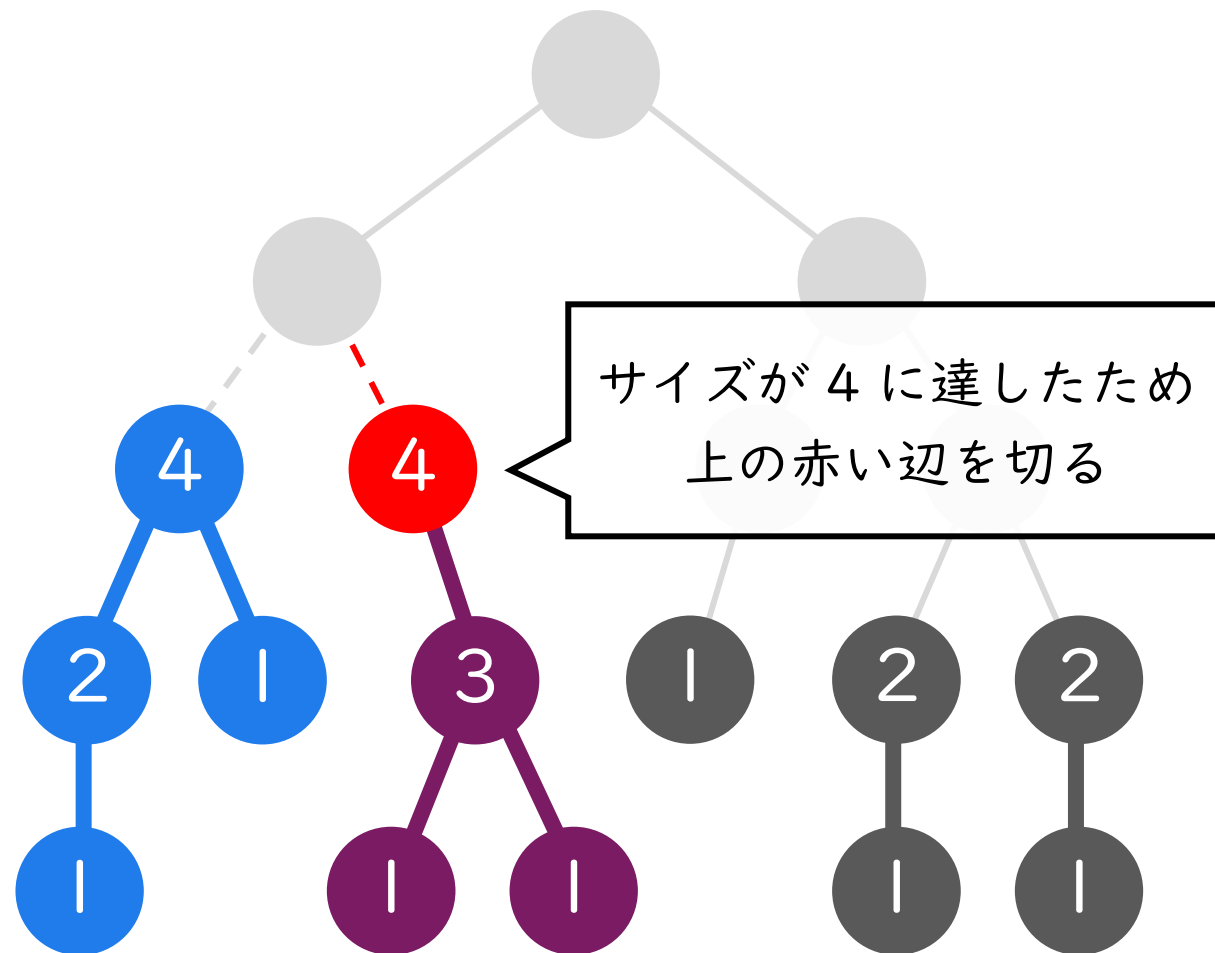
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

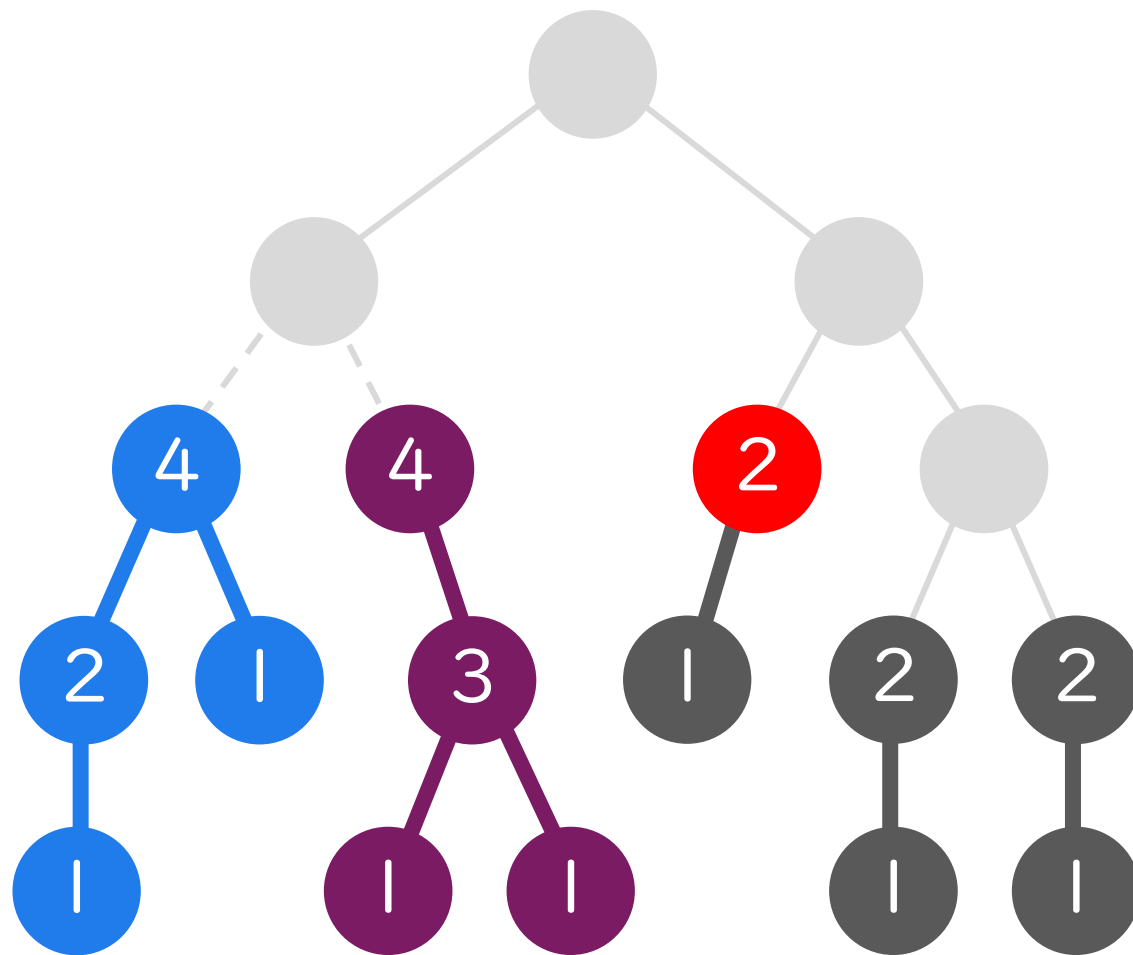
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

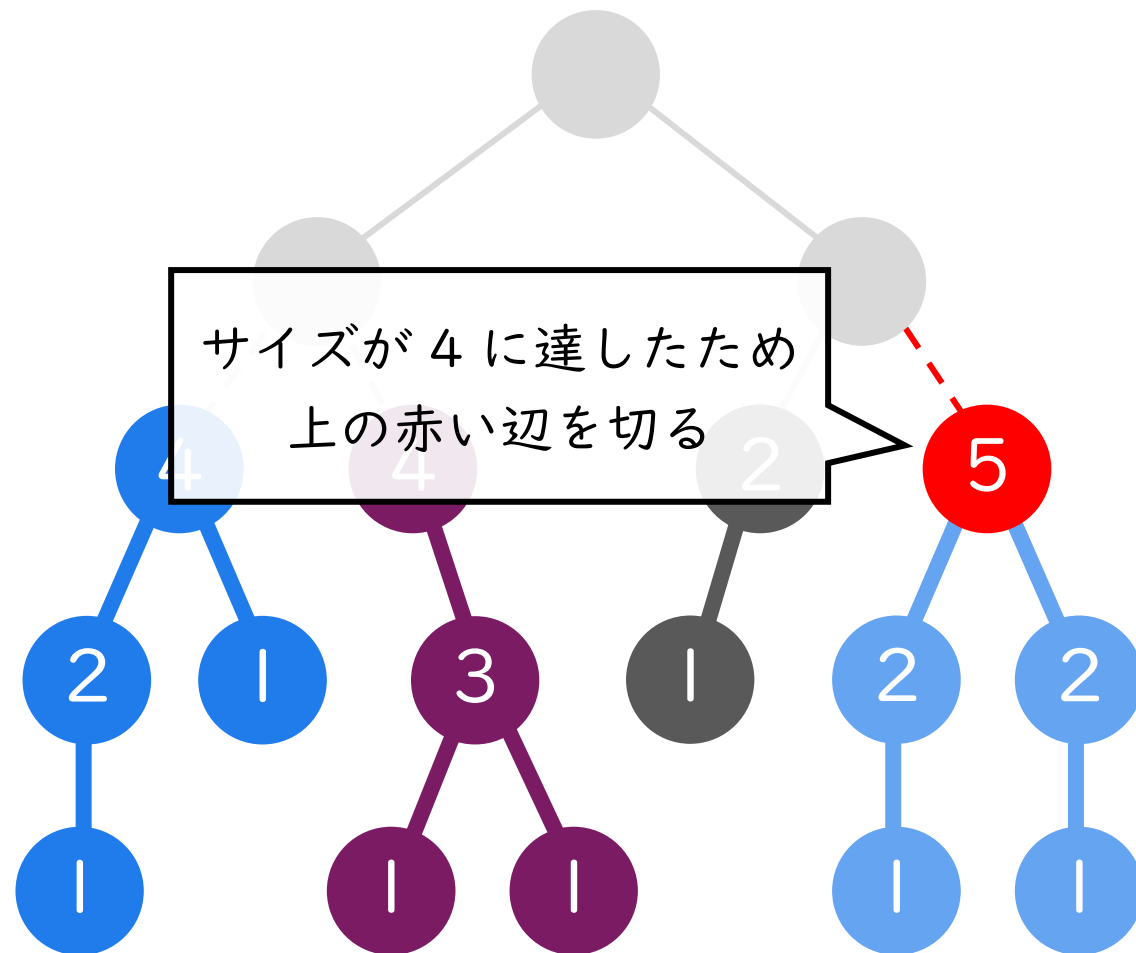
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

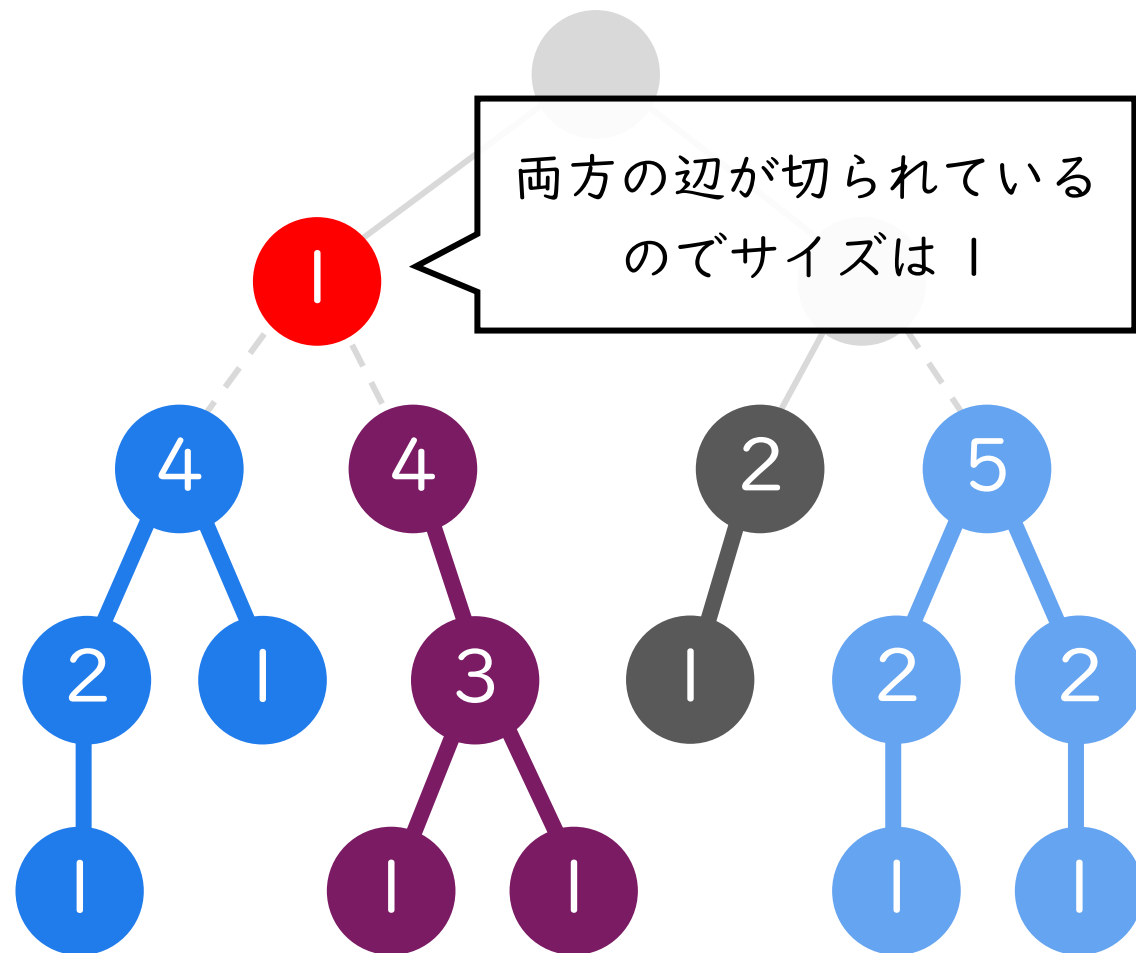
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

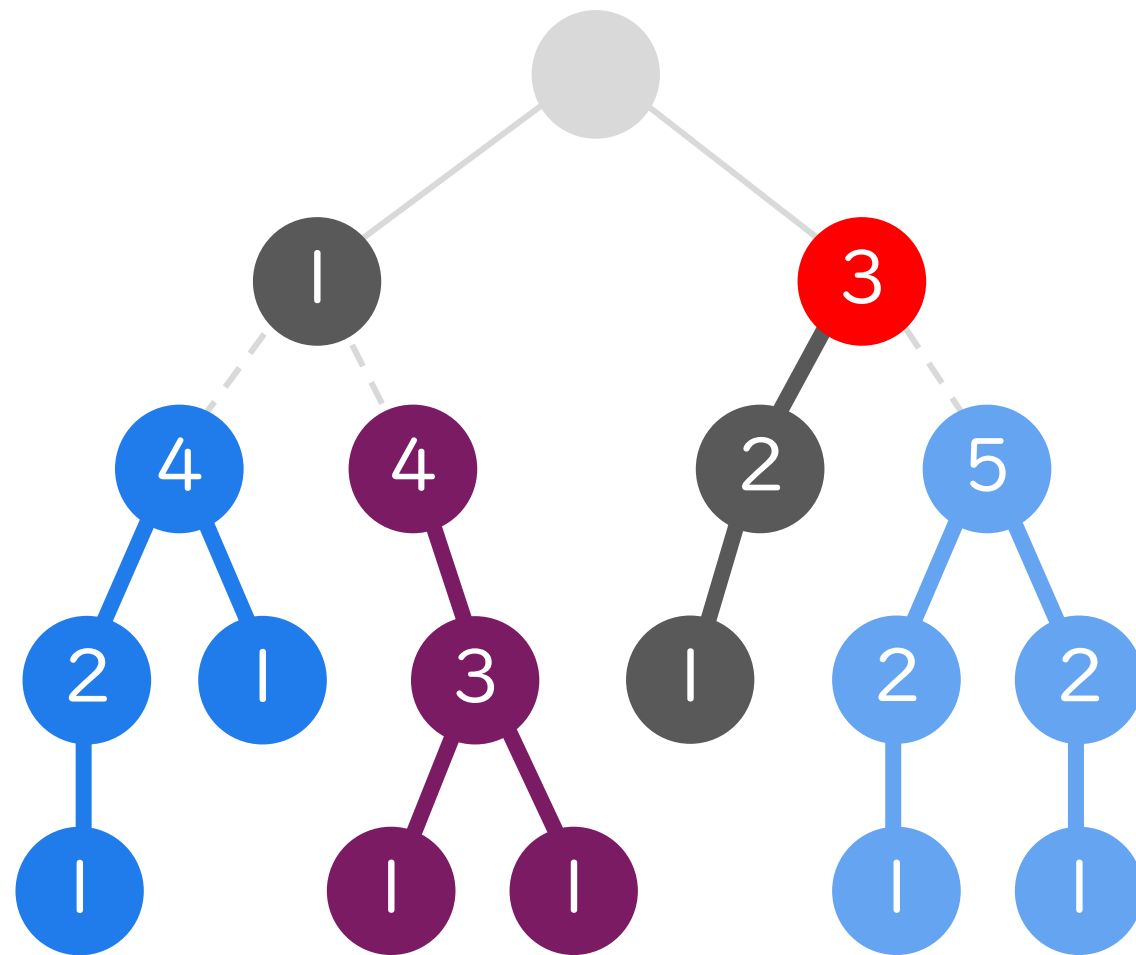
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

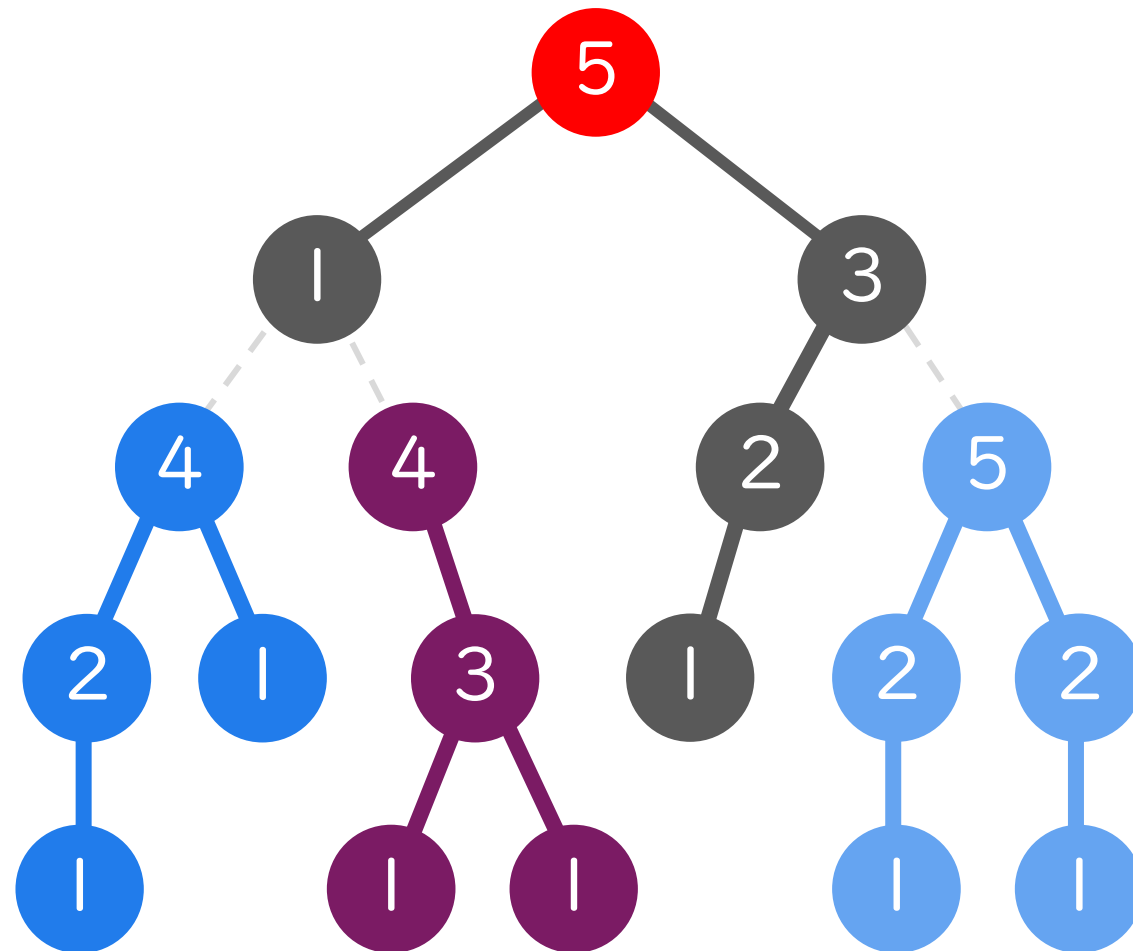
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

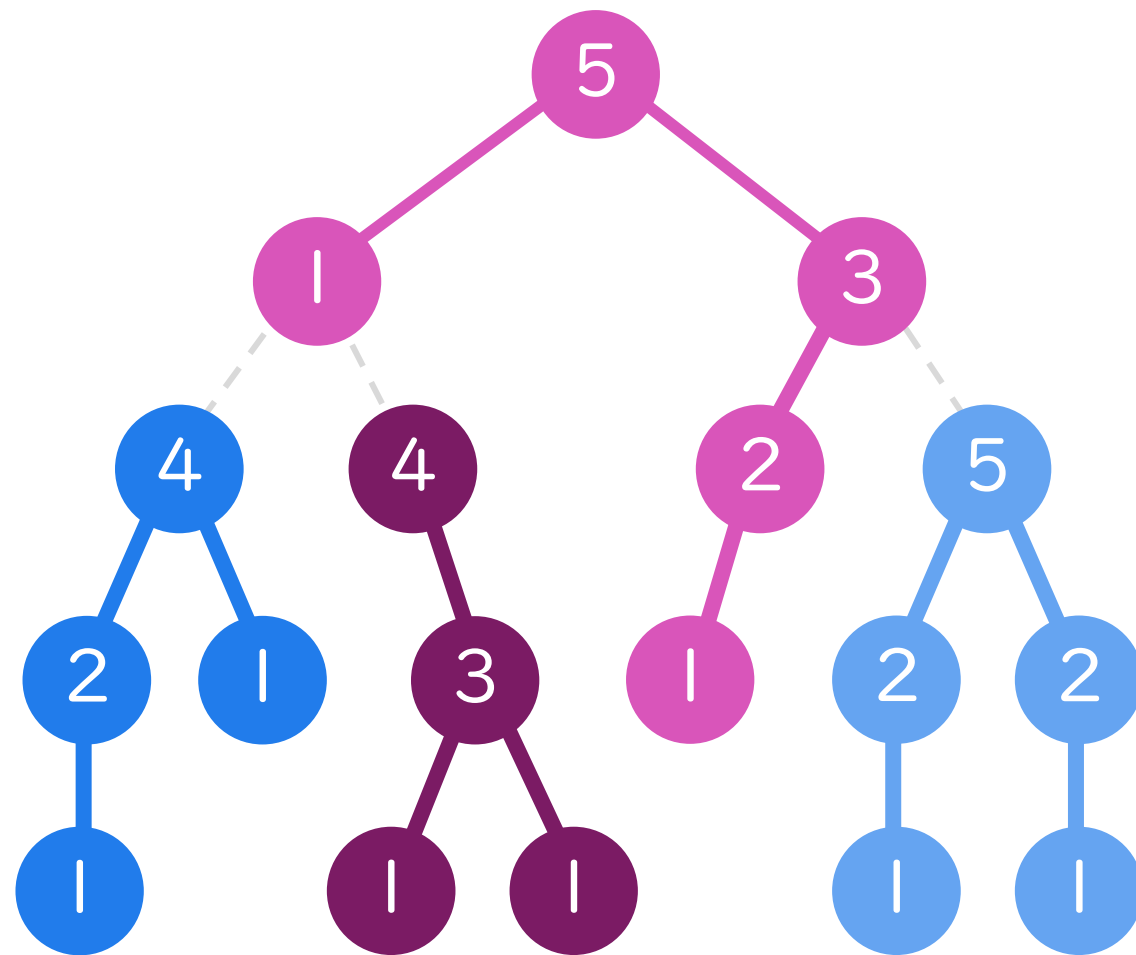
※頂点の数字は
現在のグループのサイズ



木の分解方法

- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

※頂点の数字は
現在のグループのサイズ



木の分解方法

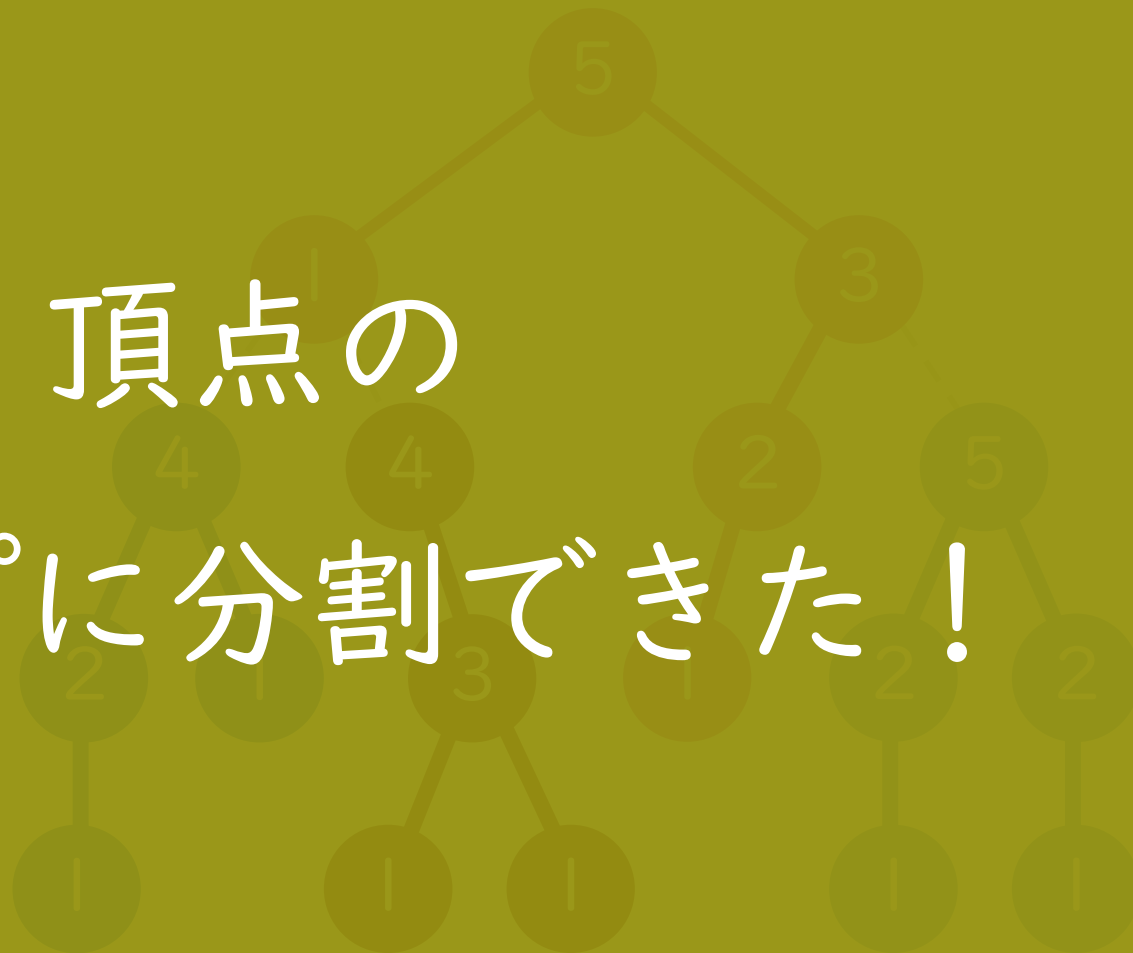
- 下から貪欲に分解していく
- グループのサイズが定数 B を超えた場合、辺を切る
- 右図は $B = 4$ の例

最大 5 頂点の

4 つのグループに分割できた！

※頂点の数字は

現在のグループのサイズ



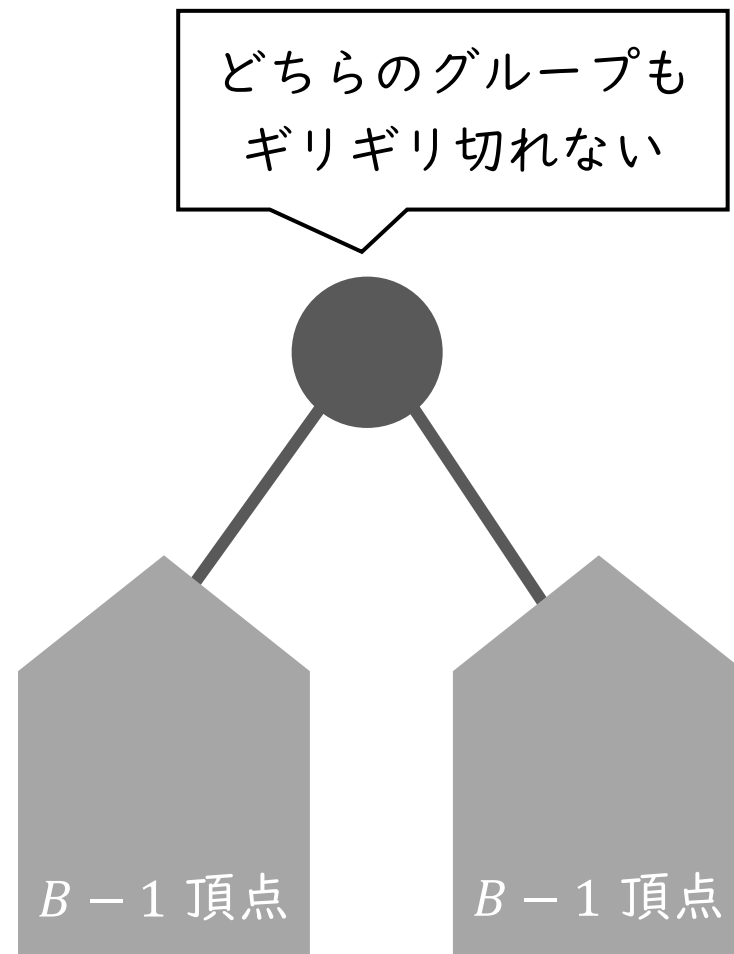
Q. この方法を使うと
最も大きいグループの頂点数はどうか？

次数は 3 以下なので...

根付き木の出次数は 2 以下

最悪ケースは右図の通り

最大 $2B-1$ 頂点



次数は 3 以下なので...

どちらのグループも
ギリギリ切れない

最大サイズ $2B - 1$ 以下の

高々 $\left\lceil \frac{N}{B} \right\rceil$ 個のグループに分割可能

最大 $2B - 1$ 頂点

$B - 1$ 頂点

$B - 1$ 頂点

※根を含むグループ以外は、頂点数 B 以上 $2B - 1$ 以下

たとえば最小サイズ $B=7$ で木をグループ分けしたとする



どのような戦略で通信するか？



ステップ 1

空港の ID の設定

空港の ID の設定

$14 \times (\text{グループ番号})$
 $+ (\text{グループ内頂点番号})$

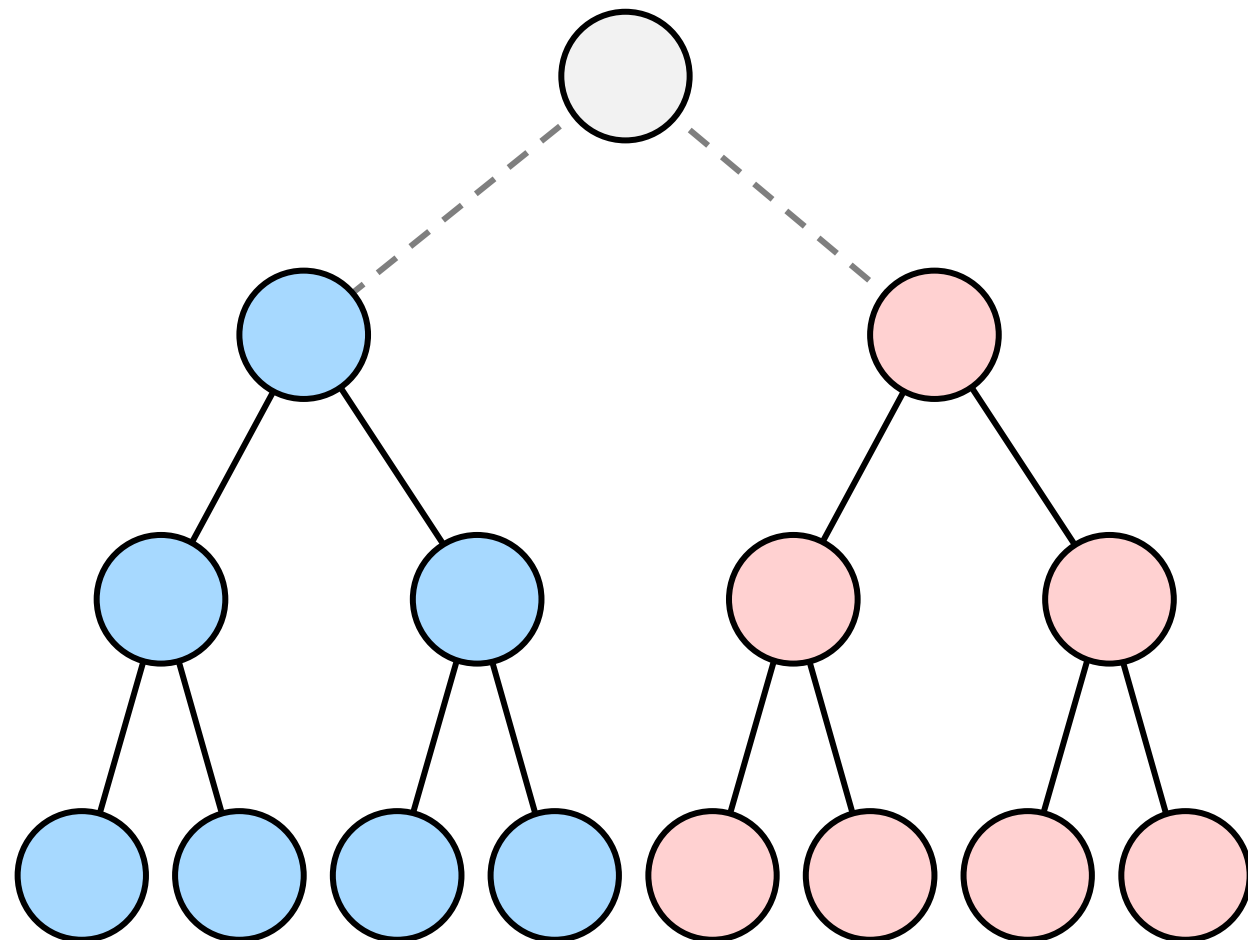
※グループ内頂点番号は適当に割り振る

1 グループの
頂点数は 13 以下

空港の ID の設定

$14 \times (\text{グループ番号})$
 $+ (\text{グループ内頂点番号})$

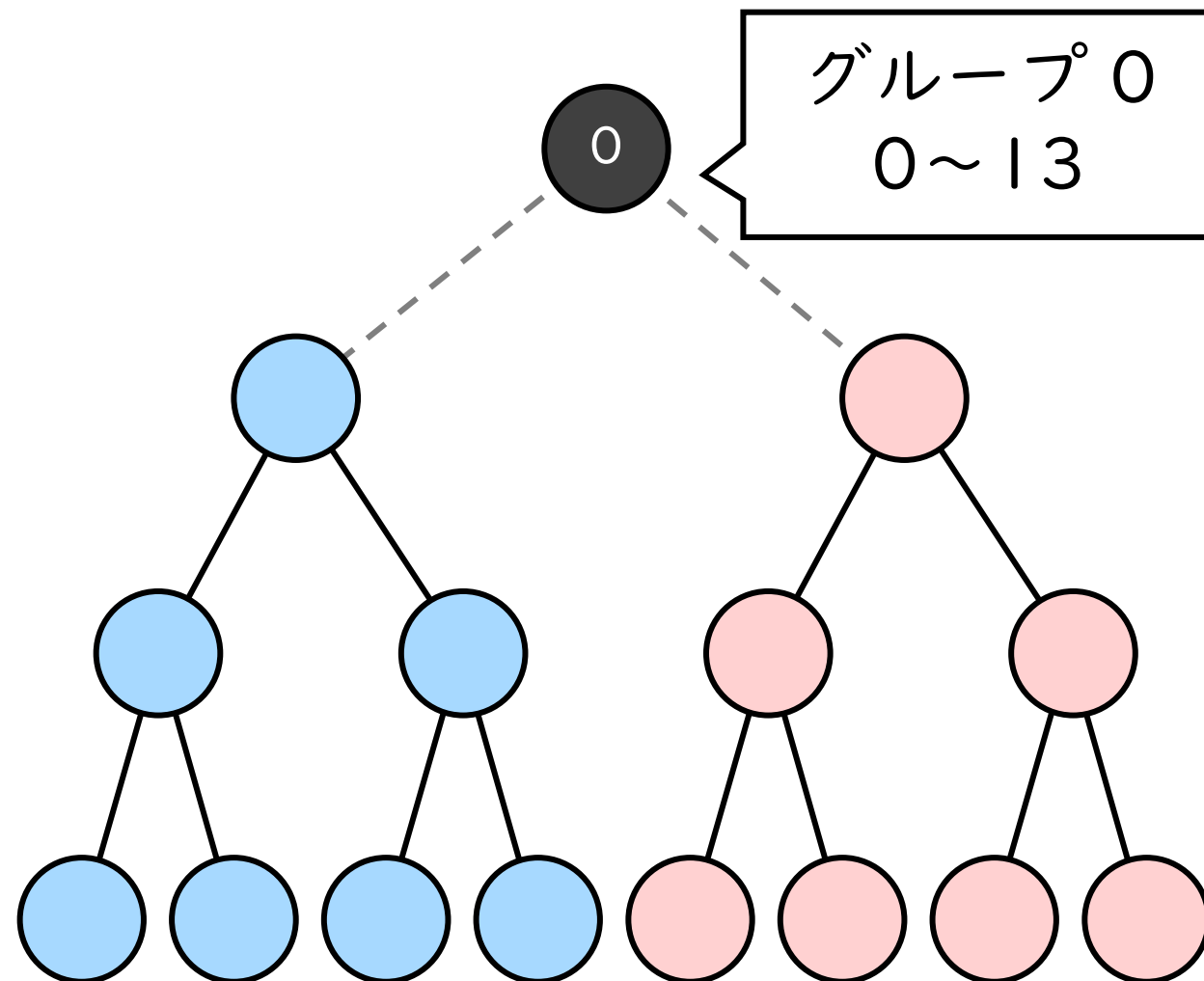
※グループ内頂点番号は適当に割り振る



空港の ID の設定

$14 \times (\text{グループ番号})$
 $+ (\text{グループ内頂点番号})$

※グループ内頂点番号は適当に割り振る

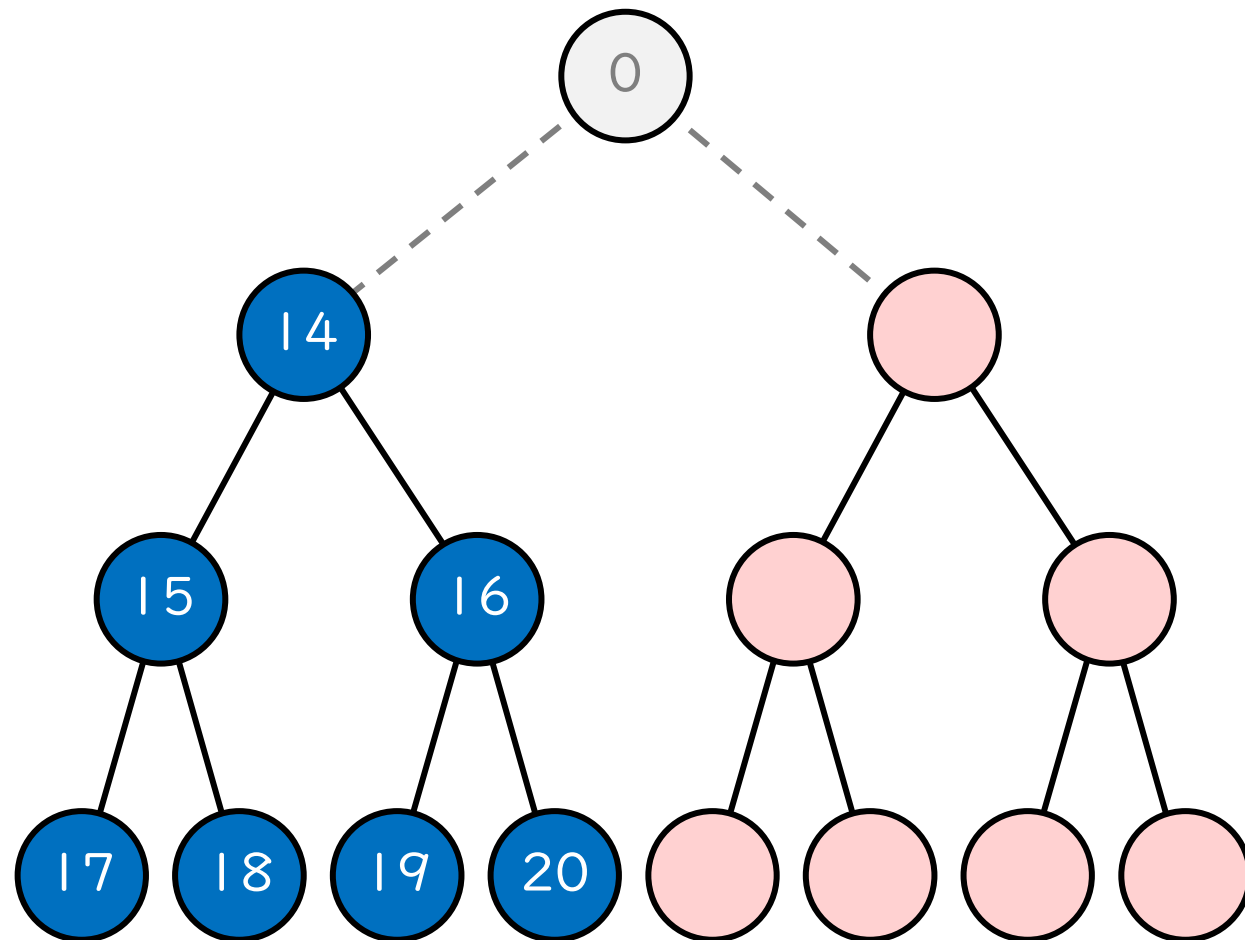


空港の ID の設定

$14 \times (\text{グループ番号})$
 $+ (\text{グループ内頂点番号})$

※グループ内頂点番号は適当に割り振る

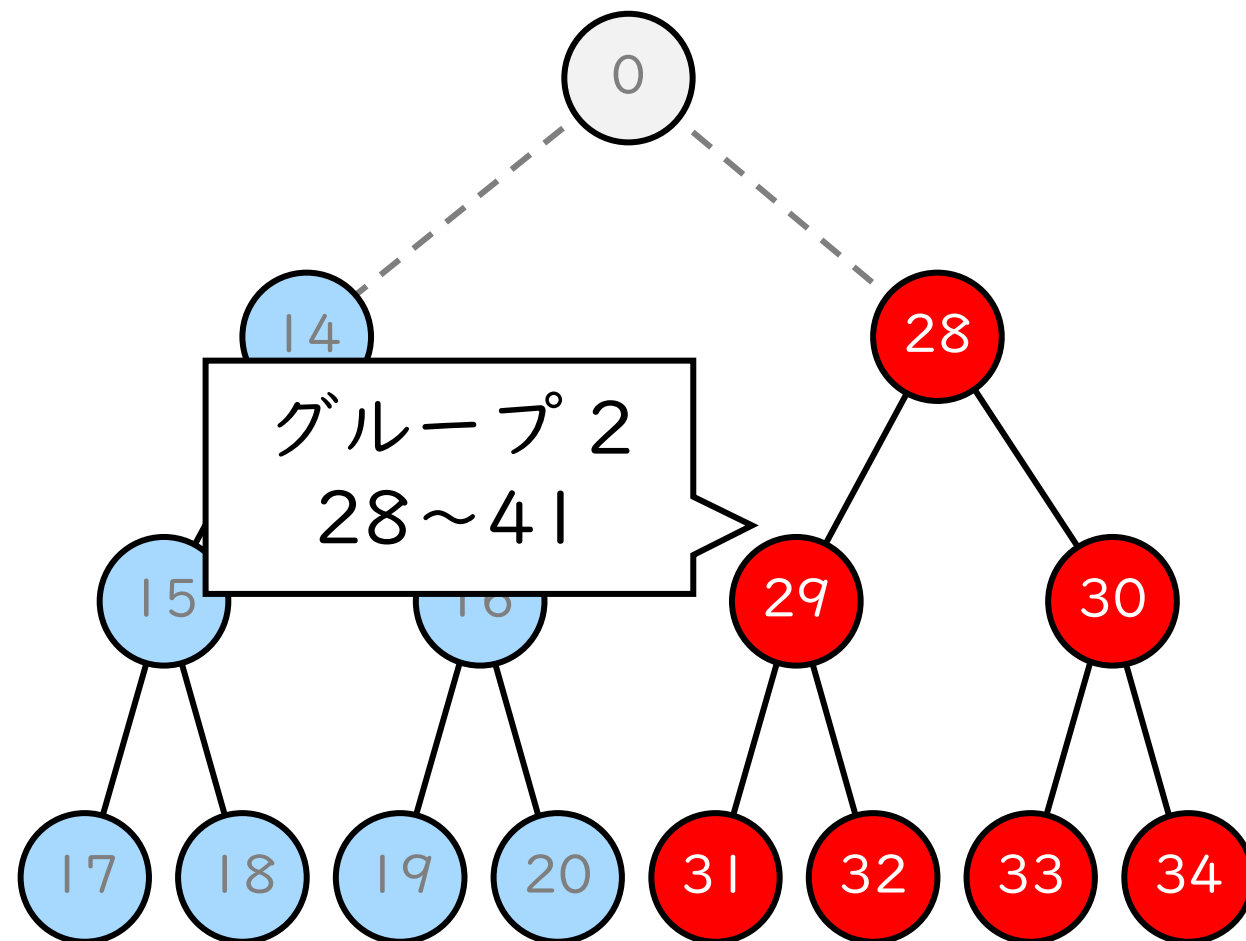
グループ 1
14 ~ 27



空港の ID の設定

$14 \times (\text{グループ番号})$
 $+ (\text{グループ内頂点番号})$

※グループ内頂点番号は適当に割り振る





ステップ 2

Ali → Benjamin の通信



Benjamin が送る情報

遊園地のグループ番号 G_X

温泉のグループ番号 G_Y

3 59 点解法：ステップ 2

112
222

Benjamin が送る情報

遊園地のグループ番号 G_X

温泉のグループ番号 G_Y

グループの数は最大で

$$10000 \div 7 = 1429 \text{ 個}$$

識別すべきパターン数は

やばい、20 ビットで
送れない！

$$1429^2 > 2^{20}$$

3 59 点解法：ステップ 2

113
222

Benjamin が送る情報

遊園地のグループ番号 G_X

温泉のグループ番号 G_Y

グループの数は最大で

$$10000 \div 7 = 1429 \text{ 個}$$

識別すべきパターン数は

$$\cancel{1429^2} > \cancel{2^{20}}$$

$G_X \leq G_Y$ を仮定しても良い
(復習：32 点解法)

$$1429^2 \div 2 < 2^{20}$$



ステップ 3

Ali → Benjamin の通信



重要な性質

グループ間を行き来するために

絶対通るパスが存在

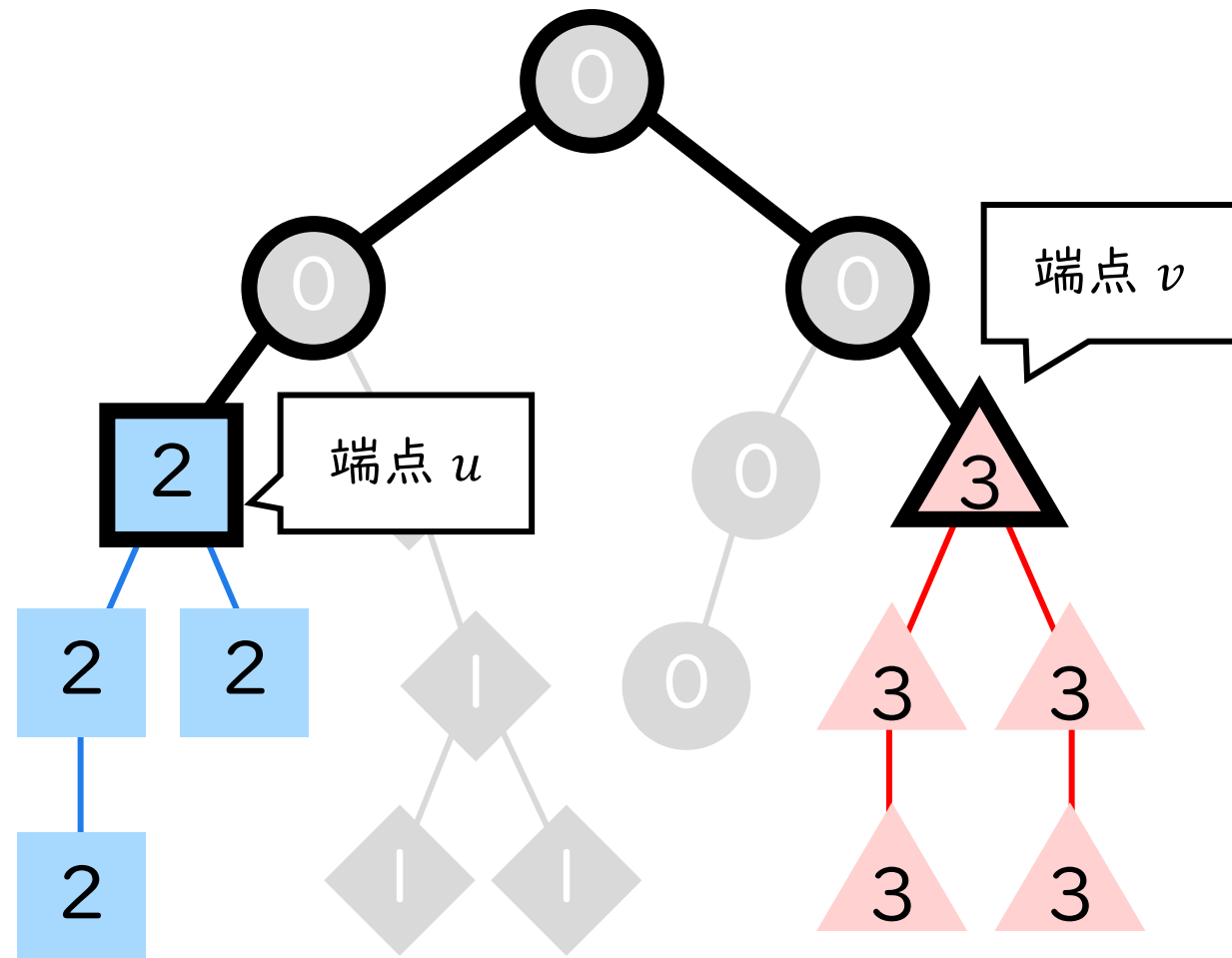
重要な性質

グループ間を行き来するために

絶対通るパスが存在

※遊園地（青色）側の端点を u

温泉（赤色）側の端点を v とする



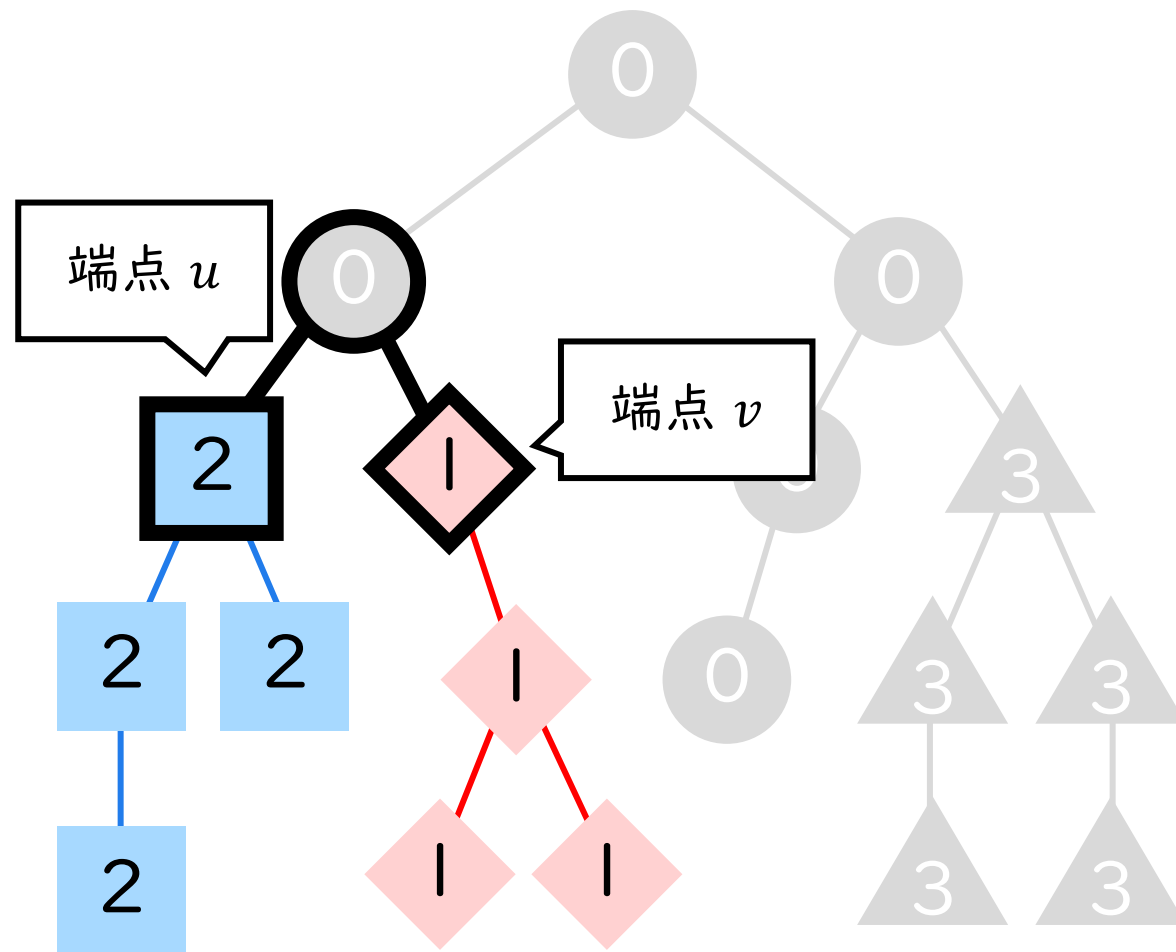
重要な性質

グループ間を行き来するために

絶対通るパスが存在

※遊園地（青色）側の端点を u

温泉（赤色）側の端点を v とする



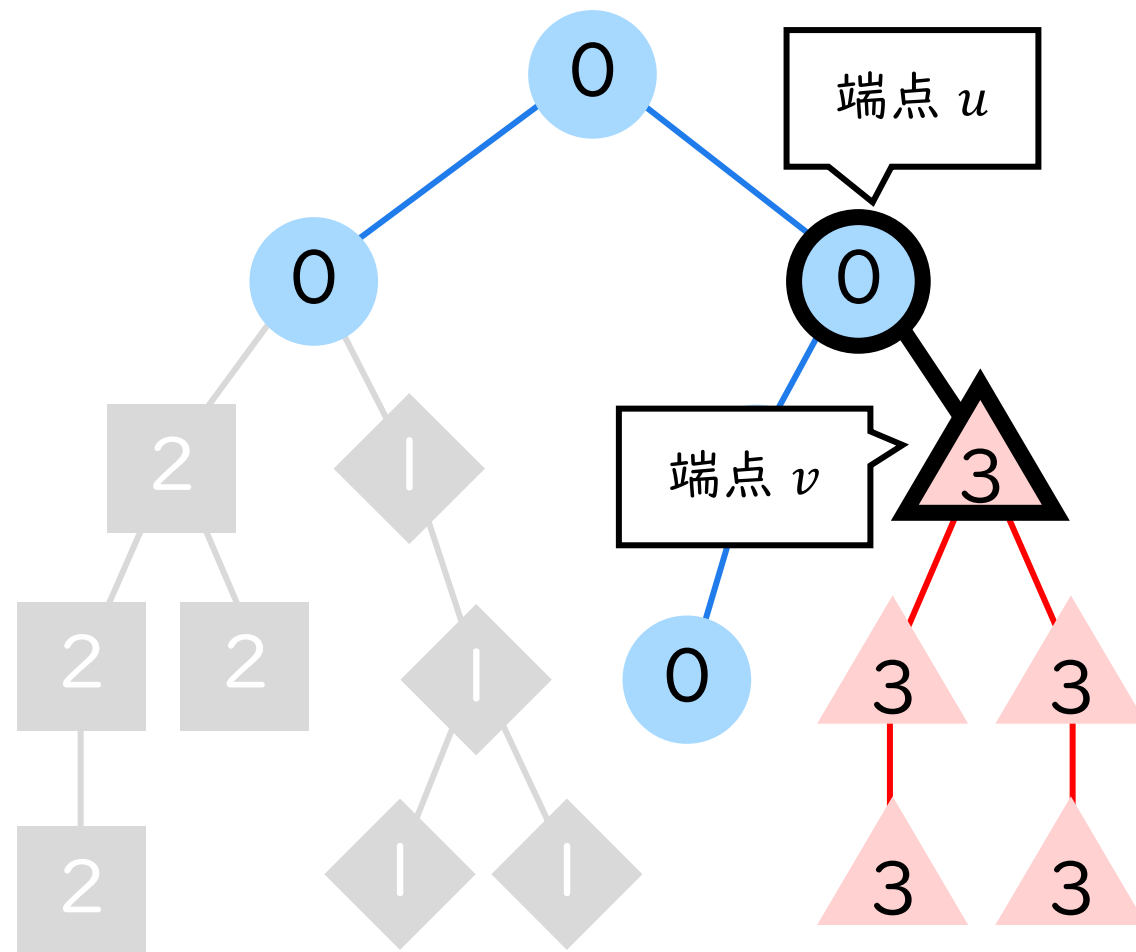
重要な性質

グループ間を行き来するために

絶対通るパスが存在

※遊園地（青色）側の端点を u

温泉（赤色）側の端点を v とする





Q. Ali はどんな情報を
送れば良いのか？

3 59 点解法：ステップ 3

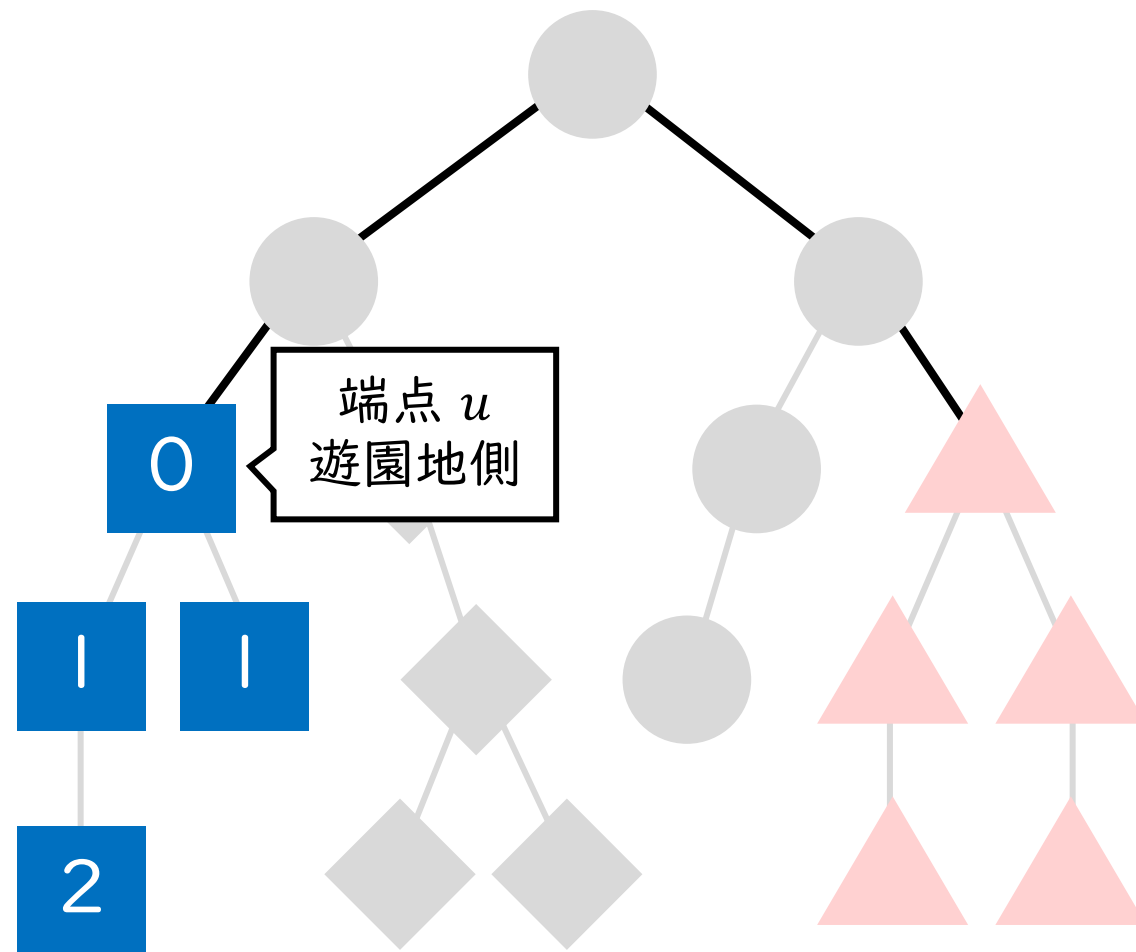
120
222

Ali が送る情報

1 端点 u から各頂点への距離

2 端点 v から各頂点への距離

3 $u - v$ 間の距離



3 59 点解法：ステップ 3

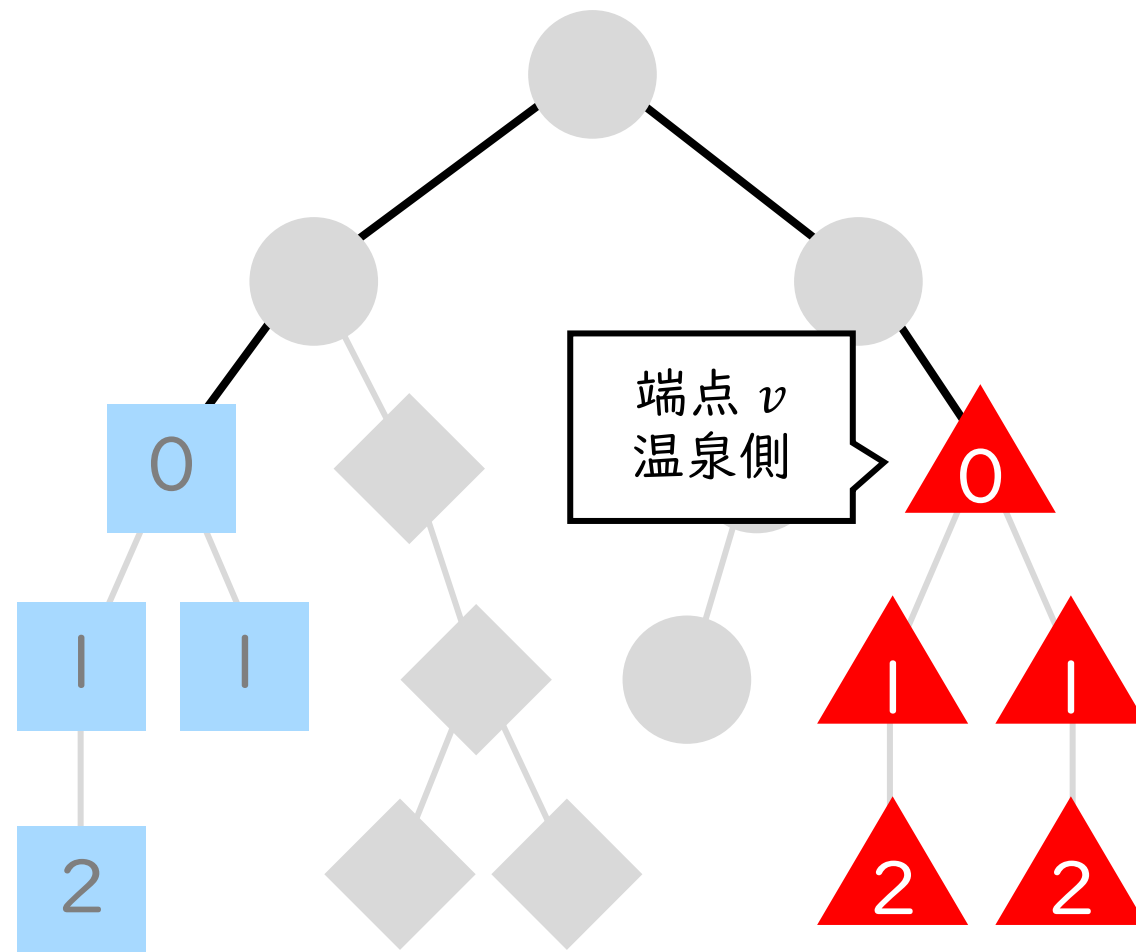
121
222

Ali が送る情報

1 端点 u から各頂点への距離

2 端点 v から各頂点への距離

3 $u - v$ 間の距離



3 59 点解法：ステップ 3

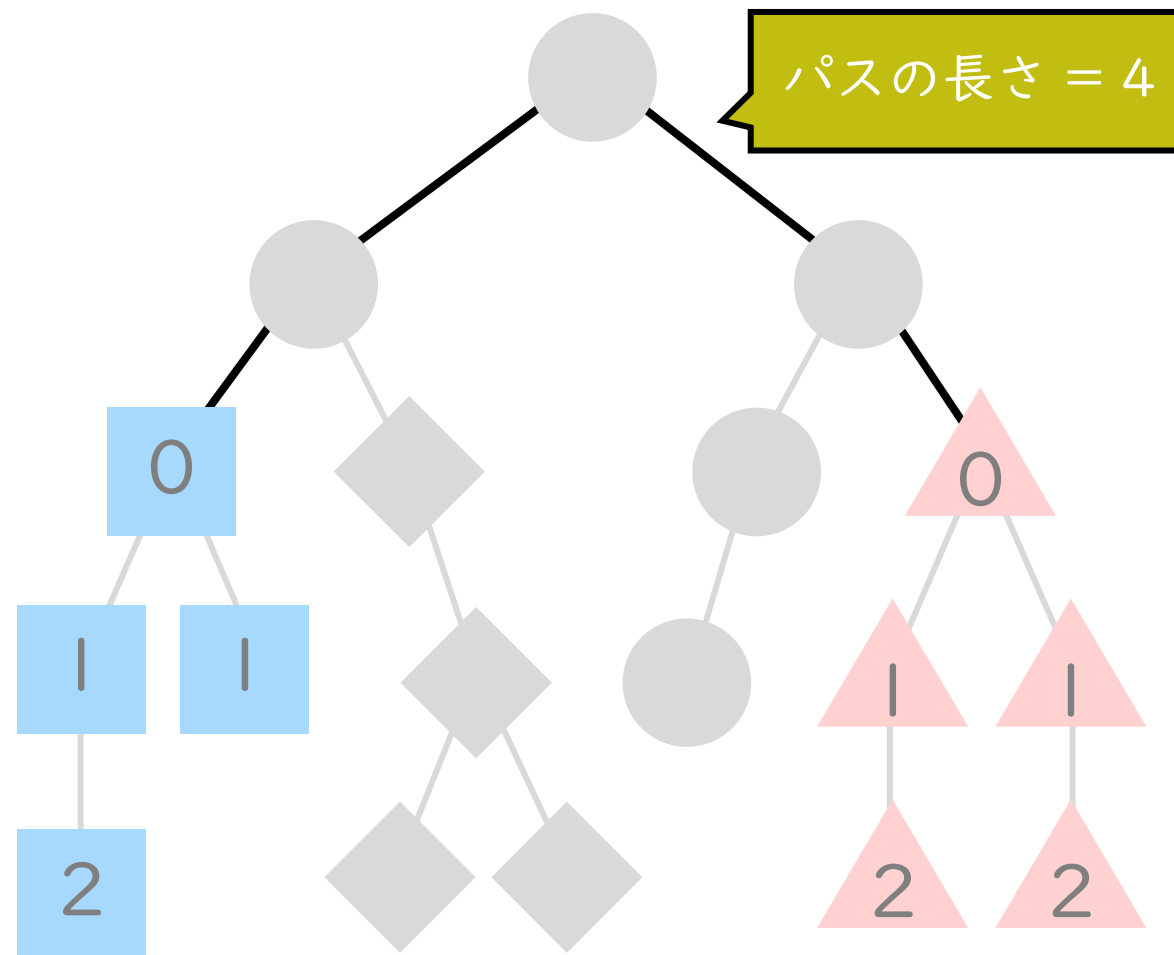
122
222

Ali が送る情報

1 端点 u から各頂点への距離

2 端点 v から各頂点への距離

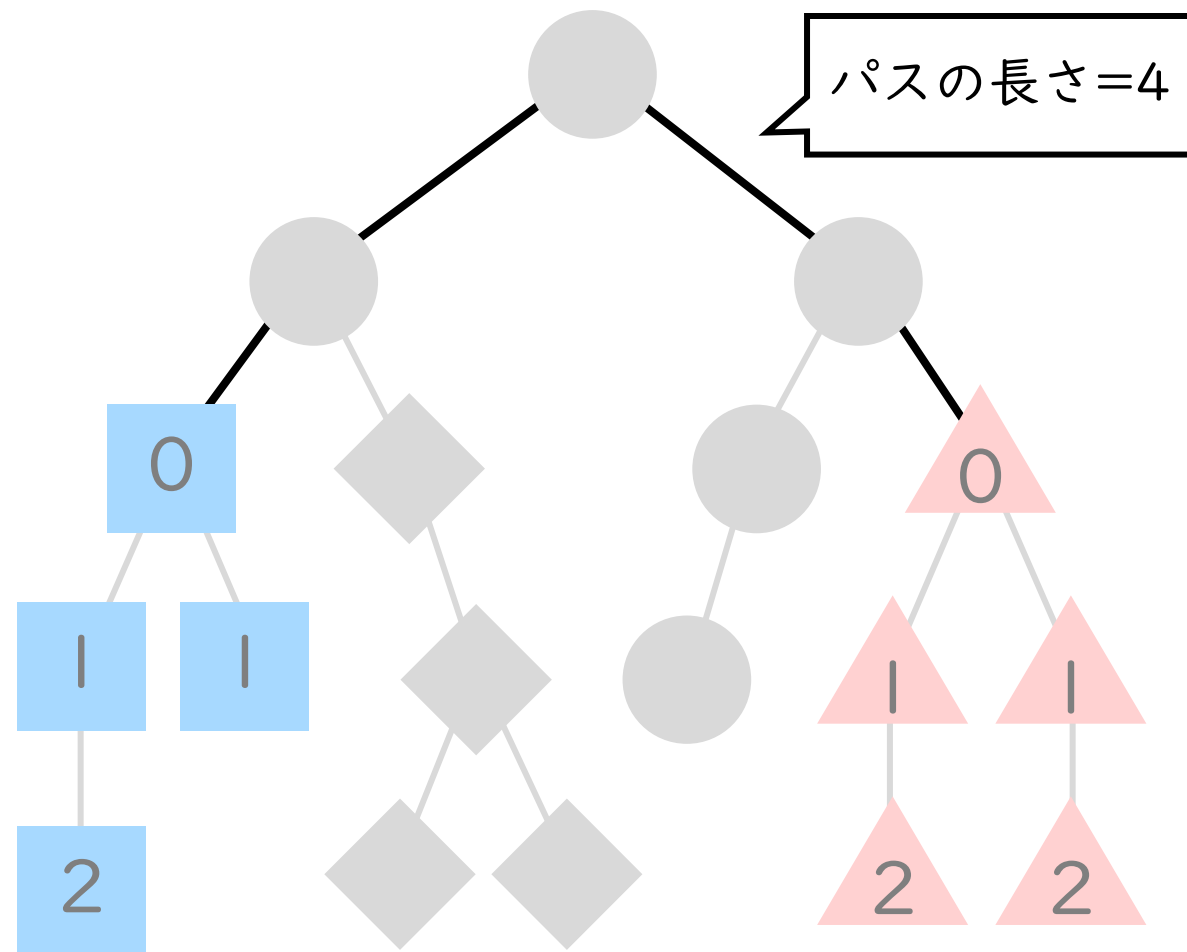
3 $u - v$ 間の距離





本当にこの情報だけで
最短距離が求められるのか？

実は全部足すだけで
最短距離が求められる



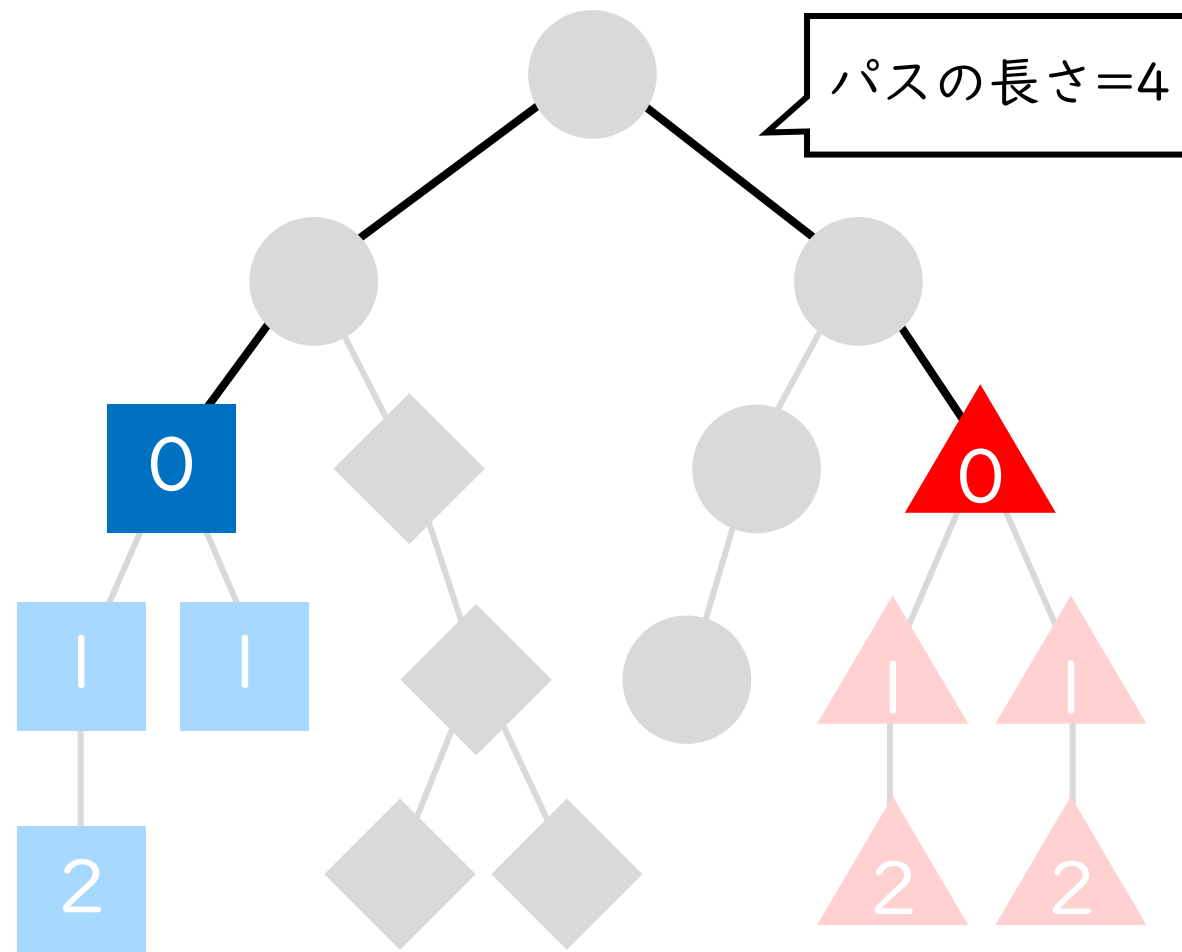
3 59 点解法：ステップ 3

125
222

実は全部足すだけで
最短距離が求められる

最短距離

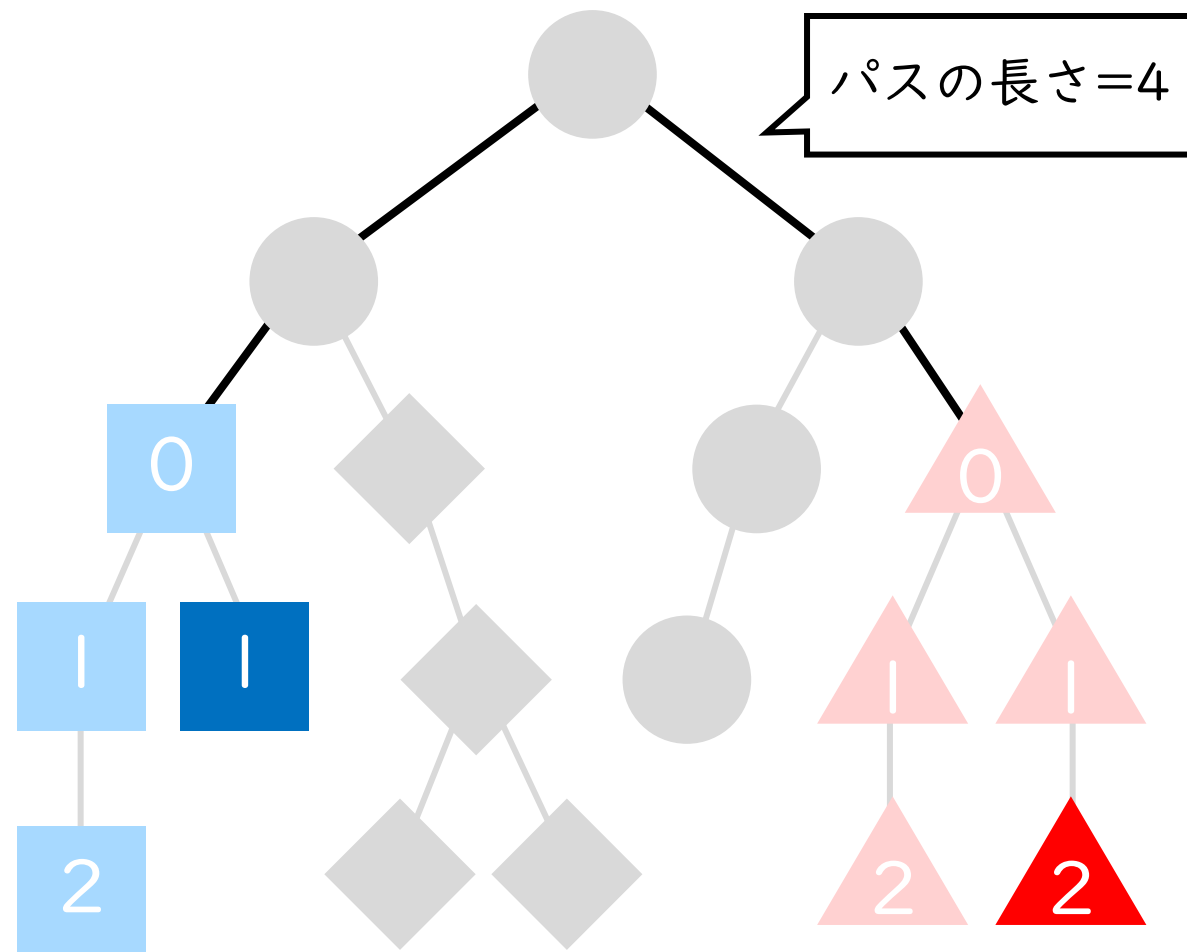
$$4 + 0 + 0 = 4$$



実は全部足すだけで
最短距離が求められる

最短距離

$$4 + 1 + 2 = 7$$



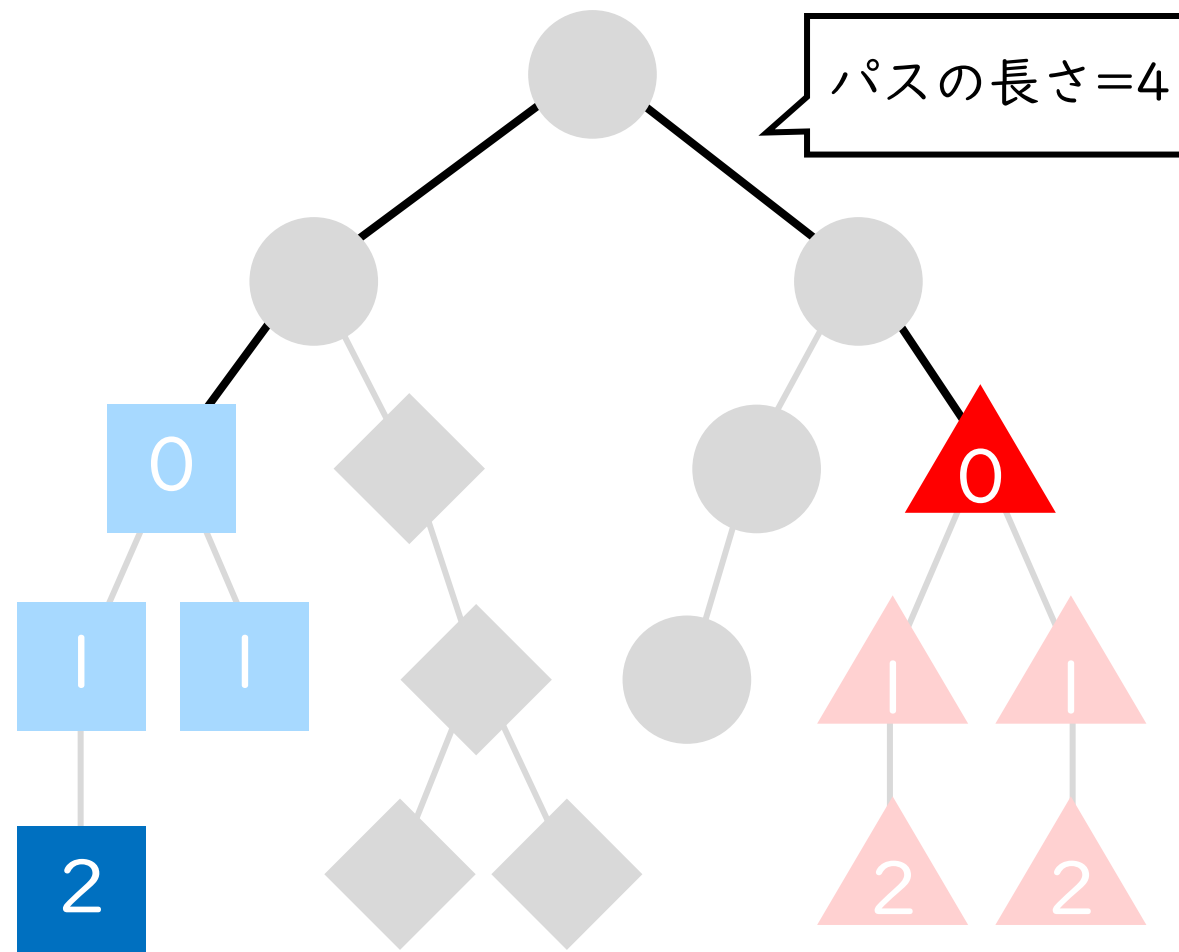
3 59 点解法：ステップ 3

127
222

実は全部足すだけで
最短距離が求められる

最短距離

$$4 + 2 + 0 = 6$$





最後に、Alice は
何ビット送る必要があるか？

3 59 点解法：ステップ 3

129
222

必要なビット数は？

1. 端点 u からの距離

$$\rightarrow 13 \text{ 個} \times 4 \text{ ビット} = 52 \text{ ビット}$$

一つのグループの
最大頂点数は 13

最大距離は 13
 $2^4=16$ で十分

3 59 点解法：ステップ 3

130
222

必要なビット数は？

1. 端点 u からの距離

$$\rightarrow 13 \text{ 個} \times 4 \text{ ビット} = 52 \text{ ビット}$$

2. 端点 v からの距離

$$\rightarrow 13 \text{ 個} \times 4 \text{ ビット} = 52 \text{ ビット}$$

一つのグループの
最大頂点数は 13

最大距離は 13
 $2^4=16$ で十分

3 59 点解法：ステップ 3

131
222

必要なビット数は？

1. 端点 u からの距離

$$\rightarrow 13 \text{ 個} \times 4 \text{ ビット} = 52 \text{ ビット}$$

2. 端点 v からの距離

$$\rightarrow 13 \text{ 個} \times 4 \text{ ビット} = 52 \text{ ビット}$$

3. $u - v$ 間の距離

$$\rightarrow 14 \text{ ビット}$$

合計ビット数
118

3 59 点解法：ステップ 3

132
222

必要なビット数は？

1. 端点 u からの距離

→ 13 個 $\times$ 4 ビット = 52 ビット

2. 端点 v からの距離

→ 13 個 $\times$ 4 ビット = 52 ビット

3. $u - v$ 間の距離

→ 14 ビット

ここまでの点数：59 点

合計ビット数

118

Stage 4

40 ビットまでの解法

Q. もっと効率的に
各グループの情報を送れないか？

II 点解法

Benjamin が頂点番号 X を送り

Ali が番号 X から各頂点への距離を送る



合計ビット数
14万

11 点解法

Benjamin が頂点番号 X を送り
Ali が番号 X から各頂点への距離を送る



合計ビット数
14万

15 点解法

Ali が木を $2(N - 1)$ ビットの文字列として送る
Benjamin は何もしない



合計ビット数
2万



11 点解法 グループ内の距離を送るより
Benjamin が頂点番号 X を送り
木を $2(N-1)$ ビットで送った方が得なのでは？



15 点解法

Ali が木を $2N$ ビットとして送る
Benjamin は何もしない

検証してみよう

合計ビット数
14万

合計ビット数
2万

Ali が必ず送るべき情報は以下の 4 つ

1

遊園地グループの木の情報

2

温泉グループの木の情報

3

端点 $u - v$ 間の距離

4

端点 u, v のグループ内番号

Ali が必ず送るべき情報は以下の 4 つ

1

遊園地グループの木の情報

2

温泉グループの木の情報

3

端点 $u-v$ 間の距離

4

端点 u, v のグループ内番号

木の情報を送っただけでは
最短距離が求められないことに注意



Alice は何ビット送る必要があるか？

Ali が必ず送るべき情報は以下の 4 つ

最大頂点数は $N = 13$
ビット数は $2(N - 1) = 24$

1

遊園地グループの木の情報

$$\Rightarrow 2 \times 12 = 24 \text{ ビット}$$

2

温泉グループの木の情報

$$\Rightarrow 2 \times 12 = 24 \text{ ビット}$$

3

端点 $u - v$ 間の距離

$$\Rightarrow 14 \text{ ビット}$$

4

端点 u, v のグループ内番号

$$\Rightarrow 2 \times 4 = 8 \text{ ビット}$$

合計ビット数
70



Ali が必ず送るべき情報は以下の 4 つ

最大頂点数は $N = 13$
ビット数は $2(N - 1) = 24$

1 遊園地グループの木の情報

→ $2 \times 12 = 24$ ビット

2 ここまでの点数: **67** 点

3 端点 u, v 間の距離

→ 14 ビット

4 端点 u, v のグループ内番号

→ $2 \times 4 = 8$ ビット

合計ビット数
70

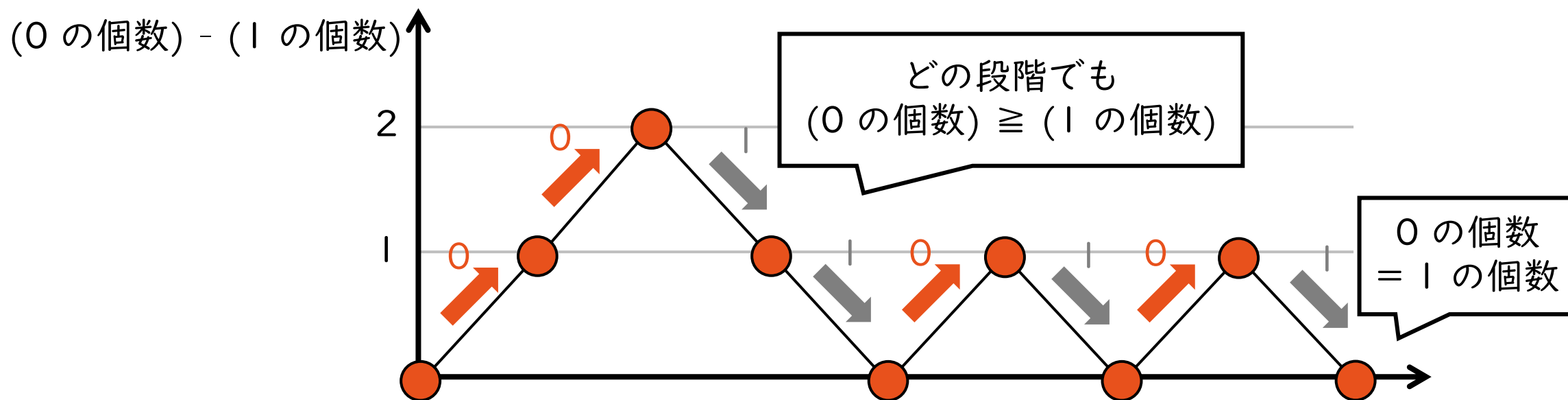
長さ $2(N - 1)$ の 01 文字列すべてが
木を表すとは限らない

× “11111111”

× “00111100”

重要な考察

木を表す文字列は **カッコ列** と同じ条件を満たす





パターン数はどれくらいか？

長さ 24 の文字列 : 2^{24} 通り ▶ 24 ビット

パターン数はどれくらいか？

※カッコ列の数はカタラン数と関連します。
興味のある方は調べてみてください。

~~長さ 24 の文字列~~ : ~~2²⁴~~ 通り  ~~24~~ ビット

長さ 24 以下のカッコ列 : 29万 通り  19 ビット

合計ビット数
60



パターン数はどれくらいか？

※カッコ列の数はカタラン数と関連します。
興味のある方は調べてみてください。

ここまでの点数：**72**点

長さ 24 以下のカッコ列：29万通り

合計ビット数

60



実は木のパターン数を
さらに減らすことができます

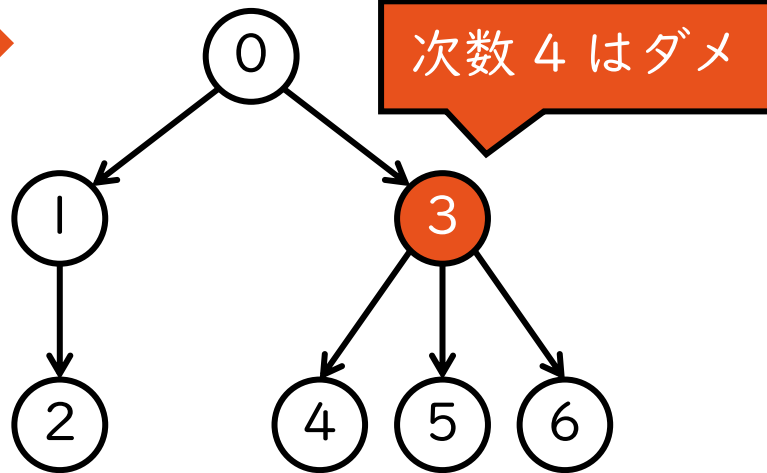
根付き木の出次数は 2 以下

根付き木の出次数は 2 以下

ポイント 1

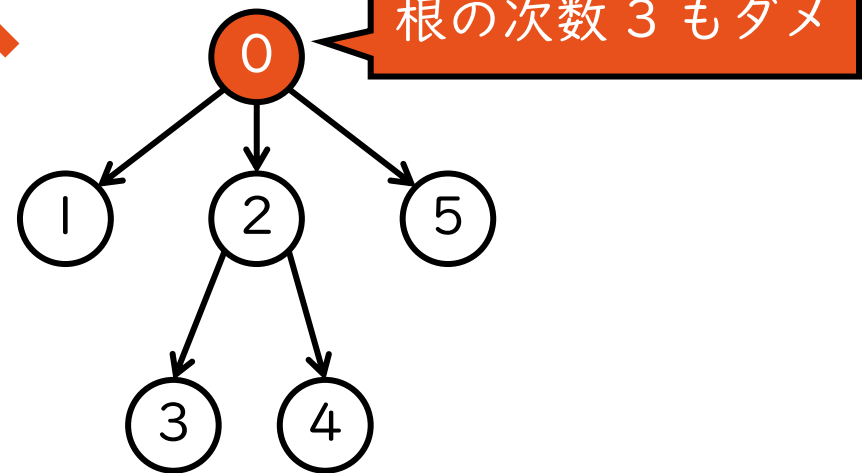
根付き木の出次数は 2 以下

✗



“001100101011”

✗



“0100101101”



ポイント 1

根付き木の出次数は 2 以下

ポイント 2

根以外の部分木のサイズは 6 以下

※サイズ 7 だと貪欲法では切られてしまう！

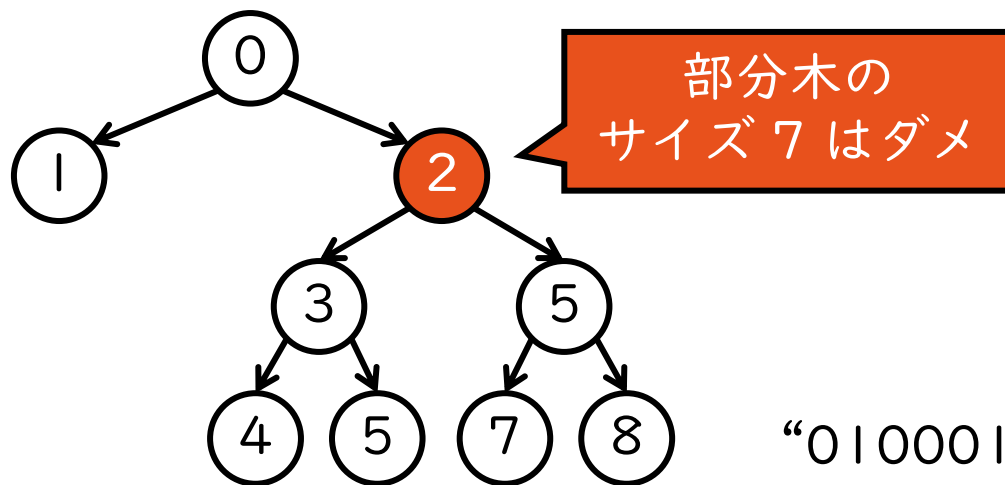
ポイント 1

根付き木の出次数は 2 以下

ポイント 2

根以外の部分木のサイズは 6 以下

※サイズ 7 だと貪欲法では切られてしまう！



“0100010110010111”



パターン数はどれくらいか？

長さ 24 以下のカッコ列： 29万 通り ▶ 19 ビット

パターン数はどれくらいか？

合計ビット数
40

~~長さ 24 以下のカッコ列：29 万 通り ▶ 19 ビット~~

条件を満たす木の個数 : 325 通り ▶ 9 ビット

※同型な木を同じとみなさない場合は 1483 通り (11 ビット、80 点)

4



84 点解法

155
222

パターン数はどれくらいか？

合計ビット数
40

ここまでの点数：84 点

条件を満たす木の個数：325 通り ▶ 9 ビット

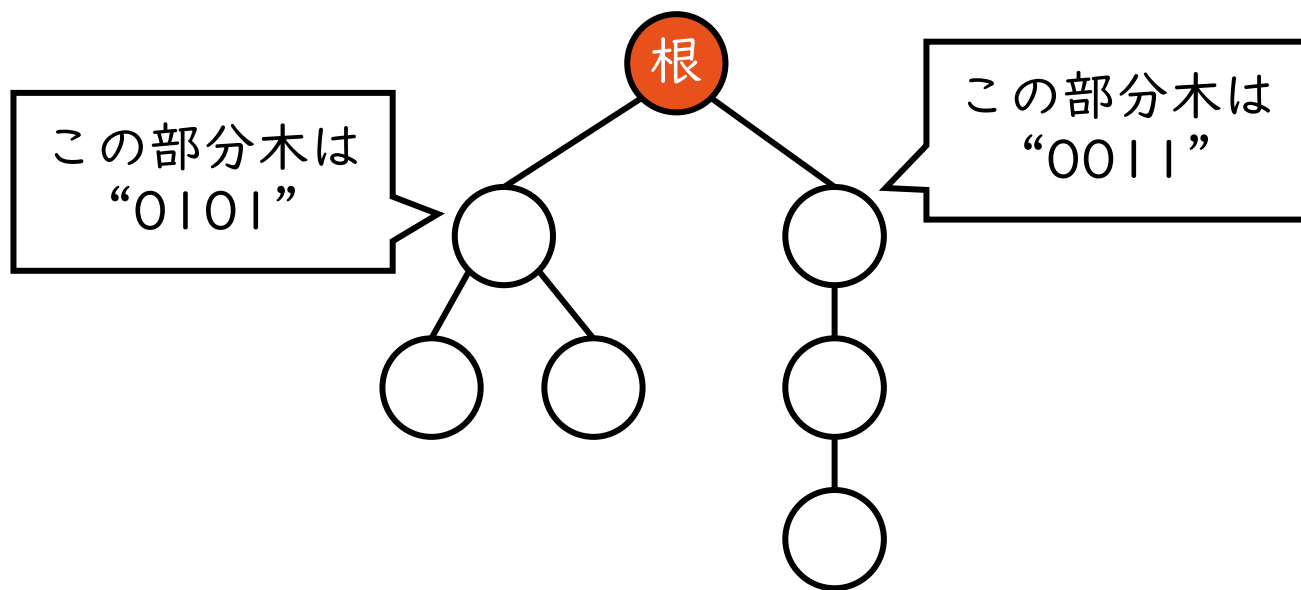
※同型な木を同じとみなさない場合は 1483 通り（11 ビット、80 点）

84 点を獲得するためには、同型な木を「同じもの」だとしてビットを送らなければならない（そうしなければ 80 点になる）

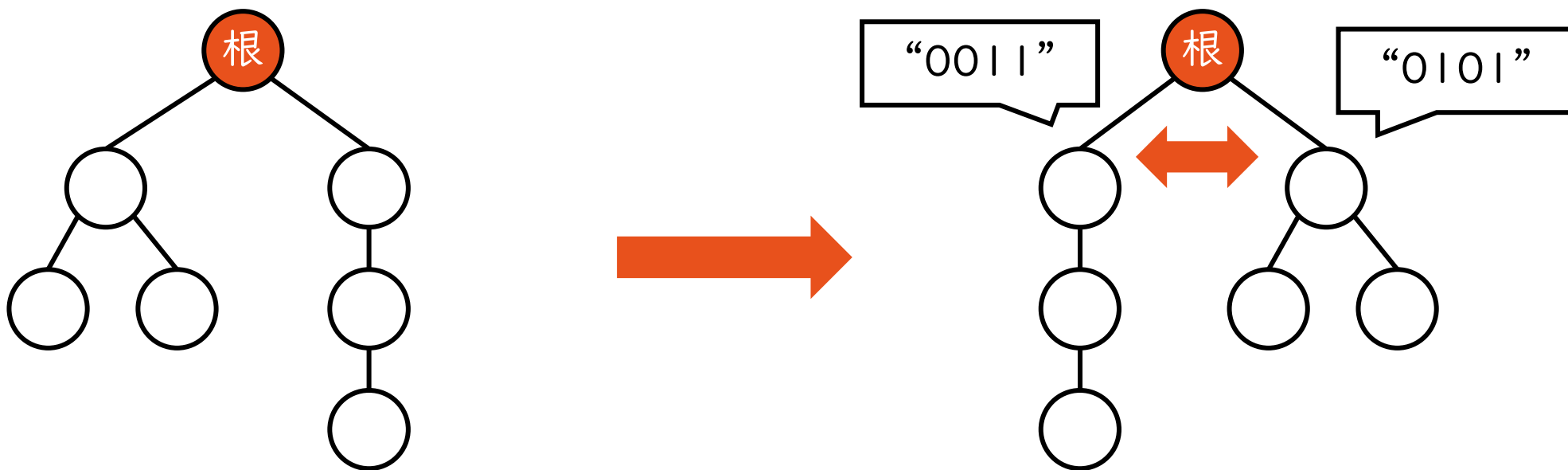


2つの木が同型かどうかを判定するためには、部分木を表す文字列の辞書順に辺を付け替えれば良い

2つの木が同型かどうかを判定するためには、**部分木を表す文字列の辞書順**に辺を付け替えれば良い



2つの木が同型かどうかを判定するためには、**部分木を表す文字列の辞書順**に辺を付け替えれば良い



Stage 5

29 ビットまでの解法



現時点で使っているビット数は、次の通り

1

遊園地グループの木の情報

9ビット

2

温泉グループの木の情報

9ビット

3

端点 $u-v$ 間の距離

14ビット

4

端点 u, v のグループ内番号

8ビット

現時点で使っているビット数は、次の通り

1	遊園地グループの木の情報
---	--------------

9ビット

2	温泉グループの木の情報
---	-------------

9ビット

3	端点 $u-v$ 間の距離
---	---------------

14ビット

4	端点 u, v のグループ内番号
---	--------------------

8ビット ➡ これを減らせないか？



重要な性質

端点 u, v のうち少なくとも木全体の
の根から遠い方は...

グループ内の根

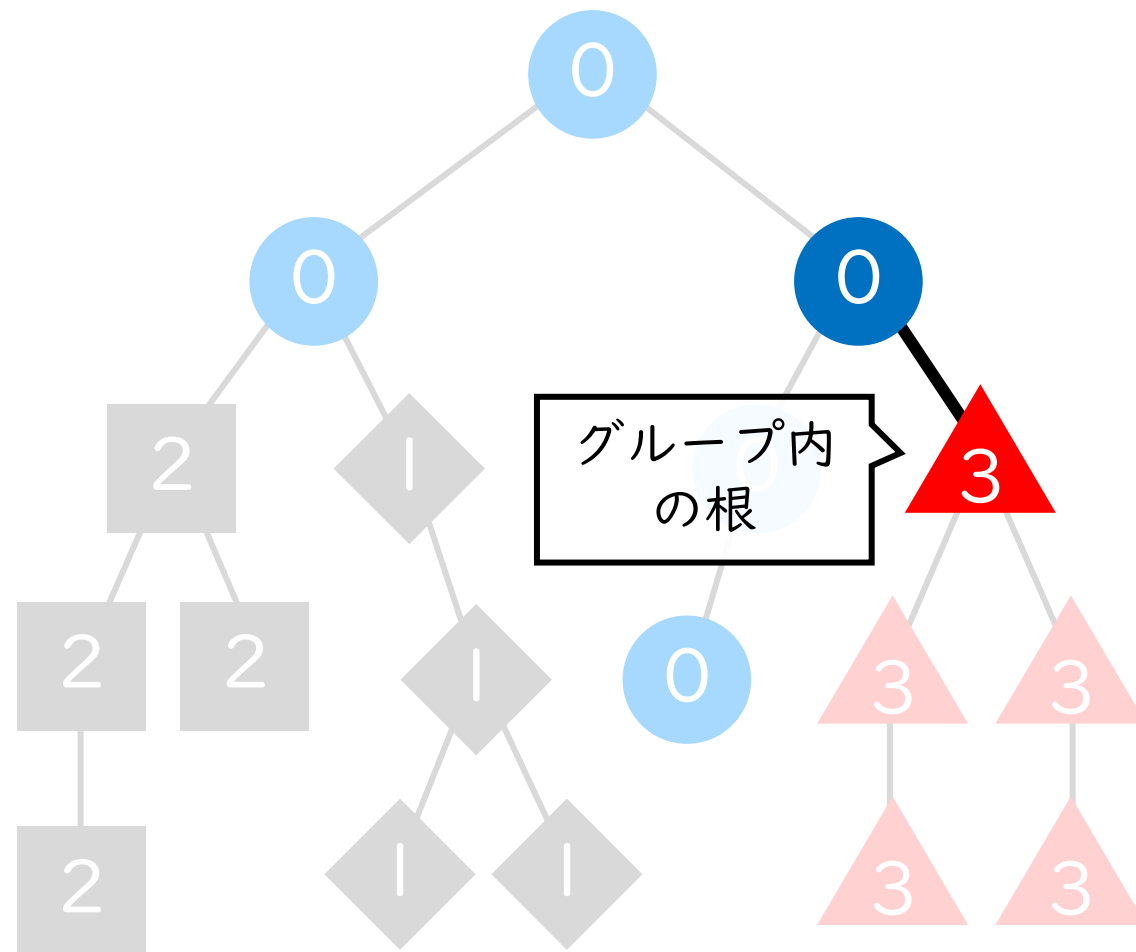
になっている

重要な性質

端点 u, v のうち少なくとも木全体の根から遠い方は...

グループ内の根

になっている

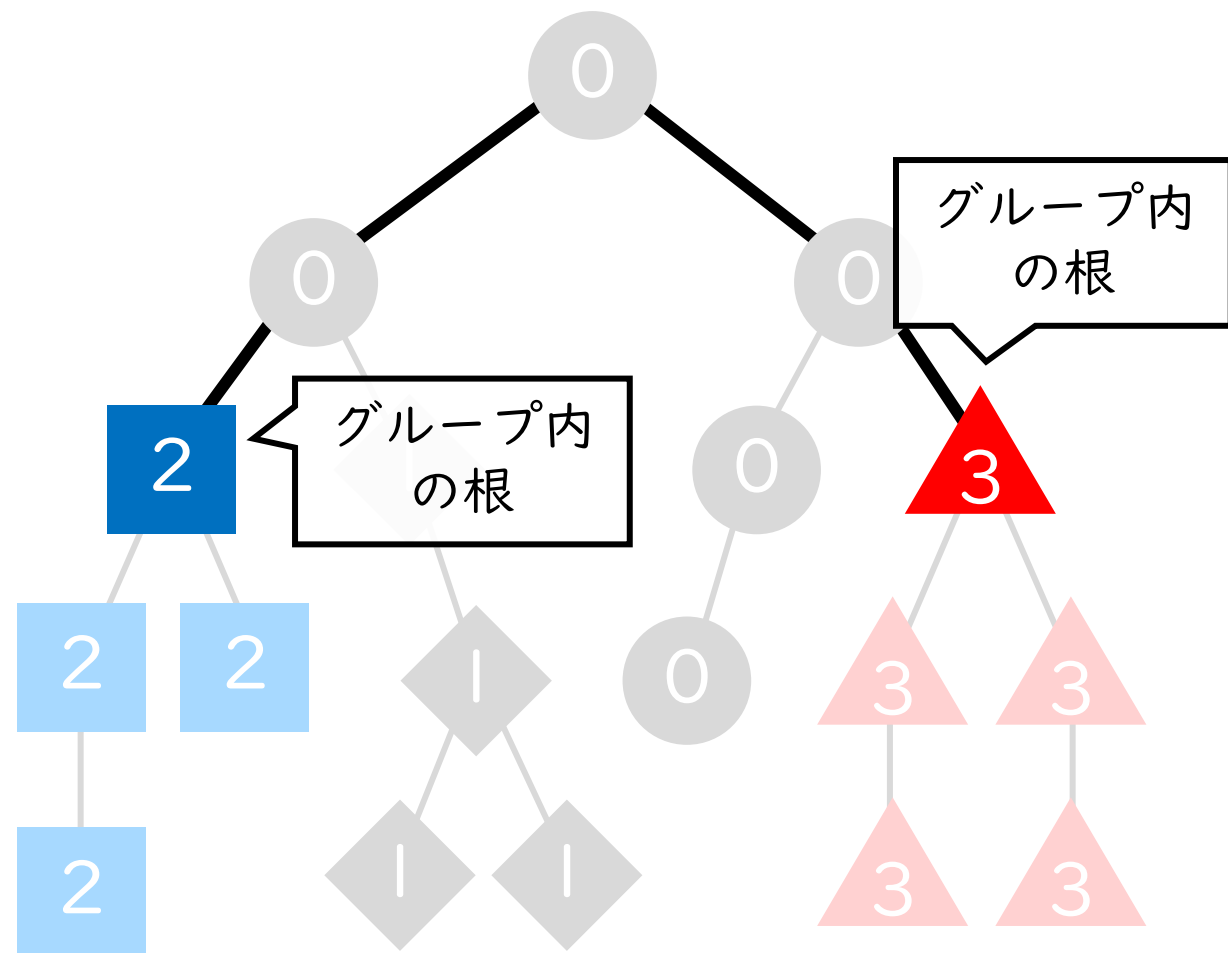


重要な性質

端点 u, v のうち少なくとも木全体の根から遠い方は...

グループ内の根

になっている





この性質を使うと…

端点 u, v のグループ内頂点番号のうち
グループ内の根である方は送らなくて良い

この性質を使うと…

端点 u, v のグループ内頂点番号のうち
グループ内の根である方は送らなくて良い



4 ビット減る！

合計ビット数
36

※ただし、 u と v どちらが「グループ内の根」であるかを判別するために
木全体の根に近い方のグループ番号を小さくするなどの工夫が必要



この性質を使うと…

端点 u, v のグループ内頂点番号のうち
グループ内の根である方は送らなくて良い

ここまでの点数： 88 点
4 ビット減る！

合計ビット数
36

※ただし、 u と v どちらが「グループ内の根」であるかを判別するために
木全体の根に近い方のグループ番号を小さくするなどの工夫が必要



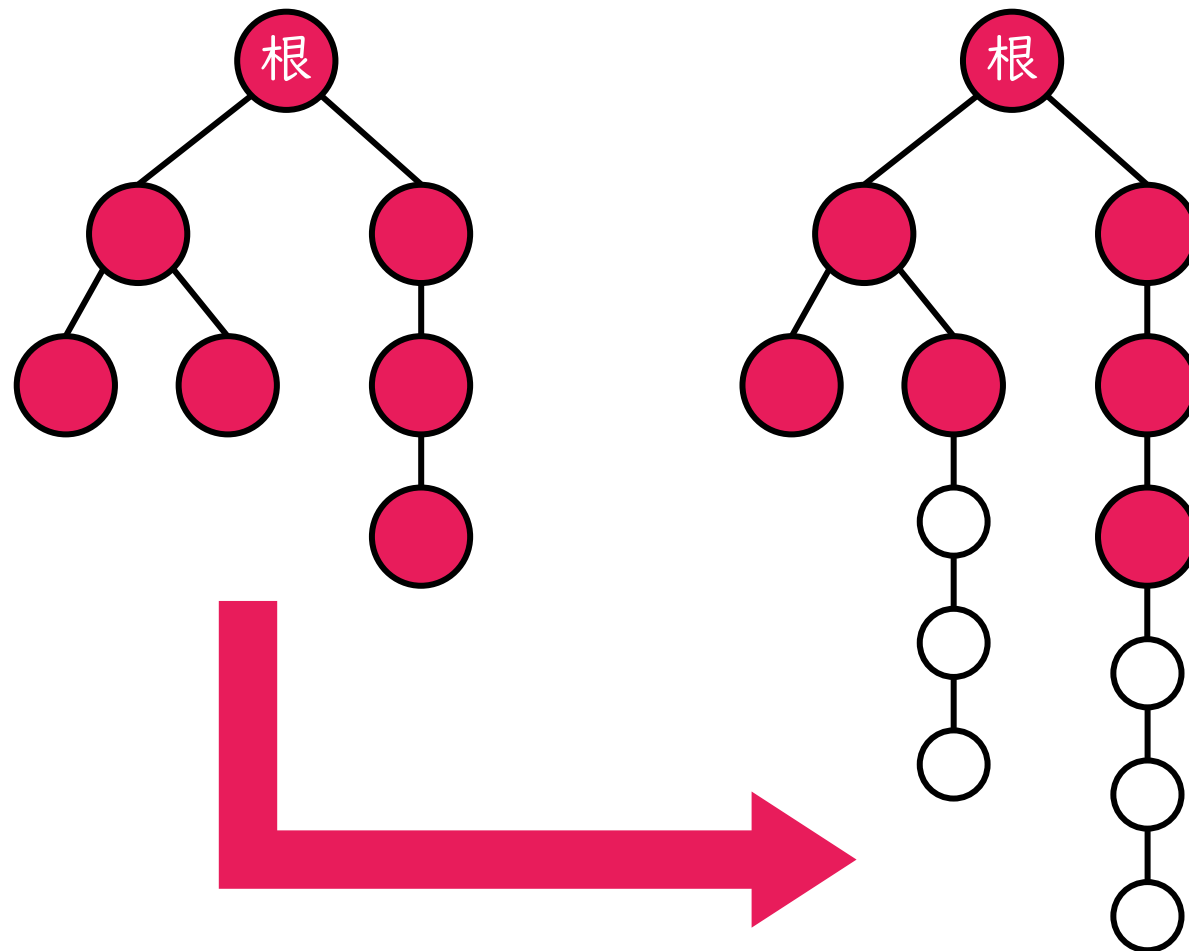
実は 1 ~ 13 頂点の木
すべてを考える必要はない

重要な性質

各グループの木に頂点を適当に追加することで、

- 次数 ≤ 3
- 部分木サイズ ≤ 6

を満たす 13 頂点の木が生成できる





パターン数はどれくらいか？

13 頂点以下の木： 325 通り ▶ 9 ビット



パターン数はどれくらいか？

~~13 頂点以下の木 : 325 通り~~  ~~9 ビット~~

13 頂点の木 : 66 通り  7 ビット

合計ビット数
32



パターン数はどれくらいか？

ここまでの点数：

92 点

13 頂点の木

： 66 通り

7 ビット

合計ビット数

32

これまでは、以下の 4 つの情報を独立に送っていた

情報	パターン数
1 つ目の木の情報	66 通り
2 つ目の木の情報	66 通り
接点のグループ内頂点番号	14 通り
接点 u, v 間の距離	10000 通り

➡ 7 ビット

➡ 7 ビット

➡ 4 ビット

➡ 14 ビット

これらをまとめて送ると 30 ビットまで減る

情報	パターン数
1 つ目の木の情報	66 通り
2 つ目の木の情報	66 通り
接点のグループ内頂点番号	14 通り
接点 u, v 間の距離	10000 通り

$$66 \times 66 \times 14 \times 10000 \\ = 609840000 \text{ 通り}$$

30 ビットで十分



これらをまとめて送ると 30 ビットまで減る

情報	パターン数
1 つ目の木の情報	66 通り
2 つ目の木の情報	88 通り
接点のグループ内頂点番号	14 通り
接点 u, v 間の距離	10000 通り

ここまでの点数:

94 点

$$66 \times 88 \times 14 \times 10000 = 609840000 \text{ 通り}$$

30 ビットで十分

これまでは Ali が 30 ビットの情報しか送らなかった



Ali が 1 ビットから **29** ビットまで全部使った場合

ほぼ 2^{30} 通りを識別できる※

合計ビット数
29



これまでは Ali が 30 ビットの情報しか送らなかった

ここまでの点数：**95**点

Ali が 1 ビットから 29 ビットまで全部使った場合

ほぼ 2^{30} 通りを識別できる※

合計ビット数
29

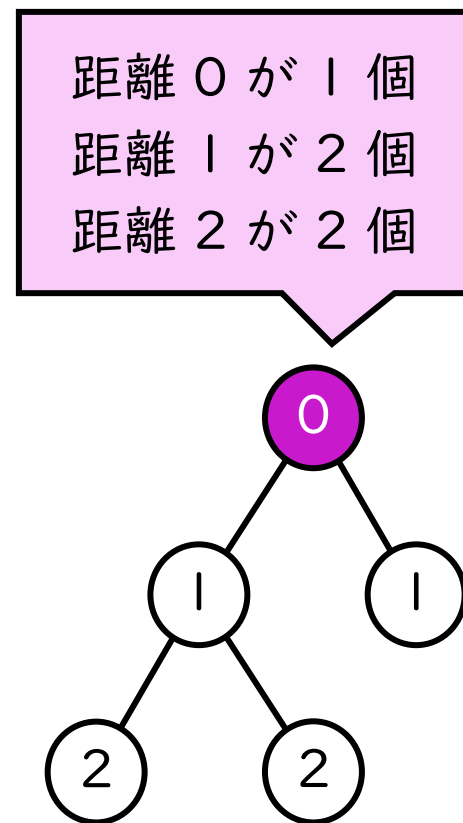
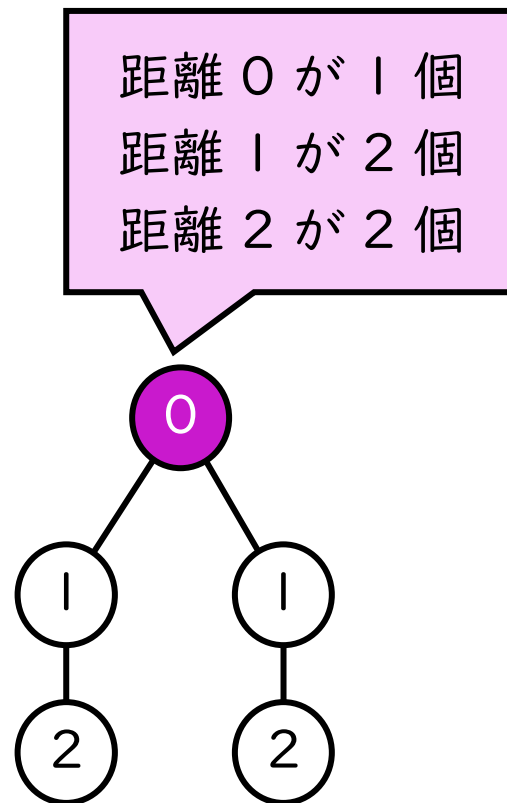
※ $2^1 + 2^2 + 2^3 + \dots + 2^{29} = 2^{30} - 2$

Extra Stage

満点 (24 ビット) へ

重要な性質

- 実は、木の代わりに距離分布を送った方が得
- たとえば以下の 2 つの木を「同じ」とみなせる

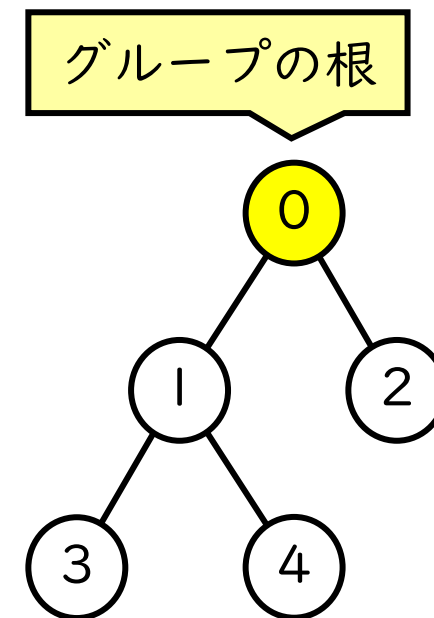
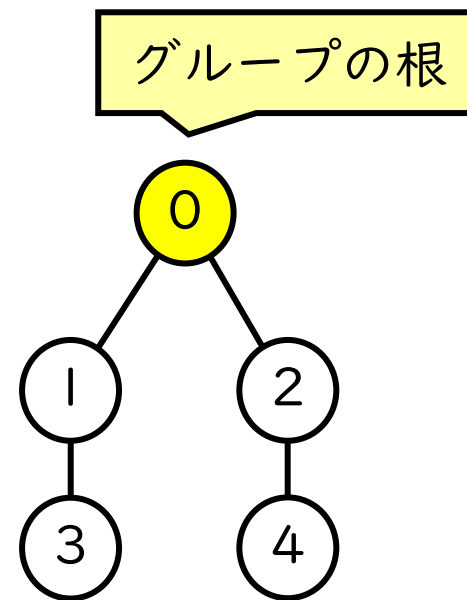




具体的にどうやって実現するか？

手順 I

各グループについて
グループ内番号を BFS 順に設定



手順 1

各グループについて
グループ内番号を BFS 順に設定

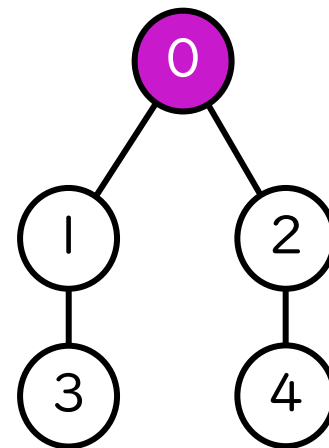


手順 2

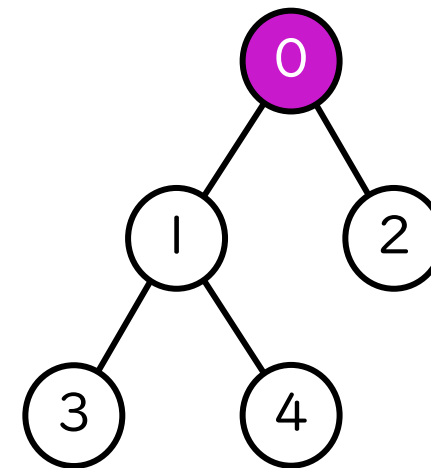
以下の組を送る

- 端点のグループ内番号
- 端点から ID=0 までの距離
:
- 端点から ID=12 までの距離

端点 0 からの距離
ID=0 から順に
(0, 1, 1, 2, 2)



端点 0 からの距離
ID=0 から順に
(0, 1, 1, 2, 2)



異なる木でも同じ情報を

- # 送っていい

送っていて得である！





パターン数はどれくらいか？

端点が根の場合

木を送る 距離分布を送る
 $66_{\text{通り}} \rightarrow 38_{\text{通り}}$



パターン数はどれくらいか？

端点が根の場合

端点が根とは限らない場合

木を送る 距離分布を送る

66通り $\rightarrow$ 38通り

858通り $\rightarrow$ 676通り※
[66×13 通り]

※38×13 通りまでは行かないが、66×13 通りよりは少ない



パターン数はどれくらいか？

端点が根の場合

27

ビット

97

点

端点が根とは限らない場合

木を送る

距離分布を送る

66通り $\rightarrow$ 38通り

木を送る

距離分布を送る

858通り $\rightarrow$ 676通り※

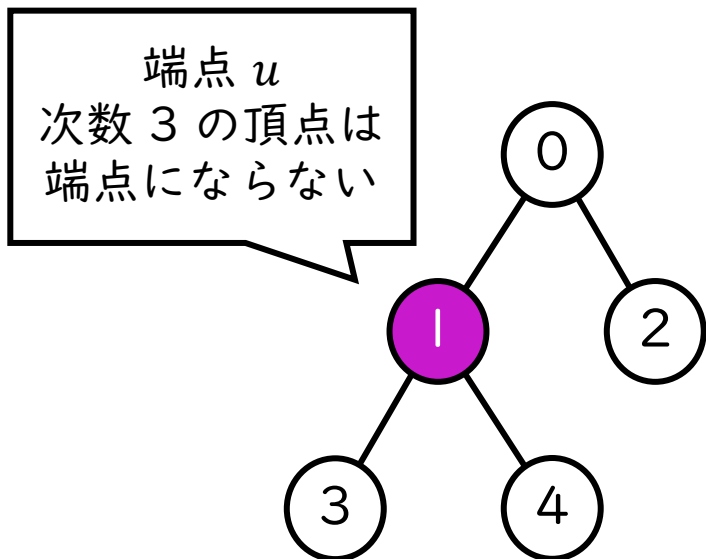
[66×13 通り]

※38×13 通りまでは行かないが、66×13 通りよりは少ない

重要な考察

端点と木の組合せとして

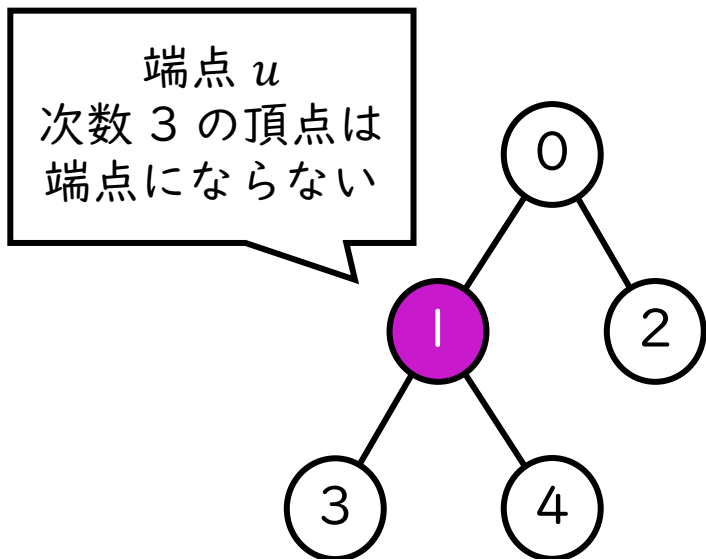
以下のようなものは絶対にあり得ない



重要な考察

端点と木の組合せとして

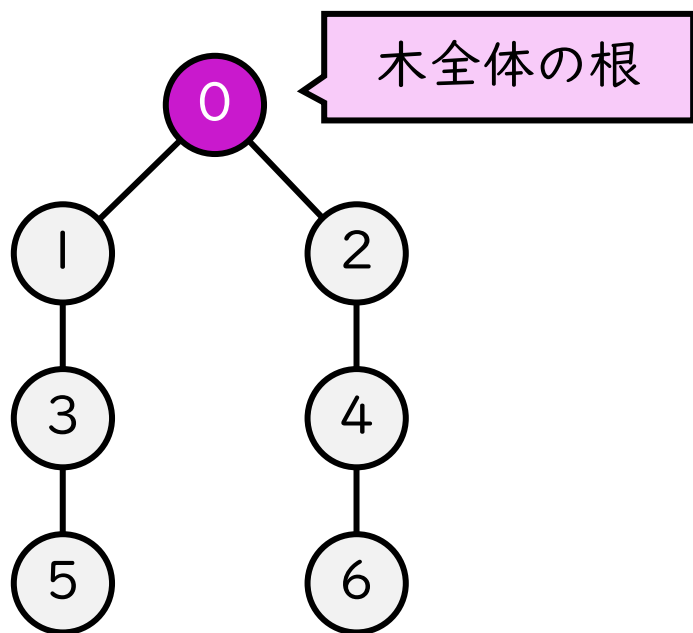
以下のようなものは絶対にあり得ない



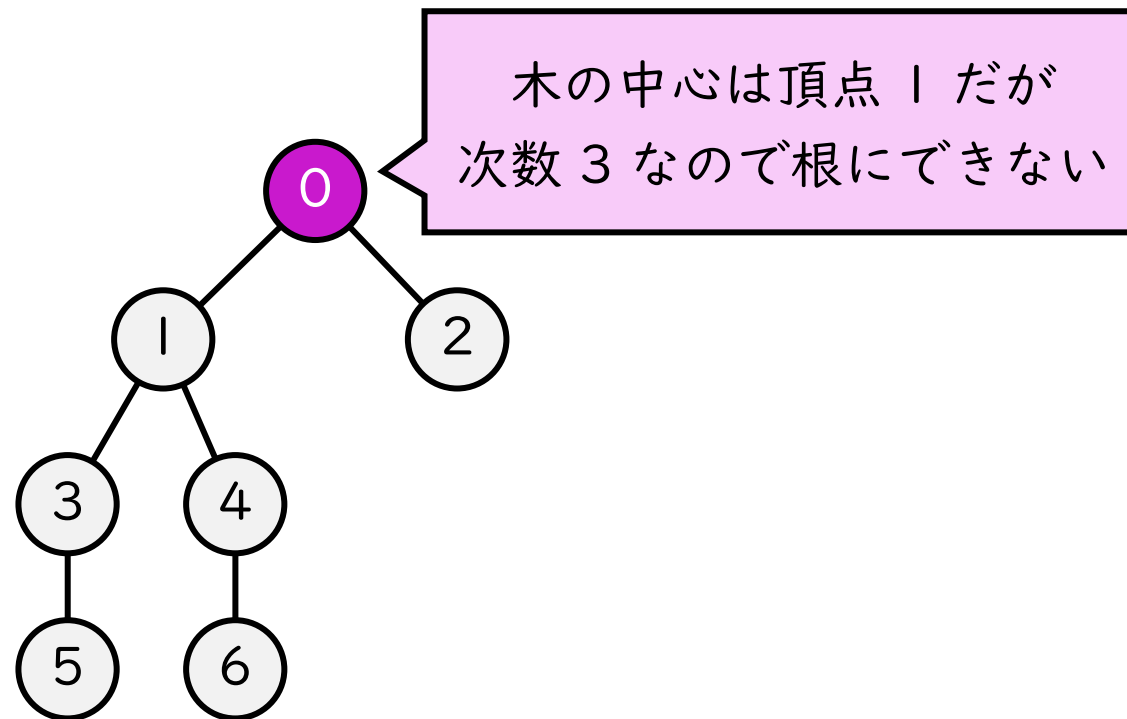
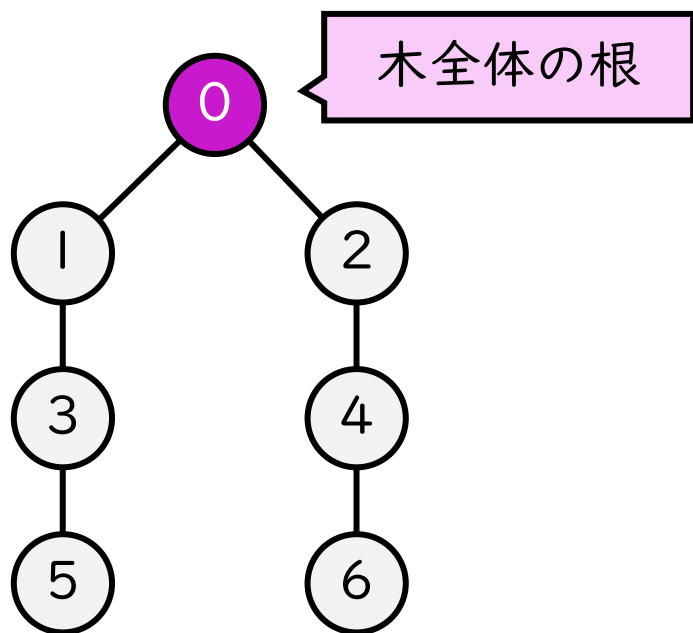
このようなものを排除すると...

676 通り ▶ 553 通り

できるだけ木の中心に近い次数 2 以上の頂点を
木全体の根にすることを考える



できるだけ木の中心に近い次数 2 以上の頂点を
木全体の根にすることを考える

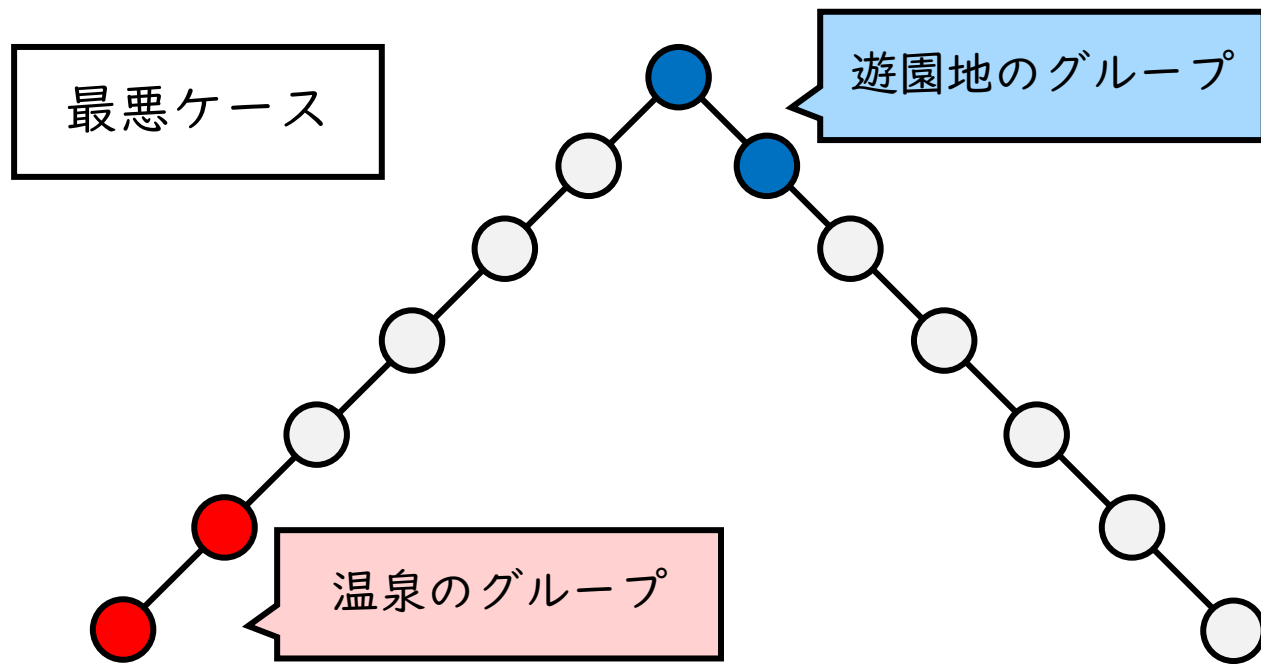


根を上手く設定すると...

端点 u, v の片方が「グループ内の根」ではないケースでは

$u - v$ 間距離

5000 以下





パターン数はどれくらいか？

u, v 両方がグループ内の根

➡ $38 \times 38 \times 10000$ 通り

そうでない場合

➡ $38 \times 515 \times 5000$ 通り※

合計 112,290,000 通り

※ 515 通りは、全体 553 通りから「端点がグループ内の根 (38 通り)」を引いた値



パターン数はどれくらいか？

u, v 両方がグループ内の根



$38 \times 38 \times 100000$ 通り

そうでない場合



$38 \times 515 \times 50000$ 通り※

26 ビット 98 点

合計 112,290,000 通り

※ 515 通りは、全体 553 通りから「端点がグループ内の根（38 通り）」を引いた値

実は頂点数を 14 まで増やしても
ID が $2N + 19$ を超えない

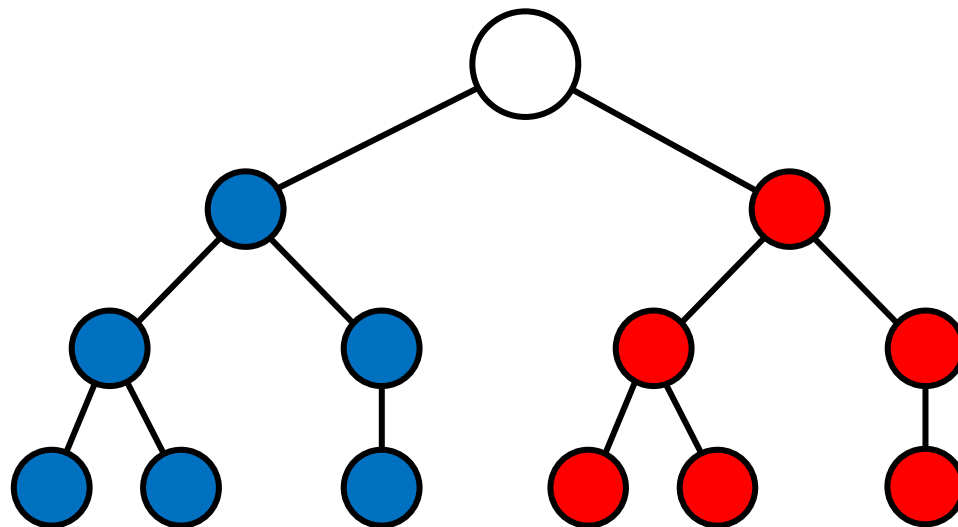


1 頂点追加して木の種類数を減らせないか？

重要な性質

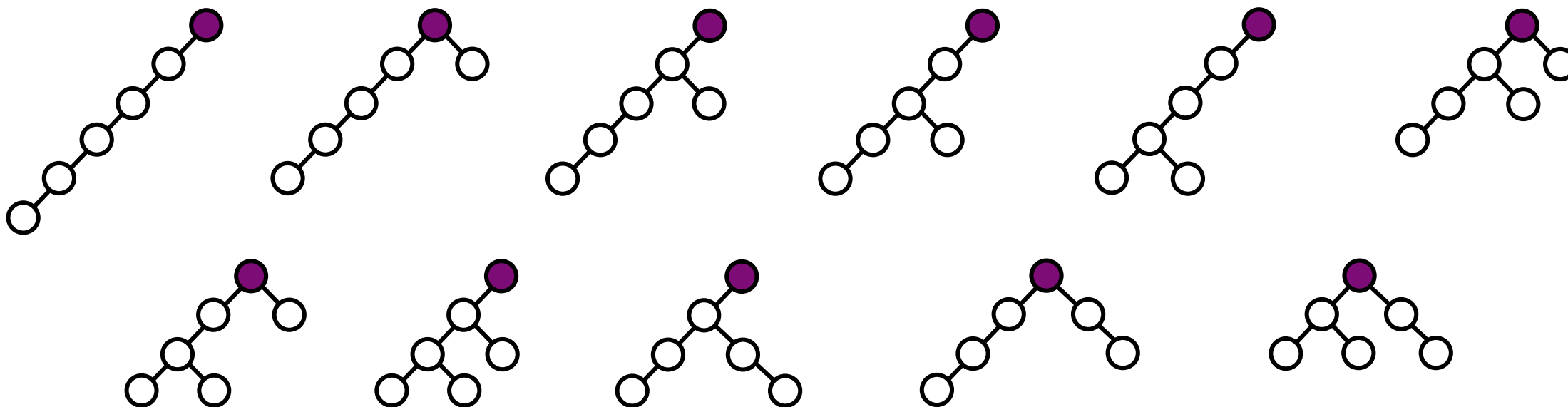
13 頂点の木の構成は

左部分木 6 + 中央 1 + 右部分木 6



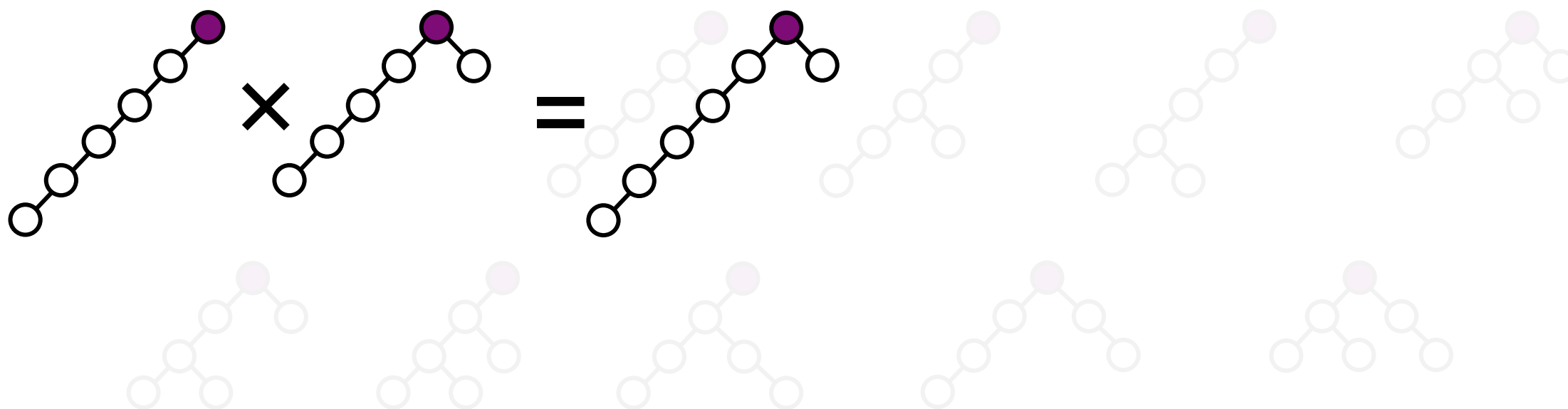
そこで、6 頂点の根付き二分木は全部で 11 個ある

※このことから、木の個数が $12 \times 11 \div 2 = 66$ 個と計算できる



しかしながら、頂点を 1 個追加すると 2 つの木を合成できる

※ サイズ 7 の二分木は、サイズ 6 の二分木を両方含む



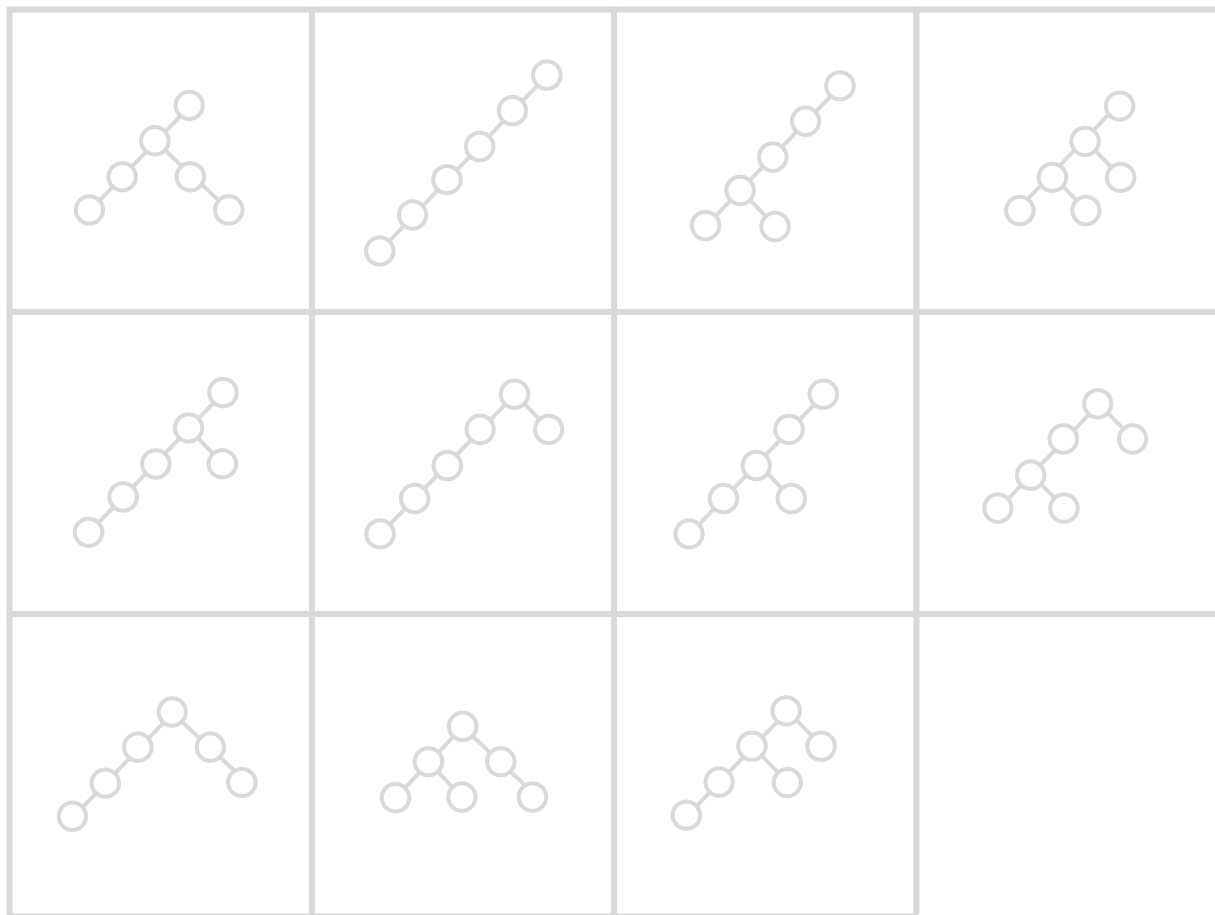


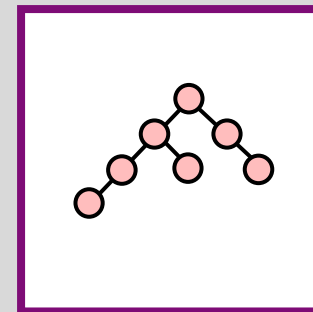
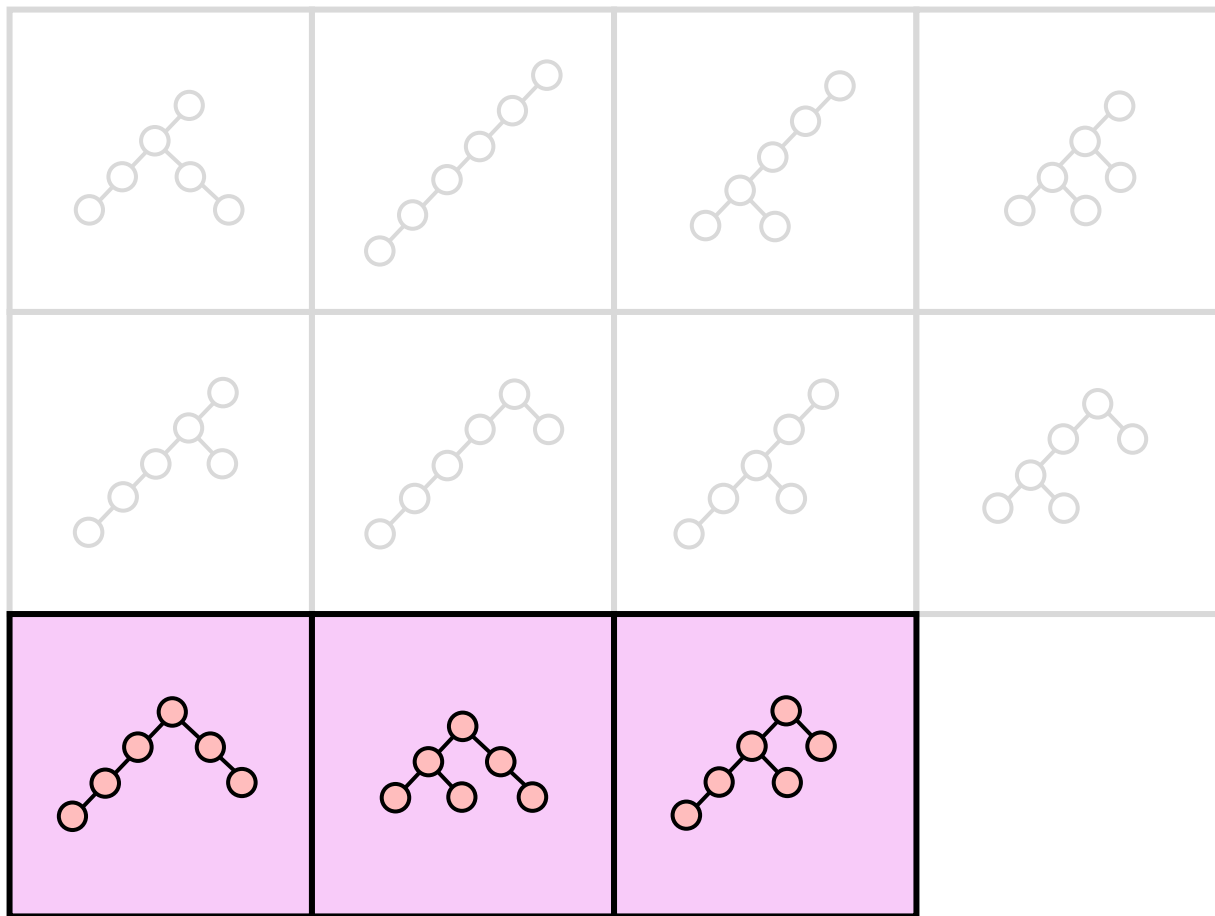
しかしながら、頂点を 1 個追加すると 2 つの木を合成できる

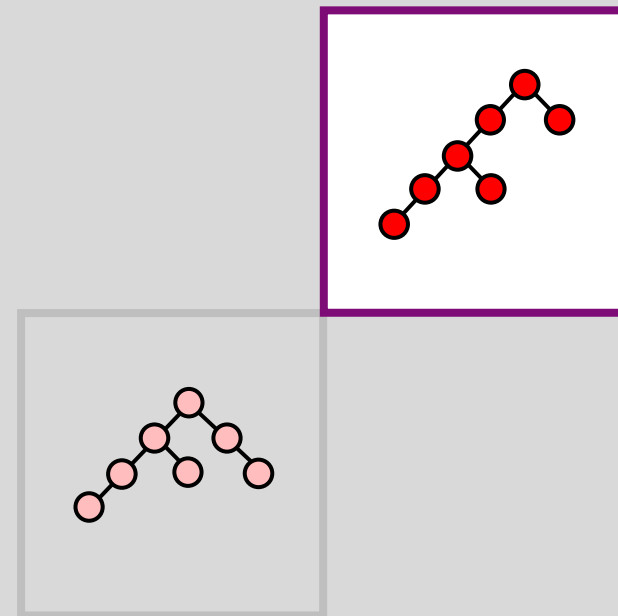
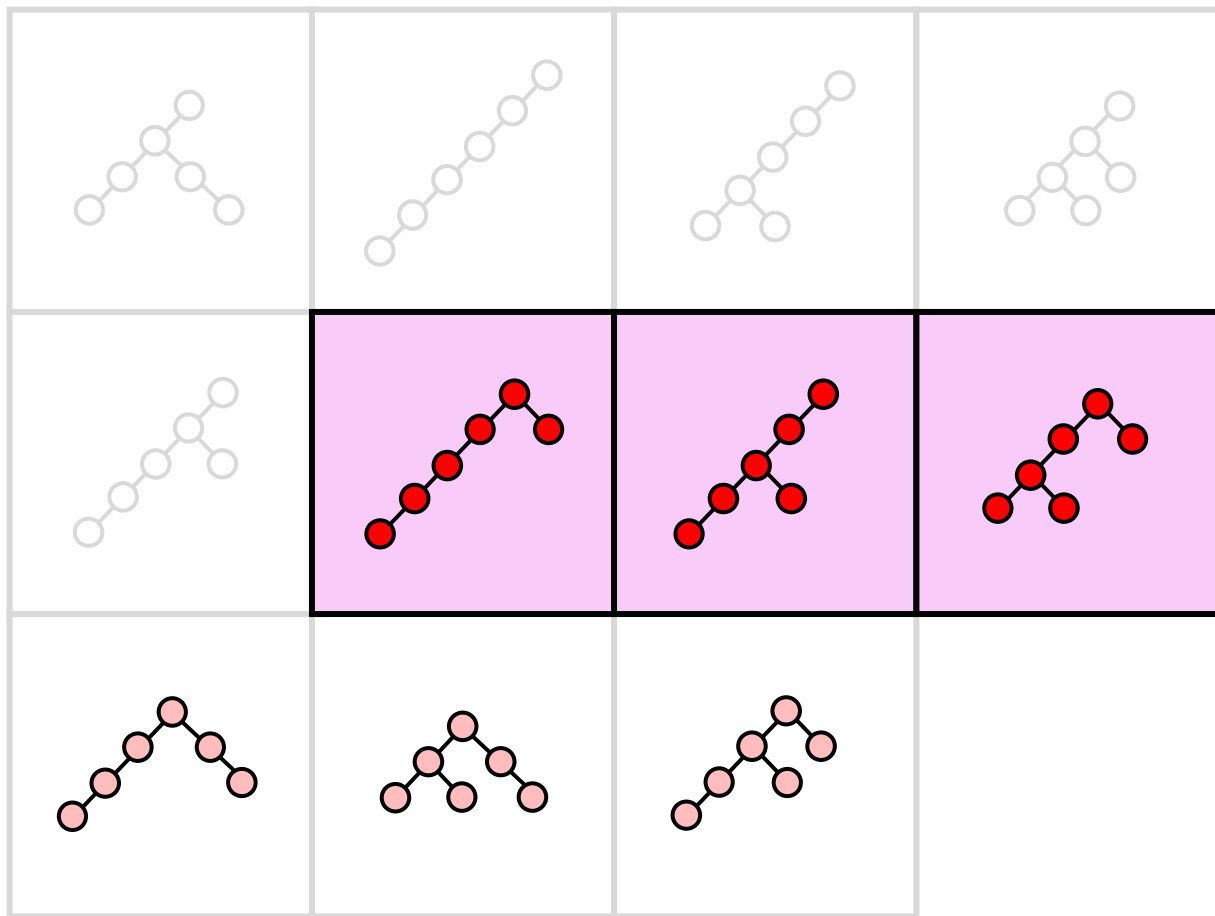
※ サイズ 7 の二分木は、サイズ 6 の二分木を両方含む

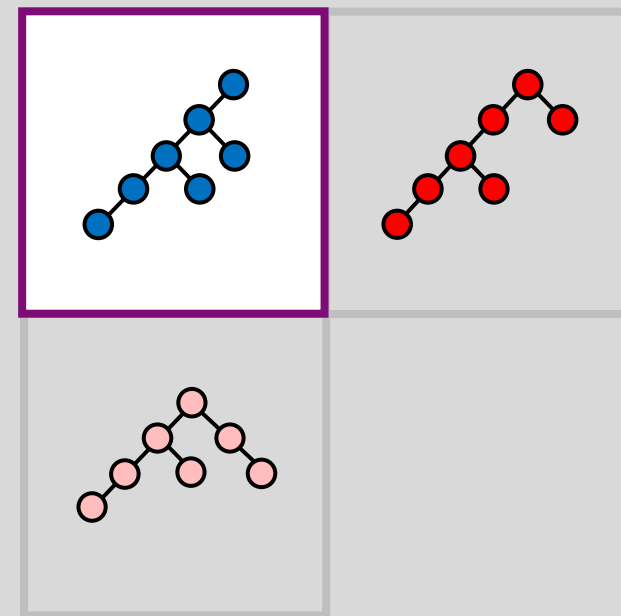
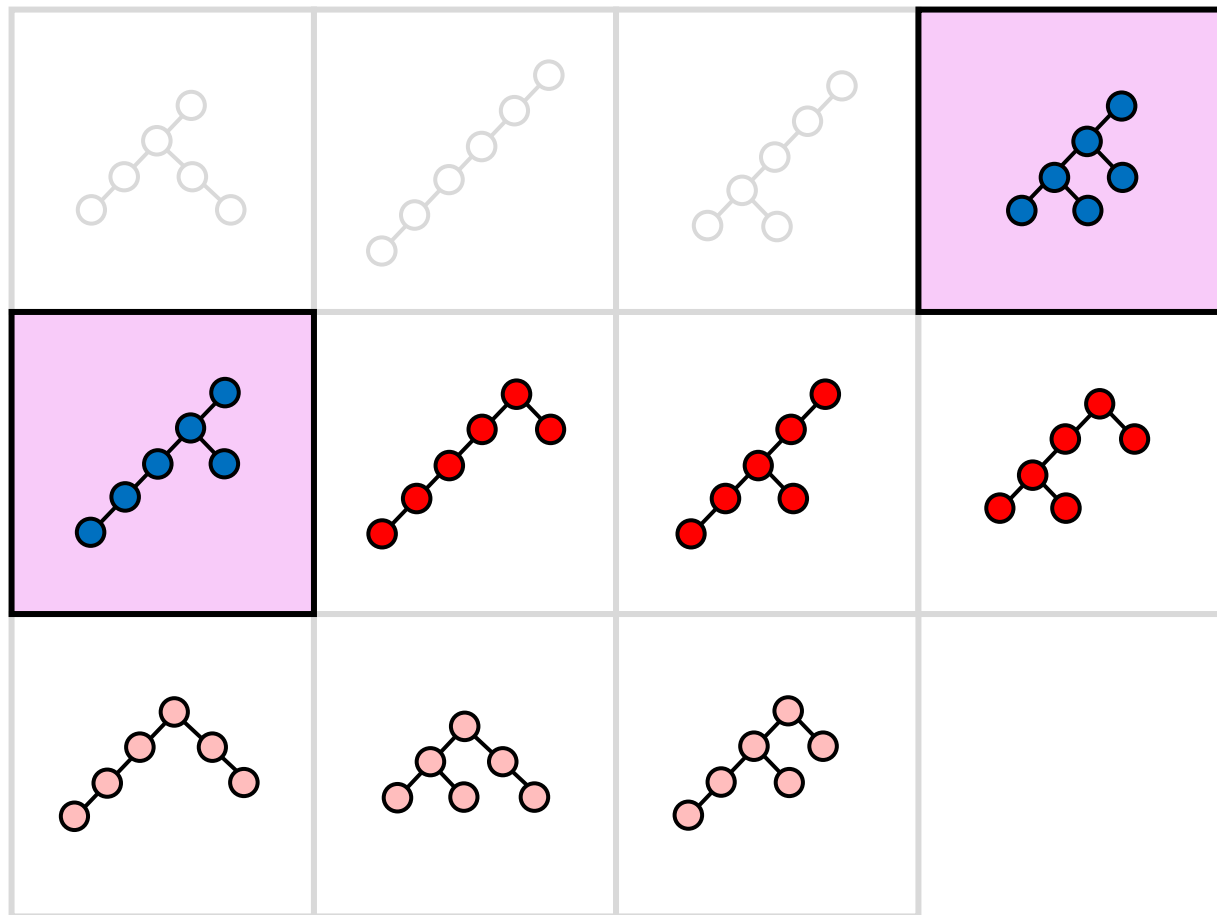
どう合成すれば

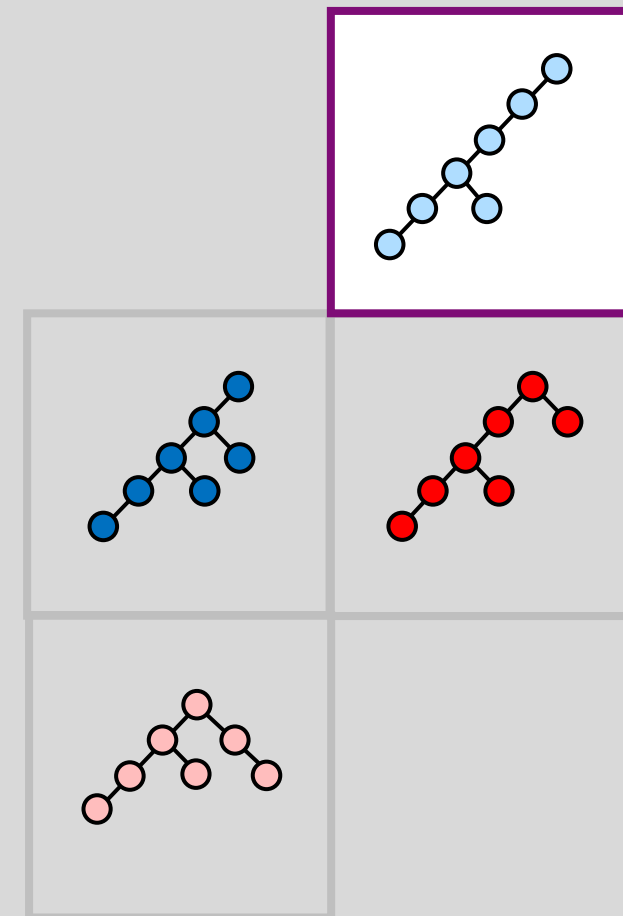
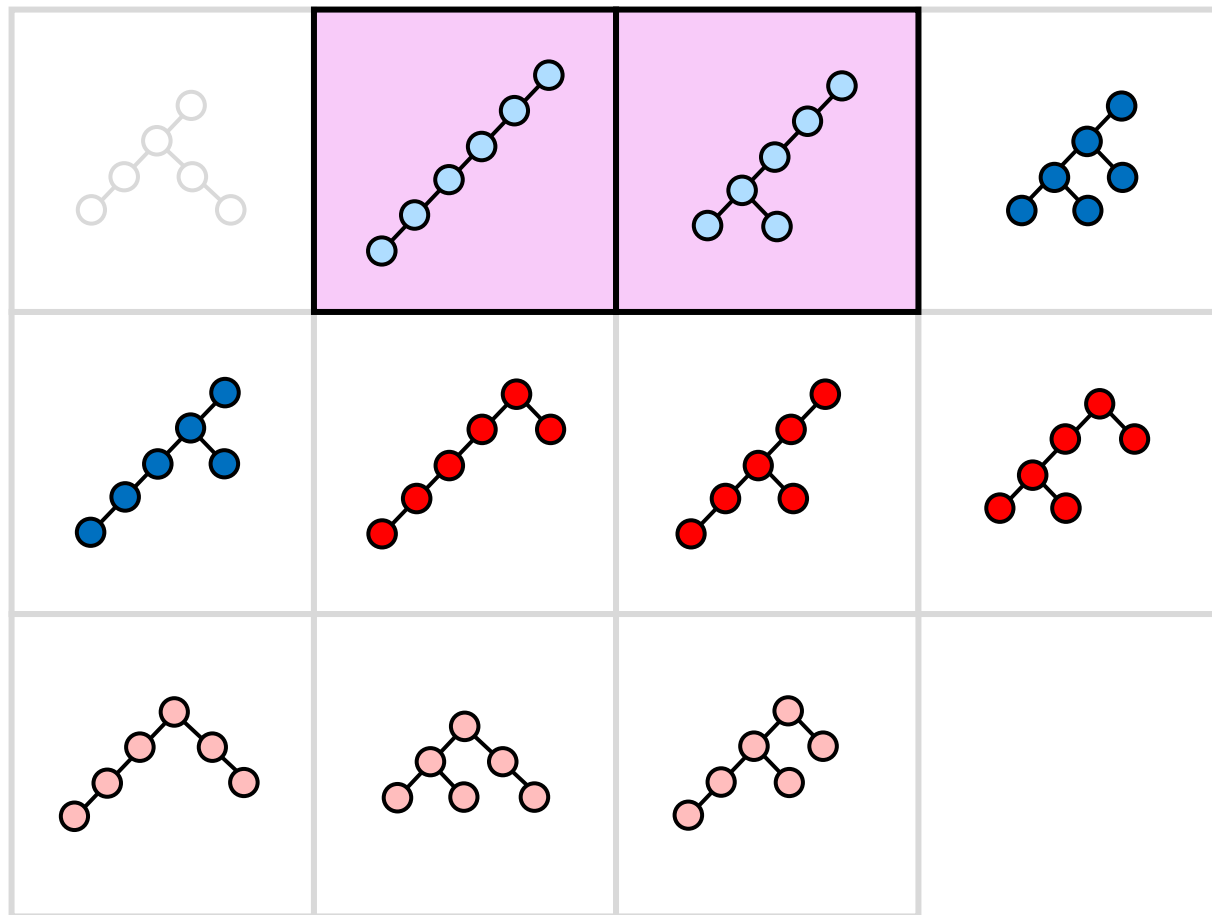
木の個数が少なくなるのか？

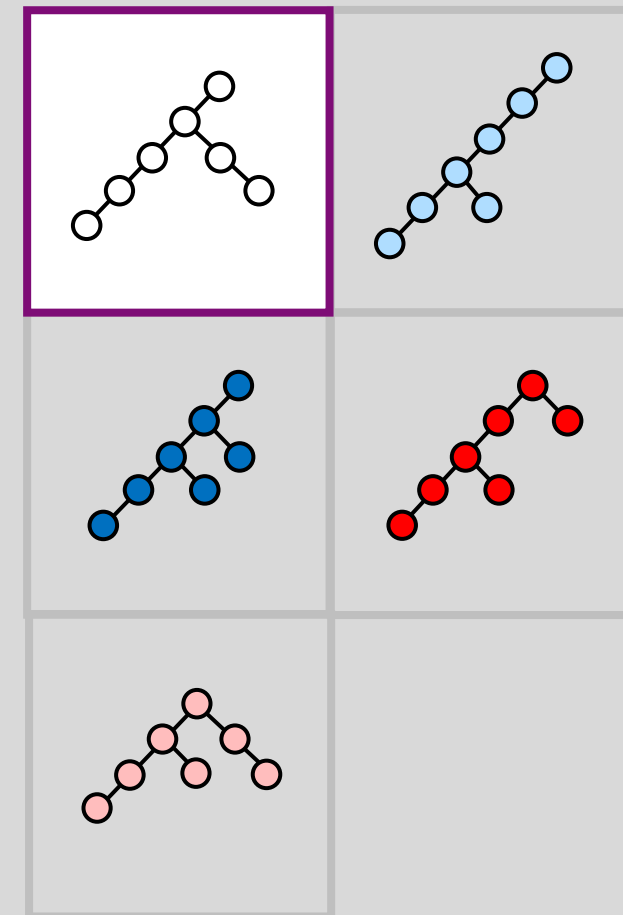
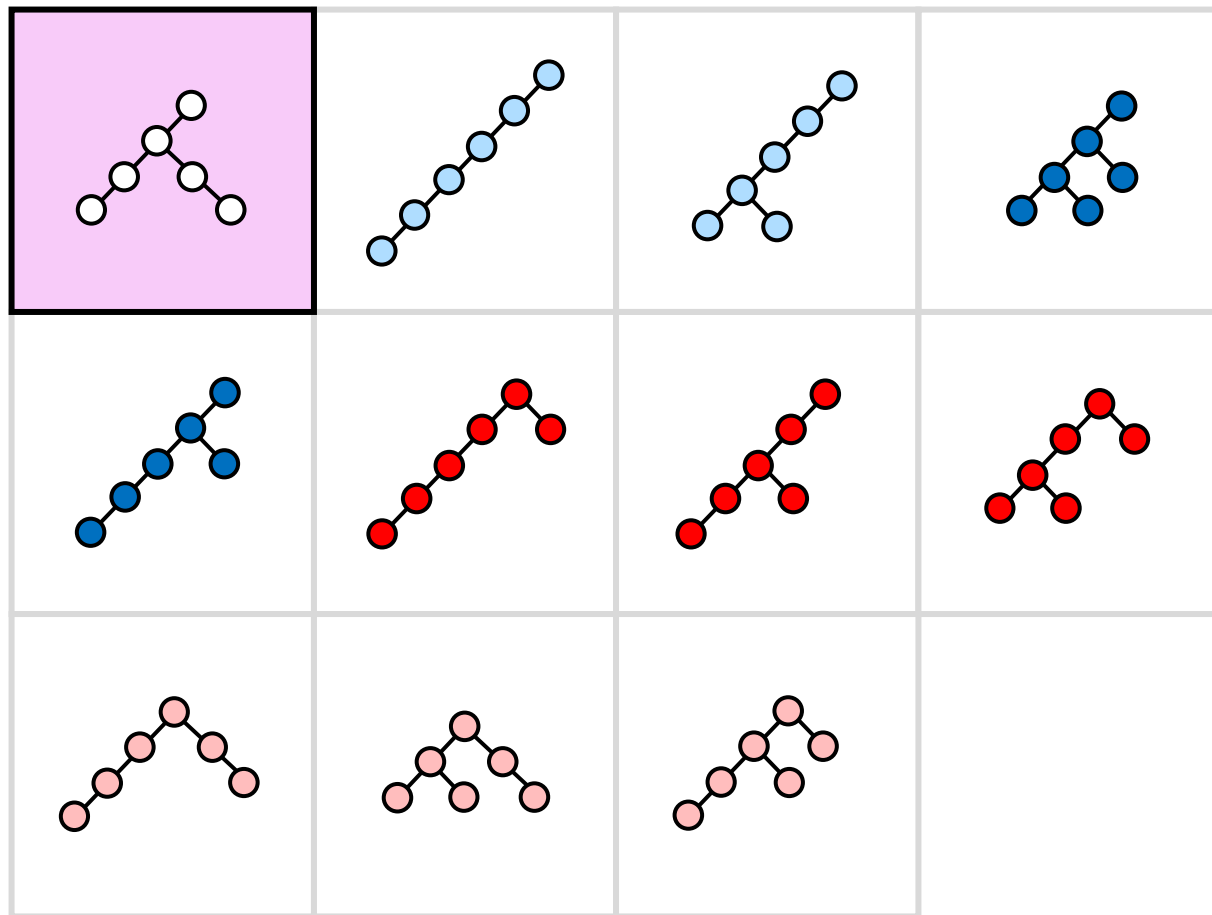














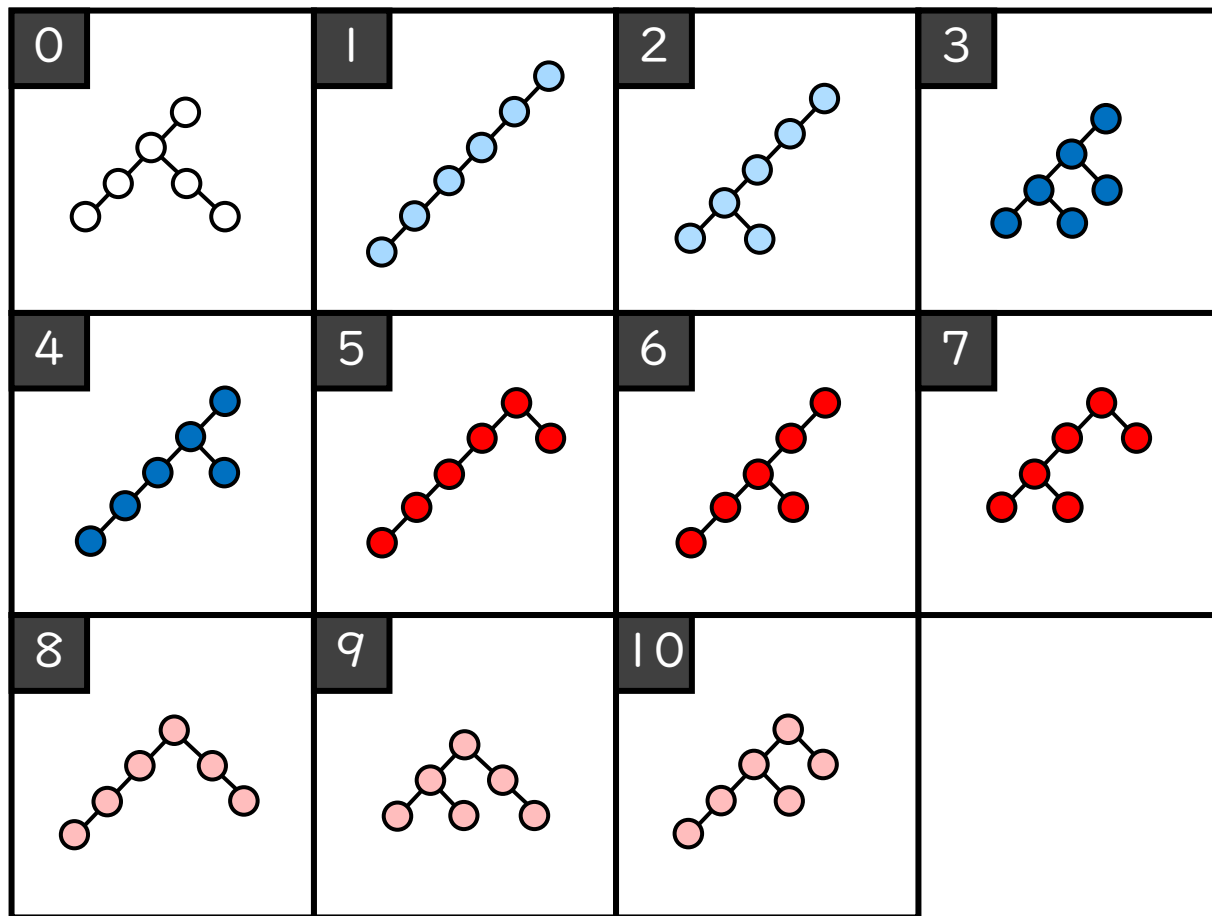
11 通りあった木が
5 通りに減った！

```
graph TD; Root(( )) --- L1L(( )); Root --- L1R(( )); L1L --- L2L(( )); L1L --- L2R(( )); L1R --- L2RL(( )); L1R --- L2RR(( )); L2L --- L3LL(( )); L2L --- L3LR(( )); L2R --- L3RL(( )); L2R --- L3RR(( )); L3LL --- L4LLL(( ))
```

5 × 11 = 55通り

左部分木 右部分木

に減ったように思える



しかし実際はもっと減る

白色の二分木から順に 0, 1, 2, ..., 10
の番号を付けて、番号の大きい方に
頂点を追加すると...



木の数は **28** 通り

$$\text{※ } 5+4+4+3+3+2+2+2+1+1+1=28$$



この方針で距離分布を送ると、一部重複が出るのでパターン数はもっと減る（次の通り）。

端点が根 22通り

全部で 304通り



$$22 \times (304 + 22) \times 5000$$

$$= 3586 \text{ 万通り}$$

2^{25} は約 3355 万なので

惜しくも 25 ビットになってしまう…



この方針で距離分布を送ると、一部重複が出るのでパターン数はもっと減る（次の通り）。

25 ビット → 99 点

端点が見え

22 通り

全部で

304 通り

$$2^2 \times (304 + 12) \times 5000 = 3586 \text{ 万通り}$$

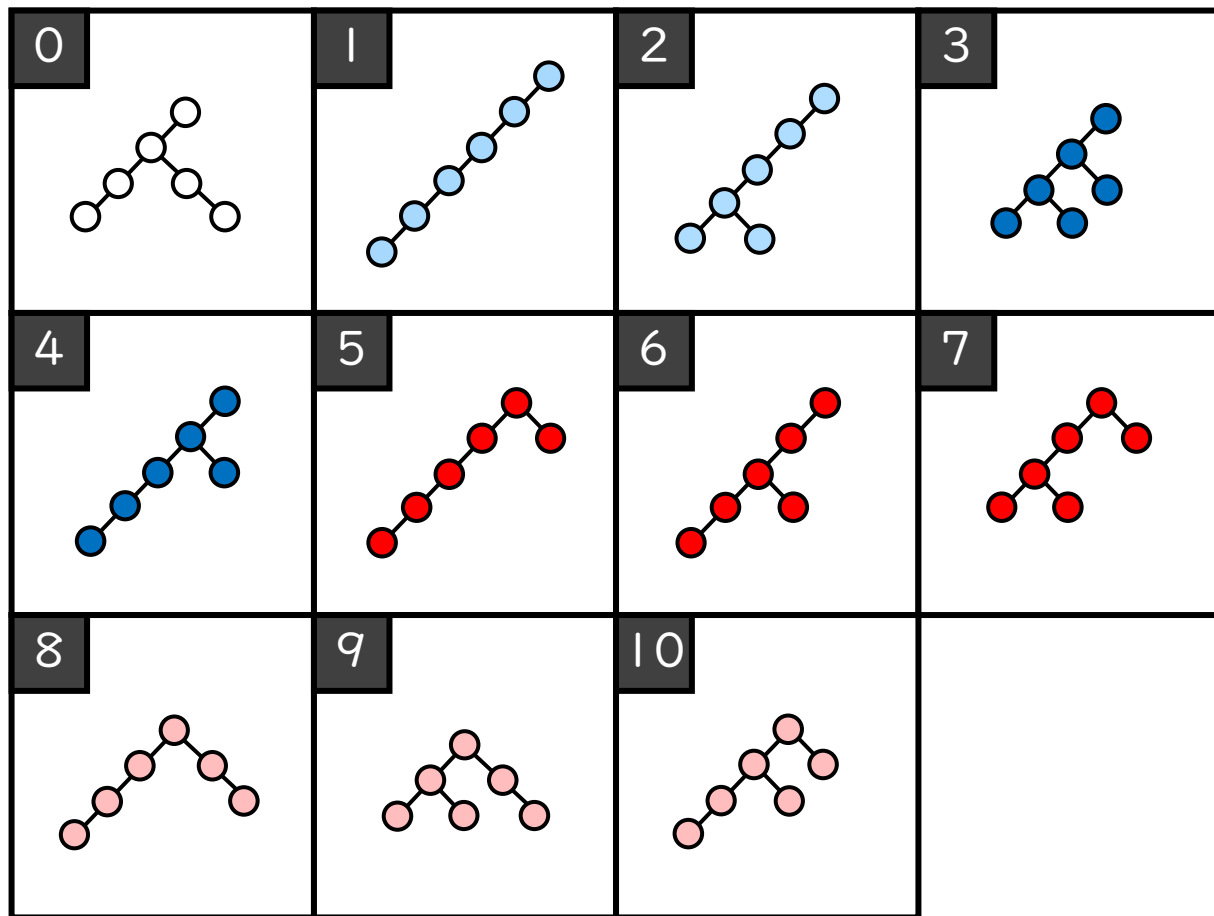
2^{25} は約 3355 万なので

惜しくも 25 ビットになってしまう…



最後の一押し

番号の振り方を変えよう！



端点が根 22通り

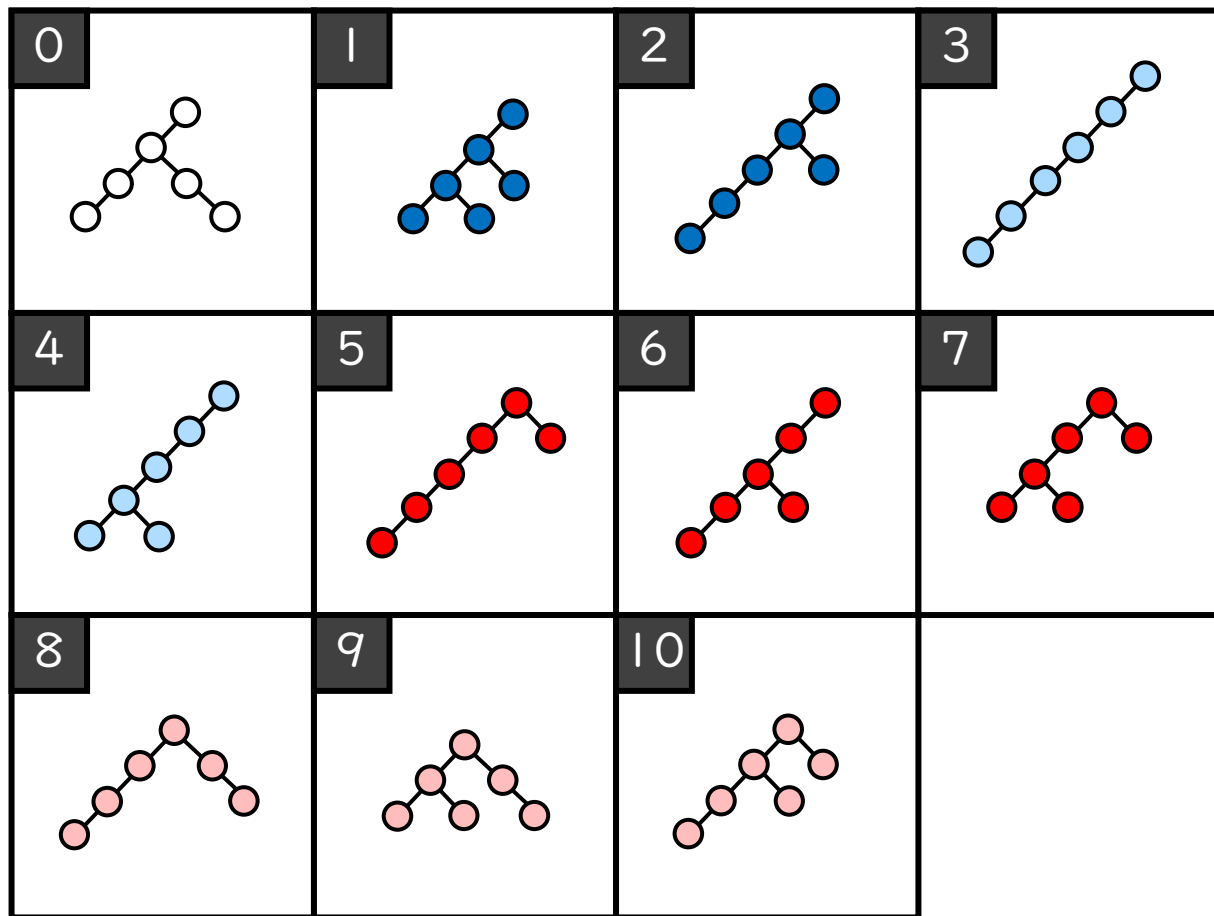
全部で 304通り



識別すべきパターン数は

3586万通り

25ビット



端点が根 22通り

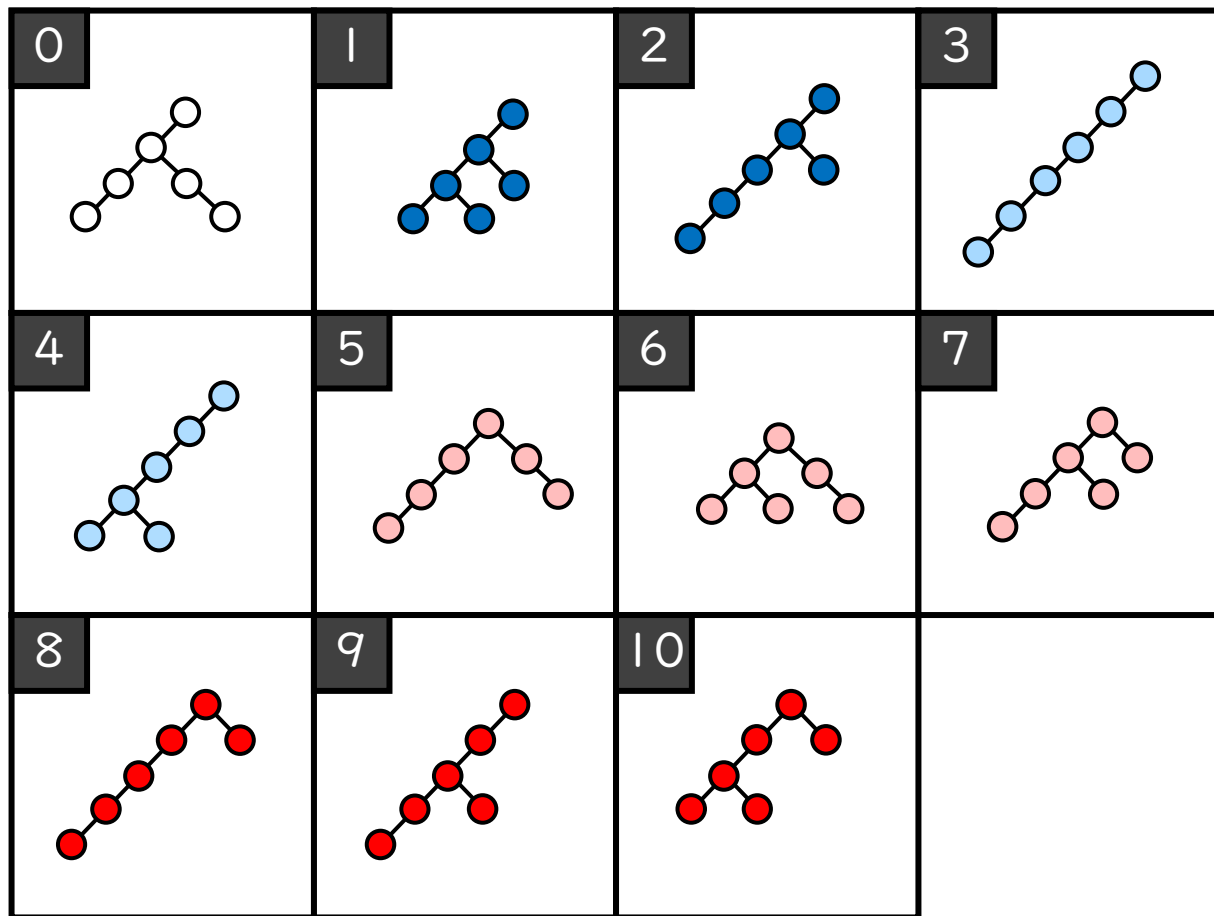
全部で 296通り



識別すべきパターン数は

3498万通り

25ビット



端点が根 21通り

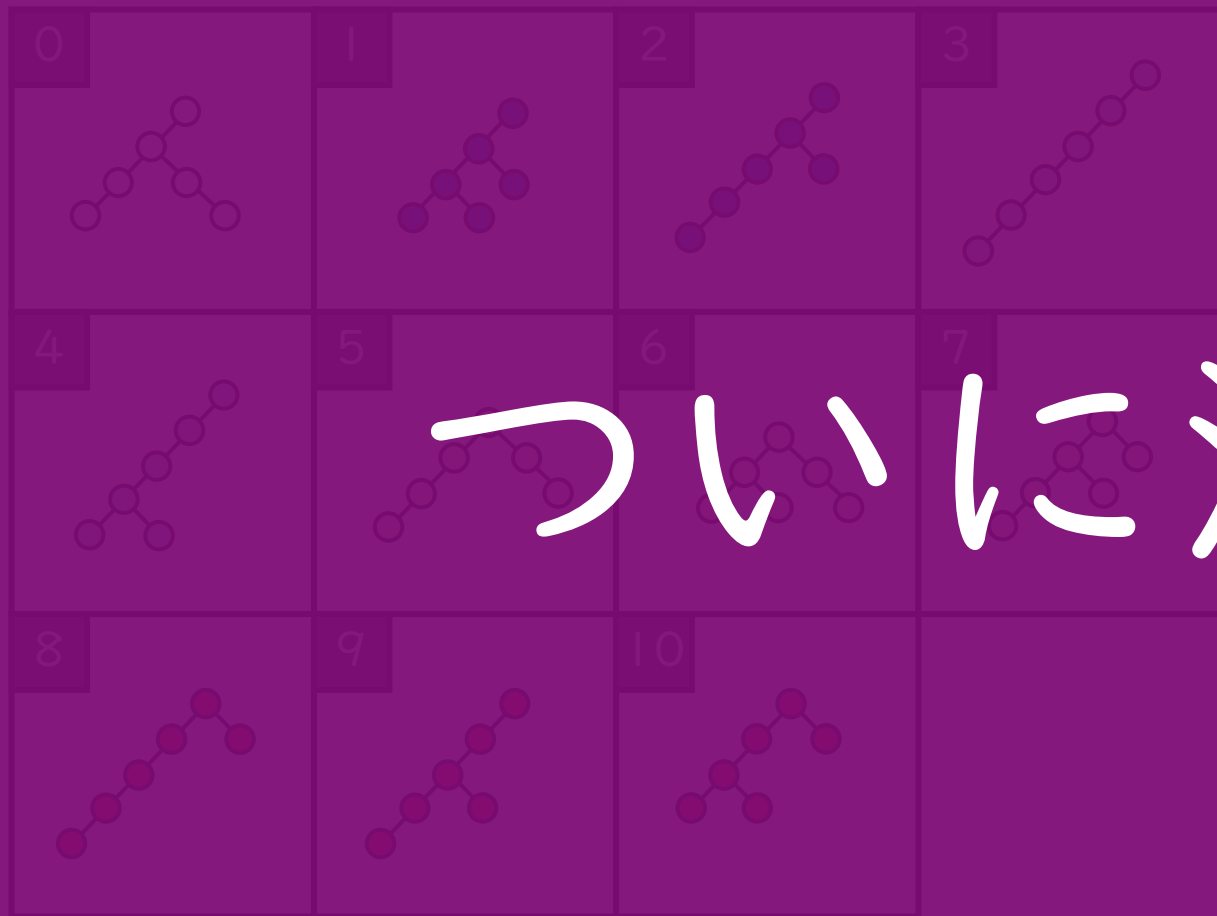
全部で 290通り



識別すべきパターン数は

3265万通り

24ビット



端点が根

21通り

全部で

290通り

識別すべきパターン数は

3265万通り

24ビット

解法のまとめ

- 木を連結なグループに分解することを考える
 - 下から貪欲に切っていくことで、根以外の全グループの頂点数を 7～13 にできる
 - グループ数は 1429 以下なので、遊園地・温泉の順序が関係ないことを使えば、Benjamin は 20 ビットで送れる



ここまで基礎点 59 点

さらなる改善として、次のようなものが考えられる

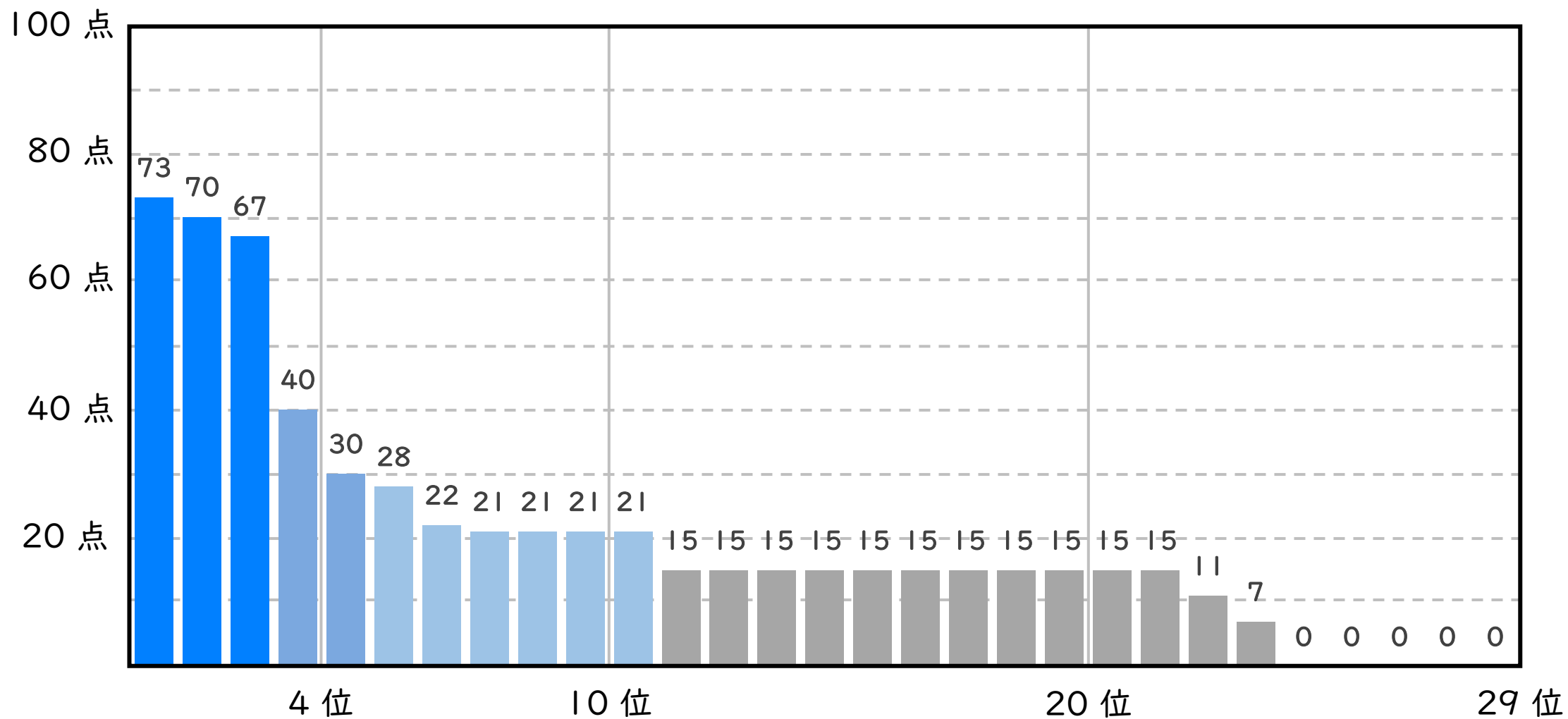
ステップ	改善の内容	ビット数	得点
1	N 頂点の木を $2N$ ビットで送る	70	67 点
2	木が高々「カタラン数」通りであることを使う	60	72 点
3	次数 ≤ 3 などの条件を追加してパターン数を減らす	44	80 点
4	同型な木を同一視してパターン数を減らす	40	84 点
5	接点 u, v のうち片方しか送らなくて良い	36	88 点
6	すべてのグループを 13 頂点にする	32	92 点
7	4 つの要素をすべてまとめる	30	94 点

さらなる改善として、次のようなものが考えられる

ステップ	改善の内容	ビット数	得点
8	1 ~ L ビット全部利用できることに気づく	29	95 点
9	木の代わりに距離分布を送る	27	97 点
10	根を「木の中心」付近にする	26	98 点
11	上手く追加して各グループ 14 頂点にする	24	100 点



- この問題は、少なくとも昨年度までのどの JOI 過去問よりも難易度が高いと思います
- 満点を取るのは非常に難しいので、**部分点**を積極的に狙う、ほかの問題に手を付けるなどの戦略が有効です





チーム戦 (Team Contest)

JOI / JOIG 2022 春合宿 Day 2 解説

東京大学 1 年 米田寛峻 (square1001)

はじめに

2 / 54

Q. 競プロで重要となる能力は何だと思いますか？

1



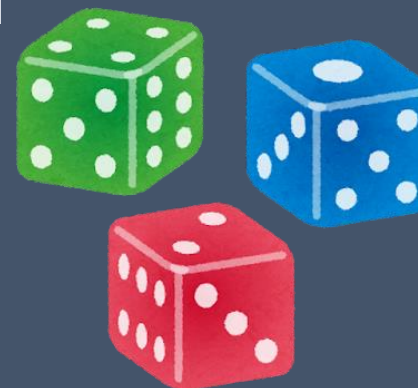
考察力

2



実装力

3



運

問題概要

3 / 54

1 N 人の選手がいます

選手 i ($1 \leq i \leq N$) の考察力は x_i 、実装力は y_i 、運は z_i



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 5
運 = 5



考察力 = 4
実装力 = 4
運 = 2



考察力 = 5
実装力 = 2
運 = 3

問題概要

4 / 54

2 この中で 3 人の代表選手を選びたいです



考察力 = 3
実装力 = 1
運 = 4



考察力 = 4
実装力 = 4
運 = 2



考察力 = 5
実装力 = 2
運 = 3

問題概要

5 / 54

3 このとき、チームの全員に長所があるようにしたいです
(つまり、全員に「他の 2 人より値が大きい」能力がある)



考察力 = 3
実装力 = 1
運 = 4



考察力 = 4
実装力 = 4
運 = 2



考察力 = 5
実装力 = 2
運 = 3

問題概要

6 / 54

4 (考察力の最大値) + (実装力の最大値) + (運の最大値) は最大いくつにできますか？



考察力 = 3
実装力 = 1
運 = 4



考察力 = 4
実装力 = 4
運 = 2



考察力 = 5
実装力 = 2
運 = 3

考察力 5 + 実装力 4 + 運 4
= 総合力 13

各小課題の制約

7 / 54

本解説では [1] → [3] → [4] → [5] → [2] → [7] の順に解説します

	配点	N の上限	X_i, Y_i, Z_i の上限
小課題 1	10 点	300	-
小課題 2	25 点	4,000	-
小課題 3	10 点	150,000	5
小課題 4	10 点	150,000	20
小課題 5	10 点	150,000	300
小課題 6	10 点	150,000	4,000
小課題 7	25 点	150,000	-

CONTENTS

8 / 54



はじめに・問題概要



小課題 1 $[N \leq 300]$ の解説



小課題 3 $[X_i, Y_i, Z_i \leq 5]$ の解説



小課題 4 $[X_i, Y_i, Z_i \leq 20]$ の解説



小課題 5 $[X_i, Y_i, Z_i \leq 300]$ の解説



小課題 2 $[N \leq 4000]$ の解説・満点解法に向けて



満点解法 $[N \leq 150000]$ の解説



おまけ

小課題 1 — $N \leq 300$

9 / 54

$N \leq 300$ のように制約が小さい場合…

全探索

をまず考えよう！

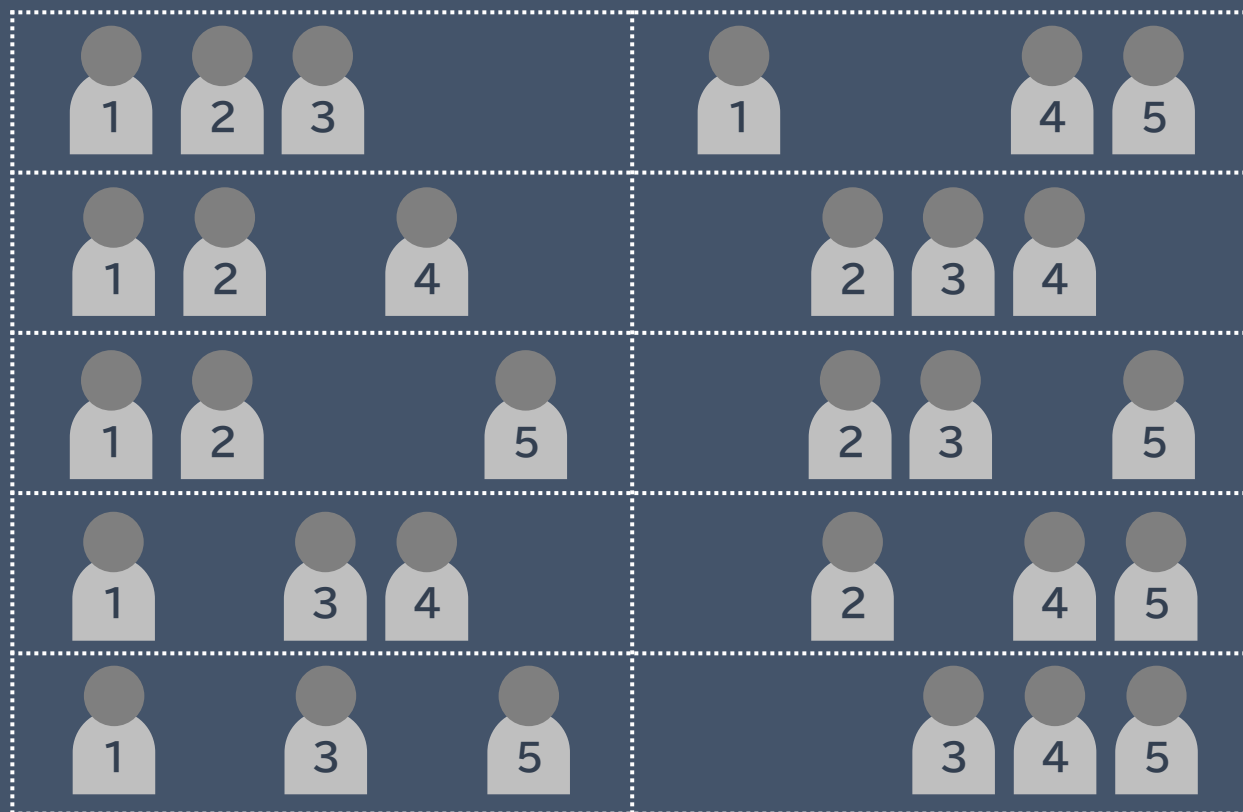
小課題 1 — $N \leq 300$

10 / 54

3 人の選び方は ${}_NC_3 = N(N-1)(N-2)/6$ 通り

これを全パターン試します

→ $N = 300$ でも 450 万通り
十分高速！



全探索の実装

11 / 54

具体的な実装について考えてみよう

選んだ 3 人が選手 i, j, k だったとする

- 選手 i に長所がある: $X_i > \max(X_j, X_k)$ または $Y_i > \max(Y_j, Y_k)$ または $Z_i > \max(Z_j, Z_k)$
- 選手 j に長所がある: $X_j > \max(X_k, X_i)$ または $Y_j > \max(Y_k, Y_i)$ または $Z_j > \max(Z_k, Z_i)$
- 選手 k に長所がある: $X_k > \max(X_i, X_j)$ または $Y_k > \max(Y_i, Y_j)$ または $Z_k > \max(Z_i, Z_j)$
- 総合力 = $\max(X_i, X_j, X_k) + \max(Y_i, Y_j, Y_k) + \max(Z_i, Z_j, Z_k)$

このように状況を整理して、実装してみよう！

全探索の実装 (1 ページ目)

12 / 54

```
#include <iostream>
#include <algorithm>
using namespace std;

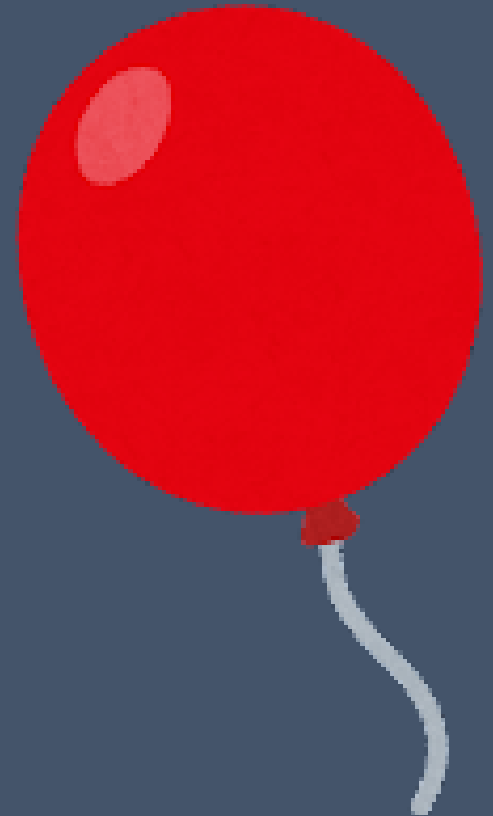
int N, X[309], Y[309], Z[309];

int get_score(int i, int j, int k) {
    if (!(X[i] > max(X[j], X[k]) || Y[i] > max(Y[j], Y[k]) || Z[i] > max(Z[j], Z[k]))) {
        return -1;
    }
    if (!(X[j] > max(X[k], X[i]) || Y[j] > max(Y[k], Y[i]) || Z[j] > max(Z[k], Z[i]))) {
        return -1;
    }
    if (!(X[k] > max(X[i], X[j]) || Y[k] > max(Y[i], Y[j]) || Z[k] > max(Z[i], Z[j]))) {
        return -1;
    }
    return max(X[i], X[j], X[k]) + max(Y[i], Y[j], Y[k]) + max(Z[i], Z[j], Z[k]);
}
```


全探索の実装 (2 ページ目)

13 / 54

```
int main() {  
    cin >> N;  
    for (int i = 0; i < N; i++) {  
        cin >> X[i] >> Y[i] >> Z[i];  
    }  
    int answer = -1;  
    for (int i = 0; i < N; i++) {  
        for (int j = i + 1; j < N; j++) {  
            for (int k = j + 1; k < N; k++) {  
                answer = max(answer, get_score(i, j, k));  
            }  
        }  
    }  
    cout << answer << endl;  
}
```



CONTENTS

14 / 54



はじめに・問題概要



小課題 1 [$N \leq 300$] の解説



小課題 3 [$X_i, Y_i, Z_i \leq 5$] の解説



小課題 4 [$X_i, Y_i, Z_i \leq 20$] の解説



小課題 5 [$X_i, Y_i, Z_i \leq 300$] の解説



小課題 2 [$N \leq 4000$] の解説・満点解法に向けて



満点解法 [$N \leq 150000$] の解説



おまけ

小課題 3 — $X_i, Y_i, Z_i \leq 5$

15 / 54

X_i, Y_i, Z_i の値が非常に小さいです

N が大きければ、「考察力・実装力・運がすべて同じ」人がたくさん出てきます

重要な性質

「考察力・実装力・運」がすべて同じ 2 人は
同じチームには入れない

理由

その 2 人には「自分だけの長所」が 1 つもないから

小課題 3 — $X_i, Y_i, Z_i \leq 5$

16 / 54

X_i, Y_i, Z_i の値が非常に小さいです

N が大きい時に「考察力・実装力・運」がすべて同じ人はいくつか出てきます

「全能力が同じ」人が複数いれば

1人にまとめても良い！

重要な性質

「考察力・実装力・運」がすべて同じ2人は
同じチームには入れない

理由

その 2 人には「自分だけの長所」が 1 つもないから

小課題 3 — $X_i, Y_i, Z_i \leq 5$

17 / 54

要するにということです
「同じものは 1 つ以外（探索に）
必要ない！」



考察力 = 1
実装力 = 2
運 = 5



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 2
運 = 5



考察力 = 3
実装力 = 1
運 = 2



考察力 = 1
実装力 = 2
運 = 5



すると、選手が $5^3 = 125$ 人以下になる

→ 計算量 $O(N^3)$ の全探索ができる！

(計算量は $O(MAX^9)$ になる)



考察力 = 1
実装力 = 2
運 = 5



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 2
運 = 5



考察力 = 3
実装力 = 1
運 = 2



考察力 = 1
実装力 = 2
運 = 5

CONTENTS

18 / 54



はじめに・問題概要



小課題 1 [$N \leq 300$] の解説



小課題 3 [$X_i, Y_i, Z_i \leq 5$] の解説



小課題 4 [$X_i, Y_i, Z_i \leq 20$] の解説



小課題 5 [$X_i, Y_i, Z_i \leq 300$] の解説



小課題 2 [$N \leq 4000$] の解説・満点解法に向けて



満点解法 [$N \leq 150000$] の解説



おまけ

小課題 4 — $X_i, Y_i, Z_i \leq 20$

19 / 54

計算量を良くするためには

全探索の工夫

が重要になることもある

小課題 4 — $X_i, Y_i, Z_i \leq 20$

20 / 54

次のような「工夫した全探索」を考えましょう

工夫

考察力の最大値 a 、実装力の最大値 b 、運の最大値 c を全探索

小課題 4 — $X_i, Y_i, Z_i \leq 20$

21 / 54

(a, b, c) が決まると、以下の 3 人でチームを組むことになります

選手 i



「考察力」重点

$$X_i = a, Y_i < b, Z_i < c$$

選手 j



「実装力」重点

$$X_i < a, Y_i = b, Z_i < c$$

選手 k



「運」重点

$$X_i < a, Y_i < b, Z_i = c$$

小課題 4 — $X_i, Y_i, Z_i \leq 20$

22 / 54

この 3 条件を満たす選手が 1 人ずついれば「この (a, b, c) でチームが作れる」
(このときの総合力は $a + b + c$ と決まる)

選手 i



「考察力」重点

$$X_i = a, Y_i < b, Z_i < c$$

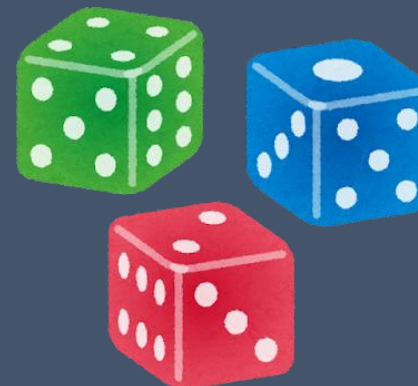
選手 j



「実装力」重点

$$X_i < a, Y_i = b, Z_i < c$$

選手 k



「運」重点

$$X_i < a, Y_i < b, Z_i = c$$

小課題 4 — $X_i, Y_i, Z_i \leq 20$

23 / 54

すると、この問題が計算量 $O(MAX^6)$ で解ける

- 1 (a, b, c) の組み合わせ MAX^3 通りを全探索する
- 2 3 人の選手を探すのに $O(N)$ かけると、全体計算量は $O(MAX^3 \times N)$ になる
- 3 しかし、全部同じ能力の選手をまとめると $N \leq MAX^3$ になるので全体計算量が $O(MAX^6)$ になる

これで小課題 4 に AC !



CONTENTS

24 / 54



はじめに・問題概要



小課題 1 [$N \leq 300$] の解説



小課題 3 [$X_i, Y_i, Z_i \leq 5$] の解説



小課題 4 [$X_i, Y_i, Z_i \leq 20$] の解説



小課題 5 [$X_i, Y_i, Z_i \leq 300$] の解説



小課題 2 [$N \leq 4000$] の解説・満点解法に向けて



満点解法 [$N \leq 150000$] の解説



おまけ

小課題 5 — $X_i, Y_i, Z_i \leq 300$

25 / 54

小課題 5 を説明する前に、「累積和」について説明します

累積和とは？

長さ N の配列 $[A_1, A_2, \dots, A_N]$ が与えられる
区間の合計 $A_l + A_{l+1} + \dots + A_{r-1}$ が計算量 $O(1)$ で求まる

累積和について知りたい方はこちら

- 「アルゴリズム×数学」p. 140～144 など

(JOIG 2022 本選 3「投票 (Voting)」を解いている方は知っているはず)

小課題 5 — $X_i, Y_i, Z_i \leq 300$

26 / 54

実は、累積和の 2 次元バージョン「2 次元累積和」もあります

1	0	1	2	0
2	1	1	0	1
1	2	1	2	0
0	1	1	2	0
1	0	1	2	1

この範囲の値の合計は $1+1+0+2+1+2+1+1+2=11$

こういうのを計算量 $O(1)$ で求めたい

小課題 5 — $X_i, Y_i, Z_i \leq 300$

27 / 54

ステップ 1

まず、横方向に累積和をとる

1	→	0	→	1	→	2	→	0
2	→	1	→	1	→	0	→	1
1	→	2	→	1	→	2	→	0
0	→	1	→	1	→	2	→	0
1	→	0	→	1	→	2	→	1



1	1	2	4	4
2	3	4	4	5
1	3	4	6	6
0	1	2	4	4
1	1	2	4	5

小課題 5 — $X_i, Y_i, Z_i \leq 300$

28 / 54

ステップ 2

次に、縦方向に累積和をとる

1	1	2	4	4
↓	↓	↓	↓	↓
2	3	4	4	5
↓	↓	↓	↓	↓
1	3	4	6	6
↓	↓	↓	↓	↓
0	1	2	4	4
↓	↓	↓	↓	↓
1	1	2	4	5



1	1	2	4	4
3	4	6	8	9
4	7	10	14	15
4	8	12	18	19
5	9	14	22	24

小課題 5 — $X_i, Y_i, Z_i \leq 300$

29 / 54

ステップ 3

すると、4 つの値の足し算・引き算で「範囲の値の合計」が求まる

1	1	2	4	4
3	4	6	8	9
4	7	10	14	15
4	8	12	18	19
5	9	14	22	24

$$18 + 1 - 4 - 4 = 11$$

小課題 5 — $X_i, Y_i, Z_i \leq 300$

30 / 54

ステップ 3

すると、4 つの値の足し算・引き算で「範囲の値の合計」が求まる

実は、これと同じように

「3次元累積和」もできる

1	1	2	4	4
3	4	6	8	9
4	7	10	14	15
4	8	12	18	19
5	9	14	22	24

$$18 + 1 - 4 - 4 = 11$$

小課題 5 — $X_i, Y_i, Z_i \leq 300$

31 / 54

ここで「考察力 x 、実装力 y 、運 z の選手の数 $V[x][y][z]$ 」を 3 次元累積和すると…
「考察力重点」「実装力重点」「運重点」にできる選手の数が、それぞれ $O(1)$ で求まる

選手 i



「考察力」重点

$$X_i = a, Y_i < b, Z_i < c$$

選手 j



「実装力」重点

$$X_i < a, Y_i = b, Z_i < c$$

選手 k



「運」重点

$$X_i < a, Y_i < b, Z_i = c$$

小課題 5 — $X_i, Y_i, Z_i \leq 300$

32 / 54

ここで「考察力 x 、実装力 y 、運 z の選手の数 $V[x][y][z]$ 」を 3 次元累積和すると…

「考察力重点」「実装力重点」「運重点」にできる選手の数が、それぞれ $O(1)$ で求まる
すると、各 (a, b, c) に対して

計算量 $O(1)$ で判定できる！

選手 i



「考察力」重点

$$X_i = a, Y_i < b, Z_i < c$$

選手 j



「実装力」重点

$$X_i < a, Y_i = b, Z_i < c$$

選手 k



「運」重点

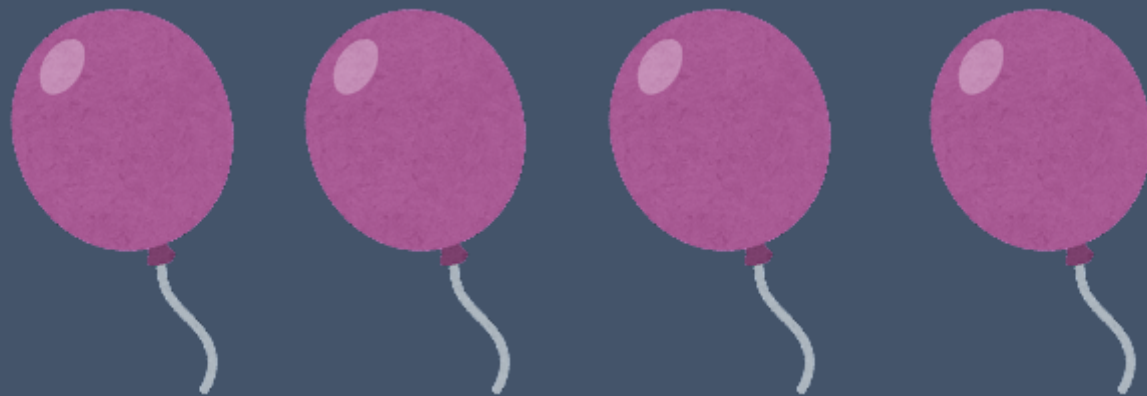
$$X_i < a, Y_i < b, Z_i = c$$

小課題 5 — $X_i, Y_i, Z_i \leq 300$

33 / 54

よって、小課題 4 の解法を「3 次元累積和」で高速化すると
全体計算量が $O(MAX^3)$ になる

→ 小課題 5 に通る！



CONTENTS

34 / 54



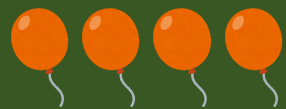
はじめに・問題概要



小課題 1 [$N \leq 300$] の解説



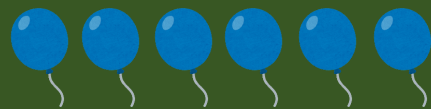
小課題 3 [$X_i, Y_i, Z_i \leq 5$] の解説



小課題 4 [$X_i, Y_i, Z_i \leq 20$] の解説



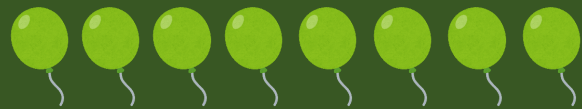
小課題 5 [$X_i, Y_i, Z_i \leq 300$] の解説



小課題 2 [$N \leq 4000$] の解説・満点解法に向けて



満点解法 [$N \leq 150000$] の解説



おまけ

小課題 2 — $N \leq 4000$

35 / 54

注意

ここからは解法がガラッと変わります

この点に注意しておいてください

満点解法への考察

36 / 54

もし「考察力が最大の人」「実装力が最大の人」「運が最大の人」が分かれていたら…？



考察力 = 3
実装力 = 1
運 = 5



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 5
運 = 4



考察力 = 5
実装力 = 4
運 = 2



考察力 = 4
実装力 = 2
運 = 3



明らかに、この 3 人を選ぶのが最適



考察力 = 3
実装力 = 1
運 = 5



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 5
運 = 4



考察力 = 5
実装力 = 4
運 = 2



考察力 = 4
実装力 = 2
運 = 3

満点解法への考察

37 / 54

では、もし分かれていなかったら？



「最大値のスキル」を 2 つ持ってる人は、チームには入れられない！

(理由は、他の 2 人の選手が合計 1 個しか長所を持てなくなるから)



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3

満点解法への考察

38 / 54

1 人消えたので、残った 4 人の中で考えると…？



「最大値のスキル」を 2 つ持ってる人は、チームには入れられない！

(理由は、他の 2 人の選手が合計 1 個しか長所を持てなくなるから)



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3

満点解法への考察

39 / 54

また 1 人消えたので、残った 3 人
の中で考えると…？



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3



「最大値のスキル」が分かれてる
ので、この組み方が最適！



考察力 = 3
実装力 = 1
運 = 4



考察力 = 2
実装力 = 3
運 = 1



考察力 = 1
実装力 = 4
運 = 5



考察力 = 5
実装力 = 5
運 = 2



考察力 = 4
実装力 = 2
運 = 3

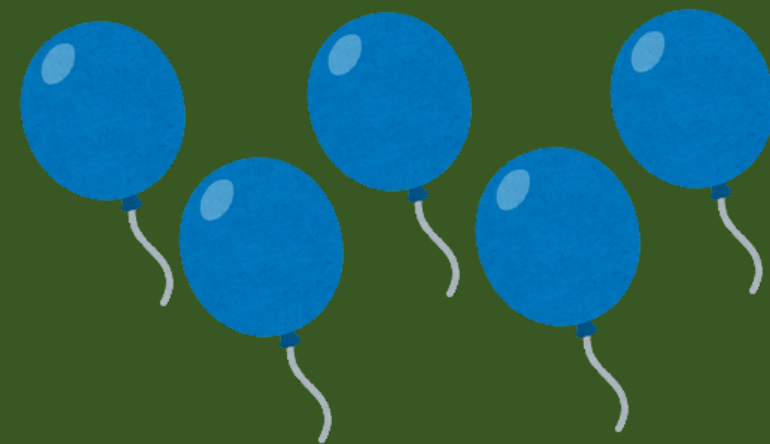
小課題 2 — $N \leq 4000$

40 / 54

このように、以下のようなアルゴリズムで最適解が求められる

- 1 『最大値のスキル』を複数持っている人がいたら消す」ことをできなくなるまで繰り返す
- 2 残った人の中で、最大値のスキルを持つ 3 人でチームを組むのが最適
誰もいなくなったら「チームを組むのが不可能」

各ステップに $O(N)$ かかるので、全体計算量は $O(N^2)$



CONTENTS

41 / 54



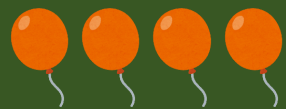
はじめに・問題概要



小課題 1 [$N \leq 300$] の解説



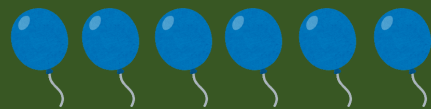
小課題 3 [$X_i, Y_i, Z_i \leq 5$] の解説



小課題 4 [$X_i, Y_i, Z_i \leq 20$] の解説



小課題 5 [$X_i, Y_i, Z_i \leq 300$] の解説



小課題 2 [$N \leq 4000$] の解説・満点解法に向けて



満点解法 [$N \leq 150000$] の解説



おまけ

満点解法 — $N \leq 150000$

42 / 54

先ほどの解法の問題点は、各ステップに $O(N)$ かかっていたことで
全体計算量が $O(N^2)$ になっていたこと

→ これを高速化したい！

満点解法 — $N \leq 150000$

43 / 54

ここで、以下の 3 つのリストを用意しておく

- N 人の選手を考察力 x_i の小さい順に並べたリスト
- N 人の選手を実装力 y_i の小さい順に並べたリスト
- N 人の選手を運 z_i の小さい順に並べたリスト

考察力 x_i

 1	 2	 5	 5	 8
---	---	---	---	---

実装力 y_i

 3	 4	 5	 6	 7
---	---	---	---	---

運 z_i

 2	 2	 3	 8	 9
---	---	---	---	---

現在の最大値:

(8, 7, 9)

満点解法 — $N \leq 150000$

44 / 54

リストの末尾にいる 3 人をチェック

- 青の選手 (5, 4, 9) は「最大値のスキル」が 1 個だけなので OK
- 紫の選手 (8, 7, 2) は「最大値のスキル」が 2 個なので NG

考察力 X_i

 1	 2	 5	 5	 8
---	---	---	---	---

実装力 Y_i

 3	 4	 5	 6	 7
---	---	---	---	---

運 Z_i

 2	 2	 3	 8	 9
---	---	---	---	---

現在の最大値:

(8, 7, 9)

満点解法 — $N \leq 150000$

45 / 54

リストの末尾にいる 3 人をチェック

- 青の選手 (5, 4, 9) は「最大値のスキル」が 1 個だけなので OK
- 紫の選手 (8, 7, 2) は「最大値のスキル」が 2 個なので NG → 全リストから消す

考察力 X_i

 1	 2	 5	 5	 9
---	---	---	---	---

実装力 Y_i

 3	 4	 5	 6	 7
---	---	---	---	---

運 Z_i

 2	 2	 3	 8	 9
---	---	---	---	---

現在の最大値:

(8, 7, 9)

満点解法 — $N \leq 150000$

46 / 54

リストの末尾にいる 3 人をチェック

- 赤の選手 (5, 3, 3)、紫の選手 (1, 6, 2) は「最大値のスキル」が 1 個だけなので OK
- 青の選手 (5, 4, 9) は「最大値のスキル」が 2 個なので NG

考察力 X_i

 1	 2	 5	 5	
---	---	---	---	--

実装力 Y_i

 3	 4	 5	 6	
---	---	---	---	--

運 Z_i

 2		 3	 8	 9
---	--	---	---	---

現在の最大値:

(5, 6, 9)

満点解法 — $N \leq 150000$

47 / 54

リストの末尾にいる 3 人をチェック

- 赤の選手 (5, 3, 3)、紫の選手 (1, 6, 2) は「最大値のスキル」が 1 個だけなので OK
- 青の選手 (5, 4, 9) は「最大値のスキル」が 2 個なので NG → 全リストから消す

考察力 X_i

 1	 2	 5	 5	
---	---	---	---	--

実装力 Y_i

 3	 4	 5	 6	
---	---	---	---	--

運 Z_i

 2		 3	 8	 9
---	--	---	---	---

現在の最大値:

(5, 6, 9)

満点解法 — $N \leq 150000$

48 / 54

リストの末尾にいる 3 人をチェック

- 3 人全員が「最大値のスキル」が 1 個だけなので OK
- よって、総合力の最大値は $5 + 6 + 8 = 19$

考察力 X_i

 1	 2		 5	
---	---	--	---	--

実装力 Y_i

 3		 5	 6	
---	--	---	---	--

運 Z_i

 2		 3	 8	
---	--	---	---	--

現在の最大値:

(5, 6, 8)

満点解法 — $N \leq 150000$

49 / 54

このようにシミュレーションすると、各ステップが $O(1)$ でできる
最初のソートに一番時間がかかるので、全体計算量は $O(N \log N)$

→ 100 点満点が取れる！



CONTENTS

50 / 54



はじめに・問題概要



小課題 1 [$N \leq 300$] の解説



小課題 3 [$X_i, Y_i, Z_i \leq 5$] の解説



小課題 4 [$X_i, Y_i, Z_i \leq 20$] の解説



小課題 5 [$X_i, Y_i, Z_i \leq 300$] の解説



小課題 2 [$N \leq 4000$] の解説・満点解法に向けて



満点解法 [$N \leq 150000$] の解説



おまけ

ICPC について

51 / 54

JOI/JOIG に参加できるのは高校生までですが、大学生になると ICPC があります

ICPC = 国際大学対抗プログラミングコンテスト



ICPC について

52 / 54

同じ大学から 3 人 1 組のチームで出場

3～5 時間で 10 問程度を解きます

勝ち抜くと世界大会に進めます

*ACM International Collegiate Programming Contest
Asia Regional Contest, Fukuoka, 2011-11-13*

Problem H ASCII Expression

Input: Standard Input
Time Limit: 30 seconds

Mathematical expressions appearing in old papers and old technical articles are printed with typewriter in several lines, where a fixed-width or monospaced font is required to print characters (digits, symbols and spaces). Let us consider the following mathematical expression.

$$\left(1 - \frac{4}{3^2}\right)^2 \times -5 + 6$$

It is printed in the following four lines:

```
      4   2
( 1 - ---- ) * - 5 + 6
      2
      3
```

where “- 5” indicates unary minus followed by 5. We call such an expression of lines “ASCII expression”.

For helping those who want to evaluate ASCII expressions obtained through optical character recognition (OCR) from old papers, your job is to write a program that recognizes the structure of ASCII expressions and computes their values.

For the sake of simplicity, you may assume that ASCII expressions are constructed by the following rules. Its syntax is shown in Table H.1.

ICPC について

53 / 54

ICPC では 1 問解くごとに風船がもらえます

なので、会場には風船がたくさん
これも ICPC の数ある特徴の 1 つです

大学生になったらぜひ ICPC に参加し
てみましょう！



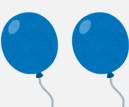
得点分布

54 / 54

JOI

100 点 

73 点 

64 点 

55 点 

17 点 

8 点 

JOIG

100 点 

26 点 

17 点 

8 点 

0 点 



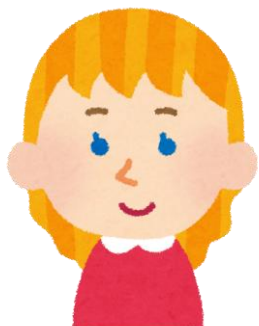
JOI 2021/2022 壊れた機器 2 (Broken Device 2)

解説: 平木 康傑

問題概要

- Anna が Bruno に整数を伝言したいが，伝達内容に制限がある
 - Anna は同じ長さのバイナリ列 2 つを送る
 - 長さは 1 以上 2000 以下．上限を L とおく
 - Bruno はそれを「混ぜた」バイナリ列を受け取り，整数を復元する

42 を伝えたい



0010
1101

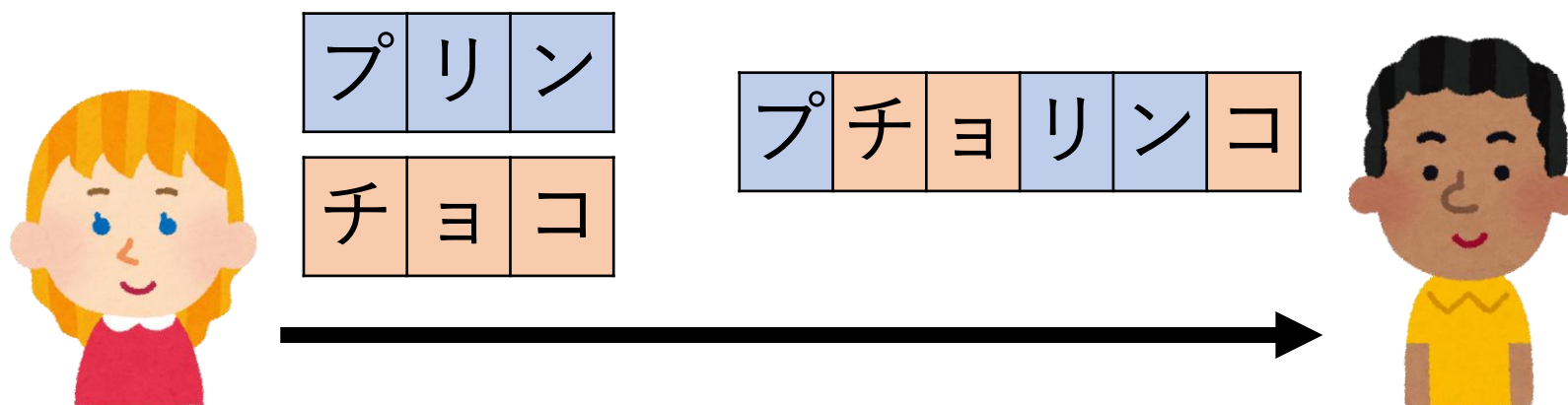
10010101



42 が分かれば OK

「混ぜ方」の概要

- ある $|S|$ 文字をそのままの順番で抜きだすと S に一致し、
残りの $|T|$ 文字をそのままの順番で抜きだすと T に一致する



ケツメイシ 『譲れない事』 歌い出し

- 時代がどんなに大きく変わっても
変わらないものはあるんだって信じて
時代が自由を奪おうとしても
譲れない事があるんだ

不変量に注目しよう

- どんな混ぜ方をしても一定になる量を見つければ、それを利用して伝えたいものを表せる
- 例えばどんなものがある？

不変量の例

- 1 の個数

不変量の例

- 1 の個数
 - たとえば Anna が $(1111, 1100)$ を送れば, それをどう混ぜても Bruno が受け取る数列には必ず 1 がちょうど 6 個入っている
 - 0 個から $2L = 4000$ 個までの $2L + 1 = 4001$ 通りを表せて, 小課題 1 は解ける (5 点)

不変量の例

- 1 の個数
- 長さ
 - こっちでも小課題 1 は解ける

不変量の例

- 1 の個数
- 長さ
 - この 2 つは組み合わせられる
 - (長さ, 1 の個数) の組は全部で $3 + 5 + 7 + \dots + (2L+1) = L^2 + 2L$ 通り
 - 上限 $L = 2000$ なので 4,000,000 までをカバーできた
 - これで小課題 2 も通る (+5 → 10 点)

不変量の例

- 1 の個数
- 長さ
- 端の文字
 - S, T の先頭がともに 1 の場合, 混ぜても先頭は 1
 - S, T の先頭がともに 0 の場合, 混ぜても先頭は 0
 - S, T の末尾がともに 1 の場合, 混ぜても末尾は 1
 - S, T の末尾がともに 0 の場合, 混ぜても末尾は 0

不変量の例

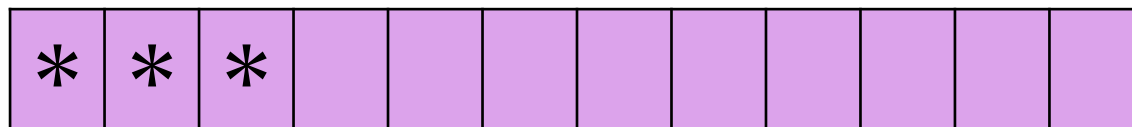
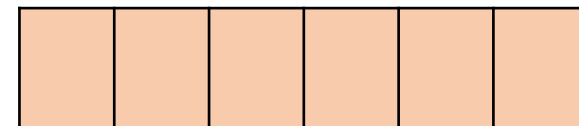
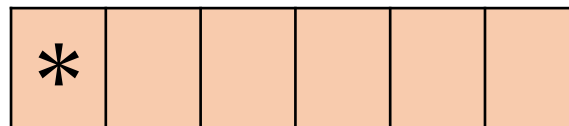
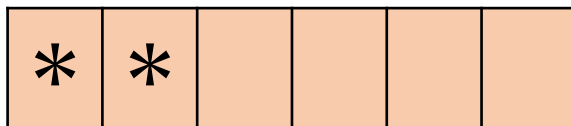
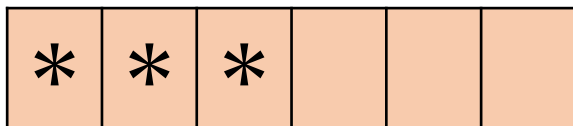
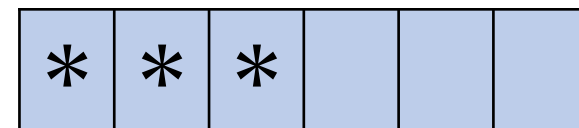
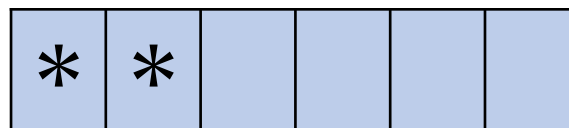
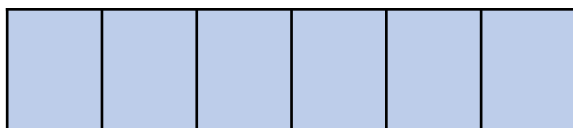
- 1 の個数
- 長さ
- 端の文字
 - S, T の {先頭, 末尾} がともに $[0, 1]$ の場合, 混ぜても $\{\text{〃}\}$ は $[\text{〃}]$
 - 少々工夫はあるが, これを使って表せる範囲を 4 倍にできる
 - これで小課題 3 も通る ($+3 \rightarrow 13$ 点)

不変量の例

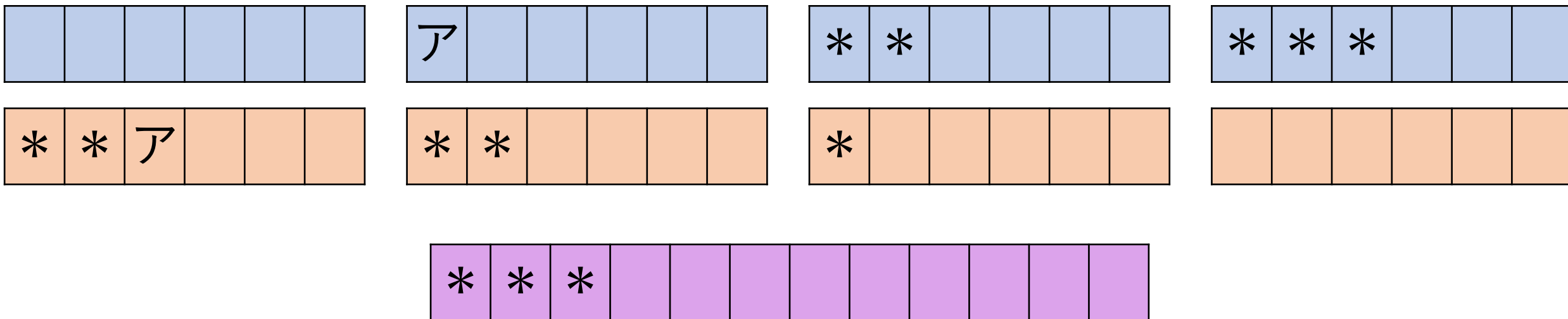
- 1 の個数
- 長さ
- 端の文字 ← 1 文字しか使わないなんてもったいない！
 - ~~• S, T の {先頭, 末尾} がともに $[0, 1]$ の場合, 混ぜても {カ} は {カ}~~
 - → _____ の場合, どう混ぜても先頭の _____ 文字のうち 1 は _____ 個

累積和不変量になるような構成

- 端の____文字
 - _____の場合, どう混ぜても先頭の____文字のうち 1 は____個
- 例えば, 先頭の 3 文字に選ばれる組合せは以下の通り

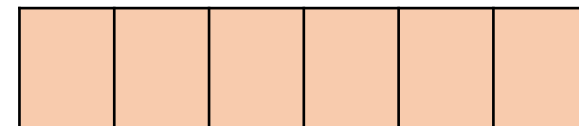
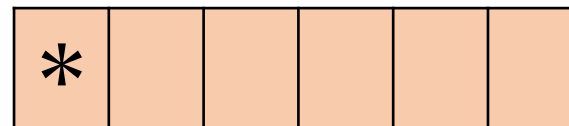
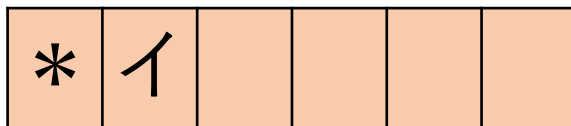
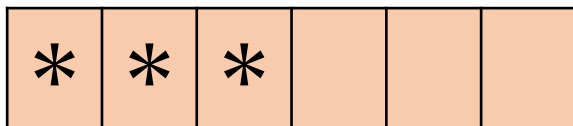
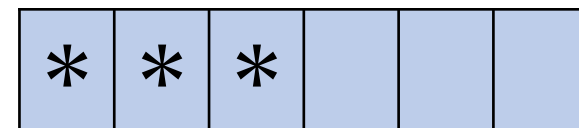
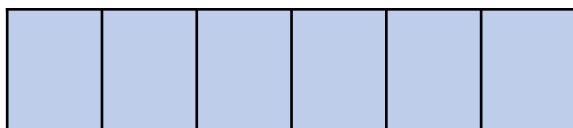


- 端の____文字
 - _____の場合，どう混ぜても先頭の____文字のうち 1 は____個
- 例えば，先頭の 3 文字に選ばれる組合せは以下の通り



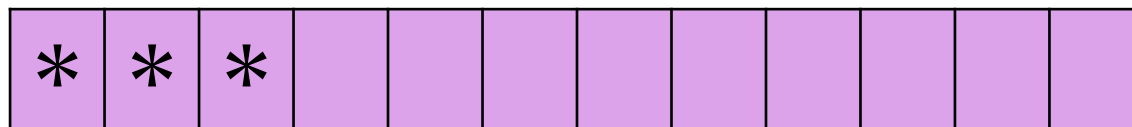
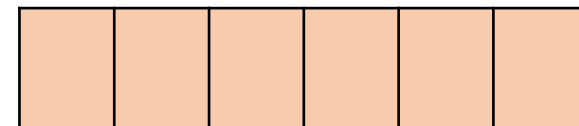
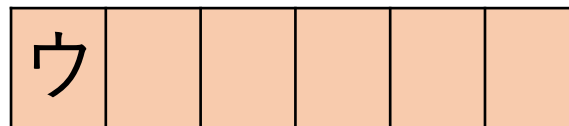
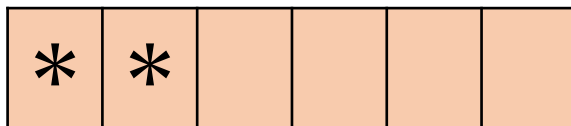
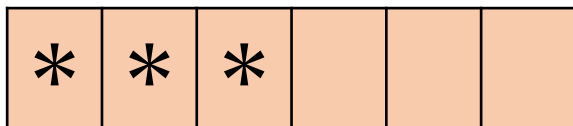
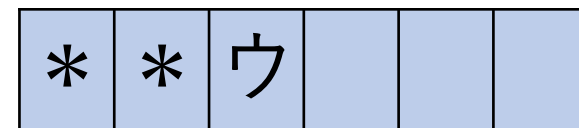
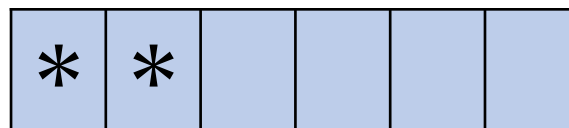
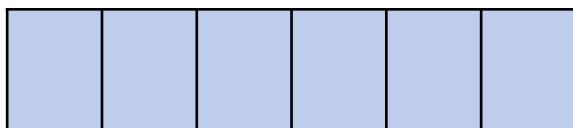
累積和不変量になるような構成

- 端の____文字
 - _____の場合, どう混ぜても先頭の____文字のうち 1 は____個
- 例えば, 先頭の 3 文字に選ばれる組合せは以下の通り



累積和不変量になるような構成

- 端の____文字
 - _____の場合, どう混ぜても先頭の____文字のうち 1 は____個
- 例えば, 先頭の 3 文字に選ばれる組合せは以下の通り



累積和不変量になるような構成

- 端の____文字
 - _____の場合, どう混ぜても先頭の____文字のうち 1 は____個
- 先頭 3 文字に選ばれる組合せの差分を取ることで,
以下のア, イ, ウがそれぞれ同じなら
先頭 3 文字に含まれる 1 の個数も一定とわかる

ア	イ	ウ			
---	---	---	--	--	--

ウ	イ	ア			
---	---	---	--	--	--

累積和不変量になるような構成

- 端の____文字
 - _____の場合, どう混ぜても先頭の____文字のうち 1 は____個
- 以下のア, イ, ウがそれぞれ同じなら
先頭 3 文字に含まれる 1 の個数も一定とわかる
- 一般に, S の先頭 k 文字 = $\text{reverse}(T$ の先頭 k 文字) のとき,
どう混ぜても
先頭 k 文字に含まれる 1 の個数は一定

ア	イ	ウ			
---	---	---	--	--	--

ウ	イ	ア			
---	---	---	--	--	--

累積和が不変量になるような構成

- S の先頭 k 文字 = $\text{reverse}(T \text{ の先頭 } k \text{ 文字})$ の場合,
どう混ぜても先頭の k 文字に含まれる 1 の個数は一定
- たとえば $S = "11100\dots"$, $T = "00111\dots"$ とすれば,
どう混ぜても先頭 5 文字に 1 はちょうど 3 つ含まれる

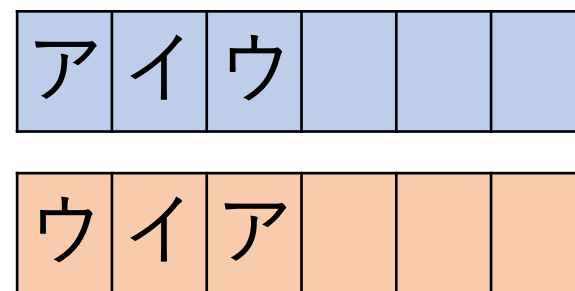
ア	イ	ウ			
---	---	---	--	--	--

- 末尾についても同様のことが言える

ウ	イ	ア			
---	---	---	--	--	--

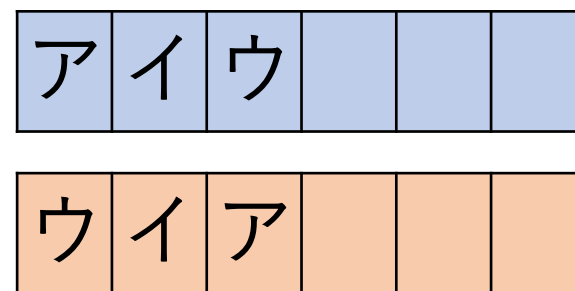
不変量の重ねがけ

- 「長さ $|S|$ 」 「先頭 $|S|/2$ 文字」 「末尾 $|S|/2$ 文字」 を組み合わせる
 - 長さ $|S|$ に対して,
 - 「先頭」で $|S|/2$ 通り,
 - 「末尾」で $|S|/2$ 通り,掛けあわせて $|S|^2/4$ 通りの不変量を伝えられる
 - $|S| = 1, 2, \dots, 2000$ で合わせて $2000^3/12 (+\alpha)$ 通りになる
 - つまり 6.6×10^8 通り
 - 小課題 4 も通る (+12 $\rightarrow$ 25 点)



不変量の重ねがけ

- 「長さ $|S|$ 」 「先頭 $|S|/3$ 文字」 「末尾 $|S|/3$ 文字」
さらに「全体の 1 の個数」を組み合わせる
 - 長さ $|S|$ に対して,
 - 「先頭」で $|S|/3$ 通り,
 - 「末尾」で $|S|/3$ 通り,
 - 「全体の 1 の個数」で $2|S|/3$ 通り,あわせて $2/27 |S|^3$ 通りの不変量を伝えられる
 - $|S| = 1, 2, \dots, 2000$ で合わせて $1/54 |S|^4 (+\alpha)$ 通り
 - 小課題 5 も通る (+15 $\rightarrow$ 40 点)



不変量の重ねがけ

- 「先頭 k 文字」そのものをたくさん重ねがけすることもできる
- DP + 経路復元によって長さごとの最適な配置を求めることで、 $L = 937$ で 10^{18} まで区別できる
- +23 → 63 点

ア	イ	ウ	エ	オ	ア	イ	ウ
ウ	イ	ア	オ	エ	ウ	イ	ア

落ち着こう

- 「先頭 k 文字」を 1 回重ねがけるごとに長さが倍増するので、140 には届きそうにない

落ち着こう

- 「先頭 k 文字」を 1 回重ねがけるごとに長さが倍増するので、140 には届きそうにない
- 少し見方を変えて、「 $+1/-1$ としたときの累積和」に注目してみる

+1/-1 累積和

- +1 と -1 を交互に置いた数列で先頭から累積和をとるとどうなる？

+1/-1 累積和

- +1 と -1 を交互に置いた数列で先頭から累積和をとるとどうなる？
- -1, 0, 1 のどれかになる

+1/-1 累積和

- +1 と -1 を交互に置いた数列で先頭から累積和をとるとどうなる？
- -1, 0, 1 のどれかになる
- つまり, 0 からのズレが高々 1

高々 1 だけズレる

- T を $[+1, -1, +1, -1, \dots]$ として, S に割り込んだときの累積和は？

高々 1 だけズレる累積和

- T を $[+1, -1, +1, -1, \dots]$ として, S に割り込んだときの累積和は？
- 純粹に S の要素だけだったときと比べて, 高々 1 しかズレない

高々 1 だけズレる累積和

- T を $[+1, -1, +1, -1, \dots]$ として, S に割り込んだときの累積和は？
- 純粹に S の要素だけだったときと比べて, 高々 1 しかズレない
- つまり, S が十分声を張れば T はかき消されると考えられる！

ノイズの入る累積和

- $S = 1 + \sum_{k=1}^{60} b_k \times (\text{直前と同じビットは 2 個, 異なるビットは 3 個})$
- $T = 10101010 \dots$
- 最初のビットを読んで「直前のビットは 1 だった」ことにする
- 以下を 60 回繰り返す
 - +1/-1 累積和を始めて, +2 か -2 になったら停止してビットを特定する

ノイズの入る累積和

$L=181$ なので, 累計 80 点

- $S = 1 + \sum_{k=1}^{60} b_k \times (\text{直前と同じビットは 2 個, 異なるビットは 3 個})$
- $T = 10101010 \dots$
- 最初のビットを読んで「直前のビットは 1 だった」ことにする
- 以下を 60 回繰り返す
 - +1/-1 累積和を始めて, +2 か -2 になったら停止してビットを特定する
- 停止時点で, 直前のビット b_{k-1} に対し, 以下のいずれかが成り立つ
 - S のビットがちょうど読まれ, T の次のビットは b_{k-1} と同じ
 - S のビットが 1 つだけ残り, T の次のビットは b_{k-1} と異なる

+「長さ」+「長さに収まる最適な割り当て」

- $S = 1 + \sum_{k=1}^{???} b_k \times (\text{直前と同じビットは2個, 異なるビットは3個})$
- $T = 10101010 \dots$
- 最初のビットを読んで「直前のビットは1だった」ことにする
- 以下を ~~60回繰り返す~~ 長さが尽きるまで繰り返す
 - +1/-1 累積和を始めて, +2 か -2 になったら停止してビットを特定する
- 停止時点で, 直前のビット b_{k-1} に対し, 以下のいずれかが成り立つ
 - S のビットがちょうど読まれ, T の次のビットは b_{k-1} と同じ
 - S のビットが1つだけ残り, T の次のビットは b_{k-1} と異なる

L=142 なので, 累計 96 点

+「長さ」+「長さに収まる最適な割り当て」
+「端の文字」

- $S = 1 + \sum_{k=1}^{???} b_k \times (\text{直前と同じビットは2個, 異なるビットは3個})$
- $T = 10101010 \dots$

100 点

L=142 なので,

- 最初のビットを読んで「直前のビットは1だった」ことにする

- 以下を ~~60回繰り返す~~ 長さが尽きるまで繰り返す

- +1/-1 累積和を始めて, +2 か -2 になったら停止してビットを反転する

- 停止時点で、送受信するビットを全て反転する
- 停止時点で、送受信するビットを全て反転する

伝える整数が奇数なら, 送受信するビットを全て反転する

- S のビットが1つだけ残り, T の次のビットは b_{k-1} と異なる

[illegible]

0点: 1人 $\sum_{k=1}^n b_k \times$ (直前と同じビットは2点: 1人 3点: 7人
• T = 10 点: 12人

- 最初のビットを読んで「直前のビットと異なっていた」ことにする
- 以下を ~~60回繰り返す~~ 長さが尽きるまで繰り返す

40 点: +1 人 積和を始めて, +2 か -2 になったら停止, 100 点に達する

• 停止時点で、送信するビットを全 80 点: 3 イ
 伝える整数が奇数なら、以下の 0 点: 0 イ
 • S のビットが 1 つだけ残り、 T の次のビットは b_{k-1} と異なる

JOI 2021/22 春合宿 Day 3 スプリンクラー (Sprinkler) 解説

解説担当 : tatyam

問題概要

N 頂点の木があり、頂点 i には整数 H_i が書かれています。また、整数 L が与えられます。 Q クエリ进行处理してください。

- $1\ X\ D\ W$: 頂点 X から距離 D 以下の頂点に書かれている数を W 倍する。
- $2\ X$: 頂点 X に書かれている数 $\bmod L$ を出力する。

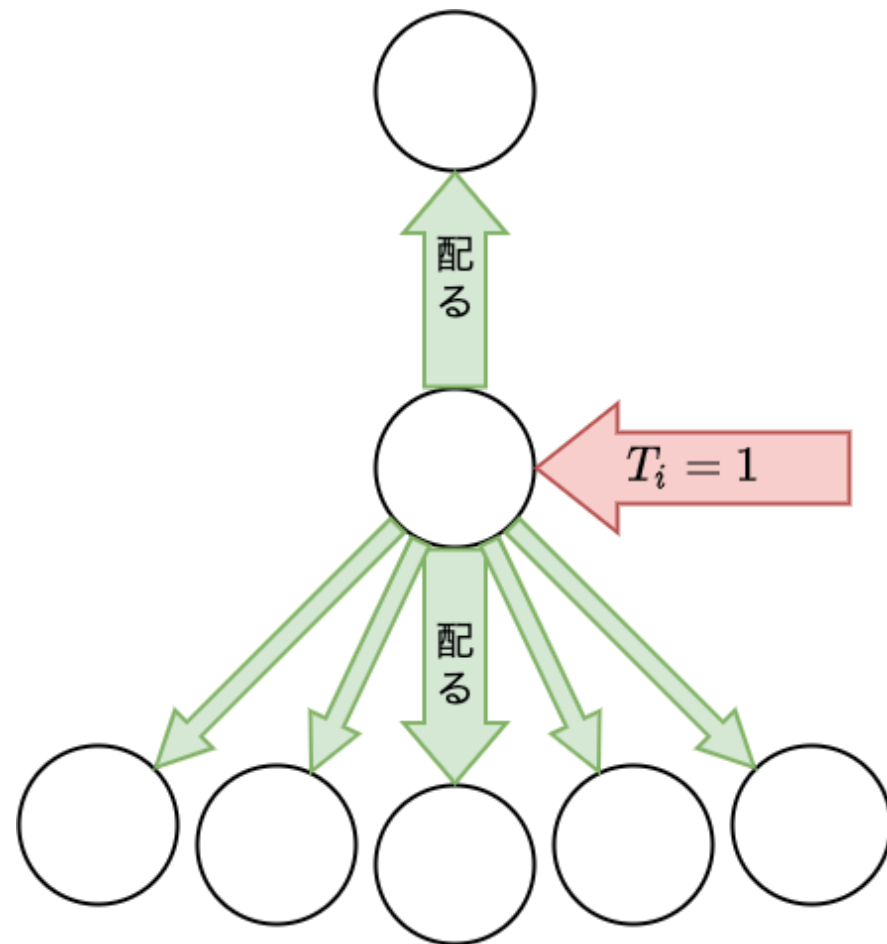
$$N \leq 2 \times 10^5, Q \leq 4 \times 10^5, D \leq 40$$

小課題

番号	点数	追加の制約	想定解
1	3 点	$N, Q \leq 10^3$	$O(NQ)$
2	9 点	$D \leq 1$	$O(N + Q), O(N + Q\sqrt{N})$ など
3	29 点	$D \leq 2$	$O(ND + QD^2), O(ND + QD \log N)$ など
4	12 点	$W = 0$	$O(ND + Q), O(ND + QD),$ $O(ND + QD^2/\text{word})$ など
5	30 点	$W = 2$	$O(N \log N + Q(\log N)^2)$
6	17 点	なし	$O((N + Q)D)$

小課題 1 (3 点)

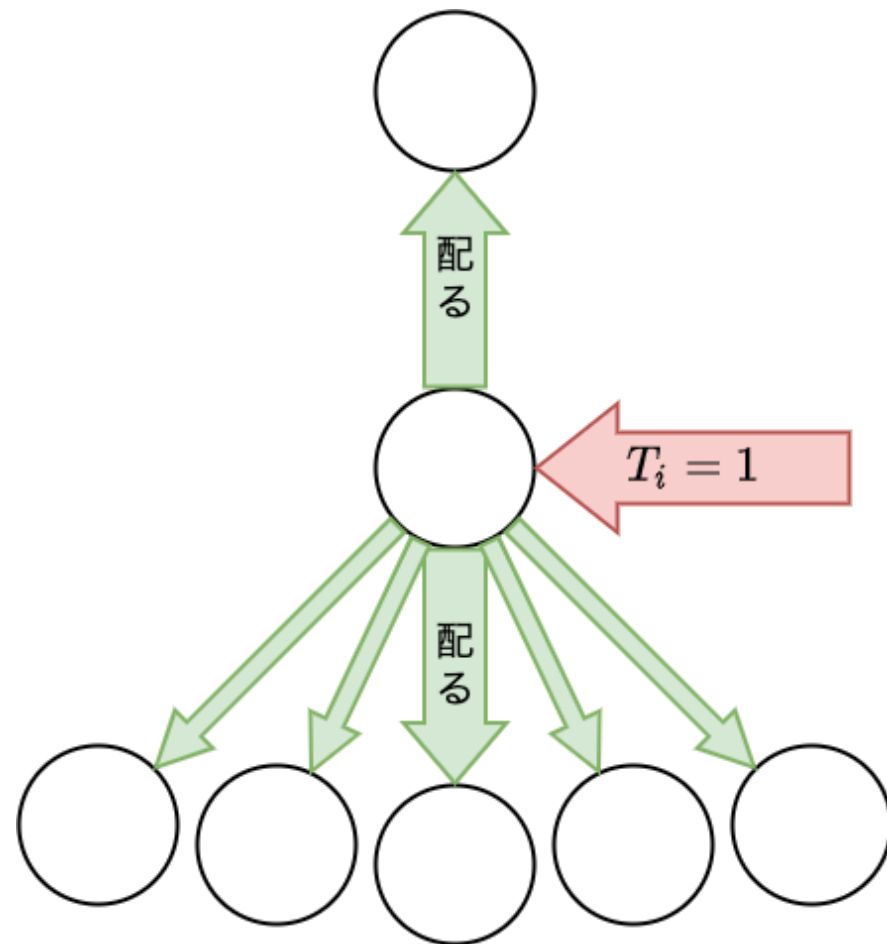
- $N, Q \leq 10^3$
- まずは愚直な解法を考えてみよう
- 「頂点 X から距離 D 以下の頂点」を DFS 等で列挙する
- これを各クエリで毎回やると $O(NQ) \sim 10^6$ なので、間に合う



小課題 2 (9 点)

- $D \leq 1$
- DFS でタイプ 1 のクエリを配ると、次数の大きい頂点を指定されて $O(NQ)$ にかかってしまう
- 逆に、タイプ 2 のクエリが来た時に DFS で、今までに周囲に来たタイプ 1 のクエリをもらいに行くこともできる。これも次数の大きい頂点を指定されて $O(NQ)$ にかかってしまう

→ これを組み合わせよう！

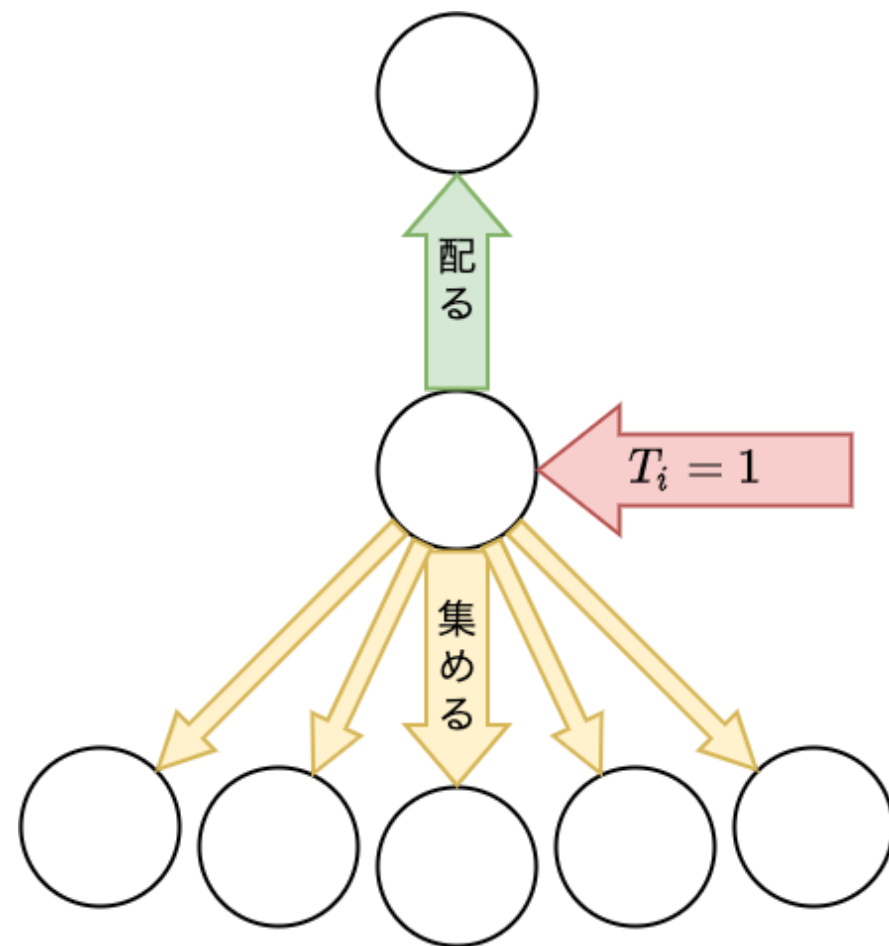


小課題 2 (9 点)

- 適当に根を取り根付き木とする
- 各頂点の親は 1 つまで、子はたくさんあるかも

→ 親にはタイプ 1 のクエリを配り、子には配らずタイプ 2 のクエリが来た時に取りに来てもらう

どちらも $O(1)$ なので、全体で $O(N + Q)$



小課題 2 (9 点)

根付き木としない場合

- 度数が多い (R 以上) 頂点にタイプ 1 のクエリが来た場合のみ配らない
- 各頂点に隣接する度数が多い頂点を列挙し、タイプ 2 のクエリが来たときに取りに行く
- 度数が R 以上の頂点は $2(N - 1)/R$ 個以下 (度数の合計は $2(N - 1)$ なので)
- $R \in \Theta(\sqrt{N})$ とすれば、全体で $O(N + Q\sqrt{N})$

木であるという制約を使わずに解けた！

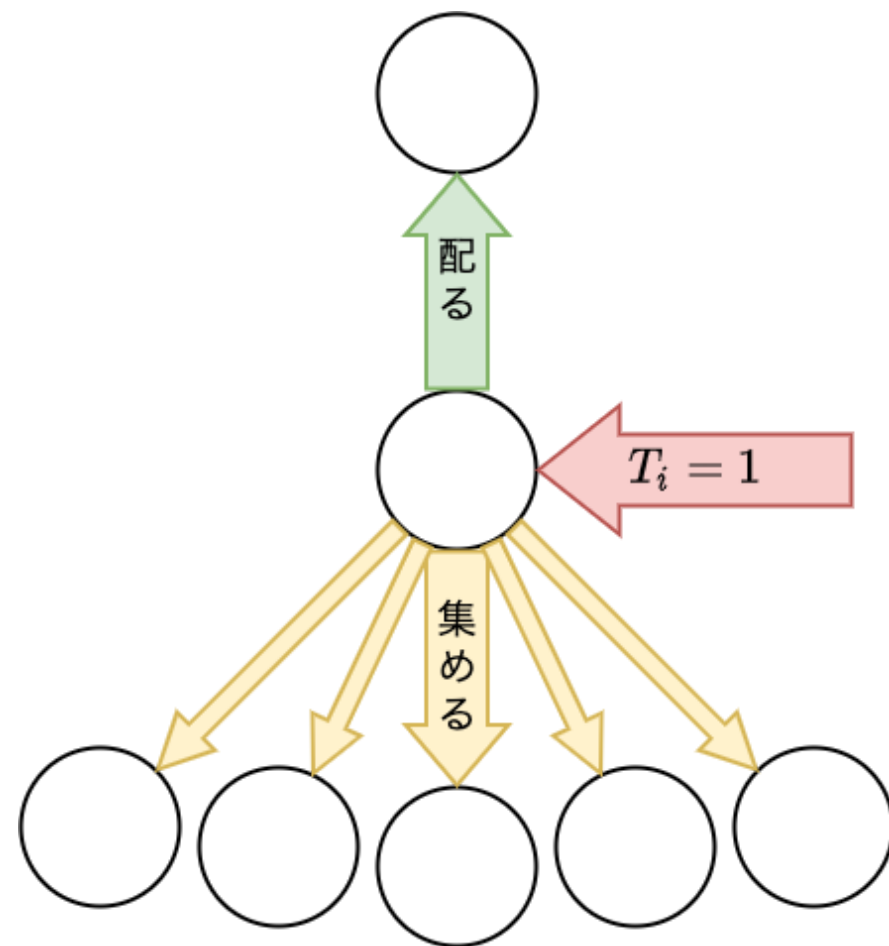
小課題 3 (29 点)

- $D \leq 2$

小課題 2 を発展させてみよう

- 2 個上の親まで配る
- 2 個下の子から取りに来てもらう

→ これで全部？



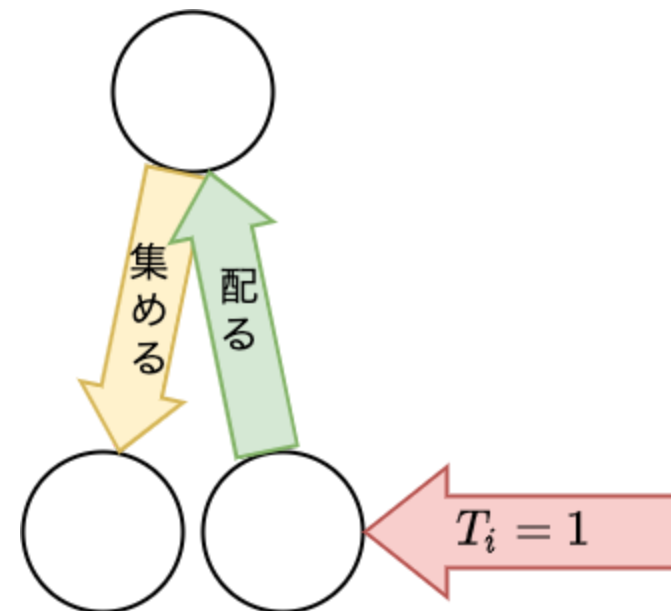
小課題 3 (29 点)

- $D \leq 2$

小課題 2 を発展させてみよう

- 2 個上の親まで配る
- 2 個下の子から取りに来てもらう

→ 上がって下がる場合に対応できていないが、上がる部分を配り、下がる部分を集めれば良い



小課題 3 (29 点)

まとめ

- 「距離 **2** 以下の頂点を W 倍する」クエリを、「距離 **2** の頂点を W 倍する」「距離 **1** の頂点を W 倍する」「距離 **0** の頂点を W 倍する」に分解
- 上がる部分をタイプ **1** のクエリのとときに配り、下がる部分をタイプ **2** のクエリのとときに集めると、 $O(ND + QD^2)$

小課題 4 (12 点)

- $W = 0$
- 水を掛けたところから高さが **0** になっていく JOI くんはなにをやっているのでしょうか.....

→ クエリを先読みして、「最初に水を掛けられるのはいつか」を計算しておく、タイプ **2** のクエリに答えられる

小課題 4 (12 点)

- クエリを先読みして、「最初に水を掛けられるのはいつか」を計算しておく、タイプ 2 のクエリに答えられる
- $dp[i][j]$ で頂点 i から距離 j 以内の頂点が最初に水を掛けられる時刻を管理する
- クエリを先読み: 時刻 t にタイプ 1 のクエリが来たら、 $dp[X][D]$ に t を入れておく
- その後、頂点 x に隣り合う頂点 y について、 $dp[y][j - 1]$ を $dp[x][j]$ で更新
- 全部の頂点で「最初に水を掛けられるのはいつか」を求められたので、タイプ 2 のクエリに答えられる

$$O(ND + Q)$$

小課題 4 (12 点)

- 小課題 3 を発展させてみよう
- 小課題 3 では「距離 2 以下の頂点を W 倍する」クエリを、「距離 2 の頂点を W 倍する」「距離 1 の頂点を W 倍する」「距離 0 の頂点を W 倍する」に分解した
- これは W 倍を 2 回数えてしまうのを防ぐため
- 0 倍は 2 回数えても構わないので、うまくやると配る方も集める方も $O(D)$ になる

$$O(ND + QD)$$

小課題 4 (12 点)

- $flag[i][j]$ で、頂点 i から下へ距離 j 以内の頂点に水を掛けられたかを管理する
- 上がる時は配る : $flag[X$ から i 個上がった親 $[D - i]$ に 1 を代入
- 下がる時は集める : $O(D^2)$ 個の $flag[i][j]$ を見る必要があるが、bit 演算なので $O(D^2/\text{word})$ で求められる

$$O(ND + QD^2/\text{word})$$

小課題 5 (30 点)

- $W = 2$
- 水のかかった回数のみ数えれば良い (この観点で小課題 4 を含む)
- 掛け算 $\text{mod } L$ では逆元がなかったが、これは逆元がある
- 何ができる？

小課題 5 (30 点)

- 集めたり配ったりすることが難しいなら、木を分解して分割統治を試みよう
- 方針: 重心分解をして、タイプ 1 のクエリで、各重心が「頂点 X から重心を通過して距離 D 以内の頂点へ」の分を管理する
- 重心分解をすると、各頂点につきちょうど 1 回、その頂点を重心とする木が現れる
- 重心分解の深さは $O(\log N)$ なので、各頂点は N 個の木のうち $O(\log N)$ 個に現れる

小課題 5 (30 点)

- タイプ 1 のクエリで、頂点 X が属する各木について、頂点 X と重心の間の距離を d として、重心から距離 $D - d$ 以内の頂点に 1 を加算する
- これは、各重心ごとに $a[i] = (\text{重心から距離 } d \text{ の頂点へのクエリ回数})$ の双対 (セグ木 or BIT) を持てば良い
- ただし、頂点 X から重心を通過して頂点 X の方向に戻る分を数えてはいけないので、その分を -1 加算して打ち消したい
- これは、各重心につながる各辺ごとに $a[i] = (\text{重心からその辺の向きに距離 } d \text{ の頂点へのクエリ回数})$ の双対 (セグ木 or BIT) を持てば良い
- セグ木の大きさは D で抑えて良いので、 $O(N \log N + Q \log N \log D)$
- $D \leq 40$ を使わなくても $O(N \log N + Q(\log N)^2)$ で解ける (!)

小課題 6 (17 点)

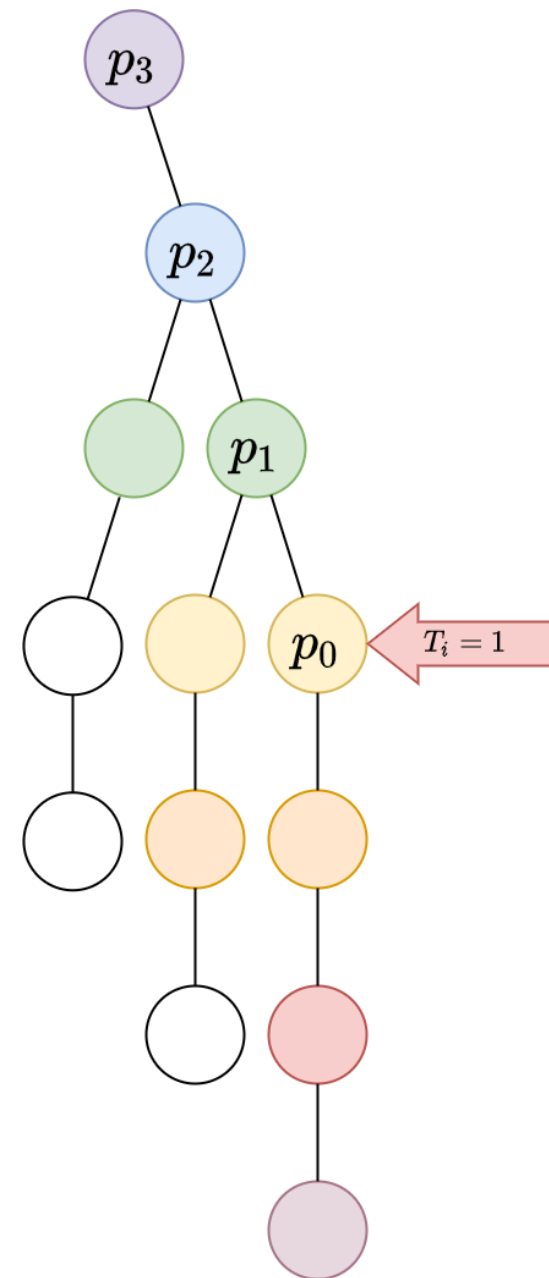
- 追加の制約はない
- 小課題 3 を発展させてみよう
- 小課題 3 では「距離 2 以下の頂点を W 倍する」クエリを、「距離 2 の頂点を W 倍する」「距離 1 の頂点を W 倍する」「距離 0 の頂点を W 倍する」に分解した
- これは W 倍を 2 回数えてしまうのを防ぐため

→ これをうまくできないか？

小課題 6 (17 点)

- 「頂点 p_0 から距離 3 以内」は、
 - 「頂点 p_0 から下に距離 3」
 - 「頂点 p_0 から下に距離 2」
 - 「頂点 p_1 から下に距離 2」
 - 「頂点 p_1 から下に距離 1」
 - 「頂点 p_2 から下に距離 1」
 - 「頂点 p_2 から下に距離 0」
 - 「頂点 p_3 から下に距離 0」

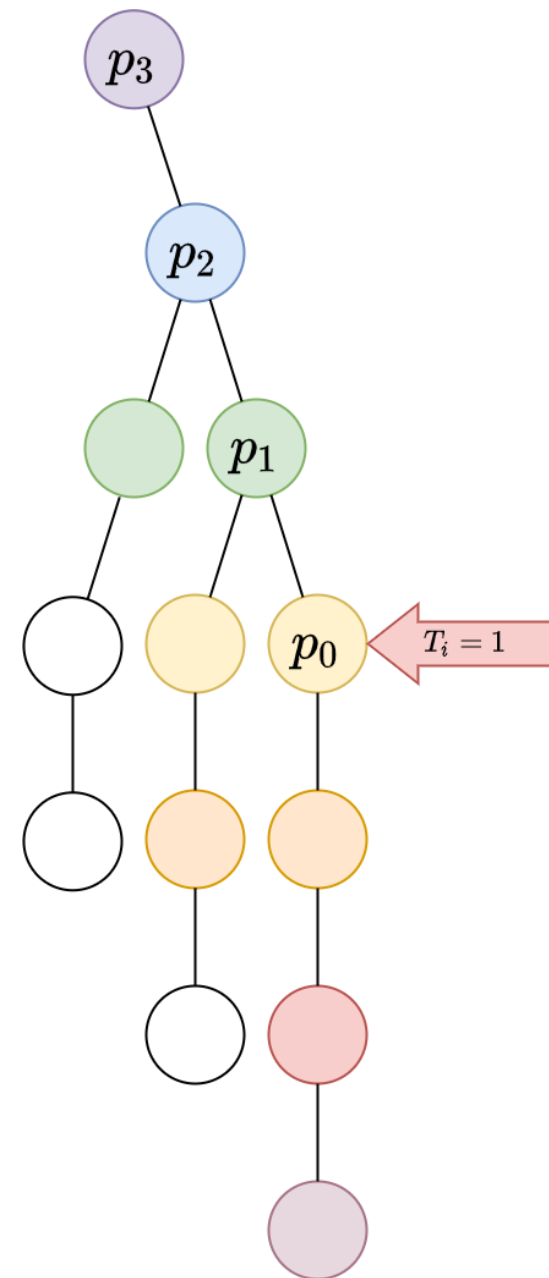
の 7 つに過不足なく分割できる



小課題 6 (17 点)

- 同様にして、頂点 X から距離 D 以内の頂点は $O(D)$ 個の「頂点 i から下に距離 j 」の形の領域に分割できる
- $a[i][j] = (\text{頂点 } i \text{ から下に距離 } j \text{ の頂点に掛かる倍率})$ を管理すれば、配るのも集めるのも $O(D)$

全体で $O(ND + QD)$ で解けた 🎉 100



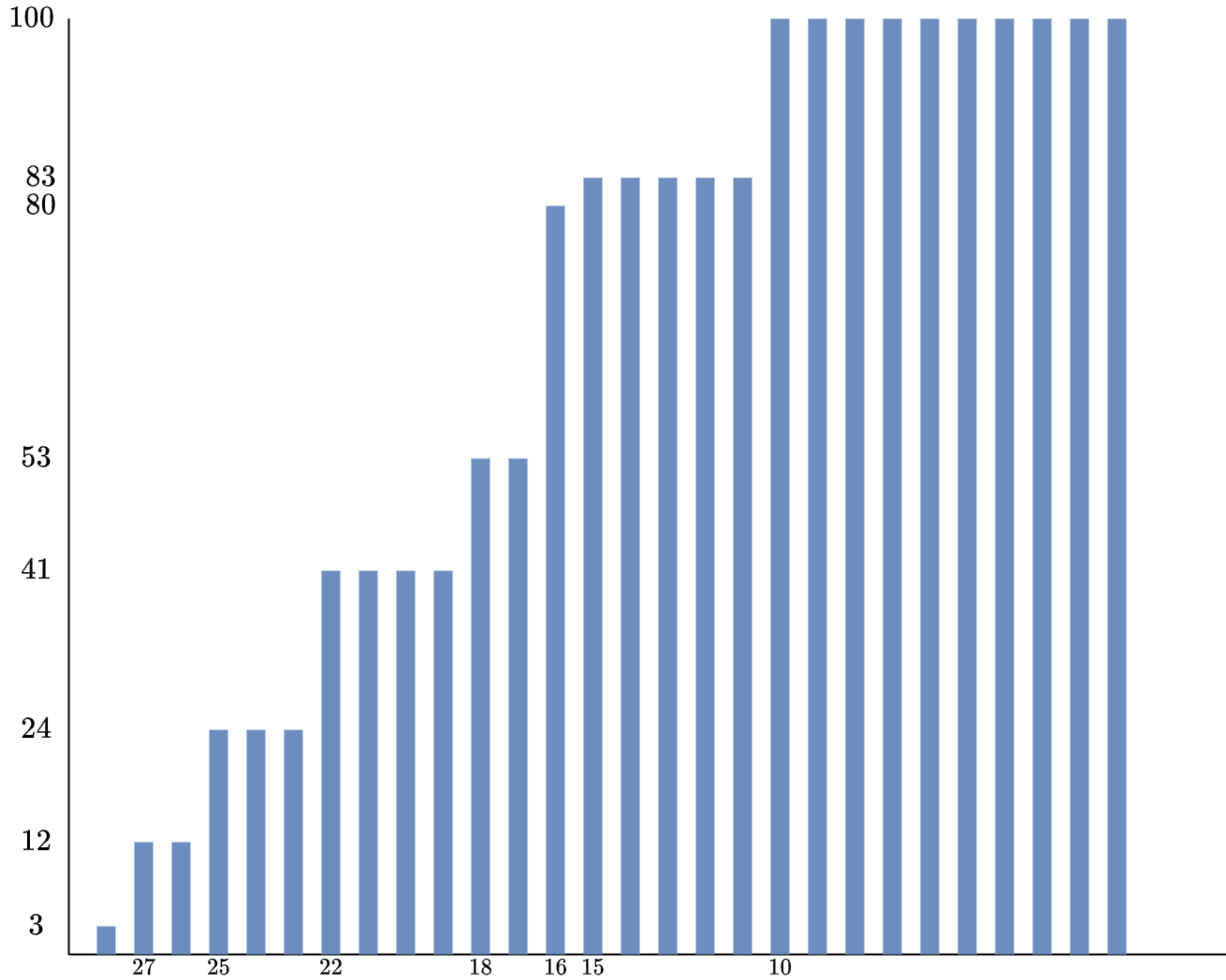
おまけ : BFS Numbering

- [yukicoder No.899 yatheree](#) Niuez 氏発祥 (?) のテクニック
- オイラーツアー (DFS 順) ではなく、BFS 順に番号を振り直すと、「頂点 i から下に距離 j 」の形の領域の頂点番号が連続するので、区間クエリとして扱えるようになる
- これを小課題 6 に使うと $O(ND + QD \log D)$ (TLE)

おまけ：頑張って逆元を持たせる

- $\text{mod } L$ での掛け算は頑張ると逆元を持たせることができる
- L を素因数分解して、 $L = p_0^{a_0} \times p_1^{a_1} \times \dots$ とする
- まず $p_0, p_1, \dots$ が何回掛かったかを持たせる。これには明らかに逆元がある
- そして、掛かった $p_0, p_1, \dots$ 以外の素因数の総積 $\text{mod } L$ も持つ。これは L と互いに素なので、逆元がある
- $p_0, p_1, \dots$ が何回掛かったかと、それ以外の素因数の総積が分かっているので、元の総積 $\text{mod } L$ を復元できる
- これで小課題 5 を小課題 6 に使えるようになる
 $O(N \log N \log L + Q \log N \log D \log L)$ (TLE)

得点分布



蟻と角砂糖

解説担当：川崎 理玖

問題概要

- 直線上に蟻と角砂糖が置かれている
- 各蟻を距離 L 以内の角砂糖に向かわせる
- 食べられる角砂糖の個数を最大化
- クエリで蟻や角砂糖が追加される

問題概要

- 距離 L 以下の蟻と角砂糖がマッチングできるとして、最大マッチングを求めればいい

小課題1

- 各クエリごとに $O(N)$ で答えを求める. (N はそれまでに処理したクエリの数)
- 蟻, 角砂糖それぞれについて, (位置, 個数) のリストを位置の昇順に持っておく.

小課題1

- 座標最小の蟻と座標最小の角砂糖に注目
- マッチングできるならする
- マッチングできないなら，座標の小さい方は以降無視
- あとで損することがないため，上の貪欲が最適

小課題1

- 普通に実装すれば $O(N)$
- 全体 $O(Q^2)$
- 6 点
- ちなみに余計な $\log$ をつけても通ります

小課題2

- 蟻と角砂糖の座標が偶奇で分かれている
- $L=1$ なので、ちょうど隣のものだけペアにできる

小課題2

- Hall の定理を使うとわかりやすくなる
- IOI Syllabus 上での Hall の定理の扱いは（多分）グレー
- JOI春合宿ではこうした問題も当然のように出題

小課題2

- この問題に適用してみる
- 今存在する蟻の総数を A とおく
- 蟻の部分集合 S を選び, S とマッチング可能な角砂糖の集合を T とおく

小課題2

- マッチングサイズが $A - |S| + |T|$ 以下であるのは自明
- 実は, $A - |S| + |T|$ の最小値が最大マッチングのサイズに一致 (Hall の定理を使う)
- Hall を経由しなくても示せる (小課題1の貪欲をよく観察する) ので実はこれが想定かも

小課題2

- 同じ座標にいる蟻は，すべて選ぶか選ばないかの2択
- 各座標 $1, 3, 5, \dots$ について，その座標にいる蟻を選ぶかどうか決める
- 便利のため，蟻が 0 匹の座標を選んでもいい

小課題2

- 座標 $1, 3, 5, \dots$ を選ぶかどうかを $0, 1$ 列 $x_0, x_1, \dots$ で表すことにする
- x_i を決めたときの $A - |S| + |T|$ への寄与：
 x_i が 1 なら (座標 $i*2-1$ にいる蟻の数) $\cdot (-1)$
 x_i か x_{i+1} が 1 なら (座標 $i*2$ にある角砂糖の個数)

小課題2

- 全体のスコアが局所的なスコアの和で表わせた
- Segment Tree に乗せてみる

小課題2

- 各ノードは，そのノード内だけ見たときの $-|S|+|T|$ の min を持つ
- ノードのマージを考えると，ノード内の左端，右端を選んだかどうかで 4 通り持っておくべき

小課題2

- 区間 $[a,b],[b+1,c]$ をマージする際は, $b,b+1$ の間にある角砂糖の個数を参照して適宜計算
- クエリあたり $O(\log Q)$, 全体 $O(Q \log Q)$
- 16 点

小課題3,4

- L の値が大きいため、小課題 2 のようにスコアを局所的なスコアの和に分解できない
- と思いきや、実はできる

小課題3,4

- 小課題2と同様，蟻がいる座標を昇順にならべ，それを使う/使わないを x_i で表す.
- ある角砂糖の寄与は，以下の形になる.
- $x_l, x_{l+1}, \dots, x_r$ のうち 1 つ以上が 1 なら，
(+1) の寄与

小課題3,4

- これを次のように変形する
- x_i が 1 なら $(+1)$ の寄与 ($1 \leq i \leq r$)
- x_i と x_{i+1} が両方 1 なら (-1) の寄与 ($1 \leq i \leq r-1$)

小課題3,4

- 変形により，寄与が本来より小さくなることはない
- 最終的に最小化するので，不正に小さい解を出力することはない
- 最適解をちゃんと達成できることを示す

小課題3,4

- $l \leq a < b \leq r$ に対して, $x_a=1, x_b=1$ であり, $x_i=0$ ($a < i < b$) だったとする
- このとき, $a < i < b$ を満たすすべての i で x_i を 1 に変えても問題ない

小課題3,4

- $a < i < b$ を満たす何らかの i に対して, x_i に作用するような角砂糖を考える
- マッチングの条件を考えると, この角砂糖は必ず x_a, x_b の少なくとも一方に作用している
- よって, 前ページの変形を行っても損しない

小課題3,4

- 結局、どの角砂糖に対して
も、 $x_l, x_{l+1}, \dots, x_r$ のうちある連続区間が 1
になるという形になる
- つまり、角砂糖のスコアへの寄与は正しく求まる

小課題3

- 角砂糖の位置が予めすべて与えられているなら，蟻の更新は小課題2と全く同様に行える
- （問題では蟻→角砂糖の順番なので役割を入れかえる）
- 全体で $O(Q \log Q)$ となり，26 点

小課題4

- 小課題2,3で使った Segment Tree に更に操作を追加する
- 角砂糖の追加を区間加算クエリの要領で処理する

小課題4

- 角砂糖が $x_l, x_{l+1}, \dots, x_r$ に作用する時, 区間 $[l, r]$ に対してクエリを呼ぶ
- 今, あるノード全体にこのクエリが飛んだとする

小課題4

- このノードは自分以下での $-|S|+|T|$ の min を保持している
- 角砂糖を W 個追加するなら，この min はすべて $(+W)$ されるはず
- ただし， $|S|>0$ でないときは違う

小課題4

- ノードが持つデータを, $-|S|+|T|$ の min (ただし, $|S|>0$ であると仮定) としておけばよい

小課題4

- ノードのマージの際、マージする2つのノードの境界に作用する角砂糖の個数を知る必要がある
- これを別個にBITで管理すると余計な $\log$ がつく（が高速 BIT なら通るかもしれない）

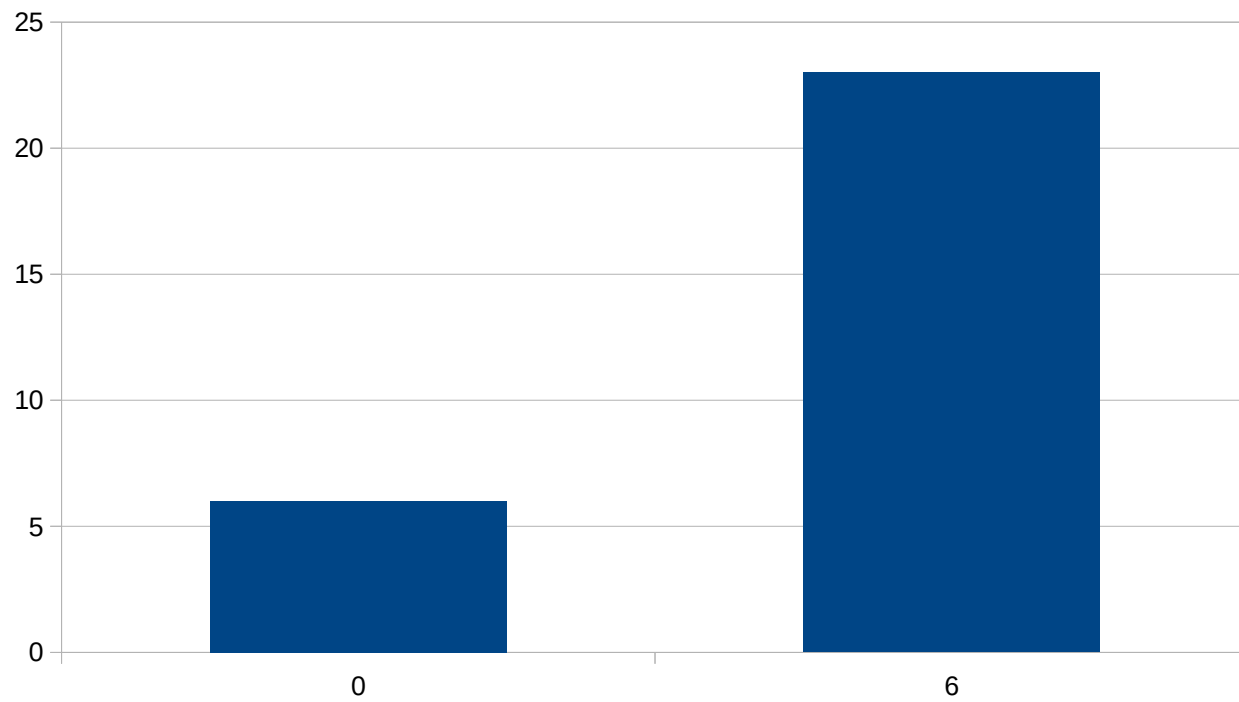
小課題4

- 実は同じ Segment Tree 内で処理できる
- 各ノードに、「その2つの子の境界に作用する角砂糖の個数」を記録することにする
- 区間加算 Segment Tree と同様に、遅延評価を用いると効率的に更新できる

小課題4

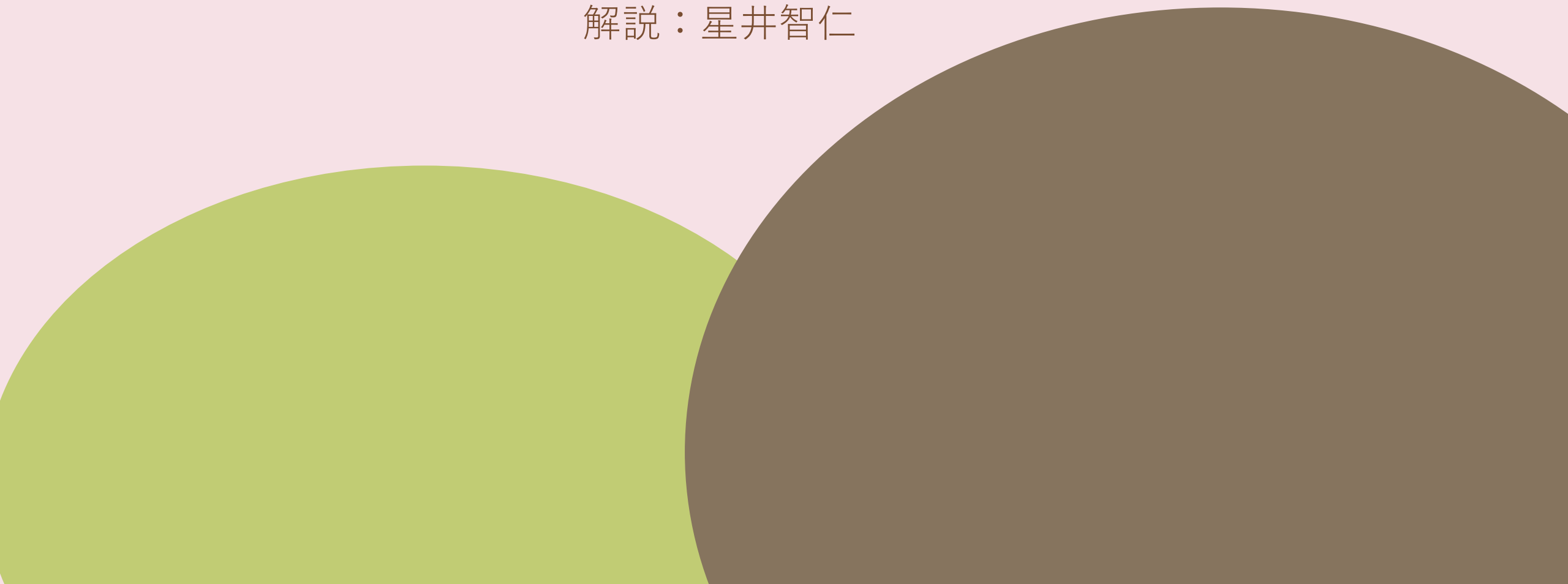
- 計算量はクエリあたり $O(\log Q)$ になり,
全体で $O(Q \log Q)$
- 満点

得点分布



一流の団子職人

解説：星井智仁



問題概要

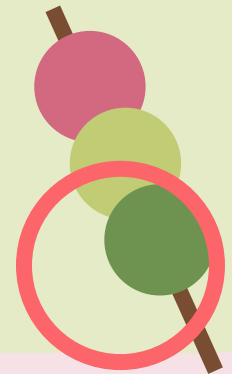
1

N色の団子がそれぞれM個ずつ 合計NM個ある

NM個ある団子を色が相異なるようにN個ずつM組に分けたい

使う団子を団子チェッカーにいくつか指定すると作れる串の数を教えてくれる

ただし団子チェッカーを使える回数は50000回以下である



小課題

2

1. $N=4, M=4$
2. $N=100, M=10$
3. $N=200, M=25$
4. $N=400, M=25$

小課題

3

1. $N=4, M=4$
2. $N=100, M=10$
3. $N=200, M=25$
4. $N=400, M=25$

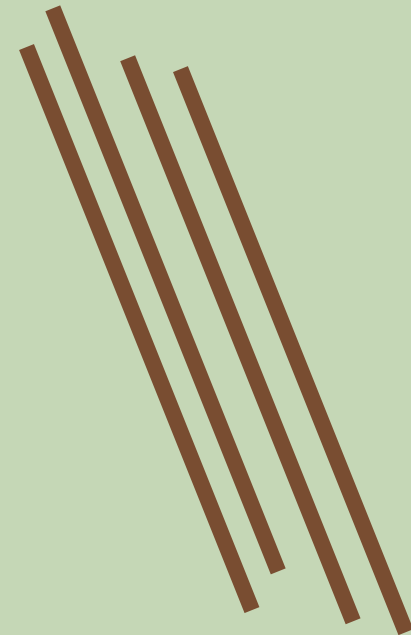
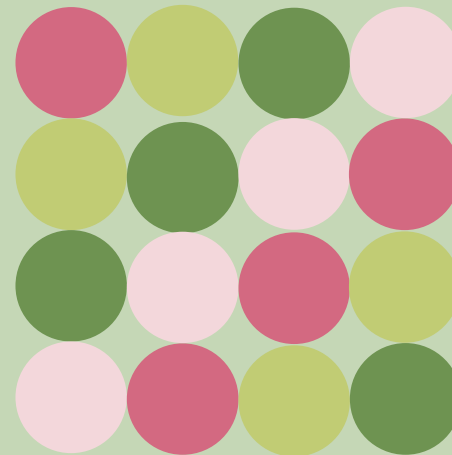
小課題 1 $N=4$ $M=4$

4

$N=4$, $M=4$

団子は $NM=16$ 個ある

全通り試せるかな



小課題 1 $N=4$ $M=4$

5

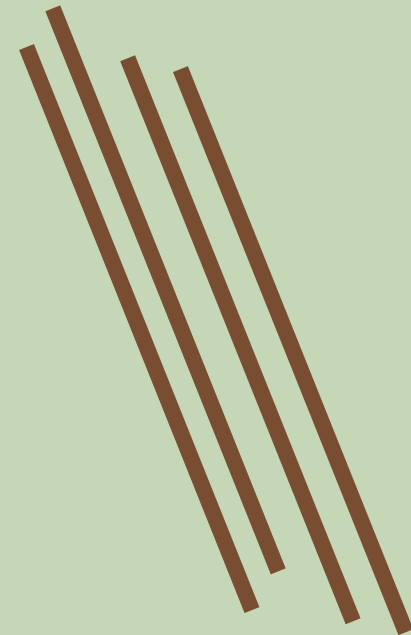
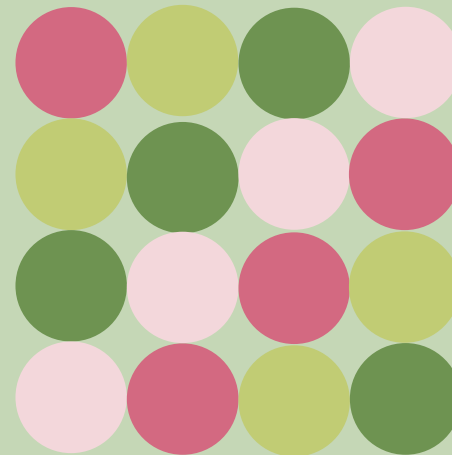
$N=4$, $M=4$

団子は $NM=16$ 個ある

全通り試せるかな

$$2^{16} = 65536 > 50000$$

試せない



小課題 1 $N=4$ $M=4$

6

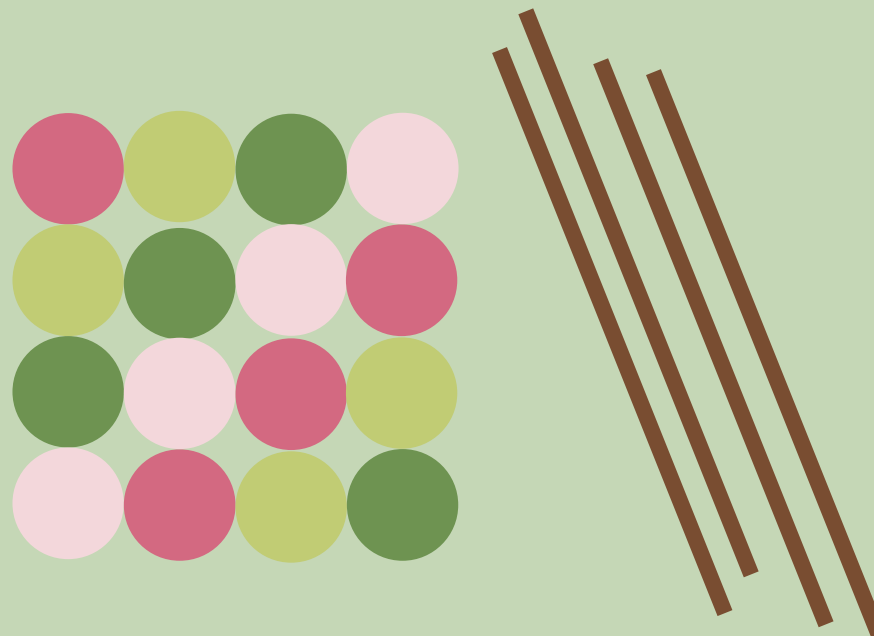
$N=4$, $M=4$

団子は $NM=16$ 個ある

全通り試せるかな

$${}_{16}C_4 = 1820 \leq 50000$$

試せる



小課題

7

1. $N=4, M=4$
2. $N=100, M=10$
3. $N=200, M=25$
4. $N=400, M=25$

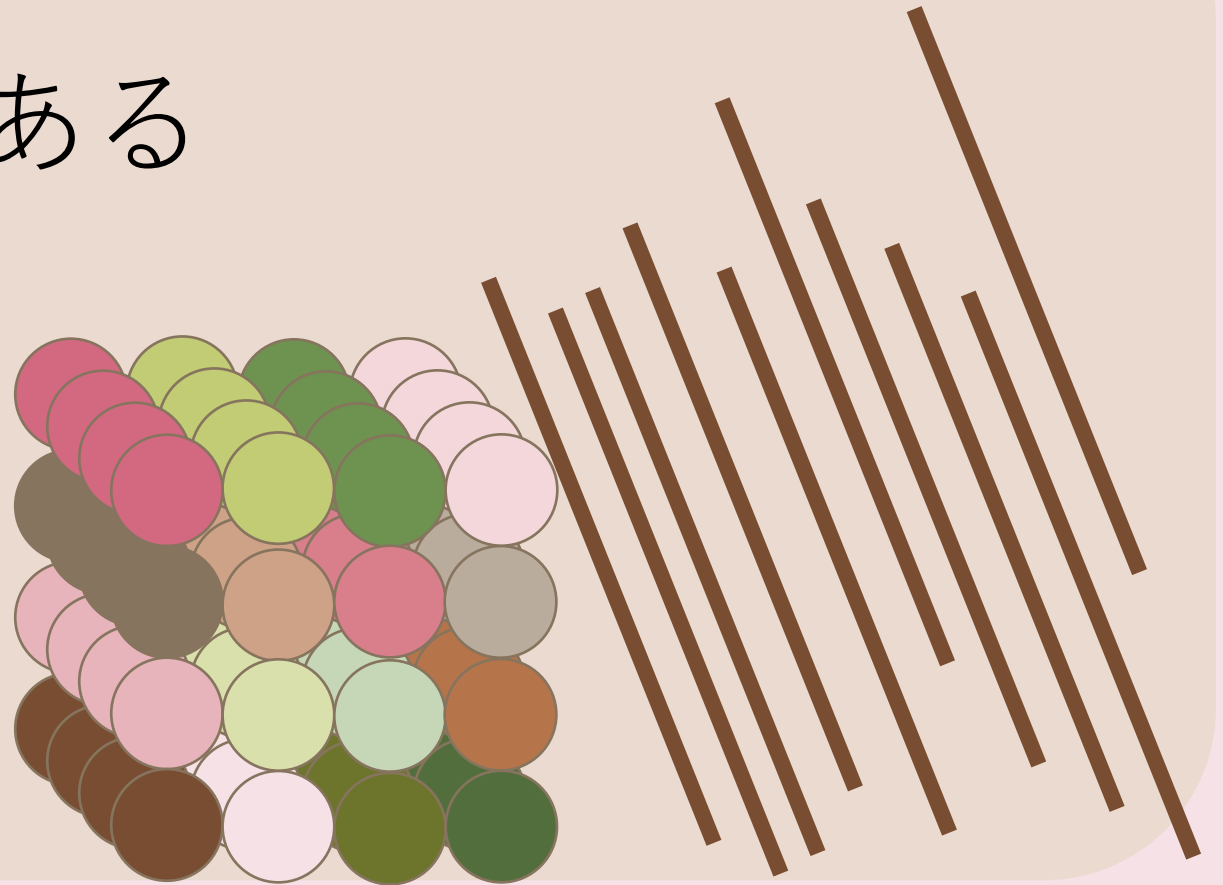
小課題 2 $N=100$ $M=10$

8

$N=100$, $M=10$

団子は $NM=1000$ 個ある

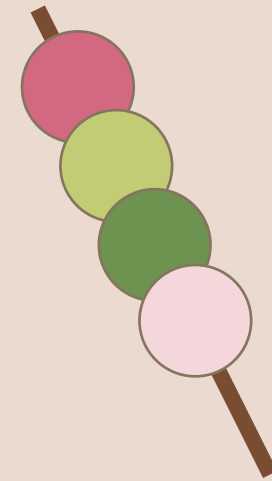
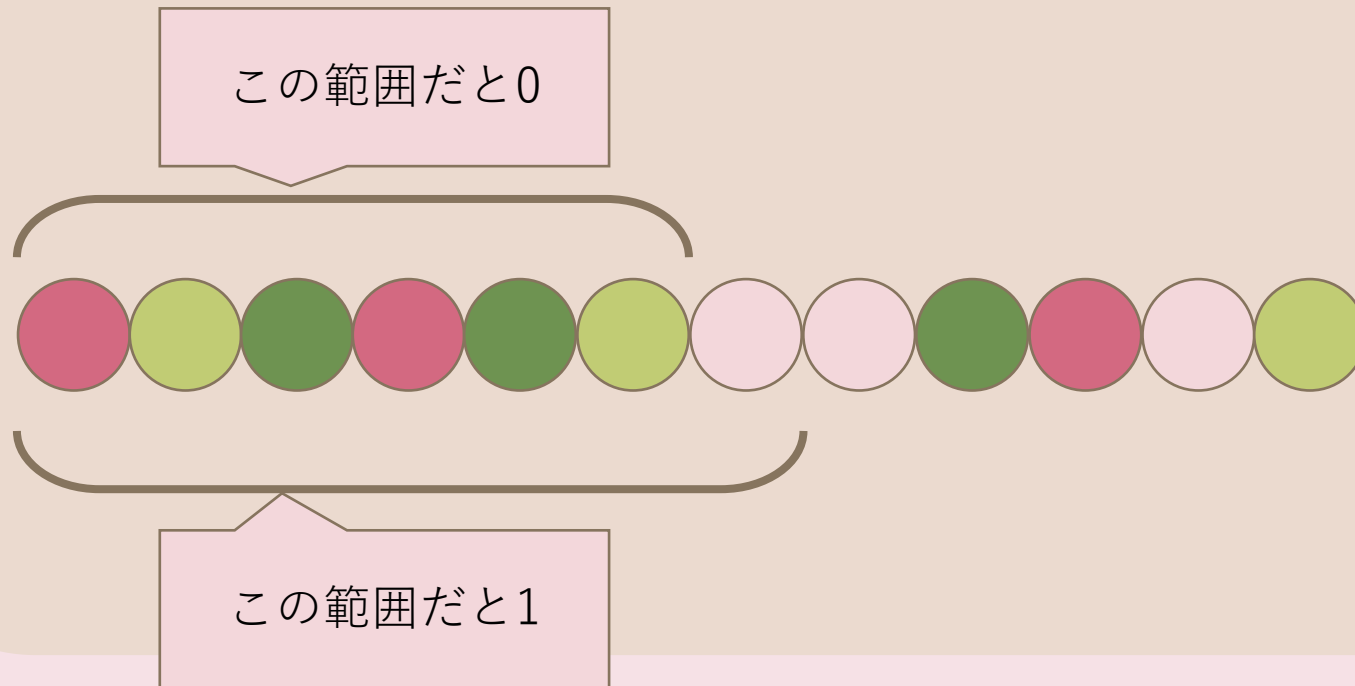
$${}_{1000}C_{10} \approx 2 \times 10^{23}$$



小課題 2 $N=100$ $M=10$

9

団子チェッカーで「団子 1 串作れる」と判定される条件
→すべての色が揃う必要あり=1つでも欠けていると判定されない

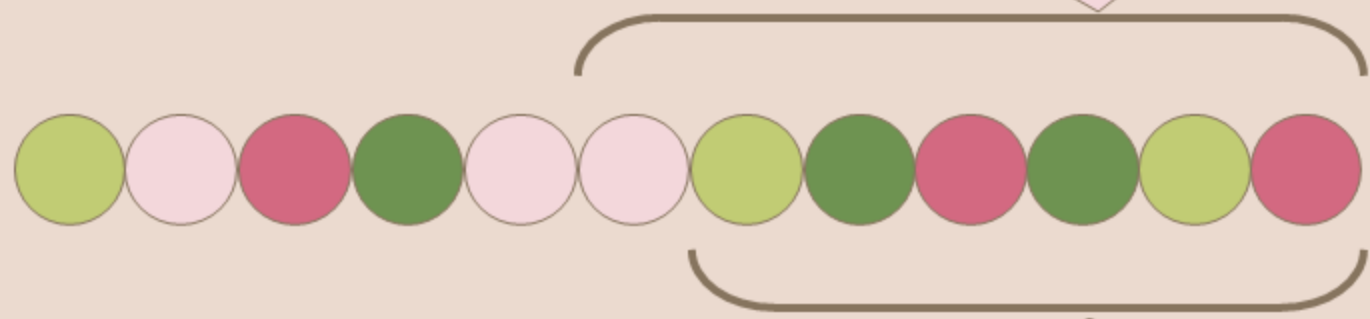


小課題2 N=100 M=10

発想を逆転させよう

因子マシーンで「因子1串作れる」と判定される条件

→オバマの角が描るが西ちいー1つズレたはマインズと判定されたい



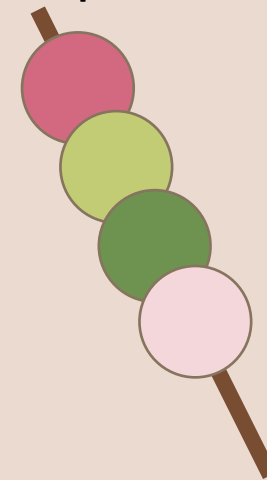
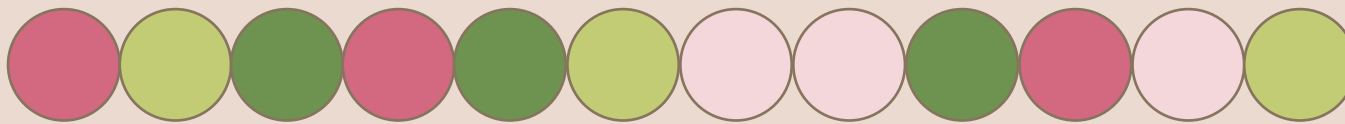
小課題 2 $N=100$ $M=10$

11

団子チェッカーで「団子 1 串作れる」と判定される条件

→すべての色が揃う必要あり=1つでも欠けていると判定されない

→全選択し、いくつか取り除いて判定



小課題 2 $N=100$ $M=10$

12

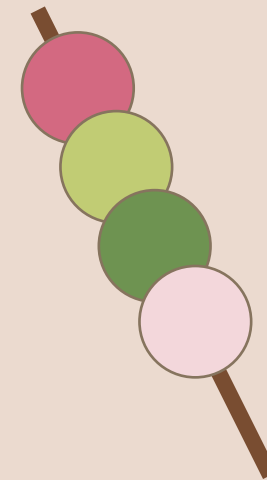
団子チェッカーで「団子 1 串作れる」と判定される条件

→すべての色が揃う必要あり=1つでも欠けていると判定されない

→全選択し、いくつか取り除いて判定

最悪はざっくり $N \times M^2/2 = 50000$ 回

これはざっくり見積もりすぎで、もうちょっと回数は少ない



小課題

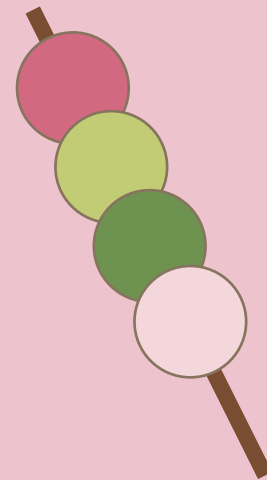
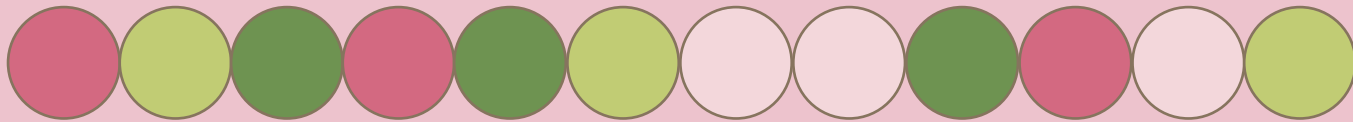
13

1. $N=4, M=4$
2. $N=100, M=10$
3. $N=200, M=25$
4. $N=400, M=25$

小課題 3 $N=200$ $M=25$

14

小課題 2: 全選択し、いくつか取り除いて判定
最悪はざっくり $N \times M^2/2$ 回

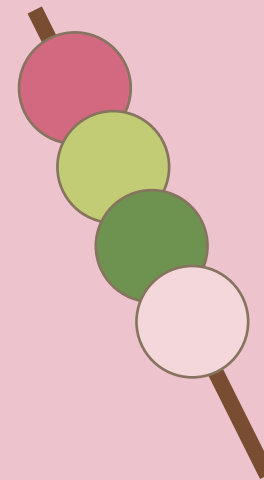
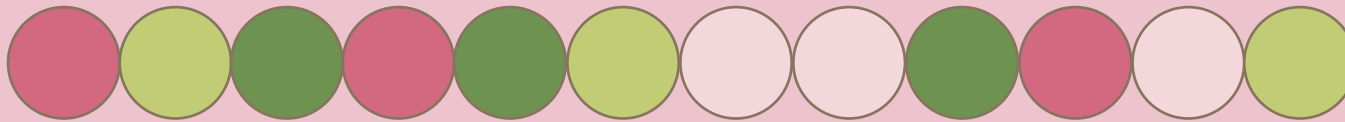


小課題 3 $N=200$ $M=25$

15

小課題 2: 全選択し、いくつか取り除いて判定

最悪はざっくり $N \times M^2/2$

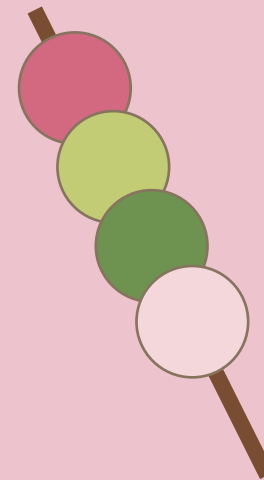
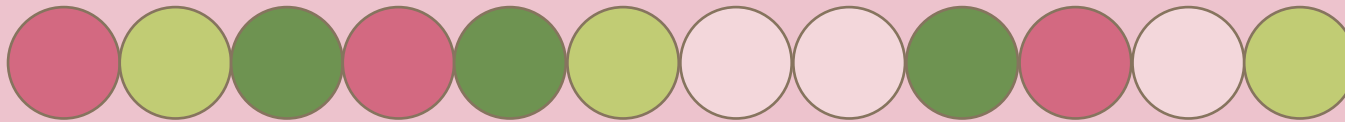


小課題 3 $N=200$ $M=25$

16

小課題 2: 全選択し、いくつか取り除いて判定

最悪 はざっくり $N \times M^2/2$



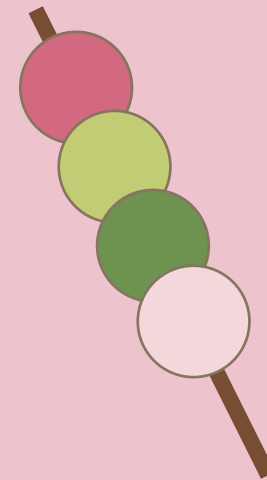
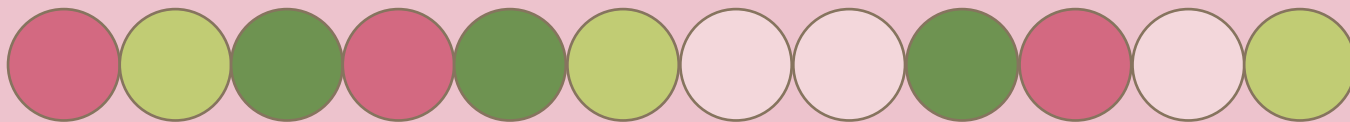
小課題 3 $N=200$ $M=25$

17

小課題 2: 全選択し、いくつか取り除いて判定

平均 はざっくり $NM \log_e N_{\text{回}}$

ランダムに選ぶと調和級数に落ちる 27000回くらい



これを撃墜するジャッジがこの世に存在する気がないので、書いてみると通ります

小課題 3 $N=200$ $M=25$

18

あまりに酷い解法を紹介してしまったので
マシな解法も紹介します

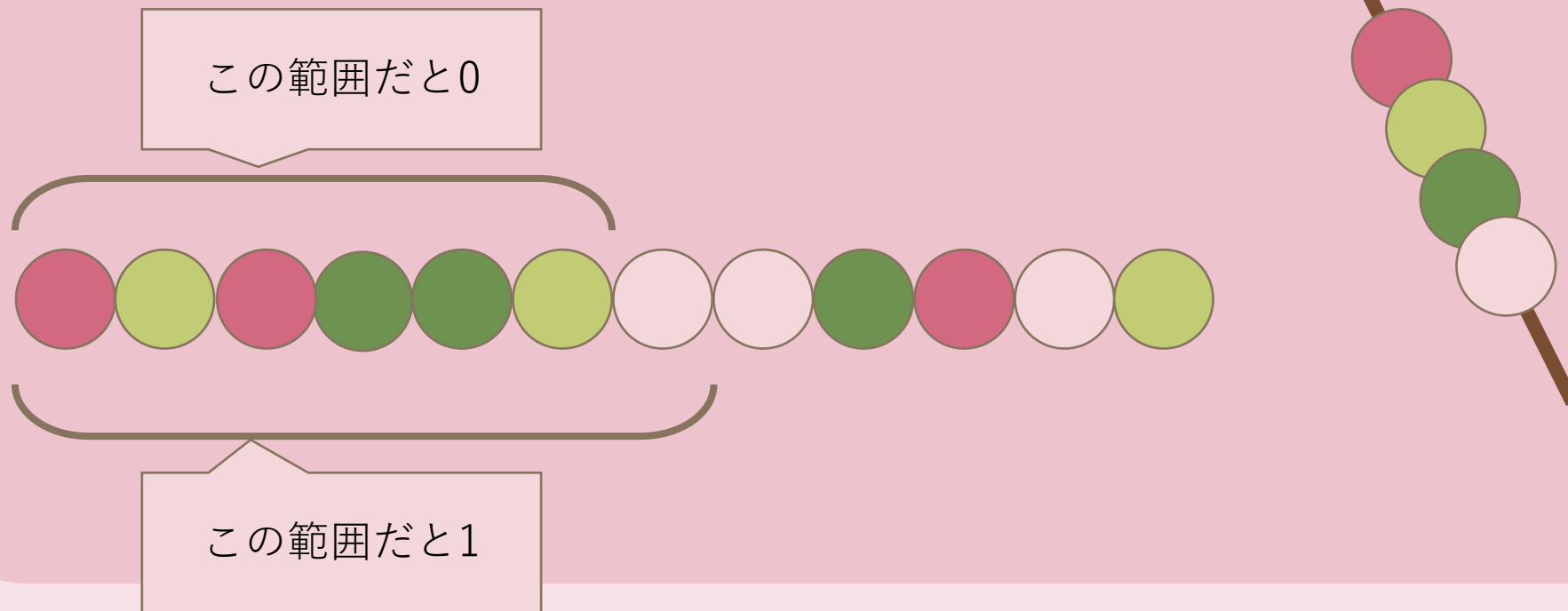
小課題 3 $N=200$ $M=25$

19

思い出そう

団子チェッカーで「団子 1 串作れる」と判定される条件

→すべての色が揃う必要あり



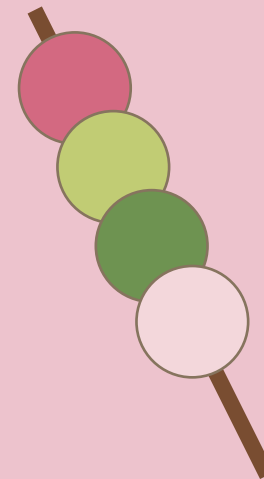
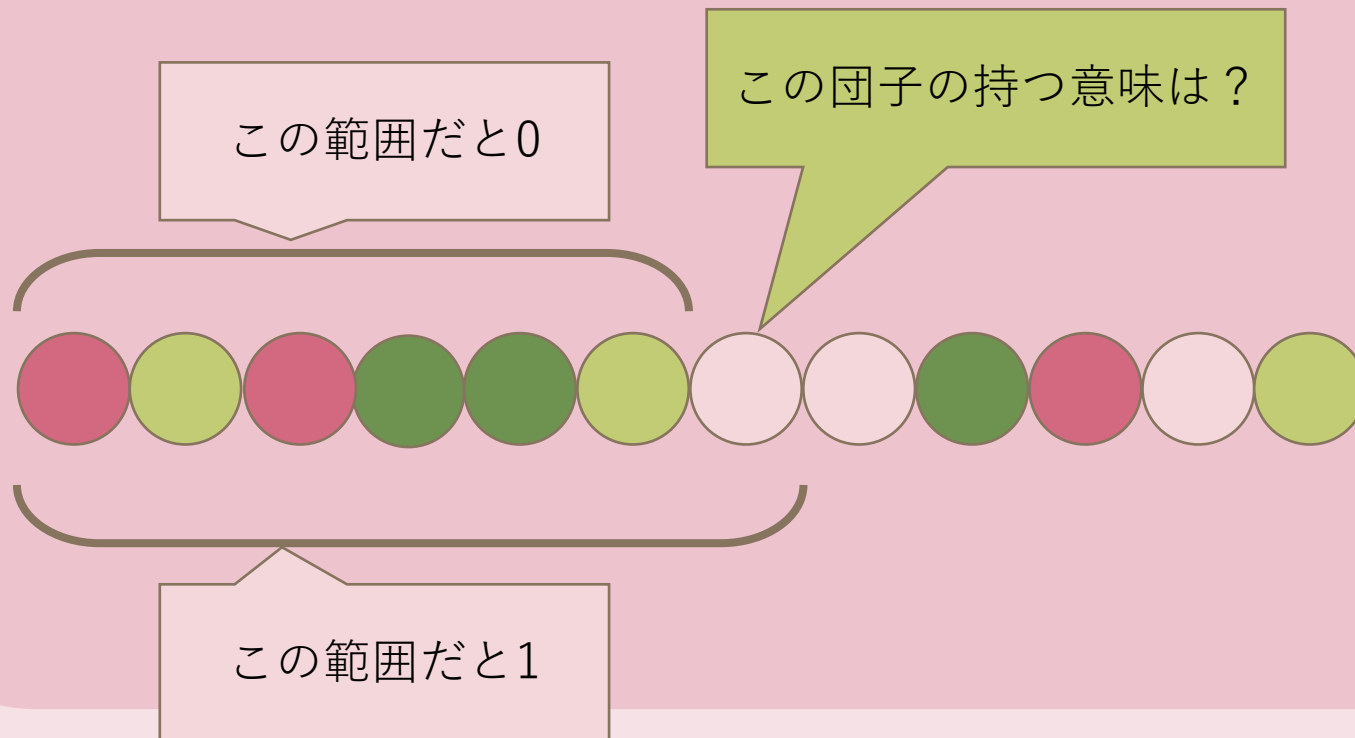
小課題 3 $N=200$ $M=25$

20

思い出そう

団子チェッカーで「団子 1 串作れる」と判定される条件

→すべての色が揃う必要あり



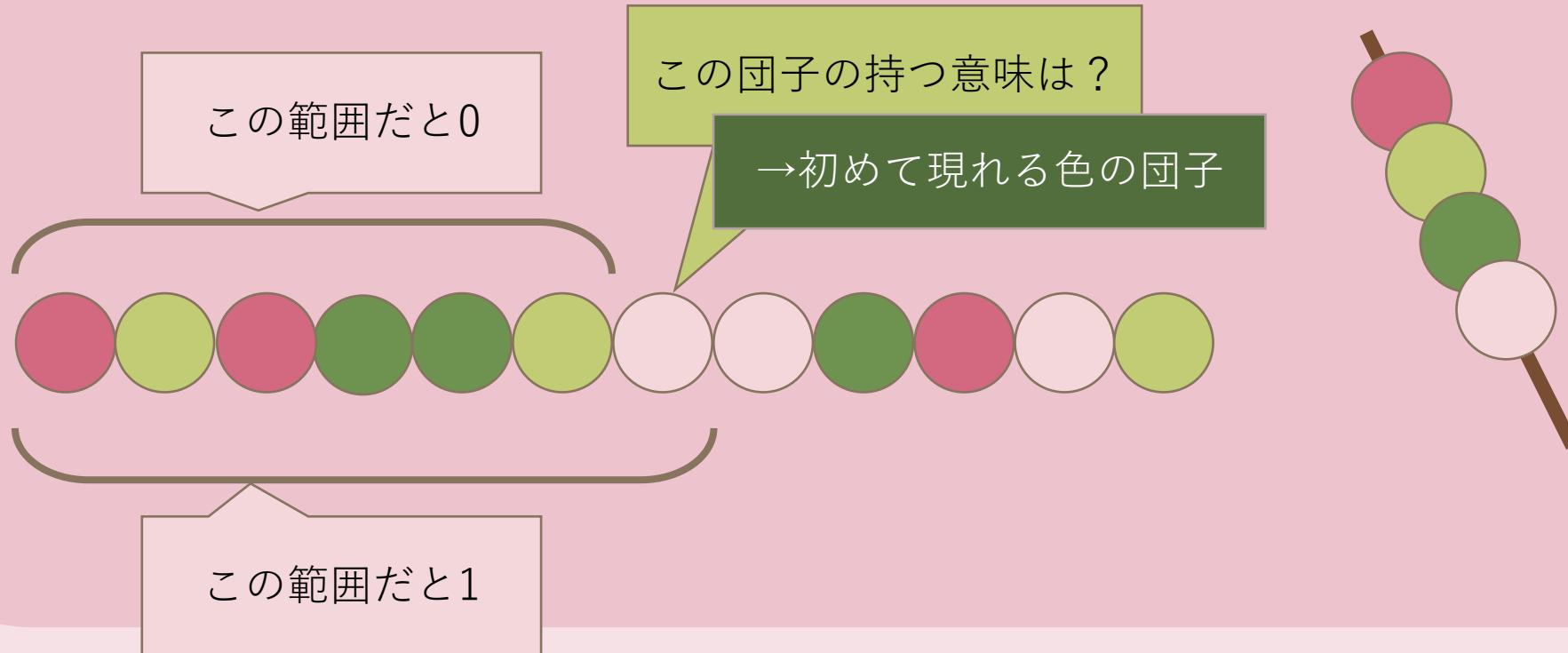
小課題 3 $N=200$ $M=25$

21

思い出そう

団子チェッカーで「団子 1 串作れる」と判定される条件

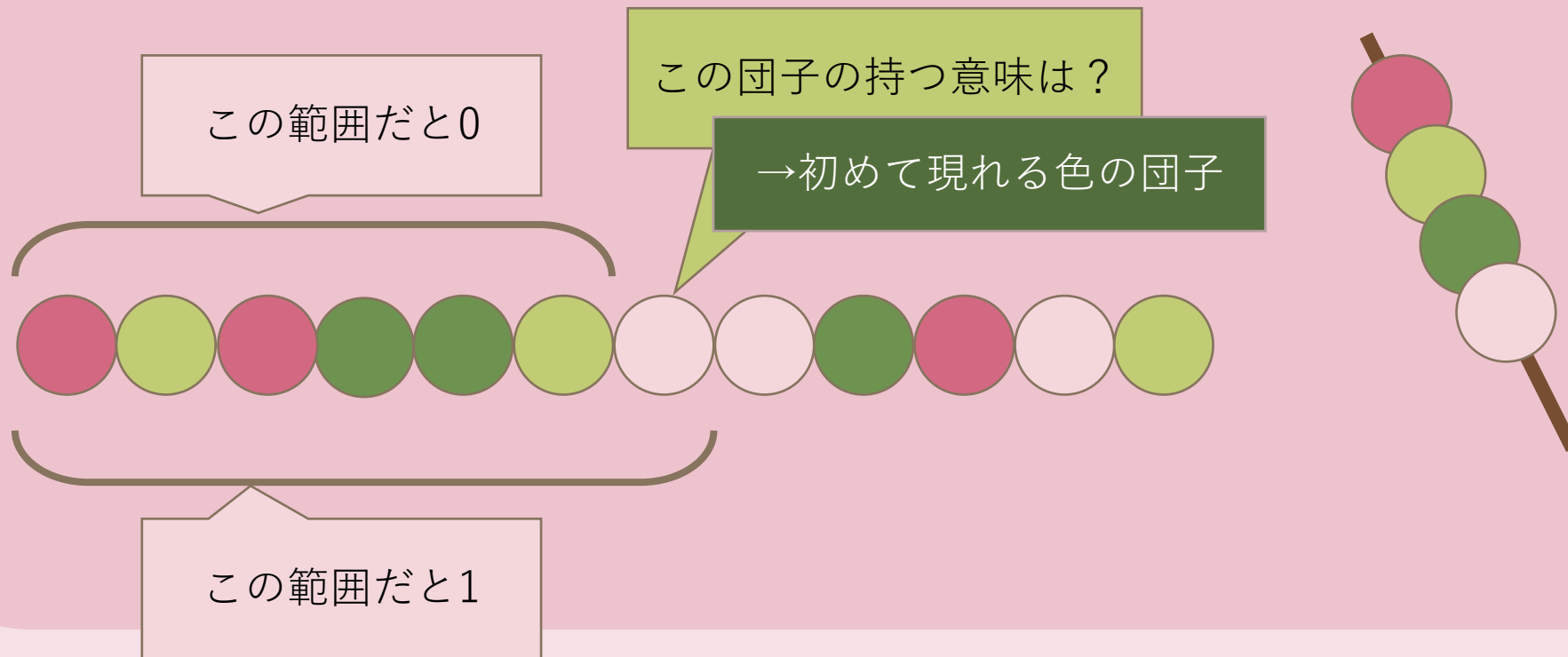
→すべての色が揃う必要あり



小課題 3 $N=200$ $M=25$

22

二分探索で「初出の団子」の位置が $\log(NM)$ 回でわかる

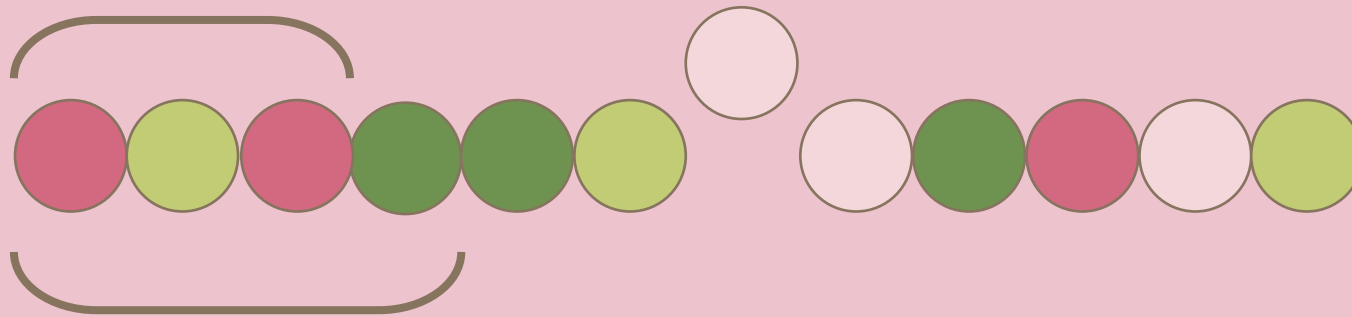


小課題 3 $N=200$ $M=25$

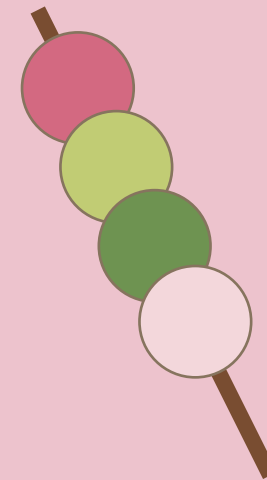
23

二分探索で「初出の団子」の位置が $\log(NM)$ 回でわかる

この範囲+  だと0



この範囲+  だと1

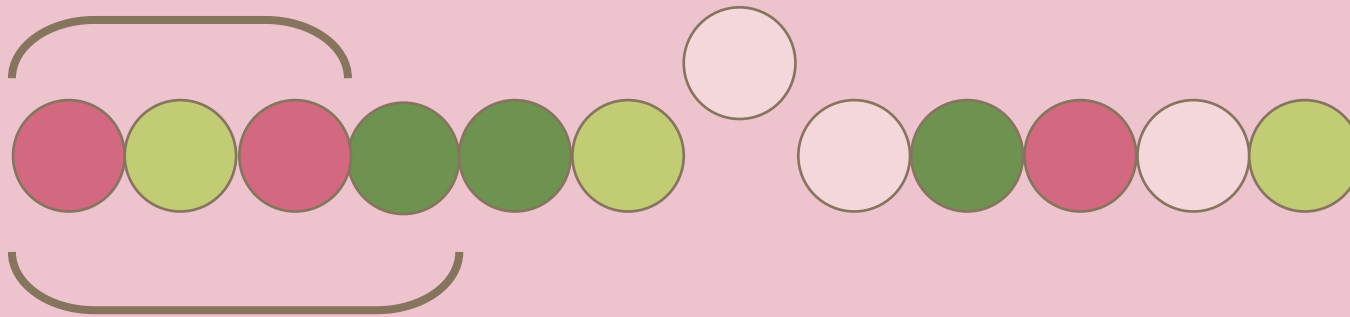


小課題 3 $N=200$ $M=25$

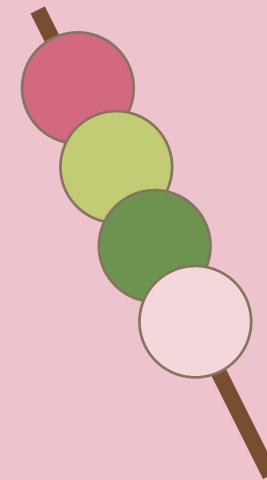
24

二分探索で「初出の団子」の位置が $\log(NM)$ 回でわかる
繰り返すと $N\log(NM)$ 回で1本の串団子が作れる

この範囲+  だと0



この範囲+  だと1



小課題 3 $N=200$ $M=25$

25

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる (スライドでは二分探索をしていない)



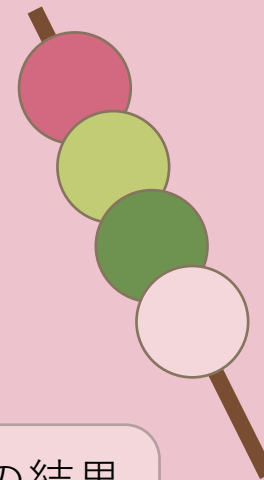
使う

余り



クエリの結果

1

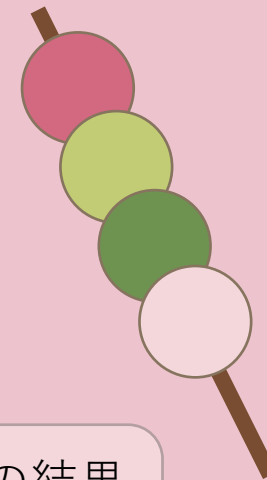
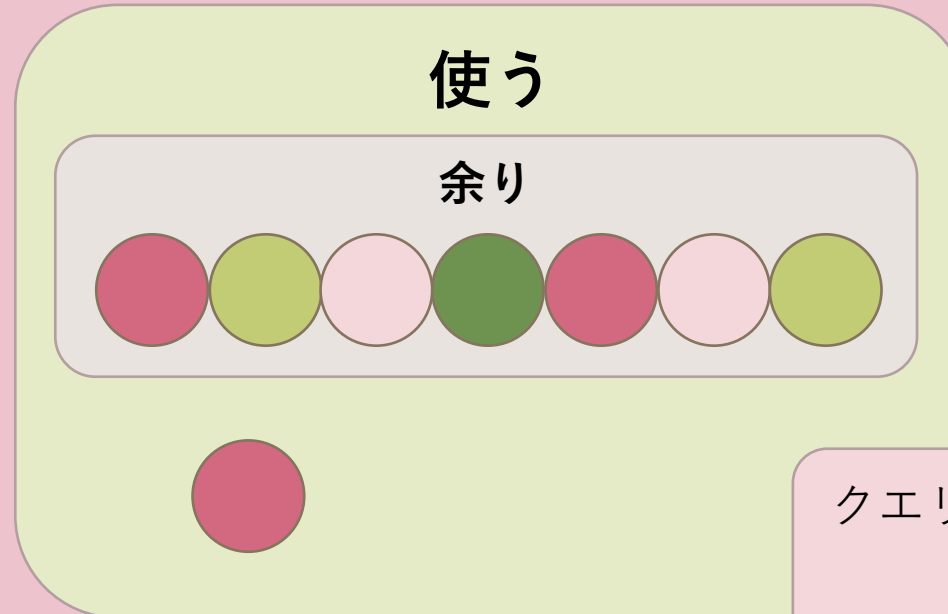


小課題 3 $N=200$ $M=25$

26

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる (スライドでは二分探索をしていない)

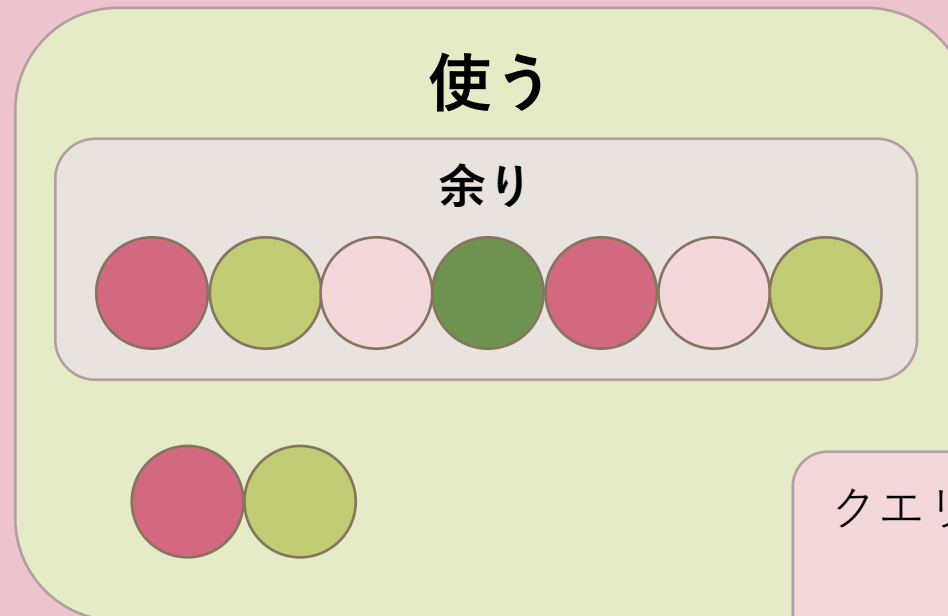
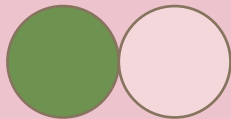


小課題 3 $N=200$ $M=25$

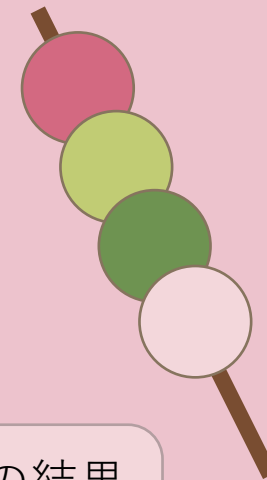
27

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる (スライドでは二分探索をしていない)



クエリの結果
1

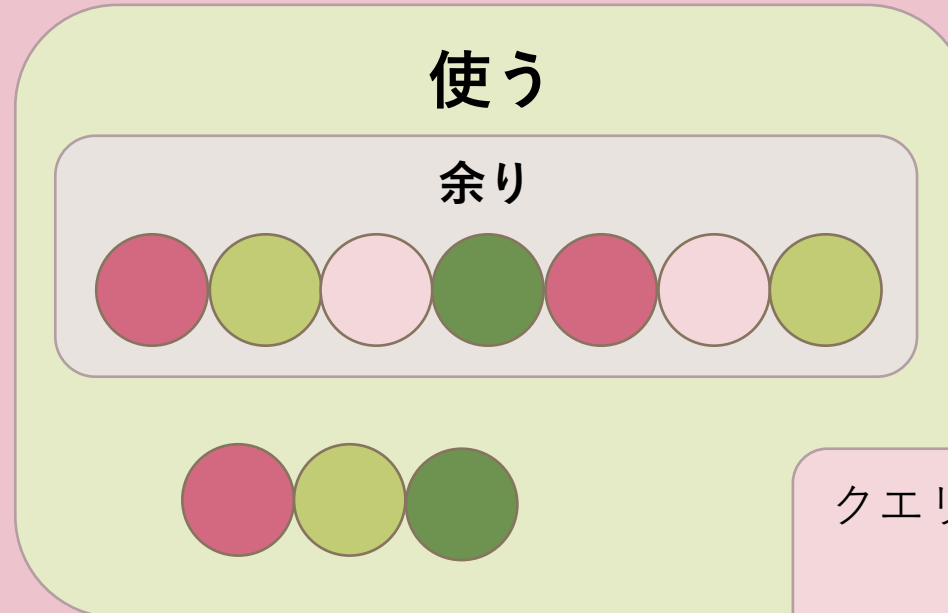


小課題 3 $N=200$ $M=25$

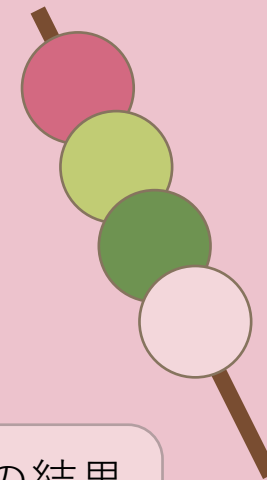
28

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる (スライドでは二分探索をしていない)



クエリの結果
2

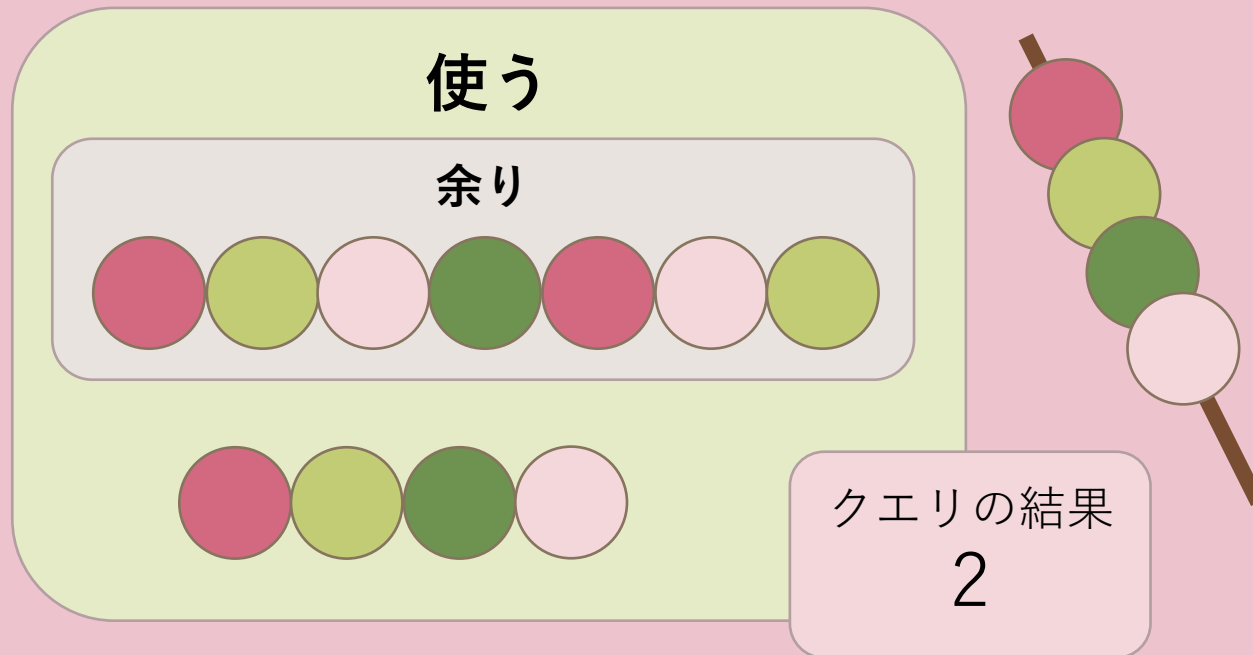


小課題 3 $N=200$ $M=25$

29

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる (スライドでは二分探索をしていない)



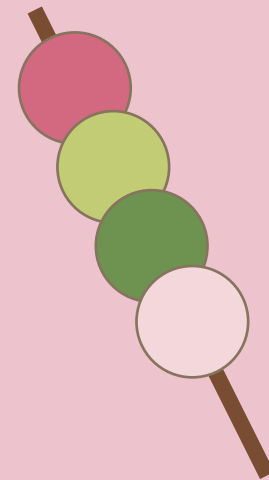
小課題 3 $N=200$ $M=25$

30

団子が1串作れたら

残ったそれぞれの団子について全体から取り除いて質問して
どの色であるか二分探索ができる

$$N \log(NM) + NM \log N = 41000 \text{ 回くらい}$$



小課題

31

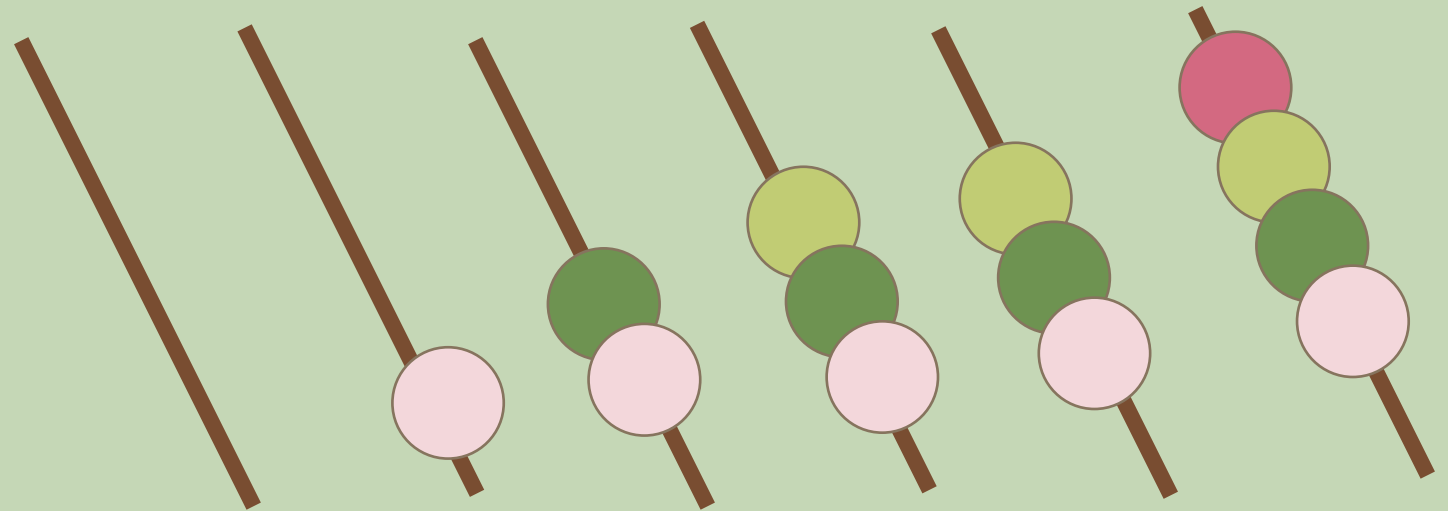
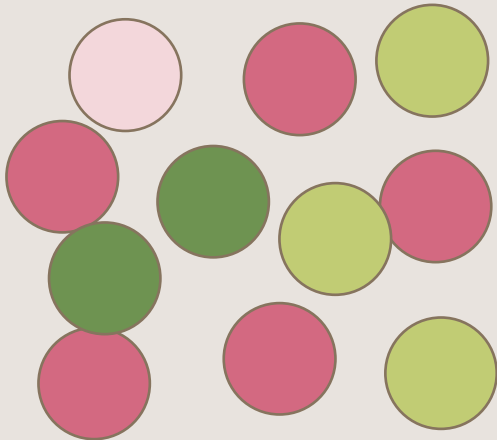
1. $N=4, M=4$
2. $N=100, M=10$
3. $N=200, M=25$
4. $N=400, M=25$

小課題 4 $N=400$ $M=25$

32

串をM本用意して 串に団子を刺していくことを考える

未確定



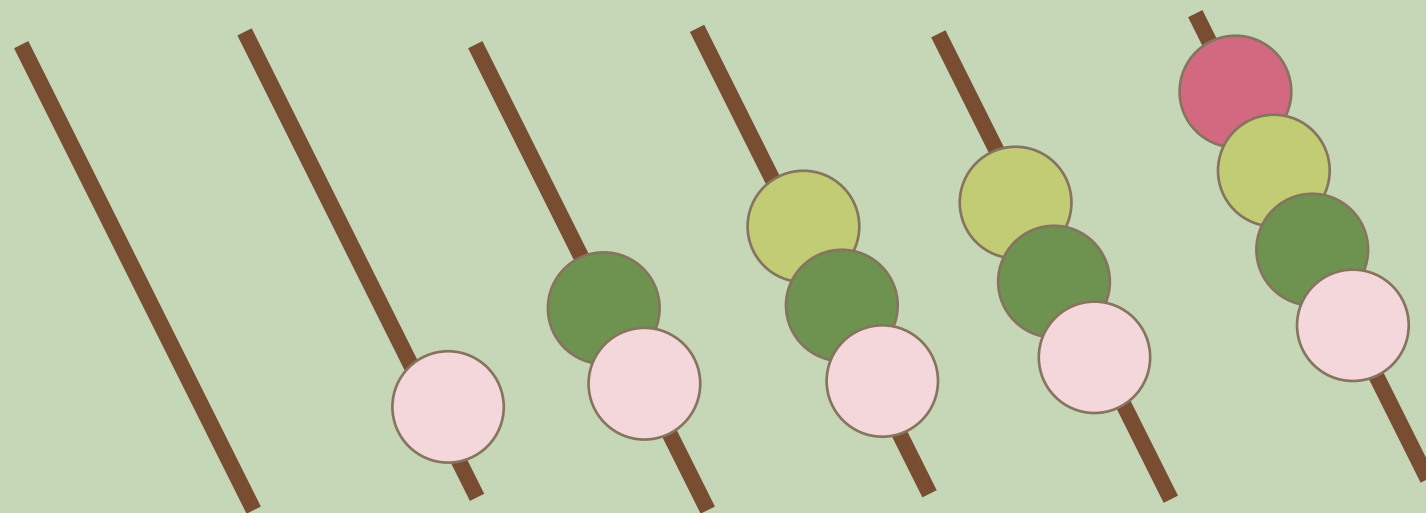
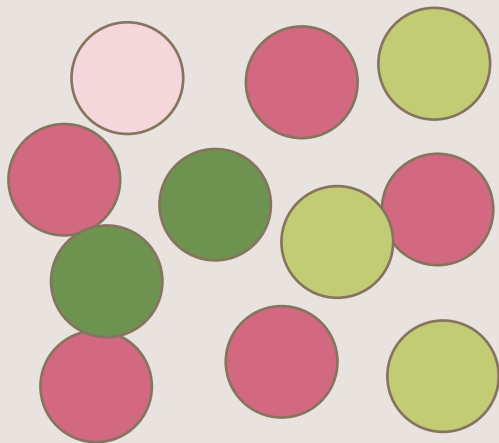
この後に説明する操作をすると 上図のように単調性をもつように串に団子を刺せる

小課題 4 $N=400$ $M=25$

33

串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する

未確定



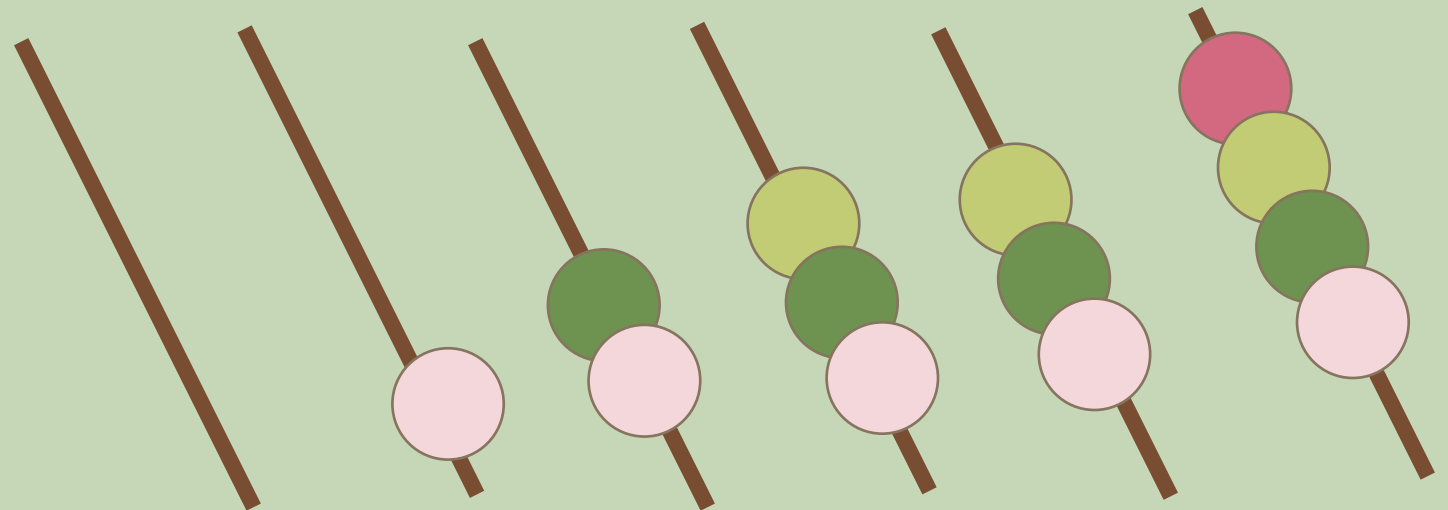
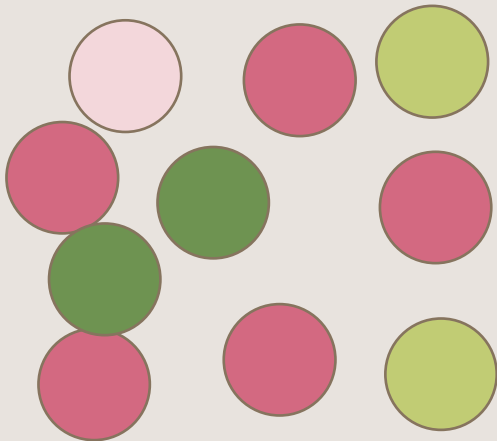
この後に説明する操作をすると 上図のように単調性をもつように串に団子を刺せる

小課題 4 $N=400$ $M=25$

34

串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)

未確定



←どこに刺せるか知りたい

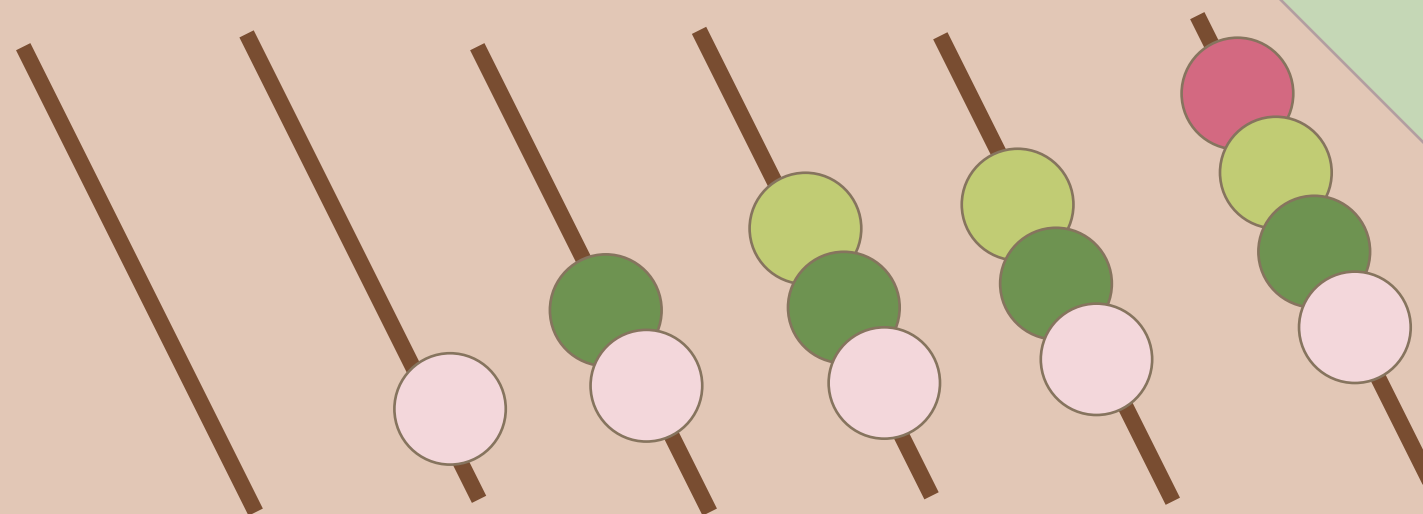
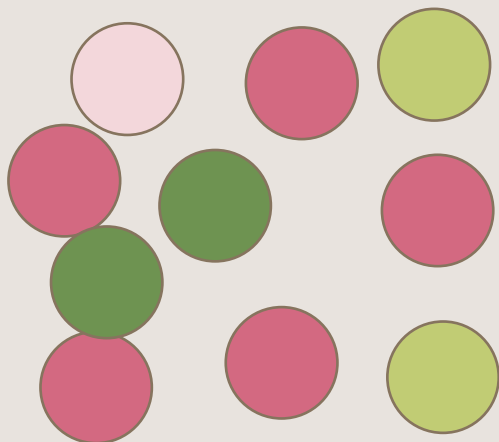
小課題 4 $N=400$ $M=25$

35

串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)

使う

未確定



クエリの結果
5

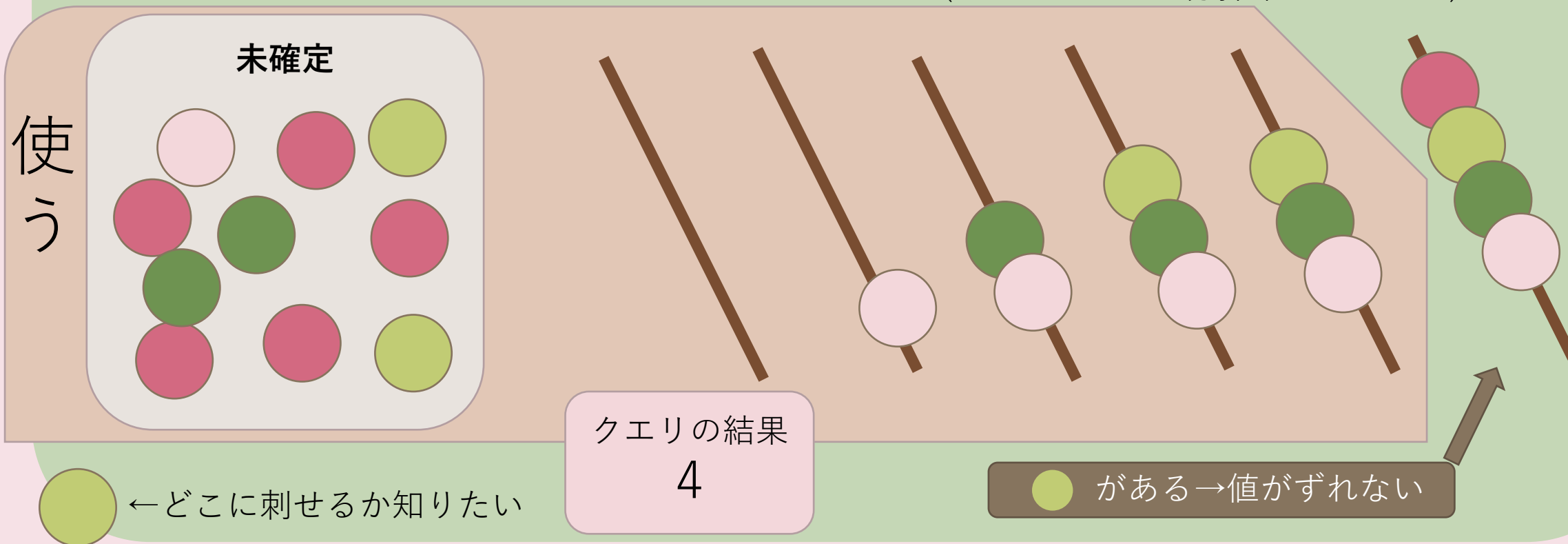
←どこに刺せるか知りたい

● がある → 値がずれない

小課題 4 $N=400$ $M=25$

36

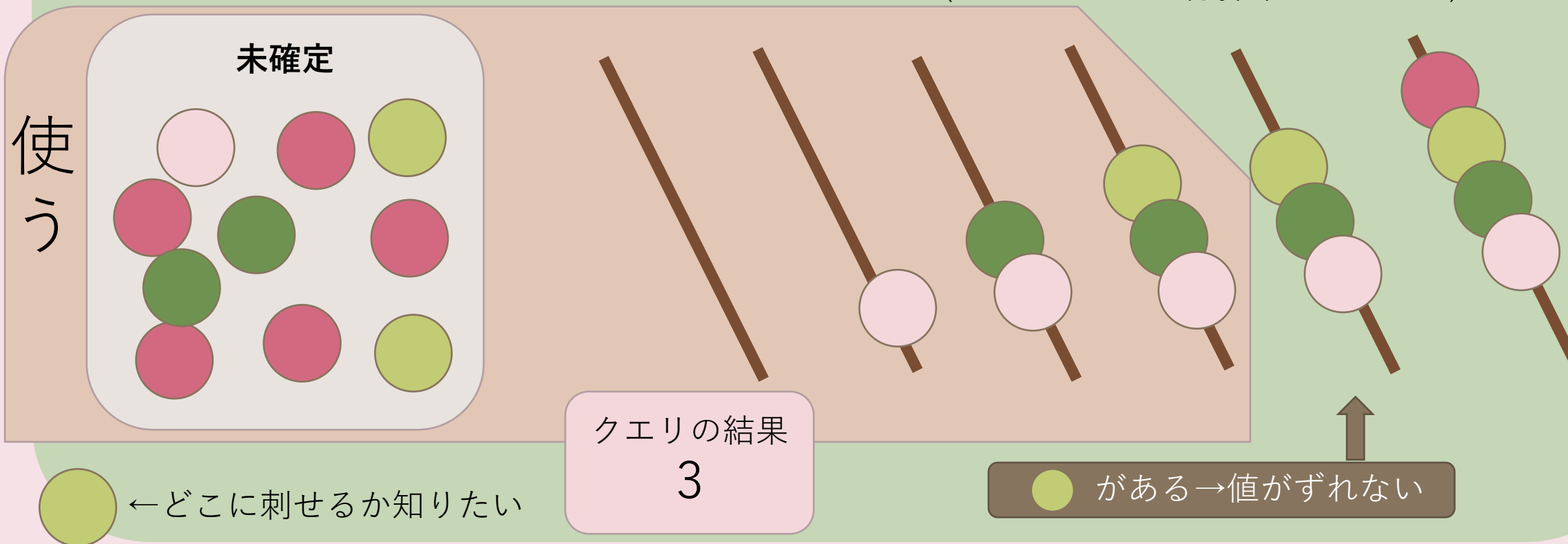
串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)



小課題 4 $N=400$ $M=25$

37

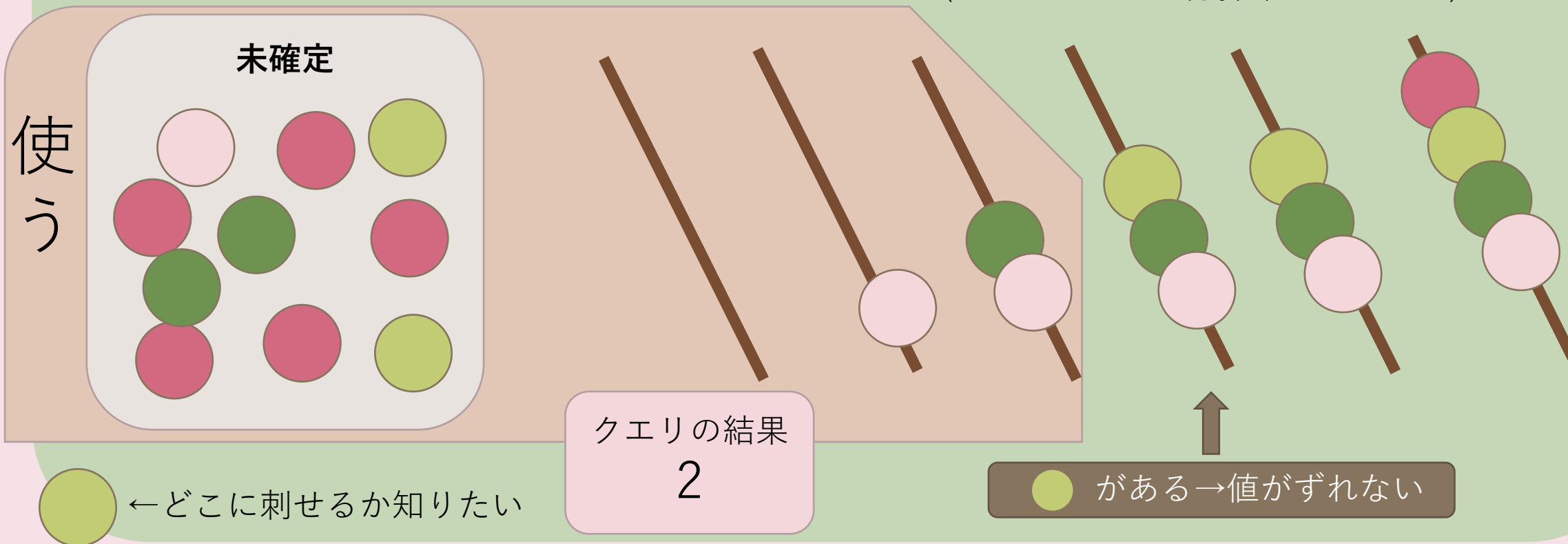
串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)



小課題 4 $N=400$ $M=25$

38

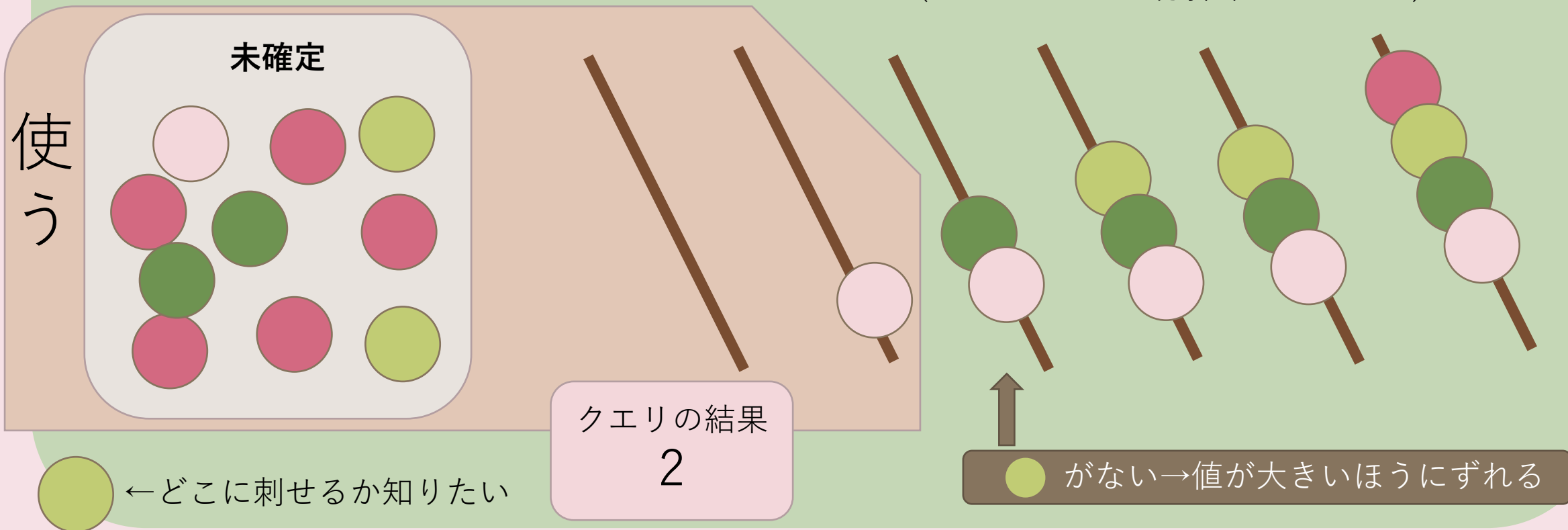
串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)



小課題 4 $N=400$ $M=25$

39

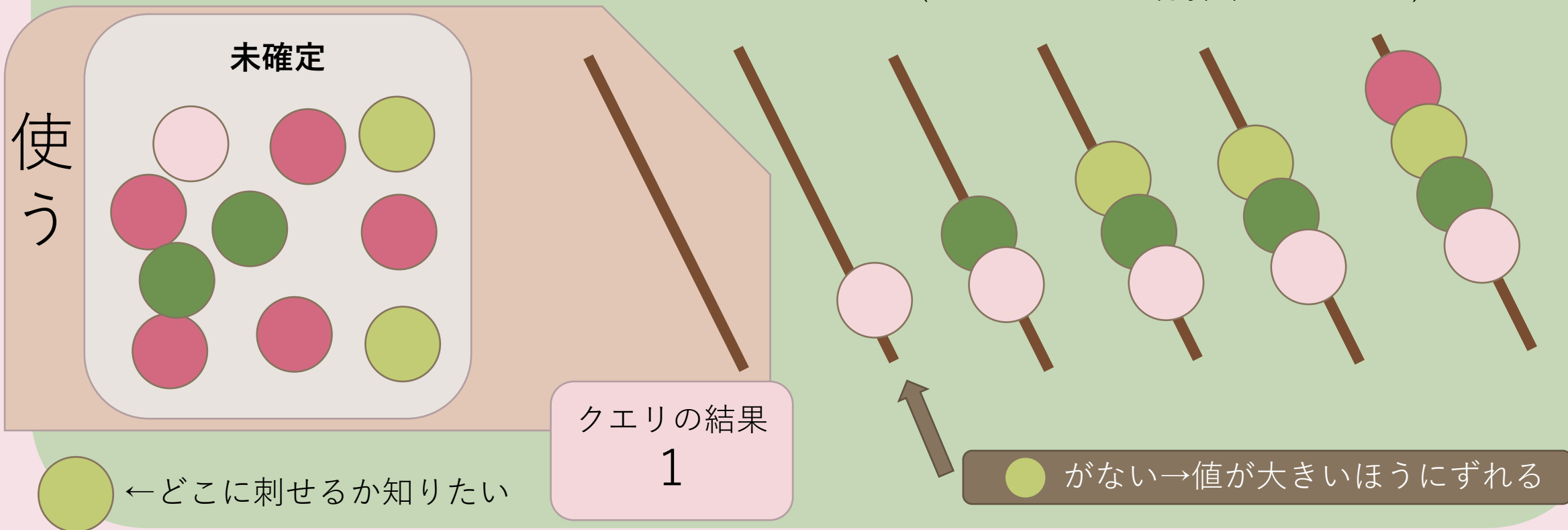
串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)



小課題 4 $N=400$ $M=25$

40

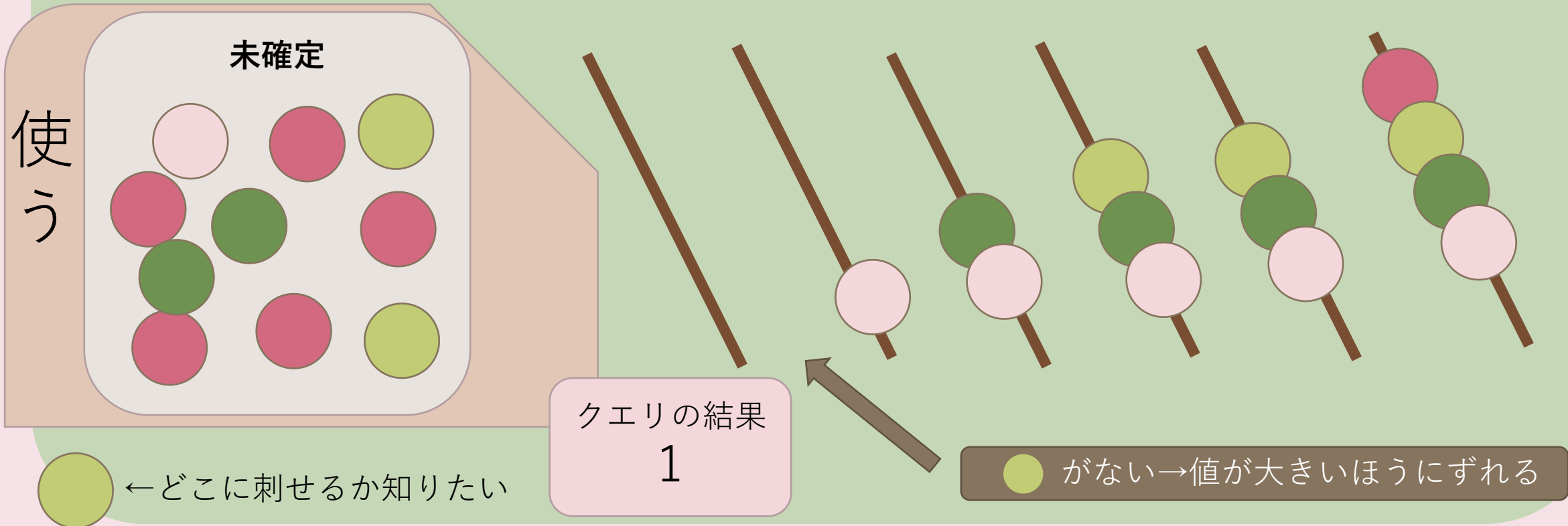
串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)



小課題 4 $N=400$ $M=25$

41

串をM本用意して 串に団子を刺していくことを考える
どこの串に刺せるかを二分探索する (スライドだと二分探索していない)

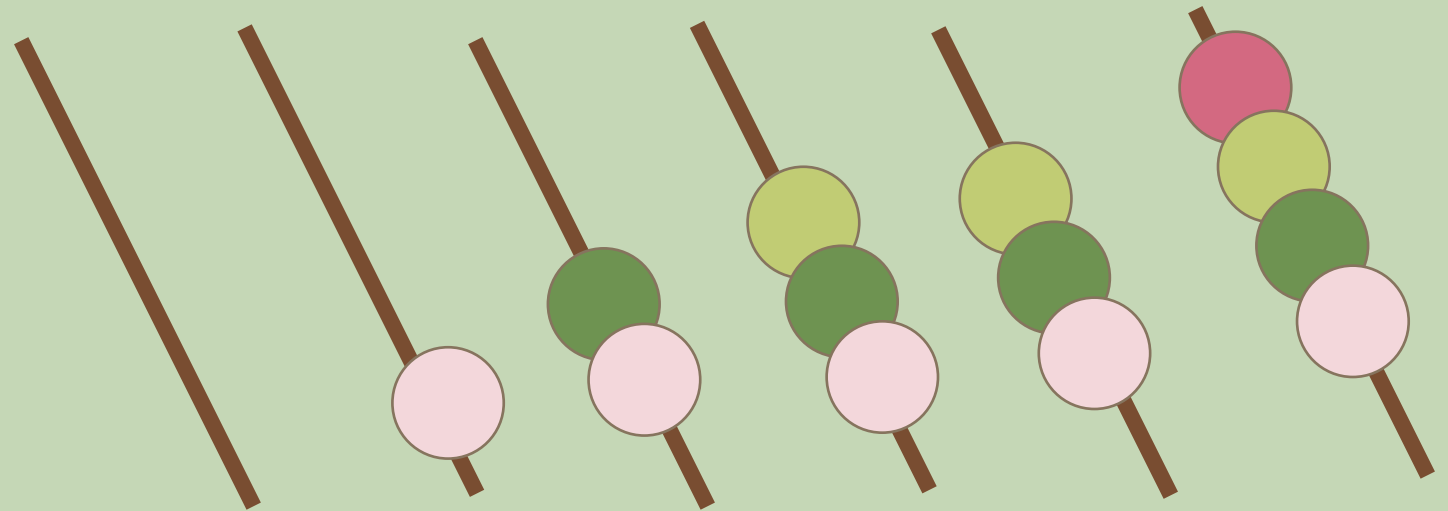
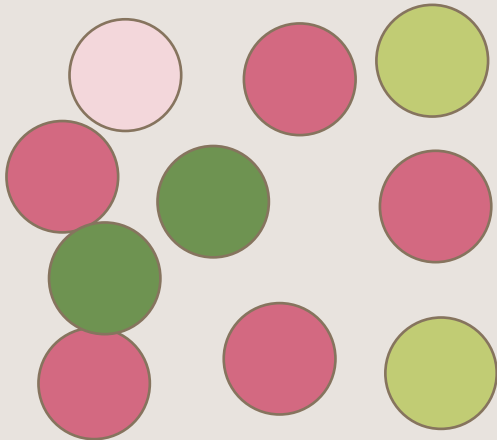


小課題 4 $N=400$ $M=25$

42

$NM \log M = 45000$ くらい

未確定



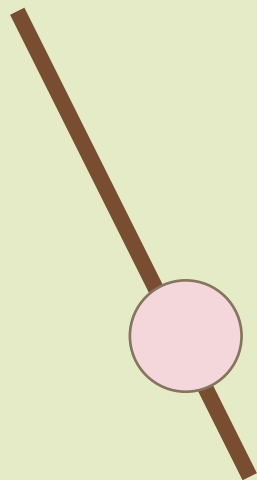
←どこに刺せるか知りたい

得点分布 JOI

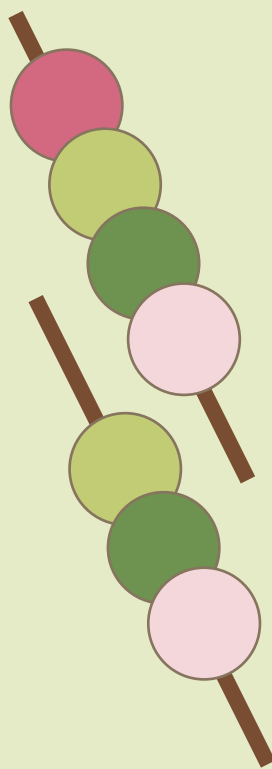
43



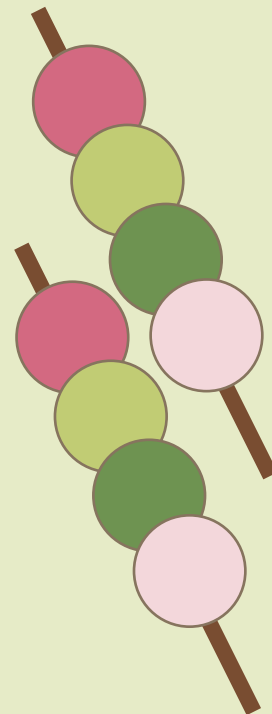
0



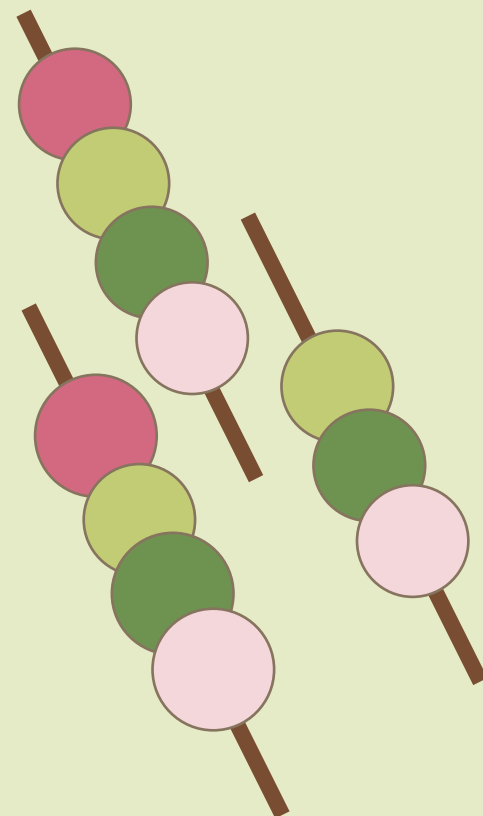
2



7



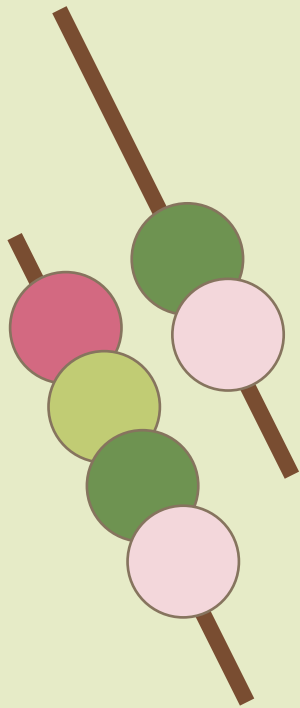
22



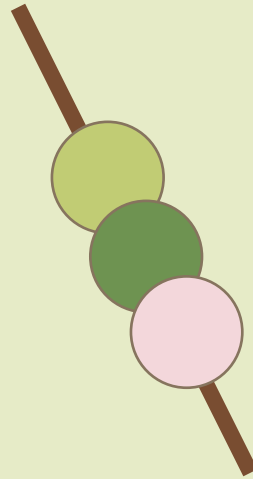
100

得点分布 JOIG

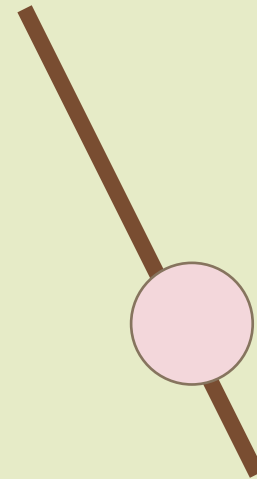
44



0



5



100

魚2 (Fish2)

解説 nxteru (戸高空)

問題概要

- N 匹の魚が横一列に並んでいる
- 左から i 番目の魚の大きさは A_i である
- Q 個のクエリ
- $T_j = 1$ X_j 番目の魚の大きさを Y_j にする
- $T_j = 2$ L_j から R_j 番目までの魚を取りだして以下の問題を解く

問題概要

- ある隣合う2匹の魚で大きい方が小さい方を食べて、食べた魚の大きさ分だけ大きくなるということが残り1匹になるまで繰り返される
- 最後の1匹として残る可能性がある魚の数を求めよ
- $N, Q \leq 100000$ 、魚の大きさ $\leq 10^9$

簡単な考察

- ある魚 s が残ることができるかを考える
 - このとき s が隣り合う魚を食べる以外は考えなくてよい
 - また s が隣り合う魚を食べられる場合は左右どちらでも食べてよい
-
- よって「 s が隣り合う魚を食べられるなら食べる」を繰り返してすべての魚を食べられるかが分かればよい

小課題1

- $N, Q \leq 500$
- すべての魚についてその魚が残ることができるかを調べる
- 各魚 $O(N)$ でシミュレーションできるのでクエリ当たり $O(N^2)$ で解ける
- $O(QN^2)$

小課題2

- $Q = 1, T_j = 2, L_j = 1, R_j = N$
- クエリが1回
- シミュレーションを高速化する

小課題2

- 右に向かってできるだけ食べることを考えると、今の大きさ以下の魚が連続していたら、その魚たちは大きさの加算分を計算しなくても食べられることが分かる
- 初めて計算が必要になるのは今の大きさより大きい魚が初めて現れる場所
- そのような場所は例えばsegment tree上の二分探索で $O(\log N)$ で求められる

小課題2

- そのような右に自分より大きな魚が初めてでてくる場所の手前まで食べてしまい、その合計だけ大きさの加算する
- このように右に進めるだけ進む、左に進めるまで進むを繰り返してシミュレーションする
- この左右に伸ばす回数は $\log_2 \max\{A\}$ 回で抑えられる
- 前の自分より大きな魚を食べると大きさが2倍以上になるためである
- $O(N \log A \log N)$

考察

- シミュレーションにおいて状態は $[l, r]$...「 $l \sim r$ 番目の魚を食べて今の大きさは $A_l + A_{l+1} + \dots + A_r$ であり左隣が $l - 1$ 、右隣が $r + 1$ である」というような区間であらわされる
- クエリは $[i, i]$ からスタートして $[L, R]$ に到達できるような i はいくつあるかを答える問題である

考察

- ここで左にも右にも進めない状態、つまり

$$A_l + A_{l+1} + \cdots + A_r < A_{r+1} \text{ かつ}$$

$$A_l + A_{l+1} + \cdots + A_r < A_{l-1}$$

を満たす区間を閉塞区間とする

考察

- 実は「ある魚が区間 $[L, R]$ に到達できる $\Leftrightarrow$ その魚が $[L, R]$ の範囲のどの閉塞区間にも含まれていない」が成り立つ
- 閉塞区間 $[l, r]$ に含まれている $\Rightarrow$ 魚 $l - 1$, 魚 $r + 1$ を食べることができない
- 閉塞区間に含まれていない $\Rightarrow$ 今の状態が閉塞区間でないなら必ず左右のどちらかに区間を伸ばせる. 閉塞区間に到達することはないので、これを繰り返すと $[L, R]$ に到達できる

イメージ(時間があれば追加する)

- 時間がありませんでした

小課題3

- この閉塞区間を求める
- 閉塞区間 $[l, r]$ に関して

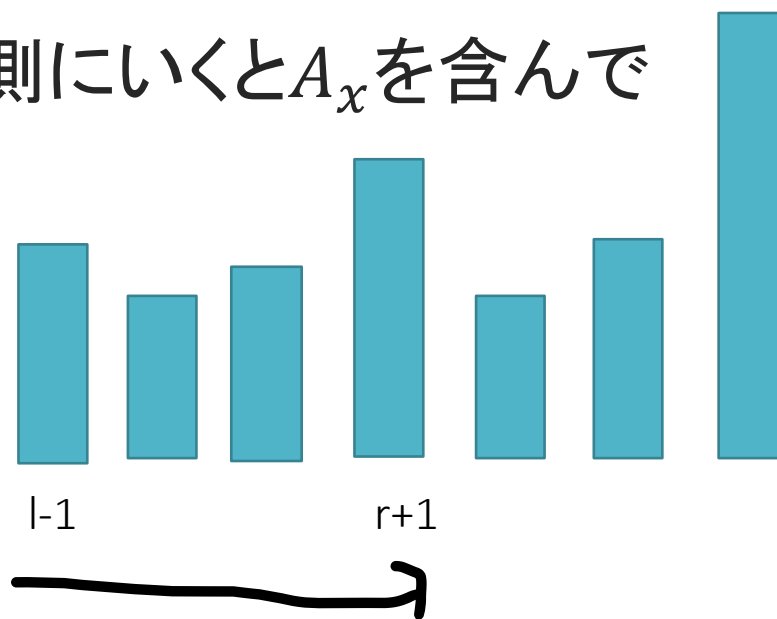
$$A_l, A_{l+1}, \dots, A_r < A_{r+1}$$

$$A_l, A_{l+1}, \dots, A_r < A_{l-1}$$

が成り立つがこのような $l - 1, r + 1$ の組は $O(N)$ 個しかない

小課題3

- $A_{l-1} < A_{r+1}$ であるような組を考える
- $l-1$ を固定したとき $r+1$ として考えられるのは $A_{l-1} < A_x$ を初めて満たす x のみである
- なぜなら $r+1$ がそのような x より右側にいくと A_x を含んで $A_l, A_{l+1}, \dots, A_r < A_{l-1}$ を満たさなくなる



小課題3

- 同様に $A_{l-1} \geq A_{r+1}$ であるような組についてもそれぞれの右端について左端の候補は1つである
- これらの条件を満たす組はstackを用いて左から列を走査することで $O(N)$ ですべて求めることができる

小課題3

- A_i を見るとき、stackのtopが A_i 以下の間stackをpopし続ける
- そして A_i をstackに入れる
- このpopのときpopされる側にとって A_i が右側で初めて自分以上の場所
- A_i をstackに入れるときのstackのtopが i にとって左側で初めて自分より大きくなる場所

小課題3

- クエリあたり $O(N)$ で閉塞区間を求めて、それらの区間に含まれていない魚の数を求められる
- $O(QN)$

閉塞区間についての考察

- 閉塞区間同士は交差せず内包するか重ならないかのどちらかである
- 右下のように $[a + 1, b - 1]$ と $[c + 1, d - 1]$ が交差したとすると
- c が閉塞区間 $[a + 1, b - 1]$ に含まれるので $c < b$
- b が閉塞区間 $[c + 1, d - 1]$ に含まれるので $b < c$
- で矛盾



小課題4

- $T_j = 2$ (更新クエリがない)
- 最初に閉塞区間を求めておくと更新がないのでそれらを使うことができる

小課題4

- クエリ $[L, R]$ にこたえる
- 範囲 $[L, R]$ のみを考えたとき $L - 1$ に進めなくなるため, $L - 1$ を左の壁とした新たな閉塞区間ができることがある
- 同様に $R + 1$ を右の壁とした閉塞区間ができる

小課題4

- そのような $L - 1$ を左の壁とする閉塞区間にどこまで含まれるか、つまり右の壁として条件を満たす一番右のものを求める
- その条件は $sum[i]$ を $A_1 + A_2 + \dots + A_{i-1}$ として

$$sum[x] - sum[L] < A_x \Leftrightarrow -sum[L] < A_x - sum[x]$$

である x だから $A_x - sum[x]$ の配列で $-sum[L]$ 以上のもので R 以下もつとも右を求められればいい

Segment tree 上の二分探索で $O(\log N)$

小課題4

- こうして新たにできた閉塞区間に含まれる左右の部分を除いた範囲をあらたに $[L, R]$ としてクエリにこたえる
- すると $[1, N]$ の範囲で求めた最初からある閉塞区間のみを考えて、 $[L, R]$ の範囲でそれに含まれない場所の数を求める問題になる

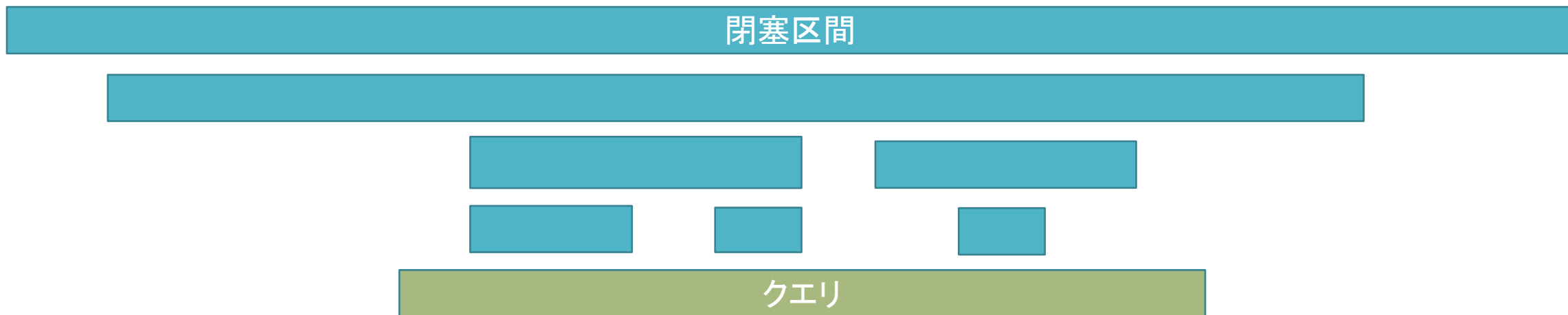
小課題4

- もう一つのうれしい性質
- この新たなクエリの区間と閉塞区間は交差せず、内包するか重ならないかのどちらかである
- 例えば図のように交差しているとすると、 $A_{a+1} + \dots + A_{b-1}$ は A_a 以下だからクエリの $a + 1$ 以降の部分はさきほどの処理で除かれているはずである



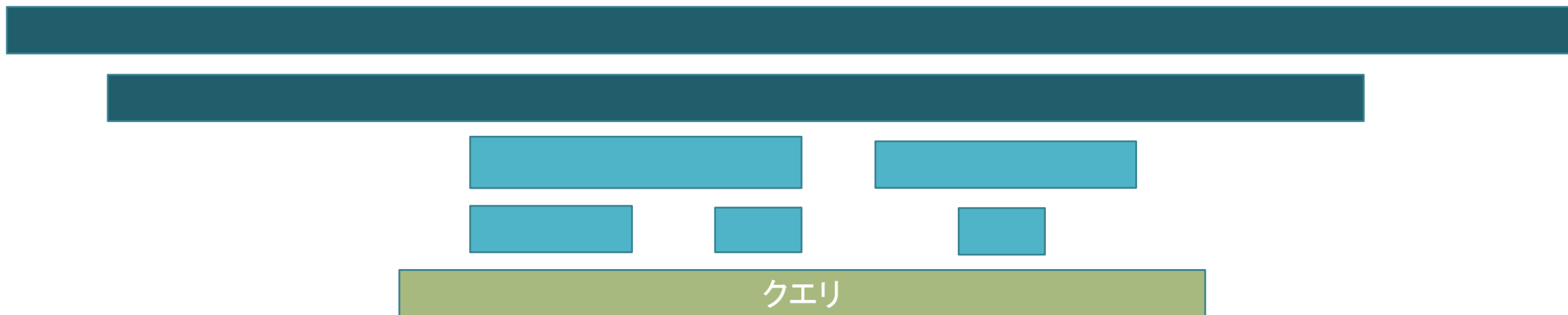
小課題4

- 閉塞区間がクエリを含むか、クエリが閉塞区間を含むという関係



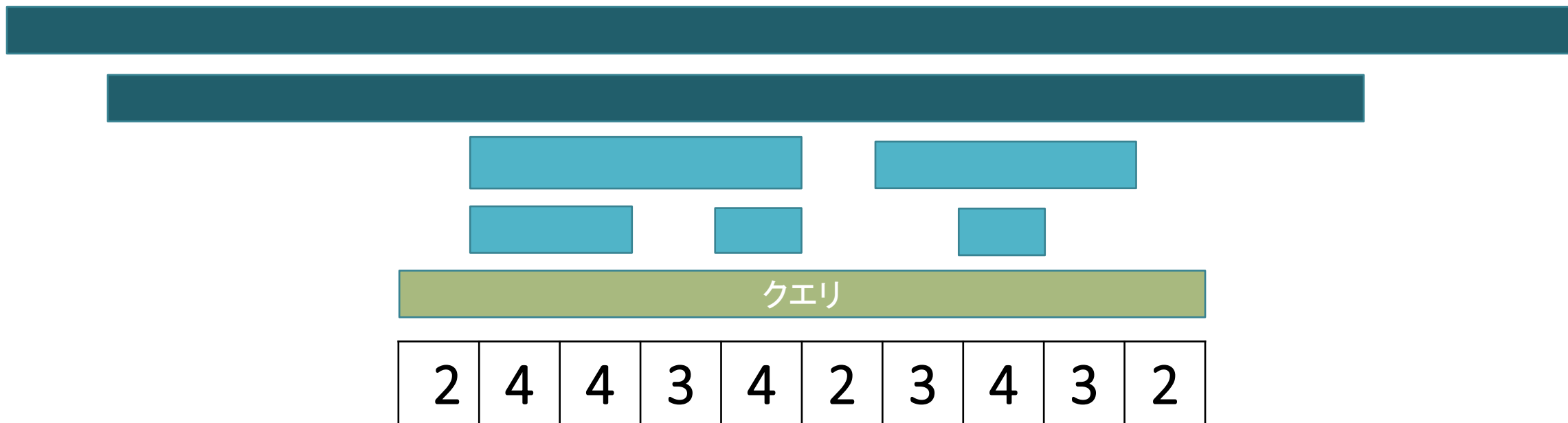
小課題4

- 閉塞区間がクエリを含むか、クエリが閉塞区間を含むという関係
- クエリが含む閉塞区間のみを考えたときクエリの範囲でそれらに含まれない場所の数を求めたい



小課題4

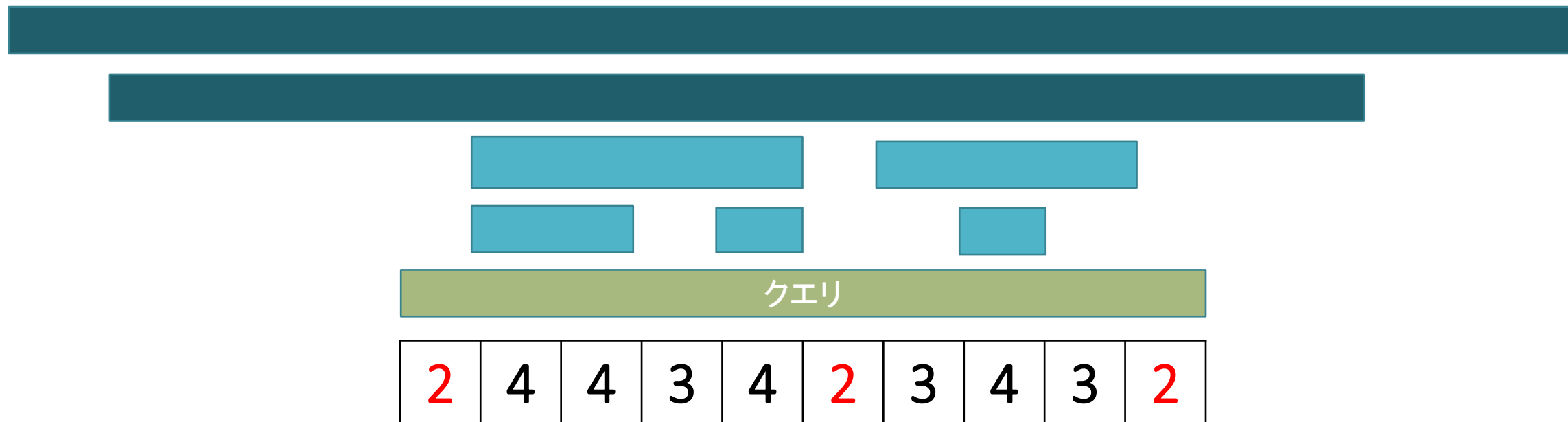
- それぞれの場所がいくつの閉塞区間に含まれているかを考える



小課題4

●クエリが含む閉塞区間のみを考えたときそれらに含まれない場所

⇔その場所が含む閉塞区間の数がクエリの範囲内で最小＝(クエリを丸ごと含む閉塞区間の数)



小課題4

- クエリは以下の問題に帰着
- B_i ... 場所 i が含む閉塞区間の数の配列
- クエリ $[L, R]$
- $\min\{B_L, B_{L+1}, \dots, B_R\} = B_i$ を満たす i の個数 ($L \leq i \leq R$) を求める

小課題4

- クエリ $[L, R]$
- $\min\{B_L, B_{L+1}, \dots, B_R\} = B_i$ を満たす i の個数($L \leq i \leq R$)を求める
- これはsegment treeでできる

小課題4

- 各ノードで持つ情報は最小値 mi と最小値の個数 cnt
- ノード A とノード B のマージは

$mi = \min(A.mi, B.mi), cnt = 0;$

$If(mi == A.mi) cnt += A.cnt$

$If(mi == B.mi) cnt += B.cnt$

小課題5

- 小課題5は満点とやることはほとんど同じなので省略します
- ただクエリが $[1, N]$ のみなので満点と比べて簡単にできたり省略できたりします

満点

- 更新あり
- 答えを求めるクエリは小課題4と同じようにする
- そのために配列 B_i ... 場所 i が含む閉塞区間の数を適切に更新する必要がある

閉塞区間についての考察2

- ある場所 x を含む閉塞区間の数は最大で $\log_2 \max\{A\}$ 個である(同様に区間の重なり深さも $\log_2 \max\{A\}$ 個以下である)

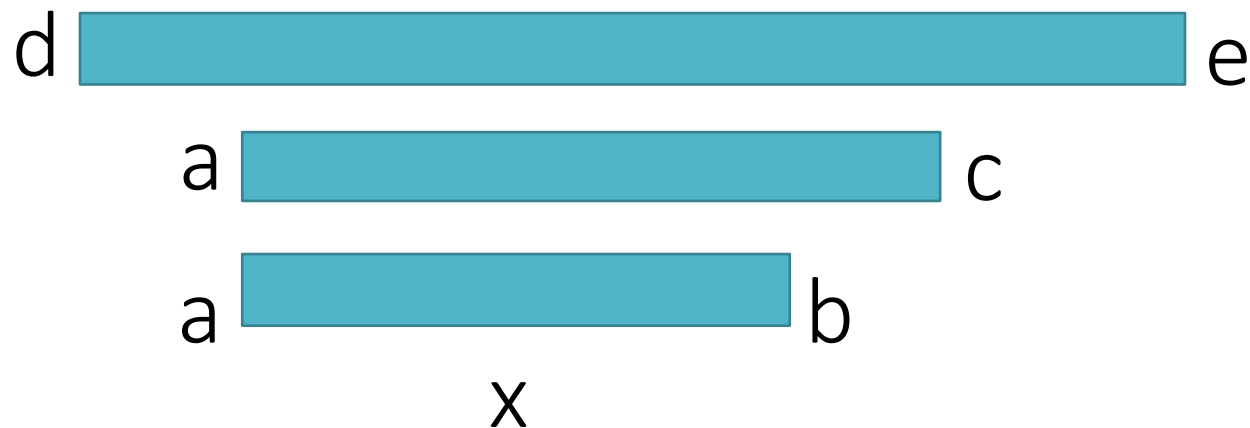
- 右下のように重なっていると

- $x < \min(a, b)$

- $x + b(> 2x) < \min(a, c)$

- $x + b + c(> 4x) < \min(d, e)$

左右の壁の下限が2倍になっていく



満点

- 場所 x を含む閉塞区間の数は $O(\log A)$ 個
- 値が更新されたときに消えたり増えたりする閉塞区間の数も $O(\log A)$ 個である
- 方針としてはその時点での閉塞区間をすべて管理する

満点

- まずは配列 B_i の区間最小値個数の segment tree に閉塞区間追加、削除の機能=区間加算クエリの機能を追加する
- これは遅延 segment tree にすればできる

満点

- A_x を y に更新するクエリ
- ①場所 x を含む閉塞区間をすべて削除する($O\log(A)$ 個)
- ② A_x を y に更新
- ③場所 x を含む閉塞区間をすべて見つける($O\log(A)$ 個)
- ④見つけた閉塞区間を追加する($O\log(A)$ 個)

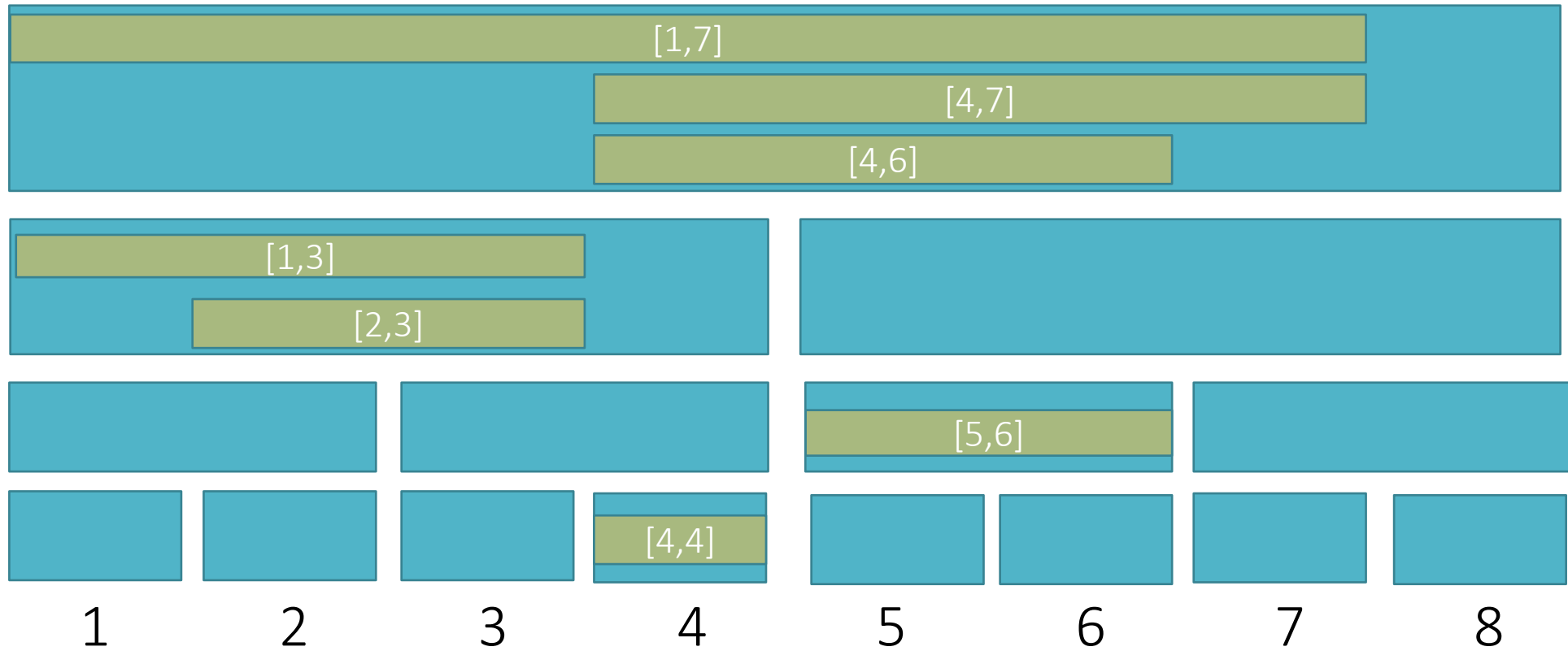
区間を管理するデータ構造

- segment treeに区間を入れていく
- 区間 $[l, r]$ はその区間を完全に格納できるノードのうち最も小さいものに入れる.

区間を管理するデータ構造

- 管理する区間は交差しないので、各ノードではそれが対応する範囲を $[l, r]$ として $(l + r)/2$ を含むような区間が入っていく
- それらの格納される区間たちは区間の大きい順に次に大きい区間を内包するというように内包の関係が $(l + r)/2$ を中心に一系列になっている
- よって各ノードでは `stack` を持ってそこに区間を小さい順に積んでいく
- 区間の追加や削除は `stack` の一番上のみで行う

区間を管理するデータ構造



区間を管理するデータ構造

- ①場所 x を含む閉塞区間をすべて削除する
- Segment treeの葉ノード x から順番に上に上りつつ、各ノードではstackの一番上(一番大きな区間)が x を含む間それらをpopする
- $O(\log A + \log N)$ でできる

区間を管理するデータ構造

- ④見つけた $O(\log A)$ 個の区間を追加する
- 追加する区間を小さい順に持っておく
- Segment treeの葉ノード x から順番に上に上りつつ、各ノードではまだ追加していない区間で最小のものが追加できる間それをpushする
- $O(\log A + \log N)$

満点

- ③場所 x を含む閉塞区間をすべて見つける($O(\log(A))$ 個)

満点

- 場所 x の右の壁となる候補は A_x より大きい必要があるのでまず $r < i$ かつ $A_x < A_r$ を満たす最小の r が考えられる
- その右で次の候補を考えると, 区間は x と r を含むので $r < r_2$ かつ $A_x + A_r < A_{r_2}$ を満たす最小の r_2
- このように右の候補を求めていくと A の値が2倍になっていくので $O(\log A)$ 個しかない
- また $0 < i$ かつ $\Delta < A_i$ を満たす最小の i は seg 木上の二分探索で $O(\log N)$ で求まる
- $O(\log A \log N)$ で右の壁の候補をすべて列挙できる

満点

- 同様に左の壁の候補も $O(\log A)$ 個求められる
- この左の壁の候補 × 右の壁の候補の組み合わせをすべて試す
- $O(\log A \log A)$

細かい話

- 最初に与えられる値も $1, 1, \dots, 1$ (閉塞区間なし) から i 番目を A_i に更新するとして N 個の更新クエリに帰着すると楽
- クエリを縮めるパートは壁の候補を列挙するようにすると、計算量は増えるが、区間を見つけるパートと同じ処理でできる
- 更新クエリで削除、追加するのは更新する場所 x が壁になる区間、つまり $x - 1, x, x + 1$ を含む区間も処理する必要がある. 壁の候補も $x - 1, x + 1$ から始めて見つけるといい

満点(まとめ)

- A_x を y に更新するクエリ
- ①場所 x を含む閉塞区間をすべて削除する($\log A$ 個)
区間の管理 $O(\log A + \log N)$ 、 B の配列の更新 $O(\log A \log N)$
- ② A_x を y に更新
- ③場所 x を含む閉塞区間をすべて見つける($\log A$ 個)
 $O(\log A \log N + \log A \log A)$
- ④見つけた閉塞区間を追加する($\log A$ 個)
区間の管理 $O(\log A + \log N)$ 、 B の配列の更新 $O(\log A \log N)$

満点(まとめ)

- $[L, R]$ にこたえるクエリ
- ⑤左右で新たにできる閉塞区間に含まれる部分を除く $O(\log N)$
- ⑥配列 B (各場所が含む閉塞区間の数) で区間最小値の個数を求める $O(\log N)$
- 全体で $O((N + Q)(\log N \log A + \log A \log A))$

得点分布

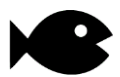
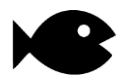
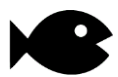
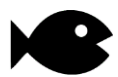
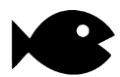
48点



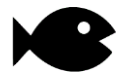
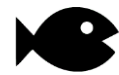
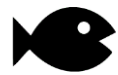
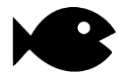
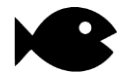
36点



25点



13点



8点



5点



0点



復興事業 解説

松尾 凜太郎 / QCFium

問題概要

辺に整数が書かれている無向グラフが与えられる。
以下の形のクエリを Q 個処理せよ：

整数 x が与えられる。コスト 1 で辺に書かれている整数を 1 増減させることで、 x が書かれている辺のみで全頂点間を行き来できるようにするときの最小コストを求める

問題概要

辺に整数が書かれている無向グラフが与えられる。
以下の形のクエリを Q 個処理せよ：

整数 x が与えられる。整数 c が書かれている辺のコストを $|c - x|$ としたとき、グラフの最小全域木を求める

$$N \leq 500$$

$$M \leq 100000$$

$$Q \leq 1000000$$

小課題 1

$$M \leq 16, Q \leq 10$$

全域木に使う辺の集合を全探索 (2^M 通り)

→ 本当に連結になっているならば使った辺についてコストを計算する

$$O(Q \times 2^M \times M)$$

クラスカル法

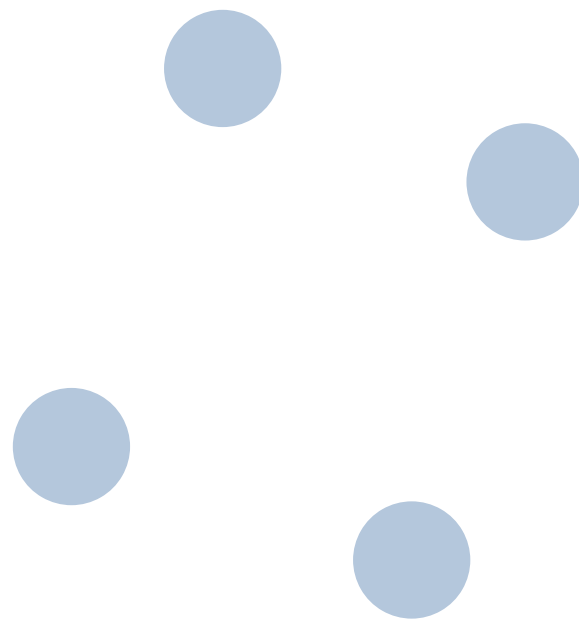
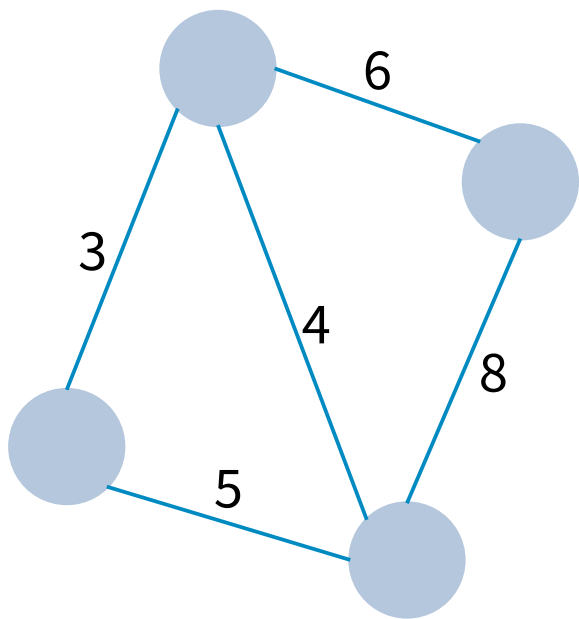
全ての辺を、コストの小さい順に見ていく

まだその辺が繋ぐ2点が連結でないならばその辺を追加する
既に連結ならばスキップする
最終的に追加された辺が最小全域木となる

辺の追加 & 連結判定 → UnionFind

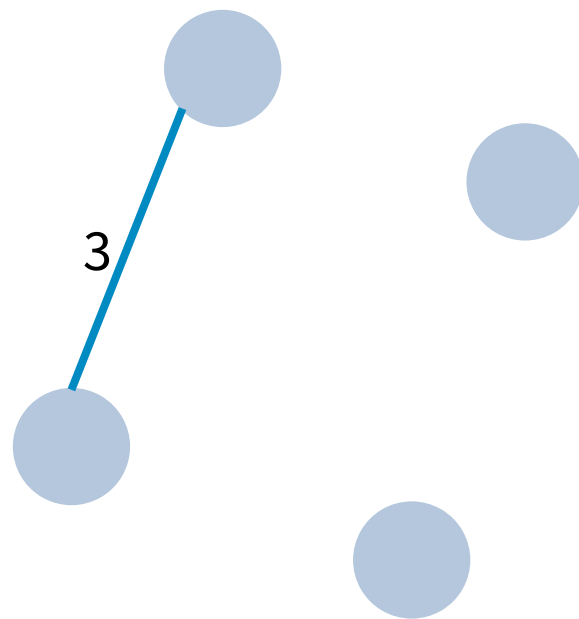
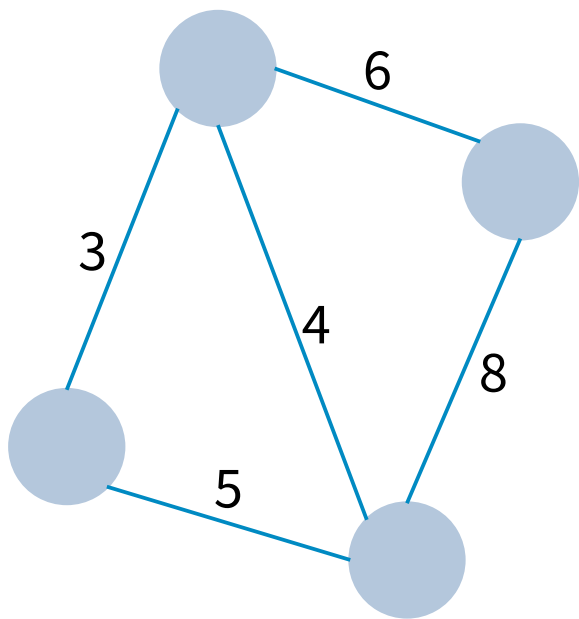
クラスカル法

全ての辺を、コストの小さい順に見ていく



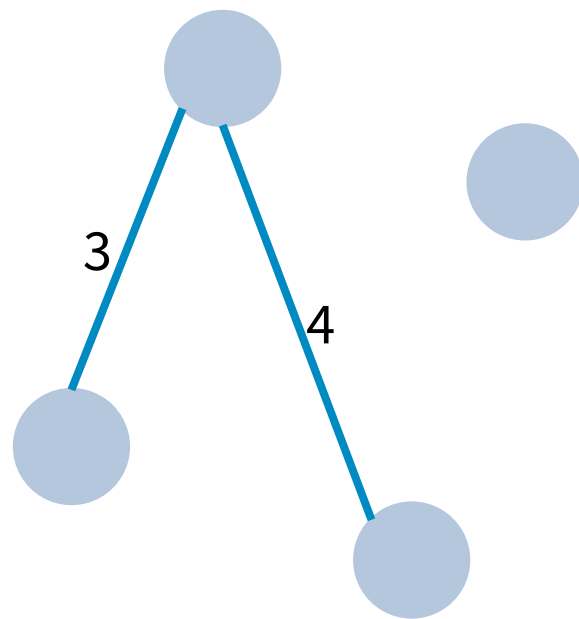
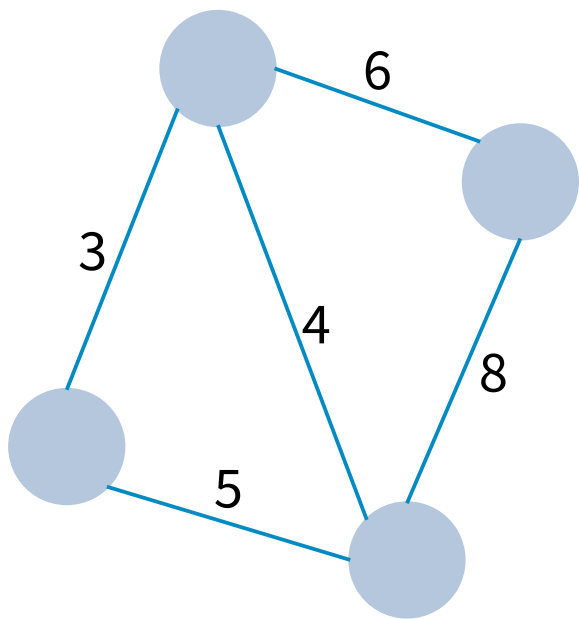
クラスカル法

全ての辺を、コストの小さい順に見ていく



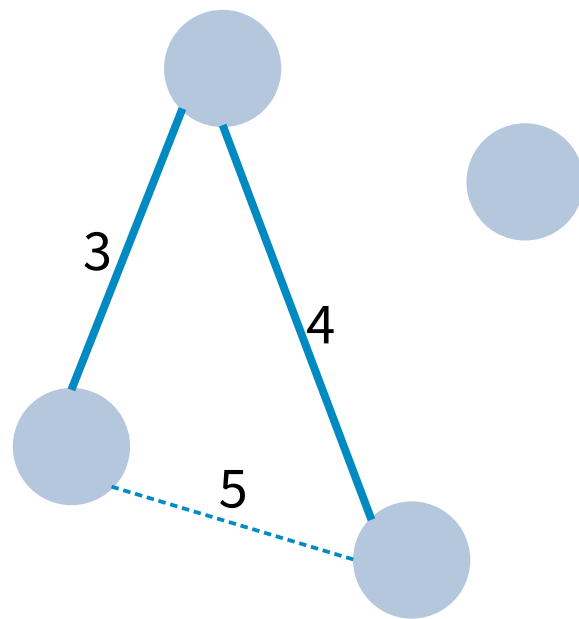
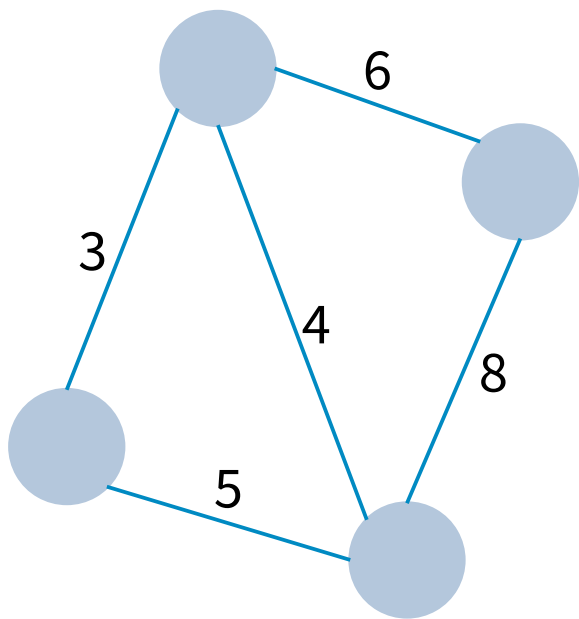
クラスカル法

全ての辺を、コストの小さい順に見ていく



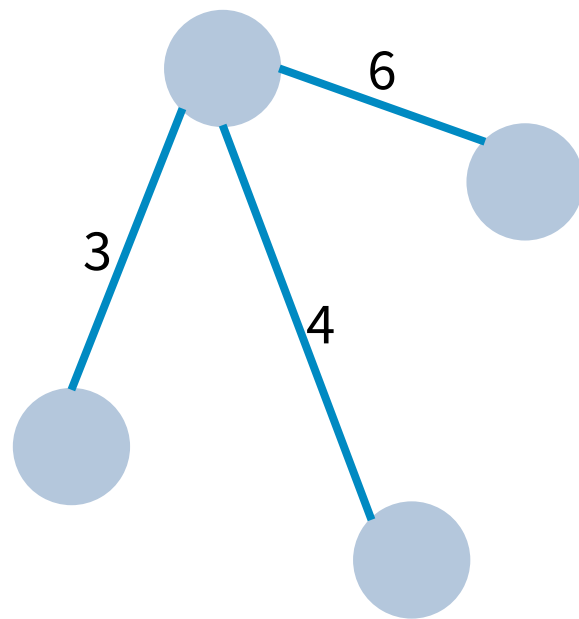
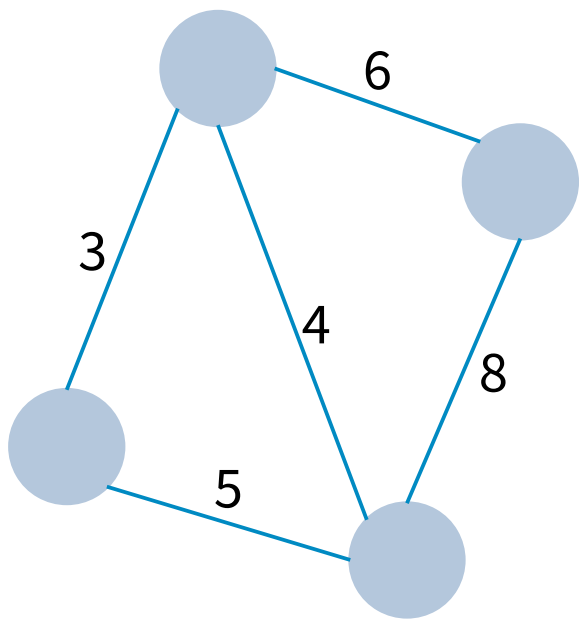
クラスカル法

全ての辺を、コストの小さい順に見ていく



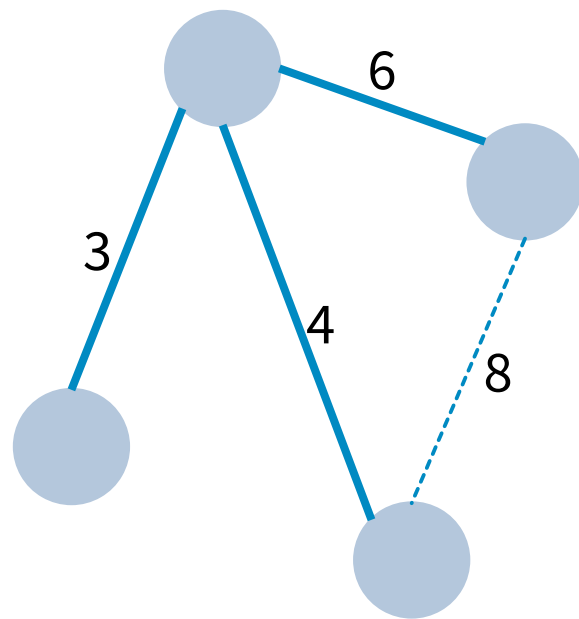
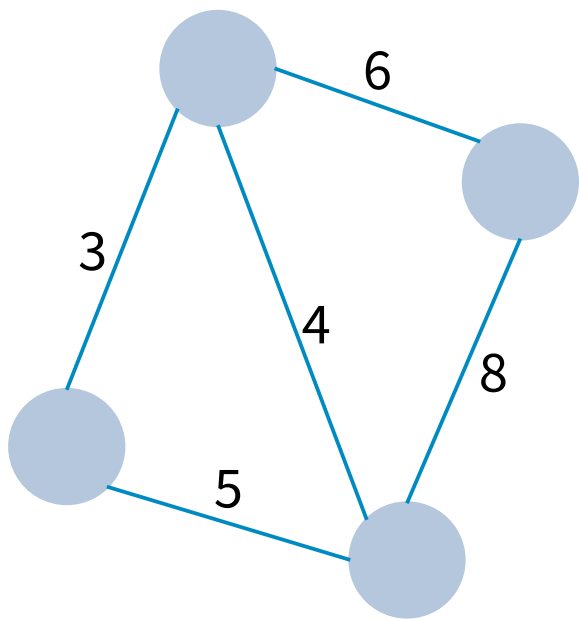
クラスカル法

全ての辺を、コストの小さい順に見ていく



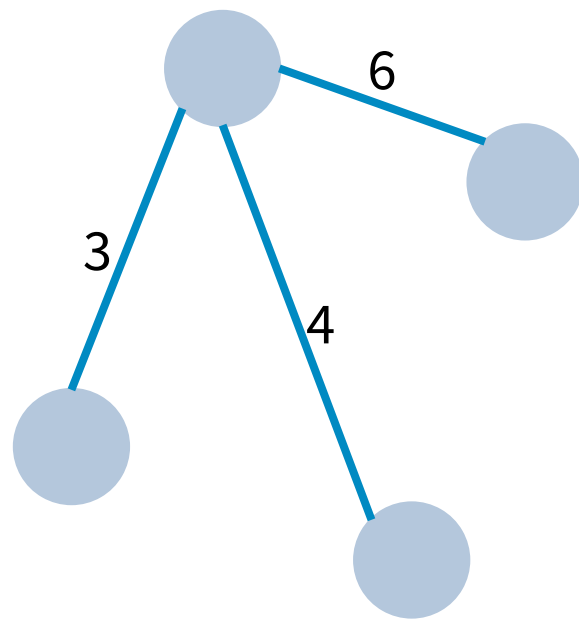
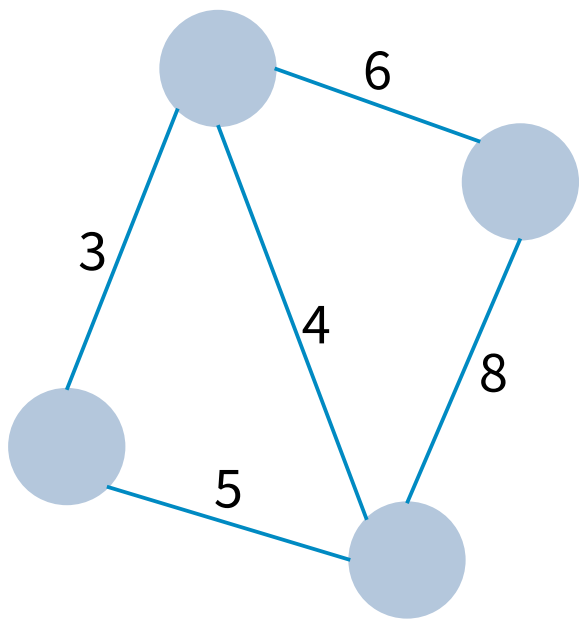
クラスカル法

全ての辺を、コストの小さい順に見ていく



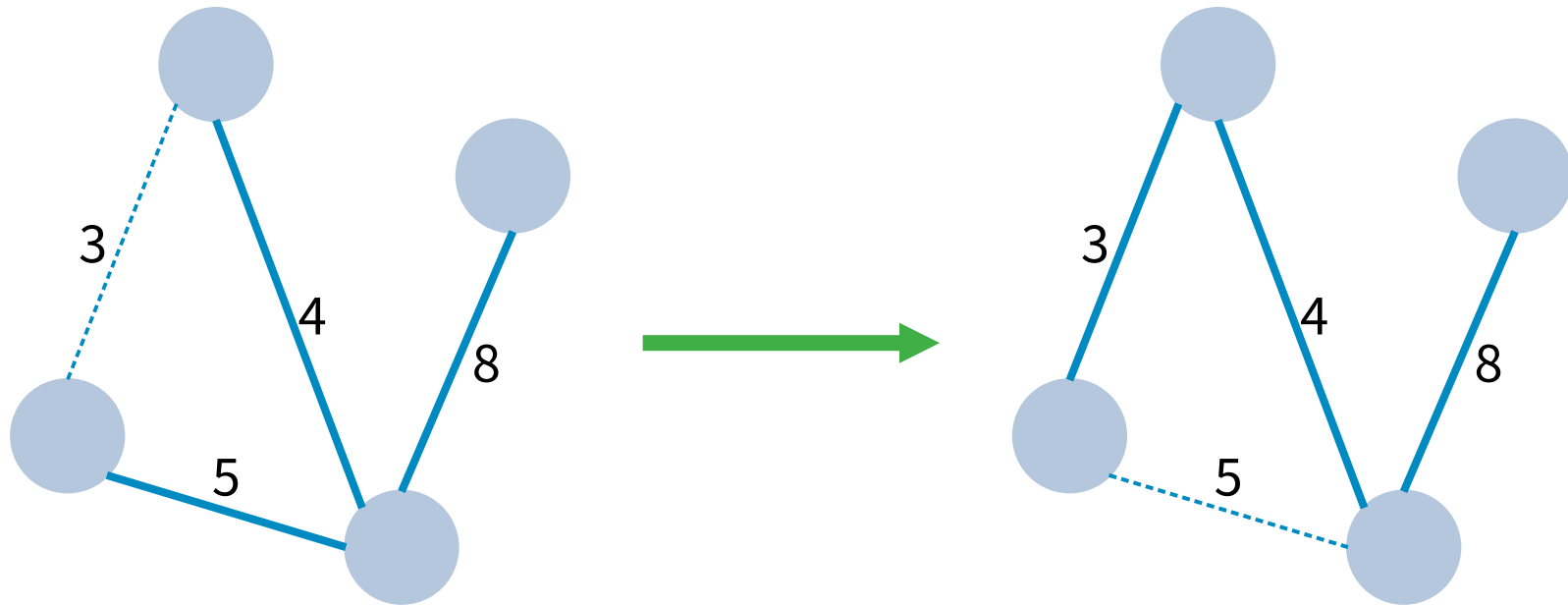
クラスカル法

全ての辺を、コストの小さい順に見ていく



クラスカル法 証明

なぜこれでよいのか
最小コストの辺を使わないのが最小だったとすると ...

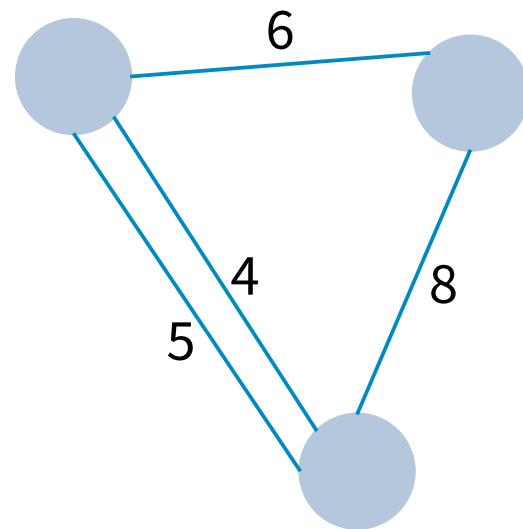
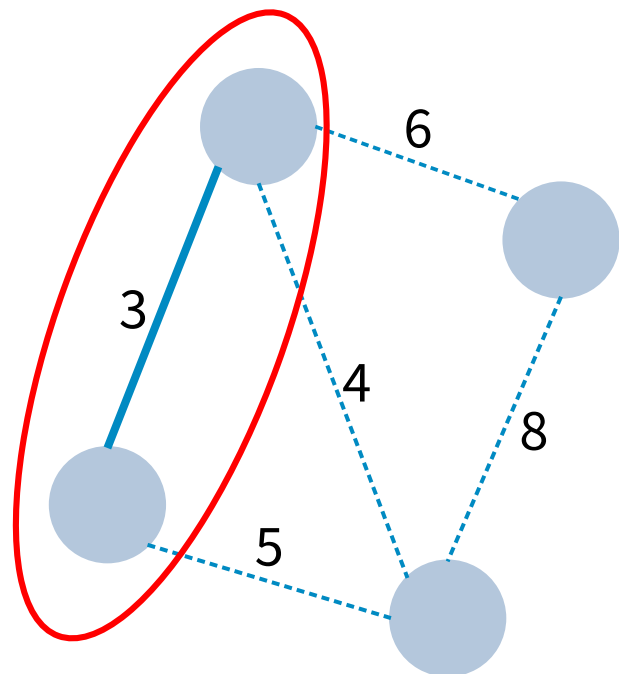


最小コストの辺を使っても損をしない

クラスカル法 証明

なぜこれでよいのか

最小コストの辺を使うことにしたので縮約して同じことを繰り返す
→ クラスカル法と同じことをしている



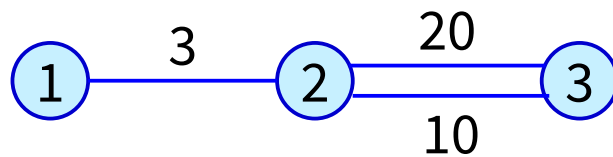
小課題 2

$$Q \leq 10$$

各クエリで各辺のコストを普通に計算 $\rightarrow$ クラスカル法
 $O(QM\log(M))$

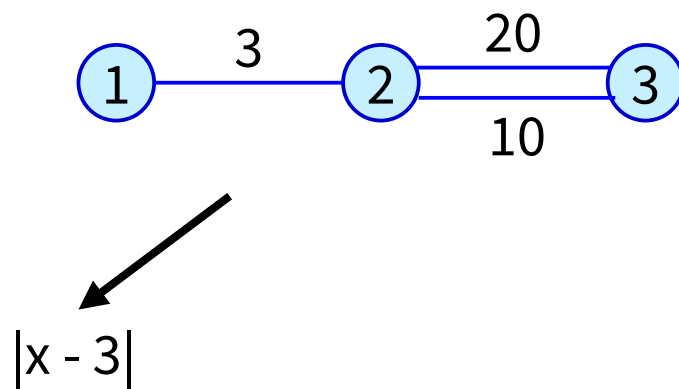
小課題 3

$$B_i = A_i + 1$$



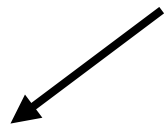
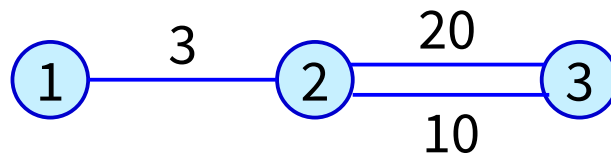
小課題 3

$$B_i = A_i + 1$$



小課題 3

$$B_i = A_i + 1$$

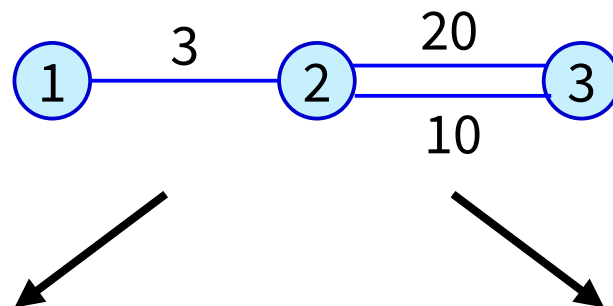


$$x < 3 \rightarrow 3 - x$$

$$x \geq 3 \rightarrow x - 3$$

小課題 3

$$B_i = A_i + 1$$



$$x < 3 \rightarrow 3 - x$$

$$x \geq 3 \rightarrow x - 3$$

$$x \leq 10 \rightarrow 10 - x$$

$$10 \leq x < 15 \rightarrow x - 10$$

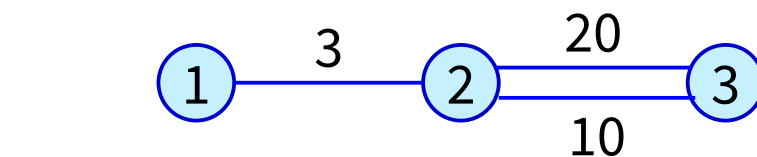
$$15 \leq x < 20 \rightarrow 20 - x$$

$$20 \leq x \rightarrow x - 20$$

コストは x の範囲ごとに x の 1 次関数

小課題 3

$$B_i = A_i + 1$$



$$\begin{aligned} x < 3 &\rightarrow 3 - x \\ x \geq 3 &\rightarrow x - 3 \end{aligned}$$

$+ (2x - 6)$

$$\begin{aligned} x &\leq 10 \rightarrow 10 - x \\ 10 &\leq x < 15 \rightarrow x - 10 \\ 15 &\leq x < 20 \rightarrow 20 - x \\ 20 &\leq x \rightarrow x - 20 \end{aligned}$$

1 次関数は変化する

小課題 3

最初 : $13 - 2x$

$x = 3 : +(2x - 6)$

$x = 10 : +(2x - 20)$

$x = 15 : +(-2x + 30)$

$x = 20 : +(2x - 40)$



小課題 3

最初 : $13 - 2x$

$x = 1$



$x = 3 : +(2x - 6)$

$x = 10 : +(2x - 20)$

$x = 15 : +(-2x + 30)$

$x = 20 : +(2x - 40)$

小課題 3

最初 : $13 - 2x$

$x = 3 : +(2x - 6)$

$x = 5$ →

$x = 10 : +(2x - 20)$

$x = 15 : +(-2x + 30)$

$x = 20 : +(2x - 40)$

小課題 3

最初 : $13 - 2x$

$x = 3 : +(2x - 6)$

$x = 10 : +(2x - 20)$

$x = 15 : +(-2x + 30)$

$x = 18$ →

$x = 20 : +(2x - 40)$

小課題 3

$$B_i = A_i + 1$$

全体のコストの 1 次式が変化するタイミングを列挙 & ソート
→ クエリを昇順に処理

$$O(M \log(M) + Q)$$

小課題 4,5

小課題 4 : $M \leq 1000$

小課題 5 : $Q \leq 20000$

毎クエリ全ての辺のコストを計算しては間に合わない
→ 視点を変える

小課題 4,5

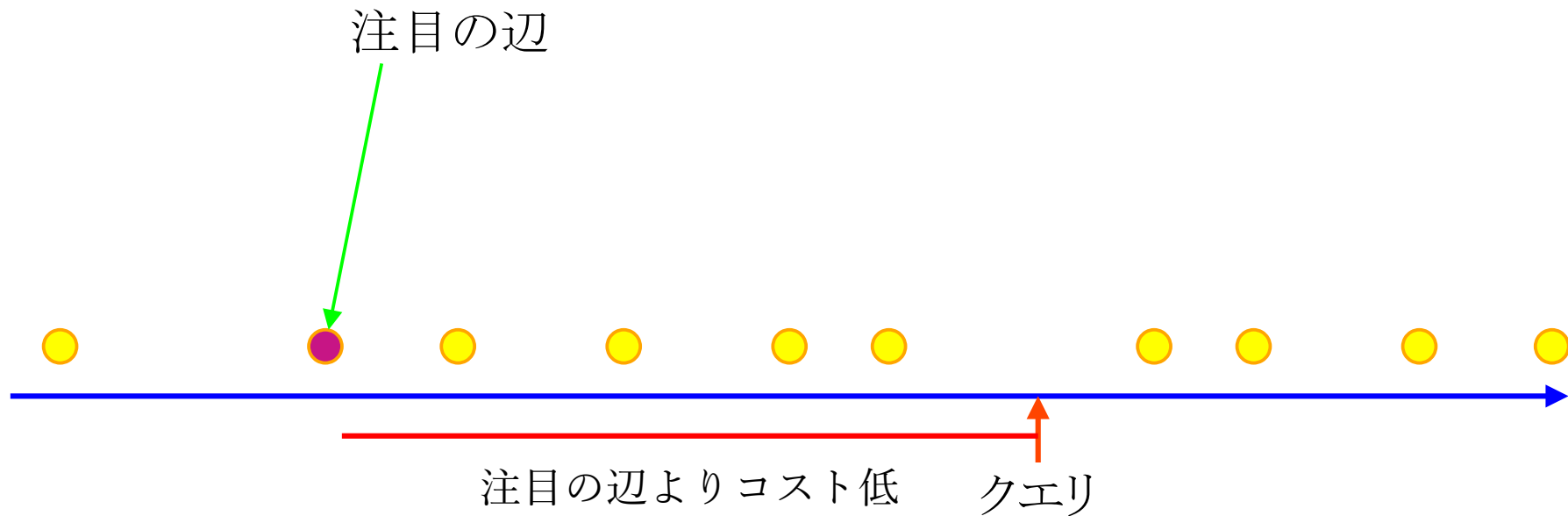
小課題 4 : $M \leq 1000$

小課題 5 : $Q \leq 20000$

毎クエリ全ての辺のコストを計算しては間に合わない
→ 視点を変える

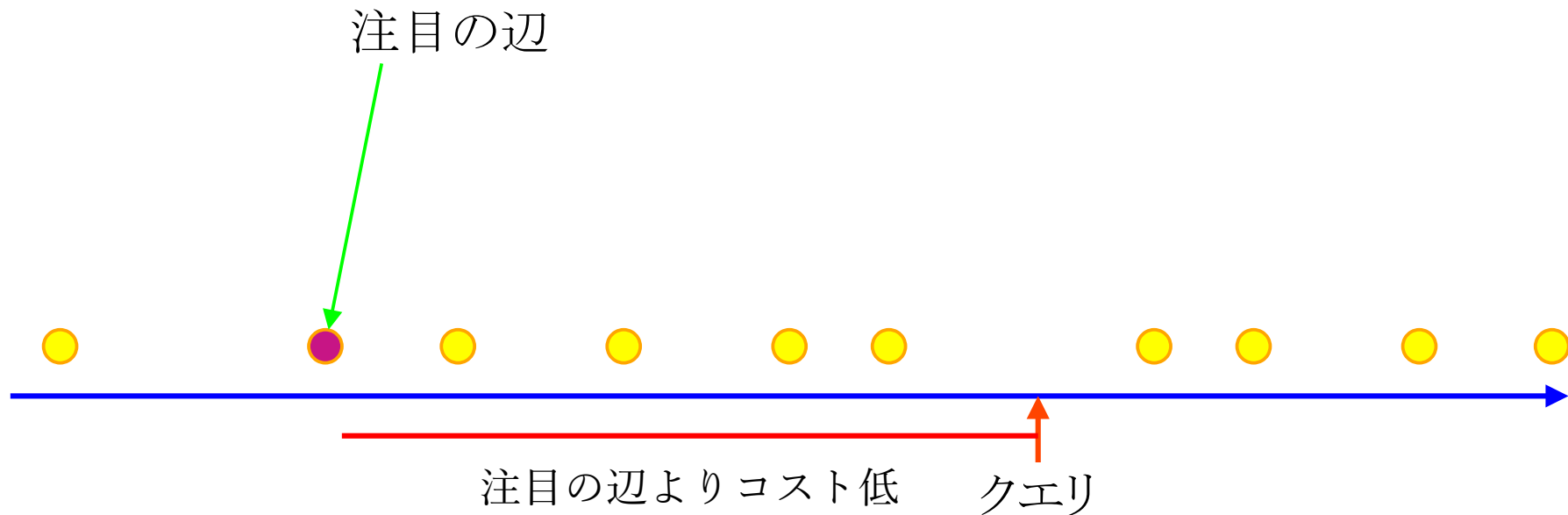
ある辺がクラスカル法で使われる可能性があるのは x がどんな値のときか？

考察



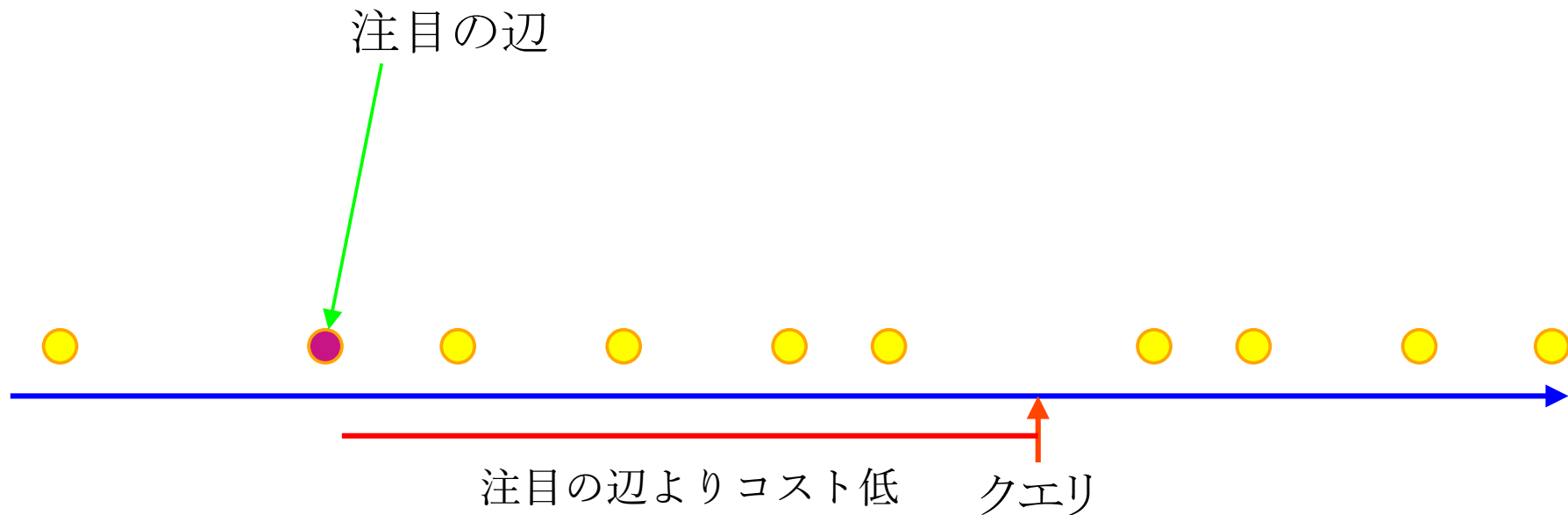
→ 注目の辺が頂点 a, b を繋いでいるとして、赤線だけで a, b が連結ならば注目の辺は使われない

考察



→ 注目の辺より右にある辺を順に追加していって a, b が連結になったら、そこより右にあるクエリでは注目の辺は使われない

考察



→ 注目の辺より右にある辺を順に追加していったら、a,b が連結になったら、そこより右にあるクエリでは注目の辺は使われない

左右で同じことをする

考察

各辺についてそれが使われる可能性のある x の範囲を求めるのを愚直にやっても $O(M^2 \alpha(N))$

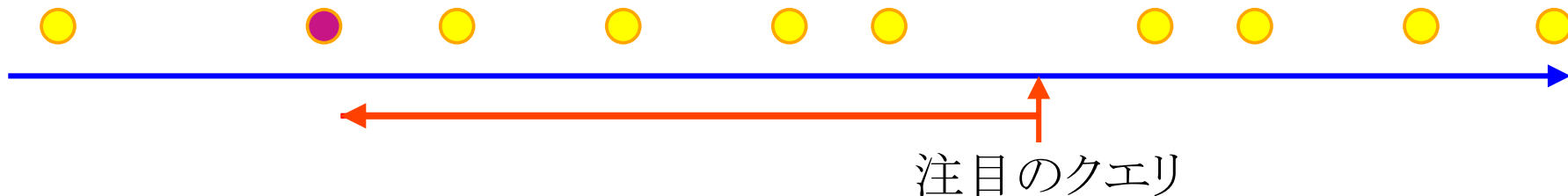
各クエリで絞った $2(N - 1)$ 本をソートしてクラスカル
→ $O(QN \log(N))$

小課題 4 ($M \leq 1000$) では $QN \log(N)$, 小課題 5 ($Q \leq 20000$) では $M^2 \alpha(N)$ が大きすぎる

小課題 5

$$Q \leq 20000$$

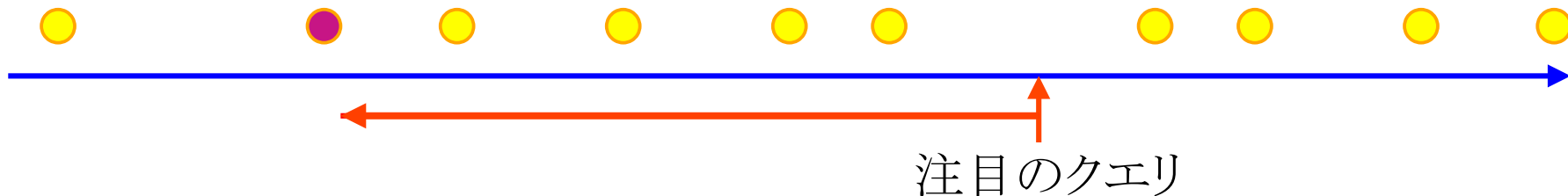
実は先ほどの解法はそのまま通る
クエリ側から使う可能性のある辺の数を考える



小課題 5

$Q \leq 20000$

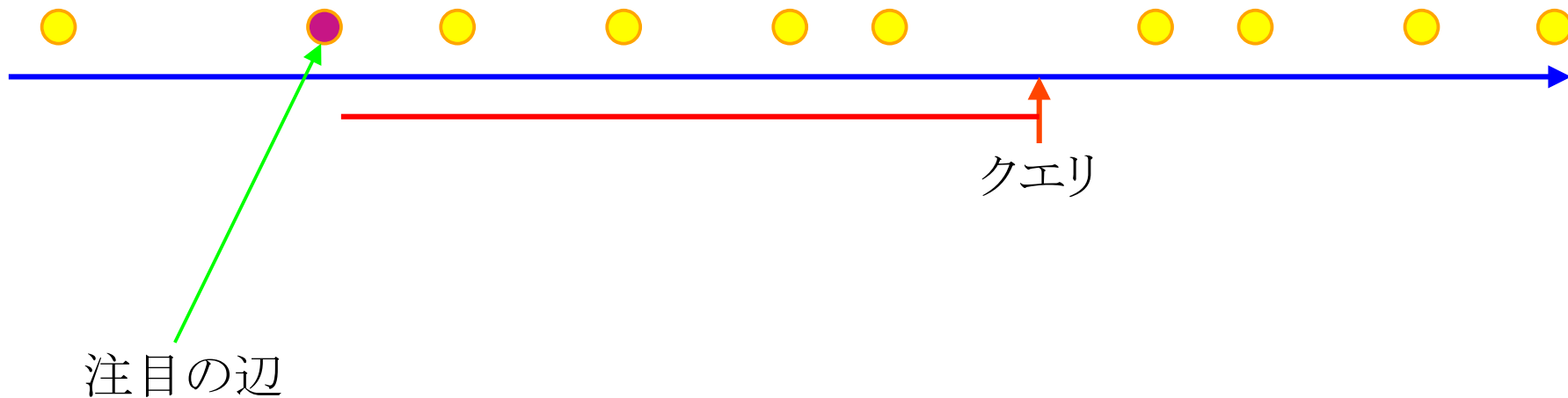
実は先ほどの解法はそのまま通る
クエリ側から使う可能性のある辺の数を考える



オレンジの範囲の辺で a, b が非連結 $\rightarrow$ オレンジの矢印の向きにクラスカル法をすると注目の辺は使われる
 $\rightarrow$ 各クエリ左右それぞれ $N - 1$ 本以下

小課題 5

同じ辺と辺の区間にあるクエリを同一視すると使われ得る辺とクエリの組は合計 $O(MN)$ 個
→ 実は先ほどの解法はそのまま通る ($O(MN \alpha(N) + QN \log(N))$)



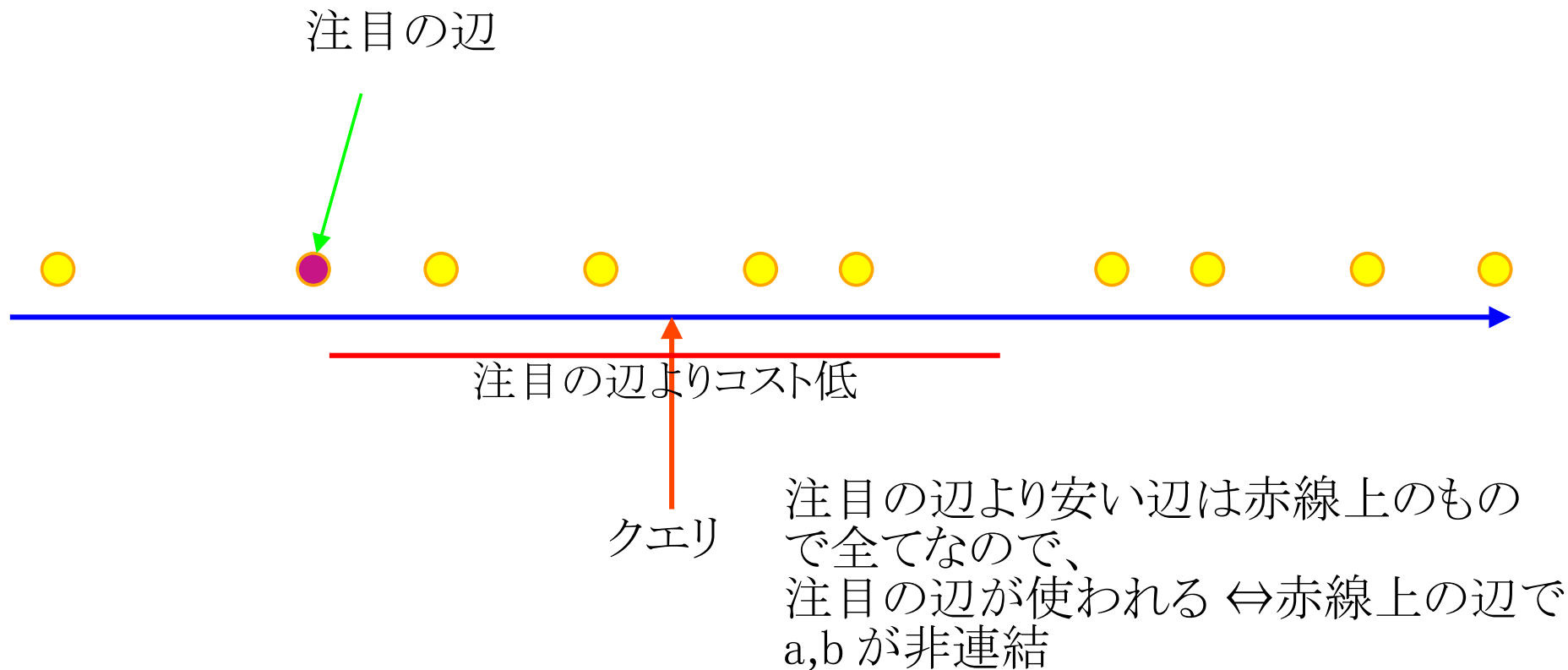
小課題 4

$$M \leq 1000$$

各クエリで $2(N - 1)$ 本の候補となる辺でクラスカル法を行う
部分 ($O(QN \log(N))$) が間に合わない
→ 「候補」ではなく使う辺を確定できないか

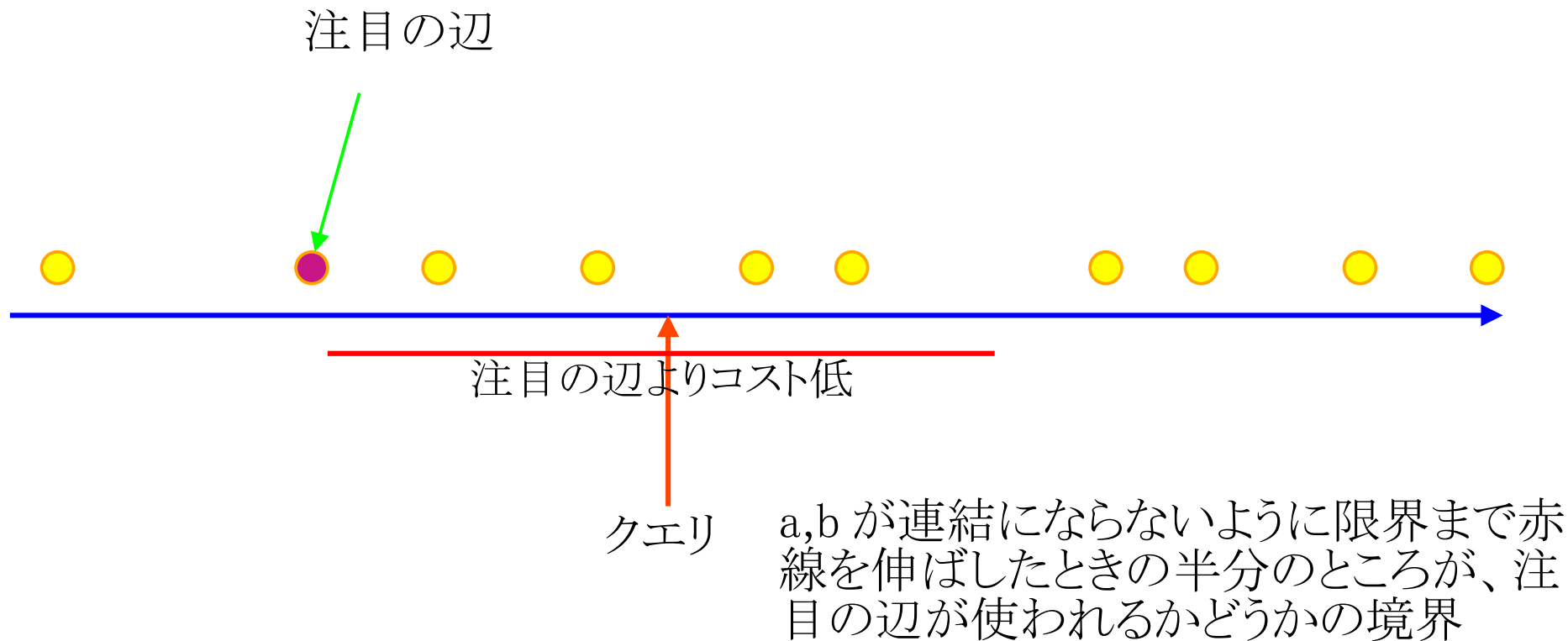
小課題 4

$M \leq 1000$



小課題 4

$M \leq 1000$



小課題 4

$$M \leq 1000$$

前処理 : $O(M^2 \alpha(N))$

クエリ処理 : $O(QN)$ or $O(M + Q \log(N))$

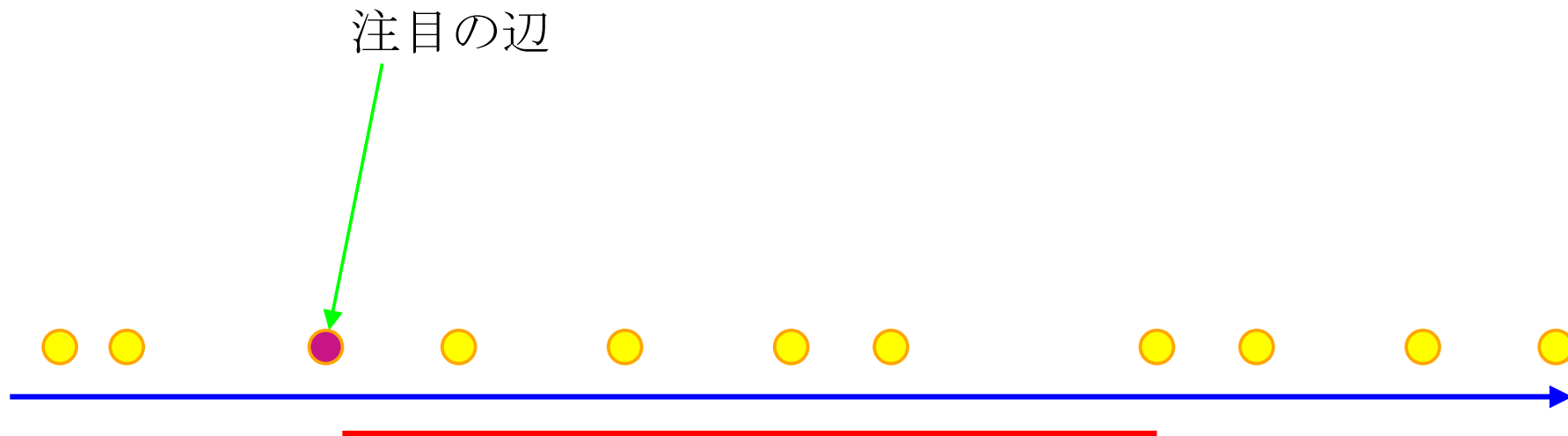
小課題 6 (満点)

追加制約なし

前処理 : $O(M^2 \alpha(N)) \rightarrow$ 実は $O(MN \alpha(N))$

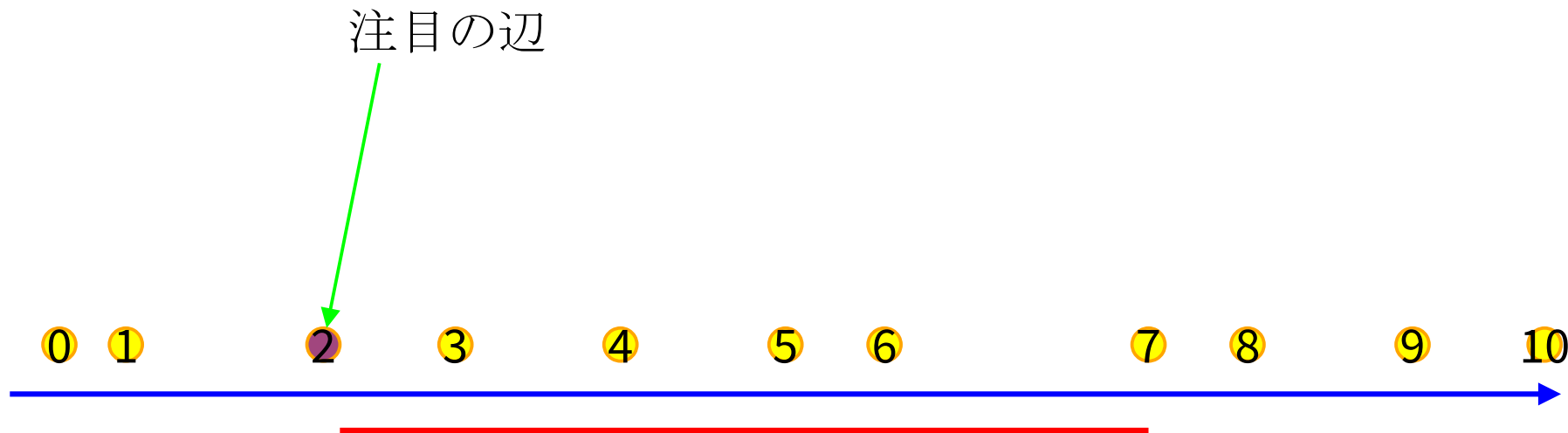
クエリ処理 : $O(QN)$ or $O(M + Q \log(N))$

小課題 6 (満点) 別解



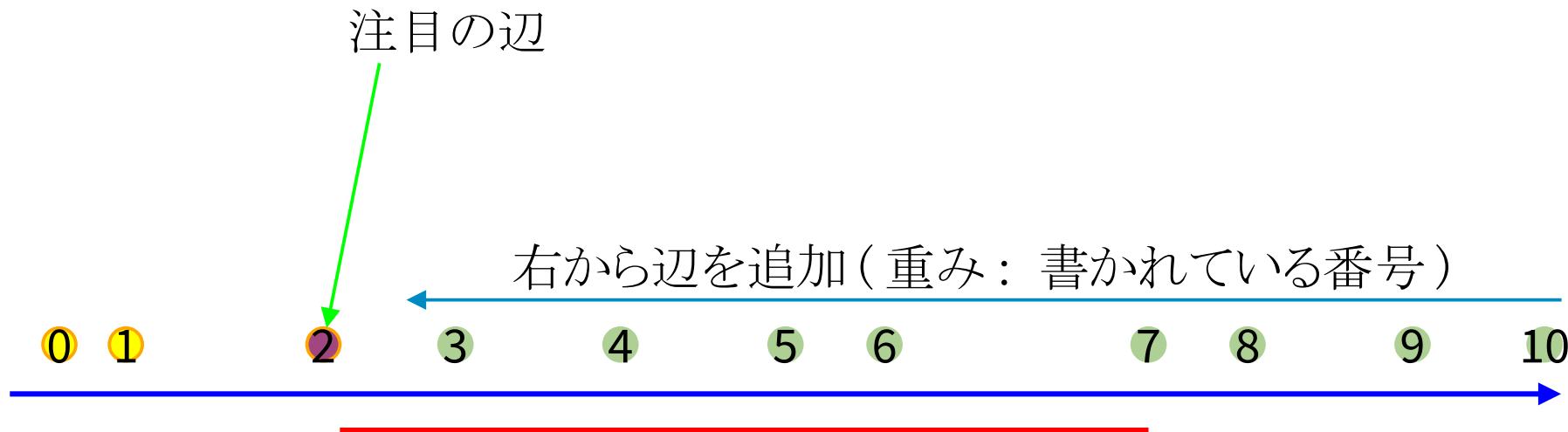
注目の辺が頂点 a, b を繋いでいるとして、どこまで右に行ったら a と b が連結になってしまうか

小課題 6 (満点) 別解



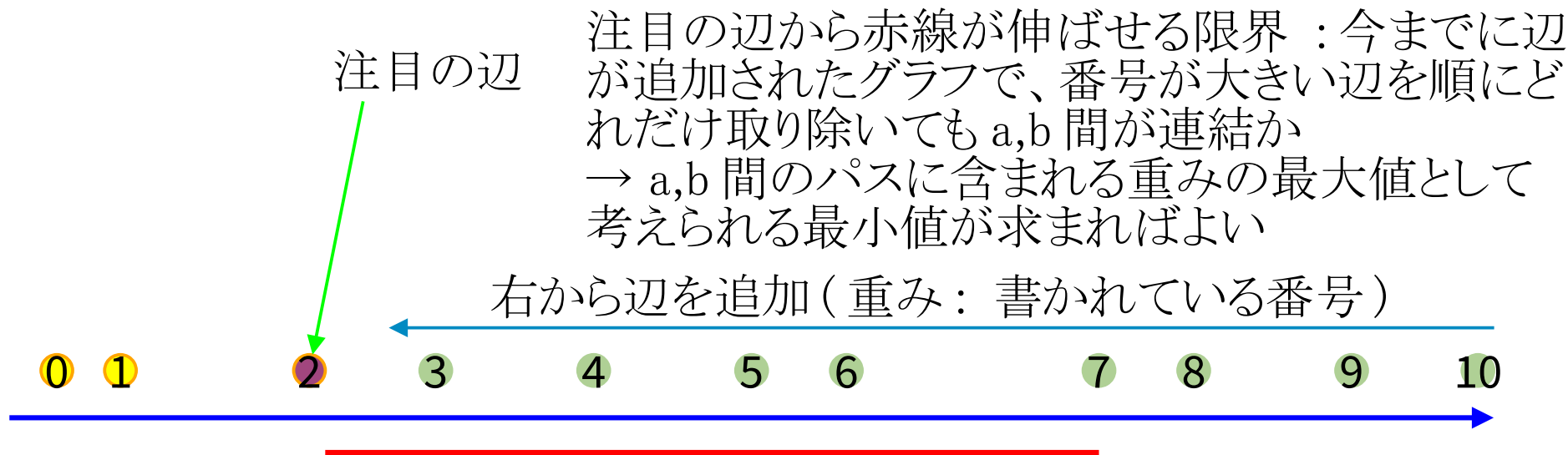
注目の辺が頂点 a, b を繋いでいるとして、どこまで右に行ったら a と b が連結になってしまうか

小課題 6 (満点) 別解



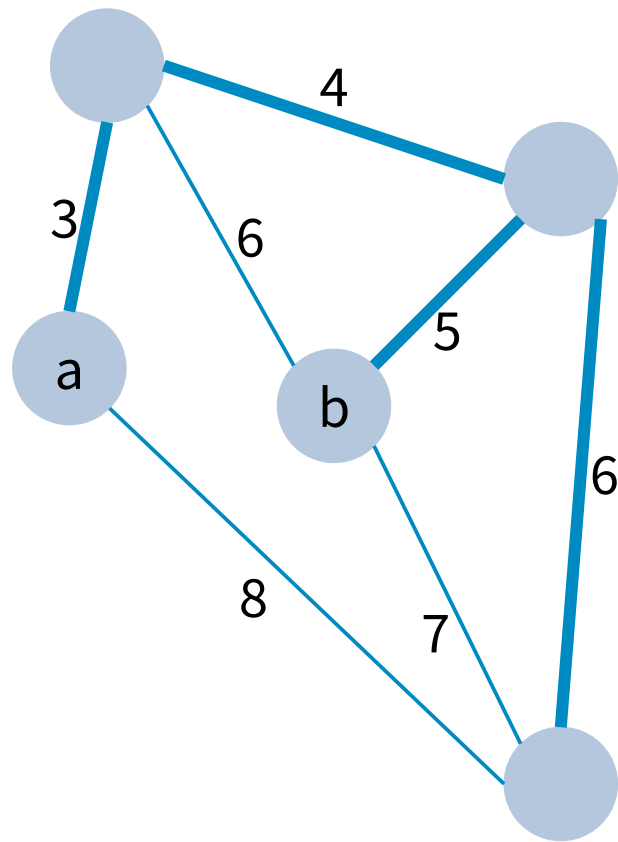
注目の辺が頂点 a, b を繋いでいるとして、どこまで右に行ったら a と b が連結になってしまうか

小課題 6 (満点) 別解



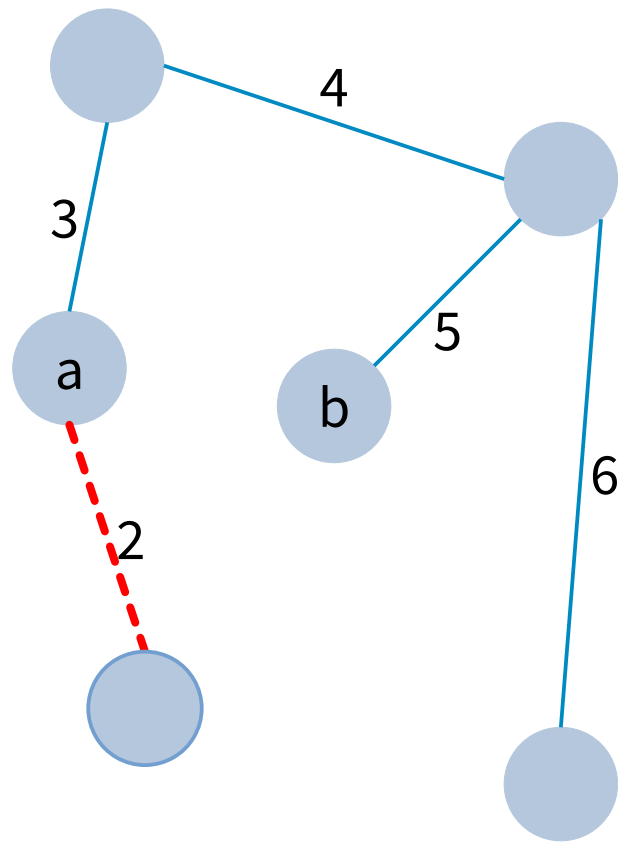
注目の辺が頂点 a, b を繋いでいるとして、どこまで右に行ったら a と b が連結になってしまうか

小課題 6 (満点) 別解



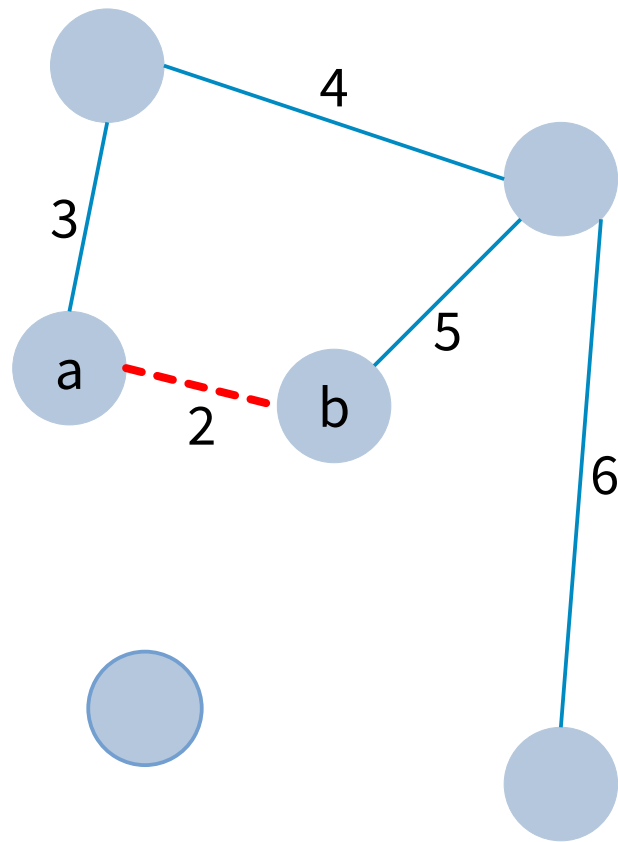
a,b 間のパスに含まれる重みの最大値として考えられる最小値：
グラフの最小全域木上での a,b の最短パス (1 つだけ) が最適であることが証明できる ([1])
→ 辺の追加ごとに最小全域木を保持できればよい

小課題6（満点）別解



辺を追加して最小全域木
を保持する
繋ぐ頂点が非連結のとき
→ 繋ぐだけ

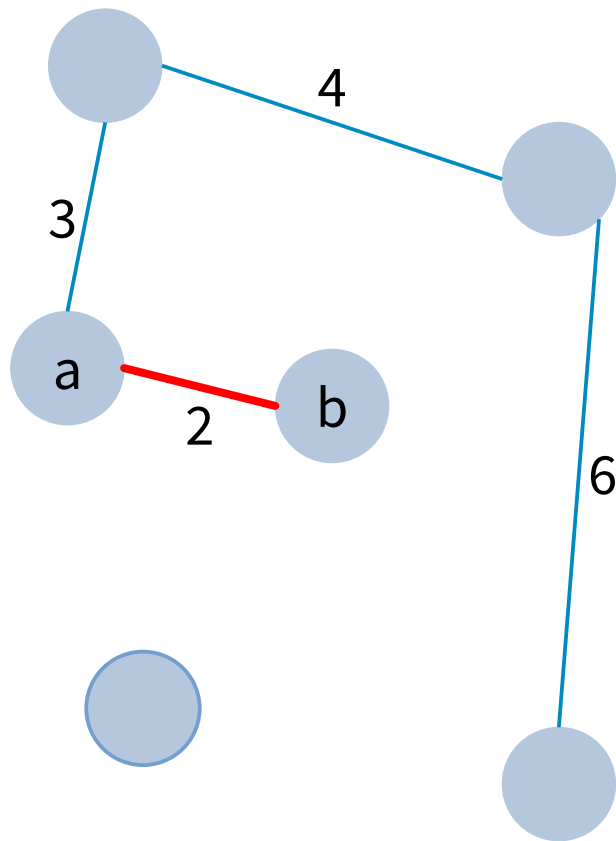
小課題 6 (満点) 別解



辺を追加して最小全域木を保持する

繋ぐ頂点が連結のとき：
辺は右から追加しているので追加する辺の重みは他のどの辺の重みよりも小さい
→ ループの中で重み最大のを消す

小課題 6 (満点) 別解



辺を追加して最小全域木を
保持する

繋ぐ頂点が連結のとき：
辺は右から追加しているので追
加する辺の重みは他のどの辺
の重みよりも小さい
→ ループの中で重み最大のも
のを消す

小課題 6 (満点) 別解

重み付き森に対して

- 指定した 2 頂点のパス上で重みが最大の辺の特定
- 辺追加 (閉路はできない)
- 辺削除

この問題では各 $O(N)$ で良いので簡単 $\rightarrow$ 合計 $O(MN + Q)$

小課題 6 (満点) 別解

重み付き森に対して

- 指定した 2 頂点のパス上で重みが最大の辺の特定
- 辺追加 (閉路はできない)
- 辺削除

~~この問題では各 $O(N)$ で良いので簡単 $\rightarrow$ 合計 $O(MN + Q)$~~

もっと減らせる

Link-Cut Tree 等で各 $O(\log(N)) \rightarrow$ 合計 $O(N + M\log(N) + Q)$

参考文献

[1] "Widest path problem", Wikipedia

https://en.wikipedia.org/wiki/Widest_path_problem